Second Edition — iOS 16 · Xcode 14.2 · Swift 5.7

SwiftUI Apprentice

Beginning iOS Programming With SwiftUI

By the Kodeco Team

Caroline Begbie & Audrey Tam



Kodeco

SwiftUI Apprentice

By Audrey Tam & Caroline Begbie

Copyright ©2023 Kodeco Inc.

Notice of Rights

All rights reserved. No part of this book or corresponding materials (such as text, images, or source code) may be reproduced or distributed by any means without prior written permission of the copyright owner.

Notice of Liability

This book and all corresponding materials (such as source code) are provided on an "as is" basis, without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, and noninfringement. In no event shall the authors or copyright holders be liable for any claim, damages or other liability, whether in action of contract, tort or otherwise, arising from, out of or in connection with the software or the use of other dealing in the software.

Trademarks

All trademarks and registered trademarks appearing in this book are the property of their own respective owners.

Table of Contents: Overview

Book License 14

Before You Begin 15

What You Need 16

Book Source Code & Forums 17

How to Read This Book 20

Section I: Your First App: HIITFit 21

Chapter 1: Checking Your Tools 22

Chapter 2: Planning a Paged App 62

Chapter 3: Prototyping the Main View 88

Chapter 4: Prototyping Supplementary Views 121

Chapter 5: Moving Data Between Views 150

Chapter 6: Observing Objects 173

Chapter 7: Saving Settings 189

Chapter 8: Saving History Data 215

Chapter 9: Refining Your App 241

Chapter 10: Working With Datasets 273

Chapter 11: Managing Data With Property Wrappers 309

Chapter 12: Apple App Development Ecosystem 332

Section II: Your Second App: Cards 355

Chapter 13: Outlining a Photo Collage App 356

Chapter 14: Gestures 379
Chapter 15: Structures, Classes & Protocols 406
Chapter 16: Adding Assets to Your App 433
Chapter 17: Adding Photos to Your App 464
Chapter 18: Paths & Custom Shapes 493
Chapter 19: Saving Files 524
Chapter 20: Delightful UX — Layout 551
Chapter 21: Delightful UX — Final Touches 582
Section III: Your Third App: TheMet 611
Chapter 22: Lists & Navigation 612
Chapter 23: Just Enough Web Stuff 638
Chapter 24: Downloading Data 657
Chapter 25: Widgets 685
Conclusion 722

Table of Contents: Extended

Book License 14

Before You Begin 15

What You Need 16

Book Source Code & Forums 17

About the Authors 19

About the Editors 19

How to Read This Book 20

Section I: Your First App: HIITFit 21

Chapter 1: Checking Your Tools 22

Getting Started 23

Creating a New Xcode Project 25

A Quick Tour of Xcode 29

ContentView.swift 33

Creating a New SwiftUI View File 40

What Else is in Your Project? 44

Xcode Settings 45

Running Your Project 53

Key Points 61

Chapter 2: Planning a Paged App 62

Making Lists: Views & Actions 63

Creating Pages 65

Grouping Files 72

Passing Parameters 74

Looping 77

Key Points 87

Where to Go From Here? 87

Chapter 3: Prototyping the Main View 88
Creating the Exercise View 89
Creating the Header View 91
Creating the Exercise Structure 104
Playing a Video 108
Creating Timer, Buttons & Rating 112
Challenges 119
Key Points 120
Where to Go From Here? 120
Chapter 4: Prototyping Supplementary Views 121
Laying Out the History View 122
Structuring HistoryView Data 127
Dismissing HistoryView 131
Laying Out the Welcome View 134
Challenge 147
Key Points 149
Where to Go From Here? 149
Chapter 5: Moving Data Between Views 150
Managing Your App's Data 151
Using State & Binding Properties 152
Setting & Tapping Images 159
Showing & Hiding Modal Sheets 164
One More Thing 170
Key Points 172
Chapter 6: Observing Objects 173
Showing/Hiding the Timer 174
Using an EnvironmentObject 182
Challenge 188
Key Points 188
Chapter 7: Saving Settings 189

Data Persistence 190
Saving Ratings to UserDefaults 190
Data Directories 192
Swift Dive: Strings 200
Thinking of Possible Errors 205
Multiple Scenes 207
Apps, Scenes and Views 210
Restoring Scene State With SceneStorage 211
Key Points 214

Chapter 8: Saving History Data 215
Using Optionals 216
Debugging HistoryStore 219
Swift Error Checking 225
Alerts 226
Saving History 229
Property List Serialization 234
Ignoring the Loading Error 238
Key Points 240

Chapter 9: Refining Your App 241
Neumorphism 242
Creating a Neumorphic Button 243
Styles 244
Abstracting Your Button 248
The Embossed Button 251
@ViewBuilder 254
ViewBuilder Container View 259
Designing WelcomeView 262
Gradients 265
Challenge 271
Key Points 272

Chapter 10: Working With Datasets 273
Accumulating Data 276
Lists 279
Adding Data to the List 285
Charts 294
Privacy 306
Challenge 307
Key Points 308
Where to Go From Here? 308
Chapter 11: Managing Data With Property Wrappers 309
Getting Started 310
Tools for Managing Data 311
Managing UI State Values 313
Accessing Environment Values 320
Managing Model Data Objects 322
Wrapping Up Property Wrappers 328
Key Points 331
Chapter 12: Apple App Development Ecosystem 332
A Brief History of SwiftUI 333
SwiftUI vs. UIKit 334
Apple Developer 337
Housekeeping & Trouble-Shooting 347
Key Points 354
Section II: Your Second App: Cards 355
Chapter 13: Outlining a Photo Collage App 356
Initial App Idea 357
Creating the Project 358
Creating the First View for Your Project 359
Refactoring the View 360
Transitioning From List to Card 363

The Navigation Toolbar 365
The Bottom Toolbar 367
Adding Modal Views 373
Challenge 377
Key Points 378

Chapter 14: Gestures 379
Creating the Resizable View 380
Creating Transforms 381
Creating a Drag Gesture 383
Creating a Rotation Gesture 387
Creating a Scale Gesture 391
Creating Custom View Modifiers 392
Other Gestures 398
Type Properties 399
Challenge 404
Key Points 405
Where to Go From Here? 405

Chapter 15: Structures, Classes & Protocols 406
Data Structure 407
Value and Reference Types 408
Swift Dive: Structure vs Class 409
Creating the Card Store 410
Class Inheritance 412
Protocols 413
The Preview Data 416
Listing the Cards 417
Mutability 420
Understanding @State and @Binding Property Wrappers 426
Key Points 432
Where to Go From Here? 432

Chapter 16: Adding Assets to Your App 433
The Asset Catalog 434
Launch Screen 440
Adding Sticker Images to Your App 445
Using Lazy Grid Views 456
Challenges 462
Key Points 463
Where to Go From Here? 463
Chapter 17: Adding Photos to Your App 464
The PhotosUI Framework 465
The PhotosPicker View 465
The Transferable Protocol 470
Drag and Drop From Other Apps 475
Pasting From Another App 482
Adding a Pop-up Menu 484
Challenge 491
Key Points 492
Chapter 18: Paths & Custom Shapes 493
The Starter Project 494
Shapes 494
Paths 496
Strokes and Fills 504
Selecting an Element 506
Associated Types 509
Adding the Frame Picker Modal to the Card 514
Challenges 521
Key Points 523
Chapter 19: Saving Files 524
The Starter Project 525
The Saved Data Format 525

When to Save the Data 526
JSON Files 529
Codable 529
Encoding and Decoding Custom Types 533
Loading Cards 542
Creating new Cards 544
Challenges 547
Key Points 550

Chapter 20: Delightful UX — Layout 551
The Starter app 552
Designing the Cards List 553
Layout 555
Adding a Lazy Grid View 559
Scaling the Card to fit the Device 569
Alignment 576
Challenges 579
Key Points 581

Chapter 21: Delightful UX — Final Touches 582
The Starter Project 583
Animated Splash Screen 583
SwiftUI Animation 588
Animated Transitions 594
Supporting Multiple View Types 597
Sharing the Card 600
Challenges 607
Key Points 610
Where to Go From Here? 610

Section III: Your Third App: TheMet 611

Chapter 22: Lists & Navigation 612
Getting Started 613

List . . . 613
NavigationStack . . . 614
Using the Internet . . . 617
navigationDestination . . . 622
Using Custom Colors . . . 630
One Last Thing . . . 633
Key Points . . . 637

Chapter 23: Just Enough Web Stuff . . . 638
Servers & Resources . . . 639
REST API . . . 641
Sending & Receiving HTTP Messages . . . 642
Exploring metmuseum.org . . . 646
POST Request & Authentication . . . 652
Key Points . . . 656

Chapter 24: Downloading Data . . . 657
Getting Started . . . 658
URLSession . . . 658
Creating a REST Request URL . . . 660
Sending the Request With URLSession . . . 665
Decoding JSON . . . 666
Downloading Objects . . . 670
Downloading Data in Your App . . . 671
Showing a Progress View . . . 683
Key Points . . . 684

Chapter 25: Widgets . . . 685
Getting Started . . . 686
WidgetKit . . . 686
Adding a Widget Extension . . . 687
Creating Entries From Your App's Data . . . 694
Creating Widget Views . . . 697

Providing a Timeline Of Entries 706
Deep-Linking Into Your App 715
A Few Last Things 719
Key Points 721
Conclusion 722

Book License

By purchasing *SwiftUI Apprentice*, you have the following license:

- You are allowed to use and/or modify the source code in *SwiftUI Apprentice* in as many apps as you want, with no attribution required.
- You are allowed to use and/or modify all art, images and designs that are included in *SwiftUI Apprentice* in as many apps as you want, but must include this attribution line somewhere inside your app: "Artwork/images/designs: from *SwiftUI Apprentice*, available at www.kodeco.com".
- The source code included in *SwiftUI Apprentice* is for your personal use only. You are NOT allowed to distribute or sell the source code in *SwiftUI Apprentice* without prior authorization.
- This book is for your personal use only. You are NOT allowed to reproduce or transmit any part of this book by any means, electronic or mechanical, including photocopying, recording, etc. without previous authorization. You may not sell digital versions of this book or distribute them to friends, coworkers or students without prior authorization. They need to purchase their own copies.

All materials provided with this book are provided on an "as is" basis, without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose and noninfringement. In no event shall the authors or copyright holders be liable for any claim, damages or other liability, whether in an action of contract, tort or otherwise, arising from, out of or in connection with the software or the use or other dealings in the software.

All trademarks and registered trademarks appearing in this guide are the properties of their respective owners.

Before You Begin

This section tells you a few things you need to know before you get started, such as what you'll need for hardware and software, where to find the project files for this book, and more.

What You Need

To follow along with this book, you'll need the following:

- A **Mac computer with an Intel or ARM processor**. Any Mac that you've bought in the last few years will do, even a Mac mini or MacBook Air.
- **Xcode 14.2 or later**. Xcode is the main development environment for building iOS Apps. It includes the Swift compiler, the debugger and other development tools you'll need. You can download the latest version of Xcode for free from the Mac App Store.
- Optionally, an **iPhone, iPad or iPod Touch** running iOS 16.

Book Source Code & Forums

Where to Download the Materials for This Book

The materials for this book can be cloned or downloaded from the GitHub book materials repository:

- https://github.com/kodecocodes/suia-materials/tree/editions/2.0

Forums

We've also set up an official forum for the book at https://forums.kodeco.com/c/books/swiftui-apprentice. This is a great place to ask questions about the book or to submit any errors you may find.

"To the kodeco.com community, who help create my happy place."

— *Audrey Tam*

"To my children, Robin and Kayla, who make my real life a better place."

— *Caroline Begbie*

About the Authors



Audrey Tam is an author of this book. As a retired computer science academic, she's a technology generalist with expertise in translating new knowledge into learning materials. Audrey attends many concerts at Tempo Rubato and does most of her writing and zooming at Rubato Upstairs. She also enjoys long train journeys, knitting, and trekking in the Aussie wilderness.



Caroline Begbie is another author of this book. Caroline ventured out into the world as an unemployed classicist. She then taught herself to code and started up a software company in the UK and later in the US. Retiring to Australia prematurely, she performed marionette shows for pre-schools until she purchased the original iPad. She recognized the creative possibilities and became an indie iOS developer. Now she loves tinkering with vertices and watching Disney movies.

About the Editors



Libranner Santos is the tech editor on this book. He's a software engineer with more than 10 years of experience. Basketball fan and player, and a decent dancer. Love learning and teaching at all levels. You can follow him on Twitter as @libranner.



Richard Critz did double duty as editor and final pass editor for this book. He has been doing software professionally for over 40 years, working on products as diverse as CNC machinery, network infrastructure, and operating systems. He discovered the joys of working with iOS beginning with iOS 6. Yes, he dates back to punch cards and paper tape. He's a dinosaur; just ask his kids.

V How to Read This Book

This book is designed to take you from zero to hero! Each chapter builds on the code and concepts from its predecessors, so you'll want to work your way through them in order. To help you navigate on your journey, here are some conventions we use:

- Filenames, text you enter into dialog boxes, items you look for on screen all appear in **bold**.
- Names of things you find in your code — such as variables, properties, types, protocols and method names — appear in a `monospaced typeface`.
- The ➤ icon indicates an instruction step for you to follow.
- Quick tips about Xcode are marked with **Xcode Tip**.
- Quick tips about Swift are marked with **Swift Tip**.
- Deeper explanations of Swift language topics are marked with **Swift Dive**.
- Watch for **Skills you'll learn in this section** to get a quick overview of specific new things you'll learn.

Section I: Your First App: HIITFit

At WWDC 2019, Apple surprised and delighted the developer community with the introduction of SwiftUI, a declarative way of building user interfaces. With SwiftUI, you build your user interface by combining fundamental components such as colors, buttons, text labels, lists and more into beautiful and functional views. Your views react to changes in the data they display, updating automatically without any intervention from you!

In this section, you'll begin your journey to becoming a SwiftUI developer by developing an app called HIITFit, a High Intensity Interval Training Fitness tracker. Along the way, you'll:

- Learn how to use Xcode.
- Discover how to plan and prototype an app.
- Explore the basic components of SwiftUI.
- Understand how data moves in a SwiftUI app and how to make it persist.
- Learn fundamental concepts of Swift, the programming language, needed to build your app.

Chapter 1: Checking Your Tools

By Audrey Tam

You're eager to dive in and create your first iOS app. If you've never used Xcode before, take some time to work through this chapter. You want to be sure your tools are working and learn how to use them efficiently.

Getting Started

To develop iOS apps, you need a Mac with Xcode installed. If you have an account on GitHub or similar, you can connect to that from Xcode.

macOS

To use the SwiftUI canvas, you need a Mac running **Catalina (V10.15)** or later. To install Xcode, your user account must have **administrator** status.

Xcode

To install Xcode, you need **23 GB** free space on your Mac's drive.

➤ Open the **App Store** app, then search for and **GET Xcode**. This is a large download — almost 8GB — so it takes a while. Fix yourself a snack while you wait or, to stay in the flow, browse Chapter 12, "Apple App Development Ecosystem".

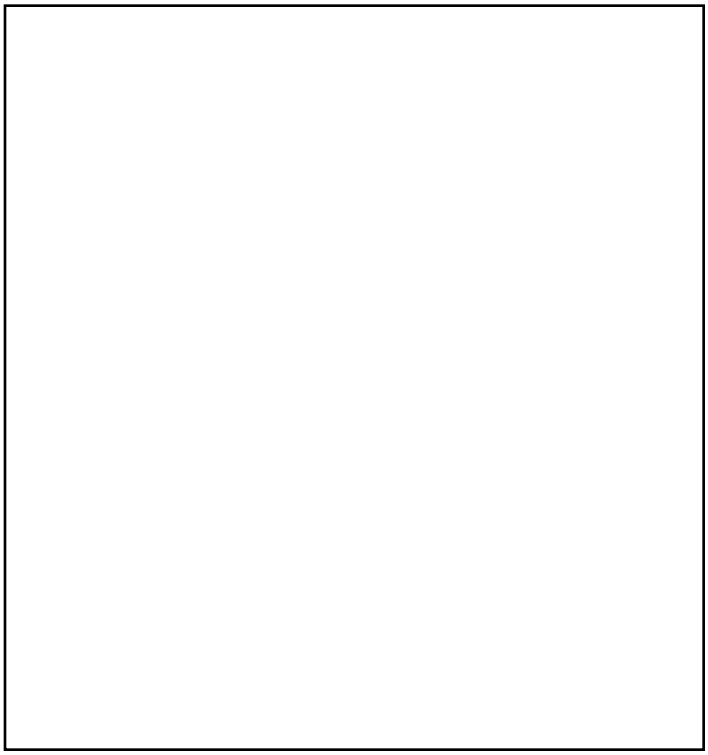
➤ When the installation finishes, **OPEN** it from the App Store page:



Open Xcode after installing it from the App Store.

> **Note**: You probably have your favorite way to open a Mac application, and it will work with Xcode, too. Double-click it in **Applications**. Or search for it in **Spotlight**. Or double-click a project's **.xcodeproj** file.

The *first* time you open Xcode after App Store installation, you'll see this window:



Select the platforms you would like to install:

➤ iOS and macOS are built into the 23 GB. Click **Install** and enter your **Mac login password** in the window that appears. This doesn't take long, so don't go away.

➤ When this installation process finishes, you'll see this **Welcome** window:



Welcome to Xcode window

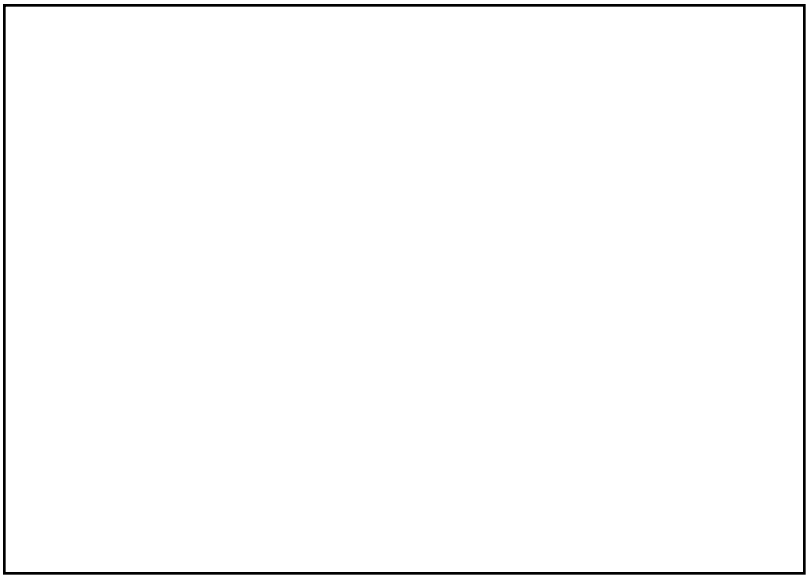
If you don't want to see this window every time you open Xcode, uncheck "Show this window when Xcode launches". You can manually open this window from the Xcode menu **Window ▸ Welcome to Xcode** or press **Shift-Command-1**. And, there is an Xcode menu item to perform each of the actions listed in this window.

Creating a New Xcode Project

You'll create a new Xcode project just for this chapter. The next chapter provides a starter project that you'll build on for the rest of Section 1.

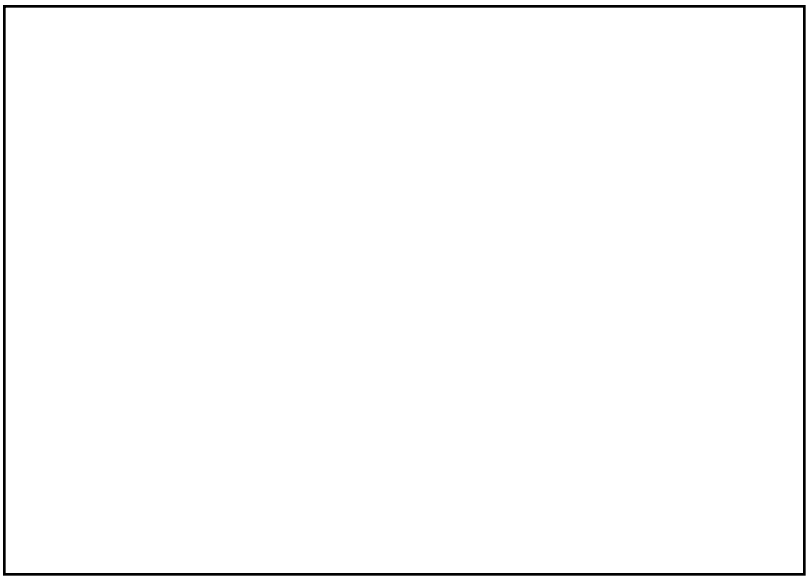
➤ Click **Create a new Xcode project**. Or, if you want to do this without the Welcome window, press **Shift-Command-N** or select **File ▸ New ▸ Project…** from the menu.

A large set of choices appears:



Choose a template for your new project.

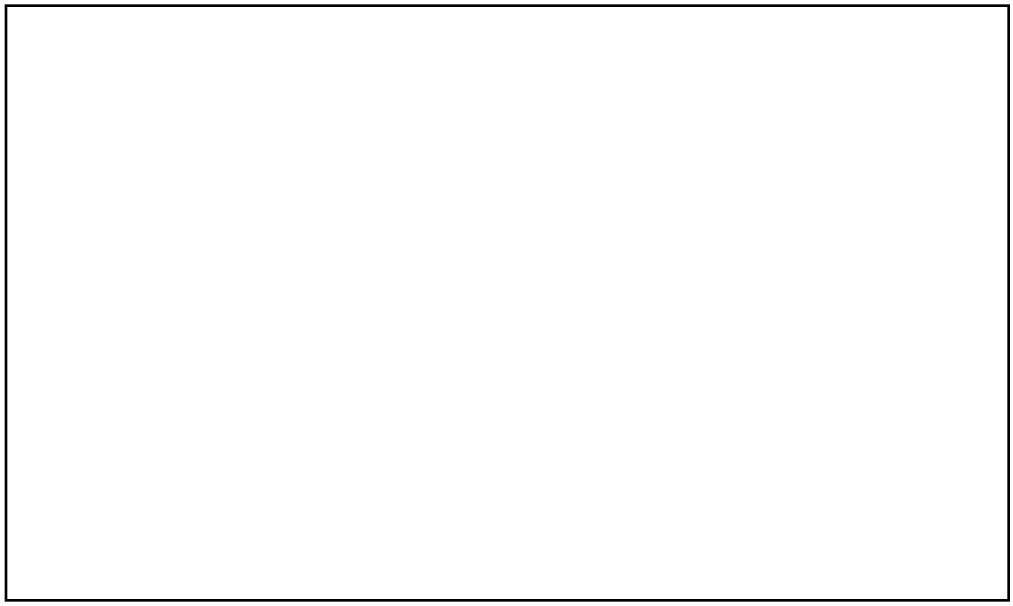
➤ Select **iOS** ‣ **App** and click **Next**. Now, you get to name your project:



Choose options for your new project.

- For **Product Name**, type **MyFirst**.
- Skip **Team** as **None** for now.
- For **Organization Identifier**, type the *reverse-DNS* of your domain name. If you don't have a domain name, just type something that follows this pattern, like **org.audrey**. The grayed-out **Bundle Identifier** changes to **your-org-id.MyFirst**. When you submit your app to the App Store, this bundle identifier uniquely identifies your app.
- For **Interface**, select **SwiftUI**.
- For **Language**, select **Swift**.
- Uncheck the checkboxes.

➤ Click **Next**. Here's where you decide where to save your new project.

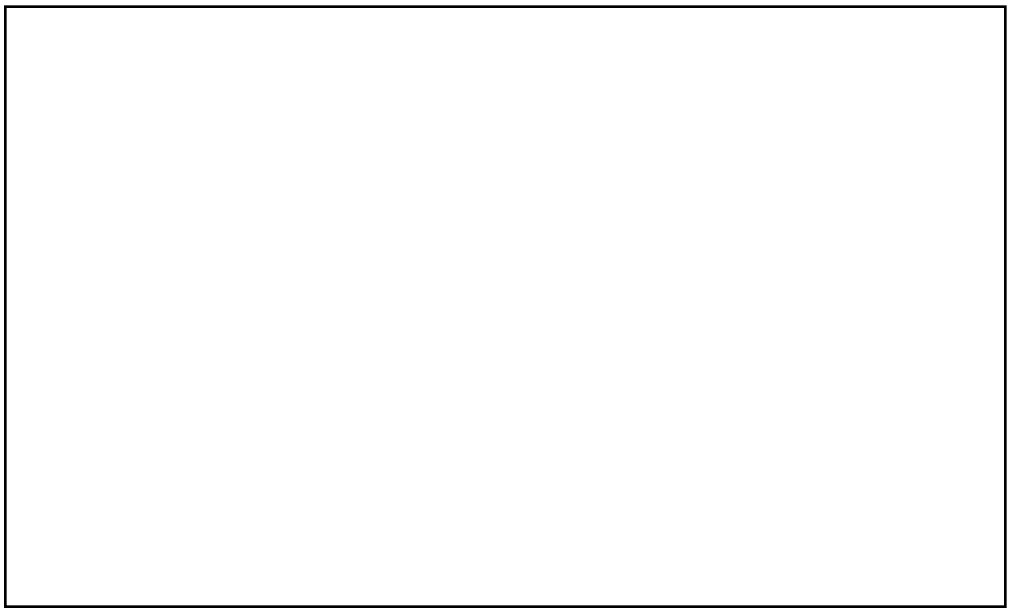


Decide where to save your project.

> **Note**: When a project is open, you can find its location by selecting **File ▸ Show in Finder** from the Xcode menu. If you forget where you saved a project, try looking in the Xcode menu **File ▸ Open Recent**.

➤ If you're saving this project to a location that is **not** currently under source control, click the **Source Control** checkbox to create a local Git repository. Later in this chapter, you'll learn how to connect this to a remote repository.

➤ Click **Create**. Your new project appears, displaying **ContentView.swift** in the editor pane.



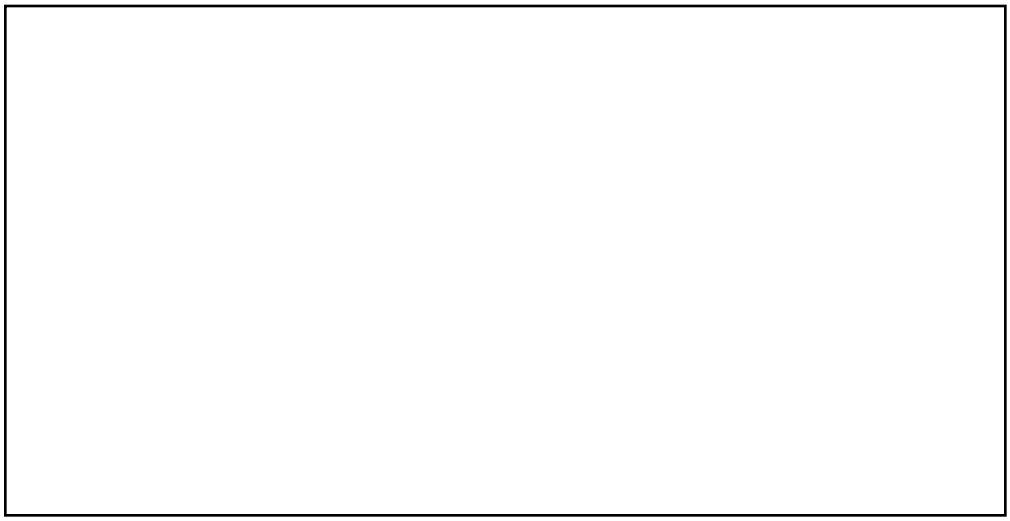
New project appears with ContentView.swift in the editor.

Looks like there's a lot going on! Don't worry, most iOS developers know enough about Xcode to do their job, but almost no one knows how to use *all* of it. Plus, Apple changes and adds to it every year. The best (and only) way to learn it is to jump in and start using it.

Ready, set, jump!

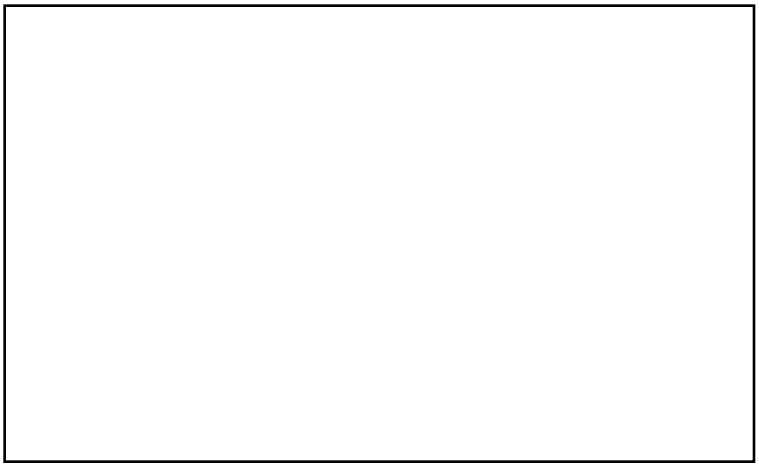
A Quick Tour of Xcode

You'll spend most of your time working in a **.swift** file:



Xcode window panes

Note: If you don't see the file extension `.swift` in the **Project** navigator, press **Command-,** to open **Settings...** then, in the **General** tab, set **File Extensions** to **Show All**:



Show all file extensions.

The Xcode window has three main panes: **Navigator**, **Editor** and **Inspectors**. When the app is running, the **Debug Area** opens below the **Editor**. When you're viewing a SwiftUI View file in the **Editor**, you can view the preview canvas side-by-side with the code.

The toolbar button just above the Navigator pane hides or shows it. The same is true for the Inspectors pane. The debug area has a hide/show button in its own toolbar. You can also hide any of these three panes by dragging its border to the edge of the Xcode window.

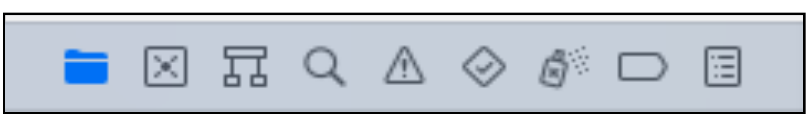
And all three have keyboard shortcuts:

- Hide/show Navigators: **Command-0**
- Hide/show Inspectors: **Option-Command-0**
- Hide/show Debug Area: **Shift-Command-Y**

> **Note**: There's a handy cheat sheet of Xcode keyboard shortcuts in the assets folder for this chapter. It's not a complete list, but it covers the ones people use most.

Navigator

The **Navigator** has nine tabs. When the navigator pane is hidden, you can open it directly in one of its tabs by pressing **Command-1** to **Command-9** (from left to right):



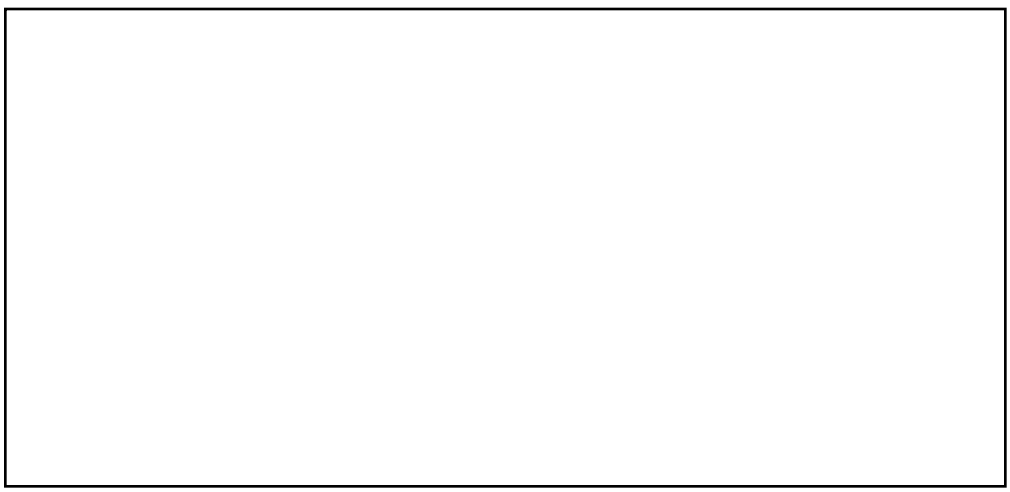
Navigator bar

1. **Project**: Add, delete or group files. Open a file in the editor.
2. **Source Control**: View Git repository working copies, branches, commits, tags, remotes and stashed changes.
3. **Symbol**: Hierarchical or flat view of the named objects and methods.
4. **Find**: Search tool.
5. **Issue**: Build-time and runtime errors and warnings.

6. **Test**: Create, manage and run unit and UI tests.
7. **Debug**: Information about CPU, memory, disk and network usage while your app is running.
8. **Breakpoint**: Add, delete, edit and manage breakpoints.
9. **Report**: View or export reports and logs generated when you build and run the project.

The **Filter** field at the bottom is different for each tab. For example, the **Project Filter** lets you show only the files you recently worked on. This is handy for projects with a lot of files in a deeply nested hierarchy.

Editor



Editor with Minimap

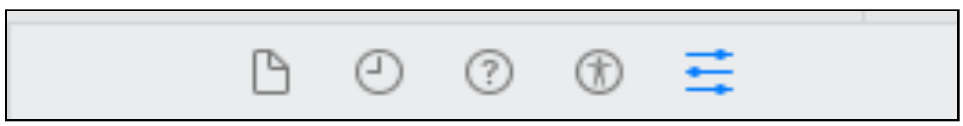
When you're working in a code file, the **Editor** shows the code and a **Minimap**. The minimap is useful for long code files with many properties and methods. You can hover the cursor over the minimap to locate a specific property, then click to go directly to it. You don't need it for the apps in this book, so you may want to hide it via the **Adjust Editor Options** button in the upper right corner of the editor.

When you're working in a SwiftUI file, **Option-Command-Return** shows or hides the preview **canvas**.

The editor has browser features like tab and go back/forward. Keyboard shortcuts for tabs are the same as for web browsers: **Command-T** to open a new tab, **Shift-Command-[** or **-]** to move to the previous or next tab, **Command-W** to close the tab and **Option-click** a tab's close button to close all the *other* tabs. The back/forward button shows a list of previous/next files, but the keyboard shortcuts are **Control-Command-right** or **-left arrow**.

Inspectors

The **Inspectors** pane has three, four or five tabs, depending on what's selected in the **Project** navigator. When this pane is hidden, you can open it directly in one of its tabs by pressing **Option-Command-1** to **Option-Command-5**:



Inspector bar

1. **File**: Name, Full Path, Target Membership.
2. **History**: Source Control log.
3. **Quick Help**: Short form of Developer Documentation if you select a symbol in the editor.
4. **Accessibility**: Accessibility information if you select a symbol in the preview.
5. **Attributes**: Properties of the symbol you select in the editor.

All five tabs appear when you select a file in the Project navigator. If you select a folder, you get only the first three tabs. If you select **Assets.xcassets**, you don't get the accessibility tab.

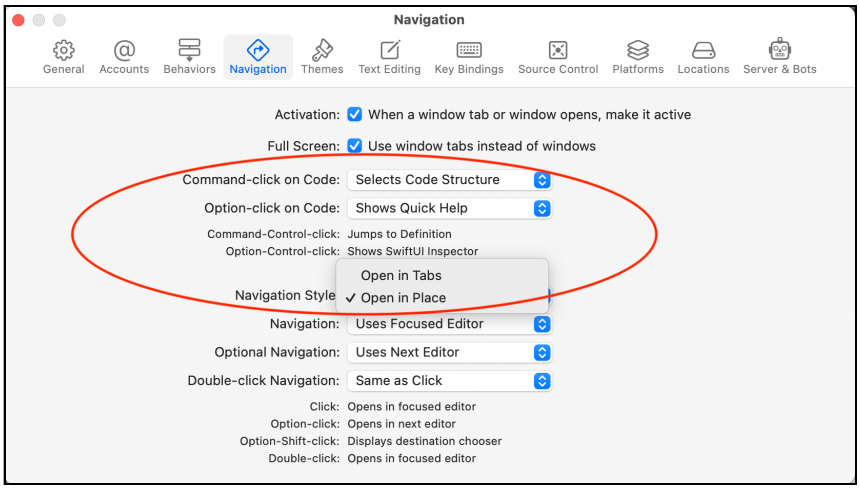
This quick tour just brushes the surface of what you can do in Xcode. Next, you'll use a few of its tools while you explore your new project.

Navigation Settings

In this book, you'll use keyboard shortcuts to examine and structure your code. Unlike the fixed keyboard shortcuts for opening navigator tabs or inspectors, you can specify settings for which shortcut does what. To avoid confusion while working through this book, you'll adjust your settings to match the instructions you'll see.

➤ Press **Command-,** to open **Settings**. In the **Navigation** tab, set:

- **Command-click on Code** to **Selects Code Structure**
- **Option-click on Code** to **Shows Quick Help**
- **Navigation Style** to your choice of **Open in Tabs** or **Open in Place**.



Adjust Navigation Settings.

ContentView.swift

The heart of your new project is in **ContentView.swift**, where your new project opened. This is where you'll lay out the initial view of your app.

➤ If **ContentView.swift** isn't in the editor, select it in the Project navigator.

The first several lines are comments that identify the file and you, the creator.

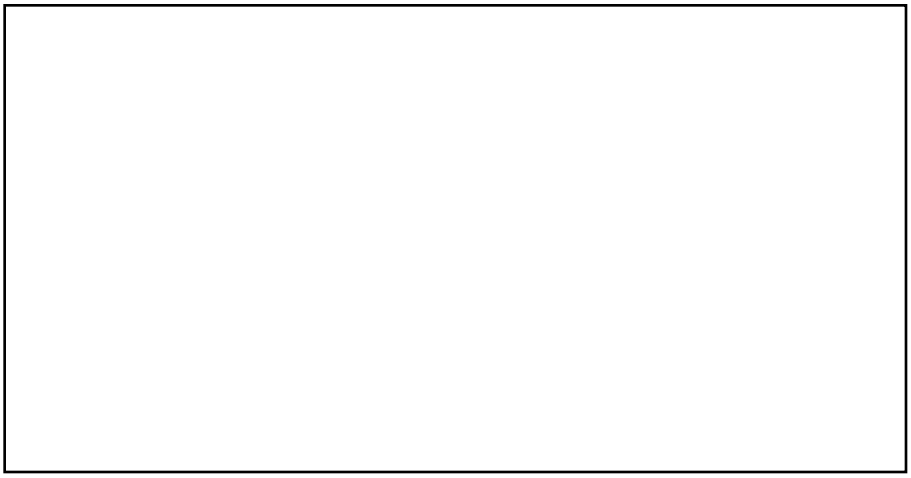
import

The first line of code is an `import` statement:

```
import SwiftUI
```

This works just like most programming languages: It allows your code to access everything in the built-in SwiftUI module. See what happens if it's missing.

➤ Click the `import` statement line, then press **Command-/**.



What happens if import SwiftUI is missing

> **Note**: All code on our site uses 2-space indentation to save space. Xcode defaults to 4-space indentation. You can change this in **Xcode Settings ▸ Text Editing ▸ Indentation**.

You commented out the `import` statement, so compiler errors appear, complaining about `View` and `PreviewProvider`.

➤ Press **Command-Z** to undo.

Below the `import` statement are two `struct` definitions. A *structure* is a named data type that encapsulates properties and methods.

struct ContentView

The name of the first structure matches the name of the file. Nothing bad happens if they're different, but most developers follow and expect this convention.

```
struct ContentView: View {
  var body: some View {
    VStack {
      Image(systemName: "globe")
        .imageScale(.large)
        .foregroundColor(.accentColor)
      Text("Hello, world!")
```

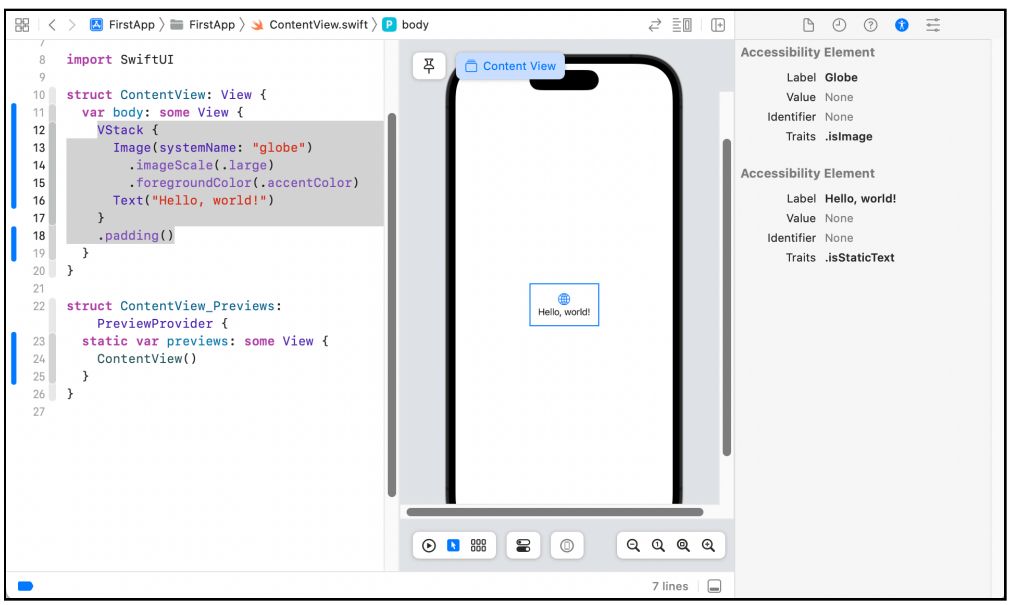
```
      }
      .padding()
    }
  }
```

Looking at `ContentView: View`, you might think `ContentView` *inherits* from `View`, but Swift structures don't have inheritance. `View` is a *protocol*, and `ContentView` *conforms* to this protocol.

The required component of `View` is the computed property `body`, which returns a `View`. In this case, it returns a `VStack` (vertical stack) that displays a globe image and the usual "Hello, world!" text.

> **Swift Tip**: A computed property returns the computed value. If there's only a single code statement, you don't need to explicitly use the `return` keyword.

The `VStack` view has a `padding` modifier — an instance method of `View` — that adds space around the stack. You can see it in this screenshot:



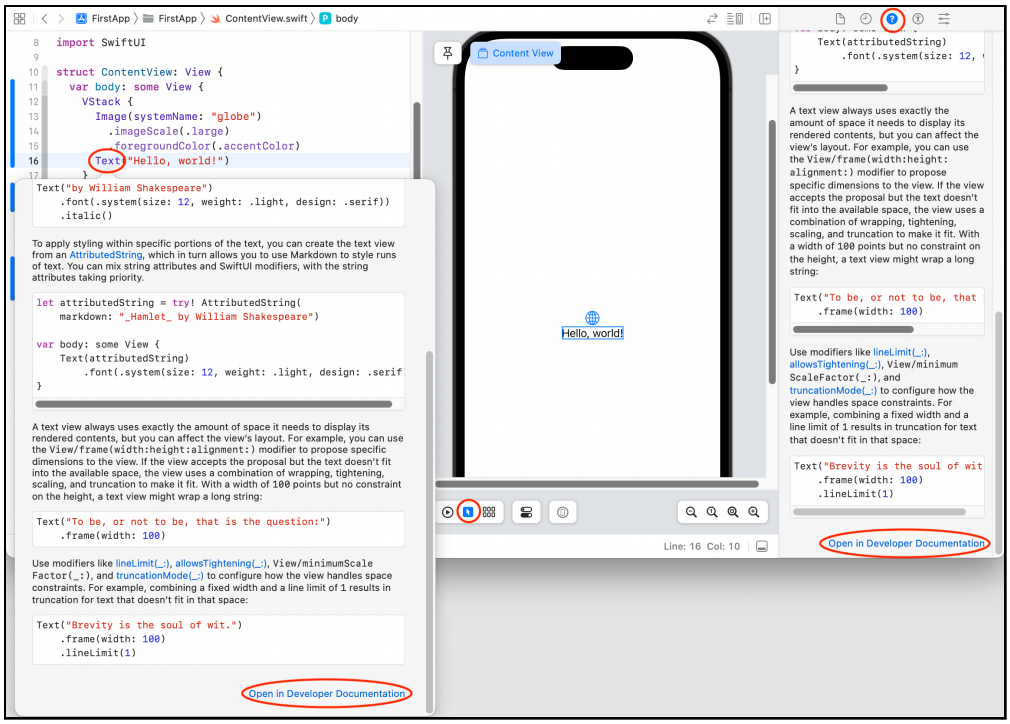
VStack padding and Accessibility Inspector

This also shows the **Accessibility** inspector for the `Image` and `Text` subviews in `VStack` — label, value, identifier and traits.

`Image(systemName: "globe")` and its modifiers display a globe symbol. You'll learn about the `Image` view in Chapter 3, "Prototyping the Main View".

`Text("Hello, world!")` displays the string "Hello, world!".

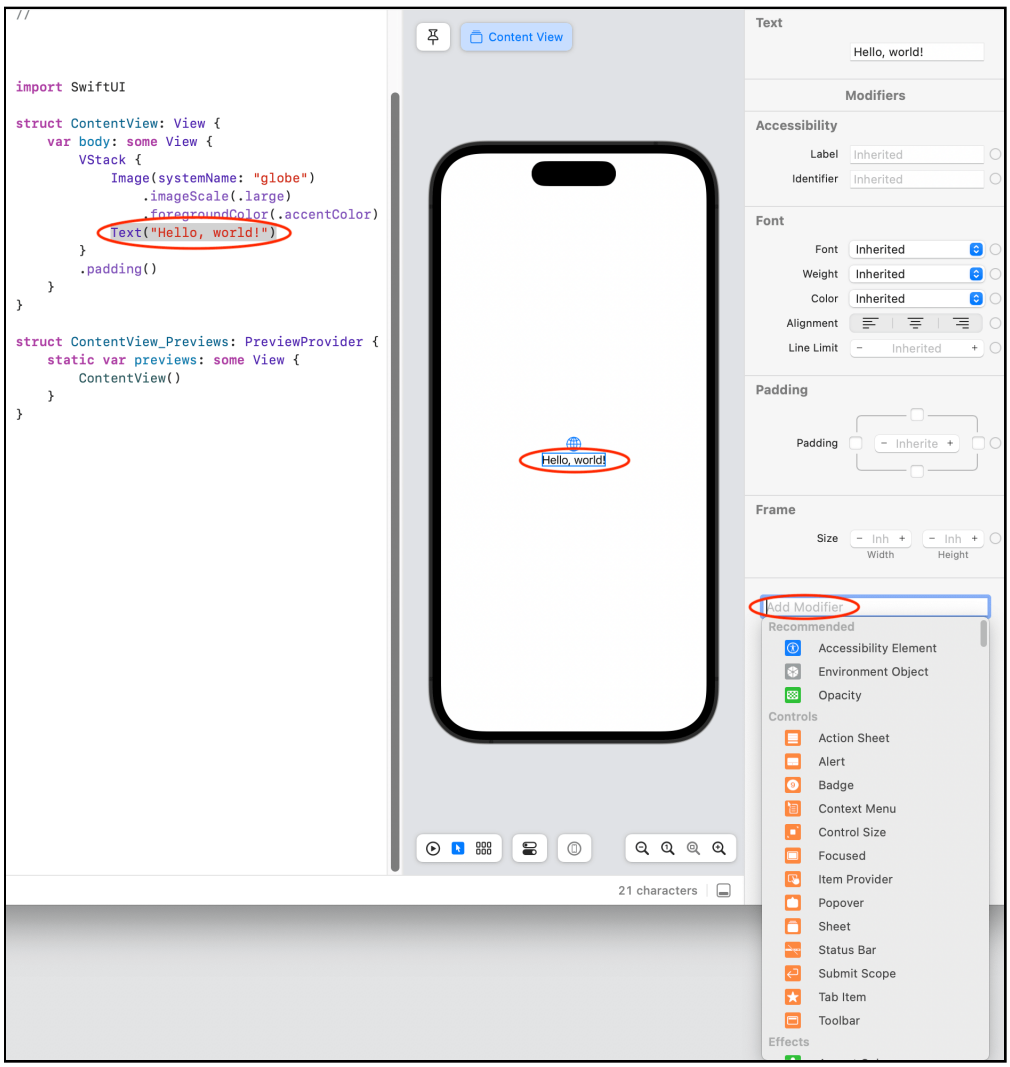
➤ Click the **Selectable** button below the preview canvas, then click the `Text` view in the canvas and select the **Quick Help inspector**.



Text Quick Help and Developer Documentation

If you don't want to use screen real estate for this inspector, **Option-click** `Text` in the code editor to see the same information in a pop-up window. Scroll down in either window to see the **Open in Developer Documentation** button, which opens a window with more information.

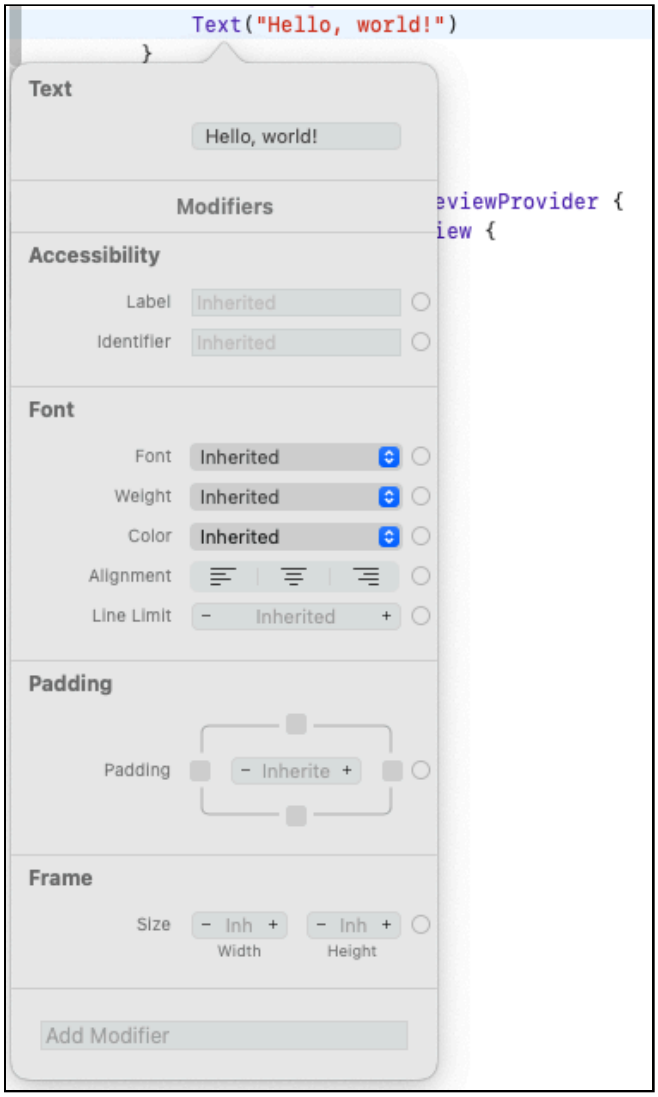
➤ Now, select the **Attributes inspector**. Click in the **Add Modifier** field and wait a short while until the modifiers menu appears:



Text attributes inspector and modifiers menu

Scrolling through this list goes on and on and on.

This inspector is useful when you want to add several modifiers to a `View`. If you just need to add one modifier, **Control-Option-click** the view **in the code editor** to open the Attributes inspector pop-up window.



Attributes inspector pop-up window

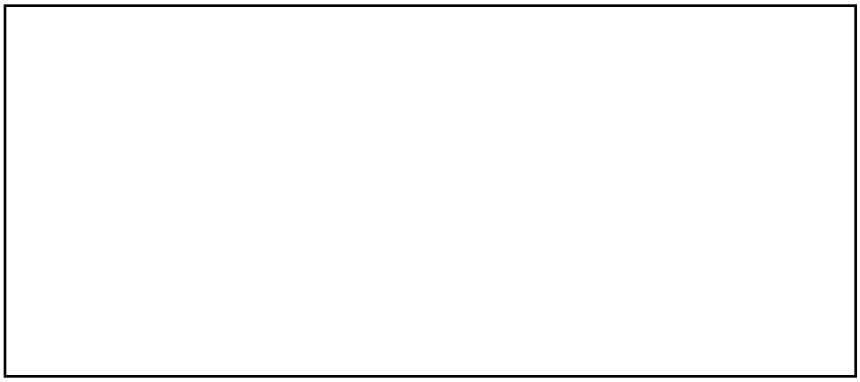
struct ContentView_Previews

Below `ContentView` is a `ContentView_Previews` structure.

```
struct ContentView_Previews: PreviewProvider {
  static var previews: some View {
    ContentView()
  }
}
```

The `ContentView_Previews` structure is what appears in the canvas on the right of the code editor. See what happens if it's missing.

➤ Select the five lines of `ContentView_Previews` and press **Command-/**.



No PreviewProvider so there's nothing in the canvas.

Without `ContentView_Previews`, there's nothing in the canvas.

➤ Press **Command-Z** to undo or, if the five lines are still selected, press **Command-/** to uncomment them.

You'll sometimes want to give more space to the code editor, so your code doesn't have to wrap.

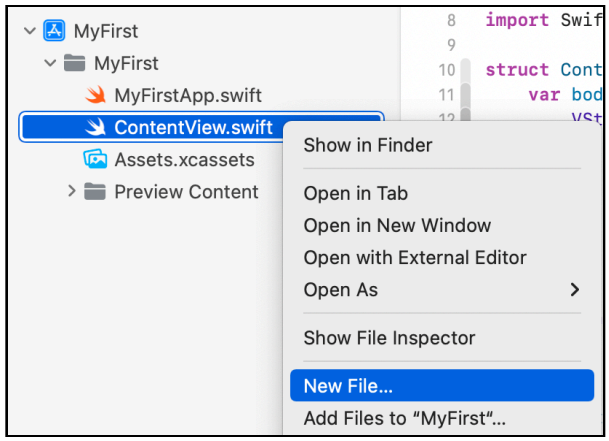
➤ Press **Option-Command-Return** to hide the canvas. Press the same keyboard shortcut to show the canvas.

For most apps, **ContentView.swift** is just the starting point. Often, `ContentView` only defines the app's organization, orchestrating several subviews. And usually, you'll define these subviews in separate files.

Creating a New SwiftUI View File

Everything you see in a SwiftUI app is a `View`. Apple encourages you to create as many subviews as you need in order to avoid redundancy and organize your code to keep it manageable. The compiler takes care of creating efficient machine code so your app's performance won't suffer.

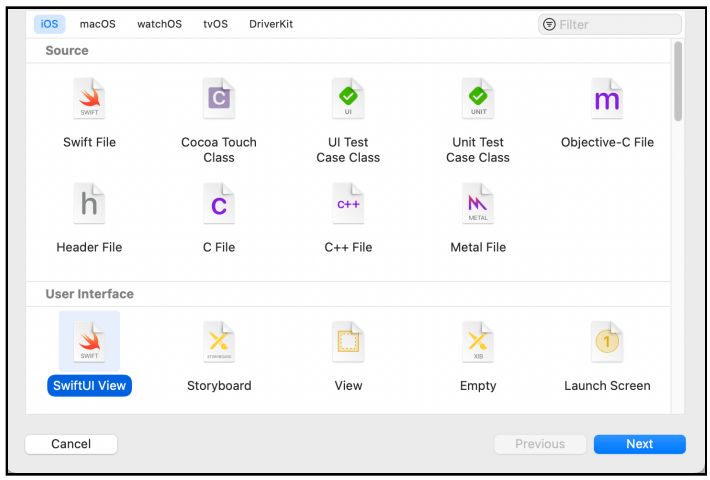
➤ In the Project navigator, select **ContentView.swift** and press **Command-N**. Alternatively, right-click **ContentView.swift** then select **New File…** from the menu.



Select New File... from right-click menu.

Xcode Tip: A new file appears in the Project navigator below the currently selected file. If that's not where you want it, drag it to where you want it to appear in the Project navigator.

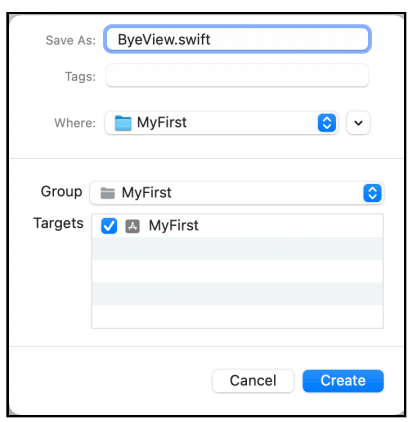
The new file window displays a lot of options! The one you want is **iOS ▸ User Interface ▸ SwiftUI View**. In Chapter 3, "Prototyping the Main View", you'll get to create a **Swift File**.



Choose iOS ▸ User Interface ▸ SwiftUI View.

Naming a New SwiftUI View

➤ Select **SwiftUI View** then click **Next**. The next window lets you specify a file name. By default, the name of the new view will be the same as the file name. You'll define the `ByeView` in this file, so replace **SwiftUIView** with **ByeView**.



SwiftUI view file name matches the new SwiftUI View.

> **Swift Tip**: Swift convention is to name types (like `struct`) with **UpperCamelCase** and properties and methods with **lowerCamelCase**.

This window also lets you specify where in the project to create your new file. The default location is usually correct — in this project, in this group (folder) and in this target.

➤ Click **Create** to finish creating your new file.

The template code for a SwiftUI view is simpler than the `ContentView` of a new project.

```
import SwiftUI

struct ByeView: View {
  var body: some View {
    Text("Hello, World!")
  }
}

struct ByeView_Previews: PreviewProvider {
  static var previews: some View {
    ByeView()
  }
}
```

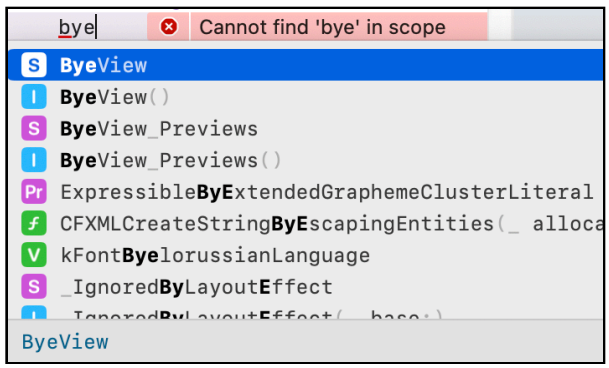
The view's body contains only `Text("Hello, World!")` — no `Image`, so you don't need a `VStack`, and no `padding`. Another subtle difference: The "Hello, World!" string is a *token*. It's just a placeholder. Clicking anywhere in it highlights the token so you can type in a new value.

Using Your New SwiftUI View

➤ Click the "Hello, World!" token and change it so the line looks like this:

```
Text("Bye bye, World!")
```

➤ Next, in **ContentView.swift**, in the code editor, delete the `Text` view, then type **bye**. Xcode suggests some auto-completions:



Xcode suggests auto-completions.

Notice you don't have to type the correct capitalization of `ByeView`.

> **Xcode Tip**: Descriptive names for your types, properties and methods is good programming practice, and auto-completion is one way Xcode helps you do the right thing. You can also turn on spell-checking from the Xcode menu: **Edit ▸ Format ▸ Spelling and Grammar ▸ Check Spelling While Typing**.

➤ Select **ByeView()** from the list, so the line looks like this:

```
ByeView()
```

You're calling the initializer of `ByeView` to create an instance of the view.

➤ In the canvas, if the preview is paused, click the refresh button:



Replace Text view with ByeView.

Your new `ByeView` has replaced the original `Text` view. You'll create many new SwiftUI view files and Swift files to develop the apps in this book.

What Else is in Your Project?

The Project navigator lists several files and folders.

- **MyFirstApp.swift**: This file contains the code for your app's entry point. This is what actually launches your app.

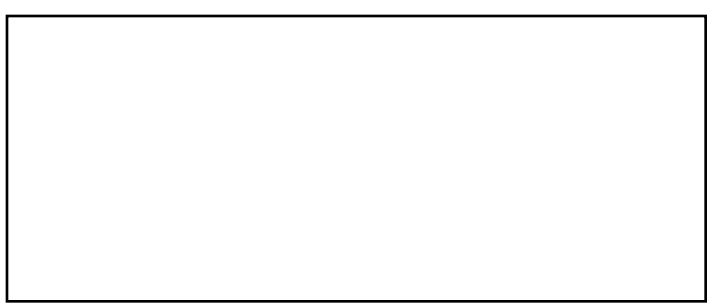
```
@main
struct MyFirstApp: App {
  var body: some Scene {
    WindowGroup {
      ContentView()
    }
  }
}
```

The `@main` attribute marks `MyFirstApp` as the app's entry point. You might be accustomed to writing a `main()` method to actually launch an app. The `App` protocol takes care of this. The `App` protocol requires only a computed property named `body` that returns a `Scene`. And a `Scene` is a container for the root view of a view hierarchy.

For an iOS app, the default setup is a `WindowGroup` scene containing `ContentView()` as its root view. A common customization is to set different root views, depending on whether the user has logged in.

In an iOS app, the view hierarchy fills the entire display. In a macOS or iPadOS app, `WindowGroup` can manage multiple windows.

- **Assets.xcassets**: Store your app's images and colors here. AppIcon is a special image set for all the different sizes and resolutions of your app's icon. Using Xcode 14, you can supply a single 1024x1024-point app icon image that Xcode resizes to get all the icon sizes for iOS, iPadOS and watchOS. Simply select **Single Size** in the Attributes inspector:



Assets: A Single Size app icon

- **Preview Content**: If your views need additional code and sample data or assets while you're developing your app, store them here. They won't be included in the final distribution build of your app.

In this list, the last item is a *group*. Groups in the Project navigator appear to be folders, but they don't necessarily match up with folders in Finder.

> **Note**: Don't rename or delete any of these files or groups. Xcode stores their path names in the project's build settings and will flag errors if it can't find them.

You'll learn how to use these files in the rest of this book.

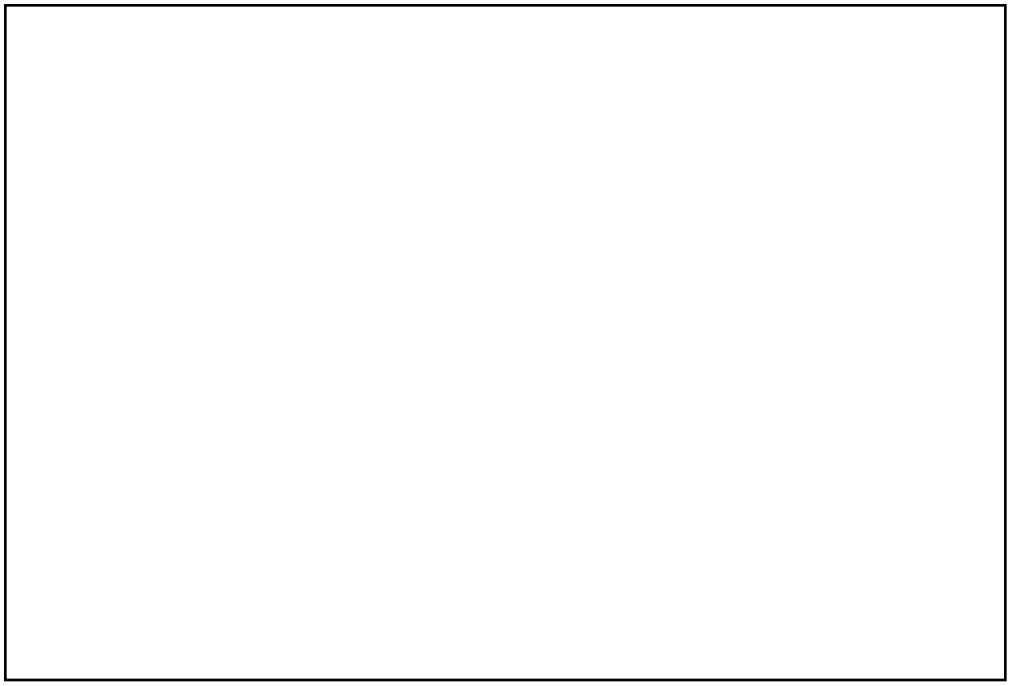
Xcode Settings

Xcode has a huge number of settings you can adjust to be more productive.

Themes

You'll be spending a lot of time working in the code editor, so you want it to look good and also help you distinguish the different components of your code. Xcode provides several pre-configured font and color themes for you to choose from or modify.

➤ Press **Command-,** to open **Settings**, then select the **Themes** tab:



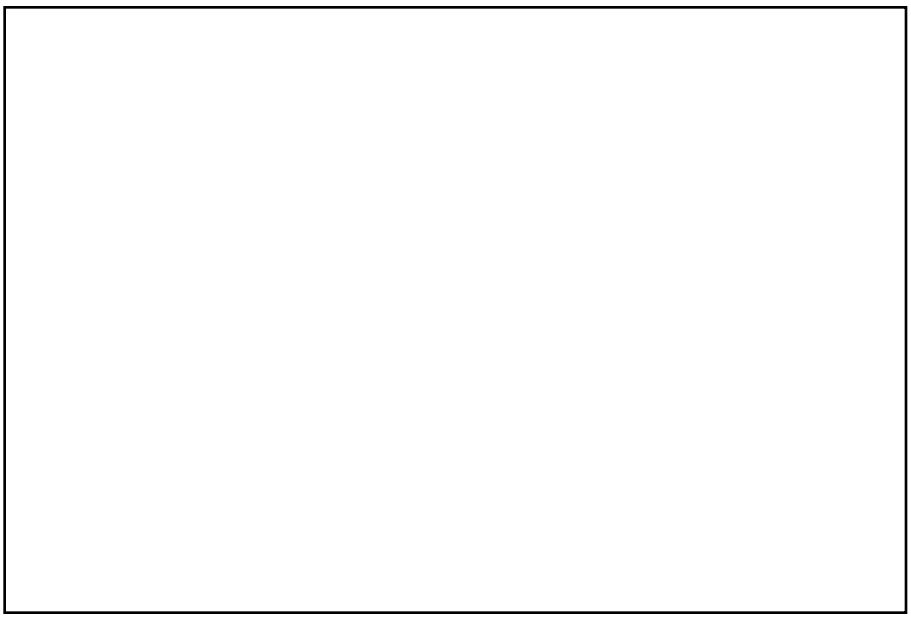
Settings: Font and color themes

Go ahead and explore these. You can customize them or create your own. I'll wait here. ;]

Matching Delimiters

SwiftUI code uses *a lot* of nested closures. It's really easy to mismatch your braces and parentheses. Xcode helps you find any mismatches and tries to prevent these errors from happening.

➤ In **Settings**, select **Text Editing ▸ Editing**:

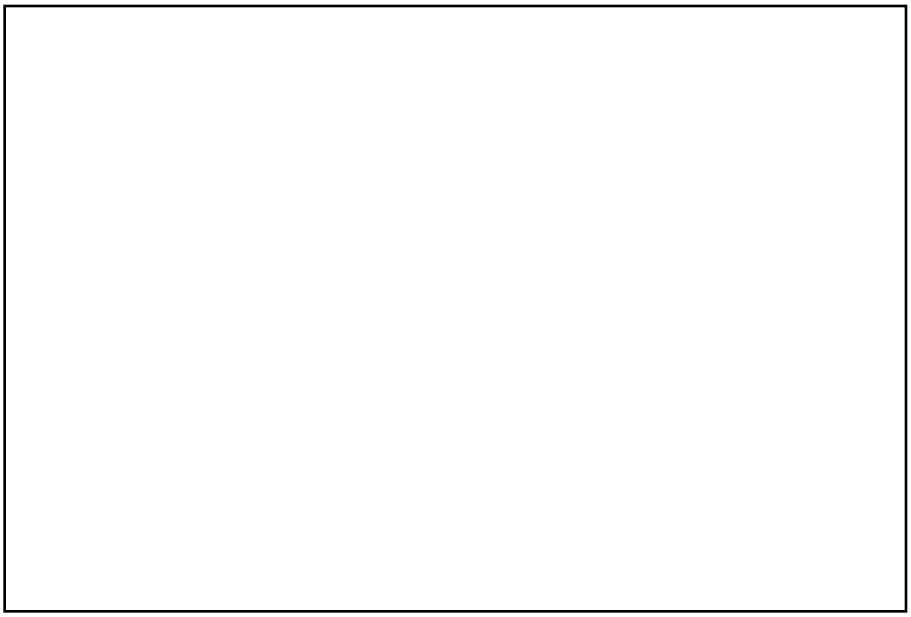


Settings ▸ Text Editing ▸ Editing

Most of the **Code Completion** items are super helpful. Although you can copy and paste code from this book, you should try to type the code as much as possible to learn how these aids work.

Here's a big hint that something's wrong or you're typing in the wrong place: You're expecting Xcode to suggest completions while you type, but nothing (useful) appears. When this happens, it's usually because you're outside the closure you need to be in.

➤ Now select the **Text Editing ▸ Display** tab. Check **Code folding ribbon** and, if you like to see them, **Line numbers**:

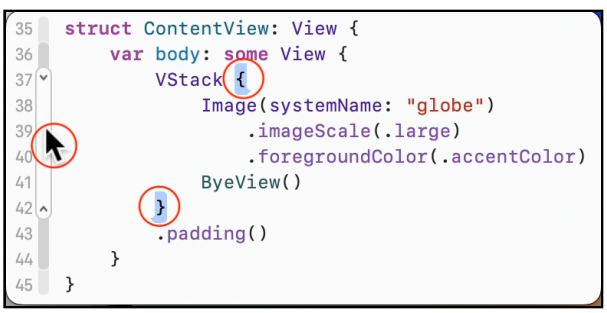


Settings ▸ Text Editing ▸ Display

So what's a *code folding ribbon*? Between the line numbers and the code, you see darker gray vertical bars.

➤ Hover your cursor over one — anywhere between `VStack {` and `padding()`.

It highlights the start and end braces of that closure:

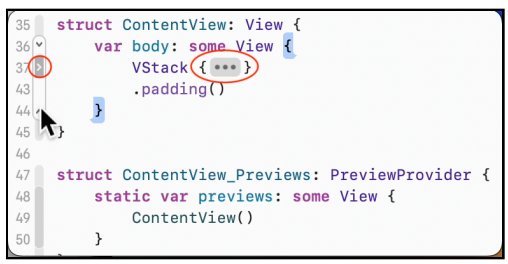


Code folding ribbon: Hover to show braces.

Other ways to see matching delimiters:

- **Option-hover** over **{**, **(**, **[** or a closing delimiter: Xcode highlights the start and end delimiters.
- **Double-click** a delimiter: Xcode selects the delimiters and their contents.

➤ Now click the bar (ribbon) to collapse (fold) those lines of code:



Code folding ribbon: Click to fold code.

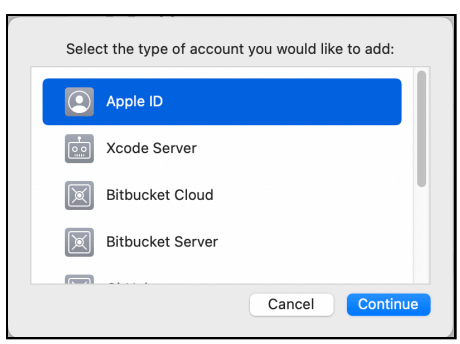
This can be incredibly useful when you're trying to find your way around some complex, deeply-nested code.

➤ Click the ribbon to unfold the code.

Adding Accounts

You can access some Xcode features by adding login information for your Apple ID and source control accounts.

➤ In **Settings**, select **Accounts**:



Settings ▸ Accounts

➤ Click the + button in the lower left corner and add your Apple ID. If you have a separate paid Apple Developer account, add that too.

To add capabilities like push notifications or Apple Pay to your app, you need to set **Team** to a Developer Program account. You'd do this in the **Signing & Capabilities** tab of the target.

- If you have an account at Bitbucket, GitHub or GitLab, add it here if you want to push your project's local git repository to a remote repository.

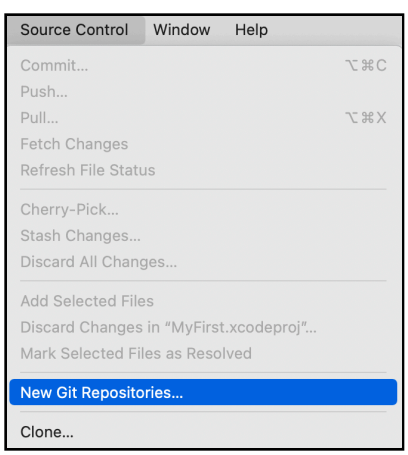


Add a source control account.

Bitbucket Server, GitHub and GitLab accounts require a *personal access token*. If the button to open GitHub's token-creation page doesn't work, open this URL (https://bit.ly/3M5HlBJ).

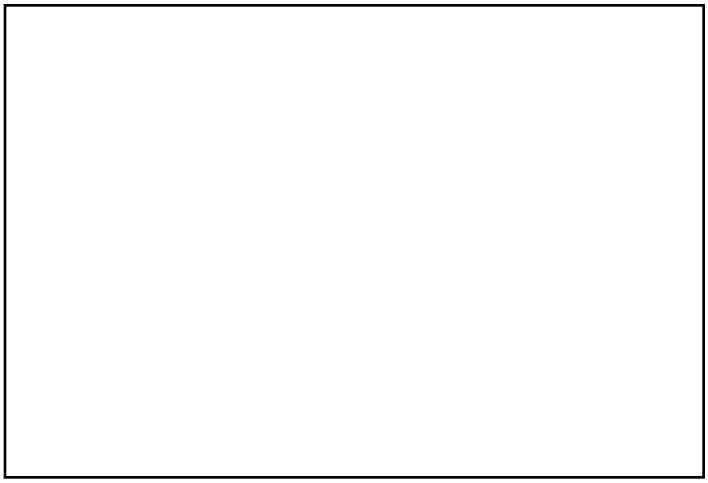
➤ To set up a remote repository, open the Xcode **Source Control** menu and select **New Git Repositories...**:

> If you remembered to check **Create Git repository on my Mac** when you created your project, you can skip this step.



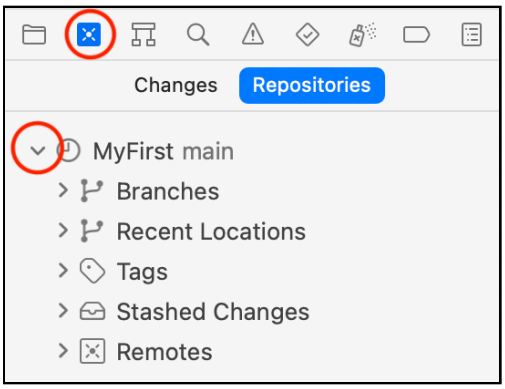
New Git Repositories...

➤ Click **Create**:



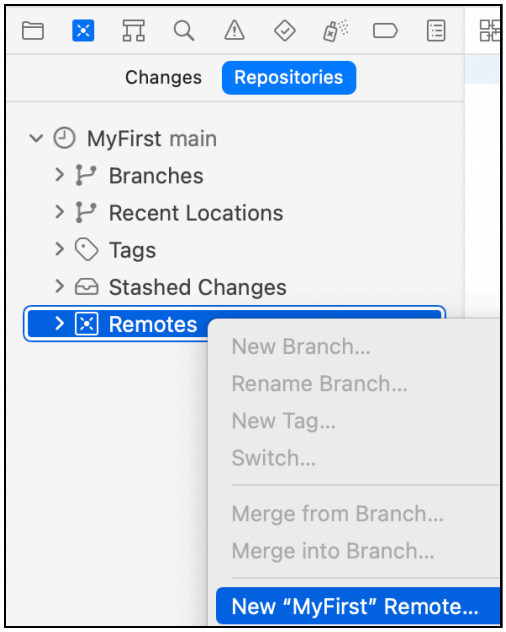
Create Git repository.

➤ Now, open the **Source Control navigator** (**Command-2**), click the **Repositories** tab and expand the repository:



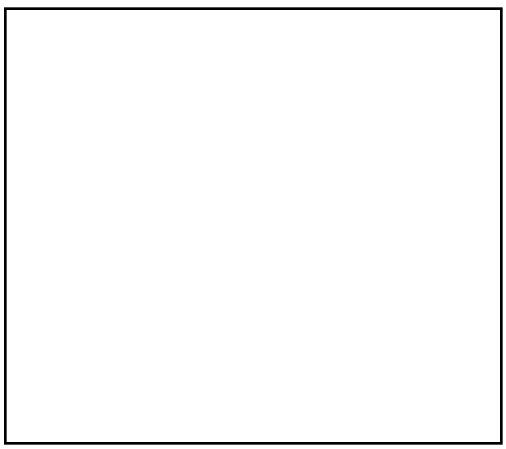
Expand Git repository in Source Control navigator.

➤ **Control-click Remotes** and select **New “MyFirst” Remote…**



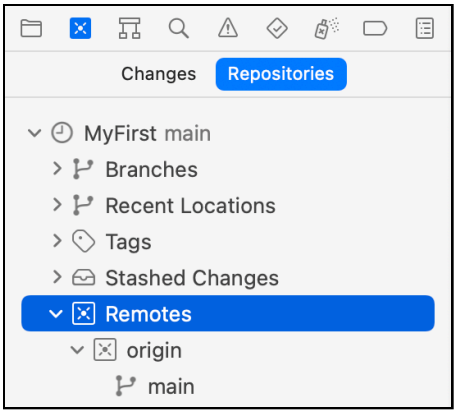
Control-click Remote, select New 'MyFirst' Remote…

➤ Select your options, then click **Create**:



Create-remote options

And here it is:



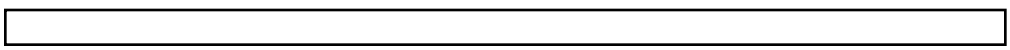
New remote repository created.

Running Your Project

So far, you've relied on Live Preview to see what your app looks like. In the next chapter, you'll use Live Preview to interact with your app. But some features don't work in Live Preview, so you need to build and run your app on a simulator. And some things will only work on an iOS device. Plus, it's fun to have something on your iPhone that you built yourself!

The Xcode Toolbar

First, a quick tour of Xcode's toolbar:



Xcode window toolbar

> **Xcode Tip**: Press **Option-Command-T** to show or hide the toolbar. If this keyboard shortcut conflicts with another app, select the command from the Xcode **View** menu.

So far, you've only used the buttons at either end of the toolbar, to show or hide the navigator or inspector panes.

Working from left to right after the navigation pane button:

- **Run** button: Build and run (**Command-R**) the project.
- **Stop** button appears when project is running: Stop (**Command-.**) the running project.
- **Source control** button: Shows branches and lets you create a pull request.
- **Scheme** menu: This button's label is the name of the app. Select, edit or manage *schemes*. Each product has a scheme. MyFirst has only one product, so it has only one scheme.
- **Run destination** menu: This menu defaults to iPhone 14 Pro. Select a connected device or a simulated device to run the project.
- **Activity** view: A wide gray field that shows the project name, status messages and warning or error indicators.
- **Library** button: This button's label is a + sign. It opens the library of views, modifiers, code snippets, media, colors stored in Assets and system symbols. **Option-click** this button to keep the library open.

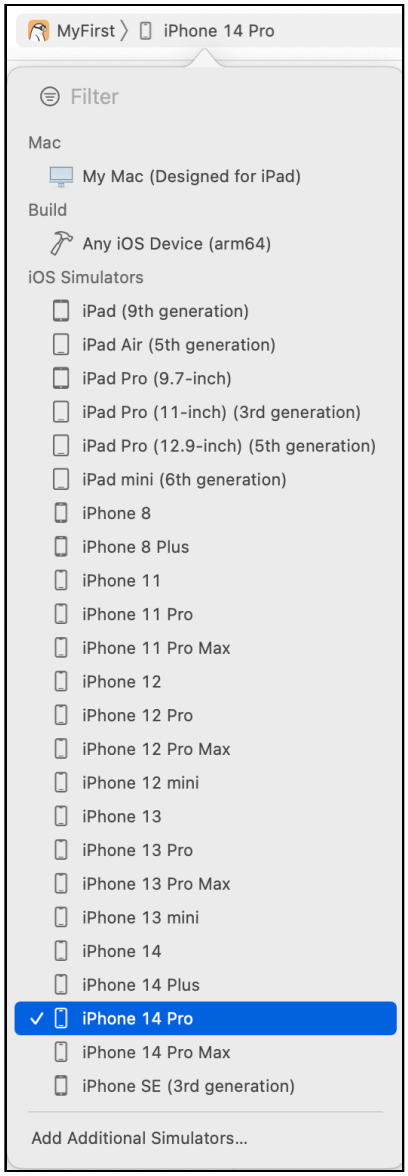
Now that you know where the controls are, it's time to use some of them.

Choosing a Run Destination or Preview Device

Apple sells iPhones and iPads in different sizes, and some have a notch. How do you know if your app looks good on every screen size?

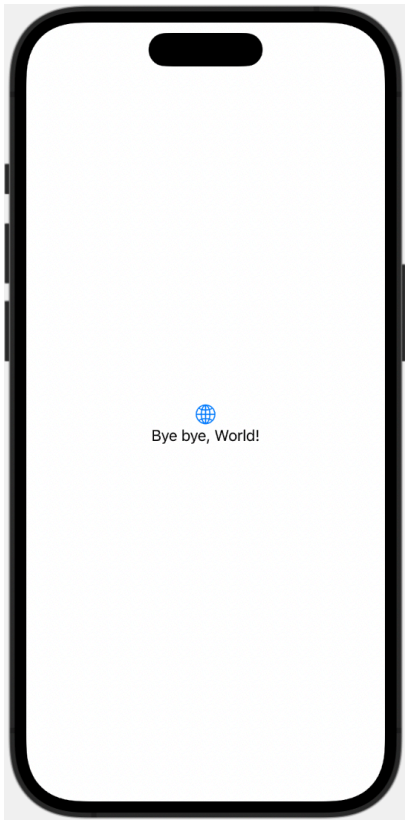
You don't need a complete collection of iOS devices. Xcode has several **Developer Tools**, and one of them is **Simulator**. The run destination menu lets you choose from a list of simulated devices.

➤ If it's not already set, click the run destination button and select **iPhone 14 Pro**.



Run destination menu

➤ In `ContentView`, if the preview is paused, refresh it:

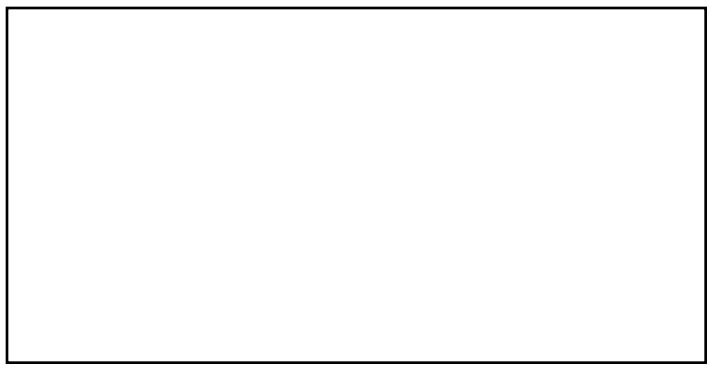


Preview device is run destination.

The preview uses the run destination device by default. You can create more than one preview and set each to a different device with the `previewDevice` modifier.

> **Note**: When you install Xcode 14, there might be only a few iPhone simulators in the destination run menu — mine lists only the iPhone 14 sizes and the iPhone SE 3rd generation. But Apple's support page lists all the iPhone models compatible with iOS 16 (https://apple.co/3l94uYm).

➤ Check your destination run menu: Is **iPhone 8** listed? If not, select **Add Additional Simulators...**, type in **iPhone 8**, then click **Create**.



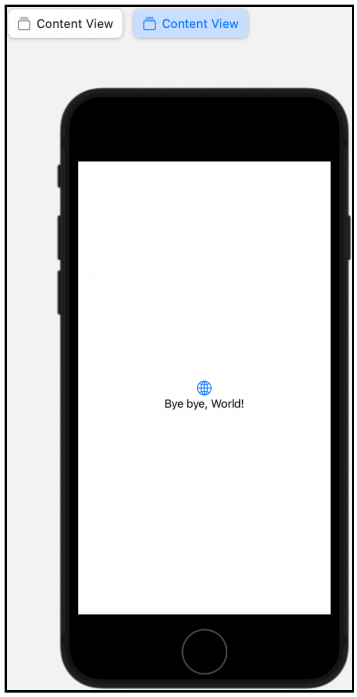
Add additional simulator: iPhone 8

➤ Then replace `ContentView_Previews` with the following:

```
struct ContentView_Previews: PreviewProvider {
  static var previews: some View {
    Group {
      ContentView()
      ContentView()
        .previewDevice(PreviewDevice(rawValue: "iPhone 8"))
    }
  }
}
```

`Group` is another SwiftUI container. It doesn't do any layout. It's just useful when you need to wrap code that's more complicated than a single view.

If you haven't used this simulator device before, Xcode starts it up, then displays it on a second *page*:



Preview of iPhone 8: Home button, no notch!

> **Note**: Enter `xcrun simctl list devicetypes` in a **Terminal** window to see a full list of supported preview device names.

The preview usually looks the same as your app running on a simulated or real device, but not always. If you feel the preview doesn't match what your code is laying out, try running it on a simulator.

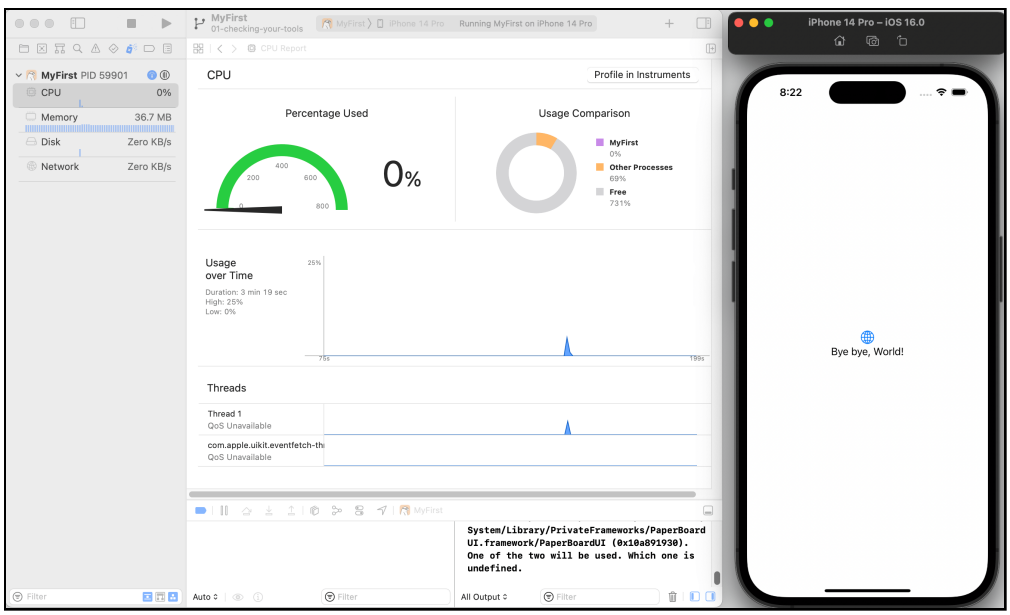
Build & Run

➤ Click the run button or press **Command-R**.

The first time you run a project on a simulated device, it starts from an "off" state, so you'll see a loading indicator. Until you quit the Simulator app, this particular simulated device is now "awake", so you won't get the startup delay even if you run a different project on it.

After the simulated device starts up, the app's launch screen appears. For MyFirst, this is just a blank screen. You'll learn how to set up your own launch screen in Chapter 16, "Adding Assets to Your App".

And now, your app is running!



Debug navigator, debug area and running app

There's not much happening in this app, but the debug toolbar appears below the editor window. For this screenshot, I showed the debug area, selected the **Debug** tab in the Project navigator pane, then selected the **CPU** item.

In the following chapters, you'll create a much more interesting app.

Not Stopping

Here's a trick that will make your Xcode life a little easier.

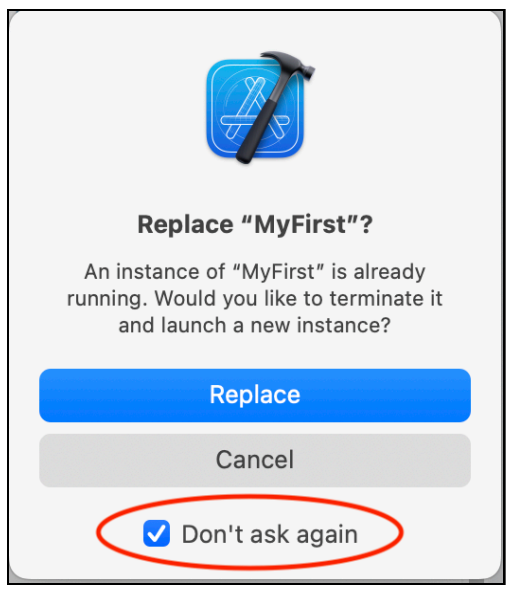
➤ **Don't click** the stop button. Yes, it's enabled. But trust me, you'll like this. :]

➤ In **ByeView.swift**, replace "Bye bye" with "Hello again":

```
Text("Hello again, World!")
```

➤ Click the run button or press **Command-R**.

Up pops this message:



Check Don't ask again.

➤ **Don't click Replace yet**, although that will work: The currently running process will stop, and the new process will run. This dialog will appear every time you forget to stop the app. It's just a moment, but it jars a little. Every time. And it's easy to get rid of.

➤ Check **Don't ask again**, *then* click **Replace**.

The app loads with your new change. But that's not what I want to show you.

➤ Once more: Click the run button or press **Command-R**.

No annoying message, no "doh!" moment, ever again! You're welcome. ;]

Key Points

- The Xcode window has Navigator, Editor and Inspectors panes, a Toolbar and a Debug Area, plus a huge number of Settings.
- You can specify navigation keyboard shortcuts in Settings, to match the instructions in this book.
- The template project defines an `App` that launches with `ContentView`, displaying a globe symbol and "Hello, world!".
- You can view Quick Help documentation in an inspector or with a keyboard shortcut. Or, you can open the Developer Documentation window.
- When you create a new SwiftUI view file, give it the same name as the `View` you'll create in it.
- Xcode's auto-completion, delimiter-matching, code-folding and spell-checking help you avoid errors.
- You can choose one of Xcode's font and color themes, modify one or create your own.
- You can run your app on a simulated device or create previews of specific devices.

Chapter 2: Planning a Paged App

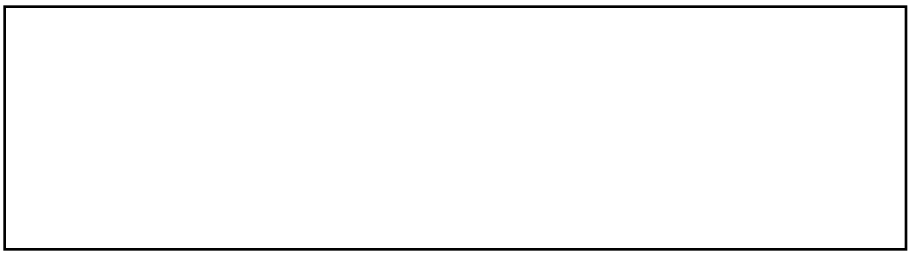
By Audrey Tam

In Section 1 of this book, you'll build an app to help you do high-intensity interval training. Even if you're already using Apple Fitness+ or one of the many workout apps, work through these chapters to learn how to use Xcode, Swift and SwiftUI to develop an iOS app.

In this chapter, you'll plan your app, then set up the paging interface. You'll start using the SwiftUI Attributes inspector to add modifiers. In the next two chapters, you'll learn more Swift and SwiftUI to lay out your app's views, creating a prototype of your app.

Making Lists: Views & Actions

The finished app will have several screens. Here's a sample to show you what it will look like:



HIITFit screens

There's a lot going on in these screens, especially the one with the exercise video. You might feel overwhelmed, wondering where to start. Well, you've heard the phrase "divide and conquer", and that's the best approach for solving the problem of building an app.

First, you need an inventory of what you're going to divide. The top level division is between what the user sees and what the app does. Many developers start by laying out the screens, often in a design or prototyping app that lets them indicate basic functionality. For example, when the user taps *this* button, the app shows *this* screen. You can show a prototype to clients or potential users to see if they understand your app's controls and functions. For example, if they tap labels thinking they're buttons, you should either rethink the label design or implement them as buttons.

Listing What Your User Sees

To start, list the screens you need to create and describe their contents:

- A **Welcome** screen with text, images and a button.
- A title and page numbers are at the top of the **Welcome** screen and a **History** button is at the bottom. These are also on the screen with the exercise video. The page numbers indicate there are four numbered pages after this page. The waving hand symbol is highlighted.
- The screen with the exercise video also has a timer, a **Start/Done** button and rating symbols. One of the page numbers is highlighted.

- The **History** screen shows the user's exercise history as a list and as a bar chart. It has a title but no page numbers and no **History** button.
- The **High Five!** screen has an image, some large text and some small gray text. Like the **History** screen, it has no page numbers and no **History** button.

In this chapter and the next, you'll lay out the basic elements of these screens. In Chapter 9, "Refining Your App", you'll fine-tune the appearance to look like the screenshots above.

Listing What Your App Does

Next, list the functionality of each screen, starting with the last two.

- The **History** and **High Five!** screens are *modal sheets* that slide up over the **Welcome** or **Exercise** screen. Each has a button the user taps to dismiss it, either a circled "X" or a **Continue** button.
- On the **Welcome** and **Exercise** screens, the matching page number is white text or outline on a black background. Tapping the **History** button displays the **History** screen.
- The **Welcome** page **Get Started** button displays the next page.
- On an **Exercise** page, the user can tap the play button to play the video of the exercise.
- On an **Exercise** page, tapping the **Start Exercise** button starts a countdown timer, and the button label changes to **Done**. Ideally, the **Done** button is disabled until the timer reaches 0. Tapping **Done** adds this exercise to the user's history for the current day.
- On an **Exercise** page, tapping one of the five rating symbols changes the color of that symbol and all those preceding it.
- Tapping **Done** on the last exercise shows the **High Five!** screen.
- Nice to have: Tapping a page number goes to that page. Tapping **Done** on an **Exercise** page goes to the next **Exercise** page. Dismissing the **High Five!** screen returns to the **Welcome** page.

You'll implement all of these in the next three chapters.

There's also the overarching page-based structure of HIITFit. This is quite easy to implement in SwiftUI, so you'll do it first, before you create any screens.

Creating Pages

> **Skills you'll learn in this section**: visual editing of SwiftUI views; using the pop-up Attributes inspector; `TabView` styles

The main purpose of this section is to set up the page-based structure of HIITFit, but you'll also learn a lot about using Xcode, Swift and SwiftUI. The short list of **Skills** at the start of each section helps you keep track of what's where.

➤ Open the starter project for this chapter. Use the Xcode menu **Source Control ▸ New Git Repositories…** to add a repository.

Canvas & Editor Always in Sync

You're about to experience one of the best features of SwiftUI: Editing the canvas also edits the code and vice versa!

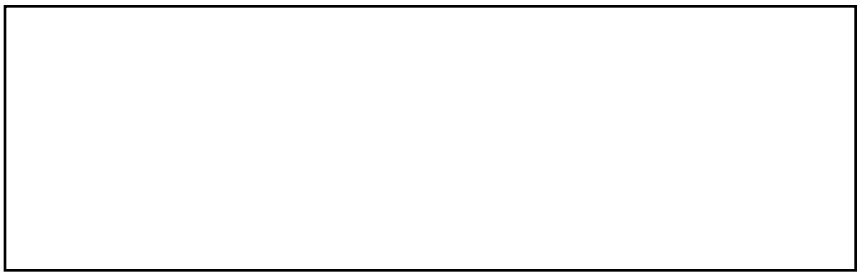
And here's your first SwiftUI vocabulary term: Everything you can see on the device screen is a *view*, with larger views containing subviews.

Your next SwiftUI term is *modifier*: SwiftUI has an enormous number of methods you can use to modify the appearance or behavior of a view.

➤ First, in **ContentView.swift**, delete `.padding()` from the `body` closure: It's a modifier that adds space around the `Text` view, and you don't need it for now.

Editing the View in Canvas Selectable Mode

➤ **In the canvas**, refresh the preview if necessary, click the **Selectable** button, then **double-click** the `Text` view: This also selects "Hello, world!" in the code:



Selection in canvas also selects text in code editor.

> **Note**: A single click selects the `Text` **view**; double-click selects the **content** of the view.

➤ Now type **Welcome**: The text changes in both the canvas and the code.

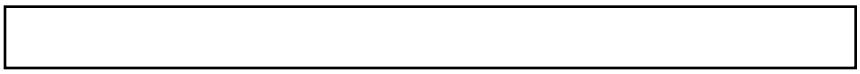


Editing the view in the canvas also changes the code.

> **Note**: Don't press **return** after typing **Welcome**. If necessary, refresh the preview.

A `Text` view simply displays a string of characters. It's useful for listing the views you plan to create, as a kind of outline. You'll use multiple `Text` views now, to see how to implement paging behavior.

➤ Still in the canvas, click anywhere outside the `Text` view to deselect "Welcome", then **single-click** to select the whole `Text` **view**. Press **Command-D**:



Command-D duplicates a view.

As you probably expected, you've *duplicated* the `Text` view in the canvas. But look at the code:

```
VStack {
  Text("Welcome")
  Text("Welcome")
}
```

Your two `Text` views are now embedded in a `VStack`! When you have more than one view, you must specify how to arrange them on the canvas. Xcode knows this, so it provided the default arrangement, which displays the two `Text` views in a *vertical stack*.

➤ Change "V" to "H" to see the two views displayed in a *horizontal stack*:



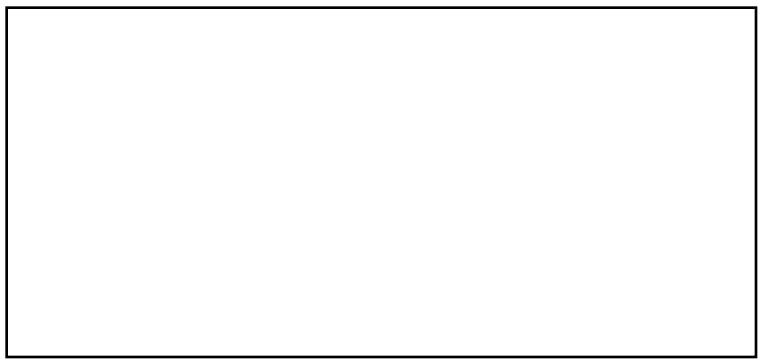
HStack stacks its views horizontally.

➤ Type **Command-Z** to undo this change. SwiftUI's defaults tend to match up well with what most people want to do.

As you can see, you can edit the view either in the code editor or in the canvas in **Selectable** mode.

Editing the View in Code Editor

➤ Type **Command-Z** again to get back to just one `Text("Welcome")` view and no `VStack`. In the code editor, click anywhere on the `Text("Welcome")` line — you don't need to select the line of code — then press **Command-D**:



Two ungrouped views display in two preview pages.

You don't get two lines of text in the preview. Instead, you get two *pages* of identical previews.

➤ In the code editor, select both `Text("Welcome")` lines, then press **{** and type `VStack` in front of the `{`:

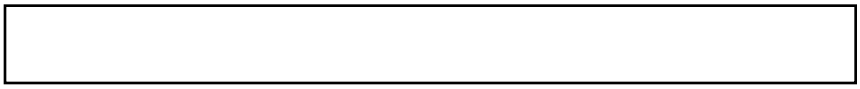


Embed two views in a VStack.

And now you're back to one preview page with two lines of text.

> **Xcode Tip**: *Delimiter auto-completion* works for curly and square braces, parentheses and quotation marks: Select the text you want to enclose, then type the start delimiter character.

➤ Still in the **code editor**, change the second `Welcome` to **`Exercise 1`**. Then duplicate `Text("Exercise 1")` and change the third string to **`Exercise 2`**.



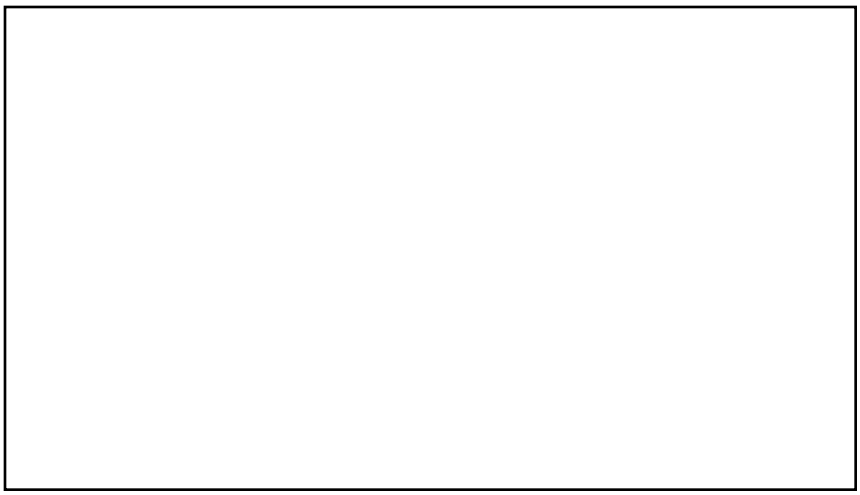
Three Text views in a VStack

You now have three distinct views to use in a `TabView`.

Using TabView

Here's how easy it is to create a `TabView`:

➤ **In the code editor**, change `VStack` to **TabView**:



A TabView has a tab bar.

Where did your Exercises go!? Well, they're now the second and third tabs of a tab view, and there's a tab bar at the bottom of the screen. It's blank, because you haven't labeled the tabs yet.

Labeling Tabs

Here's how you label the tabs. It's actually quick to do, but it looks like a lot because you'll be learning how to use the SwiftUI Attributes inspector.

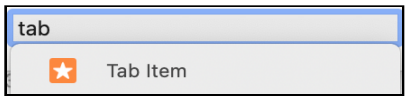
➤ Still in the code editor, **Control-Option-click** the `Text("Welcome")` view to pop up its *Attributes inspector*:



Control-Option-click Text to show its Attributes inspector.

> **Xcode Tip**: The show-inspectors button (upper right toolbar) opens the right-hand panel. The *Attributes inspector* is the right-most tab in this panel. If you're working on a small screen and just want to edit one attribute, **Control-Option-click** a view in the code editor to use the pop-up inspector. It uses less space.

➤ Click in the **Add Modifier** field, then type **tab** and select **Tab Item** from the menu:



Select the Tab Item modifier.

A new `tabItem` modifier appears in the code editor, with a placeholder for the *Item Label*:

```
Text("Welcome")
  .tabItem { Item Label }
```

And a blue **Label** appears in the tab bar:

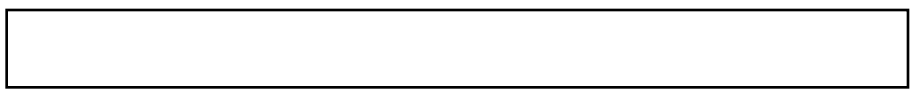


A tab item with placeholder label

➤ Select the *Item Label* placeholder and type **Text("Welcome")**:

```
.tabItem { Text("Welcome") }
```

And there it is in the tab bar:

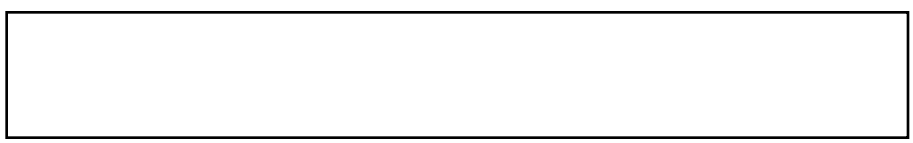


Result of replacing the placeholder tab item label

➤ Replace the entire `TabView` with the following to add the labels for the other tabs:

```
TabView {
  Text("Welcome")
    .tabItem { Text("Welcome") }
  Text("Exercise 1")
    .tabItem { Text("Exercise 1") }
  Text("Exercise 2")
    .tabItem { Text("Exercise 2") }
}
```

Now you can see the three tab labels:

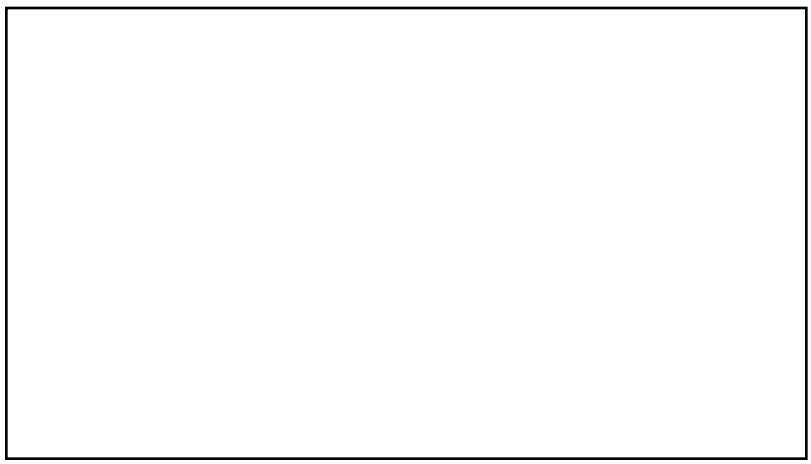


Three tab items with labels

Interacting With Live Preview

The Selectable mode of the preview canvas lets you edit views in the canvas but, at this point, you probably want to see it in action. It's time to switch back to **Live Preview**.

➤ Click the **Live Preview** button, then tap an **Exercise tab label** to switch to that tab:



In Live Preview, tab buttons work.

➤ This is the way tab views normally operate. To make the tabs behave like **pages**, add this modifier to the `TabView`:

```
  .tabViewStyle(PageTabViewStyle())
```

And now your tab labels are gone!

The page style uses small index dots, but they're white on white, so you can't see them.

➤ To make them show up, add this modifier below `tabViewStyle`:

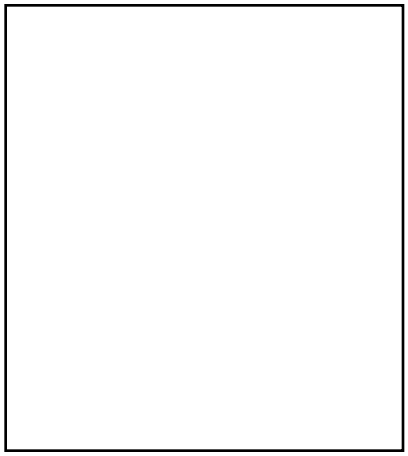
```
  .indexViewStyle(
    PageIndexViewStyle(backgroundDisplayMode: .always))
```

Now you can see the index dots:



TabView page style index dots

➤ In **Live Preview**, just swipe left or right and each page snaps into place.



Live Preview: TabView page style in mid-swipe

➤ You won't be using `tabItem` labels for this app, so delete them. This is now all the code inside the `TabView` closure:

```
TabView {
  Text("Welcome")
  Text("Exercise 1")
  Text("Exercise 2")
}
```

OK, you've set up the paging behavior, but you want the pages to be actual **Welcome** and **Exercise** views, not just text. To keep your code organized and easy to read, you'll create each view in its own file and group all the view files in a folder.

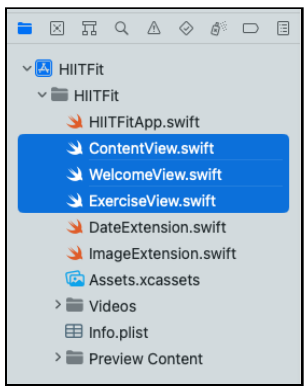
Grouping Files

Skills you'll learn in this section: creating and grouping project files

You're about to create **Welcome** and **Exercise** subviews by combining smaller subviews. SwiftUI encourages you to create reusable subviews for the same reason you create functions: *Don't Repeat Yourself*. Even if you don't reuse a subview, it makes your code much easier to read. And SwiftUI compiles subviews into efficient machine code, so you can create all the subviews you need and not worry about performance.

➤ Select **ContentView.swift** in the Project navigator. Create a **new SwiftUI View file** named **WelcomeView.swift**. Then, create another **new SwiftUI View file** named **ExerciseView.swift**.

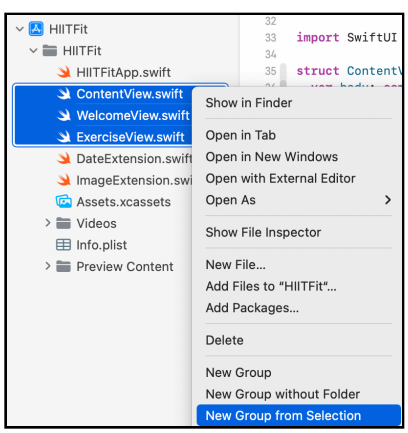
Your Project navigator now contains three view files:



Project navigator after you add two SwiftUI view files

You'll create several more view files, so now you'll create a *group folder* with these three and name it **Views**.

➤ Select the three view files, then **right-click** and select **New Group from Selection**:



Create a group folder containing the three view files.

➤ Name the group **Views**.

Group folders just help you organize all the files in your project. In Chapter 4, "Prototyping Supplementary Views", you'll create a folder for your app's data models.

Passing Parameters

Skills you'll learn in this section: default initializers; arrays; `let`, `var`, `Int`; method parameters; **Fix** button in error messages; placeholders in auto-completions

➤ Now, in `ContentView`, replace the first two `Text` placeholders with your new views:

```
TabView {
  WelcomeView()   // was Text("Welcome")
  ExerciseView()  // was Text("Exercise 1")
  Text("Exercise 2")
}
```

Swift Tip: A `View` is a *structure*, shortened to `struct` in Swift code. Like a class, it's a complex data type that encapsulates properties and methods. If a `View` has no uninitialized properties, you can create an instance of it with its default initializer. For example, `WelcomeView()` creates an instance of `WelcomeView`.

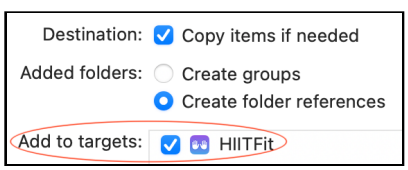
Now what? Your app will use `ExerciseView` to display the name and video for several different exercises, so you need a way to index this data and pass each index to `ExerciseView`.

Actually, first you need some sample exercise data. In the **Videos** folder, you'll find four videos:



One of the exercise videos

> **Note**: If you prefer to use your own videos, drag them from **Finder** into the Project navigator. Be sure to check the **Add to targets** check box.



If you add your own videos, check Add to targets.

➤ In Chapter 3, "Prototyping the Main View", you'll create an `Exercise` data type but, for this prototype, in **ExerciseView.swift**, simply create two arrays at the top of `ExerciseView`, just above `var body`:

```
let videoNames = ["squat", "step-up", "burpee", "sun-salute"]
let exerciseNames = ["Squat", "Step Up", "Burpee", "Sun Salute"]
```

> **Swift Tip**: An array is an ordered collection of primitive types, structure instances or class objects. All items in an array are the same type.

The video names match the names of the video files. The exercise names are visible to your users, so you use title capitalization and spaces.

➤ Still inside `ExerciseView`, above `var body`, add this property:

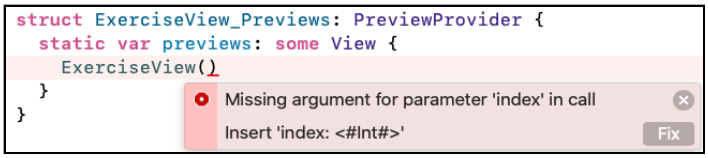
```
let index: Int
```

You declare a constant integer value named `index`.

> **Swift Tip**: Swift distinguishes between creating *constants* with `let` and creating *variables* with `var`.

Xcode now complains about `ExerciseView()` in `previews`, because it's missing the `index` parameter.

➤ Click the red error icon to display more information:



Open the error to show the Fix button.

Xcode often suggests one or more ways to fix an error. Many times, its suggestion is correct, and this is one of those times.

➤ Click **Fix** to let Xcode fill in the `index` parameter.

➤ Now there's a placeholder for the `index` value — a grayed-out `Int`. Click it to turn it blue, then type **0**. So now you have this line of code:

```
ExerciseView(index: 0)
```

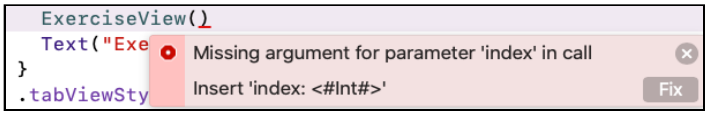
> **Swift Tip**: Like other languages descended from the C programming language, Swift arrays start counting from 0, not 1.

Now use your `index` property to display the correct name for each exercise.

➤ Change the `"Hello, World!"` placeholder to the exercise name for this `index` value.

```
Text(exerciseNames[index])
```

The canvas has been complaining it **Failed to build the scheme "HIITFit"** and, back in **ContentView.swift**, Xcode is also complaining about the missing argument for parameter `index` in the call to `ExerciseView()`.



Another error to fix

➤ Fix this error the same way you did in **ExerciseView.swift**.

Now there's a placeholder for the `index` value: What should you type there?

Looping

Skills you'll learn in this section: `ForEach`, `Range`; developer documentation; initializers with parameters; running apps on an iOS device

Well, you *could* pass the first array index:

```
ExerciseView(index: 0)
```

Then copy-paste and edit to specify the other three exercises, but there's a better way. You're probably itching to use a *loop*. Here's how you scratch that itch. ;]

➤ Replace the second and third lines in the `TabView` closure with this code:

```
ForEach(0 ..< 4) { index in
  ExerciseView(index: index)
}
```

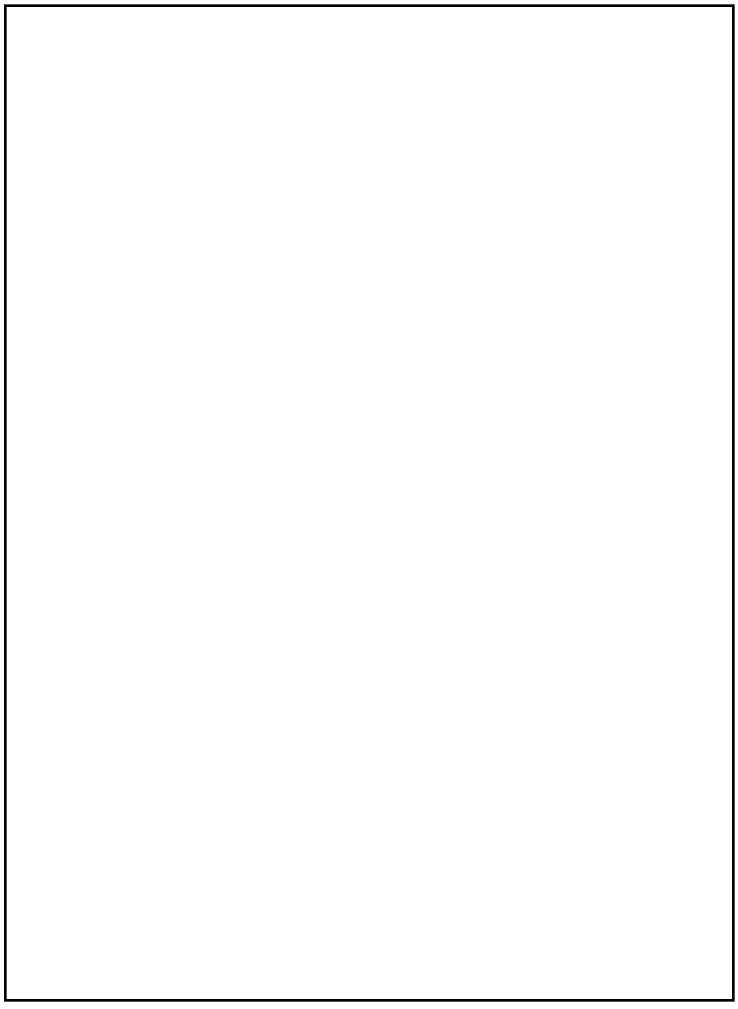
`ForEach` loops over the *range* 0 to 4 but, because of that < symbol, **not including 4**. Each integer value 0, 1, 2 and 3 creates an `ExerciseView` with that `index` value.

The local variable name `index` is up to you. You could write this code instead:

```
ForEach(0 ..< 4) { number in
  ExerciseView(index: number)
}
```

Developer Documentation

➤ This is a good opportunity to check out Xcode's built in documentation. Hold down the **Option** key, then click `ForEach`:



Option-click the ForEach keyword.

You're viewing Xcode's pop-up **Quick Help** for the `ForEach` keyword. You can also view this information in the **Quick Help inspector**.

➤ To see more detailed information, scroll down to the bottom of the Quick Help text, click **Open in Developer Documentation**, then scroll down to the **Topics** section:

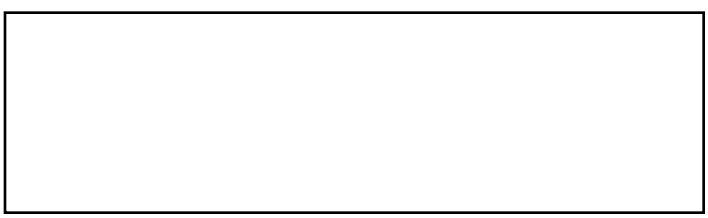


Developer Documentation for ForEach: the first initializer

The topic **Creating a collection from a range** contains the initializer you're using to loop over the array indices.

```
init(Range<Int>, content: (Int) -> Content)
```

➤ This `ForEach` initializer requires `Range<Int>`. Click this line to open the `init(_:content:)` page, then click `Range` in its **Declaration** to open the `Range` page:



Developer Documentation for Range

`Range` is "A half-open interval from a lower bound up to, but not including, an upper bound", and you "create a Range instance by using the half-open range operator (..<)", which is what you did in the `ForEach` argument.

➤ Close the documentation window.

➤ You won't need the `TabView` index dots. Open **ContentView.swift** and change:

```
.tabViewStyle(PageTabViewStyle())
.indexViewStyle(
  PageIndexViewStyle(backgroundDisplayMode: .always))
```

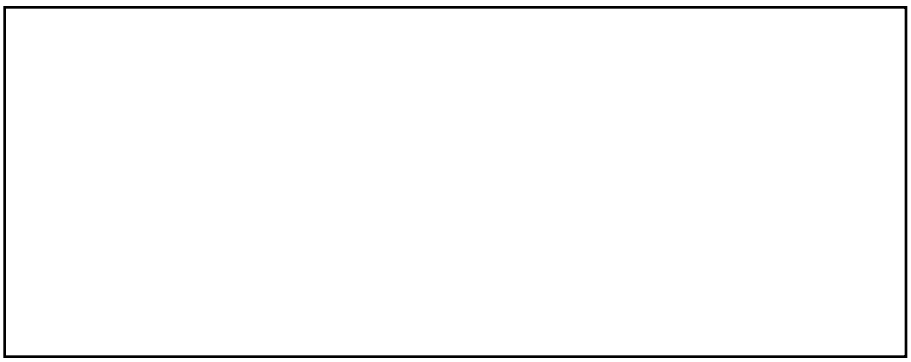
to:

```
.tabViewStyle(PageTabViewStyle(indexDisplayMode: .never))
```

Now, you'll never show the index dots.

> **Xcode Tip**: This is a good place to commit the changes you've made to your project into your local Git repository. Select **Source Control ▸ Commit…** or press **Option-Command-C**. If asked, check all the changed files. Enter a commit message like "Set up paging tab view", then click **Commit**.

➤ You're still in `ContentView`, so **Live Preview** your app. Swipe from one page to the next to see the different exercise names.



HIITFit pages

Running Your Apps on an iOS Device

> **Note**: This book's apps expect your iOS device is running **iOS 16**.

Live Preview is a convenient way to see what your app looks like and give you some idea how it behaves. But some features don't work in Live Preview, so then you need to build and run your app on a simulator.

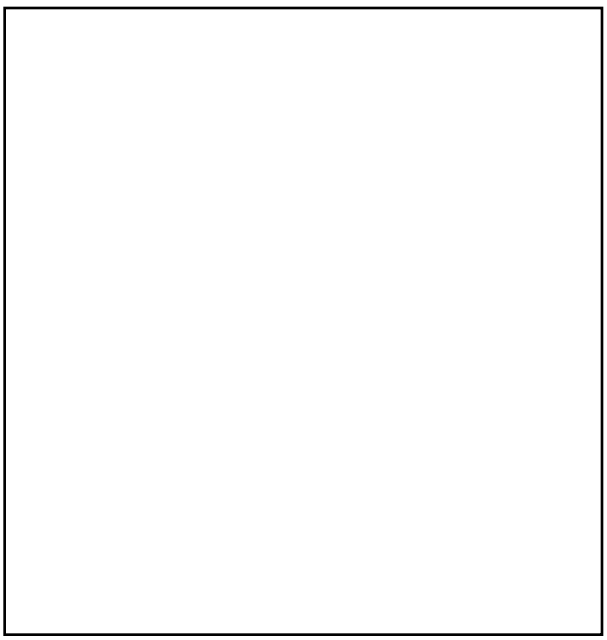
If your app doesn't look or behave quite right on the simulated device, running it on a real device is the final word. It might look just as you expect, or it might agree with the preview and simulator that you've got more work to do.

Also, there are features like motion and camera that you can't test in a simulator. For these, you must install your app on a real device. Plus, it's fun to have something on your iPhone that you built yourself!

Enabling Developer Mode on Your iOS Device

First, you must enable **Developer Mode** on your iOS device. Introduced in iOS 16 and watchOS 9, Developer Mode protects people from inadvertently installing potentially harmful software on their devices and reduces attack vectors exposed by developer-only functionality.

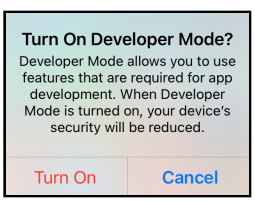
➤ Open the **Settings** app and tap **Privacy & Security**. Scroll down and tap **Developer Mode**, then tap the switch to turn it on.



Enable Developer Mode in Privacy & Security setting.

You'll see an alert warning you that Developer Mode reduces the security of your device.

➤ Tap the alert's **Restart** button. After your device restarts and you unlock it, you'll see an alert asking you to confirm that you want to turn on Developer Mode:

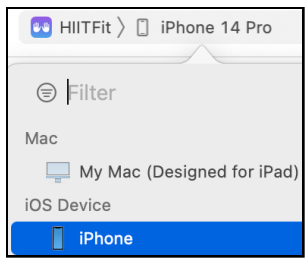


Turn on Developer Mode after restarting device.

➤ Tap **Turn On** to acknowledge the reduction in security protection in exchange for allowing Xcode and other tools to execute code, then enter your device passcode when prompted.

Your device is now ready to install and run apps from Xcode. After enabling Developer Mode, Xcode doesn't ask again unless you disable Developer Mode by turning off the switch in **Privacy & Security** and restarting your device.

➤ Connect your device to your Mac with a cable. Use an Apple cable, as other-brand cables might not work for this purpose. Select your device from the run destination menu: It appears near the top, above the simulators:



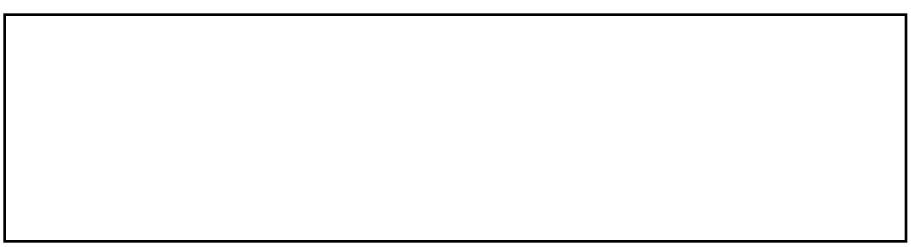
Select your device as the run destination.

Xcode will start preparing your device for development. This can take a while, so continue to the next step while it's busy.

Getting a Signing Certificate

Apple does its best to protect its users from malicious apps. Part of this protection is ensuring Apple knows who is responsible for every app on your device. Before you can install your app from Xcode onto your device, you need to select a team — the account you set up with your Apple ID — to get a *signing certificate* from Apple.

➤ In the project page, select the **target**. In the **Signing & Capabilities** tab, change the organization name in the **Bundle Identifier** to something that's uniquely yours, like org.audrey for me:



Personalize the Bundle Identifier.

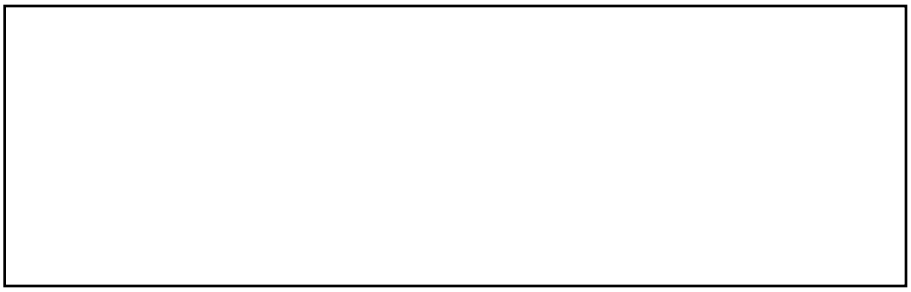
> **Note**: The apps in this book have starter projects with **com.kodeco** as the organization. If you want to run these apps on an iOS device, you need to personalize the bundle identifier. This is because one of the authors has already signed the app with the original bundle identifier, and you're not a member of our teams.

➤ Next, check **Automatically manage signing**, tap **Enable Automatic** in the confirmation dialog, then select your account from the **Team** menu:



Enable Automatic and select Team.

After some activity spinning, you'll see a **Provisioning Profile** and a **Signing Certificate**. Xcode has created these and stored the certificate in your Mac's keychain.



Provisioning Profile and a Signing Certificate

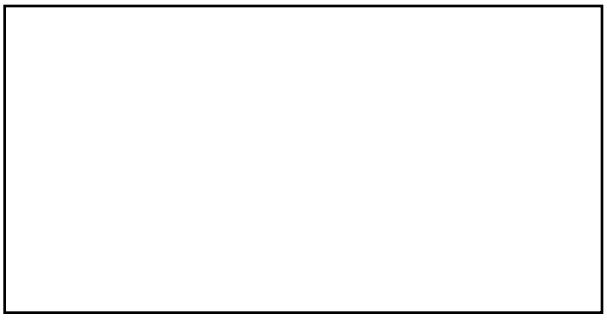
➤ Unlock your device, then build and run your project. Keep your device screen active until the app launches on your device.

Trusting Yourself

> **Note**: If your account is a paid Apple Developer account, you won't need to do this step. Running your app on your device will just work. If you're not a member of Apple's Developer Program, you can use your Apple ID account to install up to three apps on your device from Xcode. The app works for seven days after you install it. Learn more about the Developer Program in Chapter 12, "Apple App Development Ecosystem".

If this is the first time you're running an app on this device, Apple makes you perform one more step to make sure nothing nasty installs itself on your device.

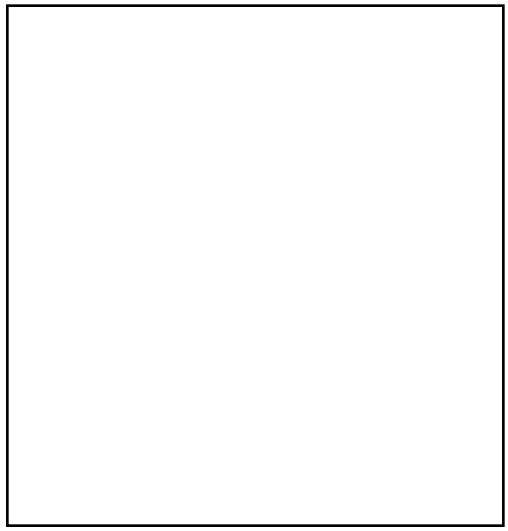
The app icon appears on the home screen of your device, but error messages appear in Xcode and on your device:



Could not launch / Untrusted Developer

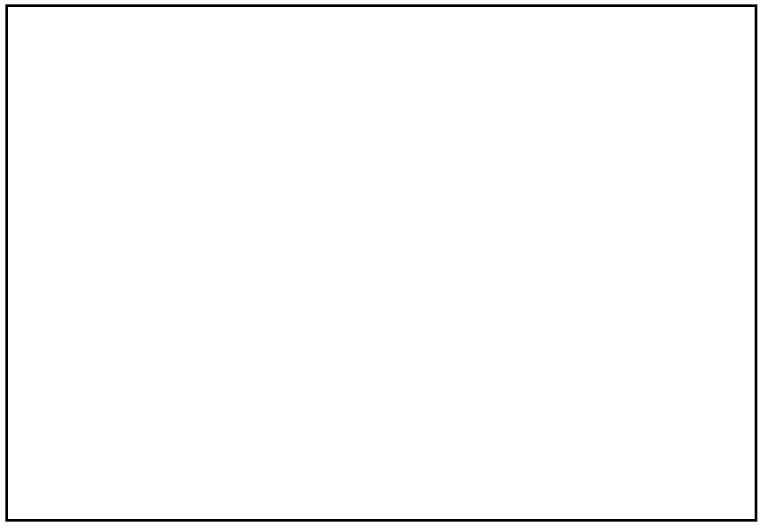
Of the three possible reasons in the Xcode message, it's the last one that's holding things up: *its profile has not been explicitly trusted by the user*. Apple *really* doesn't want just anyone installing potentially malicious apps on your device. *You* have to say it's OK. The problem is, *You can allow using these apps in Settings* doesn't really tell you what to do.

➤ Open **Settings** to see what's there. You'll never guess where to look, so here's what you do: Tap **General**, then scroll down and tap **VPN & Device Management**. On that page, tap **Apple Development…**:



Settings ▸ General ▸ VPN & Device Management

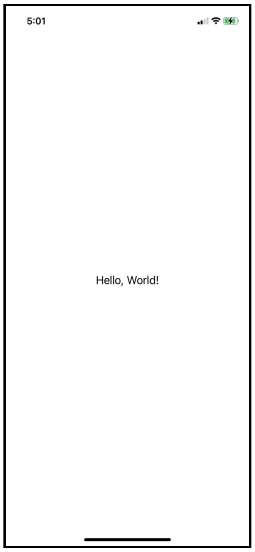
➤ Next, tap **Trust "Apple Development…** and finally, tap **Trust** in the alert:



Trust developer.

You won't need to do this again unless you delete all your apps from this device.

➤ Now close **Settings** and tap the **HIITFit** icon:



HIITFit running on an iPhone

> **Note**: If your device uses dark mode, the background and text will have a different color. By default, SwiftUI respects the device configuration and uses colors accordingly.

The app doesn't actually look or behave any different to Live Preview, but you're now all set up to run your own projects on this device. When you really want to get something running *right away*, you won't have to stop and deal with any of this **Trust** business.

➤ After you disconnect your device from your Mac, select a simulator device in the run destination menu. Otherwise, if it's stuck on *Any iOS Device*, you won't see anything in the preview canvas.

Key Points

- Plan your app by listing what the user will see and what the app will do.
- Build your app with views and subviews, customized with modifiers.
- The canvas and code editor are always in sync: Changes you make in one also appear in the other.
- Layout multiple views vertically in a `VStack` or horizontally in an `HStack`.
- The Attributes inspector helps you to modify a view or a preview.
- `ForEach` lets you loop over a half-open range of numbers.
- `TabView` can behave like a tab view or like a page controller.
- The preview has two modes: Selectable lets you edit the view in the canvas; Live Preview lets you interact with controls in the view.
- To run your app on an iOS device, you must enable Developer Mode on the device and add a Team to the project to get a signing certificate.
- The first time you run your project on an iOS device, Apple requires you to complete a "Trust this developer" step.

Where to Go From Here?

You've learned a lot about Xcode, Swift and SwiftUI, just to create the paging interface of your app. Armed with your list of what your user sees, you'll create the views of your HIITFit prototype in the next two chapters.

Chapter 3: Prototyping the Main View

By Audrey Tam

Now for the fun part! In this chapter, you'll start creating a prototype of your app, which has four full-screen views:

- Welcome
- Exercise
- History
- Success

Creating the Exercise View

You'll start by laying out the **Exercise** view, because it contains the most subviews. Here's the list of what your user sees in this view:

- A title and page numbers are at the top of the view and a **History** button is at the bottom.
- The page numbers indicate there are four numbered pages.
- The exercise view contains a video player, a timer, a **Start/Done** button and rating symbols.

And here's the list rewritten as a list of subviews:

- Header with page numbers
- Video player
- Timer
- Start/Done button
- Rating
- History button

You *could* sketch your screens in an app like Sketch or Figma before translating the designs into SwiftUI views. But SwiftUI makes it easy to lay out views directly in your project, so that's what you'll do.

The beauty of SwiftUI is it's declarative: You simply *declare* the views you want to display, in the order you want them to appear. If you've created web pages, it's a similar experience.

Outlining the Exercise View

➤ Continue with your project from the previous chapter or open the project in this chapter's **starter** folder.

Start by creating an outline with placeholder `Text` views.

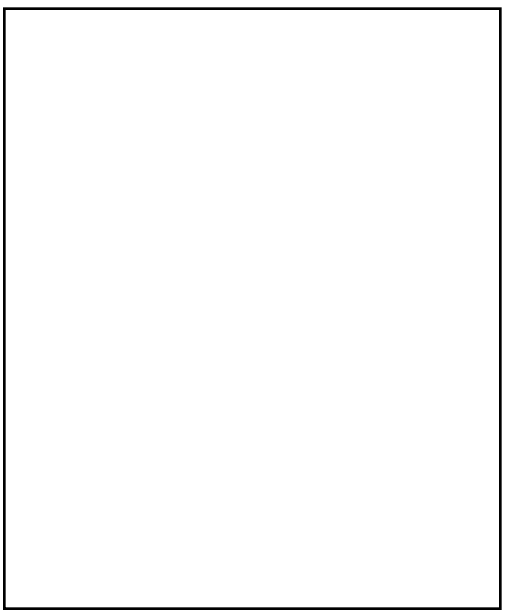
➤ Open **ExerciseView.swift**.

➤ You'll start by laying out the iPad version of HIITFit, so select an **iPad simulator**:



Select an iPad simulator.

➤ Zoom to fit the iPad in the canvas:

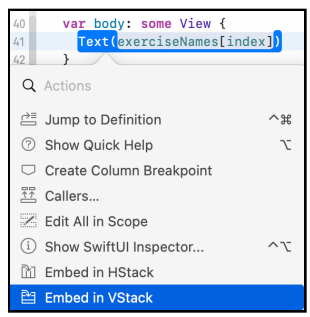


Zoom to fit the iPad in the canvas.

➤ `ExerciseView` has six subviews, so duplicate the `Text(exerciseNames[index])` view, then edit the arguments — in the code or in the canvas — to create this list:

```
VStack {
  Text(exerciseNames[index])
  Text("Video player")
  Text("Timer")
  Text("Start/Done button")
  Text("Rating")
  Text("History button")
}
```

Xcode Tip: You *could* create the six `Text` views, then do the curly brace auto-completion trick. Another way is to embed the single `Text` view in a `VStack` — **Command-click** `Text` and select **Embed in VStack** from the menu — and then duplicate and edit your views. **Embed in ...** only works on a **single** line of code and the preview canvas must be open.



Embed single view in VStack.

Creating the Header View

Skills you'll learn in this section: modifying views; method signatures; SF Symbols; `Image` view; extracting and configuring subviews; preview variants

The first `Text` view is the starting point for the **Header** view. You'll add code to it, here in `ExerciseView`, then you'll extract this code as a subview and move it to its own file.

➤ To prepare for later extraction, use the **Command-click** menu to embed the first `Text` view in a `VStack`. Now, this `Text` view is in a `VStack`, nested in the top-level `VStack`:

```
VStack {
  VStack {
    Text(exerciseNames[index])
  }
```

The Many Ways to Modify a View

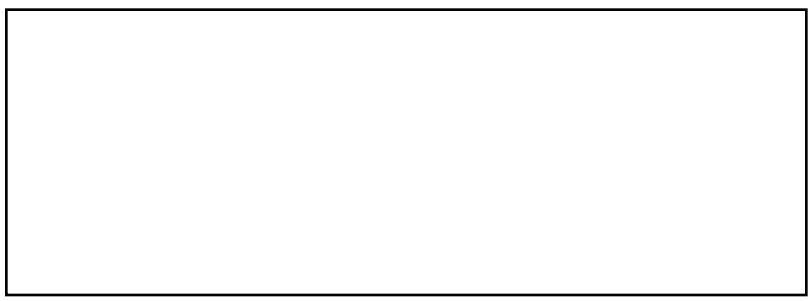
➤ Open the **Attributes inspector**: Press **Option-Command-4** or click the **inspectors** button in the toolbar, then select the **Attributes inspector**. In the canvas **Selectable** mode, select the "Squat" `Text` view:



Open the Attributes inspector, select Squat view in preview.

This inspector has sections for the most commonly-used modifiers: Accessibility, Font, Padding and Frame. You *could* select a font size from the **Font ▸ Font** menu, but you'll use the search field this time. This is a more general approach to adding modifiers.

➤ Click in the **Add Modifier** field, type **font**, then select **Font** from the menu:



Select Font from the Add Modifier menu.

The font size of "Squat" changes in both the canvas and in code:

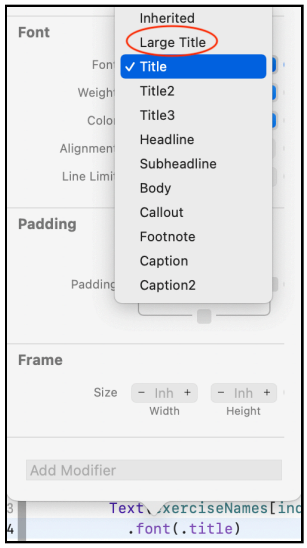
```
Text(exerciseNames[index])
  .font(.title)
```

> **Note**: Putting the modifier on its own line is a SwiftUI convention. A view often has several modifiers, each on its own line. This makes it easy to move a modifier up or down, because sometimes the order makes a difference.

➤ Xcode suggests the font size `title`, but this is only a placeholder. To accept this value, click `.title`, then press **Return**.

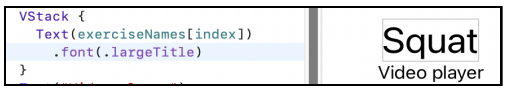
> **Note**: Xcode and SwiftUI auto-suggestions and default options are often what you want.

➤ To see other options, **Control-Option-click** `font` or `title`. This opens the `font` modifier's pop-up Attributes inspector. In the **Font** section, click the selected **Font** option **Title** to see the **Font** menu:



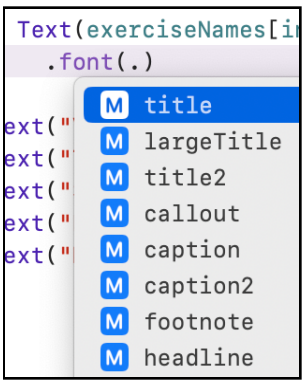
Show the Font menu in the pop-up Attributes inspector.

➤ Select **Large Title** from the menu: "Squat" is even bigger now!



Text with Large Title font

➤ Here's another way to see the `font` menu. Replace `.largeTitle` with `.`:

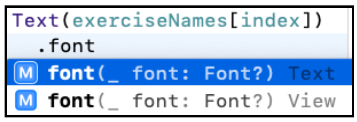


Xcode's auto-suggestions while you type code

Xcode auto-suggests the possible values.

➤ Select **largeTitle** from the menu.

➤ Once you're familiar with SwiftUI modifiers, you might prefer to just type. Delete `.font(.largeTitle)` and type **.font**. Press the right-arrow key to see the second `font(_ font:)` method:



Xcode's auto-suggestions for font methods

➤ Select either method and Xcode auto-completes with a `(font: Font?)` placeholder. Change this to **.largeTitle**.

> **Swift Tip**: The method signature `func font(_ font: Font?) -> Text` indicates this method takes one parameter of type `Font?` and returns a `Text` view. The "_" means there's no *external parameter name* — you call it with `font(.title)`, not with `font(font: .title)`.

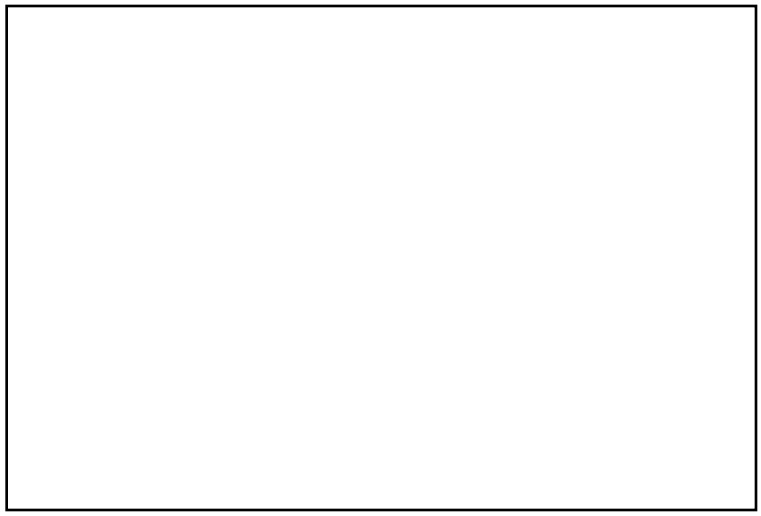
Creating Page Numbers With SF Symbols

In addition to the name of the exercise, the header should display the page numbers with the current page number highlighted.

You *could* just display `Text("1")`, `Text("2")` and so on, but Apple provides a wealth of configurable icons as **SF Symbols**.

➤ The **SF Symbols app** is the best way to browse and explore the collection. Download it from SF Symbols 4 (https://apple.co/3hWxn3G) and install it. Some symbols must be used only for specific Apple products like FaceTime or AirPods. You can check symbols for restrictions at sfsymbols.com.

➤ After installing the **SF Symbols app**, open it, select the **Indices** category, then scroll more than halfway down to the numbers:



SF Symbols app: Indices category (partial)

There are black numbers on a white background or the other way around, in a circle or a square. The fill version could represent the current page, with no-fill numbers for the other pages.

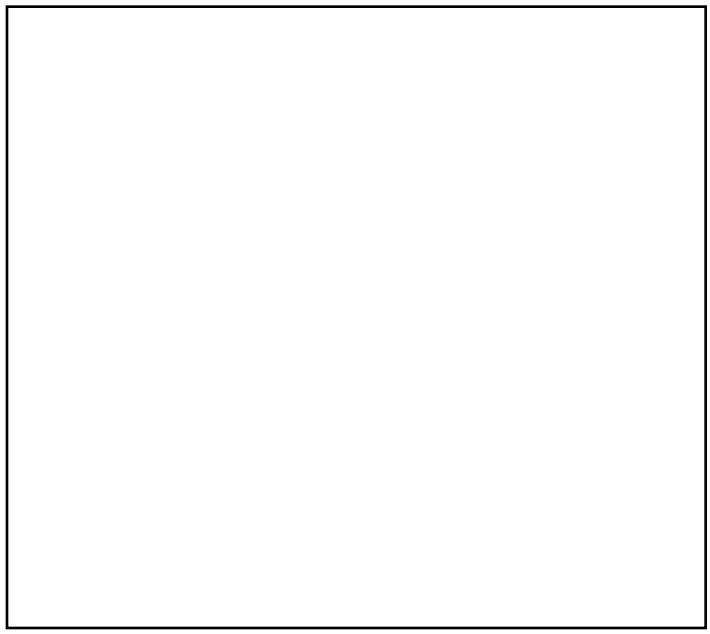
You can copy SF Symbol names from the app with **Shift-Command-C**. However, there's an easier way, if you know part of the symbol name you want.

➤ In the code editor, add this line below the title `Text` view in the nested `VStack`:

```
Image(systemName: "")
```

`Image` is another built-in SwiftUI view, and it has an initializer that takes an SF Symbol name as a `String`.

➤ Position the cursor between the quotation marks, then open the **Library**: Click the + button in the toolbar or press **Shift-Command-L**. Select the **Symbols** tab and search for "1 circle":



Search Library for symbol.

> **Note**: By default, the Library disappears as soon as you click somewhere else. If you want it to stay open, hold down the **Option** key while you open it.

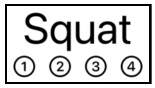
➤ Now, double-click **1.circle**. The symbol name appears at the cursor location:

```
Image(systemName: "1.circle")
```

➤ Before adding more numbers, embed this `Image` in an `HStack`, so the numbers will appear side by side. Then duplicate and edit more `Image` views to create the other three numbers:

```
HStack {
  Image(systemName: "1.circle")
  Image(systemName: "2.circle")
  Image(systemName: "3.circle")
  Image(systemName: "4.circle")
}
```

And here's your header:



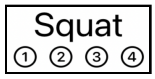
Header with title and page numbers

The page numbers look too small. However, because SF Symbols are integrated into the San Francisco system font — that's the "SF" in SF Symbols — you can treat them like text and use `font` to specify their size.

➤ You *could* add `.font(.title2)` to each `Image`, but it's quicker and neater to add it to the `HStack` container:

```
HStack {
  Image(systemName: "1.circle")
  Image(systemName: "2.circle")
  Image(systemName: "3.circle")
  Image(systemName: "4.circle")
}
.font(.title2)
```

The font size applies to all views in the `HStack`:

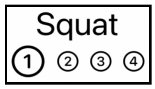


SF Symbols with title2 font size

➤ You can modify an `Image` to override the `HStack` modifier. For example, modify the first number to make it extra large:

```
Image(systemName: "1.circle")
  .font(.largeTitle)
```

Now the first symbol is larger:



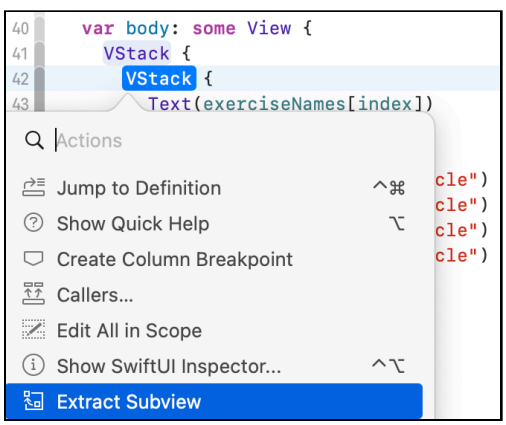
Overriding the stack's font size for the first symbol

➤ Delete the `Image` font modifier, so all the numbers are the same size.

Your `ExerciseView` now has a header, which you'll reuse in `WelcomeView`. So you're about to *extract* the header code to create a `HeaderView`.

Extracting a Subview

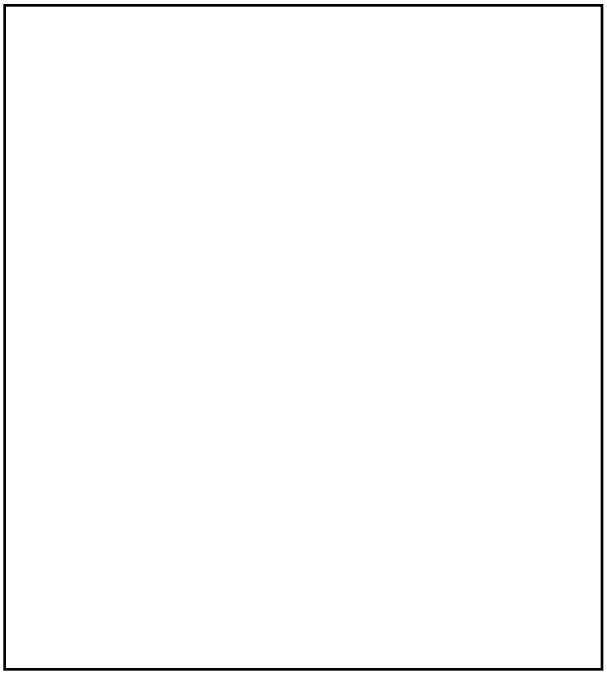
➤ **Command-click** the `VStack` containing the title `Text` and the page numbers `HStack`, then select **Extract Subview** from the menu:



Command-click VStack, select Extract Subview.

Xcode moves the whole `VStack` into the `body` property of a new view with the placeholder name `ExtractedView`.

And `ExtractedView()` is where the `VStack` used to be:



Code extracted to ExtractedView

➤ If the placeholders are highlighted, type **HeaderView** and press **Return**. If not, select `ExtractedView` and right-click. Choose **Refactor ▸ Rename…** from the menu:



Refactor ▸ Rename… ExtractedView.

➤ Type **HeaderView** to replace both placeholders at once, then click **Rename**:



Rename ExtractedView to HeaderView.

Adding a Parameter

The error flag in `HeaderView` shows where you need a parameter. The `index` property is local to `ExerciseView`, so you can't use it in `HeaderView`. You *could* pass `index` to `HeaderView` and ensure it can access the `exerciseNames` array. But it's always better to pass just enough information. This makes it easier to set up the preview for `HeaderView`. Right now, `HeaderView` needs only the exercise name.

➤ Add this property to `HeaderView`, above the body property:

```
let exerciseName: String
```

➤ And replace `exerciseNames[index]` in `Text`:

```
Text(exerciseName)
```

➤ Scroll up to `ExerciseView`, where Xcode is complaining about a missing argument in `HeaderView()`. Click the error icon to click **Fix**, then complete the line to read:

```
HeaderView(exerciseName: exerciseNames[index])
```

Moving a Subview to a New File

➤ Now, press **Command-N** to create a **new SwiftUI View file** and name it **HeaderView.swift**. Because you were in **ExerciseView.swift** when you pressed **Command-N**, the new file appears below it and in the same group folder.

Your new file opens in the editor with two error flags:

1. Invalid redeclaration of 'HeaderView'.
2. Missing argument for parameter 'exerciseName'.

➤ To fix the first, in **ExerciseView.swift**, select all 17 lines of your new `HeaderView` structure and press **Command-X** to *cut* it — *copy* it to the clipboard and *delete* it from **ExerciseView.swift**.

➤ Back in **HeaderView.swift**, replace the boilerplate `HeaderView` with what's in the clipboard.

➤ To fix the second error, in `previews`, let Xcode add the missing parameter, then enter any exercise name for the argument:

```
HeaderView(exerciseName: "Squat")
```

Because you pass only the exercise name to `HeaderView`, the preview doesn't need access to the `exerciseNames` array.

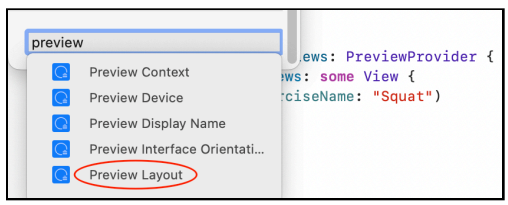
Working With Previews

SwiftUI previews can do a lot more than what you've see so far.

Layout: Size That Fits

The preview still uses the iPad simulator, which takes up a lot of space. You can modify the preview to show only the header.

➤ In `HeaderView_Previews`, **Control-Option-click** `HeaderView(...)` then type **preview** in the **Add Modifier** field:



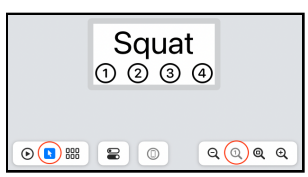
Selecting Preview Layout from the Attributes inspector for Header view

➤ Select **Preview Layout** to add this modifier:

```
.previewLayout(.sizeThatFits)
```

➤ The placeholder value `sizeThatFits` is what you want, but you must accept it: Click **sizeThatFits**, then press **Return**.

➤ Switch to **Selectable** mode and **Zoom to 100%** to see just the header:

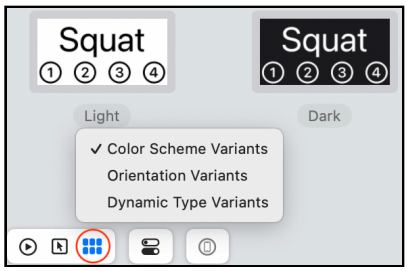


Selectable mode: Preview is just big enough to show the view.

A small view makes it easier to see some of the possibilities of previews.

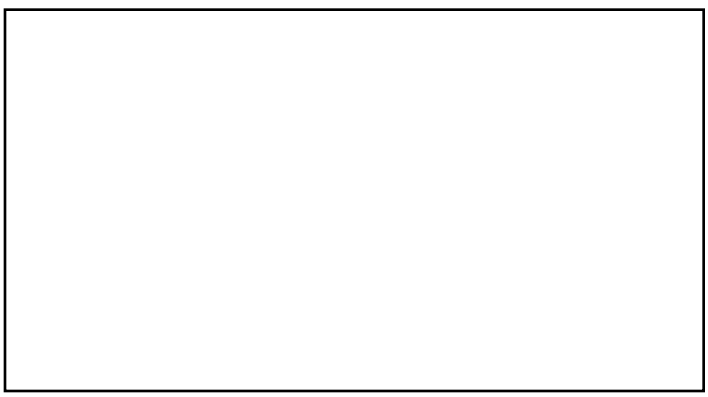
Variants

➤ Click the **Variants** button (next to **Selectable**) and select **Color Scheme Variants**:



Color Scheme Variants

➤ See what you get with **Dynamic Type Variants**:

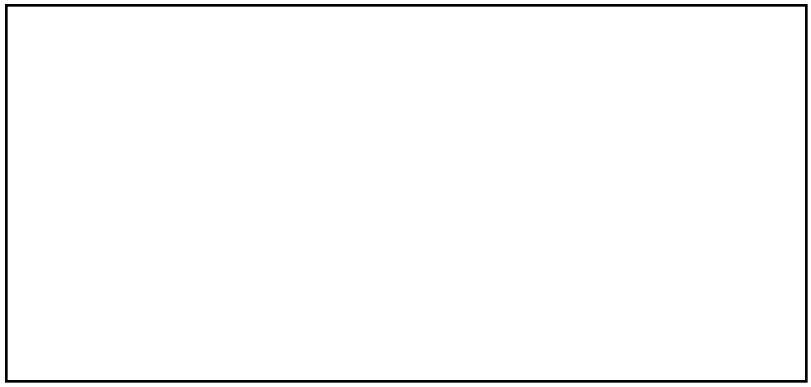


Dynamic Type Variants

iOS device users can set a preferred text size in Settings ➤ Display & Brightness ➤ Text Size — from extra small (to fit more on the screen) up to XXX Large. Accessibility ➤ Display & Text Size ➤ Larger Text goes up to Accessibility size 5. Your app supports these settings by using semantic font sizes like `title2` instead of a fixed value like 36 points.

Dynamic Type Variants show how this view appears for different text size settings, enabling you to adapt your layout so important elements remain readable.

➤ Return to **ExerciseView.swift** to see the **Orientation Variants**:

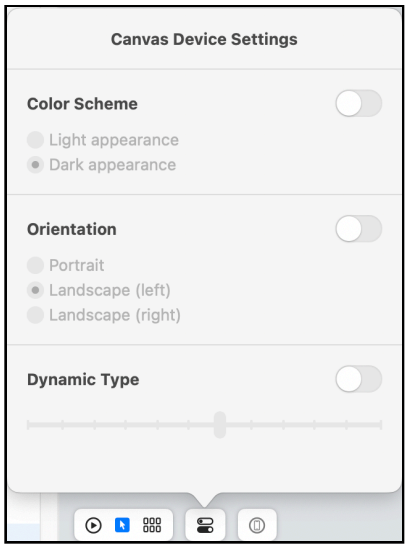


Orientation Variants

That's how easy it is to see how your views appear on a device with these settings.

➤ Click the **Live Preview** or **Selectable** button to stop showing the variants.

It's also easy to select specific variants:



Canvas Device Settings

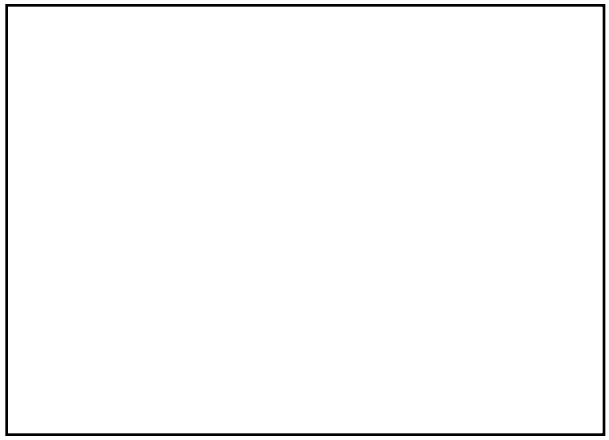
Creating the Exercise Structure

Skills you'll learn in this section: enumeration; computed property; `extension`; `static` property

Currently, your app uses two arrays of strings for the exercise and video file names. This simple approach helped you pass just enough data to the extracted `HeaderView`, keeping its preview manageable. But, if you add more videos, you must manually ensure the strings match up across the two arrays. It's safer to encapsulate them as properties of a named type.

First, you'll create an `Exercise` structure with the properties you need. Then, you'll create an array of `Exercise` instances and loop over this array to create the `ExerciseView` pages of the `TabView`.

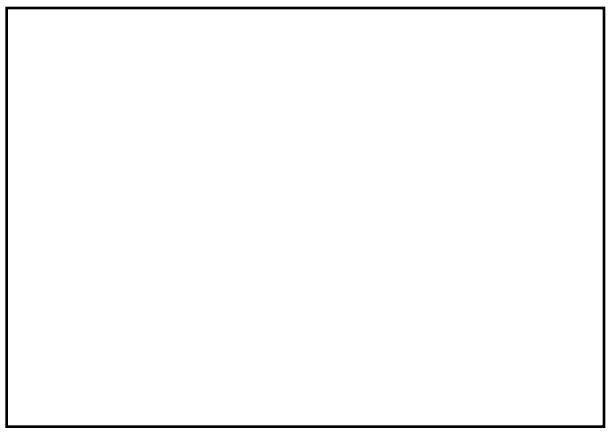
➤ Create a **new Swift file** (**Command-N**):



New Swift File

Note: Up to now, you've created and used new **SwiftUI views**. This `Exercise` structure *models* your app's data, encapsulating the `exerciseName` and `videoName` properties. It isn't a view, so you create it in a new Swift file.

➤ Name the file **Exercise.swift**. Select the **HIITFit** group so it doesn't go into the **Views** group:



Save new Swift file.

If it does end up in the Views group, just drag it out of the group.

➤ In **Exercise.swift**, add the following code below `import Foundation`:

```
struct Exercise {
  let exerciseName: String
  let videoName: String

  enum ExerciseEnum: String {
    case squat = "Squat"
    case stepUp = "Step Up"
    case burpee = "Burpee"
    case sunSalute = "Sun Salute"
  }
}
```

Enumerating Exercise Names

`enum` is short for *enumeration*. A Swift enumeration is a named type and can have methods and computed properties. It's useful for grouping related values so the compiler can help you avoid mistakes like misspelling a string.

> **Swift Tip**: A *stored property* is one you declare with a type and/or an initial value, like `let name: String` or `let name = "Audrey"`. You declare a *computed property* with a type and a closure where you compute its value, like `var body: some View { ... }`.

Here, you create an enumeration for the four exercise names. The case names are camelCase: If you start typing `ExerciseEnum.sunSalute`, Xcode will suggest the auto-completion.

Because this enumeration has `String` type, you can specify a `String` as the *raw value* of each case. Here, you specify the title-case version of the exercise name, like "Sun Salute" for `sunSalute`. Then `ExerciseEnum.sunSalute.rawValue` is `"Sun Salute"`.

Creating an Array of Exercise Instances

Use your enumeration to create your `exercises` array.

➤ **Below** `Exercise`, completely outside its braces, add this code:

```
extension Exercise {
  static let exercises = [
    Exercise(
      exerciseName: ExerciseEnum.squat.rawValue,
      videoName: "squat"),
    Exercise(
      exerciseName: ExerciseEnum.stepUp.rawValue,
      videoName: "step-up"),
    Exercise(
      exerciseName: ExerciseEnum.burpee.rawValue,
      videoName: "burpee"),
    Exercise(
      exerciseName: ExerciseEnum.sunSalute.rawValue,
      videoName: "sun-salute")
  ]
}
```

Swift Tips: Type Property, Array Literal, Type Extension

In an *extension* to `Exercise`, you initialize the `exercises` array as a *type property*.

`exerciseName` and `videoName` are *instance properties*: Each `Exercise` instance has its own values for these properties. A *type property* belongs to the type, and you declare it with the `static` keyword. The `exercises` array doesn't belong to an `Exercise` instance. There's only one `exercises` no matter how many `Exercise` instances you create. You access it with the type name: `Exercise.exercises`.

You create `exercises` with an *array literal*: a comma-separated list of values, enclosed in square brackets. Each value is an instance of `Exercise`, supplying the raw value of an enumeration case and the corresponding video file name.

As the word suggests, an *extension* extends a named type. The starter project includes two extensions: **DateExtension.swift** and **ImageExtension.swift**. `Date` and `Image` are built-in SwiftUI types but, by creating an `extension`, you can add custom methods and computed or type properties.

`Exercise` is your own custom type, so why do you have an extension? It's housekeeping: You're keeping this task — initializing an array of `Exercise` values — separate from the core definition of your structure — stored properties and any custom initializers.

Developers also use extensions to encapsulate the requirements for protocols, one for each protocol. Organizing code like this makes it easy to see where to add features or look for bugs.

Refactoring ContentView & ExerciseView

Now, you'll modify `ContentView` and `ExerciseView` to use your new `Exercise.exercises` array.

➤ In **ContentView.swift**, replace the `ForEach` line with this:

```
ForEach(Exercise.exercises.indices, id: \.self) { index in
```

Instead of `0 ..< 4`, you use the `exercises` array's built-in range. Because the range is no longer fixed, you must provide an `id` for each array element. `\.self` means each element is its own unique identifier.

➤ In **ExerciseView.swift**, delete `videoNames` and `exerciseNames`. The error flags tell you where you need to use `Exercise.exercises`.

You *could* replace `exerciseNames[index]` with `Exercise.exercises[index].exerciseName`, but you'll need to use `Exercise.exercises[index]` several times in `ExerciseView`. This is a good reason to define a **computed property**.

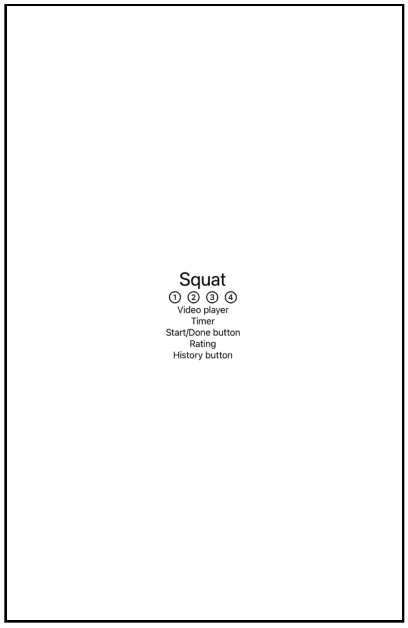
➤ Add this near the top of `ExerciseView`, below `let index: Int`:

```
var exercise: Exercise {
  Exercise.exercises[index]
}
```

➤ Then replace `exerciseNames[index]` with `exercise.exerciseName`:

```
HeaderView(exerciseName: exercise.exerciseName)
```

➤ **Live Preview** `ContentView` to check everything still works:



Exercise views work after refactoring.

Playing a Video

> **Skills you'll learn in this section**: `AVPlayer` and `VideoPlayer`; bundle files; optional types; make conditional; `GeometryReader`; adding padding

➤ In **ExerciseView.swift**, add this statement just below `import SwiftUI`:

```
import AVKit
```

Importing `AVKit` lets you use high-level types like `AVPlayer` to play videos with the usual playback controls.

➤ Now replace `Text("Video player")` with this line:

```
VideoPlayer(player: AVPlayer(url: url))
```

Xcode complains it "cannot find 'url' in scope", so you'll define this value next.

Getting the URL of a Bundle File

You need the URL of the video file for this exercise. The `videoName` property is the name part of the file. All the files have file extension **.mp4**.

These files are in the project folder, which you can access as `Bundle.main`. Its method `url(forResource:withExtension:)` gets you the URL of a file in the main app bundle **if it exists**. Otherwise, it returns `nil` which means *no value*. The return type of this method is an `Optional` type, `URL?`.

> **Swift Tip**: Swift's `Optional` type helps you avoid many hard-to-find bugs that are common in other programming languages. It's usually declared as a type like `Int` or `String` followed by a question mark: `Int?` or `String?`. If you declare `var index: Int?`, `index` can contain an `Int` or no value at all. If you declare `var index: Int` — with no `?` — `index` must *always* contain an `Int`. Use `if let index {...}` to check whether an optional has a value. The condition is `true` if `index` has a value. You can also check `index != nil`.

> **Note**: You'll learn more about the app bundle in Chapter 7, "Saving Settings" and about optionals in Chapter 8, "Saving History Data".

So you need to wrap an `if let` around the `VideoPlayer`. Yet another pair of braces! It can be hard to keep track of them all. But Xcode is here to help. ;]

➤ **Command-click** `VideoPlayer` and select **Make Conditional**:

```
if true {
  VideoPlayer(player: AVPlayer(url: url))
} else {
  EmptyView()
}
```

An `if-else` closure wraps `VideoPlayer`, with placeholders `true` and `EmptyView()`.

> **Xcode Tip**: Take advantage of features like **Embed in HStack** and **Make Conditional** to let Xcode keep your braces matched. To adjust what's included in the closure, use **Option-Command-[** or **Option-Command-]** to move the closing brace up or down.

➤ Now replace `if true {` with:

```
if let url = Bundle.main.url(
  forResource: exercise.videoName,
  withExtension: "mp4") {
```

➤ In the `else` closure, replace `EmptyView()` with:

```
Text("Couldn't find \(exercise.videoName).mp4")
  .foregroundColor(.red)
```

> **Swift Tip**: The *string interpolation* code `\(exercise.videoName)` inserts this value into the string literal.

➤ In **Live Preview**, click above or below the video to show the play button. Or just click the video to start/stop it.

Getting the Screen Dimensions

The video takes up a lot of space on the screen. You could set the width and height of its frame to some constant values that work on most devices, but it's better if these measurements adapt to the size of the device.

➤ In `body`, **Command-click** `VStack` and select **Embed...**. Change the `Container {` placeholder to this line:

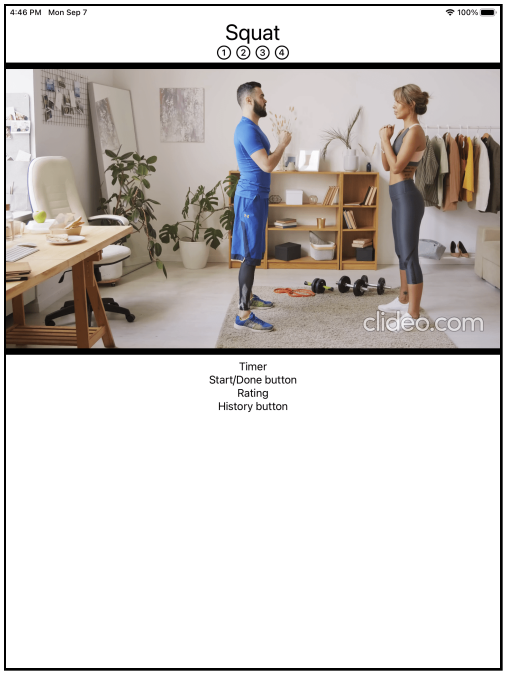
```
GeometryReader { geometry in
```

`GeometryReader` is a container view that provides you with the screen's measurements for whatever device you're previewing or running on.

➤ Add this modifier to `VideoPlayer`:

```
.frame(height: geometry.size.height * 0.45)
```

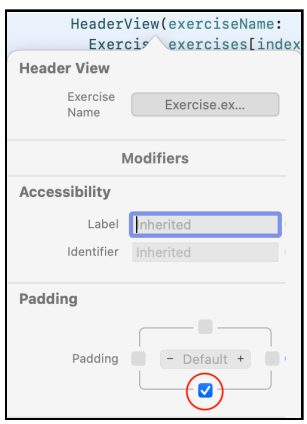
The video player now uses only 45% of the screen height:



Video player uses 45% of screen height.

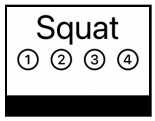
Adding Padding

➤ The header looks a little squashed. **Control-Option-click** `HeaderView` to add `padding` to its **bottom**:



Add bottom padding to Header view in Exercise view.

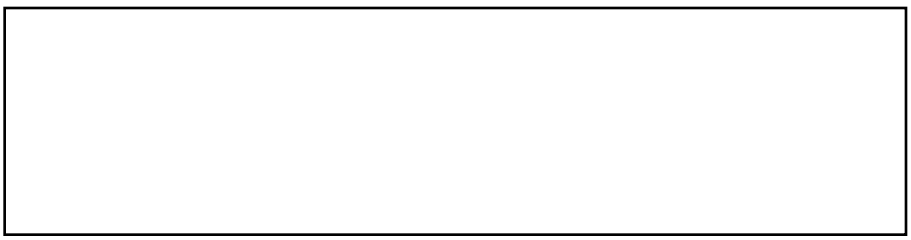
This gives you a new modifier `padding(.bottom)` and now there's space between the header and the video:



Padding under Header view

> **Note**: You *could* have added padding to the `VStack` *in* **HeaderView.swift**, but `HeaderView` is a little more reusable without padding. You can choose whether to add padding and how to customize it whenever you use `HeaderView` in another view.

➤ Head back to **ContentView.swift** and **Live Preview** your app. Swipe from one page to the next to see the different exercise videos.



HIITFit pages

Creating Timer, Buttons & Rating

> **Skills you'll learn in this section**: `Text` with date and style parameters; types in Swift; `Date()`; `Button`, `Spacer`, `foregroundColor`; repeating a view; unused closure parameter

Creating the Timer View

➤ Add this property to `ExerciseView`, above `body`:

```
let interval: TimeInterval = 30
```

These are high-intensity interval exercises, so the timer counts down from 30 seconds.

➤ Replace `Text("Timer")` with this code:

```
Text(Date().addingTimeInterval(interval), style: .timer)
  .font(.system(size: geometry.size.height * 0.07))
```

The default initializer `Date()` creates a value with the current date and time. The `Date` method `addingTimeInterval(_ timeInterval:)` adds `interval` seconds to this value.

➤ The Swift `Date` type has a lot of methods for manipulating date and time values. **Option-click** `Date` and **Open in Developer Documentation** to scan what's available. You'll dive a little deeper into `Date` when you create the **History** view.

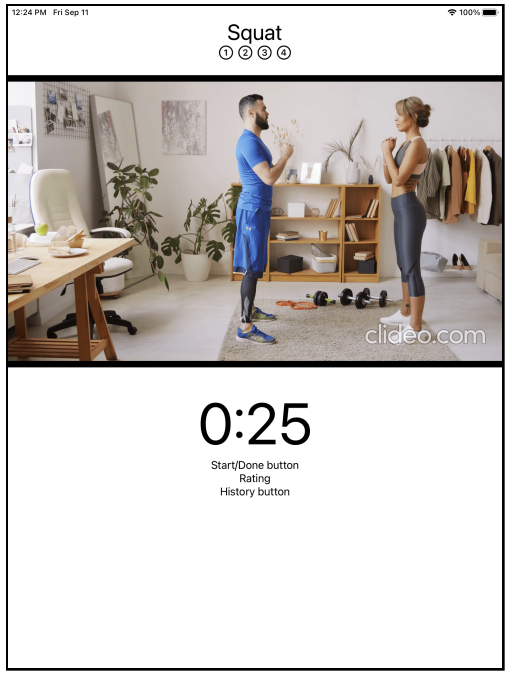
The `timeInterval` parameter's type is `TimeInterval`. This is simply an alias for `Double`. If you say `interval` is of type `Double`, you won't get an error, but `TimeInterval` describes the value's purpose more accurately.

> **Swift Tip**: Swift is a *strongly typed language*. This means that you must use the correct type. When using numbers, you can usually pass a value of a wrong type to the initializer of the correct type. For example, `Double(myIntValue)` creates a `Double` value from an `Int` and `Int(myDoubleValue)` truncates a `Double` value to create an `Int`. If you write code in languages that allow automatic conversion, it's easy to create a bug that's very hard to find. Swift makes sure you, and people reading your code — including "future you", know that you're converting one type to another.

You're using the `Text` view's `(_:style:)` initializer for displaying dates and times. The `timer` and `relative` styles display the time interval between the current time and the `date` value, formatted as "mm:ss" or "mm min ss sec", respectively. These two styles update the display every second.

You set the system font size to `geometry.size.height * 0.07` to make a really big timer — around 95 points for a 12.9" iPad and 47 points for the much smaller iPhone 8.

➤ Click **Live Preview** to watch the timer count down from 30 seconds:



Exercise view with 30-second timer

Because you set `date` to 30 seconds in the future, the displayed time interval decreases by 1 every second, as the current time approaches `date`. If you wait until it reaches 0 (change `interval` to 3 so you don't have to wait so long), you'll see it start counting **up**, as the current time moves away from `date`. Don't worry, this `Text` timer is just for the prototype. You'll replace it with a *real* timer in Chapter 6, "Observing Objects".

Creating Buttons

Creating buttons is simple, so you'll do both now.

➤ Replace `Text("Start/Done button")` with this code:

```
Button("Start/Done") { }
  .font(.title3)
  .padding()
```

Here, you gave the `Button` the label **Start/Done** and an empty action. You'll add the action in Chapter 6, "Observing Objects". Then, you enlarged the font of its label and added padding all around it.

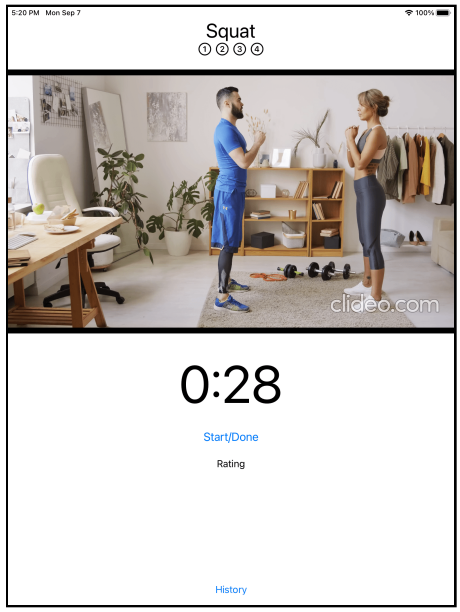
➤ Replace `Text("History button")` with this code:

```
Spacer()
Button("History") { }
  .padding(.bottom)
```

The `Spacer` pushes the History button to the bottom of the screen. The `padding` pushes it back up a little, so it doesn't look squashed.

You'll add this button's action in Chapter 5, "Moving Data Between Views".

Here's what `ExerciseView` looks like now:



Exercise view with buttons

Creating the Rating View

➤ Create a **new SwiftUI View file** in the **Views** group named **RatingView.swift**. This will be a small view, so add this modifier to its preview:

```
.previewLayout(.sizeThatFits)
```

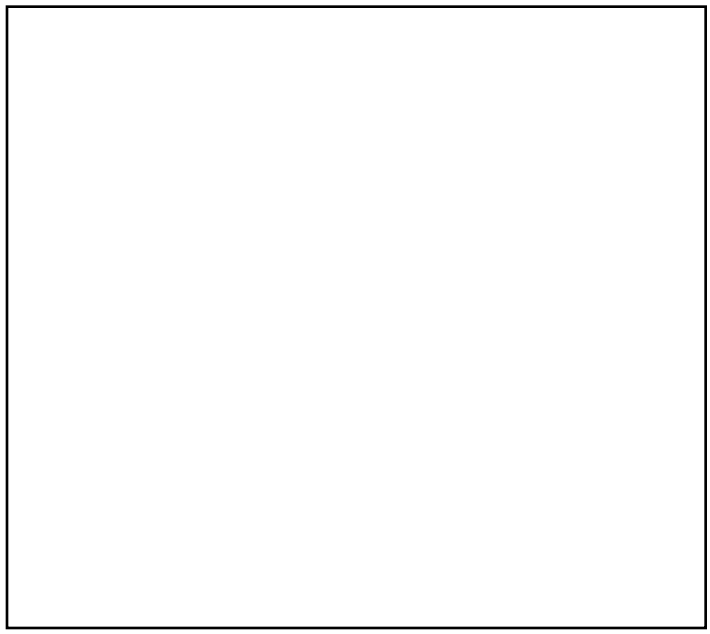
➤ Switch to **Selectable** mode and **Zoom to 100%**.

➤ Replace the boilerplate `Text` with this code, leaving the cursor between the double quotation marks:

```
Image(systemName: "")
  .foregroundColor(.gray)
```

A rating view is usually five stars or hearts, but the rating for an exercise should reflect the user's exertion. Something heart-related...

➤ Open the **Library** and search **Symbols** for "ecg":



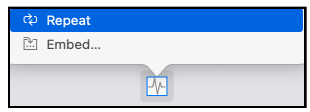
SF Symbols Health category: ECG waveform

➤ The ECG wave form seems just right for rating high-intensity exercises! Double-click it to insert its name between the double quotation marks:

```
Image(systemName: "waveform.path.ecg")
  .foregroundColor(.gray)
```

A rating view needs five of these symbols, arranged horizontally.

➤ In the canvas or in the editor, **Command-click** the `Image` and select **Repeat** from the menu:



Command-click Image, select Repeat.

Xcode gives you a loop, with placeholder range `0 ..< 5`:

```
ForEach(0 ..< 5) { item in
  Image(systemName: "waveform.path.ecg")
    .foregroundColor(.gray)
}
```

➤ Click this range and press **Return** to accept it.

In the canvas, you see five separate preview pages! Xcode should have embedded them in a stack, like when you duplicated a view, but it didn't.

➤ **Command-click** `ForEach` and embed it in an `HStack`.

Now your code looks like this:

```
HStack {
  ForEach(0 ..< 5) { item in
    Image(systemName: "waveform.path.ecg")
      .foregroundColor(.gray)
  }
}
```

That's better! And the symbols are all in a row. But they're very small.

➤ Remember, you can use `font` to specify the size of SF Symbols. So add this modifier to the `Image`:

```
.font(.largeTitle)
```

Bigger is better!



SF Symbols with largeTitle font size

One last detail: The code Xcode created for you contains an unused *closure parameter* `item`:

```
ForEach(0 ..< 5) { item in
```

➤ You don't use `item` in the loop code, so replace `item` with `_`:

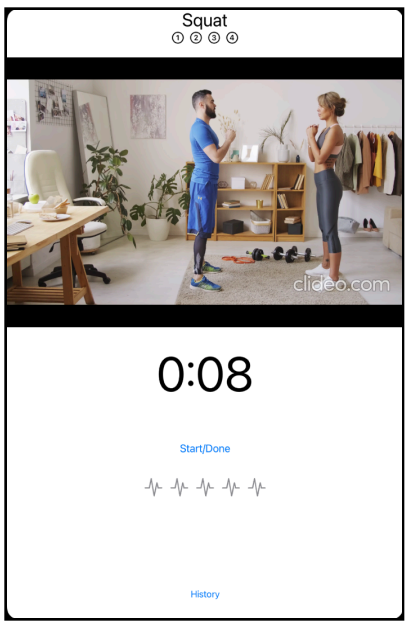
```
ForEach(0 ..< 5) { _ in
```

> **Swift Tip**: It's good programming practice to replace unused parameter names with _. The alternative is to create a throwaway name, which takes a non-zero amount of time and focus and will confuse you and other programmers reading your code.

➤ Now head back to **ExerciseView.swift** to use your new view. Replace `Text("Rating")` with this code:

```
RatingView()
  .padding()
```

Your ECG wave forms now march across the screen!



Exercise view with Rating subview

In Chapter 5, "Moving Data Between Views", you'll add code to let the user set a rating value and represent this value by setting the right number of symbols to red. And, in Chapter 7, "Saving Settings", you'll save the rating values so they persist across app launches.

Challenges

`ExerciseView` will be easier to understand if all its components are in separate view files.

Challenge: Create VideoPlayerView

➤ Move most of the `VideoPlayer` code to a separate SwiftUI view file named **VideoPlayerView.swift**, so you can call it in `ExerciseView` like this:

```
VideoPlayerView(videoName: exercise.videoName)
  .frame(height: geometry.size.height * 0.45)
```

The solution to this challenge is in this chapter's **challenge** folder.

Key Points

- Declare SwiftUI views in the order you want them to appear.
- Create separate views for your user interface elements. This makes your code easier to read and maintain.
- Put each view modifier on its own line. This makes it easy to move or delete a modifier.
- Xcode and SwiftUI auto-suggestions and default values are often what you want.
- Let Xcode help you avoid errors: Use the **Command**-menu to embed or extract views.
- The **SF Symbols** app and Xcode's **Symbols Library** provide icon images you can configure like text.
- Preview variants make it easy to check your interface for different user settings.
- An *enumeration* is a named type, useful for grouping related values so the compiler can help you avoid mistakes like misspelling a string.
- Swift is a strongly typed programming language.
- `GeometryReader` enables you to set a view's dimensions relative to the screen dimensions.

Where to Go From Here?

In the next chapter, you'll lay out views for History, Welcome and Success.

Chapter 4: Prototyping Supplementary Views

By Audrey Tam

Your app still needs three more full-screen views:

- Welcome
- History
- Success

In the previous chapter, you laid out the **Exercise** view and created an `Exercise` structure. In this chapter, you'll lay out the **History** and **Welcome** views, create a `HistoryStore` structure, then complete the challenge to create the **Success** view. And your app's prototype will be complete.

Laying Out the History View

Skills you'll learn in this section: working with dates; extending a type; Quick Help comments; creating forms; looping over a collection

You'll start with a mock-up of the list view. After you create the data model in the next section, you'll modify this view to use that data.

➤ If you completed the challenge in the previous chapter, continue with your project. Or open the project in this chapter's **starter** folder.

➤ In the **Views** group, create a **new SwiftUI View file** named **HistoryView.swift**. For this mock-up, add some sample history data to `HistoryView`, above `body`:

```
let today = Date()
let yesterday = Date().addingTimeInterval(-86400)

let exercises1 = ["Squat", "Step Up", "Burpee", "Sun Salute"]
let exercises2 = ["Squat", "Step Up", "Burpee"]
```

You'll display exercises completed over two days.

➤ Replace `Text("Hello, World!")` with this code:

```
VStack {
  Text("History")
    .font(.title)
    .padding()
  // Exercise history
}
```

You've created the title for this view with some padding around it.

Creating a Form

SwiftUI has a container view that automatically formats its contents to look organized.

➤ Inside the `VStack`, replace `// Exercise history` with this code:

```
Form {
  Section(
    header:
    Text(today.formatted(as: "MMM d"))
```

```
        .font(.headline)) {
      // Section content
    }
    Section(
      header:
      Text(yesterday.formatted(as: "MMM d"))
        .font(.headline)) {
      // Section content
    }
  }
```

Inside the `Form` container view, you create two sections. Each `Section` has a header with the date, using `headline` font size.

This code takes yesterday and today's date as the section headers, so your view will have different dates from the one below:



History Form with two Sections

Extending the Date Type

When you created the timer view, you had a quick look at the Swift `Date` type and used one of its methods. It's now time to learn a little more about it.

> **Swift Tip**: A `Date` object is just some number of seconds relative to January 1, 2001 00:00:00 UTC. To display it as a calendar date in a particular time zone, you must use a `DateFormatter`. This class has a few built-in styles named `short`, `medium`, `long` and `full`, described in links from the developer documentation page for `DateFormatter.Style`. You can also specify your own format as a `String`.

➤ Open **DateExtension.swift**. The first method shows how to use a `DateFormatter`.

```
func formatted(as format: String) -> String {
  let dateFormatter = DateFormatter()
  dateFormatter.dateFormat = format
  return dateFormatter.string(from: self)
}
```

`DateFormatter` has only the default empty initializer. You create an instance, then configure it by setting the properties you care about. This method uses its `format` argument to set the `dateFormat` property.

In `HistoryView`, you pass `"MMM d"` as `format`. This specifies three characters for the month — so you get SEP or OCT — and one character for the day — so you get a number. If the number is a single digit, that's what you see. If you specify `"MM dd"`, you get numbers for both month and day, with leading 0 if the number is single digit: **09 02** instead of **SEP 2**.

Once you've configured `dateFormatter`, its `string(from:)` method returns the date string.

You don't have to worry about time zones if you simply want the user's current time zone. That's the default setting.

Formatting Quick Help Comments

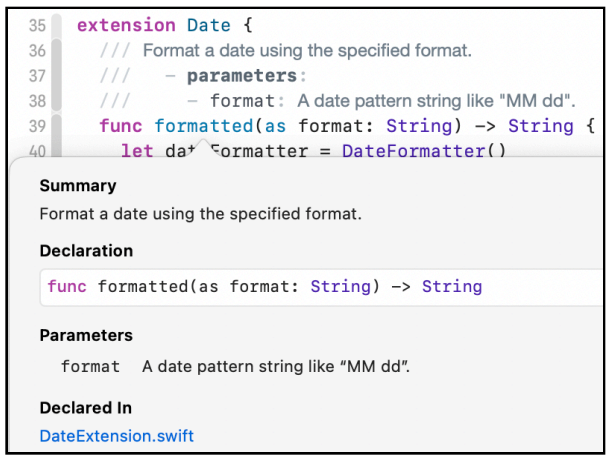
Extending the `Date` class with `formatted(as:)` makes it easy to get a `Date` in the format you want: `today.formatted(as: "MMM d")`.

> **Swift Tip**: You can add methods to extend any type, including those built into the software development kit, like `Image` and `Date`. Then, you can use them the same way you use the built-in methods.

➤ Look at the comment above the `formatted(as:)` method:

```
/// Format a date using the specified format.
///   - parameters:
///     - format: A date pattern string like "MM dd".
```

This is a special kind of comment. It appears in Xcode's **Quick Help** when you **Option-click** the method name:



DIY Quick Help documentation comment

It looks just like all the built-in method summaries!

It's good practice to document all the methods you write this way. See Apple's Formatting Quick Help (https://apple.co/33hohbk) documentation for more details.

Looping Over a Collection

➤ Now, head back to **HistoryView.swift** to fill in the `Section` content.

To display the completed exercises for each day, you'll use `ForEach` to loop over the elements of `exercises1` and `exercises2`.

➤ In the first `Section`, replace `// Section content` with this code:

```
ForEach(exercises1, id: \.self) { exercise in
  Text(exercise)
}
```

In `ContentView`, you looped over a number range. Here, you're using the third `ForEach` initializer for **Creating a collection from data**:

```
init(Data, id: KeyPath<Data.Element, ID>, content:
(Data.Element) -> Content)
```

`exercises1` is the `Data` and `\.self` is the key path to each array element's identifier. `\.self` means each element of the array is its own unique identifier.

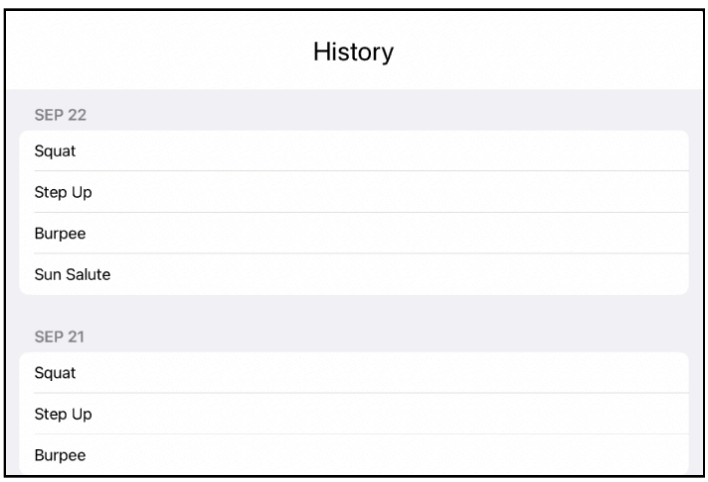
As the loop visits each array element, you assign it to the local variable `exercise`, which you display in a `Text` view.

➤ In the second `Section`, replace `// Section content` with the almost identical code:

```
ForEach(exercises2, id: \.self) { exercise in
  Text(exercise)
}
```

This time, you display `exercises2`.

➤ Refresh the preview to admire your exercise history:



History list for two days

Of course, `HistoryView` needs to display any number of days, with any collection of exercises on each day. You need a data structure that enables you to loop over a *collection* of days. For each day, you'll create a `Section`, where you loop over that day's exercises.

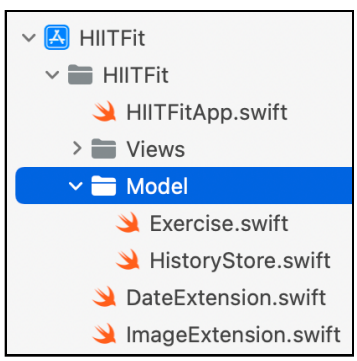
Structuring HistoryView Data

Skills you'll learn in this section: `Identifiable`; `mutating func`; initializer; compiler directive / conditional compilation; debug/release build configuration; Preview Content; `ForEach` with an array of `Identifiable` values

In this section, you'll create a data structure to store the user's activity, to replace the hard-coded dates and exercise lists in your mock-up.

Creating HistoryStore

➤ Outside the **Views** group, create a **new Swift file** and name it **HistoryStore.swift**. Group it with **Exercise.swift** and name the group folder **Model**:



Model group with Exercise and HistoryStore

➤ In **HistoryStore.swift**, add the following code below `import Foundation`:

```
struct ExerciseDay: Identifiable {
  let id = UUID()
  let date: Date
  var exercises: [String] = []
}

struct HistoryStore {
  var exerciseDays: [ExerciseDay] = []
}
```

An `ExerciseDay` has properties for the date and a list of exercise names completed by the user on that date.

`ExerciseDay` conforms to `Identifiable`. This protocol is useful for named types that you plan to use as elements of a collection, because you usually want to loop over these elements or display them in a list.

When you loop over a collection with `ForEach`, it must have a way to uniquely identify each of the collection's elements. The easiest way is to make the element's type conform to `Identifiable` and include `id: UUID` as a property. `UUID` is a basic `Foundation` type, and `UUID()` is the easiest way to create a unique identifier whenever you create an `ExerciseDay` instance.

The only property in `HistoryStore` is the array of `ExerciseDay` values you'll loop over in `HistoryView`.

In Chapter 8, "Saving History Data", you'll extend `HistoryStore` with a method to save the user's history to persistent storage and another method to load the history. Soon, you'll add a `HistoryStore` property to `HistoryView`, which will initialize it.

In the meantime, you need some sample history data and an initializer to create it.

➤ **Below** `HistoryStore`, completely outside its braces, add this code:

```
extension HistoryStore {
  mutating func createDevData() {
    // Development data
    exerciseDays = [
      ExerciseDay(
        date: Date().addingTimeInterval(-86400),
        exercises: [
          Exercise.exercises[0].exerciseName,
          Exercise.exercises[1].exerciseName,
          Exercise.exercises[2].exerciseName
        ]),
      ExerciseDay(
        date: Date().addingTimeInterval(-86400 * 2),
        exercises: [
          Exercise.exercises[1].exerciseName,
          Exercise.exercises[0].exerciseName
        ])
    ]
  }
}
```

The exercise lists are slightly different to the mock-up data in HistoryView, and they're stored in your new `Exercise` and `ExerciseDay` structures. In Chapter 6, "Observing Objects", you'll add a new `ExerciseDay` item, so I've set the development data to yesterday and the day before yesterday.

You create this sample data in a method named `createDevData()`. This method changes, or mutates, `exerciseDays`, so you must mark it with the `mutating` keyword. And you create this method in an extension because it's not part of the core definition. But there's another reason, too — coming up soon!

➤ Now, in the main `HistoryStore`, create an initializer for `HistoryStore` that calls `createDevData()`:

```
init() {
  #if DEBUG
  createDevData()
  #endif
}
```

You don't want to call `createDevData()` in the release version of your app, so you use a *compiler directive* to check whether the current **Build Configuration** is **Debug**:



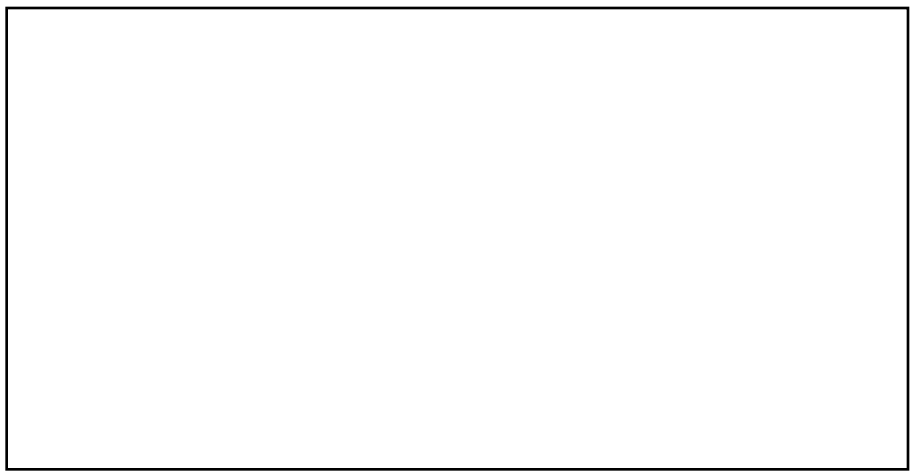
Debug build configuration

Note: To see this window, click the Scheme menu button (next to the run destination menu). Select **Edit Scheme...**, then select the **Info** tab.

Moving Development Code Into Preview Content

In fact, you don't want `createDevData()` to ship in your release version at all. Xcode provides a place for development code and data: **Preview Content**. Anything you put into this group will not be included in your release version. So handy!

➤ In the **Preview Content** group, create a **new Swift file** named **HistoryStoreDevData.swift** and move the `HistoryStore` extension into it:



HistoryStore extension in Preview Content

And this is the other reason `createDevData()` is in an extension: You can store extensions in separate files. This means you never have to scroll through very long files.

Using HistoryStore in HistoryView

➤ Now, in **HistoryView.swift**, delete the `Date` properties and the exercise arrays, then add this property:

```
let history = HistoryStore()
```

`HistoryStore` now encapsulates all the information in the stored properties `today`, `yesterday` and the exercises arrays.

The `Form` closure currently displays each day in a `Section`. Now that you have an `exerciseDays` array, you should loop over it.

➤ Replace the `Form` closure with the following:

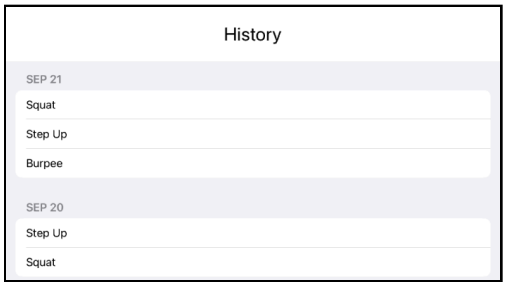
```
Form {
  ForEach(history.exerciseDays) { day in
    Section(
      header:
        Text(day.date.formatted(as: "MMM d"))
        .font(.headline)) {
      ForEach(day.exercises, id: \.self) { exercise in
```

```
            Text(exercise)
          }
        }
      }
    }
```

Instead of `today` and `yesterday`, you use `day.date`. And, instead of the named exercises arrays, you use `day.exercises`.

The code you just replaced looped over `exercises1` and `exercises2`, which were arrays of `String`. The `id: \.self` argument told `ForEach` to use the instance itself as the unique identifier. The `exercises` array also contains `String` instances, so you still need to specify this `id` value.

➤ Check out the preview:



History view works after refactoring.

Congratulations, your data structure works in your view. Just one finishing touch remains.

Dismissing HistoryView

Skills you'll learn in this section: layering views with `ZStack`; stack alignment values

Creating a Button in Another Layer

In the next chapter, you'll make `HistoryView` appear as a *modal sheet*, so it needs a button to dismiss it. You'll often see a dismiss button in the upper right corner of a modal sheet. The easiest way to place it there, without disturbing the layout of the rest of `HistoryView`, is to put it in its own layer.

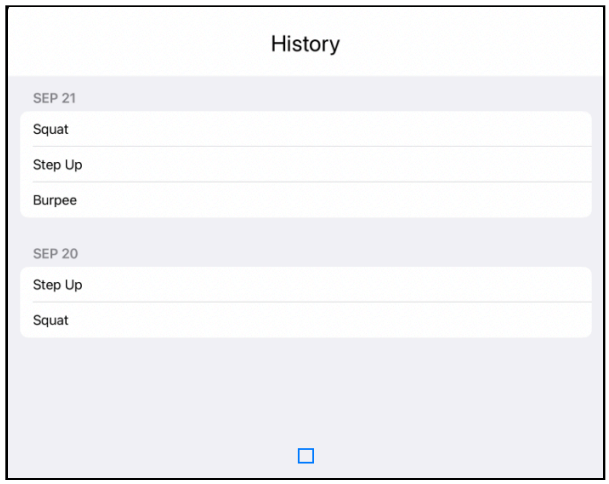
ZStack

If you think of an `HStack` as arranging its contents along the device's x-axis and a VStack arranging views along the y-axis, then the `ZStack` container view stacks its contents along the *z-axis*, perpendicular to the device screen. Think of it as a **depth** stack, displaying views in layers.

➤ **Command-click** `VStack` to embed it in a `ZStack`, then add this code at the top of `ZStack` above the `VStack`:

```
Button(action: {}) {
  Image(systemName: "xmark.circle")
}
```

➤ Switch the preview to **Selectable** mode to see the outline of the button:



Dismiss button outline

The button is centered in the view, because the default *stack alignment* is `center`. Because you added the `Button` code *above* the `VStack` in the source code, it's *underneath* the `VStack` on screen, so you see only its outline.

The arrangement is a little counter-intuitive unless you think of it as placing the first view down on a flat surface, then layering the next view on top of that, and so on. So declaring the button as the *first* view places it on the *bottom* of the stack. If you want the button in the *top layer*, declare it *last* in the `ZStack`.

It doesn't matter in this case, because you're about to move the button into the top right corner of the view, where there's nothing in the `VStack` to cover it.

Stack Alignment

You can specify an `alignment` value for any kind of stack, but they all use different `alignment` values. `VStack` `alignment` values are horizontal: `leading`, `center` or `trailing`. `HStack` `alignment` values are vertical: `top`, `center`, `bottom`, `firstTextBaseline` or `lastTextBaseline`.

To specify the alignment of a `ZStack`, you must set both horizontal and vertical alignment values. You can either specify separate horizontal and vertical values, or a combined value like `topTrailing`.

➤ Replace `ZStack {` with this:

```
ZStack(alignment: .topTrailing) {
```

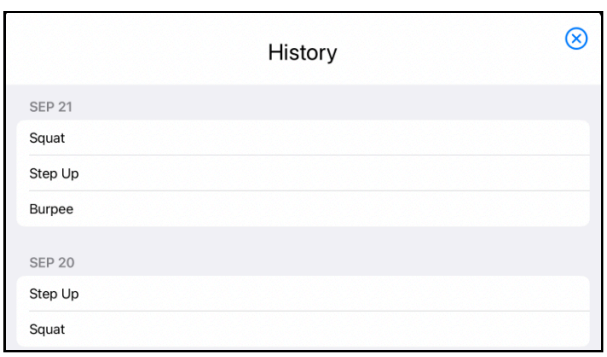
You set the `ZStack` `alignment` parameter to position the button in the top right corner of the view. Other views in the `ZStack` have their own `alignment` values, so the `ZStack` `alignment` value doesn't affect them.

The button is now visible, but it's small and a little too close to the corner edges.

➤ Add these modifiers to the `Button` to adjust its size and position:

```
.font(.title)
.padding(.trailing)
```

➤ Check out the preview:



History view with dismiss button in top trailing corner

You're finished with `HistoryView` for now. Next up: `WelcomeView`.

Laying Out the Welcome View

> **Skills you'll learn in this section**: refactoring/renaming a parameter; modifying images; using a custom modifier; `Button` label with text and image

➤ Open **WelcomeView.swift**.

`WelcomeView` is the first page in your app's page-style `TabView`, so it should have the same header as `ExerciseView`.

➤ Replace `Text("Hello, World!")` with this line:

```
HeaderView(exerciseName: "Welcome")
```

You want the title of this page to be "Welcome", so you pass this as the value of the `exerciseName` parameter. `HeaderView` also displays the page numbers of the four exercises:



Welcome view header: First try

Refactoring HeaderView

Using `HeaderView` here raises two issues:

1. There's no page number for the **Welcome** page.
2. The parameter name `exerciseName` isn't a good description of "Welcome".

The first issue is easy to resolve. The app has only one non-exercise page, so you just need to add another page "number" in `HeaderView`.

➤ In **HeaderView.swift**, duplicate the first `Image`, then change the now-first `Image` to display a hand wave:

```
Image(systemName: "hand.wave")
```



Header with hand-wave symbol for Welcome page number

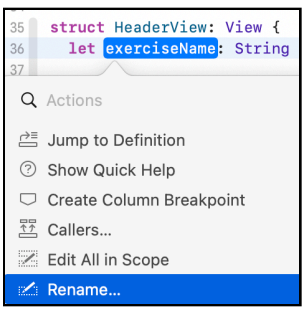
That'll do nicely.

Now you need to rename the `exerciseName` property. Its purpose is really to be the title of the page, so `titleText` is a better name for it.

You could search for all occurrences of `exerciseName` in your app, then decide for each whether to change it to `titleText`. In a more complex app, this approach almost guarantees you'll forget one or change one that shouldn't change.

Xcode has a safer way! You've already used it to rename `ExtractedView`.

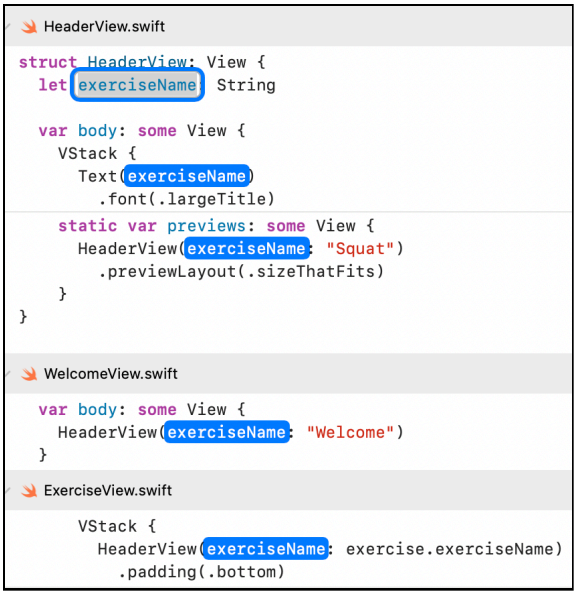
➤ **Command-click** the first occurrence of `exerciseName` and select **Rename...** from the menu:



Command-click exerciseName, select Rename.

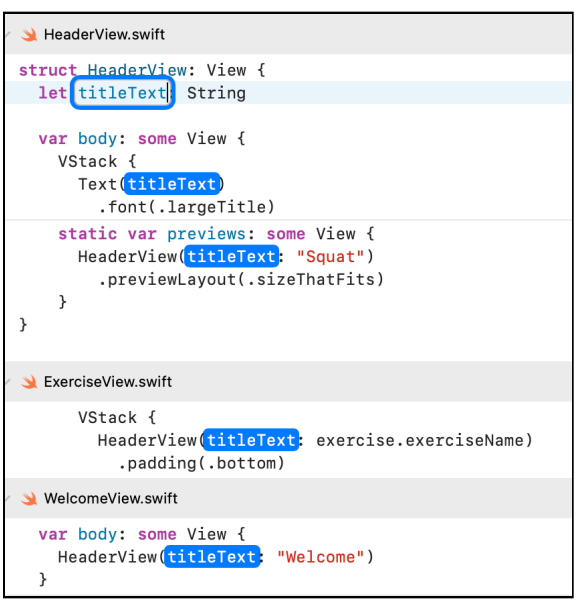
> **Note**: If you **Command-click** `exerciseName` in `Text(exerciseName)`, you'll see the longer menu that includes *Embed in HStack* etc. **Rename...** is at the bottom of this menu.

Xcode displays the code statements in three files that need to change:



Xcode shows code affected by name change.

➤ The first instance is highlighted differently. Type **titleText**, and all the instances change:



Change exerciseName to titleText.

➤ Click the **Rename** button in the upper right corner to confirm these changes, then head back to **WelcomeView.swift** to see the results:



Welcome view with refactored Header view: issues resolved

That's better! The user sees a page icon, and the programmer sees a descriptive parameter.

More Layering With ZStack

So far, so good, but the header should be at the top of the page. A **History** button should be at the bottom of the page. The main content should be centered in the view, independent of the heights of the header and button.

In `HistoryView`, you used a `ZStack` to position the dismiss button in the upper right corner (`topTrailing`), without affecting the layout of the other content.

In this view, you'll use a `ZStack` to put the header and **History** button in one layer, to push them apart. Then you'll create the main content in another layer, centered by default.

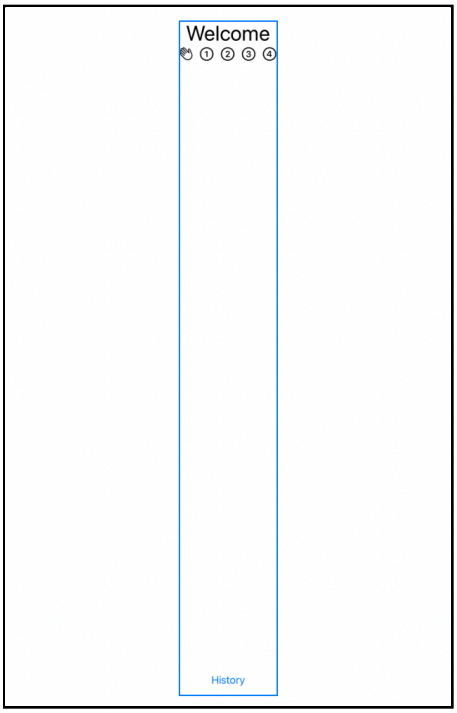
➤ First, embed `HeaderView` in a `VStack`, then embed that `VStack` in a `ZStack`.

```
ZStack {
  VStack {
    HeaderView(titleText: "Welcome")
  }
}
```

➤ In the `VStack`, below `HeaderView`, add this code:

```
Spacer()
Button("History") { }
  .padding(.bottom)
```

You have the header and the **History** button in a `VStack`, with a `Spacer` to push them apart and some padding so the button isn't too close to the bottom edge:

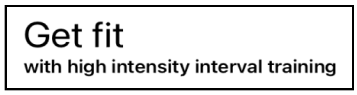


Welcome view header and footer

➤ Now to fill in the middle space. Add this layer to the `ZStack`:

```
VStack {
  HStack {
    VStack(alignment: .leading) {
      Text("Get fit")
        .font(.largeTitle)
      Text("with high intensity interval training")
        .font(.headline)
    }
  }
}
```

> **Note**: You can add this `VStack` either above or below the existing `VStack`. It doesn't matter because there's no overlapping content in the two layers.



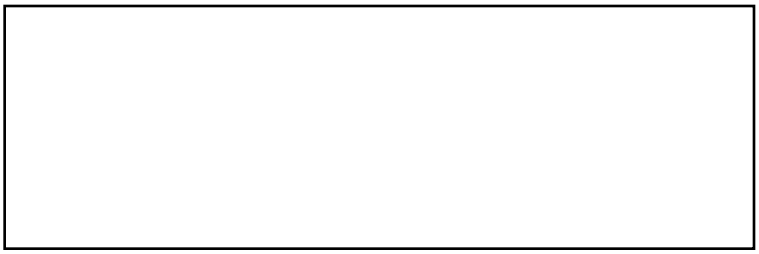
Welcome view center text

The inner `VStack` contains two `Text` views with different font sizes. You set its alignment to `leading` to left-justify the two `Text` views.

This `VStack` is in an `HStack` because you're going to place an `Image` to the right of the text. And the `HStack` is in an outer `VStack` because you'll add a `Button` below the text and image.

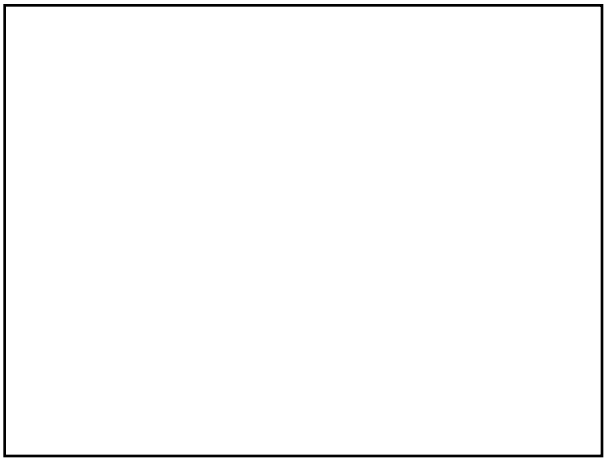
Using an Image

➤ Look in **Assets.xcassets** for the **step-up** image:



step-up image in Assets

➤ Back in **WelcomeView.swift**, open the **Library** with **Shift-Command-L** (or click the + toolbar button) and select the media tab:

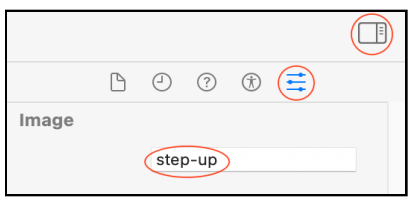


Add image from Xcode media library.

➤ To insert **step-up** in the correct place, it's easiest to drag it into the **code editor**. Hold onto it while nudging the code with the cursor, until a line opens, just below the `VStack` of two `Text` views. Let go of the image, and it appears in your code:

```
HStack {
  VStack(alignment: .leading) {
    Text("Get fit")
      .font(.largeTitle)
    Text("with high intensity interval training")
      .font(.headline)
  }
  Image("step-up")  // your new code appears here
}
```

➤ You usually have to add several modifiers to an `Image`, so open the **Attributes inspector** in the **inspectors panel**:

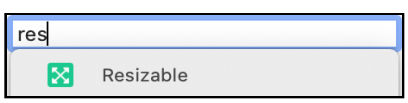


Open Attributes inspector in the inspector panel.

> **Note**: If you don't see **Image** with a value of **step-up**, select the image or select another inspector then re-select **Attributes**.

Modifying an Image

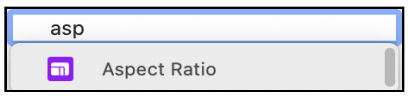
➤ First, you must add a modifier that lets you *resize* the image. In the **Add Modifier** field, type **res** then select **Resizable**.



Select Resizable modifier.

Don't worry if the image stretches. You'll fix that with the next modifier.

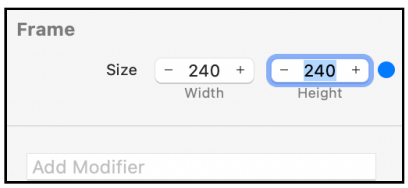
➤ When resizing an image, you usually want to preserve the *aspect ratio*. So search for an **aspect** modifier and select **Aspect Ratio**:



Select Aspect Ratio modifier.

➤ The suggested `contentMode` value is `fill`, which is what you usually want, so accept it.

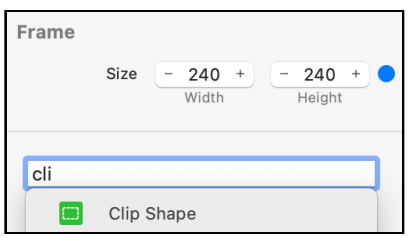
➤ Now the image looks more normal, but it's too big. In the **Frame** section, set the **Width** and **Height** to **240**:



Set Frame Width and Height to 240.

That's looking pretty good! How about *clipping* it to a circle?

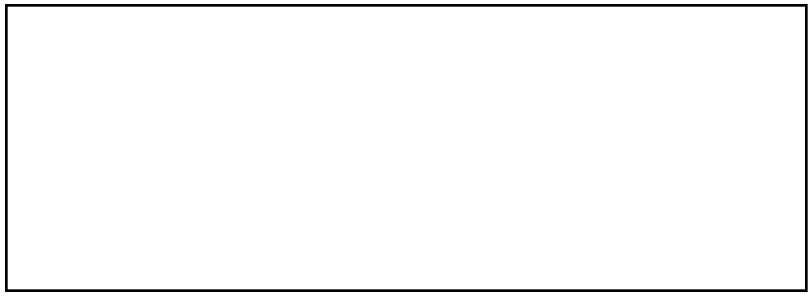
➤ Search for a **clip** modifier and select **Clip Shape**:



Select Clip Shape modifier.

➤ Again, the suggestion `Circle()` is what you want, so accept it.

Your `HStack` code and preview now look like this:

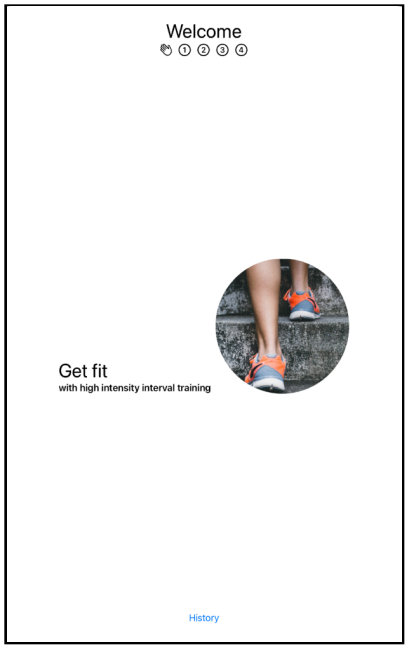


Welcome view center view

➤ You need just one more tweak: The text would look better if you align it with the bottom of the image. Just change the *alignment* of the enclosing `HStack`:

```
HStack(alignment: .bottom)
```

And here's your **Welcome** page:



Welcome view center view with text aligned to bottom of HStack

You've done enough to make it look welcoming. :] In Chapter 9, "Refining Your App", you'll add a few more images.

Using a Custom Modifier

You'll use this triplet of `Image` modifiers *all the time*:

```
.resizable()
.aspectRatio(contentMode: .fill)
.frame(width: 240.0, height: 240.0)
```

Everyone does, although the frame dimensions won't always be 240. In **ImageExtension.swift**, you'll find `resizedToFill(width:height:)` which encapsulates these three modifiers:

```
func resizedToFill(width: CGFloat, height: CGFloat)
-> some View {
  return self
    .resizable()
    .aspectRatio(contentMode: .fill)
    .frame(width: width, height: height)
}
```

It extends the `Image` view, so `self` is the `Image` you're modifying with `resizedToFill(width:height:)`.

➤ To use this custom modifier, head back to **WelcomeView.swift**. Comment out (**Command**-/) or delete the first three modifiers of `Image("step-up")`, then add this custom modifier:

```
.resizedToFill(width: 240, height: 240)
```

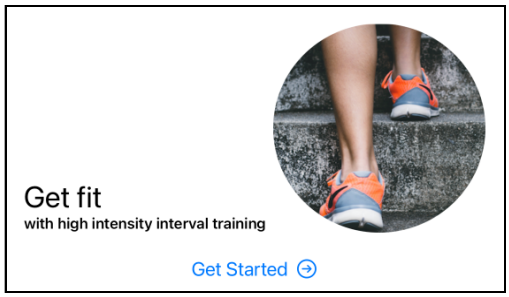
And the view looks the same, but there's a little less code.

Labeling a Button With Text & Image

The final detail is a `Button`. The user can tap this to move to the first exercise page, but the label also has an arrow image to indicate they can swipe to the next page. The other buttons you've created have only text labels. But it's easy to label a `Button` with text *and* an image.

➤ In the center view `VStack`, below the `HStack` with the image, add this code:

```
Button(action: { }) {
  Text("Get Started")
  Image(systemName: "arrow.right.circle")
}
.font(.title2)
.padding()
```



Welcome view Get Started button

This code is quite different from the other buttons you've created and requires some explanation. SwiftUI uses a lot of *syntactic sugar*: Instead of using the official method calls, SwiftUI lets you write code that's much simpler and more readable.

(action, label) vs. (String, action)

The *official* `Button` signature is:

```
Button(action: () -> Void, label: () -> Label)
```

- `action` is a method or a closure containing executable code.
- `label` is a view describing the button's `action`.

Both parameter values can be closures, so `action` can be more than one executable statement, and `label` can be more than one view.

The buttons you've created so far use the simplest `Button` syntax: The button's label is simply a `String`, and the button's action is in a trailing closure. For example:

```
Button("History") { }
```

> **Swift Tip**: You can move the last closure argument of a function call outside the parentheses into a *trailing closure*.

This simple `Button` syntax reverses the official signature, and it's only for the case where `label` is a string.

If you want more than a string in your `label`, its content must be in a closure. It's the last closure argument of this function call, so it can be a trailing closure:

```
Button(action: {} ) {
  <Content>
}
```

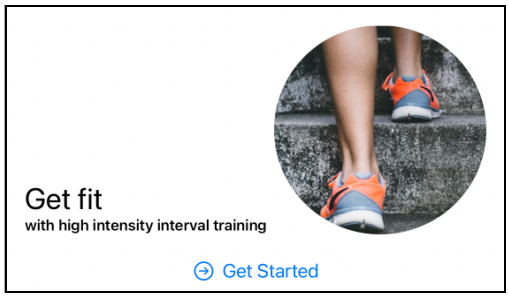
This is the syntax used in the "Get Started" `Button` above, with the `Text` and `Image` views in an implicit `HStack`.

The Label View

➤ The `Label` view is another way to label a `Button` with text and image. Comment out (**Command**-/) the `Text` and `Image` lines, then write this line in the `label` closure:

```
Label("Get Started", systemImage: "arrow.right.circle")
```

Look closely: Do you see what changed?



Welcome view Get Started button with Label view

Note: You can modify a `Label` with `labelStyle` to show only the text or only the image.

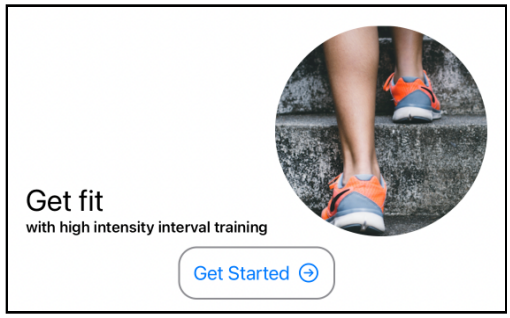
The image is on the *left side* of the text. This looks wrong to me: An arrow pointing right should appear *after* the text. Unfortunately for this particular `Button`, there's no way to make the image appear to the right of the text, unless you're using a language like Arabic that's written right-to-left. `Label` is ideal for icon-text lists, where you want the icons nicely aligned on the leading edge.

➤ Delete the `Label` and uncomment the `Text` and `Image`.

A Border For Your Button

➤ Just for fun, give this button a border. Add this modifier **below** `padding()`:

```
.background(
  RoundedRectangle(cornerRadius: 20)
  .stroke(Color.gray, lineWidth: 2))
```

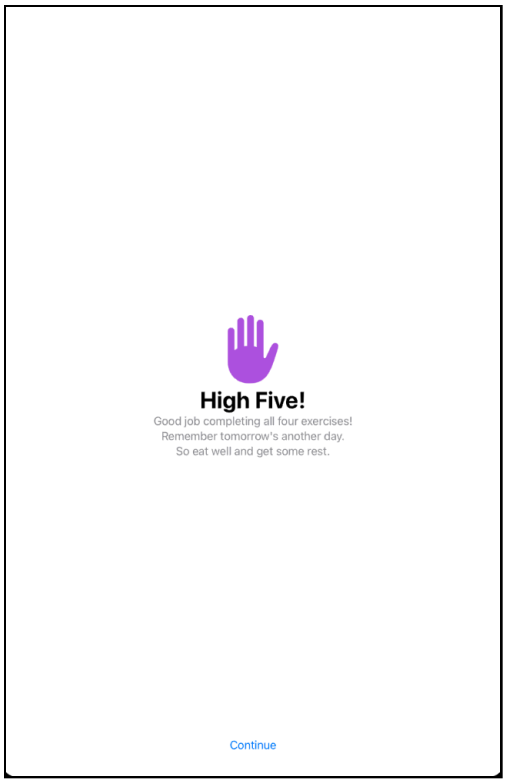


Welcome view Get Started button with border

You put a rounded rectangle around the *padded* button, specifying the corner radius, line color and line width.

Challenge

When your users tap **Done** on the last exercise page, your app will show a modal sheet to congratulate them on their success. Your challenge is to create this `SuccessView`:

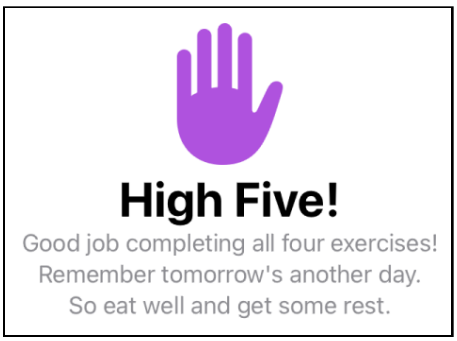


Challenge: Create this Success view.

Challenge: Creating the Success View

1. Create a **new SwiftUI View file** named **SuccessView.swift**.
2. Replace its `Text` view with a `VStack` containing the `hand.raised.fill` symbol and the text in the screenshot.
3. The symbol is in a 75 by 75 frame and colored purple. **Hint**: Use the custom `Image` modifier.
4. For the large "High Five!" title, you can use the `fontWeight` modifier to emphasize it more.
5. For the three small lines of text, you could use three `Text` views. Or refer to our Swift Style Guide (https://bit.ly/30cHeeL) to see how to create a *multi-line string*. `Text` has a `multilineTextAlignment` modifier. This text is colored gray.
6. Like `HistoryView`, `SuccessView` needs a button to dismiss it. Center a **Continue** button at the bottom of the screen. **Hint**: Use a `ZStack` so the "High Five!" view remains vertically centered.

Here's a close-up of the "High Five!" view:



Success view center view

You'll find the solution to this challenge in the **challenge** folder for this chapter.

Key Points

- The `Date` type has many built-in properties and methods. You need to configure a `DateFormatter` to create meaningful text to show your users.
- Use the `Form` container view to quickly lay out table data.
- `ForEach` lets you loop over the items in a collection.
- To use a collection in a `ForEach` loop, it needs to have a way to uniquely identify each of its elements. The easiest way is to make it conform to `Identifiable` and include `id: UUID` as a property.
- Use compiler directives to create development data only while you're developing and not in the release version of your app.
- **Preview Content** is a convenient place to store code and data you use only while developing. Its contents won't be included in the release version of your app.
- `ZStack` is useful for keeping views in one layer centered while pushing views in another layer to the edges.
- You can specify vertical alignment values for `HStack`, horizontal alignment values for `VStack` and combination alignment values for `ZStack`.
- Xcode helps you to refactor the name of a parameter quickly and safely.
- `Image` often needs the same three modifiers. You can create a custom modifier so you Don't Repeat Yourself.
- A `Button` has a label and an action. You can define a `Button` a few different ways.

Where to Go From Here?

Your views are all laid out. You're eager to implement all the button actions. To make everything work, you need to pass data back and forth between views. You already know how to pass data **to** a view. But some of your views need to change values and send them back. Excitement awaits!

Chapter 5: Moving Data Between Views

By Audrey Tam

In the previous chapter, you structured your app's data to be more efficient and less error-prone. In this chapter, you'll implement most of the functionality your users expect when navigating and using your app. Now, you'll need to *manage* your app's data so values flow smoothly through the views and subviews of your app.

Managing Your App's Data

SwiftUI has two guiding principles for managing how data flows through your app:

- **Data access = dependency**: Reading a piece of data in your view creates a *dependency* for that data in that view. Every view is a function of its data dependencies — its inputs or state.
- **Single source of truth**: Every piece of data that a view reads has a source of truth, which is either *owned* by the view or *external* to the view. Regardless of where the source of truth lies, you should always have a *single* source of truth.

Tools for Data Flow

SwiftUI provides several tools to help you manage the flow of data in your app. The SwiftUI framework takes care of creating views when they should appear and updating them whenever there's a change to data they depend on.

Property wrappers augment the behavior of properties. SwiftUI-specific wrappers like `@State`, `@Binding`, and `@EnvironmentObject` declare a view's dependency on the data represented by the property.

Some of the data flow in HIITFit

Each wrapper indicates a different *source of data*:

- A `@State` property is a source of truth. One view owns it and passes either its value or a reference, known as a binding, to its subviews.
- A `@Binding` property is a reference to a `@State` property owned by another view. It gets its initial value when the other view passes it a binding, using the `$` prefix. Having this reference to the source of truth enables the subview to change the property's value, and this changes the state of any view that depends on this property.
- `@EnvironmentObject` declares dependency on some shared data — data that's visible to all views in a sub-tree of the app. It's a convenient way to pass data indirectly instead of passing data from parent view to child to grandchild, especially if the in-between child view doesn't need it.

You'll learn more about these, and other, property wrappers in Chapter 11, "Managing Data With Property Wrappers".

Using State & Binding Properties

> **Skills you'll learn in this section**: using `@State` and `@Binding` properties; pinning a preview; adding `@Binding` parameters in previews

Here's your first feature: Set up `TabView` to use `tag` values. When a button changes the value of `selectedTab`, `TabView` displays that tab.

➤ Continue with your project from the previous chapter or open the project in this chapter's **starter** folder.

Passing the Binding of a State Property

➤ In **ContentView.swift**, add this property to `ContentView`:

```
@State private var selectedTab = 9
```

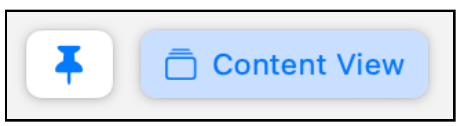
> **Note**: You almost always mark a `State` property `private`, to emphasize that it's owned and managed by this view specifically. Only this view's code in this file can access it directly. An exception is when the `App` needs to initialize `ContentView`, so it needs to pass values to its `State` properties. Learn more about access control in *Swift Apprentice*, Chapter 18, "Access Control, Code Organization & Testing" (https://bit.ly/37EUQDk).

Declaring `selectedTab` as a `@State` property in `ContentView` means `ContentView` *owns* this property, which is the *single source of truth* for this value.

Other views will *use* the value of `selectedTab`, and some will *change* this value to make `TabView` display another page. But, you won't declare it as a `@State` property in any other view.

The initial value of `selectedTab` is **9**, which you'll set as the `tag` value of the welcome page.

➤ If the preview is paused, refresh it, then click the **pin** button to *pin* the preview of `ContentView`:



Pin the preview of ContentView.

You'll soon be editing **WelcomeView.swift** and **ExerciseView.swift**. Pinning the preview of `ContentView` means you'll be able to preview the results without needing to go back to **ContentView.swift**.

➤ Next, replace the entire `body` closure of `ContentView` with the following code:

```
var body: some View {
  TabView(selection: $selectedTab) {
    WelcomeView(selectedTab: $selectedTab)  // 1
      .tag(9)  // 2
    ForEach(Exercise.exercises.indices, id: \.self) { index in
      ExerciseView(selectedTab: $selectedTab, index: index)
        .tag(index)  // 3
    }
  }
  .tabViewStyle(PageTabViewStyle(indexDisplayMode: .never))
}
```

1. You pass the *binding* `$selectedTab` to `WelcomeView` and `ExerciseView` so `TabView` can respond when they change its value. Xcode complains you're passing an extra argument because you haven't yet added a `selectedTab` property to `WelcomeView` or `ExerciseView`. You'll do that soon.
2. You use **9** for the tag of `WelcomeView`.
3. You tag each `ExerciseView` with its index in `Exercise.exercises`.

Adding a Binding Property to a View

➤ Now, in **ExerciseView.swift**, add this property to `ExerciseView`, **above** `let index: Int`:

```
@Binding var selectedTab: Int
```

You'll soon write code to make `ExerciseView` change the value of `selectedTab`, so it can't be a plain old `var selectedTab`. Views are structures, which means you can't change a property value unless you mark it with a property wrapper like `@State` or `@Binding`.

`ContentView` owns the *source of truth* for `selectedTab`. You don't declare `@State private var selectedTab` here in `ExerciseView` because that would create a *duplicate* source of truth, which you'd have to keep in sync with the `selectedTab` value in `ContentView`. Instead, you declare `@Binding var selectedTab` — a *reference* to the `@State` variable owned by `ContentView`.

➤ You need to update `previews` because it creates an `ExerciseView` instance. Add this new parameter like this:

```
ExerciseView(selectedTab: .constant(1), index: 1)
```

You just want the preview to show the second exercise, but you can't pass `1` as the `selectedTab` value. You must pass a `Binding`, which is tricky in a standalone situation like this, where you don't have a `@State` property to bind to. Fortunately, SwiftUI provides the `Binding` type method `constant(_:)` to create a `Binding` from a constant value.

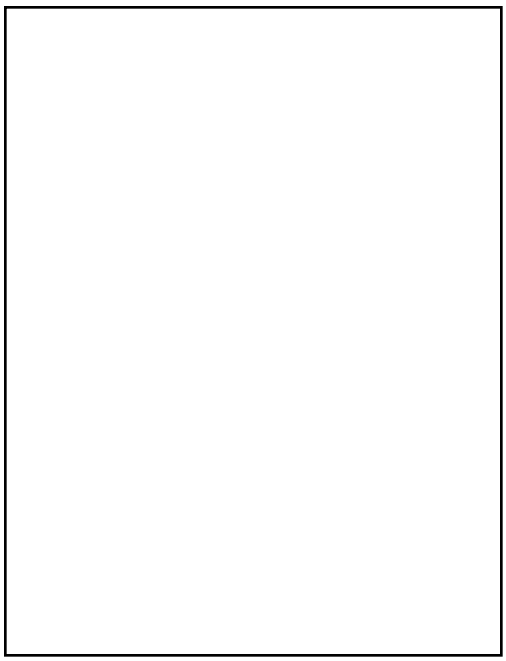
➤ Now, add the same property to `WelcomeView` in **WelcomeView.swift**:

```
@Binding var selectedTab: Int
```

➤ And add this parameter in its `previews`:

```
WelcomeView(selectedTab: .constant(9))
```

Now that you've fixed the errors, you can preview `ContentView` while you're still in **WelcomeView.swift**:



Previewing pinned ContentView in WelcomeView.swift

Changing a Binding Property

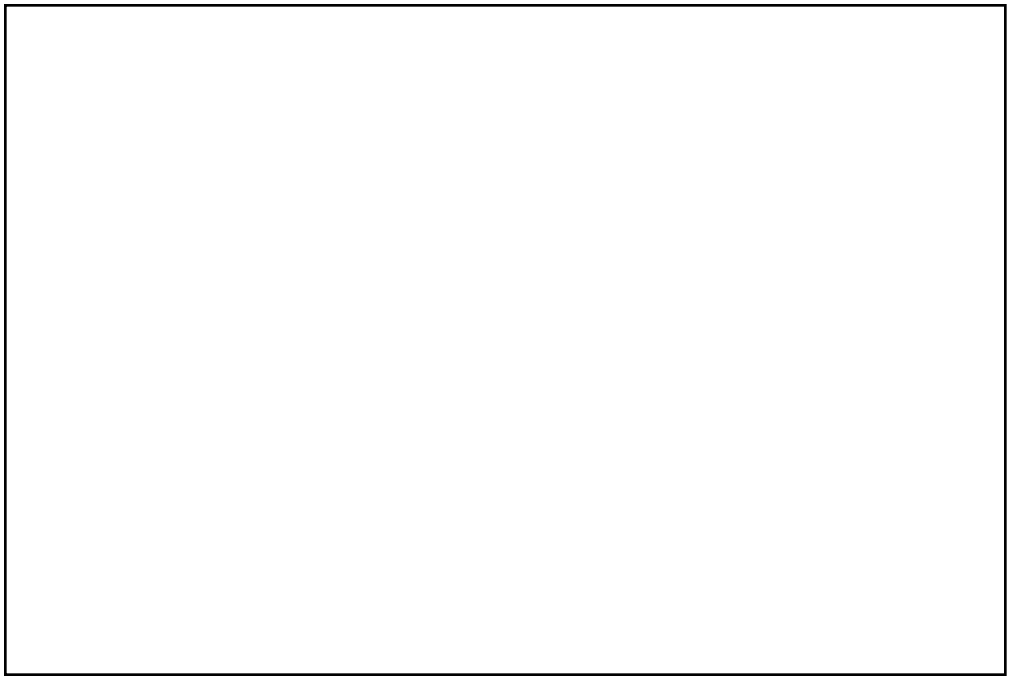
Next, you'll implement the **Welcome** page **Get Started** button action to display the first `ExerciseView`.

➤ In **WelcomeView.swift**, replace `Button(action: { }) {` with this:

```
Button(action: { selectedTab = 0 }) {
```

You've used `selectedTab` to navigate from the welcome page to the first exercise!

➤ Now, in the `ContentView` **Live Preview**, tap **Get Started**.



Tap Get Started to show first exercise.

> **Note**: You can't preview this action in the `WelcomeView` preview because it doesn't include `ExerciseView`. Tapping **Get Started** doesn't go anywhere.

Next, you'll work even more magic in **ExerciseView.swift**.

Using the Ternary Conditional Operator

Your users will be exerting a lot of physical energy to perform the exercises. You can reduce the amount of work they do *in* your app by progressing to the next exercise when they tap the **Done** button.

➤ First, simplify your life by separating the **Start** and **Done** buttons in `ExerciseView`. In **ExerciseView.swift**, replace `Button("Start/Done") { }` with this `HStack`:

```
HStack(spacing: 150) {
  Button("Start Exercise") { }
  Button("Done") { }
}
```

Keep the `font` and `padding` modifiers on the `HStack`, so both buttons use `title3` font size, and the padding surrounds the `HStack`.

➤ Fix the indentation: Select the `font` and `padding` modifiers, then press **Control-I**.

Now you're ready to implement your time-saving action for the **Done** button: Tapping **Done** goes to the next `ExerciseView`, and tapping **Done** in the last `ExerciseView` goes to `WelcomeView`.

➤ First, add this to the other properties in `ExerciseView`:

```
var lastExercise: Bool {
  index + 1 == Exercise.exercises.count
}
```

You created a computed property to check whether this is the last exercise.

➤ Now, back to `Button("Done") { }` and replace it with the following code:

```
Button("Done") {
  selectedTab = lastExercise ? 9 : selectedTab + 1
}
```

> **Swift Tip**: The *ternary conditional operator* tests the condition specified before `?`, then evaluates the first expression after `?` if the condition is true. Otherwise, it evaluates the expression after `:`.

Later in this chapter, you'll show `SuccessView` when the user taps **Done** on the last `ExerciseView`. Then dismissing `SuccessView` will progress to `WelcomeView`.

Computed Properties for Buttons

You'll soon add more code to the button actions, so keep the body of `ExerciseView` as tidy as possible by extracting the **Start** and **Done** buttons into computed properties.

➤ Add these properties to `ExerciseView`:

```
var startButton: some View {
  Button("Start Exercise") { }
}

var doneButton: some View {
  Button("Done") {
    selectedTab = lastExercise ? 9 : selectedTab + 1
  }
}
```

➤ In `body`, replace the two Button views in the HStack with your new properties:

```
HStack(spacing: 150) {
  startButton
  doneButton
}
```

➤ Now, in the `ContentView` **Live Preview**, tap **Get Started** to load the first exercise. Tap **Done** on each exercise page to progress to the next. Tap **Done** on the last exercise to return to the welcome page.



Tap your way through the pages.

Next-page navigation is great, but your users might want to jump directly to their favorite exercise. You'll implement this soon.

Setting & Tapping Images

Skills you'll learn in this section: passing a value vs. passing a `Binding`; making `Image` tappable

Using ?: to Set an Image

Users expect the page numbers in `HeaderView` to indicate the current page. A convenient indicator is the *fill* version of the symbol. In light mode, it's a white number on a black background.



Light mode 2.circle and 2.circle.fill

➤ In **HeaderView.swift**, replace the contents of `HeaderView` with the following code:

```
@Binding var selectedTab: Int  // 1
let titleText: String

var body: some View {
  VStack {
    Text(titleText)
      .font(.largeTitle)
    HStack {  // 2
      ForEach(Exercise.exercises.indices, id: \.self) { index in
// 3
        let fill = index == selectedTab ? ".fill" : ""
        Image(systemName: "\(index + 1).circle\(fill)")  // 4
      }
    }
    .font(.title2)
  }
}
```

1. `HeaderView` doesn't change the value of `selectedTab`, but it needs to redraw itself when other views change this value. You create this dependency by declaring `selectedTab` as a `@Binding`.
2. The **Welcome** page doesn't really need a page "number", so you delete the `"hand.wave"` symbol from the `HStack`.\

3. To accommodate any number of exercises, you create the `HStack` by looping over the `exercises` array, just like in `ContentView`.
4. You create each symbol's name by joining together a `String` representing the integer `index + 1`, the text `".circle"` and either `".fill"` or the empty `String`, depending on whether `index` matches `selectedTab`. You use a ternary conditional expression to choose between `".fill"` and `""`.

➤ Now `previews` needs this new parameter, so replace it with the following:

```
HeaderView(selectedTab: .constant(0), titleText: "Squat")
  .previewLayout(.sizeThatFits)
```

Next, you need to update the instantiations of `HeaderView` in `WelcomeView` and `ExerciseView`.

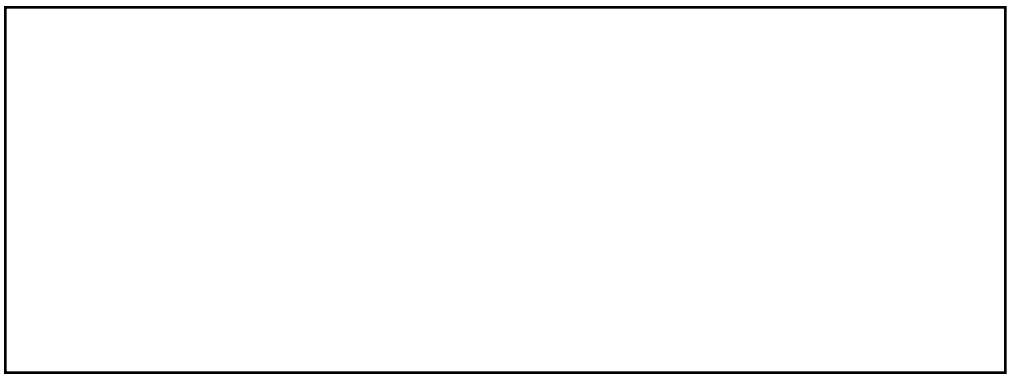
➤ In **WelcomeView.swift**, change `HeaderView(titleText: "Welcome")` to the following:

```
HeaderView(selectedTab: $selectedTab, titleText: "Welcome")
```

➤ In **ExerciseView.swift**, change `HeaderView(titleText: Exercise.exercises[index].exerciseName)` to the following:

```
HeaderView(
  selectedTab: $selectedTab,
  titleText: Exercise.exercises[index].exerciseName)
```

➤ In the `ContentView` **Live Preview**, tap **Get Started** to load the first exercise. The **1** symbol is filled. Tap **Done** on each exercise page to progress to the next and see the symbol for each page highlight.



ExerciseView with page numbers

Using onTapGesture

Making Page Numbers Tappable

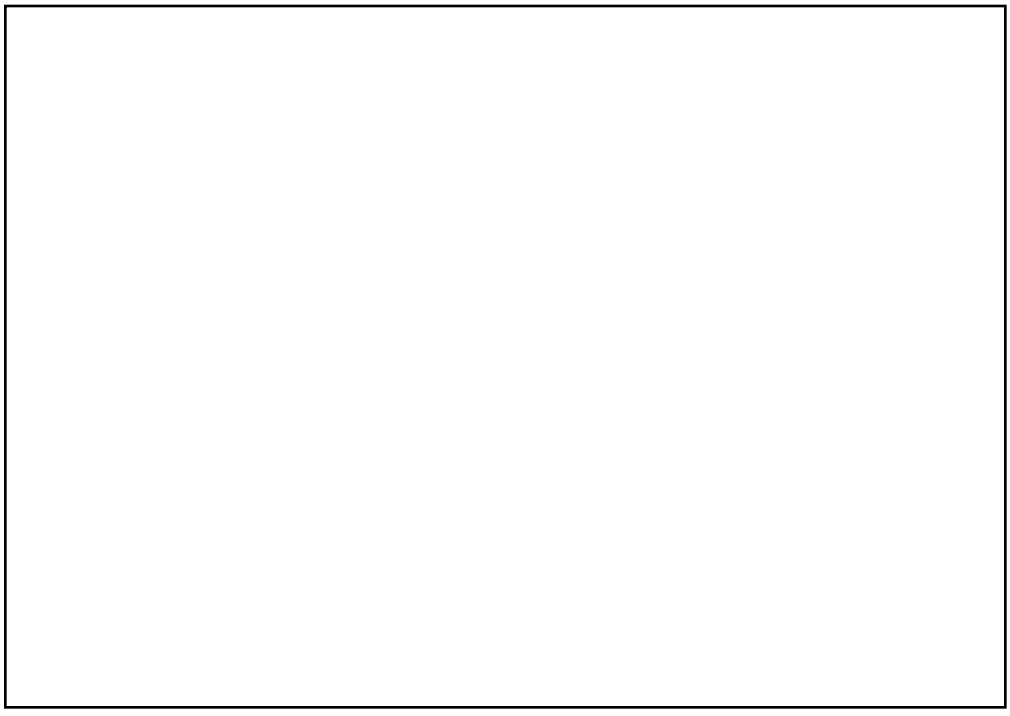
Many users expect page numbers to respond to tapping by going to that page.

➤ In **HeaderView.swift**, add this modifier to `Image(systemName:)`:

```
.onTapGesture {
  selectedTab = index
}
```

This modifier reacts to the user tapping the `Image` by setting the value of `selectedTab`.

➤ In the `ContentView` **Live Preview**, tap a page number to navigate to that exercise page:



Tap page number to jump to last exercise.

Congratulations, you've improved your app's user experience out of sight by providing all the navigation features your users expect.

Indicating & Changing the Rating

The `onTapGesture` modifier is also useful for making `RatingView` behave the way everyone expects: Tapping one of the five rating symbols changes the color of that symbol and all those preceding it to red. The remaining symbols are gray.



Rating view: rating = 3

➤ First, add a `rating` property to `ExerciseView`. In **ExerciseView.swift**, add this to the other properties:

```
@State private var rating = 0
```

In Chapter 7, "Saving Settings", you'll save the `rating` value along with the `exerciseName`, so `ExerciseView` needs this `rating` property. You use the property wrapper `@State` because `rating` must be able to change, and `ExerciseView` owns this property.

➤ Now scroll down to `RatingView()` and replace it with this line:

```
RatingView(rating: $rating)
```

You pass a binding to `rating` to `RatingView` because that's where the actual value change will happen.

➤ In **RatingView.swift**, in `RatingView_Previews`, replace `RatingView()` with this line:

```
RatingView(rating: .constant(3))
```

➤ Now replace the contents of `RatingView` with the following code:

```
@Binding var rating: Int  // 1
let maximumRating = 5  // 2

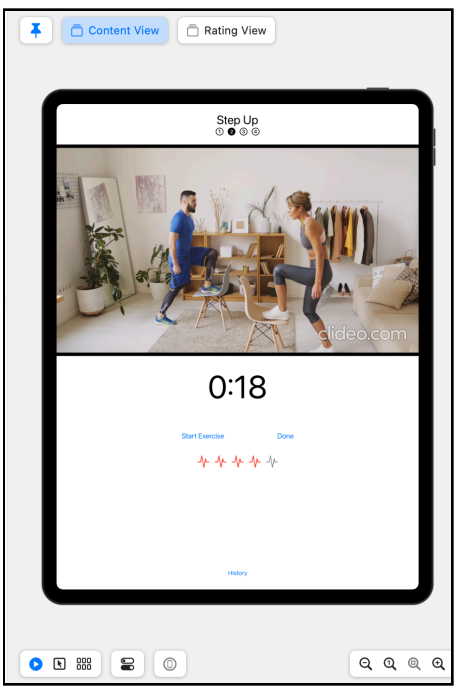
let onColor = Color.red  // 3
let offColor = Color.gray

var body: some View {
  HStack {
    ForEach(1 ..< maximumRating + 1, id: \.self) { index in
      Image(systemName: "waveform.path.ecg")
        .foregroundColor(
          index > rating ? offColor : onColor)  // 4
        .onTapGesture {  // 5
          rating = index
```

```
            }
        }
    }
    .font(.largeTitle)
}
```

1. `ExerciseView` passes to `RatingView` a binding to its `@State` property `rating`.
2. Most apps use a 5-level rating system, but you can set a different value for `maximumRating`.
3. When `rating` is an integer between 1 and `maximumRating`, the first `rating` symbols should be the `onColor`, and the remaining symbols should be the `offColor`.
4. In the `HStack`, you still loop over the symbols, but now you set the symbol's `foregroundColor` to `offColor` if its `index` is higher than `rating`.
5. When the user taps a symbol, you set `rating` to that `index`.

➤ In the `ContentView` **Live Preview**, tap a page number to navigate to that exercise page. Tap different symbols to see the colors change:



Rating view

➤ Navigate to other exercise pages and set their ratings, then navigate through the pages to see the ratings are still the values you set.

➤ Click the **pin** button to unpin the `ContentView` preview.

Showing & Hiding Modal Sheets

> **Skills you'll learn in this section**: more practice with `@State` and `@Binding`; using a Boolean flag to show a modal sheet; dismissing a modal sheet by toggling the Boolean flag or by using `@Environment(\.dismiss)`

`HistoryView` and `SuccessView` are modal sheets that slide up over `WelcomeView` or `ExerciseView`. You dismiss the modal sheet by tapping its **circled-x** or **Continue** button, or by dragging it down.

Showing a Modal With a Binding

One way to show or hide a modal sheet is with a Boolean flag.

➤ In **WelcomeView.swift**, add this `State` property to `WelcomeView`:

```
@State private var showHistory = false
```

When this view loads, it doesn't show `HistoryView`.

➤ Replace `Button("History") { }` with the following:

```
Button("History") {
  showHistory.toggle()
}
.sheet(isPresented: $showHistory) {
  HistoryView(showHistory: $showHistory)
}
```

Tapping the **History** button toggles the value of `showHistory` from `false` to `true`. This causes the `sheet` modifier to present `HistoryView`. You pass a binding `$showHistory` to `HistoryView` so it can change this value back to `false` when the user dismisses `HistoryView`.

➤ You'll edit `HistoryView` to do this soon. But first, repeat the steps above in **ExerciseView.swift**.

Hiding a Modal With a Binding

There are actually two ways to dismiss a modal sheet. This way is the easiest to understand. You set a flag to `true` to show the sheet, so you set the flag to `false` to hide it.

➤ In **HistoryView.swift**, add this property:

```
@Binding var showHistory: Bool
```

This matches the argument you passed to `HistoryView` from `WelcomeView`.

➤ Add this new parameter in `previews`:

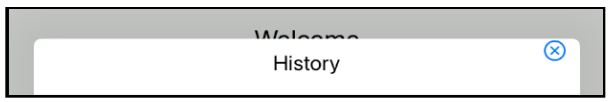
```
HistoryView(showHistory: .constant(true))
```

➤ Now replace `Button(action: {}) {` with the following:

```
Button(action: { showHistory.toggle() }) {
```

You toggle `showHistory` back to `false`, so `HistoryView` goes away.

➤ Go back to **WelcomeView.swift** and, in **Live Preview**, tap **History**:



History dismiss button position problem

`HistoryView` slides up over `WelcomeView`, as it should but, when `HistoryView` displays as a modal sheet, its dismiss button is too close to the top edge.

➤ Pin the `WelcomeView` preview, then go to `HistoryView` and fix the button padding:

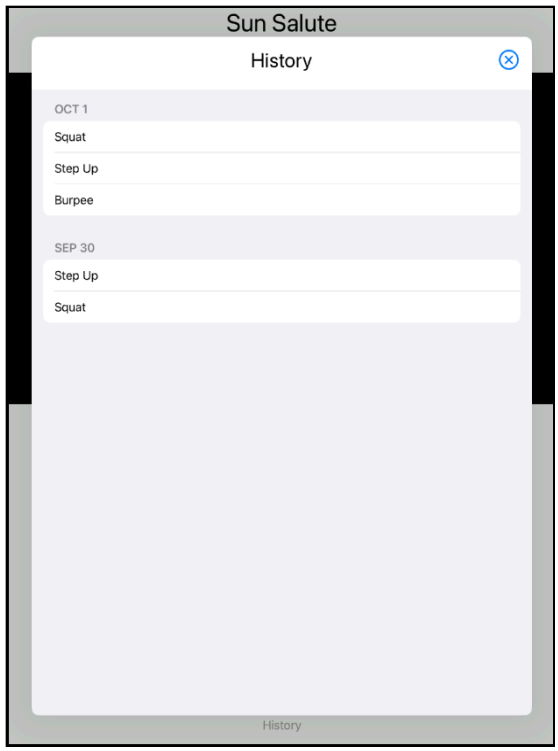
```
Button(action: { showHistory.toggle() }) {
  Image(systemName: "xmark.circle")
}
.font(.title)
.padding()  // delete .trailing
```



History dismiss button position fixed

➤ That's better! Now, tap the dismiss button to hide it. You can also drag down on `HistoryView`. Unpin the `WelcomeView` preview.

➤ Also check the **History** button in **ExerciseView.swift**:



Testing ExerciseView History button

Your app has another modal sheet to show and hide. You'll show it the same way as `HistoryView`, but you'll use a different way to hide it.

Showing a Modal Without a Binding

In **ExerciseView.swift**, you'll modify the action of the **Done** button so when the user taps it on the last exercise, it displays `SuccessView`.

➤ First, add the `@State` property:

```
@State private var showSuccess = false
```

➤ Then, in the `doneButton` closure, replace the **Done** button action with an `if-else` statement:

```
Button("Done") {
  if lastExercise {
    showSuccess.toggle()
  } else {
    selectedTab += 1
  }
}
```

➤ Finally, in the `HStack`, add this modifier to `doneButton`:

```
.sheet(isPresented: $showSuccess) {
  SuccessView()
}
```

Notice you **don't** pass `$showSuccess` to `SuccessView()`. You're going to use a different way to dismiss `SuccessView`. And the first difference is that it won't use the Boolean flag.

Dismissing a Modal Sheet With dismiss

The internal workings of this way are complex, but it simplifies your code because you don't need to pass a parameter to the modal sheet. And you can use exactly the same two lines of code in every modal view.

➤ In **SuccessView.swift**, add this property to `SuccessView`:

```
@Environment(\.dismiss) var dismiss
```

`@Environment(\.dismiss)` gives read access to the environment variable referenced by the key path `\.dismiss`.

Every view's environment has properties like `colorScheme`, `locale` and the device's accessibility settings. Many of these are inherited from the app, but a view's `dismiss` is specific to the view. It's the instance for the current `Environment` of the `DismissAction` structure. This structure has a `callAsFunction` method, but you don't call it directly. Instead, you use "syntactic sugar".

➤ In **SuccessView.swift**, replace `Button("Continue") { }` with the following:

```
Button("Continue") {
  dismiss()
}
```

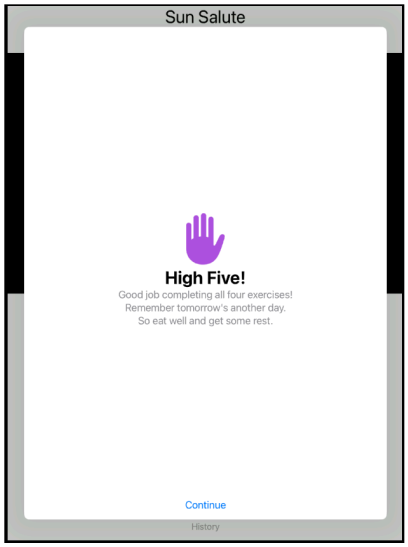
You "call" the `dismiss` structure, and SwiftUI then calls its `callAsFunction` method. This method isn't a toggle. It dismisses the view if it's currently presented. It does nothing if the view isn't currently presented.

➤ Go back to **ExerciseView.swift** and change the line in `previews` to the following:

```
ExerciseView(selectedTab: .constant(3), index: 3)
```

To test showing and hiding `SuccessView`, you'll preview the *last* exercise page.

➤ In **Live Preview**, previewing **Sun Salute**, tap **Done**:



Tap Done on the last exercise to show SuccessView.

There's a lot of empty space above and below the message. You can't change this on an iPad, but you'll soon see how to reduce the height in iOS.

➤ Tap **Continue** to dismiss `SuccessView`.

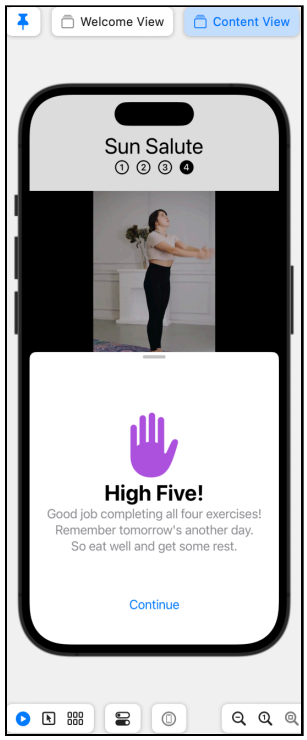
Showing Shorter Modal Sheets

➤ Change the run destination to **iPhone 14 Pro**, then add this modifier to `SuccessView()`:

```
.presentationDetents([.medium, .large])
```

You're telling SwiftUI to support two sheet sizes — `.medium` (about half height) and `.large` (full height). You can also specify `.fraction` or `.height` values.

➤ In **Live Preview**, previewing **Sun Salute**, tap **Done**:



Medium height modal sheet on iPhone

You see the half-height sheet with a resize handle to adjust the sheet to full height.

One More Thing

The **High Five!** message of `SuccessView` gives your user a sense of accomplishment. Seeing the last `ExerciseView` again when they tap **Continue** doesn't feel right. Wouldn't it be better to see the welcome page again?

➤ In **SuccessView.swift**, add this property:

```
@Binding var selectedTab: Int
```

`SuccessView` needs to be able to change this value, so it's a binding.

➤ Also add it in `previews`:

```
SuccessView(selectedTab: .constant(3))
```

➤ And add this line to the **Continue** button action:

```
selectedTab = 9
```

`WelcomeView` has tag value 9.

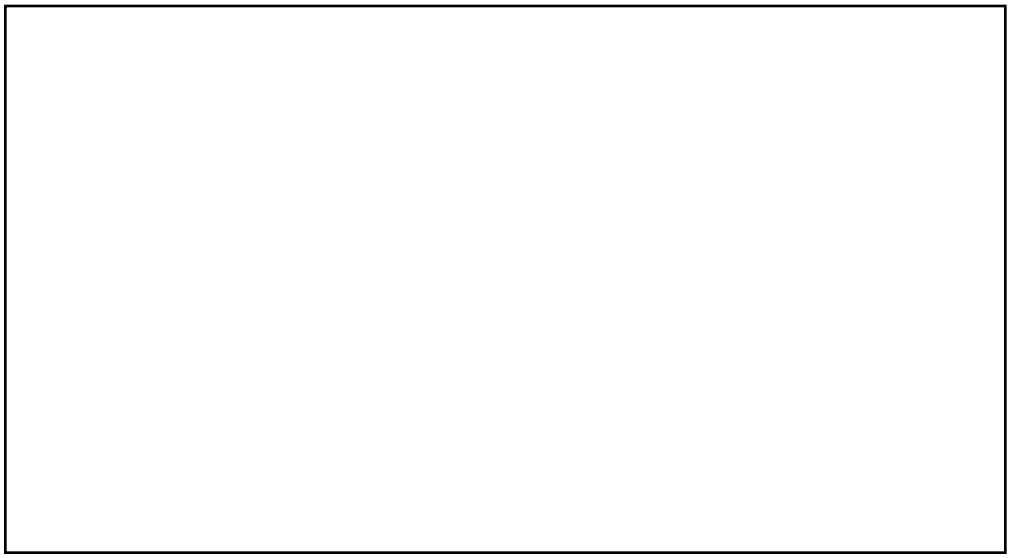
> **Note**: You can add it either above or below the `dismiss` call, but adding it above feels more like the right order of things.

Now back to **ExerciseView.swift** to pass this parameter to `SuccessView`.

➤ Change `SuccessView()` to this line:

```
SuccessView(selectedTab: $selectedTab)
```

➤ And finally, back to **ContentView.swift** to see it work. Run live preview, tap the page 4 button, tap **Done**, then tap **Continue**:



Dismissing SuccessView returns to WelcomeView.

> **Note**: If you don't see the welcome page, press **Command-B** to rebuild the app, then try again.

Tapping **Continue** on `SuccessView` displays `WelcomeView` and dismisses `SuccessView`.

You've used a Boolean flag to show modal sheets. And you've used the Boolean flag and the environment variable `.\dismiss` to dismiss the sheets.

In this chapter, you've used view values to navigate your app's views and show modal sheets. In the next chapter, you'll observe **objects**: You'll use a `TimeLineView` and rework `HistoryStore` as an `ObservableObject`.

Key Points

- Declarative app development means you declare both how you want the views in your UI to look and also what data they depend on. The SwiftUI framework takes care of creating views when they should appear and updating them whenever there's a change to data they depend on.
- Data access = dependency: Reading a piece of data in your view creates a dependency for that data in that view.
- Single source of truth: Every piece of data has a source of truth, internal or external. Regardless of where the source of truth lies, you should always have a *single* source of truth.
- Property wrappers augment the behavior of properties: `@State`, `@Binding` and `@EnvironmentObject` declare a view's dependency on the data represented by the property.
- `@Binding` declares dependency on a `@State` property owned by another view. `@EnvironmentObject` declares dependency on some shared data, like a reference type that conforms to `ObservableObject`.
- Use Boolean `@State` properties to show and hide modal sheets or subviews. Use `@Environment(\.dismiss)` as another way to dismiss a modal sheet.

Chapter 6: Observing Objects

By Audrey Tam

In the previous chapter, you managed the flow of values to implement most of the functionality your users expect when navigating and using your app. In this chapter, you'll manage some of your app's data *objects*. You'll use a `TimeLineView` and give some views access to `HistoryStore` as an `EnvironmentObject`.

Showing/Hiding the Timer

Skills you'll learn in this section: using a `TimeLineView`; showing and hiding a subview

Here's your next feature: In `ExerciseView`, tapping **Start Exercise** shows a countdown timer; the **Done** button is disabled until the timer reaches 0. Tapping **Done** hides the timer. In this section, you'll create a `TimerView`, then use a Boolean flag to show or hide it in `ExerciseView`.

Using TimelineView

Your app currently uses a `Text` view with `style: .timer`. This counts down just fine, but then it counts up and keeps going. You don't have any control over it. You can't stop it. You can't even check when it reaches zero.

Swift has a `TimelineView` container that redraws its content at scheduled times. It comes with some built-in schedule types:

- `everyMinute` schedule updates at the beginning of each minute.
- `periodic(from:by:)` schedule updates periodically with a custom start time and interval between updates.
- `explicit(_:)` schedule updates at a specific set of times.
- `animation(minimumInterval:paused:)` updates at a frequency no more quickly than the provided `interval`. This schedule can be paused, so you'll use this to implement a `TimerView` that redraws once every second as it counts down.

➤ Continue with your project from the previous chapter or open the project in this chapter's **starter** folder.

➤ Create a **new SwiftUI view file** and name it **TimerView.swift**.

➤ Replace the `View` and `PreviewProvider` structures with the following:

```
struct CountdownView: View {
  let date: Date
  @Binding var timeRemaining: Int
  let size: Double

  var body: some View {
```

```
      Text("\(timeRemaining)")  // 5
        .font(.system(size: size, design: .rounded))
        .padding()
        .onChange(of: date) { _ in  // 6
          timeRemaining -= 1
        }
    }
  }

  struct TimerView: View {
    @State private var timeRemaining: Int = 3  // 1
    @Binding var timerDone: Bool  // 2
    let size: Double

    var body: some View {
      TimelineView(  // 3
        .animation(
          minimumInterval: 1.0,
          paused: timeRemaining <= 0)) { context in
            CountdownView(  // 4
              date: context.date,
              timeRemaining: $timeRemaining,
              size: size)
          }
          .onChange(of: timeRemaining) { _ in
            if timeRemaining < 1 {
              timerDone = true  // 7
            }
          }
    }
  }

  struct TimerView_Previews: PreviewProvider {
    static var previews: some View {
      TimerView(timerDone: .constant(false), size: 90)
    }
  }
```

Here's what this does:

1. `timeRemaining` is the number of seconds the timer runs for each exercise. Normally, this is 30 seconds. But one of the features you'll implement in this section is disabling the **Done** button until the timer reaches zero. You set `timeRemaining` very small so you won't have to wait 30 seconds when you're testing this feature.

2. You'll set up the **Start Exercise** button in `ExerciseView` to show `TimerView`, passing a binding to the `timerDone` Boolean flag that enables the **Done** button. You'll change the value of `timerDone` when the timer reaches zero, but this value isn't owned by `TimerView` so it has to be a `@Binding` property.

3. You create a `TimelineView` with an `animation(minimumInterval:paused:)` schedule to update `CountdownView` every 1 second.
4. The `Content` closure receives a `TimelineView.Context` that includes the `date` that triggered the update. You send this to `CountdownView` along with a binding to `timeRemaining`.
5. `CountdownView` displays `timeRemaining` in a large rounded system font, surrounded by padding.
6. The `.onChange(of: date)` modifier in `CountdownView` updates `timeRemaining`, which also updates `timeRemaining` in `TimerView`.
7. In `TimerView`, when `timeRemaining` reaches 0, it sets `timerDone` to `true`. This enables the **Done** button in `ExerciseView`.

> **Note**: `onChange(of:perform:)` receives a new value of `date` or `timeRemaining`, but your action doesn't use this, so you acknowledge its existence with `_`.

Showing the Timer

➤ In **ExerciseView.swift**, replace `let interval: TimeInterval = 30` with the following code:

```
@State private var timerDone = false
@State private var showTimer = false
```

You'll pass `$timerDone` to `TimerView`, which will set it to `true` when the timer reaches zero. You'll use this to enable the **Done** button.

And, you'll toggle `showTimer` just like you did with `showHistory` and `showSuccess`.

➤ Next, locate the `Text` view timer:

```
Text(Date().addingTimeInterval(interval), style: .timer)
  .font(.system(size: geometry.size.height * 0.07))
```

There's an error flag on it because you deleted the `interval` property.

➤ Replace this `Text` view and `font` modifier with the following code:

```
if showTimer {
  TimerView(
    timerDone: $timerDone,
    size: geometry.size.height * 0.07
  )
}
```

You include `TimerView` when `showTimer` is `true`, passing it a binding to the `@State` property `timerDone` and the font size.

➤ Then, in `startButton`, replace `Button("Start Exercise") { }` with the following code:

```
Button("Start Exercise") {
  showTimer.toggle()
}
```

This is just like your other buttons that toggle a Boolean to show another view.

Enabling the Done Button & Hiding the Timer

➤ Now, in `doneButton`, add these two lines to the **Done** button action, *above* the if-else:

```
timerDone = false
showTimer.toggle()
```

If the **Done** button is enabled, `timerDone` is now `true`, so you reset it to `false` to disable the **Done** button.

Also, `TimerView` is showing. This means `showTimer` is currently `true`, so you toggle it back to `false`, to hide `TimerView`.

➤ Next, in the `HStack` of start and done buttons, add this modifier to `doneButton`, above the `sheet(isPresented:)` modifier:

```
.disabled(!timerDone)
```

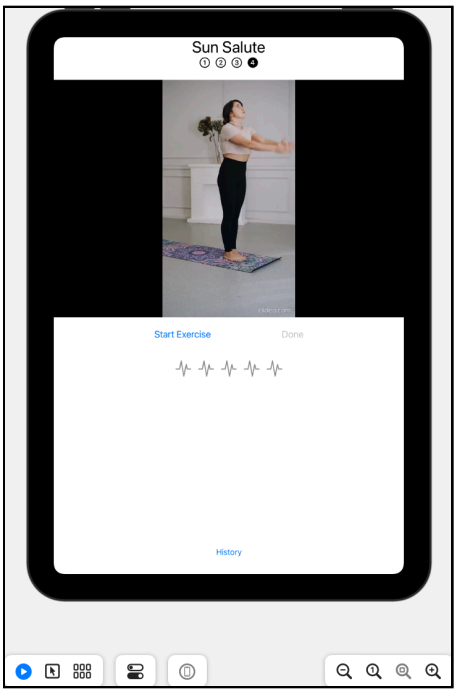
You disable the **Done** button while `timerDone` is `false`.

Testing the Timer & Done Button

➤ Now check that `previews` still shows the last exercise:

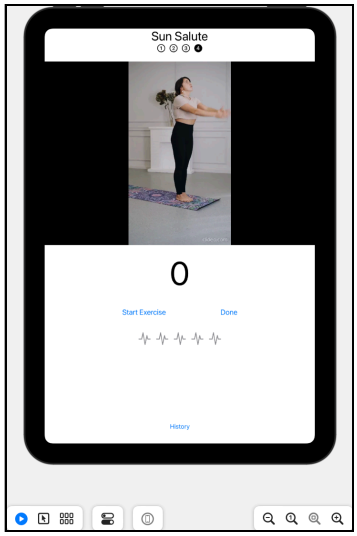
```
ExerciseView(selectedTab: .constant(3), index: 3)
```

This exercise page provides visible feedback. It responds to tapping **Done** by showing `SuccessView`. In **Live Preview**, the **Done** button is currently disabled:



ExerciseView with disabled Done button

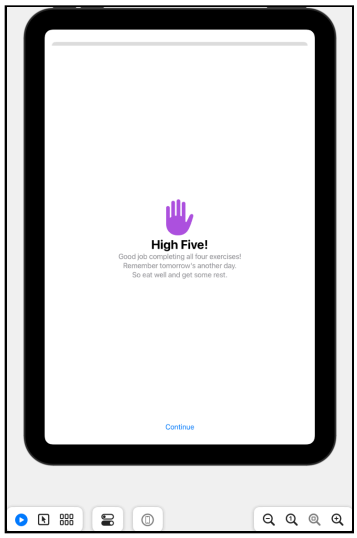
➤ Tap **Start Exercise** and wait while the timer counts down from 3:



ExerciseView with enabled Done button

When the timer reaches 0, the **Done** button is enabled.

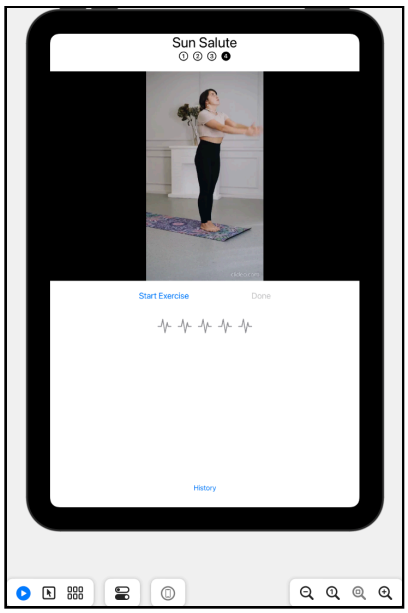
➤ Tap **Done**.



Tap Done to show SuccessView.

This is the last exercise, so `SuccessView` appears.

➤ Tap **Continue**.



ExerciseView with disabled Done button

Because you're previewing `ExerciseView`, not `ContentView`, you return to `ExerciseView`, not `WelcomeView`.

Now the timer is hidden and **Done** is disabled again.

➤ Tap **Start Exercise** to see the timer starts from 3 again.

Tweaking the UI

Tapping **Start Exercise** shows the timer and pushes the buttons and rating symbols down the screen. Tapping **Done** moves them up again. So much movement is probably not desirable, unless you believe it's a suitable "feature" for an exercise app.

To stop the buttons and ratings from doing squats, you'll rearrange the UI elements.

➤ In **ExerciseView.swift**, locate the line `if showTimer {` and the line `Spacer()`. Replace these lines, and everything between them, with the following code:

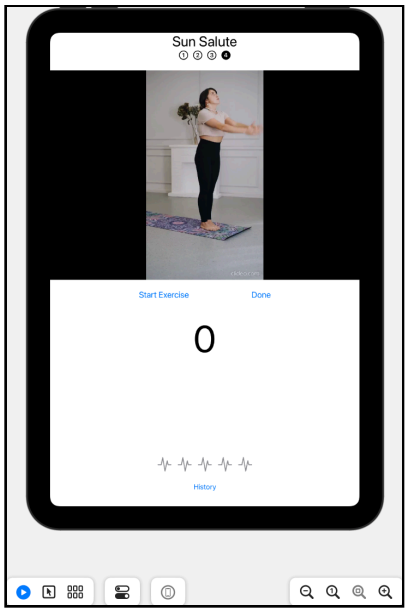
```
HStack(spacing: 150) {
  startButton
  doneButton
```

```
    .disabled(!timerDone)
    .sheet(isPresented: $showSuccess) {
      SuccessView(selectedTab: $selectedTab)
        .presentationDetents([.medium, .large])
    }
  }
  .font(.title3)
  .padding()

  if showTimer {
    TimerView(
      timerDone: $timerDone,
      size: geometry.size.height * 0.07
    )
  }
  Spacer()
  RatingView(rating: $rating) // Move RatingView below Spacer
    .padding()
```

You move the buttons above the timer and `RatingView(rating:)` below `Spacer()`. This leaves a stable space to show and hide the timer.

➤ In **Live Preview**, tap **Start Exercise**, wait for the **Done** button, then tap it. The timer appears then disappears. None of the other UI elements moves.



Show/hide timer without moving other UI elements.

> **Note**: If you preview on a small iPhone, the timer view pushes the History button off the screen. To prevent this, set the spacing of the top level `VStack` to `0`: `VStack(spacing: 0)`.

There's just one last feature to add to your app. It's another job for the **Done** button.

Using an EnvironmentObject

> **Skills you'll learn in this section**: using `@ObservableObject` and `@EnvironmentObject` to let subviews access data; class vs structure

This is the last feature: Tapping **Done** adds this exercise to the user's history for the current day. You'll add the exercise to the `exercises` array of today's `ExerciseDay` object, or you'll create a new `ExerciseDay` object and add the exercise to its array.

Examine your app to see which views need to access `HistoryStore` and what kind of access each view needs:

HistoryStore view tree

- `ContentView` calls `WelcomeView` and `ExerciseView`.
- `WelcomeView` and `ExerciseView` call `HistoryView`.
- `ExerciseView` changes `HistoryStore`, so `HistoryStore` must be either a `@State` or a `@Binding` property in `ExerciseView`.
- `HistoryView` only needs to read `HistoryStore`.
- `WelcomeView` and `ExerciseView` call `HistoryView`, so `WelcomeView` needs read access to `HistoryStore` only so it can pass this to `HistoryView`.

More than one view needs access to `HistoryStore`, so you need a single source of truth. There's more than one way to do this.

The last list item above is the least satisfactory. You'll learn how to manage `HistoryStore` so it doesn't have to pass through `WelcomeView`.

➤ Make a copy of this project **now** and use it to start the challenge at the end of this chapter.

Creating an ObservableObject

To dismiss `SuccessView`, you used its `dismiss` environment property. This is one of the system's predefined environment properties. You can define your own environment object on a view, and it can be accessed by any subview of that view. You don't need to pass it as a parameter. Any subview that needs it simply declares it as a property.

So, if you make `HistoryStore` an `@EnvironmentObject`, you won't have to pass it to `WelcomeView` just so `WelcomeView` can pass it to `HistoryView`.

To be an `@EnvironmentObject`, `HistoryStore` must conform to `ObservableObject`. An `ObservableObject` is a *publisher*.

> **Note**: Publishers are fundamental to Apple's **Combine** concurrency framework. For complete coverage of this framework, check out our book Combine: Asynchronous Programming with Swift (https://bit.ly/3sW1L3I).

And, to conform to `ObservableObject`, `HistoryStore` must be a *class*, not a structure.

> **Swift Tip**: Structures and enumerations are *value types*. If `Person` is a structure, and you create Person object `audrey`, then `audrey2 = audrey` creates a *separate copy* of `audrey`. You can change properties of `audrey2` without affecting `audrey`. Classes are *reference* types. If `Person` is a class, and you create Person object `audrey`, then `audrey2 = audrey` creates a *reference* to the *same* `audrey` object. If you change a property of `audrey2`, you also change that property of `audrey`.

➤ In **HistoryStore.swift**, replace the first two lines of `HistoryStore` with the following:

```
class HistoryStore: ObservableObject {
  @Published var exerciseDays: [ExerciseDay] = []
```

You make `HistoryStore` a class instead of a structure, and you make it conform to the `ObservableObject` protocol.

You mark `exerciseDays` with the `@Published` property wrapper. Whenever `exerciseDays` changes, it *publishes* itself to any *subscribers*, and the system redraws any affected views.

In particular, when `ExerciseView` adds an `ExerciseDay` to `exerciseDays`, `HistoryView` gets updated.

➤ Now, add the following method to `HistoryStore`, below `init()`:

```
func addDoneExercise(_ exerciseName: String) {
  let today = Date()
  if today.isSameDay(as: exerciseDays[0].date) { // 1
    print("Adding \(exerciseName)")
    exerciseDays[0].exercises.append(exerciseName)
  } else {
    exerciseDays.insert( // 2
      ExerciseDay(date: today, exercises: [exerciseName]),
      at: 0)
  }
}
```

You'll call this method in the **Done** button action in `ExerciseView`.

1. The `date` of the first element of `exerciseDays` is the user's most recent exercise day. If `today` is the same as this `date`, you *append* the current `exerciseName` to the `exercises` array of this `exerciseDay`.
2. If `today` is a new day, you create a new `ExerciseDay` object and *insert* it at the beginning of the `exerciseDays` array.

> **Note**: `isSameDay(as:)` is defined in **DateExtension.swift**.

➤ Now to fix the error in **Preview Content/HistoryStoreDevData.swift**, delete `mutating`:

```
func createDevData() {
```

You had to mark this method as `mutating` when `HistoryStore` was a structure. You **must not** use `mutating` for methods defined in a class.

> **Swift Tip**: Structures tend to be constant, so you must mark as `mutating` any method that changes a property. If you mark a method in a *class* as `mutating`, Xcode flags an error. See Chapter 15, "Structures, Classes & Protocols" for further discussion of reference and value types.

Using Your @EnvironmentObject

Now, you need to set up `HistoryStore` as an `@EnvironmentObject` in the parent view of `ExerciseView`. `ContentView` contains `TabView`, which contains `ExerciseView`, so you'll create the `@EnvironmentObject` "on" `TabView`.

➤ In **ContentView.swift**, add this modifier to `TabView(selection:)` above `.tabViewStyle(PageTabViewStyle(indexDisplayMode: .never))`:

```
.environmentObject(HistoryStore())
```

You initialize `HistoryStore` and pass it to `TabView` as an `@EnvironmentObject`. This makes it available to all views in the subview tree of `TabView`, including `HistoryView`.

➤ In **HistoryView.swift**, replace `let history = HistoryStore()` with this property:

```
@EnvironmentObject var history: HistoryStore
```

You don't want to create another `HistoryStore` object here. Instead, `HistoryView` can access `history` directly without needing it passed as a parameter.

➤ Next, add this modifier to `HistoryView(showHistory:)` in `previews`:

```
.environmentObject(HistoryStore())
```

You must tell `previews` about this `EnvironmentObject` or it will crash with the message *Preview is missing environment object "HistoryStore"*.

➤ In **ExerciseView.swift**, add the same property to `ExerciseView`:

```
@EnvironmentObject var history: HistoryStore
```

`ExerciseView` gets read-write access to `HistoryStore` without passing `history` from `ContentView` to `ExerciseView` as a parameter.

➤ Replace `ExerciseView(selectedTab:index:)` in `previews` with the following:

```
ExerciseView(selectedTab: .constant(0), index: 0)
  .environmentObject(HistoryStore())
```

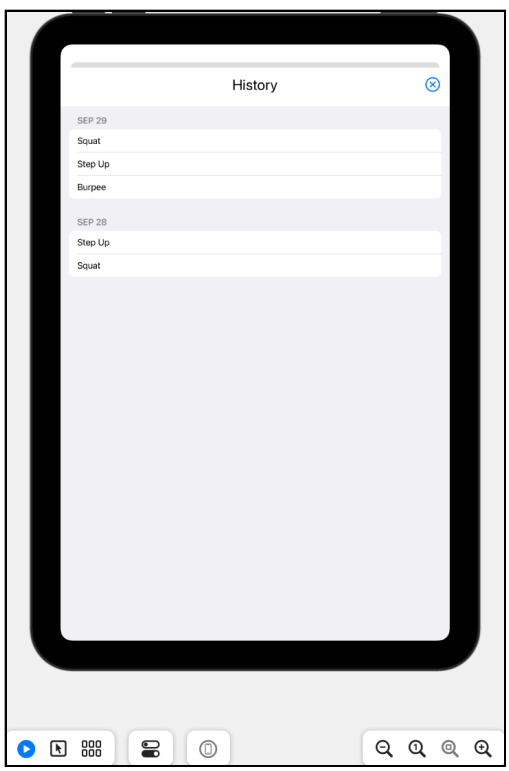
You'll preview the *first* exercise, and you attach `HistoryStore` as an `EnvironmentObject`, just like in **HistoryView.swift**.

➤ Now, in `doneButton`, add this line at the top of the button's action closure:

```
history.addDoneExercise(Exercise.exercises[index].exerciseName)
```

You add this exercise's name to `HistoryStore`.

➤ In **Live Preview**, tap **History** to see what's already there:



History: before

➤ Dismiss `HistoryView`, then tap **Start Exercise**. When **Done** is enabled, tap it. Because you're previewing `ExerciseView`, it won't progress to the next exercise.

➤ Now tap **History** again:



History: after

There's your new `ExerciseDay` with this exercise!

Your app is working pretty well now, with all the expected navigation features. But you still need to save the user's ratings and history so they're still there after quitting and restarting your app. And then, you'll finally get to make your app look pretty.

Challenge

To appreciate how well `@EnvironmentObject` works for this feature, implement it using `State` and `Binding`.

Challenge: Use @State & @Binding to Add Exercise to HistoryStore

- Start from the project copy you made just before you changed `HistoryStore` to an `ObservableObject`. Or open the **starter** project in the **challenge** folder.
- Save time and effort by commenting out `previews` in `WelcomeView`, `ExerciseView` and `HistoryView`. Just pin the preview of `ContentView` so you can inspect your work while editing any view file.
- Initialize `history` in `ContentView` and pass it to `WelcomeView` and `ExerciseView`. Use `State` and bindings where you need to.
- Pass `history` to `HistoryView` from `WelcomeView` and `ExerciseView`. In `HistoryView`, change `let history = HistoryStore()` to `let history: HistoryStore`.
- Add `addDoneExercise(_ exerciseName:)` to `HistoryStore` as a `mutating` method and call it in the action of the **Done** button in `ExerciseView`.

My solution is in the **challenge/final** folder for this chapter.

Key Points

- Implement a countdown timer by creating a `TimelineView` with an `animation(minimumInterval:paused:)` schedule to update `CountdownView` every 1 second.
- `@Binding` declares a dependency on a `@State` property owned by another view. `@EnvironmentObject` declares a dependency on some shared data, such as a reference type that conforms to `ObservableObject`.
- Use an `ObservableObject` as an `@EnvironmentObject` to let subviews access data without having to pass parameters.

Chapter 7: Saving Settings

By Caroline Begbie

Whenever your app closes, all the data entered, such as any ratings you've set or any history you've recorded, is lost. For most apps to be useful, they have to persist data between app sessions. Data persistence is a fancy way of saying "saving data to permanent storage".

In this chapter, you'll explore how to store simple data using `AppStorage` and `SceneStorage`. You'll save the exercise ratings and, if you get called away mid-exercise, your app will remember which exercise you were on and start there, instead of at the welcome screen.

You'll also learn about how to store data in Swift dictionaries and realize that string manipulation is complicated.

Data Persistence

Depending on what type of data you're saving, there are different ways of persisting your data:

- **`UserDefaults`**: Use this for saving user preferences for an app. This would be a good way to save the ratings.
- **Property List file**: A macOS and iOS settings file that stores serialized objects. Serialization means translating objects into a format that can be stored. This would be a good format to store the history data, and you'll do just that in the following chapter.
- **JSON file**: An open standard text file that stores serialized objects. You'll use this format in Section 2.
- **Core Data**: An object graph with a macOS and iOS framework to store objects. For further information, check out our book **Core Data by Tutorials (**https://bit.ly/3fiStNp**)**.

Saving Ratings to UserDefaults

> **Skills you'll learn in this section**: `AppStorage`; `UserDefaults`

`UserDefaults` is a class that enables storing and retrieving data in a **property list** (plist) file held with your app's sandboxed data. It's called "defaults" because you should only use `UserDefaults` for simple app-wide settings. You should never store data such as your history, which will get larger as time goes on.

➤ Open the project in this chapter's **starter** folder. This is the same as the previous chapter's final project.

So far you've used iPad in previews. Remember to test your app just as much using iPhone as well. To test data persistence, you'll need to run the app in Simulator so that you can examine the actual data on disk.

➤ Click the run destination button and select **iPhone 14 Pro**.

AppStorage

`@AppStorage` is a property wrapper, similar to `@State` and `@Binding`, that allows interaction between `UserDefaults` and your SwiftUI views.

You set up a ratings view that allows the user to rate the exercise difficulty from one to five. You'll save this rating to `UserDefaults` so that your ratings don't disappear when you close the app.

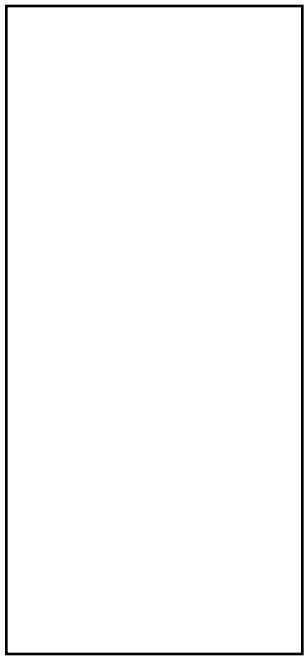
The source of truth for `rating` is currently in **ExerciseView.swift**, where you set up a state property for it.

➤ Open **ExerciseView.swift** and change `@State private var rating = 0` to:

```
@AppStorage("rating") private var rating = 0
```

The property wrapper `@AppStorage` will save any changes to `rating` to `UserDefaults`. Each piece of data you save to `UserDefaults` requires a unique key. With `@AppStorage`, you provide this key as a string in quotes — in this case, **rating**.

➤ Build and run, and choose an exercise. Tap the ratings view to score a rating for the exercise. `UserDefaults` now stores your rating.



Rating the exercise

`AppStorage` only allows a few types: `String`, `Int`, `Double`, `Data`, `Bool` and `URL`. For simple pieces of data, such as user-configurable app settings, storing data to `UserDefaults` with `AppStorage` is incredibly easy.

> **Note**: Even though `UserDefaults` is stored in a fairly secure directory, you shouldn't save sensitive data such as login details and other authentication codes there. For those you should use the keychain. See Apple's article: Storing Keys in the Keychain (https://apple.co/3evbAkA).

➤ Stop the app in Simulator, by swiping up from the bottom. Then, in Xcode, run the app again and go to the same exercise. Your rating persists between launches.

> **Note**: When using `@AppStorage` and `@SceneStorage`, always make sure you exit the app in Simulator or on your device before terminating the app in Xcode. Your app may not save data until the system notifies it of a change in state.

You've solved the data persistence problem, but caused another. Unfortunately, as you only have one **rating** key for all ratings, you are only storing a single value in `UserDefaults`. When you go to another exercise, it has the same rating as the first one. If you set a new rating, all the other exercises have that same rating.

You really need to store an array of ratings, with an entry for each exercise. For example, an array of `[1, 4, 3, 2]` would store individual rating values for exercises 1 to 4. Before fixing this problem, you'll find out how Xcode stores app data.

Data Directories

> **Skills you'll learn in this section**: what's in an app bundle; data directories; property list files; `Dictionary`

When you run your app in Simulator, Xcode creates a **sandboxed** directory containing a standard set of subdirectories. Sandboxing is a security measure so no other app will be able to access your app's files.

Conversely, your app will only be able to read files that iOS allows, and you won't be able to read files from any other app.

App sandbox and directories

The App Bundle

Inside your app sandbox are two sets of directories. First, you'll examine the app bundle and then the user data.

➤ From the Xcode menu, choose **Product ▸ Show Build Folder in Finder**.

A Finder window opens showing a subfolder inside **Xcode/DerivedData**, where Xcode builds all your apps.

> **Note**: If you have build problems, or if you need more disk space, you can clear the **DerivedData** folder.

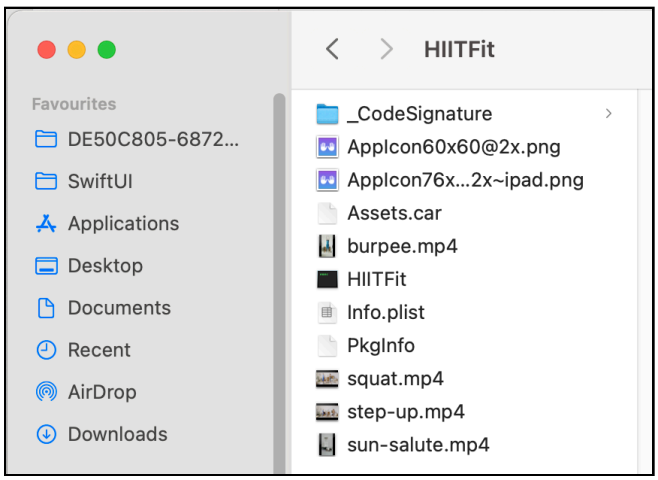
➤ Open the folders **Build ▸ Products ▸ Debug-iphonesimulator**.



App in Finder

When you build your app, Xcode creates a product with the same name as the project.

➤ **Control-click HIITFit** and choose **Show Package Contents** to see the contents of the app bundle.



App bundle contents

The app bundle contains:

- App icons for the current simulator.
- Any app assets not in **Assets.xcassets**. In this app, there are four exercise videos.
- **HIITFit** executable.
- Optimized assets from **Assets.xcassets**, held in **Assets.car**.
- Settings files — Info.plist, PkgInfo etc.

The app bundle is read-only. Once the device loads your app, you can't change the contents of any of these files inside the app. If you have some default data included with your bundle that your user should be able to change, you would need to copy the bundle data to the user data directories when your user runs the app after first installation.

> **Note**: This would be another good use of `UserDefaults`. When you run the app, store a Boolean — or the string version number — to mark that the app has run. You can then check this flag or version number to see whether your app needs to do any internal updates.

You've already used the bundle when loading your video files with `Bundle.main.url(forResource:withExtension:)`. Generally, you won't need to look at the bundle files on disk but, if your app fails to load a bundle file for some reason, it's useful to go to the actual files included in the app and do a sanity check. It's easy to forget to check Target Membership for a file in the File inspector, for example. In that case, the file wouldn't be included in the app bundle.

User Data Directories

The files and directories you'll need to check most often are the ones that your app creates and updates during execution.

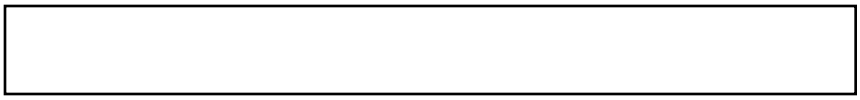
➤ In Xcode, open **HIITFitApp.swift** and add this modifier to `ContentView`:

```
.onAppear {
  print(URL.documentsDirectory)
}
```

You print the URL of the app's Documents directory to the console. Your app will run `onAppear(perform:)` every time the view appears.

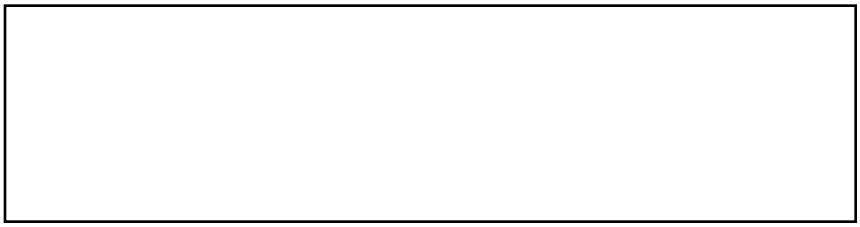
As well as the Documents directory, there are other significant directories, such as Home, Movies, Pictures. You can find these in the `URL`Apple documentation for (https://apple.co/3zylHyL) under **Type Properties**. Remember that your app is sandboxed, and each app will have its own app directories.

➤ Build and run in Simulator. Your app will print its **Documents** directory path in the debug console.



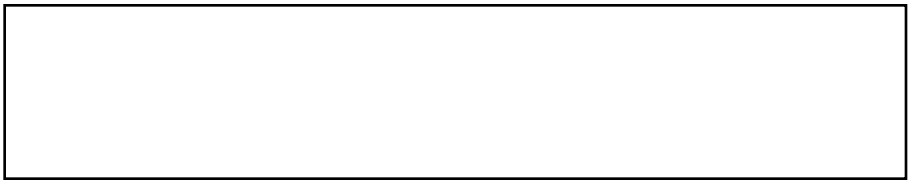
Documents directory path

➤ Highlight from `/Users..` to `/Documents/` and **Control-click** the selection. Choose **Services ▸ Show in Finder**.



Show in Finder

This will open a new **Finder** window showing Simulator's user directories:



Simulator directories

➤ The parent directory — in this example, **47379...F3198** — contains the app's sandboxed user directories. You'll see other directories also named with UUIDs that belong to other apps you may have worked on. Select the parent directory and drag it to your Favorites sidebar, so you have quick access to it.

In your app, you have access to some of these directories:

- **`URL.documentsDirectory`**: **Documents/**. The main documents directory for the app.
- **`URL.libraryDirectory`**: **Library/**. The directory for files that you don't want to expose to the user.
- **`URL.cachesDirectory`**: **Library/Caches/**. Temporary cache files. You might use this if you expand a zipped file and temporarily access the contents in your app.

iPhone and iPad backups will save **Documents** and **Library**, excluding **Library/Caches**.

Inside a Property List File

`@AppStorage` saved your `rating` to a `UserDefaults` property list file. A property list file is an XML file format that stores structured text. All property list files contain a `Root` of type `Dictionary` or `Array`, and this root contains a hierarchical list of keys with values.

For example, instead of distributing HIITFit's exercises in an array, you could store them in a property list file and read them into an array at the start of the app:



Exercises in a property list file

The advantage of this is that you could, in a future release, add an in-app exercise purchase and keep track of purchased exercises in the property list file.

Xcode formats property lists files in a readable format. This is the text version of the property list file above:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://
www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<array>
  <dict>
    <key>exerciseName</key>
    <string>Squat</string>
    <key>videoName</key>
    <string>squat</string>
  </dict>
  <dict>
    <key>exerciseName</key>
    <string>Sun Salute</string>
    <key>videoName</key>
    <string>sun-salute</string>
  </dict>
</array>
</plist>
```

The root of this file is an array that contains two exercises. Each exercise is of type `Dictionary` with key values for the exercise properties.

Swift Dive: Dictionary

A `Dictionary` is a **hash table** which consists of a hashable key and a value. A hashable key is one that can be transformed into a numeric hash value, which allows fast look-up in a table using the key.

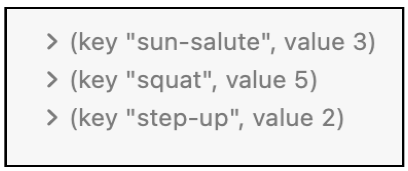
For example, you might create a `Dictionary` that holds ratings for exercises:

```
var ratings = ["burpee": 4]
```

This is a `Dictionary` of type `[String : Integer]`, where `burpee` is the key and `4` is the value.

You can initialize with multiple values and add new values:

```
var ratings = ["step-up": 2, "sun-salute": 3]
ratings["squat"] = 5 // ratings now contains three items
```



Dictionary contents

This last image is from a Swift Playground and shows you that dictionaries, unlike arrays, have no guaranteed sequential order. The order the playground shows is different than the order of creation.

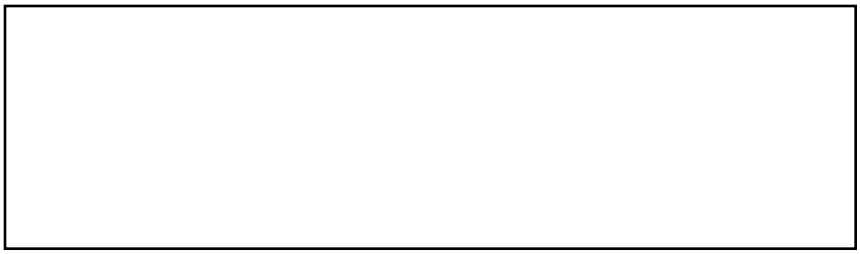
If you haven't used Swift Playgrounds before, they are fun and useful for testing snippets of code. You'll use a playground in Chapter 24, "Downloading Data".

You can retrieve values using the key:

```
let rating = ratings["squat"]  // rating = 5
```

UserDefaults Property List File

➤ In **Finder**, open your app's sandbox and locate **Library/Preferences**. In that directory, open **com.kodeco.HIITFit.plist**. This is the `UserDefaults` file where your ratings are stored. Your app automatically created this file when you first stored `rating`.



UserDefaults property list file

This file has a single entry: `rating` with a `Number` value of 2. This is the rating value that can be from one to five.

Property list files can contain:

- `Dictionary`
- `Array`
- `String`
- `Number`
- `Boolean`
- `Data`
- `Date`

With all these types available, you can see that direct storage to property list files is more flexible than `@AppStorage`, which doesn't support dictionaries or arrays. You could decide that, to store your ratings array, maybe `@AppStorage` isn't the way to go after all. But hold on — all you have to do is a little data manipulation. You could store your integer ratings as an array of characters, also known as a `String`.

You'll initially store the ratings as a string of `"0000"`. When you need, for example, the first exercise, you'll read the first character in the string. When you tap a new rating, you store the new rating back to the first character.

This is extensible. If you add more exercises, you simply have a longer string.

Swift Dive: Strings

> **Skills you'll learn in this section**: Unicode; `String` indexing; nil coalescing operator; `String` character replacement

Strings aren't as simple as they may appear. To support the ever-growing demand for emojis, a string is made up of **extended grapheme clusters**. These are a sequence of Unicode values, shown as a single character, where the platform supports it. They're used for some language characters and also for various emoji tag sequences, for example skin tones and flags.

Unicode tag sequences

The Welsh flag uses seven tag sequences to construct the single character 🏴. On platforms where the tag sequence is not supported, the flag will show as a black flag 🏴.

A `String` is a collection of these characters, very similar to an `Array`. Each element of the `String` is a `Character`, type-aliased as `String.Element`.

Just as with an array, you can iterate through a string using a `for` loop:

```
for character in "Hello World" {
  print(character) // console shows each character on a new line
}
```

Because of the complicated nature of strings, you can't index directly into a `String`. But, you *can* do subscript operations using indices.

```
let text = "Hello World"
let index = text.index(text.startIndex, offsetBy: 6)
let seventh = text[index]
// seventh = "W"
```

`text.index(_:offsetBy:)` returns a `String.Index`. You can then use this special index in square brackets, just as you would with an array.

As you will see shortly, you can also insert a `String` into another `String` at an index, using `String.insert(contentsOf:at:)`, and insert a `Character` into a `String`, using `String.insert(_:at:)`.

> **Note**: You can do so much string manipulation that you'll need Use Your Loaf's Swift String cheat sheet (https://bit.ly/3aGRjWp)

Saving Ratings

Now that you're going to store `ratings`, `RatingView` is a better source of truth than `ExerciseView`. Instead of storing `ratings` in `ExerciseView`, you'll pass the current exercise index to `RatingView`, which can then read and write the rating.

➤ Open **ExerciseView.swift**, and remove:

```
@AppStorage("rating") private var rating = 0
```

➤ Toward the end of `body`, where the compile error shows, change `RatingView(rating: $rating)` to:

```
RatingView(exerciseIndex: index)
```

You pass the current exercise index to the rating view. You'll get a compile error until you fix `RatingView`.

➤ Open **RatingView.swift** and replace `@Binding var rating: Int` with:

```
let exerciseIndex: Int
@AppStorage("ratings") private var ratings = "0000"
@State private var rating = 0
```

Here you hold `rating` locally and set up `ratings` to be a string of four zeros.

Preview holds its own version of `@AppStorage`, which can be hard to clear.

➤ Replace `RatingViewPreviews` with:

```
struct RatingView_Previews: PreviewProvider {
  @AppStorage("ratings") static var ratings: String?
  static var previews: some View {
    ratings = nil
    return RatingView(exerciseIndex: 0)
      .previewLayout(.sizeThatFits)
  }
}
```

To remove a key from the Preview `UserDefaults`, you need to set it to a `nil` value. Only optional types can hold nil, so you define `ratings` as `String?`, with the `?` marking the property as optional. You can then set the `@AppStorage` `ratings` to have a `nil` value, ensuring that your Preview doesn't load previous values. You'll take another look at optionals in the next chapter.

You pass in the exercise index from Preview, so your app will now compile.

Extracting the Rating From a String

➤ In `body`, add a new modifier to `Image`:

```
// 1
.onAppear {
  // 2
  let index = ratings.index(
    ratings.startIndex,
    offsetBy: exerciseIndex)
  // 3
  let character = ratings[index]
  // 4
  rating = character.wholeNumberValue ?? 0
}
```

Swift can be a remarkably succinct language, and there's a lot to unpack in this short piece of code:

1. Your app runs `onAppear(perform:)` every time the view appears.
2. `ratings` is labeled as `@AppStorage` so its value is stored in the `UserDefaults` property list file. You create a `String.Index` to index into the string using `exerciseIndex`.
3. You extract the correct character from the string using the `String.Index`.
4. Convert the character to an integer. If the character is not an integer, the result of `wholeNumberValue` will be an optional value of `nil`. The two question marks are known as the **nil coalescing operator**. If the result of `wholeNumberValue` is `nil`, then use the value after the question marks — in this case, zero. You'll learn more about optionals in the next chapter.

➤ Preview the view. Your stored ratings are currently `0000`, and you're previewing exercise zero.

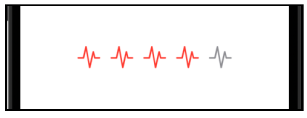


Zero Rating

➤ Change `@AppStorage("ratings") private var ratings = "0000"` to:

```
@AppStorage("ratings") private var ratings = "4000"
```

➤ Resume the preview, and the rating for exercise zero changes to four.



Rating of four

Storing Rating in a String

You're now reading the ratings from `AppStorage`. To store the ratings back to `AppStorage`, you'll index into the string and replace the character at that index.

Add a new method to `RatingView`:

```
func updateRating(index: Int) {
  rating = index
  let index = ratings.index(
    ratings.startIndex,
    offsetBy: exerciseIndex)
  ratings.replaceSubrange(index...index, with: String(rating))
}
```

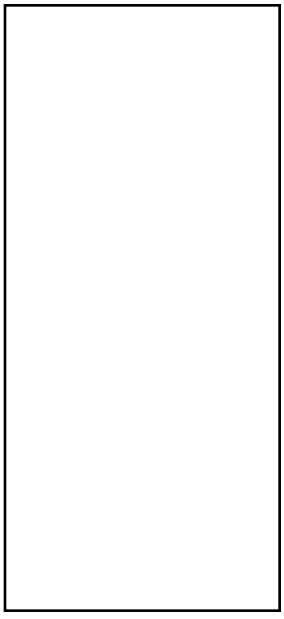
Here you create a `String.Index` using `exerciseIndex`, as you did before. You create a `RangeExpression` with `index...index` and replace the range with the new rating.

> **Note**: You can find more information about `RangeExpressions` in the official Apple documentation (https://apple.co/3qNxD8R).

➤ Replace the `onTapGesture` action `rating = index` with:

```
updateRating(index: index)
```

➤ Build and run and replace all your ratings for all your exercises. Each exercise now has its individual rating.



Rating the exercise

➤ In Finder, examine **com.kodeco.HIITFit.plist** in your app's Library/Preferences directory.



AppStorage

You can remove `rating` from the property list file, as you no longer need it. The ratings stored in the above property list file are:

- Squat: 3
- Step Up: 1
- Burpee: 2
- Sun Salute: 4

Thinking of Possible Errors

> **Skills you'll learn in this section**: custom initializer

You should always be thinking of ways your code can fail. If you try to retrieve an out of range value from an array, your app will crash. It's the same with strings. If you try to access a string index that is out of range, your app is dead in the water. It's a catastrophic error, because there is no way that the user can ever input the correct length string, so your app will keep failing. As you control the `ratings` string, it's unlikely this would occur, but bugs happen, and it's always best to avoid catastrophic errors.

You can ensure that the string has the correct length when initializing `RatingView`.

Custom Initializers

➤ Add a new initializer to `RatingView`:

```
// 1
init(exerciseIndex: Int) {
  self.exerciseIndex = exerciseIndex
```

```
    // 2
    let desiredLength = Exercise.exercises.count
    if ratings.count < desiredLength {
      // 3
      ratings = ratings.padding(
        toLength: desiredLength,
        withPad: "0",
        startingAt: 0)
    }
  }
```

Going through the code:

1. If you don't define `init()` yourself, Xcode creates a default initializer that sets up all the necessary properties. However, if you create a custom initializer, you must initialize them yourself. Here, `exerciseIndex` is a required property, so you receive it as a parameter and store it to the `RatingView` instance.
2. `ratings` must have as many characters as you have exercises.
3. If `ratings` is too short, then you pad out the string with zeros.

To test this, in Simulator, choose **Device ▸ Erase All Content and Settings...** to completely delete the app and clear caches.

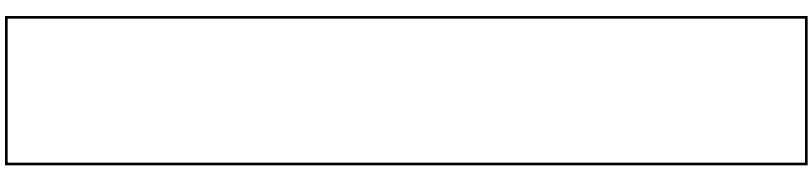
In `RatingView`, change `@AppStorage("ratings") private var ratings = "4000"` to:

```
  @AppStorage("ratings") private var ratings = ""
```

Now when `AppStorage` creates rating's `UserDefaults`, it'll be an empty string.

➤ Build and run and go to an exercise. Then locate your app in Finder. Erasing all contents and settings creates a completely new app sandbox, so open the path printed in the console.

➤ Open **Library ▸ Preferences ▸ com.kodeco.HIITFit.plist**. `ratings` will be padded out with zeros.



Zero padding

Multiple Scenes

Skills you'll learn in this section: multiple iPad windows

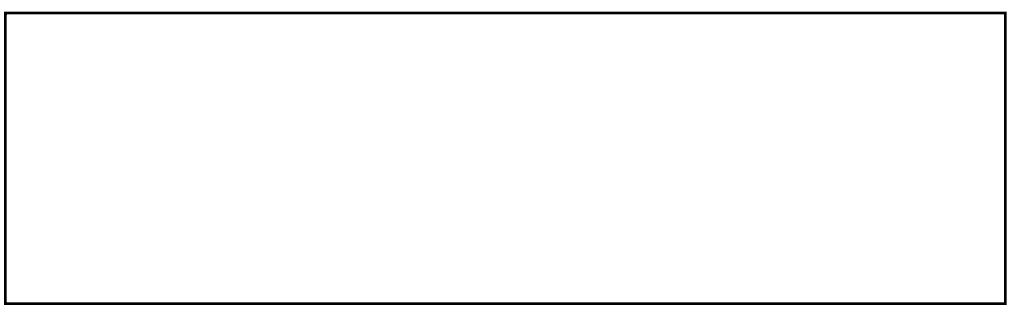
Perhaps your partner, dog or cat would like to exercise at the same time. Or maybe you're just really excited about HIITFit, and you'd like to view two exercises on iPad at the same time. In iPad Split View, you can have a second window open so you can compare your Squat to your Burpee.

First ensure that your app supports multiple windows.

➤ Select the top **HIITFit** group in the Project navigator. Select the **HIITFit** target, then the **Info** tab.

➤ Locate **Application Scene Manifest** in the **Custom iOS Target Properties**, and open the disclosure indicator. **Enable Multiple Windows** has a value of **YES**. When this value is **NO** or not present, you won't be able to have two windows of either your own app, or yours plus another app, side-by-side.

These custom target properties are held in a file **Info.plist**. They configure your app to act in certain ways. You'll add new keys to this file to configure the launch screen and allow writing of photos in Section 2.



Configure multiple windows

➤ Build and run **HIITFit** either on your iPad device or in iPad Simulator.

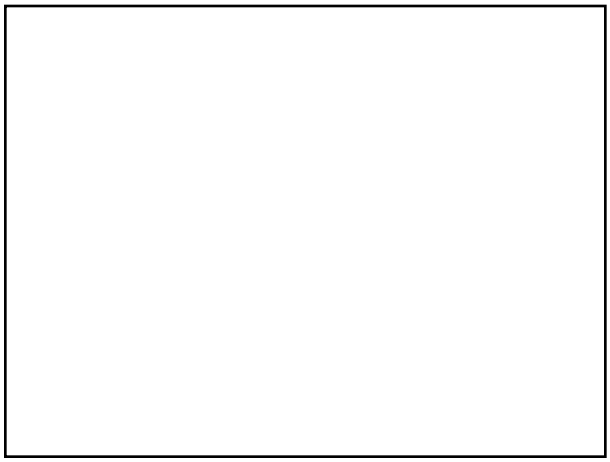
➤ Turn Simulator to landscape orientation using **Command-Right Arrow**. With the app open, tap the three dots at the very top of the screen in Simulator. Choose **Split View** from the menu, and tap the HIITFit icon in the Dock. You can resize the windows by dragging the divider between the windows.

Multiple iPad Windows

You now have two sessions open.

Making Ratings Reactive

➤ On each window, go to Exercise 1 Squat and change the rating. You'll notice there's a problem, as, although the rating is stored in `UserDefaults` using `AppStorage`, the windows reflect two different ratings. When you update the rating in one window, the rating should immediately react in the other window.



Non-reactive rating

➤ Open **RatingView.swift** and review the code.

With `AppStorage`, you hold one `ratings` value per app, no matter how many windows are open. You change `ratings` and update `rating` in `onTapGesture(count:perform:)`. The second window holds its own `rating` instance. When you change the rating in one window, the second window should react to this change and update and redraw its rating view.

Outdated rating

If you were showing a view with the `ratings` string from `AppStorage`, not the extracted integer `rating`, `AppStorage` would automatically invalidate the view and redraw it. However, because you're converting the string to an integer, you'll need to perform that code on change of `ratings`.

The code you have in `onAppear(perform:)` does what you need. However, you need it to run whenever `ratings` changes. SwiftUI provides another modifier — `onChange(of:perform:)` — which does exactly that. Instead of duplicating the code, you'll create a new method and call the method twice.

➤ In `onAppear(perform:)`, highlight:

```
let index = ratings.index(
  ratings.startIndex,
  offsetBy: exerciseIndex)
let character = ratings[index]
rating = character.wholeNumberValue ?? 0
```

➤ **Control-click** the highlighted code and choose **Refactor ▸ Extract to Method**

➤ Name the extracted method `convertRating()`.

Swift tip: Note the `fileprivate` access control modifier in the new method. This modifier allows access to `convertRating()` only inside **RatingView.swift**.

➤ Add a new modifier to `Image`:

```
.onChange(of: ratings) { _ in
  convertRating()
}
```

Here you set up a reactive method that will call `convertRating()` whenever `ratings` changes. If you were using only one window, you wouldn't notice the effect, but multiple windows can now react to the property changing in another window.

➤ Build and run the app with two windows side by side. Return to Exercise 1 in both windows and change the rating in one window. The rating view in the other window immediately redraws when you change the rating.

Apps, Scenes and Views

Skills you'll learn in this section: scenes; `@SceneStorage`

You opened two independent sessions of HIITFit in Simulator. If this app were running on macOS, users would expect to be able to open any number of HIITFit windows. You'll now take a look at how SwiftUI handles multiple windows.

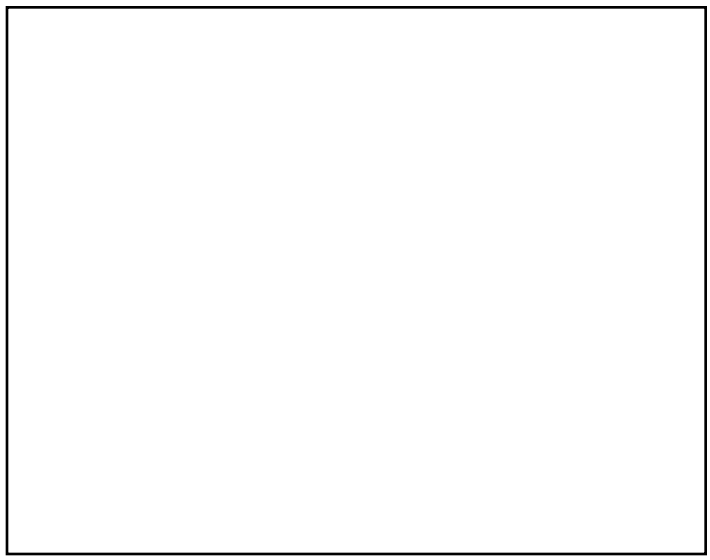
➤ Open **HIITFitApp.swift** and examine the code:

```
@main
struct HIITFitApp: App {
  var body: some Scene {
    WindowGroup {
      ContentView()
      ...
    }
  }
}
```

This simple code controls execution of your app. The `@main` attribute indicates the entry point for the app and expects the structure to conform to the `App` protocol.

`HIITFitApp` defines the basic hierarchy of your app, made up of:

- **HITTFitApp**: Conforms to `App`, which represents the entire app.
- **WindowGroup**: Conforms to `Scene`. A `WindowGroup` presents one or more windows that all contain the same view hierarchy.
- **ContentView**: Everything you see in a SwiftUI app is a `View`. Although the SwiftUI template creates `ContentView`, it's a placeholder name, and you can rename it.



The App Hierarchy

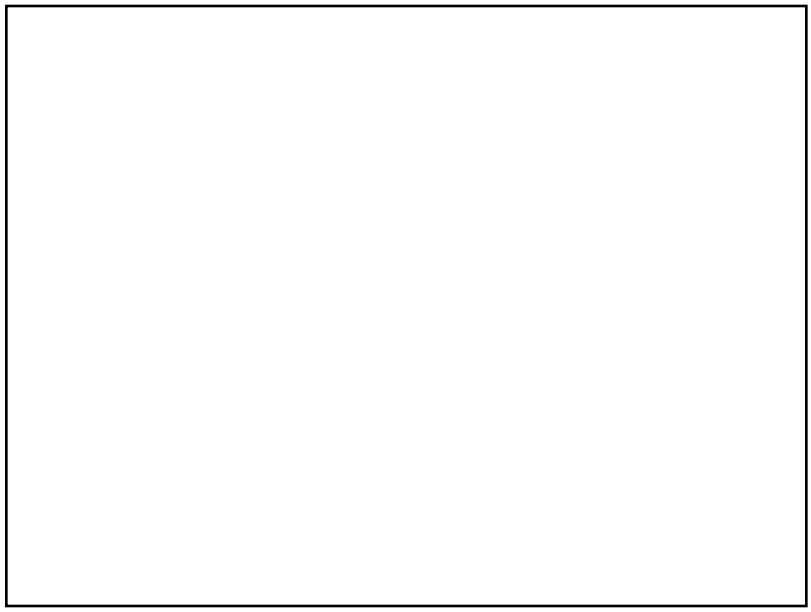
`WindowGroup` behaves differently depending on the platform. On macOS and iPadOS, you can open more than one window or scene, but on iOS, tvOS and watchOS, you can only have the one window.

Restoring Scene State With SceneStorage

Currently, when you exit and restart your app, you always start at the welcome screen. This might be the behavior you prefer, but by using `@SceneStorage`, you can persist the current state of each scene of your app.

➤ Build and run your app in iPad Simulator, in two windows, and go to Exercise 3 in the second window. Exit the app in Simulator by swiping up from the bottom. Then stop the app in Xcode. Remember that your app may not save data unless the device notifies it that state has changed.

➤ Rerun the app and, because the app is completely refreshed with new states, the app doesn't remember that you were doing Exercise 3 in one of the windows.



What exercise was I doing?

As you might guess, `@SceneStorage` is similar to `@AppStorage`. Instead of being persisted per app instance, `@SceneStorage` properties persist per scene.

➤ Open **ContentView.swift**.

The property that controls the current exercise is `selectedTab`.

➤ Change `@State private var selectedTab = 9` to:

```
@SceneStorage("selectedTab") private var selectedTab = 9
```

➤ Build and run the app. In the first window, go to Exercise 1, and in the second window, go to Exercise 3.

➤ Exit the app in Simulator by swiping up from the bottom. Then, stop the app in Xcode.

➤ Build and run again, and this time, the app remembers that you were viewing both Exercise 1 and Exercise 3 and goes straight there.

> **Note**: To reset `SceneStorage` in Simulator, you will have to clear the cache. In Simulator, choose **Device ▸ Erase All Content and Settings...** and then re-run your app.

Although you won't realize this until the next chapter, introducing `SceneStorage` has caused a problem with the way you're initializing `HistoryStore`. Currently you create `HistoryStore` in **ContentView.swift** as an environment object modifier on `TabView`. `SceneStorage` reinitializes `TabView` when it stores `selectedTab`, so each time you change the tab, you reinitialize `HistoryStore`. If you do an exercise your history doesn't save. You'll fix this in the following chapter.

Key Points

- You have several choices of where to store data. You should use `@AppStorage` and `@SceneStorage` for lightweight data, and property lists, JSON or Core Data for main app data that increases over time.
- Your app is sandboxed so that no other app can access its data. You are not able to access the data from any other app either. Your app executable is held in the read-only app bundle directory, with all your app's assets.
- Property lists store serialized objects. If you want to store custom types in a property list file, you must first convert them to a data type recognized by a property list file, such as `String` or `Boolean` or `Data`.
- String manipulation can be quite complex, but Swift provides many supporting methods to extract part of a string or append a string on another string.
- Manage scenes with `@SceneStorage`. Your app holds data per scene. iPads and macOS can have multiple scenes, but an app run on iPhone only has one.

Chapter 8: Saving History Data

By Caroline Begbie

`@AppStorage` is excellent for storing lightweight data such as settings and other app initialization. You can store other app data in property list files, in a database such as SQLite or Realm, or in Core Data. Since you've learned so much about property list files already, you'll save the history data to one in this chapter.

The saving and loading code itself is quite brief, but when dealing with data, you should always be aware that errors might occur. As you would expect, Swift has comprehensive error handling so that, if anything goes wrong, your app can recover gracefully.

In this chapter, you'll learn about error checking techniques as well as saving and loading from a property list file. Specifically, you'll learn about:

- **Optionals**: `nil` values are not allowed in Swift unless you define the property type as Optional.
- **Debugging**: You'll fix a bug by stepping through the code using breakpoints.
- **Error Handling**: You'll throw and catch some errors, which is just as much fun as it sounds. You'll also alert the user when there is a problem.
- **Closures**: These are blocks of code that you can pass as parameters or use as completion handlers.
- **Serialization**: Last but not least, you'll translate your history data into a format that can be stored.

Adding the Completed Exercise to History

➤ Continue with your project from the previous chapter, or open the project in this chapter's **starter** folder.

➤ Open **HistoryStore.swift** and examine `addDoneExercise(_:)`. This is where you save the exercise to `exerciseDays` when your user taps **Done**.

Currently, on initializing `HistoryStore`, you create a fake `exerciseDays` array. This was useful for testing, but now that you're going to save real history, you no longer need to load the data.

➤ In `init()`, comment out `createDevData()`.

➤ Build and run your app. Start an exercise and tap **Done** to save the history. Your app performs `addDoneExercise(_:)` and crashes with **Fatal error: Index out of range**.

Xcode highlights the offending line in your code:

```
if today.isSameDay(as: exerciseDays[0].date) {
```

This line assumes that `exerciseDays` is never empty. If it's empty, then trying to access an array element at index zero is out of range. When users start the app for the first time, their history will always be empty. A better way is to use optional checking.

Using Optionals

Skills you'll learn in this section: optionals; unwrapping; forced unwrapping; filtering the debug console

Swift Dive: Optionals

In the previous chapter, to remove a key from Preview's `UserDefaults`, you needed to assign `nil` to `ratings`.

So you defined `ratings` as an **optional** `String` type by adding `?` to the type:

```
@AppStorage("ratings") static var ratings: String?
```

`ratings` here can either hold a string value or `nil`.

You may have learned that Booleans can be either true or false. But an optional Boolean can hold `nil`, giving you a third alternative.

> **Swift Tip**: `Optional` is actually an enumeration with two cases: `some(Wrapped)` and `none`, where `some` has a generic value of type `Wrapped` and `none` has no value.

Checking for `nil` can be useful to prevent errors. At compile time, Xcode prevents Swift properties from containing `nil` unless you've defined them as optional. At run time, you can check that `exerciseDays` is not empty by checking the value of the optional `first`:

```
if exerciseDays.first != nil {
  if today.isSameDay(as: exerciseDays[0].date) {
    ...
  }
}
```

When `first` is `nil`, the array is empty, but if `first` is not `nil`, then it's safe to access index 0 in the array. This is true, because `exerciseDays` doesn't accept `nil` values. You can have arrays with `nil` values by declaring them like this:

```
var myArray: [ExerciseDay?] = []
```

The more common way of checking for `nil` is to use:

```
if let newProperty = optionalProperty {
  // code executes if optionalProperty is non-nil
}
```

This places a non-optional **unwrapped** result into `newProperty`. Unwrapped here means that `newProperty` is assigned the contents of `optionalProperty` as long as `optionalProperty` is not `nil`.

> **Note**: When the test is a simple one, you can shorten the test and assignment from `if let varName = varName {...}` to `if let varName {...}`.

➤ Change `if today.isSameDay(as: exerciseDays[0].date) {` to:

```
if let firstDate = exerciseDays.first?.date {
```

`if let` tells the compiler that whatever follows could result in `nil`. The property `first?` with the added `?` means that `first` is an optional and can contain `nil`.

If `exerciseDays` is empty, then `first?` will be `nil` and your app won't perform the conditional block, otherwise `firstDate` will contain the unwrapped first element in `exerciseDays`.

Swift Dive: Forced Unwrapping

If you're really sure your data is non-`nil`, then you can use an exclamation mark `!` on an optional. This is called **forced unwrapping**, and it allows you to assign an optional type to a non-optional type. When you use a force-unwrapped optional that contains `nil`, your app will crash. For example:

```
let optionalDay: ExerciseDay? = exerciseDays.first
let forceUnwrappedDay: ExerciseDay = exerciseDays.first!
let errorDay: ExerciseDay = exerciseDays.first
```

- `optionalDay` is of type `ExerciseDay?` and allows `nil` when `exerciseDays` is empty.
- `forceUnwrappedDay` is not optional and could cause a runtime error if `exerciseDays` is empty and you force-unwrap `first`.
- `errorDay` causes a compile error because you are trying to put an optional which could contain `nil` into a property that can't contain `nil`.

Unless you're really certain that the value will never contain `nil`, don't use exclamation marks to force-unwrap it!

Multiple Conditionals

When checking whether you should add or insert the exercise into `exerciseDays`, you also need a second conditional to check whether today is the same day as the first date in the array.

➤ Change `if let firstDate = exerciseDays.first?.date {` to:

```
if let firstDate = exerciseDays.first?.date,
  today.isSameDay(as: firstDate) {
```

You can stack up conditionals, separating them with a comma. Your second conditional evaluates the Boolean condition. If `firstDate` is not `nil`, and `today` is the same day as `firstDate`, then the code block executes.

➤ At the end of `addDoneExercise(_:)`, add:

```
print("History: ", exerciseDays)
```

This will print the contents of `exerciseDays` to the debug console after adding or inserting history.

➤ Build and run, complete an exercise and tap **Done**.

Your app doesn't crash, and your completed exercise prints out in the console.



Completed exercise log

> **Note**: If you have trouble finding your print commands in the console, you can filter on words that you expect to see, such as in this case **History**.

Debugging HistoryStore

> **Skills you'll learn in this section**: breakpoints

Even though the contents of `exerciseDays` appears correct at the end of `addDoneExercise(_:)`, if you tap **History**, your history data is blank. This is a real-life frustrating situation where you're pretty sure you've done everything correctly, but the history data refuses to stay put.

Time to put your debugging hat on.

The first and often most difficult debugging step is to find where the bug occurs and be able to reproduce it consistently. Start from the beginning and proceed patiently. Document what should happen and what actually happens.

➤ Build and run, complete an exercise and tap **Done**. The contents of `exerciseDays` print out correctly in the debug console. Tap **History** and the view is empty, when it should show the contents of `exerciseDays`. This error happens every time, so you can be confident at being able to reproduce it.

Error Reproduction

An Introduction to Breakpoints

When you place breakpoints in your app, Xcode pauses execution and allows you to examine the state of variables and, then, step through code.

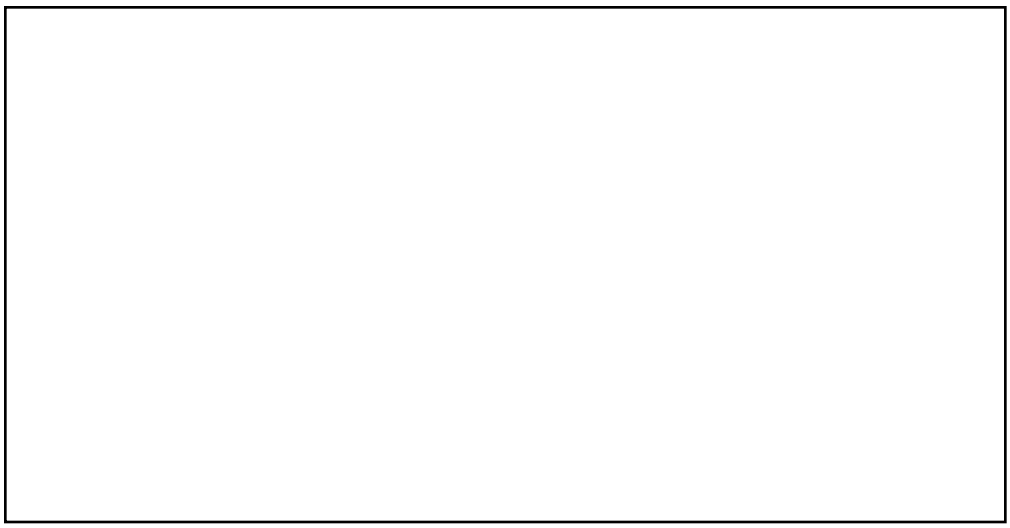
➤ Still running the app, with the first exercise done, in Xcode click the line number to the left of `let today = Date()` in `addDoneExercise(_:)`. This adds a breakpoint at that line.



Breakpoint

➤ Without stopping your app, complete a second exercise and tap **Done.**

When execution reaches `addDoneExercise(_:)`, it finds the breakpoint and pauses. The Debug navigator shows the state of the CPU, memory and current thread operations. The debug console shows a prompt — `(lldb)` — allowing you to interactively debug.



Execution paused

Above the debug console, you have icons to control execution:



Icons to control execution

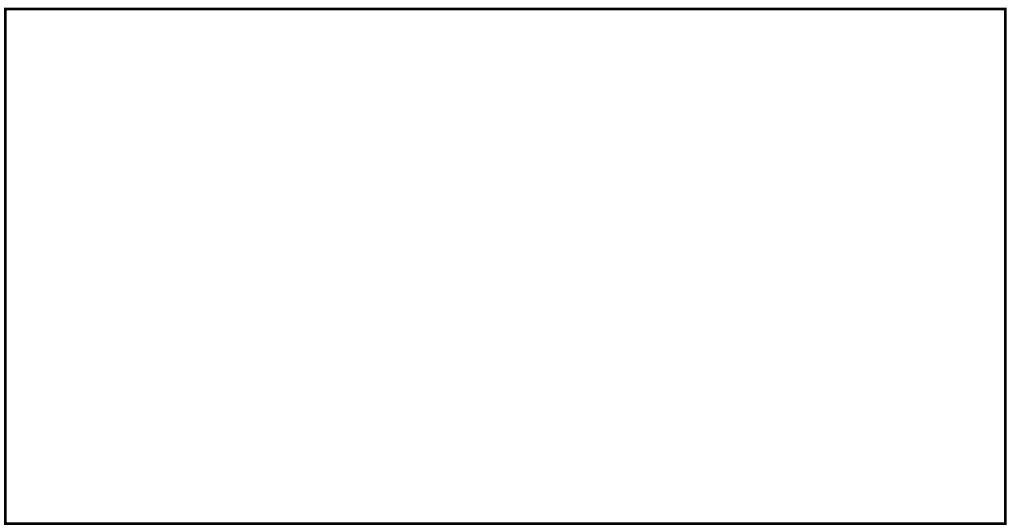
1. **Deactivate breakpoints**: Turns on and off all your breakpoints.
2. **Continue program execution**: Continues executing your app until it reaches another active breakpoint.
3. **Step over**: If the next line to execute includes a method call, stop again *after* that method completes.
4. **Step into/out**: If your code calls a method, you can step into the method and continue stepping through it. If you step over a method, it will still be executed, but execution won't be paused after every instruction.

➤ Click **Step over** to step over to the next instruction. `today` is now instantiated and contains a value.

➤ In the debug console, remove any filters, and at the `(lldb)` prompt, enter:

```
po today
po exerciseDays
```

`po` prints out in the debug console the contents of `today` and `exerciseDays`:



Printing out contents of variables

In this way, you can examine the contents of any variable in the current scope.

Even though `exerciseDays` should have data from the previous exercise, it now contains zero elements. Somewhere between tapping **Done** on two exercises, `exerciseDays` is getting reset.

➤ Step over each instruction and examine the variables to make sure they make sense to you. When you've finished, drag the breakpoint out of the gutter to remove it.

The next step in your debugging operation is to find the source of truth for `exerciseDays` and when that source of truth gets initialized. You don't have to look very far in this case, as `exerciseDays` is owned by `HistoryStore`.

➤ At the end of `init()` add:

```
print("Initializing HistoryStore")
```

➤ Build and run, and reproduce your error by performing an exercise and tapping **Done**. In the debug console, filter on **History**.

Your console should look like this:



Initializing HistoryStore

Now you can see why `exerciseDays` is empty after performing an exercise. Something is reinitializing `HistoryStore`!

➤ Open **ContentView.swift**. This is where you initialize `HistoryStore` in an environment object modifier on `TabView`.

You may remember from the end of the previous chapter that `@SceneStorage` reinitializes `TabView` when it stores `selectedTab`. The redraw re-executes `environmentObject(HistoryStore())` and incorrectly initializes `HistoryStore` with all its data.

You've now successfully debugged why your history data is empty. All you have to do now is decide what to do about it.

This first step to fix this is to move the initialization of `HistoryStore` up a level in the view hierarchy.

➤ Remove `.environmentObject(HistoryStore())` from `ContentView`'s `body`.

@StateObject

`@State`, being so transient, is incompatible with reference objects and, as `HistoryStore` is a class, `@StateObject` is the right choice here. `@StateObject` is a read-only property wrapper. You get one chance to initialize it, and you can't change the property once you set it.

➤ Open **HIITFitApp.swift** and add a new property to `HIITFitApp`:

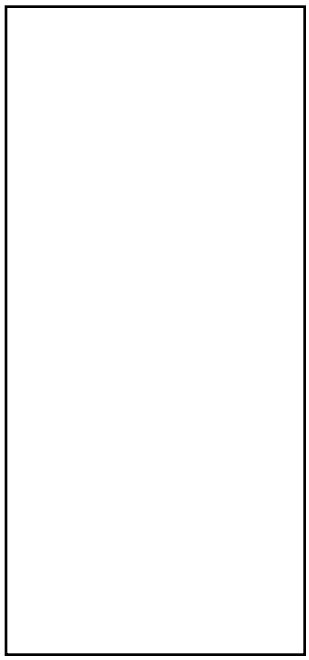
```
@StateObject private var historyStore = HistoryStore()
```

➤ In `body`, add this modifier to `ContentView()`.

```
  .environmentObject(historyStore)
```

You place the store into the environment. In case you're confused about all the property wrappers you've used so far, you'll review them in Chapter 11, "Managing Data With Property Wrappers".

➤ Build and run, perform all four exercises, tapping **Done** after each, and check your history:



Successful history store

Congratulations! You fixed your first bug! You can now remove all your print statements from **HistoryStore.swift** with pride and a sense of achievement.

Now you can continue on and save your history so that it doesn't reset every time you restart your app.

Swift Error Checking

Skills you'll learn in this section: throwing and catching errors

Saving and loading data is serious business, and if any errors occur, you'll need to know about them. There isn't a lot you can do about file system errors, but you can let your users know that there has been an error, and they need to take some action.

➤ Open **HistoryStore.swift** and add a new enumeration to `HistoryStore`:

```
enum FileError: Error {
  case loadFailure
  case saveFailure
}
```

These are the two errors that you'll check for.

To create a method that raises an error, you mark it with `throws` and add a `throw` statement.

➤ Add this new method to `HistoryStore`:

```
func load() throws {
  throw FileError.loadFailure
}
```

Here, you'll read the history data from a file on disk. Currently, this method always raises an error, but you'll come back to it later when you add the loading code. When you throw an error, the method returns immediately and doesn't execute any following code. It's the caller that should handle the error, not the throwing method.

Swift Dive: do...catch

To handle an error from a throwing method, you use the expression:

```
 do {
   try methodThatThrows()
} catch {
  // take action on error
}
```

If you don't need to handle any errors specifically, instead of enclosing the `try` in a `do..catch` statement, you can call the method with `try?`. For example, `try? load()`, converts an error result to `nil` and execution continues.

If you have several errors, you can add a pattern to `catch`. For example:

```
do {
  try load()
} catch FileError.loadFailure {
  // load failed
} catch {
  // any other error
}
```

Call your load method from `HistoryStore`'s initializer.

➤ Add this at the end of `init()`:

```
do {
  try load()
} catch {
  print("Error:", error)
}
```

`load()` throws, so you embed `try` in a `do...catch`. If there's an error, the `catch` block executes.

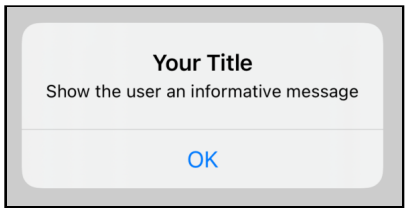
➤ Build and run. Currently `load()` always throws, so in the debug console, you'll see your printed error: **Error: loadFailure**. (Remember to clear the debug console Filter if you don't see the error.)

Alerts

Skills you'll learn in this section: `Alert` view

If loading the history data fails, you could either report a catastrophic error and crash the app or, preferably, you could report an error and continue with no history.

When you release your app, your users won't be able to see `print` statements, so you'll have to provide them with more visible communication. When you want to give the user a choice of actions, you can use an `ActionSheet` but, for simple notifications, an `Alert` is perfect. An `Alert` pops up with a title and a message and pauses app execution until the user taps **OK**.



An alert

➤ Open **HistoryStore.swift** and add a new property to `HistoryStore`:

```
@Published var loadingError = false
```

When `loadingError` is true, you'll show an alert in the view.

➤ In `init()`, replace `print("Error", error)` with:

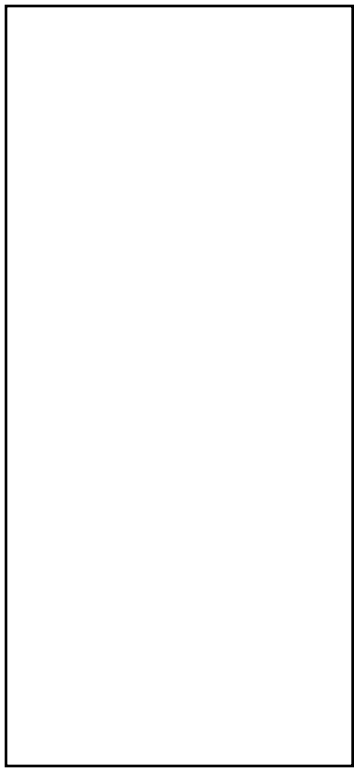
```
loadingError = true
```

➤ Open **HIITFitApp.swift**, and in `body`, add a new modifier to `ContentView`:

```
.alert(isPresented: $historyStore.loadingError) {
  Alert(
    title: Text("History"),
    message: Text(
      """
      Unfortunately we can't load your past history.
      Email support:
        support@xyz.com
      """))
}
```

When `loadingError` is true, you show an `Alert` view with the supplied `Text` title and message. Surround the string with three `"""` to format your string on multiple lines.

➤ Build and run. Instead of seeing the failure message in the console, you see an alert.



History Alert

➤ Tap **OK**. `Alert` resets `historyStore.loadingError` and your app continues with empty history data.

➤ Now that you've tested error checking, open **HistoryStore.swift** and remove `throw FileError.loadFailure` from `load()`.

> **Note**: You can find out more about error handling in our Swift Apprentice book (https://www.kodeco.com/books/swift-apprentice), which has an entire chapter on the subject.

Saving History

Skills you'll learn in this section: closures; `map(_:)`; transforming arrays

You need to have some history data stored in order to load it. So, you're now going to implement saving data and then come back and complete `load()`.

➤ Add a new property to `HistoryStore` to describe the URL:

```
var dataURL: URL {
  URL.documentsDirectory
    .appendingPathComponent("history.plist")
}
```

You add the file name to the documents path. This gives you the full URL of the file to which you'll write the history data.

You'll save the history data to a property list (plist) file. As mentioned in the previous chapter, the root of a property list file can be a dictionary or an array. Dictionaries are useful when you have a number of discrete values that you can reference by key. But in the case of history, you have an array of `ExerciseDay` to store, so your root will be an array.

Property list files can only store a few standard types, and `ExerciseDay`, being a custom type, is not one of them. In Chapter 19, "Saving Files", you'll learn about `Codable` and how to save custom types to files but, for now, the easy way is to separate out each `ExerciseDay` element into an array of `Any` and append this to the array that you will save to disk.

➤ Add a new throwing method to `HistoryStore`:

```
func save() throws {
  var plistData: [[Any]] = []
  for exerciseDay in exerciseDays {
    plistData.append([
      exerciseDay.id.uuidString,
      exerciseDay.date,
      exerciseDay.exercises
    ])
  }
}
```

For each element in the loop, you construct an array with a `String`, a `Date` and a `[String]`. You can't store multiple types in an `Array`, so you create an array of type `[Any]` and append this element to `plistData`.

`plistData` is a type `[[Any]]`. This is a **two dimensional** array, which is an array that contains an array. After saving two elements, `plistData` will look like this:

An array of type [[Any]]

The `for` loop **maps** `exerciseDays` to `plistData`. In other words, the loop transforms one set of data to another set of data. As this happens so often in code, Swift provides `map(_:)`, an optimized method on `Array`, for this transforming of data.

Swift Dive: Closures

`map(_:)` takes a **closure** as a parameter so, before continuing, you'll learn how to use closures. You've already used them many times, as SwiftUI uses them extensively.

A closure is simply a block of code between two curly braces. Closures can look complicated, but if you recognize how to put a closure together, you'll find that you use them often, just as SwiftUI does. Notice a closure's similarity to a function: Functions are closures — blocks of code — with names.

A closure

The closure is the part between the two curly braces `{...}`. In the example above, you assign the closure to a variable `addition`.

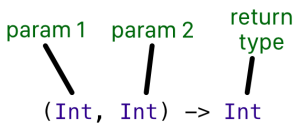
The signature of `addition` is `(Int, Int) -> Int` and declares that you will pass in two integers and return one integer.

It's important to recognize that when you assign a closure to a variable, the closure code doesn't execute. The variable `addition` contains the code `return a + b`, not the actual result.

To perform the closure code, you execute it with its parameters:

Closure result

You pass in 1 and 2 as the two integer parameters and receive back an integer:



Closure signature

Another example:

```
let aClosure: () -> String = { "Hello world" }
```

This closure takes in no parameters and returns a string.

Your current task is to convert each `ExerciseDay` element to an element of type `[Any]`.

This is the closure that would perform this conversion for a single `ExerciseDay` element:

```
let result: (ExerciseDay) -> [Any] = { exerciseDay in
  [
    exerciseDay.id.uuidString,
    exerciseDay.date,
    exerciseDay.exercises
  ]
}
```

`result` is of type `(ExerciseDay) -> [Any]`. The closure takes in a parameter `exerciseDay` and combines the `ExerciseDay` properties into an array of type `[Any]`.

Using map(_:) to Transform Data

Similar to a `for` loop, `map(_:)` goes through each element individually, transforms the data to a new element and then combines them all into a single array.

You could send `result` to `map` which returns an array of the results:

```
let plistData: [[Any]] = exerciseDays.map(result)
```

`map(_:)` takes the closure `result`, executes it for every element in `exerciseDays` and returns an array of the results.

Rather than separating out into a closure variable, it's more common to declare the `map` operation together with the closure.

➤ Replace the contents of `save()` with:

```
let plistData = exerciseDays.map { exerciseDay in
  [
    exerciseDay.id.uuidString,
    exerciseDay.date,
    exerciseDay.exercises
  ]
}
```

The full declaration of `Array.map(_:)` is:

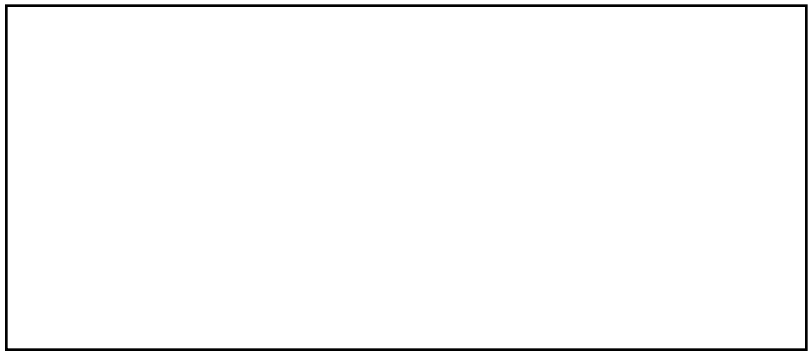
```
func map<T>(
  _ transform: (Self.Element) throws -> T) rethrows -> [T]
```

- If `map(_:)` finds any errors, it will throw.
- `T` is a generic type. You'll discover more about generics in Section 2, but here `T` is equivalent to `[Any]`.
- `transform`'s signature is `(Self.element) -> T`. You'll recognize this as the signature of a closure to which you pass a single element of `ExerciseDay` and return an array of type `[Any]`.

This is how your code matches `map(_:)`:

Deconstructing map(_:)

This code gives exactly the same result as the previous `for` loop. **Option-click** `plistData`, and you'll see that its type is `[[Any]]`, just as before.



Type of plistData

One advantage of using `map(_:)` rather than dynamically appending to an array in a `for` loop, is that you declare `plistData` as a constant with `let`. This is some extra safety, so that you know that you won't accidentally change `plistData` further down the line.

An Alternative Construct

When you have a simple transformation, and you don't need to spell out all the parameters in full, you can use `$0`, `$1`, `$2`, `$...` as replacements for multiple parameter names.

➤ Replace the previous code with:

```
let plistData = exerciseDays.map {
  [$0.id.uuidString, $0.date, $0.exercises]
}
```

Here you have one input parameter, which you can replace with `$0`. When using `$0`, you don't specify the parameter name after the first curly brace `{`.

Again, this code gives exactly the same result. **Option-click** `plistData`, and you'll see that its type is still `[[Any]]`.



Type of plistData

Property List Serialization

Skills you'll learn in this section: property list serialization

Writing Data to a Property List File

You now have your history data in an array with only simple data types that a property list can recognize. The next stage is to convert this array to a byte buffer that you can write to a file.

➤ Add this code to the end of `save()`:

```
do {
  // 1
```

```
    let data = try PropertyListSerialization.data(
      fromPropertyList: plistData,
      format: .binary,
      options: .zero)
    // 2
    try data.write(to: dataURL, options: .atomic)
  } catch {
    // 3
    throw FileError.saveFailure
  }
```

Going through the code:

1. You convert your history data to a serialized property list format. The result is a `Data` type, which is a buffer of bytes.
2. You write to disk using the URL you formatted earlier.
3. The conversion and writing may throw errors, which you catch by throwing an error.

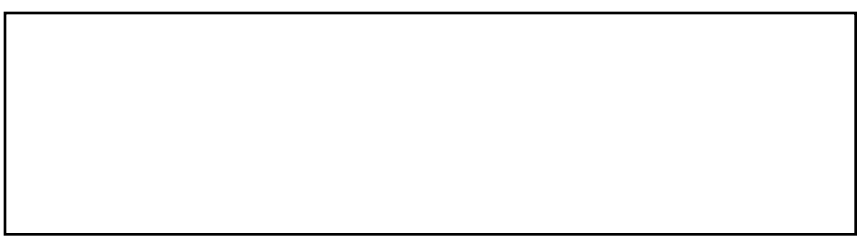
➤ Call `save()` from the end of `addDoneExercise(_:)`:

```
  do {
    try save()
  } catch {
    fatalError(error.localizedDescription)
  }
```

If there's an error in saving, you crash the app, printing out the string description of your error. This isn't a great way to ship your app, and you may want to change it later.

➤ Build and run and do an exercise. Tap **Done** and your history file will save.

➤ In **Finder**, go to your app's **Documents** directory, and you'll see **history.plist**. Double click the file to open this file in Xcode.



Saved history property list file

See how the property list file matches with your data:

- **Root**: The property list array you saved in `plistData`. This is an array of type `[[Any]]`.
- **Item 0**: The first element in `exerciseDays`. This is an array of type `[Any]`.
- **Item 0**: The id converted to `String` format.
- **Item 1**: The date of the exercise
- **Item 2**: The array of exercises that you have performed and tapped **Done** to save. In this example, the user has exercised on one day with two exercises: **Sun Salute** and **Burpee**.

Reading Data From a Property List File

You're successfully writing some history, so you can now load it back in each time the app starts.

➤ In **HistoryStore.swift**, replace `load()` with this code:

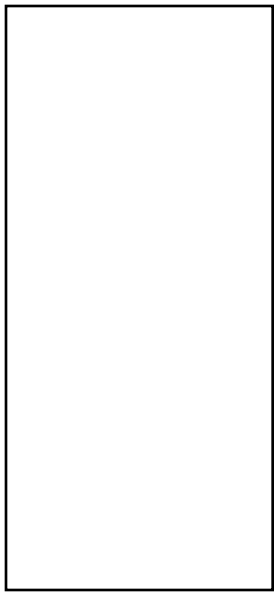
```
func load() throws {
  do {
    // 1
    let data = try Data(contentsOf: dataURL)
    // 2
    let plistData = try PropertyListSerialization.propertyList(
      from: data,
      options: [],
      format: nil)
    // 3
    let convertedPlistData = plistData as? [[Any]] ?? []
    // 4
    exerciseDays = convertedPlistData.map {
      ExerciseDay(
        date: $0[1] as? Date ?? Date(),
        exercises: $0[2] as? [String] ?? [])
    }
  } catch {
    throw FileError.loadFailure
  }
}
```

Loading is very similar to saving, but with some type checking to ensure that your data conforms to the types you are expecting.

Going through the code:

1. Read the data file into a byte buffer. This buffer is in the property list format. If **history.plist** doesn't exist on disk, `Data(contentsOf:)` will throw an error. Throwing an error is not correct in this case, as there will be no history when your user first launches your app. You'll fix this error at the end of this chapter.
2. Convert the property list format into a format that your app can read.
3. When you serialize from a property list, the result is always of type `Any`. To cast to another type, you use the **type cast operator** `as?`. This will return `nil` if the type cast fails. Because you wrote **history.plist** yourself, you can be pretty sure about the contents, and you can cast `plistData` from type `Any` to the `[[Any]]` type that you serialized out to file. If for some reason **history.plist** isn't of type `[[Any]]`, you provide a fall-back of an empty array using the nil coalescing operator `??`.
4. With `convertedPlistData` cast to the expected type of `[[Any]]`, you use `map(_:)` to convert each element of `[Any]` back to `ExerciseDay`. You also ensure that the data is of the expected type and provide fall-backs if necessary.

➤ Build and run, and tap **History**. The history you saved out to your property list file will load in the modal.

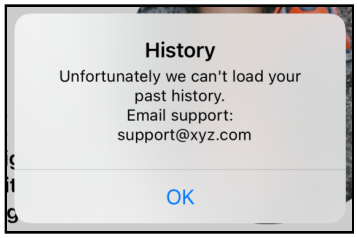


Saved history

Ignoring the Loading Error

➤ Delete **history.plist** in Finder, and build and run your app.

Because the file doesn't exist any more, your loading error appears because `load()` fails on `try Data(contentsOf:)`.



Load error

Before throwing an error, you should first check whether **history.plist** exists. On first run of your app, the plist file will never exist, so you should ignore the loading error.

➤ In **HistoryStore.swift**, in `load()`, remove:

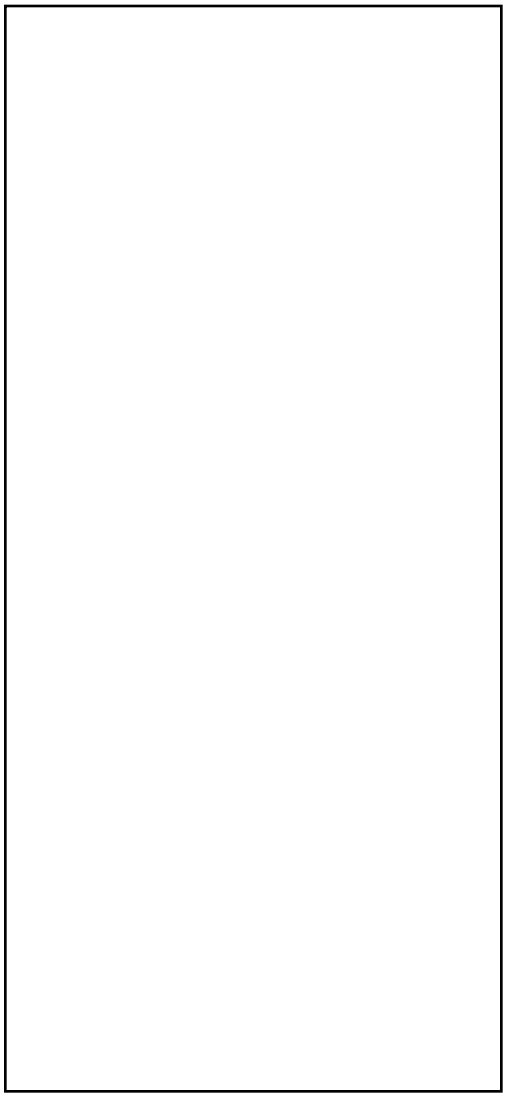
```
let data = try Data(contentsOf: dataURL)
```

➤ Before `do`, add this:

```
guard let data = try? Data(contentsOf: dataURL) else {
  return
}
```

`try?` returns `nil` if the operation fails. Using `guard`, you can jump out of a method if a condition is not met. `guard let` is similar to `if let` in that you assign an optional to a non-optional variable and check it isn't `nil`. You always provide an `else` branch with `guard`, where you specify what to do when the `guard` conditional test fails. Generally you return from the method, but you could instead use `fatalError(_:file:line:)` to crash the app.

➤ Build and run the app. The loading error has gone and you can start recording your exercises.



Final result

Key Points

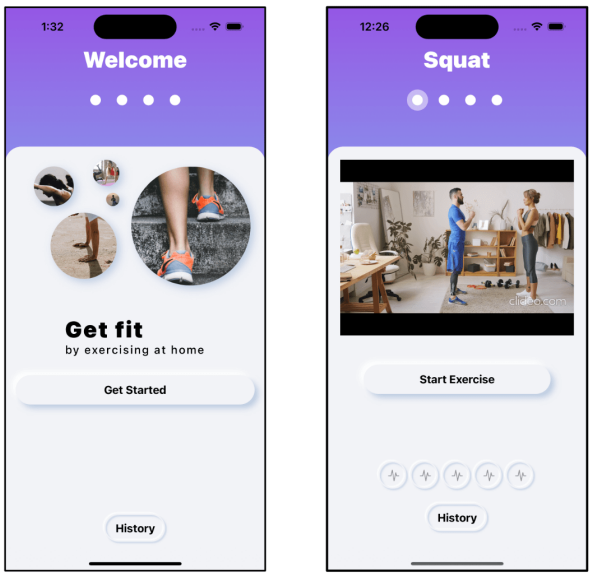
- Optionals are properties that can contain `nil`. Optionals make your code more secure, as the compiler won't allow you to assign `nil` to non-optional properties. You can use `guard let` to unwrap an optional or exit the current method if the optional contains `nil`.
- Don't force-unwrap optionals by marking them with an `!`. It is tempting to use an `!` when assigning optionals to a new property because you think the property will never contain `nil`. Instead, try and keep your code safe by assigning a fall-back value with the nil coalescing operator `??`. For example: `let atLeastOne = oldValue ?? 1`.
- Use breakpoints to halt execution and step through code to confirm that it's working correctly and that variables contain the values you expect.
- Use `throw` to throw errors in methods marked by `throws`.
- If you need to handle errors, call methods marked by `throws` with `do { try ... } catch { ... }`. `catch` will only be performed if the `try` fails. If you don't need to handle errors, you can call the method with `let result = try? method()`. `result` will contain `nil` if there is an error.
- Use `@StateObject` to hold your data store. Your app will only initialize a state object once.
- Closures are chunks of code that you can pass around just as you would any other object. You can assign them to variables or provide them as parameters to methods. `Array` has a number of methods requiring closures to transform its elements into a new array.
- `PropertyListSerialization` is just one way of saving data to disk. You could also use `JSON`, or Core Data, which manages objects and their persistence.

Chapter 9: Refining Your App

By Caroline Begbie

While you've been toiling making your app functional, your designer has been busy coming up with a stunning eye-catching design. One of the strengths of SwiftUI is that, as long as you've been encapsulating views and separating them out along the way, it's easy to restyle the UI without upsetting the main functionality.

In this chapter, you'll style some of the views for iPhone, making sure that they work on all iPhone devices.



App design

Creating individual reusable elements is a good place to start. Looking at the design, you'll have to style:

1. A raised button for **Get Started** and **Start Exercise**.
2. An embossed button for **History** and the exercise rating. The **History** button is a capsule shape, while the rating is round.
3. A shaped gray background view with a gradient behind.

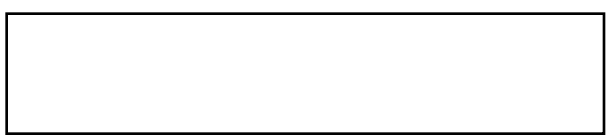
The starter app contains the colors and images that you'll need in the asset catalog. There's also some code for creating the welcome image and text in **WelcomeImages.swift**.

Neumorphism

Skills you'll learn in this section: neumorphism

The style of design used in **HIITFit**, where the background and controls are one single color, is called **neumorphism**. You achieve the look with shading rather than with colors.

In the old days, peak iPhone design had **skeuomorphic** interfaces with realistic surfaces, so you had wood and fabric textures with dials that looked real throughout your UI. iOS 7 went in the opposite direction with minimalistic flat design. The name **Neumorphism** comes from **New** + **Skeuomorphism** and refers to minimalism combined with realistic shadows.



Neumorphism

Essentially, you choose a theme color. You then choose a lighter tint and a darker shade of that theme color for the highlight and shadow. You can define colors with either red, green, blue (RGB) or hue, saturation and lightness (HSL). When shifting tones within one color, HSL is the easier model to use as you keep the same hue. The base color in the picture above is Hue: 166, Saturation: 54, Lightness: 59. The lighter highlight color has the same Hue and Saturation, but a Lightness: 71. Similarly, the darker shadow color has a Lightness: 30.

Creating a Neumorphic Button

The first button you'll create is the **Get Started** raised button.



Get Started button

➤ Open the starter project for this chapter and create a new SwiftUI View file called **RaisedButton.swift**.

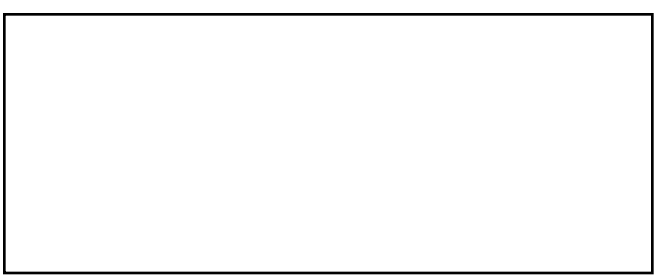
Replace the two structures with:

```
struct RaisedButton: View {
  var body: some View {
    Button(action: {}, label: {
      Text("Get Started")
    })
  }
}

struct RaisedButton_Previews: PreviewProvider {
  static var previews: some View {
    ZStack {
      RaisedButton()
        .padding(20)
    }
    .background(Color("background"))
    .previewLayout(.sizeThatFits)
  }
}
```

Here you create a plain vanilla button with a preview sized to fit the button. **Assets.xcassets** holds the background color "background".

Preview the button using **Selectable** to see only the button.



Plain button

The text style on both the raised buttons in your app have the same design.

➤ Add this code after `RaisedButton`:

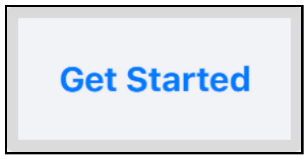
```
extension Text {
  func raisedButtonTextStyle() -> some View {
    self
    .font(.body)
    .fontWeight(.bold)
  }
}
```

Here you style the text with a bold font.

➤ In `RaisedButton`, add the new modifier to `Text("Get Started")`:

```
.raisedButtonTextStyle()
```

Abstracting the style into a modifier makes your app more robust. If you want to change the text style of the buttons, simply change `raisedButtonTextStyle()` and the changes will reflect wherever you used this style.



Styled text

Styles

Skills you'll learn in this section: view styles; button style; shadows

Apple knows that you often want to style objects, so it created a range of style protocols for you to customize (https://apple.co/3kzvD2e). You've already used one of these styles, the built-in `PageTabViewStyle`, on your `TabView`. Styling text is not on that list, which is why you created your own view modifier.

You can customize buttons by setting up a structure that conforms to `ButtonStyle`.

➤ Add this new structure to **RaisedButton.swift**:

```
struct RaisedButtonStyle: ButtonStyle {
  func makeBody(configuration: Configuration) -> some View {
    configuration.label
      .background(Color.red)
  }
}
```

Here you make a simple style giving the button text a red background. `ButtonStyle` has one required method: `makeBody(configuration:)`. The configuration gives you the button's label text and a Boolean `isPressed` telling you whether the button is currently depressed.

> **Swift Tip**: If you want to customize how the button action triggers with gestures, you can use `PrimitiveButtonStyle` instead of `ButtonStyle`.

➤ Still in **RaisedButton.swift**, add a new extension to `ButtonStyle`:

```
extension ButtonStyle where Self == RaisedButtonStyle {
  static var raised: RaisedButtonStyle {
    .init()
  }
}
```

This makes using the button style more Swift-y. Instead of adding a button style: `buttonStyle(RaisedButtonStyle())`, you can instead use: `buttonStyle(.raised)`.

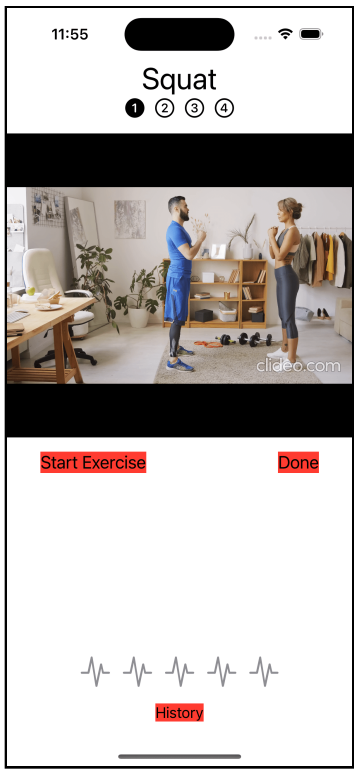
You can use this button style to change all your buttons in a view hierarchy.

➤ Open **HIITFitApp.swift** and temporarily add this new modifier to `ContentView()`:

```
.buttonStyle(.raised)
```

You tell `ContentView` that whenever there's a button in the hierarchy, it should use your custom style.

➤ Build and run the app.



Buttons with styled red background

All the buttons in your app will use your new style with the red background. Notice that when you use a style, the button text color changes from the default accent color of blue to the primary color. That's black in Light Mode and white in Dark Mode.

➤ The buttons in your app won't all use the same style so remove `.buttonStyle(.raised)` from `HIITFitApp`.

➤ Open **RaisedButton.swift** and in `RaisedButton_Previews` add a new modifier to `RaisedButton()`:

```
  .buttonStyle(.raised)
```

You can now preview your button style as you change it.

➤ In `RaisedButtonStyle`, change `makeBody(configuration:)` to:

```
func makeBody(configuration: Configuration) -> some View {
  configuration.label
    .frame(maxWidth: .infinity)
    .padding([.top, .bottom], 12)
    .background(
      Capsule()
    )
}
```

When you set `frame(maxWidth:)` to `.infinity`, you ask the view to take up as much width as its parent gives it. Add some padding around the label text at top and bottom. For the background, use a `Capsule` shape.



Initial button

When you use `Shapes`, such as `Rectangle`, `Circle` and `Capsule`, the default fill color is black, so you'll change that in your neumorphic style to match the background color.

Shadows

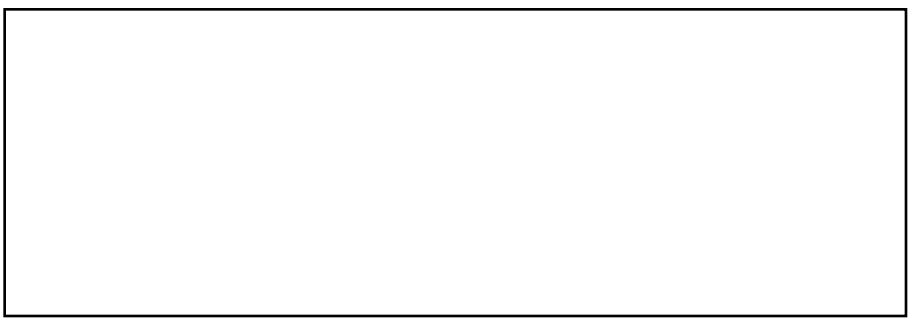
You have two choices when adding shadows. You can choose a simple all round shadow, with a radius. The radius is how many pixels to blur out to. A default shadow with radius of zero places a faint gray line around the object, which can be attractive.

The other alternative is to specify the color, the amount of blur radius, and the offset of the shadow from the center.

➤ In `makeBody(configuration:)`, add new modifiers to `Capsule()`:

```
.foregroundColor(Color("background"))
.shadow(color: Color("drop-shadow"), radius: 4, x: 6, y: 6)
.shadow(color: Color("drop-highlight"), radius: 4, x: -6, y: -6)
```

Watch the button preview change as you add these modifiers. Your darker shadow is offset by six pixels to the right and down, whereas the highlight is offset by six pixels to the left and up. When you add the highlight, the button really pops off the screen.



Button styling

The buttons work in Dark Mode too, because each color in the asset catalog has a value for both Light Mode and Dark Mode. You'll learn more about the asset catalog in Chapter 16, "Adding Assets to Your App".

Abstracting Your Button

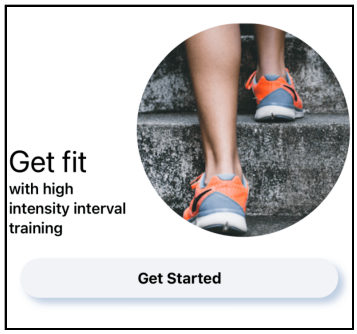
Skills you'll learn in this section: passing closures to views

Your button is finished, so you can now replace your three buttons in your app with this one.

➤ Open **WelcomeView.swift** and locate the button code for **Get Started**. Replace the button code and all the button modifiers with:

```
Button(action: { selectedTab = 0 }) {
  Text("Get Started")
    .raisedButtonTextStyle()
}
.buttonStyle(.raised)
.padding()
```

Here you use your new text and button styles to create your new button. In Live Preview, even though you haven't yet changed the background color, it looks great.



New Get Started

You could change the other button in the same way, or you could make `RaisedButton` more abstract by passing in text and an action. You became familiar with closures in the previous chapter, and here's another way you might use one.

➤ Open **RaisedButton.swift** and change `RaisedButton` to:

```
struct RaisedButton: View {
  let buttonText: String
  let action: () -> Void

  var body: some View {
    Button(action: {
      action()
    }, label: {
      Text(buttonText)
        .raisedButtonTextStyle()
    })
    .buttonStyle(.raised)
  }
}
```

You pass in the button text and an action closure. The action closure of type `() -> Void` takes no parameters and returns nothing. Inside `Button`'s `action` closure, you perform `action()`.

➤ In the preview where you have a compile error, change `RaisedButton()` to:

```
RaisedButton(
  buttonText: "Get Started",
  action: {
    print("Hello World")
  })
```

When the user taps the button marked **Get Started**, your app prints *Hello World* in the console. (Of course a preview doesn't print anything, so nothing will show.)

When a closure is the method's last parameter, the preferred way of calling it is to use special **trailing closure** syntax.

➤ Replace the code above with:

```
RaisedButton(buttonText: "Get Started") {
  print("Hello World")
}
```

With trailing closure syntax, you remove the `action` label and take the closure out of the method's calling parentheses.

Open **WelcomeView.swift** and create a new property for the **Get Started** button:

```
var getStartedButton: some View {
  RaisedButton(buttonText: "Get Started") {
    selectedTab = 0
  }
  .padding()
}
```

➤ In `body`, change your previous **Get Started** button code, including modifiers, to:

```
getStartedButton
```

That code is a lot more succinct but still descriptive and has the same functionality as before.

➤ Open **ExerciseView.swift** and replace `startButton` with:

```
var startButton: some View {
  RaisedButton(buttonText: "Start Exercise") {
    showTimer.toggle()
  }
}
```

At the end of the next chapter, the challenge project moves the **Done** button to a new modal view, so you don't need to change it here.



Start Exercise button

The Embossed Button

Skills you'll learn in this section: stroking a shape

The **History** button will have an embossed border in the shape of a capsule. If you remember from the start of the chapter, the rating view will also have an embossed border. Your new button needs to be able to contain any content, not just text. For this reason, you'll create a new button style and not a new button structure.

➤ Create a new SwiftUI View file named **EmbossedButton.swift**.

➤ Remove `EmbossedButton` entirely as you won't be needing it.

➤ Copy `RaisedButtonStyle` from **RaisedButton.swift** to **EmbossedButton.swift**, and change the name of the copied `RaisedButtonStyle` to `EmbossedButtonStyle`.

➤ Replace `previews` in `EmbossedButton_Previews` with:

```
static var previews: some View {
  Button("History") {}
    .fontWeight(.bold)
    .buttonStyle(EmbossedButtonStyle())
    .padding(40)
    .previewLayout(.sizeThatFits)
}
```

You show a **History** button using the embossed button style.

➤ Set up the Color Scheme Variants:

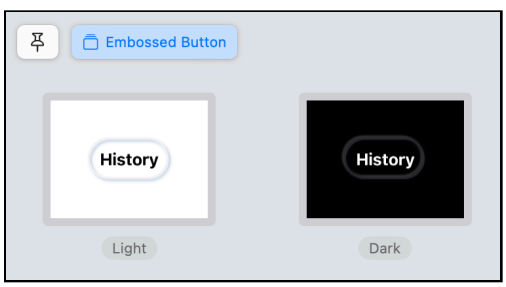


History Button before embossed styling

➤ In `EmbossedButtonStyle`, replace `makeBody(configuration:)` with:

```
func makeBody(configuration: Configuration) -> some View {
  let shadow = Color("drop-shadow")
  let highlight = Color("drop-highlight")
  return configuration.label
    .padding(10)
    .background(
      Capsule()
        .stroke(Color("background"), lineWidth: 2)
        .foregroundColor(Color("background"))
        .shadow(color: shadow, radius: 1, x: 2, y: 2)
        .shadow(color: highlight, radius: 1, x: -2, y: -2)
        .offset(x: -1, y: -1))
}
```

Here you use `stroke(_:linewidth:)` to outline the capsule instead of filling it with color. You'll learn more about shapes and fills in Chapter 18, "Paths & Custom Shapes". You offset the capsule outline by half the width of the stroke, which centers the content.



Embossed History Button

The padding doesn't look enough for the text, but different content may require minimal padding, so you'll add the padding to the content you provide for the button instead of inside the button style.

Your capsule-shaped button is now ready for use in your app. However, looking back at the design at the beginning of the chapter, the designer has placed the ratings in a circular embossed button. You can make your button more useful by allowing different shapes.

➤ Add a new enumeration to **EmbossedButton.swift**:

```
enum EmbossedButtonShape {
  case round, capsule
}
```

➤ In `EmbossedButtonStyle`, below `makeBody(configuration:)`, add a new method:

```
func shape() -> some View {
  Capsule()
}
```

Here, you will determine the shape depending on a passed-in parameter.

➤ In `makeBody(configuration:)`, replace `Capsule()` with:

```
shape()
```

You get a compile error, as `stroke(_:lineWidth:)` is only allowed on actual shapes such as `Rectangle` or `Capsule`, not on `some View`. Place your cursor on `.stroke(Color("background"), lineWidth: 2)`, and press **Option-Command-]** repeatedly to move the line down to below `Capsule()` in `shape()`. The compile error will then go away.

➤ Add a new property to `EmbossedButtonStyle`:

```
var buttonShape = EmbossedButtonShape.capsule
```

If you don't provide a shape, the embossed button will be a capsule.

➤ Change `shape()` to:

```
func shape() -> some View {
  switch buttonShape {
  case .round:
    Circle()
      .stroke(Color("background"), lineWidth: 2)
  case .capsule:
    Capsule()
      .stroke(Color("background"), lineWidth: 2)
  }
}
```

Here you return the desired shape. Unfortunately, you get a compile error. You'll look at this problem in more depth in Section 2, but for now, you just need to understand that the compiler expects `some View` to be one type of view. You're returning either a `Circle` or a `Capsule`, determined at run time, so the compiler doesn't know which type `some View` should be at compile time.

@ViewBuilder

Skills you'll learn in this section: view builder attribute

There are several ways of dealing with this problem. One way is to return a `Group` from `shape()` and place `switch` inside `Group`.

Another way is to use the function builder `@ViewBuilder`. Various built-in views, such as `HStack` and `VStack` can display various types of views, and they achieve this by using `@ViewBuilder`. Shortly, you'll create your own container view where you can stack up other views just as `VStack` does.

➤ Add this above `func shape() -> some View {`:

```
@ViewBuilder
```

Your code now magically compiles.

Internally, `@ViewBuilder` takes in up to ten views and combines them into one `TupleView`. A **tuple** is a loosely formed type made up of several items.

`@ViewBuilder` has ten `buildBlock(...)` methods and, depending on how many contained views there are, calls the appropriate method. Have you ever tried to add more than ten views in a `VStack`? Because there are only ten methods for building up views in `ViewBuilder`, you'll get a compile error: **Extra argument in call**.

This is one of the declarations of `buildBlock(...)` that takes in seven contained views and returns one `TupleView` made up of these seven views:



buildBlock(...)

The other nine `buildBlock(...)` methods are the same except for the different number of views passed in.

➤ In `EmbossedButton_Previews`, change `.buttonStyle(EmbossedButtonStyle())` to:

```
.buttonStyle(EmbossedButtonStyle(buttonShape: .round))
```



Initial round button

The circle takes its diameter from the height of the button.

➤ To visualize this, choose the selectable preview, and in `makeBody(configuration:)`, click `configuration.label` to view the text outline in the preview:



Size of the button

The size of the circle should be the larger of either the width or the height of the button contents. You've already used `GeometryReader` to find out the size of a view, and that's what you'll use here.

➤ In `makeBody(configuration:)`, embed `shape()` in `GeometryReader` and add a `size` parameter to `shape`. This is the contents of `background(_:)`:

```
.background(
  GeometryReader { geometry in
    shape(size: geometry.size)
      .foregroundColor(Color("background"))
      .shadow(color: shadow, radius: 1, x: 2, y: 2)
      .shadow(color: highlight, radius: 1, x: -2, y: -2)
      .offset(x: -1, y: -1)
  })
```

➤ Change `func shape() -> some View` to:

```
func shape(size: CGSize) -> some View {
```

You're now passing to `shape(size:)` the size of the contents of the button, so you can determine the larger of width or height.

➤ In `shape(size:)`, add this modifier to `Circle()` *after* the `stroke` modifier:

```
.frame(
  width: max(size.width, size.height),
  height: max(size.width, size.height))
```

Here you set the frame to the larger of the width or height.

In the selectable preview, you can see that the circle takes the correct diameter of the width of the button contents, but starts at the top.

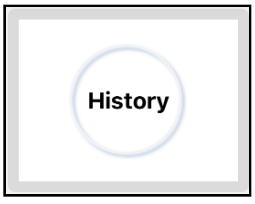


Correct diameter of the button

➤ Add this after the previous modifier:

```
.offset(x: -1)
.offset(y: -max(size.width, size.height) / 2 +
  min(size.width, size.height) / 2)
```

You offset the circle in the **x** direction by half of the width of the stroke. In the **y** direction, you offset the circle by half the diameter plus the smaller of half the width or height.



Completed round button

Your embossed button is now complete and ready to use.

➤ Open **WelcomeView.swift** and add a new property:

```
var historyButton: some View {
  Button(
    action: {
      showHistory = true
    }, label: {
      Text("History")
        .fontWeight(.bold)
        .padding([.leading, .trailing], 5)
    })
    .padding(.bottom, 10)
    .buttonStyle(EmbossedButtonStyle())
}
```

Here you format a new **History** button and use the default capsule shape for the button style.

➤ In `body`, replace:

```
Button("History") {
  showHistory.toggle()
}
.sheet(isPresented: $showHistory) {
  HistoryView(showHistory: $showHistory)
}
.padding(.bottom)
```

with:

```
historyButton
  .sheet(isPresented: $showHistory) {
    HistoryView(showHistory: $showHistory)
  }
```

➤ Copy the `var historyButton` code, open **ExerciseView.swift** and paste the code into `ExerciseView`.

➤ In `body`, replace:

```
Button("History") {
  showHistory.toggle()
}
```

with:

```
historyButton
```

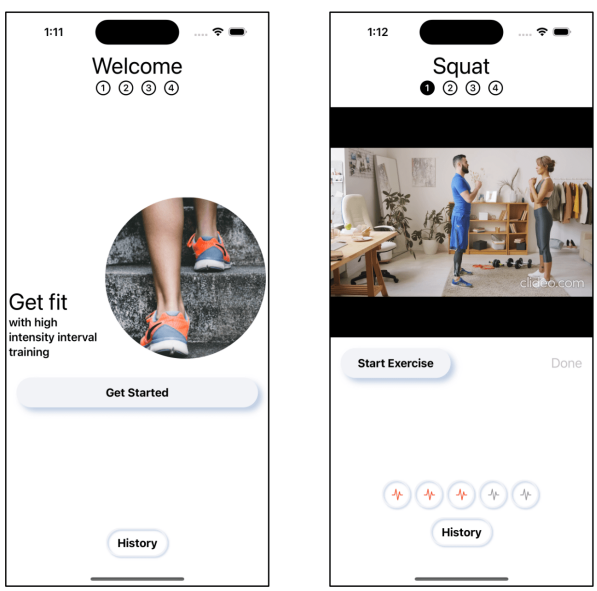
Notice as you replace `body`'s button code with properties describing the views, the code becomes a lot more readable.

➤ In **RatingView.swift**, in `body`, replace the contents of `ForEach` with the new round button:

```
Button(action: {
  updateRating(index: index)
}, label: {
  Image(systemName: "waveform.path.ecg")
    .foregroundColor(
      index > rating ? offColor : onColor)
    .font(.body)
})
.buttonStyle(EmbossedButtonStyle(buttonShape: .round))
.onChange(of: ratings) { _ in
  convertRating()
}
.onAppear {
  convertRating()
}
```

You embed `Image` inside the new embossed button as the label, and this time, you use the round embossed style.

➤ Build and run and admire your new buttons:



New buttons

ViewBuilder Container View

Skills you'll learn in this section: container views

Looking at the design at the beginning of the chapter, the tab views have a purple/blue gradient background for the header and a gray background with round corners for the rest of the view.

You can make this gray background into a container view and embed `WelcomeView` and `ExerciseView` inside it. The container view will be a `@ViewBuilder`. It will take in any kind of view content as a parameter and add its own formatting to the view stack. This is how `HStack` and `VStack` work.

➤ Create a new SwiftUI View file named **ContainerView.swift**.

➤ Change `struct ContainerView: View {` to:

```
struct ContainerView<Content: View>: View {
  var content: Content
```

`Content` is a generic. Generics make Swift very flexible and let you create methods that work on multiple types without compile errors. Here, `Content` takes on the type with which you initialize the view. You'll learn more about generics in Chapter 15, "Structures, Classes & Protocols".

➤ Create an initializer for `ContainerView`:

```
  init(@ViewBuilder content: () -> Content) {
    self.content = content()
  }
```

You'll recognize the argument of the initializer as a closure. It's a closure that takes in no parameters and returns a generic value `Content`. In the initializer, you run the closure and place the *result* of the closure in `ContainerView`'s local storage.

You mark the closure method with the `@ViewBuilder` attribute, allowing it to return multiple child views of any type.

➤ Change `body` to:

```
  var body: some View {
    content
  }
```

The view here is the result of the content closure that the initializer performed.

Now, you can test your container view in the preview.

➤ Change `ContainerView_Previews` to:

```
struct Container_Previews: PreviewProvider {
  static var previews: some View {
    ContainerView {
      VStack {
        RaisedButton(buttonText: "Hello World") {}
          .padding(50)
        Button("Tap me!") {}
          .buttonStyle(EmbossedButtonStyle(buttonShape: .round))
      }
    }
    .padding(50)
    .previewLayout(.sizeThatFits)
  }
}
```

You create a `VStack` of two buttons. You send `ContainerView` the `VStack` as the `content` closure parameter. `ContainerView` then shows the result of running the closure `content`.



Preview of ContainerView

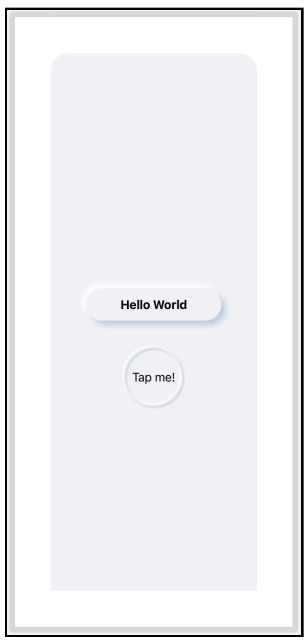
In this example, `ContainerView` merely returns the content, which is a `VStack`. Your container view will format the background on which the content resides. You can then present any content and the background will be the same.

➤ In `ContainerView` replace `body` with:

```
var body: some View {
  ZStack {
    RoundedRectangle(cornerRadius: 25.0)
```

```
        .foregroundColor(Color("background"))
      VStack {
        Spacer()
        Rectangle()
          .frame(height: 25)
          .foregroundColor(Color("background"))
      }
      content
    }
  }
```

Here you create a rounded rectangle using the background color from the asset catalog. You don't want the bottom corners to be rounded, so you add a rectangle with sharp corners at the bottom to cover up the corners.



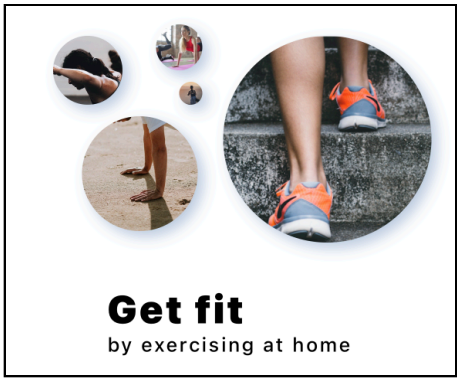
Finished ContainerView

Your container view is now finished. You can construct any views and present them with the same background. It's a good idea not to add unnecessary padding to the actual container view, as that reduces the flexibility. Here the preview provides the padding, but shortly you'll make the container view go right to the edges.

Designing WelcomeView

Skills you'll learn in this section: refactoring with view properties; the safe area

➤ Open **WelcomeImages.swift**. This is a file included in your starter project which contains some images and formatted text to use in `WelcomeView`.



Welcome images and text

One interesting formatting tip to note in `welcomeText` is the text **kerning** in the modifier `.kerning(2)`. This gives you control over the spacing between the letters.

➤ Open **WelcomeView.swift** and replace `body` with:

```
var body: some View {
  GeometryReader { geometry in
    VStack {
      HeaderView(
        selectedTab: $selectedTab,
        titleText: "Welcome")
      Spacer()
      // container view
      VStack {
        WelcomeView.images
        WelcomeView.welcomeText
        getStartedButton
        Spacer()
        historyButton
      }
    }
    .sheet(isPresented: $showHistory) {
      HistoryView(showHistory: $showHistory)
```

```
      }
    }
  }
```

Here you use the images and text from **WelcomeImages.swift**. Wherever you can refactor your code into smaller chunks, you should. This code is much clearer and easier to read. You embed the top `VStack` in `GeometryReader` so that you'll be able to determine the size available for the container view.

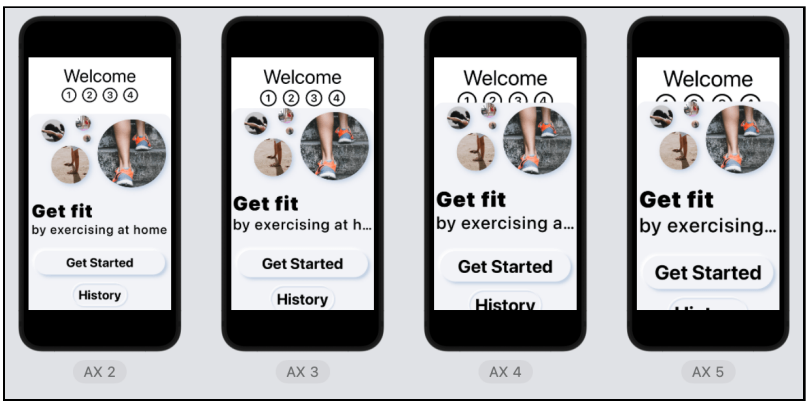
➤ Embed the second `VStack` — the one containing the images and text — in your `ContainerView` and add the modifier to determine its height:

```
// container view
ContainerView {
  VStack {
    ...
  }
}
.frame(height: geometry.size.height * 0.8)
```

Using the size given by `GeometryReader`, the container view will take up 80% of the available space. You'll take a further look at `GeometryReader` in Chapter 20, "Delightful UX - Layout".

➤ Open **ContentView.swift**. Change the run device to **iPhone SE (3rd generation)** and preview Dynamic Type Variants.

This will show the text at various accessibility levels. You want to make sure your app looks great in all circumstances.



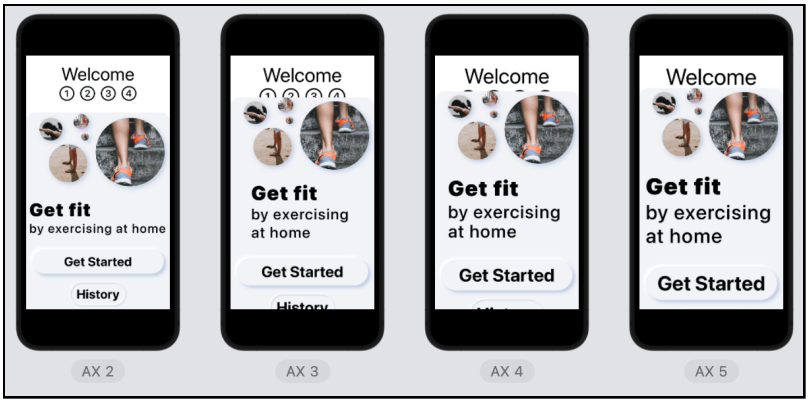
Dynamic Type Variants

The text "by exercising at home" truncates on the larger sizes.

➤ Pin **ContentView** in the Canvas and open **WelcomeImages.swift**. In `welcomeText`, add a new modifier to `Text("by exercising at home")`:

```
.fixedSize(horizontal: false, vertical: true)
```

The text fits horizontally, and you ask SwiftUI to fix an ideal vertical size.



Fixed vertical size

You fixed the text, but the **History** button is disappearing off the foot of the container view.

ViewThatFits

Using `ViewThatFits`, you can present alternative layouts. Work out what is important for interaction with your app. For the larger size text variants, you could dispense with the images.

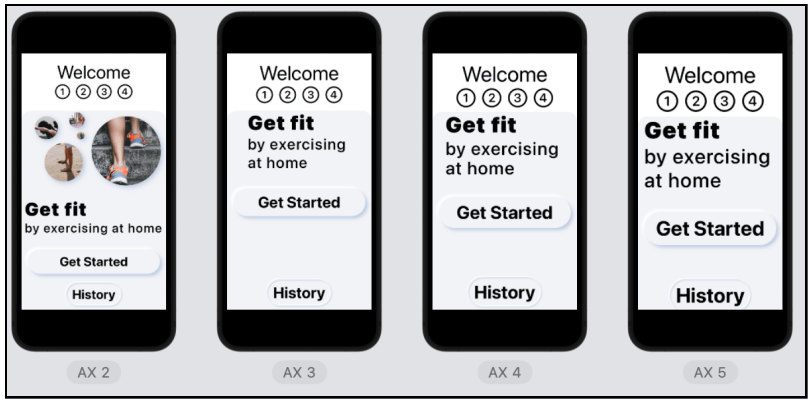
➤ Open **WelcomeView.swift**, and locate `ContainerView`.

Embed the `VStack` inside `ContainerView` in `ViewThatFits`. Then duplicate the `VStack` with its contents, and remove `WelcomeView.images` in the second `VStack`.

```
ViewThatFits {
  VStack {
    WelcomeView.images
    WelcomeView.welcomeText
    getStartedButton
    Spacer()
    historyButton
  }
  VStack {
    WelcomeView.welcomeText
    getStartedButton
```

```
      Spacer()
      historyButton
    }
  }
```

Your app will use the first `VStack` wherever it can, but when space is tight, it will use the alternative one.



ViewThatFits

The images won't show on small devices with large text. Always remember to preview your app on multiple devices with all the variants.

➤ Unpin **ContentView** and change your run destination back to **iPhone 14 Pro**.

Gradients

Skills you'll learn in this section: gradient views

SwiftUI makes using gradients really easy. You simply define the gradient colors in an array. As a background behind the header view, you're going to use a lovely purple to blue gradient, using the predefined colors in the asset catalog.

➤ Create a new SwiftUI View file called **GradientBackground.swift** and add a new property to `GradientBackground`:

```
  var gradient: Gradient {
    Gradient(colors: [
      Color("gradient-top"),
```

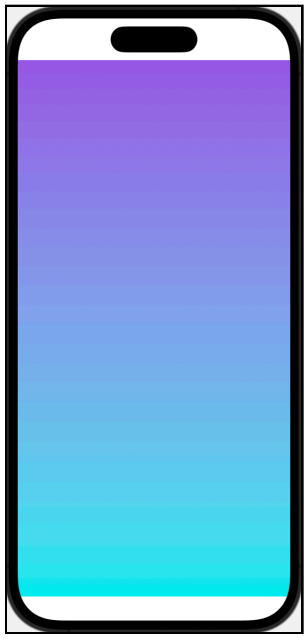
```
      Color("gradient-bottom")
    ])
  }
```

This defines the gradient colors.

➤ Change `body` to:

```
  var body: some View {
    LinearGradient(
      gradient: gradient,
      startPoint: .top,
      endPoint: .bottom)
  }
```

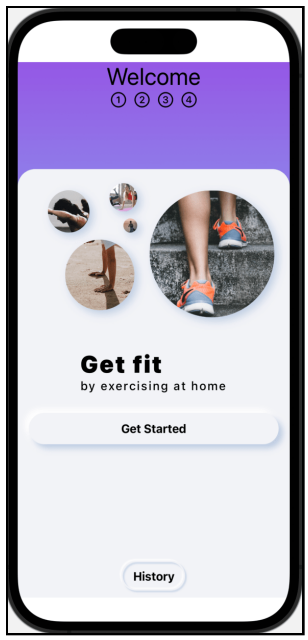
You start the gradient at the top and continue down to the bottom. If you want the gradient to be diagonal, you can use `.topLeading` as the start point and `.bottomTrailing` as the end point.



Initial gradient

➤ Open **ContentView.swift** to add your gradient background. Embed `TabView` in a `ZStack` and add your background:

```
ZStack {
  GradientBackground()
  TabView(selection: $selectedTab) {
    ...
  }
  ...
}
```



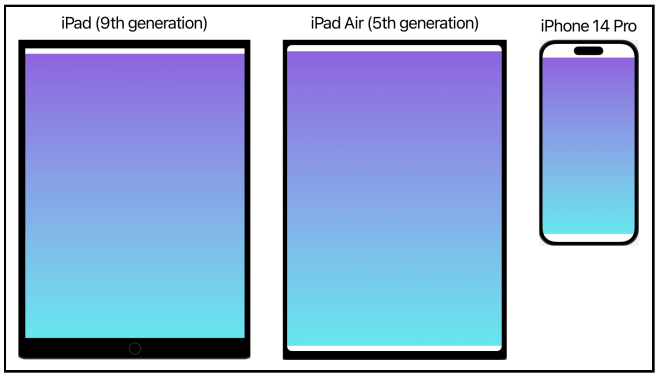
Gradient background

In Live Preview, your gradient shows behind the header view, but doesn't cover the dynamic island or the bottom of the screen.

The Safe Area

A safe area on a device, as its name suggests, is an area where you should never place interactive views. This area might be covered by the dynamic island, a navigation bar or a toolbar.

Devices without a physical home button have a safe area at the bottom of the screen where you swipe up to leave the app.



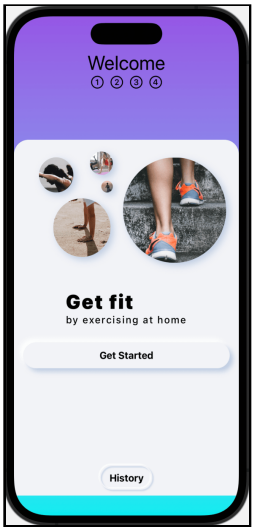
The safe area

By default, a view will size itself respecting the safe areas, but you can override this.

➤ Pin **ContentView** in the canvas again and open **GradientBackground.swift**.

➤ In `body`, add this modifier to `LinearGradient`:

```
.ignoresSafeArea()
```



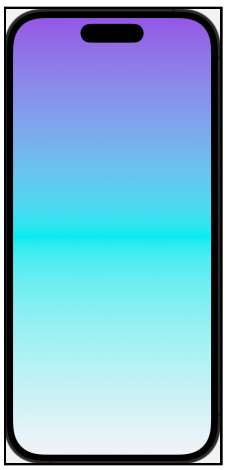
Covering the safe area

The gradient now stretches to all screen edges. This doesn't look great at the bottom of the screen, but you can cover that area with the gray background color.

➤ Include the gray background color in the list of colors:

```
Gradient(colors: [
  Color("gradient-top"),
  Color("gradient-bottom"),
  Color("background")
])
```

Introducing this third color means that the gradient is now divided equally between the three colors and gives a less pleasing purple to blue gradient.



Gray added to gradient

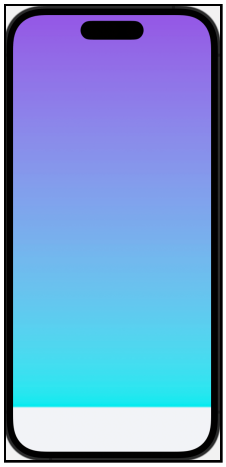
You can control where the gradient changes using stops.

➤ Replace `gradient` with:

```
var gradient: Gradient {
  let color1 = Color("gradient-top")
  let color2 = Color("gradient-bottom")
  let background = Color("background")
  return Gradient(
    stops: [
      Gradient.Stop(color: color1, location: 0),
      Gradient.Stop(color: color2, location: 0.9),
      Gradient.Stop(color: background, location: 0.9),
      Gradient.Stop(color: background, location: 1)
    ])
}
```

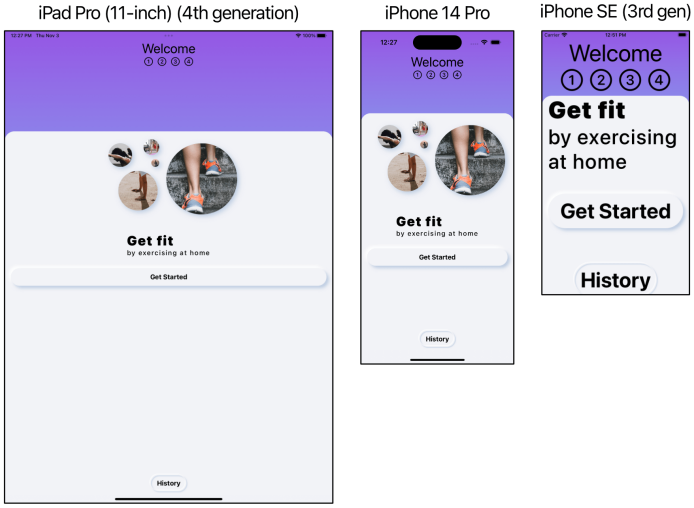
Here you use purple to blue for 90% of the gradient. At the 90% mark, you switch to the background color for the rest of the gradient. As you have two stops right next to each other, you get a sharp line across instead of a gradient.

If you want a striped background, you can achieve this using color stops in this way.



Gradient with stops

➤ Preview your result on all the devices you can. Also make sure that you check that your layout works as far as possible with accessibility dynamic type.



Multiple devices

You could come up with better layouts for iPad, and you now have all the tools at your disposal to do that.

Challenge

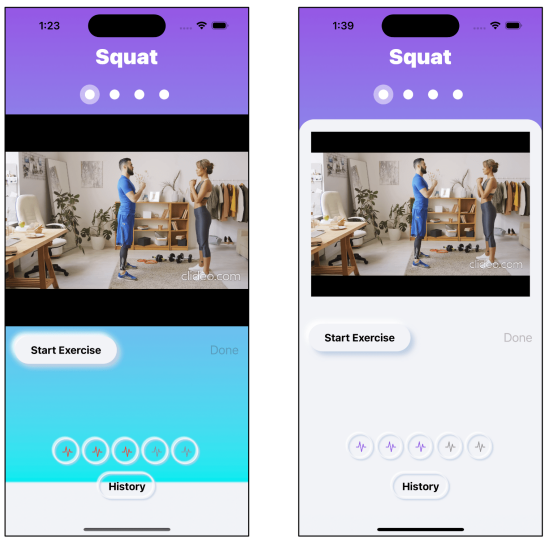
Your challenge is to continue styling. With **ContentView** pinned, style `HeaderView`.



Finished HeaderView

Functionality will remain the same, but instead of numbers, you'll have circles. A faded circle behind the circle indicates the current page. You can achieve transparency with the modifier `opacity(:)`, where opacity is between zero and one.

`ExerciseView` doesn't look so hot with the gradient background, so embed this in `ContainerView` just as you did in `WelcomeView`. Also, match the rating color with the design color using the supplied color "ratings".



Before and after styling

As always, check out the solution in the challenge folder for this chapter.

Key Points

- It's not always possible to spend money on hiring a designer, but you should definitely spend time making your app as attractive and friendly as possible. Try various designs out and offer them to your testers for their opinions.
- Neumorphism is a simple style that works well. Keep up with designer trends at https://dribbble.com.
- Style protocols allow you to customize various view types to fit in with your desired design.
- Using `@ViewBuilder`, you can return varying types of views from methods and properties. It's easy to create custom container views that have added styling or functionality.
- You can layer background colors in the safe area, but don't place any of your user interface there.
- Gradients are an easy way to create a stand-out design. You can find interesting gradients at https://uigradients.com.

Chapter 10: Working With Datasets

By Caroline Begbie

Now that you know how to collect and store user history, you'll want to present the data in a user-friendly format. In this chapter, you'll learn how to deal with sets of data.

First, you'll allow the user to modify and delete the history data. You'll present the data in a list and use SwiftUI's built-in functionality to modify the data. Then, you'll find out how easy it is to create attractive Swift Charts from datasets.

➤ Open the starter project for this chapter.

This project is the almost same as the previous chapter's challenge project with these changes:

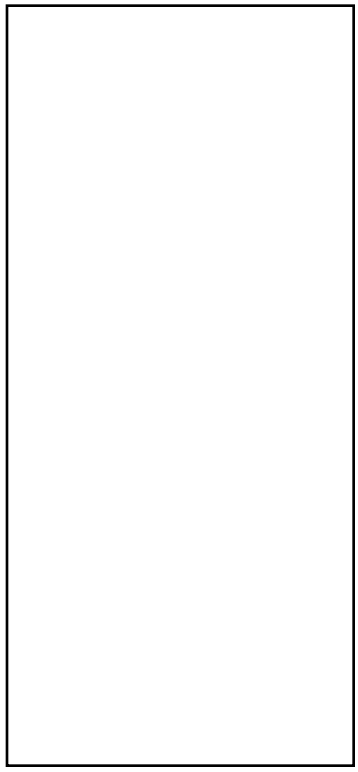
- On first run of the project on Simulator, when there is no history, the app will run `HistoryStore.copyHistoryTestData()`, in **HistoryStoreDevData.swift**. This method copies a sample **history.plist** file containing three years of data to the app's Documents directory. Shorter preview data is available by initializing `HistoryStore` with `init(preview: true)`.
- **HistoryView.swift** will get more complicated through this chapter, so subviews are now in separate properties:

Initial HistoryView subviews

- **DateExtension.swift** and **Exercise.swift** contain some new supporting code.
- **Assets.xcassets** contains some new colors.

➤ In Simulator, choose **Device ▸ Erase All Contents and Settings...**. Erase all the contents to ensure that you start with no history data.

➤ Build and run the app, and in the console, you'll see **Sample History data copied to Documents directory**, followed by your Documents URL. Tap the **History** button to see the sample data.



Sample data

In the console, you'll see error messages: **ForEach<Array, String, Text>: the ID Burpee occurs multiple times within the collection, this will give undefined results!**. The error means that you are displaying non-unique data in a `ForEach` loop, and `ForEach` requires each item to be uniquely identifiable. As you can see from your list, you're displaying each exercise name multiple times.

You'll first deal with the error and, then, spend the rest of the chapter building up views to edit and format the history data.

Accumulating Data

Skills you'll learn in this section: `Sets`; badges

Instead of showing all the exercises on each line, you'll show a list of dates, with the number of times you've performed the exercises accumulated within those dates. Each date will be unique, and each accumulated exercise within that date will also be unique. The `ForEach` loops will then show unique data with no errors.

Swift Dive: Sets

To accumulate the data, you'll create a `Set` of exercises for each day. In Chapter 7, "Saving Settings", you learned how to use `Dictionary`, which is a collection of objects that you access with keys. A `Set` is an unordered collection of unique objects. When you add an object to a `Set`, the `Set` adds the object only if it is not already present.

For example, a `Set` created from this `Array` of exercises for July 16th:

```
[Squat, Burpee, Squat, Sun Salute, Sun Salute]
```

Would contain (in no particular order):

```
[Squat, Sun Salute, Burpee]
```

Accumulating the Exercises

➤ In the **Model** group, open **HistoryStore.swift** and add a new property to `ExerciseDay`:

```
var uniqueExercises: [String] {
  Array(Set(exercises)).sorted(by: <)
}
```

Here you take the array of exercises, create a set of unique instances, and then return an array created from that set, sorted alphabetically.

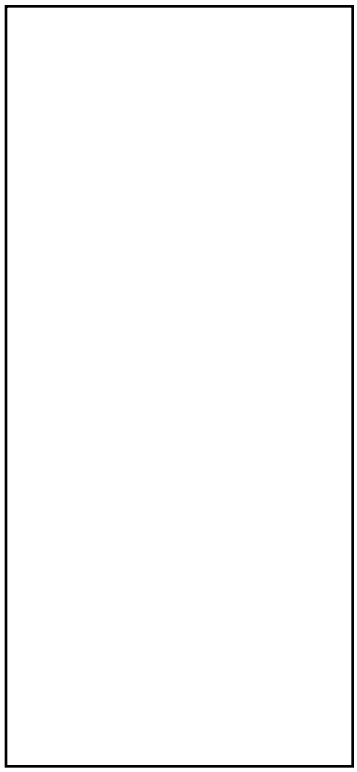
➤ In the **Views ▸ History Views** group, open **HistoryView.swift** and, in `exerciseView`, change `day.exercises` to:

```
day.uniqueExercises
```

Instead of listing all the exercises, you list only the unique instances.

➤ Build and run the app, and tap the **History** button.

You no longer get an error printed in the console as all the values listed are unique.



Unique values

Unfortunately the seventeen squats you might have achieved in a day have all been reduced to one listing. Time to add the accumulated number.

➤ Open **HistoryStore.swift** and add a new method to `ExerciseDay`:

```
func countExercise(exercise: String) -> Int {
  exercises.filter { $0 == exercise }.count
}
```

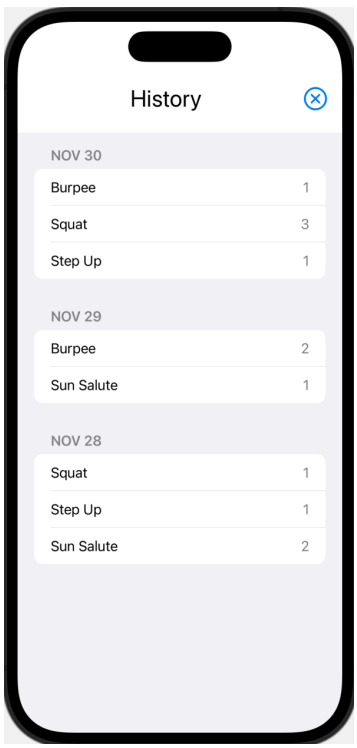
Here you pass in an exercise name and retrieve all the instances of that exercise from the exercises array. You return the count of those instances.

➤ Back in **HistoryView.swift**, in `exerciseView`, add a modifier to `Text(exercise)`:

```
.badge(day.countExercise(exercise: exercise))
```

A badge provides supplementary information in a list. This badge will show the number of times you performed an exercise.

➤ Preview the view. Instead of repeating the listings, each exercise now shows the number of times you've performed it.



Unique values

Lists

Skills you'll learn in this section: Listing data; deleting items from lists; collapsing hierarchical data; the Edit button

A `List` is a container that shows elements from a collection of data. Each element is presented on a row. This is similar to the `Form` you've been using, but your current usage is much more a listing of data than creating a form where you might ask for input from the user.

Editable Lists

Being able to edit lists of data is a common requirement. In this app, you may want to reset your daily exercise. Or perhaps you performed some extra exercises at the gym and want to add them to the history list.

The Apple standard way of editing rows in lists is to have both an **Edit** button in the navigation bar and a swipe action to delete. You'll first add the swipe action and then add the button. Most of the behavior is built-in.

With your data, each item in the file has a unique date, and the exercises for that date are held in a single array.

Your sample data

This design choice means that your data is separated by date, not by exercise. The top level of your list will be dates, which means that built-in list editing will take place on dates. Of course, with the nested `ForEach` loop, you can still list the exercises, but editing those exercises is more difficult.

To start with, simplify `dayView(day:)` so that it shows only dates.

➤ In `HistoryView`, replace the contents of `dayView(day:)` with:

```
Text(day.date.formatted(as: "d MMM YYYY"))
  .font(.headline)
```

You change the date format and list only dates.



Listing dates

➤ In `body`, replace `Form` and its contents with:

```
List($history.exerciseDays, editActions: [.delete]) { $day in
  dayView(day: day)
}
```

That's all that's required to set up deletion from a list.

When you have a `List` with a `ForEach` following, you can compress them into one. The `$` binding syntax makes the data mutable so that you can delete it. The edit action here is `delete`. Another edit action is `move`, which allows you to move rows around in the list. You can try this out, but it doesn't make sense to reorder the dates here.

➤ In Live Preview, swipe a date to the left. You'll first see the **Delete** button, and you can either continue the swipe or tap the button to delete the row.



The delete button

The row disappears from both the list and `HistoryStore.exercises`. However, you haven't yet saved the history file on disk. If you run the app in Simulator, the rows will disappear when you delete them, but on the second run, the deleted data reappears.

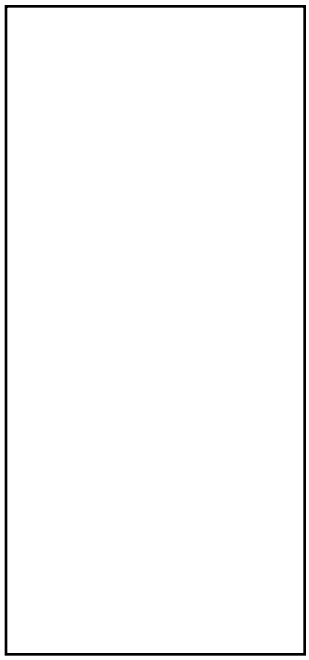
You'll save the history data when the user closes the History view.

➤ In `body`, add this modifier to `VStack`:

```
.onDisappear {
  try? history.save()
}
```

When the view disappears, save the data. However, if the user doesn't close the History view, but instead leaves the app by swiping up from the bottom, the app may be closed by the system without saving the data. You'll find out how to overcome this by checking scene phases in Chapter 19, "Saving Files".

➤ Build and run the app and tap the **History** button to test that your deletion works. To save the data permanently, make sure you close the History view in the app before exiting the app.



Date deletion

Showing Hierarchical Data

A data hierarchy has a parent and children. In your data, the date is the parent, and the exercises are the children. Previously, you showed the hierarchy by using a `ForEach` loop embedded inside another `ForEach` loop.

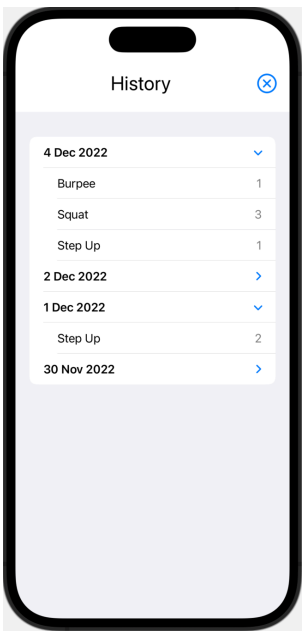
Another way of showing hierarchical data is to use a disclosure group. This view collapses its contents and you expand them using a disclosure indicator.

➤ Replace `dayView(day:)` with:

```
func dayView(day: ExerciseDay) -> some View {
  DisclosureGroup {
    exerciseView(day: day)
  } label: {
    Text(day.date.formatted(as: "d MMM YYYY"))
      .font(.headline)
  }
}
```

Here you embed your `Text` view in a `DisclosureGroup`. The disclosure group, when expanded will reveal `exerciseView(day:)`.

➤ Live Preview the view and tap each date to see your accumulated exercises for that date.



Disclosure groups

Note: If you have truly hierarchical data, such as a structure `Parent` containing a property `children: [Parent]`, where `children` is an array of

the same type as `Parent`, you can take advantage of SwiftUI's automatic disclosure by initializing the list with the format `List(parents, children: \.children) { parent in ... }`. The list will automatically list all parents, with disclosure groups for the children.

Correcting Row Deletion

Something very odd happens when you swipe left on an exercise to delete it. An entire day disappears.

Disappearing date

On each delete action, your `List` is deleting the top level of data, which in your case is the date.

➤ In `dayView(day:)`, add this modifier to `exerciseView(day:)`:

```
.deleteDisabled(true)
```

Now, you will still be able to delete the date with all the exercises, but you won't be able to delete a single exercise

> **Note**: You can of course do anything with your data in Swift. If you had the requirement of deleting a single exercise, you might set up your data differently, so that the top level of a list would be by exercise, rather than by date. Alternatively, instead of using the built in `editActions` of a list, you can use the `onDelete(perform:)` modifier for deletion and write the deletion code yourself.

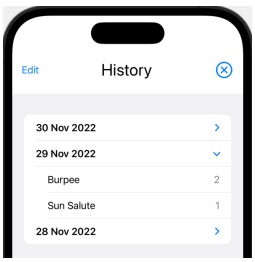
The Edit button

In addition to swipe-to-delete, you should implement an **Edit** button. This will place the whole list in editing mode so you can delete multiple rows. Apple provides a special button which does all the work for you.

➤ In `headerView`, add this as the first view in the `HStack`:

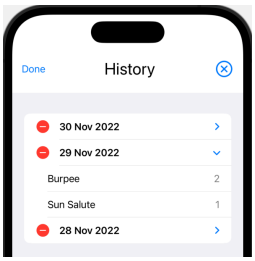
```
EditButton()
```

This places a standard **Edit** button in the header view.



Edit button

➤ In Live Preview, tap the **Edit** button to go into edit mode:



Edit mode

When you've finished deleting items, tap **Done** to return to the list.

Adding Data to the List

> **Skills you'll learn in this section**: Date picker; inverting colors; button feedback

You can now delete out-dated information, but how about adding in those exercises that you perform away from your iPhone? This isn't as easy as adding list deletion. You'll create an **Add** button that loads a calendar view from which you can select a date. You'll set up a button for each exercise and each time you tap a button, your app will add an exercise to that date.

➤ Add a new property to `HistoryView` to toggle add mode:

```
@State private var addMode = false
```

When `addMode` is true, you'll show the calendar view.

➤ In `headerView`, add a button as the first item in the `HStack`:

```
Button {
  addMode = true
} label: {
  Image(systemName: "plus")
}
.padding(.trailing)
```

Here you create the add button, which sets `addMode` to `true`.



Add button

In the **History Views** group, create a new SwiftUI View file called **AddHistoryView.swift**. This is where you'll create the calendar view.

➤ Add new properties to `AddHistoryView`:

```
@Binding var addMode: Bool
@State private var exerciseDate = Date()
```

➤ In `AddHistoryView_Previews`, change the contents of `previews` to:

```
AddHistoryView(addMode: .constant(true))
```

You'll dismiss the calendar view from `AddHistoryView`, which entails changing `addMode`. You'll also need to update the exercise date.

Apple provides a date picker that you can configure in various ways. You can customize the style and control which date components you show.

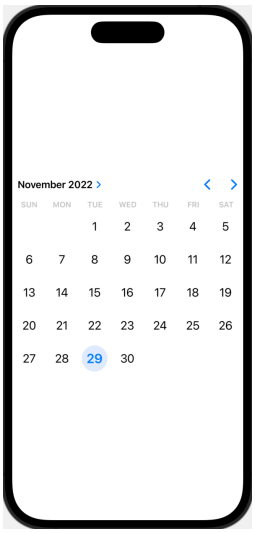
➤ In `AddHistoryView`, replace the contents of `body` with:

```
VStack {
  DatePicker(
    // 1
    "Choose Date",
    // 2
    selection: $exerciseDate,
    // 3
    in: ...Date(),
    // 4
    displayedComponents: .date)
    // 5
  .datePickerStyle(.graphical)
}
.padding()
```

Here's what you can customize on a `DatePicker`:

1. The label of the control. This may not display, depending on the style of the date picker.
2. The binding of the date being selected.
3. An optional closed range. In this case, you don't want to let the user select a future date, so you constrain the date range up to today.

4. Whether you want the date and/or the time to show.
5. The style of the picker. This can be a wheel, a compact drop-down or, in this case, a full calendar month view.



DatePicker

➤ Add this as the first item at the top of the `VStack`:

```
ZStack {
  Text("Add Exercise")
    .font(.title)
  Button("Done") {
    addMode = false
  }
  .frame(maxWidth: .infinity, alignment: .trailing)
}
```

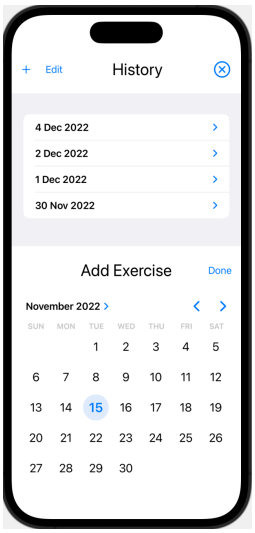
You add the text heading for the view and the button to dismiss the view. By giving the button an infinitely wide frame with trailing alignment, the text is centered, and the button is in the correct position

➤ Open **HistoryView.swift**. In `body`, under `List`, add this code:

```
if addMode {
  AddHistoryView(addMode: $addMode)
}
```

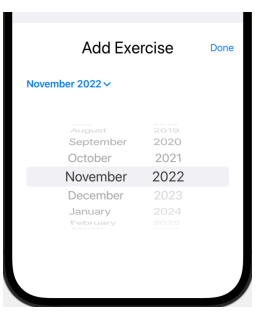
When `addMode` is true, you show the new calendar view.

➤ Try it out in Live Preview. Tap the + button to see the calendar and tap **Done** to make the `DatePicker` disappear.



AddExerciseView

To navigate through the calendar, tap the forward and back arrows. To change the month and year via a wheel picker, tap the disclosure indicator next to the month/year. Tap the disclosure indicator again when you've selected the month and year.



Change the month and year

When you're in add mode, the top navigation buttons are not relevant.

➤ In `body`, change `headerView` to:

```
Group {
  if addMode {
    Text("History")
```

```
        .font(.title)
    } else {
      headerView
    }
  }
```

Now when you're in add mode, the buttons in the navigation bar disappear. You embed the conditional in a group to keep the same padding on both views.

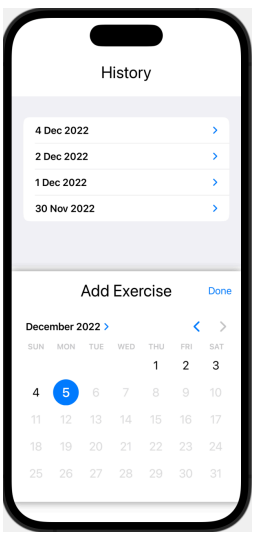
Extra Styling

Add a little pizzazz to the calendar view to make it stand out. If you add a shadow to `AddHistoryView` as a modifier, all the subviews will get a shadow, which isn't the result you want. Instead, you'll add a background color to the view, and add a shadow to that.

➤ In `body`, add this modifier to `AddHistoryView`:

```
.background(Color.primary.colorInvert()
.shadow(color: .primary.opacity(0.5), radius: 7))
```

Here you change the date picker's background color to the system's primary color, and invert it. If the system is in Light Mode, the primary color is black. When you invert black, you get white. This matches the original color of the date picker. You add to the background view a primary colored drop shadow with a 50% opacity.



Adding a shadow to the calendar view

Adding the Exercise Buttons

Open **AddHistoryView.swift** and add a new view to the file:

```
struct ButtonsView: View {
  @EnvironmentObject var history: HistoryStore
  @Binding var date: Date

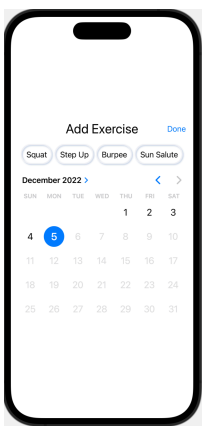
  var body: some View {
    HStack {
      ForEach(Exercise.exercises.indices, id: \.self) { index in
        let exerciseName =
Exercise.exercises[index].exerciseName
        Button(exerciseName) {
          // save the exercise
        }
      }
    }
    .buttonStyle(EmbossedButtonStyle())
  }
}
```

Here you create a view that shows a button for each exercise, using the embossed button style from the previous chapter.

➤ In `AddHistoryView`, add this to the `VStack` above `DatePicker`:

```
ButtonsView(date: $exerciseDate)
```

You show the buttons and pass the currently selected date to `ButtonsView`



The exercise buttons

When you tap one of these buttons, the interface feels curiously unresponsive.

Feedback When Tapping a Button

➤ Open **EmbossedButton.swift** and examine `EmbossedButtonStyle`. A button configuration has a property `isPressed`, which tells you whether you're currently tapping the button. You can check this property and style your button accordingly.

You'll scale the button up temporarily, just while the user is tapping the button.

➤ Add a new property to `EmbossedButtonStyle`:

```
var buttonScale = 1.0
```

➤ At the very end of `makeBody(configuration:)`, add a new modifier to `configuration.label`:

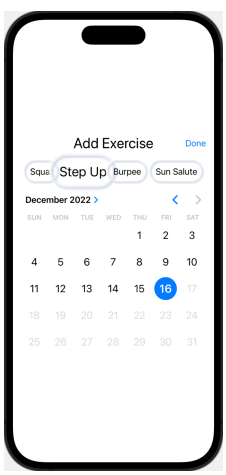
```
.scaleEffect(configuration.isPressed ? buttonScale : 1.0)
```

When the user is pressing the button, the button will scale up to the supplied value. At all other times, the button's scale won't change.

➤ Open **AddHistoryView.swift** and, in `ButtonsView.body`, change `.buttonStyle(EmbossedButtonStyle())` to:

```
.buttonStyle(EmbossedButtonStyle(buttonScale: 1.5))
```

The buttons will now scale up when you tap them, giving you feedback on your action.



The button scales on tap

Incrementing the Exercise Count

When you tap an exercise, the exercise count for that date should increment.

➤ Open **HistoryStore.swift**.

`addDoneExercise(_:)` will add or insert exercises. However, as you can now insert historical dates, you'll need a new method that inserts the date in the correct position in the array.

➤ Add a new method to `HistoryStore`:

```
func addExercise(date: Date, exerciseName: String) {
  let exerciseDay = ExerciseDay(date: date, exercises:
[exerciseName])
  // 1
  if let index = exerciseDays.firstIndex(
    where: { $0.date.yearMonthDay <= date.yearMonthDay }) {
    // 2
    if date.isSameDay(as: exerciseDays[index].date) {
      exerciseDays[index].exercises.append(exerciseName)
    // 3
    } else {
      exerciseDays.insert(exerciseDay, at: index)
    }
    // 4
  } else {
    exerciseDays.append(exerciseDay)
  }
  // 5
  try? save()
}
```

Going through the code:

1. You find the first index in the `exerciseDays` array where the date is less than or equal to the passed-in date. The `where` part is a comparison closure that returns true when the criterion is matched. The index of the first true comparison is then passed back to `index`. Here, the conditional will fail if the passed-in date is earlier than the array dates. You want to compare the dates on a daily basis, so you use `yearMonthDay` from **DateExtension.swift**, to exclude the time.
2. If you find a date in the array that's the same as the passed-in date, you append the exercise name to the already existing array element.

3. If the date doesn't already exist in the array, then insert it at the appropriate position.
4. If the date is earlier than all the dates in the array, or the array is empty, then append the date to the array.
5. Save the history data.

➤ Open **AddHistoryView.swift** and, in `ButtonsView`, replace `// save the exercise` with this:

```
history.addExercise(date: date, exerciseName: exerciseName)
```

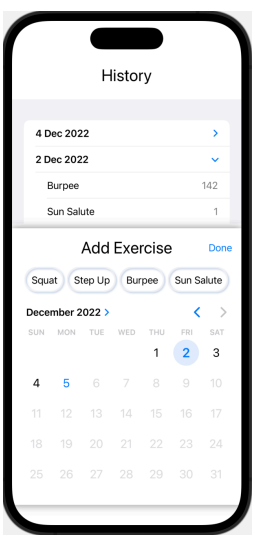
You call the new method with the currently selected date and the name on the tapped button.

➤ In `AddHistoryView_Previews`, add this modifier to `AddHistoryView(addMode:)`:

```
.environmentObject(HistoryStore(preview: true))
```

`ButtonsView` accesses `HistoryStore` through the environment. If you don't set up the environment somewhere in the hierarchy, the preview will crash.

➤ Open **HistoryView.swift** and try out adding new exercises in Live Preview. Then, try your app in Simulator to make sure that it all gets saved correctly there too.



Oh my, that's a lot of burpees!

Charts

Skills you'll learn in this section: Bar charts; organizing data for charts; line charts; stacked charts

Using your history data, Swift Charts give you an opportunity to graphically summarize how you're performing. You can show which exercise you perform the most, how often you exercise or how many exercises you perform per day or any selected time period. Discover trends so you can analyze why you sometimes have periods where you're less enthusiastic about exercising. With just a few lines of code, you can draw beautiful charts.

A chart consists of **marks** which represent the data. These marks could be points, lines, areas or rectangular bars. The data is **categorical**, meaning that it can be separated out into categories, or in this case, exercises.

Charts have two axes. One axis plots the individual categories, and the other axis plots the numerical data associated with the categories.

Bar Charts

Bar charts present data using rectangles of different heights.

➤ In the **History Views** group, create a new SwiftUI View file called **BarChartDayView.swift**, and add this code to the top of the file:

```
import Charts
```

Here you import the Swift Charts framework.

➤ Replace `BarChartDayView` with:

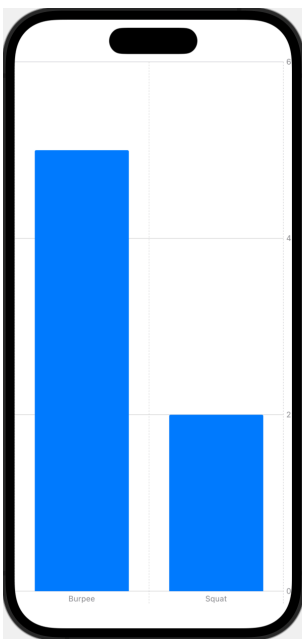
```
struct BarChartDayView: View {
  var body: some View {
  // 1
    Chart {
    // 2
      BarMark(
      // 3
        x: .value("Name", "Burpee"),
      // 4
        y: .value("Count", 5))
      // 5
```

```
        BarMark(
          x: .value("Name", "Squat"),
          y: .value("Count", 2))
      }
    }
  }
```

Creating a chart needs some explanation:

1. Declare that you are creating a Swift Chart.
2. Inside the chart, determine what sort of mark to use. This chart will be a bar chart, but it could also be an area, line or point chart.
3. On the x-axis, you create a plottable value with a label and a string value.
4. Similarly, on the y-axis, you create a plottable value with a label and an integer value.
5. Repeat the marks for each piece of data you want to chart.

Live Preview shows the bar chart with the values that you created.



First bar chart

Now you'll use the history data to create the chart.

➤ Add a new property to `BarChartDayView`:

```
let day: ExerciseDay
```

The chart will show the exercises performed on a particular day.

➤ Change `BarChartDayView_Previews` to:

```
struct BarChartDayView_Previews: PreviewProvider {
  static var history = HistoryStore(preview: true)
  static var previews: some View {
    BarChartDayView(day: history.exerciseDays[0])
      .environmentObject(history)
  }
}
```

Here you load up the history store with the preview data and pass the first day to the chart.

➤ In `BarChartDayView`, replace the contents of `body` with:

```
Chart {
  ForEach(Exercise.names, id: \.self) { name in
    BarMark(
      x: .value(name, name),
      y: .value("Total Count", day.countExercise(exercise:
name)))
    .foregroundStyle(Color("history-bar"))
  }
  RuleMark(y: .value("Exercise", 1))
    .foregroundStyle(.red)
}
.padding()
```

`Exercise.names` is defined in **Exercise.swift** and contains all the names of the exercises. The chart iterates through each exercise and creates a bar mark for each one. The x-axis will display the name of the exercise. For the y-axis, you count the number of times you performed the exercise for the day. You also add a rule mark to show that you should perform at least one of the exercises per day.

`foregroundStyle` allows you to change the colors of the chart. **history-bar** is a color defined in **Assets.xcassets**.

Daily bar chart showing Light and Dark Modes

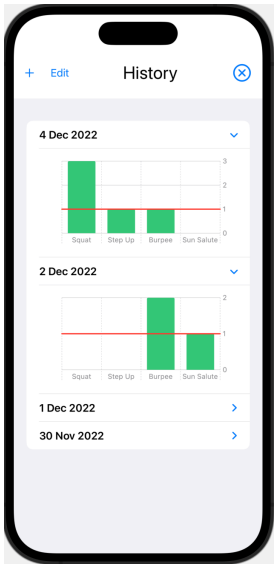
Notice that, as the data is dynamic, the chart automatically scales to the largest bar size. The y-axis numerical labels are listed down the right. Each exercise is a **group**.

This chart is ready for use. You can substitute it for your accumulated exercises in the history list.

➤ Open **HistoryView.swift**, and locate the definition of `dayView(day:)`. Replace `exerciseView(day: day)` with:

```
BarChartDayView(day: day)
```

➤ In Live Preview, check out your daily chart:



Daily chart

This chart shows you individual exercises by day.

Charting a Week's Data

Next, you'll create a bar chart that groups all the exercises by day and shows the latest week's data.

➤ In the **History Views** group, create a new SwiftUI View file called **BarChartWeekView.swift**, and replace the code with:

```
import SwiftUI
import Charts

struct BarChartWeekView: View {
  @EnvironmentObject var history: HistoryStore

  var body: some View {
    // create bar chart here
    .padding()
  }
}

struct BarChartWeekView_Previews: PreviewProvider {
  static var previews: some View {
    BarChartWeekView()
```

```
        .environmentObject(HistoryStore(preview: true))
    }
}
```

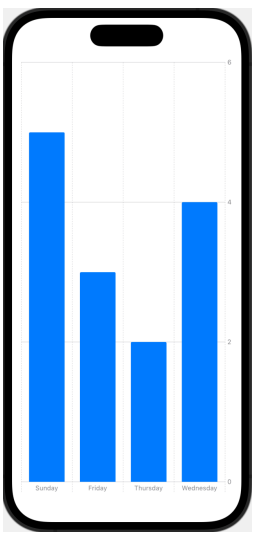
➤ In `body`, replace `// create bar chart here` with:

```
Chart(history.exerciseDays.prefix(7)) { day in
  BarMark(
    x: .value("Date", day.date.dayName),
    y: .value("Total Count", day.exercises.count))
}
```

Going through this chart:

1. When you don't need a separate `ForEach`, you can initialize `Chart` with the chart data. You use the first seven elements of `exerciseDays`. Seven is the maximum value, so if there aren't seven elements in the array, only the available elements are used.
2. For each of the days, you combine all the exercises into one bar.

➤ Look at the result in Live Preview.



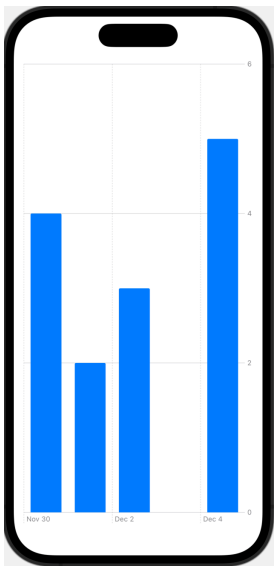
A week's worth of exercises

The preview data only has four days of data, and these show from left to right in reverse date order. It's usual to show week data with the last date on the trailing edge. The preview data skips a day, but the chart doesn't show zero exercises on that day. You can ensure that the chart shows all days by choosing a unit.

➤ Change `x: .value("Date", day.date.dayName),` to:

```
x: .value("Date", day.date, unit: .day),
```

By choosing a unit, the missing day now shows up, and the chart presents the data in standard week-date format with the latest date at the trailing edge.



A daily chart

Because the preview data only contains four days' worth of data, you don't get the full seven days. At times like this, you'll have to massage your data into a format that works with your desired chart.

➤ In `BarChartWeekView`, create a new property to hold one weeks' data:

```
@State private var weekData: [ExerciseDay] = []
```

➤ In `body`, add a new modifier to `Chart`:

```
.onAppear {
  // 1
  let firstDate = history.exerciseDays.first?.date ?? Date()
  // 2
  let dates = firstDate.previousSevenDays
  // 3
  weekData = dates.map { date in
    history.exerciseDays.first(
      where: { $0.date.isSameDay(as: date) })
    ?? ExerciseDay(date: date)
```

```
    }
  }
```

Here you create an array of seven dates. By iterating through these dates, you find out whether you performed any exercises on that date. If you have, you use that data, otherwise you create an empty daily record for that date.

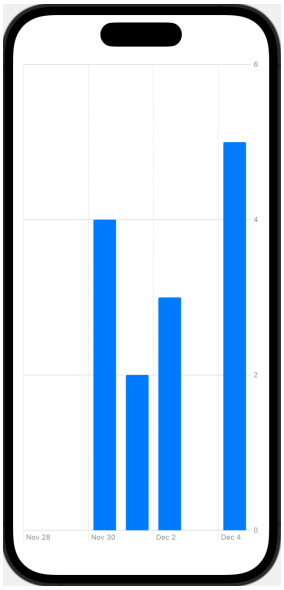
Going through the code:

1. Find out the first date in history. If there isn't one, use the current date.
2. Set up an array using a method already created for you in **DateExtension.swift**.
3. Iterate through the array of dates and for each date, locate the first entry for that date. If there isn't one, create a new blank `ExerciseDay`.

➤ In `body`, replace `Chart(history.exerciseDays.prefix(7)) { day in` with:

```
  Chart(weekData) { day in
```

In the preview, you'll now see a seven-day chart.



A seven-day chart

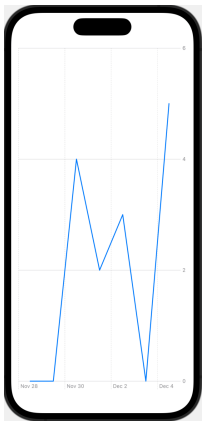
Line Charts

It's easy to replace this bar chart with a line chart.

➤ In `body`, replace `BarMark` with:

```
LineMark
```

In Live Preview, you'll see the chart change to a line chart.



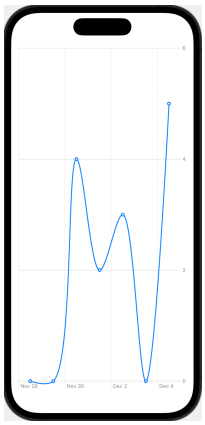
A basic line chart

You can make the chart a bit prettier with some modifiers.

➤ Add these modifiers to `LineMark`:

```
.symbol(.circle)
.interpolationMethod(.catmullRom)
```

At each data point, it draws a circle. Check the code completion in Xcode to see what other symbols you can use. A Catmull-Rom spline interpolates the points along a curve, making the chart smooth instead of linear.

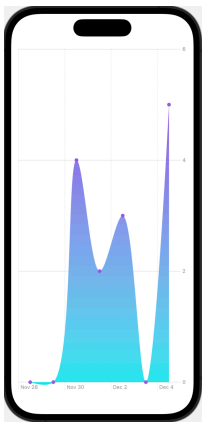


A line chart

Other Chart Styles

Try replacing `LineMark` with `PointMark`, `AreaMark` and `RectangleMark` to see the resulting charts. You can even layer marks by placing one mark after another inside `Chart { }`.

This is an area chart with a point chart layered on top of it. The area chart has a gradient foreground style, and the point chart has a purple foreground style.



An area chart with a point chart

Stacked Bar Chart

➤ Return `Chart` and its contents to:

```
Chart(weekData) { day in
  BarMark(
    x: .value("Date", day.date, unit: .day),
    y: .value("Total Count", day.exercises.count))
}
```

With this bar chart, your exercises are all counted together. This doesn't help if you want to compare your burpees to your squats. You can split out your exercises using similar code to your list when you accumulated the exercise.

➤ Replace `Chart` and its contents to:

```
Chart(weekData) { day in
  ForEach(Exercise.names, id: \.self) { name in
    BarMark(
      x: .value("Date", day.date, unit: .day),
      y: .value("Total Count", day.countExercise(exercise:
name)))
  }
}
```

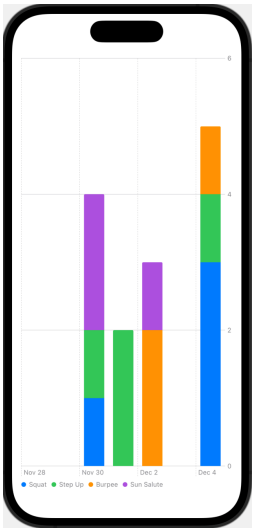
For each day, you iterate through all the four exercise names. `Exercise.names` is a property in **Exercise.swift**. You accumulate the current exercises into the bar mark. The result of this chart is currently the same as the previous chart, but you're now able to split the bars into different colors.

➤ Add a new modifier to `BarMark`:

```
.foregroundStyle(by: .value("Exercise", name))
```

Instead of using a color to determine the style, you separate out the bar by exercise.

In Live Preview, the chart displays the bars with colors marking the relative number of exercises. Beneath the chart, the legend explains what each color represents.



A stacked bar chart

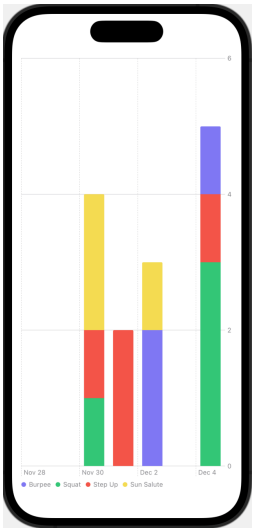
Naturally, you can customize the chart with different colors.

➤ Add a new modifier to `Chart`:

```
.chartForegroundStyleScale([
  "Burpee": Color("chart-burpee"),
  "Squat": Color("chart-squat"),
  "Step Up": Color("chart-step-up"),
  "Sun Salute": Color("chart-sun-salute")
])
```

Assets.xcassets contains these colors for your charts.

The preview updates the legend and the chart with your new colors.



Custom colors

Privacy

Skills you'll learn in this section: User privacy

Collection and analysis of data over time can be very useful. You can track weight trends with health data or wealth with financial data. Like Google and Apple, you can decide how to collect users' data, what to do with it and how present it back to your users. If your app attracts enough users, you might be able to create machine learning datasets and use those datasets for future apps.

Fortunately, Apple enforces user privacy. Apple's article Protecting the User's Privacy (https://apple.co/3P0XSGy) tells you how to access and protect user data. Remember that your users trust you!

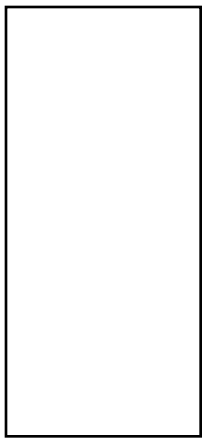
Challenge

As you can see, it's easy to design new charts. Your challenge is to incorporate new charts into your app.

In **WelcomeView.swift**, add a new button beside the **History** button called **Reports**. When you tap this button, you should show a modal view with a `Toggle` to show a bar or a line chart for the last week. Integrate your existing `BarChartWeekView` in the modal view.

Your challenge

As always, you can examine the project in your challenge folder for this chapter. In addition, the challenge project has a styled modal timer view that you can examine.



A styled modal timer view

Key Points

- A `Set` is a collection of data where each element is unique. Both `Set` and `Array` have initializers to create one from the other.
- Use `List` for lists of data. Editing lists is built-in.
- To show groups of data which you can collapse and expand, use a `DisclosureGroup`.
- Swift Charts is a framework that displays your data in gorgeous charts with minimal code.
- As well as bar charts, you can just as easily create line, point and area charts.
- You can layer charts on top of each other, such as layering points on top of lines.
- When you have groups of data, you can stack the data in a single bar. Charts will automatically create different colors for the groups.
- You can customize any chart legends and colors.

Where to Go From Here?

For more practice with Swift Charts, visit Swift Charts Tutorial: Getting Started (https://bit.ly/3VQ2Ack)

Chapter 11: Managing Data With Property Wrappers

By Audrey Tam

In your SwiftUI app, every data value or object that can change needs a single source of truth and a mechanism to enable views to change or observe it. SwiftUI's property wrappers enable you to declare how each view interacts with mutable data.

In this chapter, you'll review how you managed data values and objects in HIITFit with `@State`, `@Binding`, `@Environment`, `ObservableObject`, `@StateObject` and `@EnvironmentObject`. And, you'll build a simple app that lets you focus on how to use these property wrappers. You'll also learn about `TextField`, the `environment` modifier and property wrappers `@ObservedObject` and `@FocusState`.

To help answer the question "struct or class?", you'll see why `HistoryStore` should be a class, not a structure and learn about the natural architecture for SwiftUI apps: the Model-View-ViewModel pattern.

Getting Started

➤ Open the **TIL** project in the starter folder. The project name "TIL" is the acronym for "Today I Learned". Or, you can think of it as "Things I Learned". Here's how the app should work: The user taps the + button to add acronyms like "YOLO" and "BTW", and the main screen displays these.



TIL in action

This app embeds a `VStack` in a `NavigationStack`, which gives you the navigation bar where you display the title and a toolbar where you display the + button. You'll learn more about `NavigationStack` in Section 3.

This project has a `ThingStore`, which is like `HistoryStore` in HIITFit. This app is much simpler than HIITFit, so you can focus on how you manage the data.

Remember how you managed changes to `HistoryStore` in HIITFit:

HIITFit: HistoryStore shared as EnvironmentObject

In Chapter 6, "Observing Objects", you converted `HistoryStore` from a structure to a class conforming to `ObservableObject` and set it up as an `@EnvironmentObject` so `ExerciseView` and `HistoryView` could access it directly.

`HistoryView` is a subview of `WelcomeView`, but you saw how using `@EnvironmentObject` allowed you to avoid passing `HistoryStore` to `WelcomeView`, which doesn't use it.

If you did the challenge in that chapter, you also managed `HistoryStore` with `@State` and `@Binding`.

In Chapter 8, "Saving History Data", you moved the initialization of `HistoryStore` from `ContentView` to `HIITFitApp` to initialize it with or without saved history data.

`ThingStore` has the property `things`, which is an array of `String` values. Like the `HistoryStore` in the first version of HIITFit, it's a structure.

In this chapter, you'll first manage changes to the `ThingStore` structure using `@State` and `@Binding`, then convert it to an `ObservableObject` class and manage changes with `@StateObject` and `@ObservedObject`:

TIL: ThingStore shared as Binding and as ObservedObject

You'll learn that these two approaches are very similar.

> **Note**: Our tutorial Property Wrappers (https://bit.ly/3vLOpbl) extends this project to use `ThingStore` as an `@EnvironmentObject`.

Tools for Managing Data

You already know that a `@State` property is a *source of truth*. A view that owns a `@State` property can pass either its value or its binding to its subviews. If it passes a binding to a subview, that subview now has a reference to the source of truth. This allows the subview to update that property's value or redraw itself when that value changes. When a `@State` value changes, any view with a reference to it invalidates its appearance and redraws itself to display the new state.

Your app needs to manage changes to two kinds of data:

Managing UI values and model objects

- User interface values, like Boolean flags to show or hide views, text field text, slider or picker values.
- Data model objects, often collections of objects that model the app's data, like daily logs of completed exercises.

Property Wrappers

Property wrappers encapsulate a value or object in a structure with two properties:

- `wrappedValue` is the underlying value or object.
- `projectedValue` is a binding to the wrapped value or a projection of the object that creates bindings to its properties.

Swift syntax lets you write just the name of the property, like `showHistory`, instead of `showHistory.wrappedValue`. And, its binding is `$showHistory` instead of `showHistory.projectedValue`.

SwiftUI provides tools — mostly property wrappers — to create and modify the single source of truth for values and for objects:

- User interface *values*: Use `@State` and `@Binding` for values like `showHistory` that affect the view's appearance. The underlying type must be a *value* type like `Bool`, `Int`, `String` or `Exercise`. Use `@State` to create a source of truth in one view, then pass a `@Binding` to this property to subviews. A view can access built-in `@Environment` values as `@Environment` properties or with the `environment(_:_:)` view modifier.

- Data model *objects*: For objects like `HistoryStore` that model your app's data, use either `@StateObject` with `@ObservedObject` or `environmentObject(_:)` with `@EnvironmentObject`. The underlying object type must be a *reference* type — a class — that conforms to `ObservableObject`, and it should *publish* at least one value. Then, either use `@StateObject` and `@ObservedObject` or declare an `@EnvironmentObject` with the same type as the environment object created by the `environmentObject(_:)` view modifier.

While prototyping your app, you can model your data with structures and use `@State` and `@Binding`. When you've worked out how data needs to flow through your app, you can refactor your app to accommodate data types that need to conform to `ObservableObject`.

This is what you'll do in this chapter to consolidate your understanding of how to use these property wrappers.

Saving/Persisting App or Scene State

There are two other property wrappers you've used. `@AppStorage` wraps `UserDefault` values. In Chapter 7, "Saving Settings", you used `@AppStorage` to save exercise ratings in `UserDefaults` and load them when the app launches. In the same chapter, you used `@SceneStorage` to save and restore the state of scenes — windows in the iPad simulator, each showing a different exercise.

Managing UI State Values

`@State` and `@Binding` value properties are mainly used to manage the state of your app's user interface.

A view is a structure, so you can't change a property value unless you wrap it as a `@State` or `@Binding` property.

The view that owns a `@State` property is responsible for initializing it. The `@State` property wrapper creates persistent storage for the value outside the view structure and preserves its value when the view redraws itself. This means initialization happens exactly once.

You already got lots of practice with `@State` and `@Binding` in Chapter 5, "Moving Data Between Views" and Chapter 6, "Observing Objects":

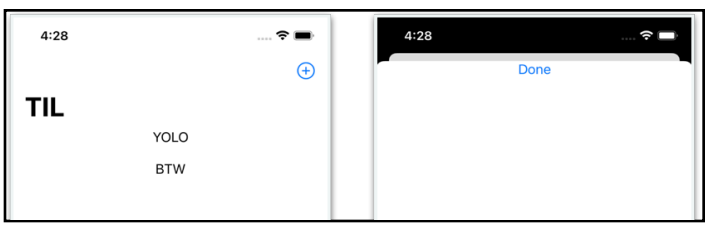
- `selectedTab` controls `TabView`.
- `showHistory`, `showSuccess`, `showTimer`, `timerDone` show or hide views.
- `rating` and `timeRemaining` values must be able to change.

In the challenge for Chapter 6, "Observing Objects", you used `@State` and `@Binding` to manage changes to `HistoryStore`. That was just an exercise to demonstrate it's possible, and it's one approach you can take to prototyping. For most apps, your final data model will involve `ObservableObject` classes.

Managing ThingStore With @State & @Binding

TIL is a very simple app, making it easy to examine different ways to manage the app's data. First, you'll manage `ThingStore` the same way as any other mutable *value* you share between your app's views.

➤ Open **ContentView.swift**, in **Live Preview** tap the `+`:



Starter TIL

TIL uses a Boolean flag, `showAddThing`, to show or hide `AddThingView`. It's a `@State` property because its value changes when you tap the `+`, and `ContentView` owns it.

➤ Add this line to `ContentView`:

```
@State private var myThings = ThingStore()
```

You'll add items to `myThings.things`, so `myThings` must be a wrapped property. In this case, it's `@State`, because `ContentView` owns it and initializes it.

➤ Now, delete the temporary array:

```
let tempThings = ["YOLO", "BTW"]  // delete this line
```

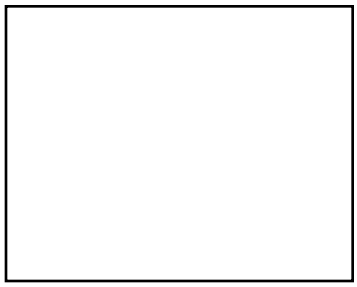
You'll store strings in `myThings.things`, so you no longer need this array.

➤ Then, update the `ForEach` argument:

```
ForEach(myThings.things, id: \.self) { thing in
```

You loop over the `things` array instead of `tempThings`.

When There Are No Things



Nothing to see here

Now, there's nothing to show in Live Preview because `myThings` initializes with an empty `things` array. It's a better user experience if you display a message, instead of this blank page, the first time your user launches your app.

➤ In **ContentView.swift**, add this code at the top of the `VStack`, before the `ForEach` line:

```
if myThings.things.isEmpty {
  Text("Add acronyms you learn")
    .foregroundColor(.gray)
}
```



First-time empty-array screen

You give your users a hint of what they can do with your app. The text is grayed out so they know it's just a placeholder until they add their own data.

`AddThingView` needs to modify `myThings`, so you need a `@Binding` in `AddThingView`.

➤ In **AddThingView.swift**, add this property to `AddThingView`:

```
@Binding var someThings: ThingStore
```

You'll soon pass this binding from `ContentView`.

➤ You'll also add a text field to get the user's input, but for now, just to have something happen when you tap **Done**, add this line to the button action, before you dismiss this sheet:

```
someThings.things.append("FOMO")
```

You append a specific string to the array.

➤ Fix this view's `previews`:

```
AddThingView(someThings: .constant(ThingStore()))
```

You create a binding for the constant initial value of `ThingStore`.

➤ Now, go back to **ContentView.swift** and fix the call to `AddThingView()`:

```
AddThingView(someThings: $myThings)
```

You pass a binding to the `ContentView` `@State` property to `AddThingView`.

> **Note**: Passing a binding gives the subview write access to everything in `ThingStore`. In this case, `ThingStore` has only the `things` array but, if it had more properties and you wanted to restrict write access to its `things` array, you could pass `$myThings.things` — a binding to only the `things` array. You'd need to initialize an array of `String` for the preview of `AddThingView`.

➤ In **Live Preview**, tap + then tap **Done**:



Adding a string works.

Great, you've got data flowing from the `AddThingView` to `ContentView` via `ThingStore`! Now, to get input from your user, you'll add a `TextField` to `AddThingView`.

➤ First, pin the preview of `ContentView` so it's there when you're ready to test your `TextField`.

Using a TextField

Many UI controls work by binding a parameter to a `@State` property of the view: These include `Slider`, `Toggle`, `Picker` and `TextField`.

To get user input via a `TextField`, you need a mutable `String` property to store the user's input.

➤ In **AddThingView.swift**, add this property to `AddThingView`:

```
  @State private var thing = ""
```

It's a `@State` property because it must persist when the view redraws itself. `AddThingView` owns this property, so it's responsible for initializing `thing`. You initialize it to the empty string.

➤ Now, add your `TextField` in the `VStack`, above the **Done** button:

```
  TextField("Thing I Learned", text: $thing)  // 1
    .textFieldStyle(.roundedBorder)  // 2
    .padding()  // 3
```

1. The label "Thing I Learned" is the *placeholder* text. It appears grayed out in the `TextField` as a hint to the user. You pass a binding to `thing` so `TextField` can set this value to what the user types.
2. You dress up this `TextField` with a rounded border.
3. You add padding so there's some space from the top of the view and also to the button.

➤ Then, edit what the button action appends:

```
if !thing.isEmpty {
  someThings.things.append(thing)
}
```

Instead of `"FOMO"`, you append the user's text input to your `things` array after checking it's not the empty string.

➤ In the `ContentView` **Live Preview**, tap +. Type an acronym like **YOLO** in the text field. It automatically capitalizes the first letter, but you must hold down the **Shift** key for the rest of the letters. Tap **Done**:



TextField input

`ContentView` displays your new acronym.

Improving the UX

You can improve your users' experience by adjusting how the text field handles their input. Acronyms should appear as all caps, but it's easy to forget to hold down the **Shift** key. Sometimes the app auto-corrects your acronym: FTW to FEW or FOMO to DINO. And focus! Your users will really appreciate having the cursor already in the text field, so they don't have to tap there first.

➤ Add these modifiers to `TextField`:

```
.autocapitalization(.allCharacters)
.disableAutocorrection(true)
```

> **Note**: `String` has an `uppercased()` modifier to automatically convert text to upper case, so you could modify the `Text` view in `ContentView` instead. But automatically capitalizing in the text field is a more immediate UX improvement.

➤ To place the cursor in `TextField`, start by adding this property to `AddThingView`:

```
@FocusState private var thingIsFocused: Bool
```

This is like the `@State` Boolean properties you use to show or hide modal sheets or alerts, but you use a value wrapped by `@FocusState` to place the cursor in an associated view.

➤ To use `thingIsFocused`, add these modifiers to `TextField`:

```
.focused($thingIsFocused)
.onAppear { thingIsFocused = true }
```

> **Note**: The value wrapped by `@FocusState` doesn't have to be `Bool`. You can use any value type, including enumerations, with `focused(_:equals:)`.

When `TextField` appears, you set `thingIsFocused` to `true`, so the `focused` modifier places the cursor in `TextField`.

➤ In **Live Preview**, tap +, type **icymi** (don't touch the **Shift** key!) then tap **Done**:



Auto-focus, auto-cap

The cursor is already in the text field and, even if you type in lower case, your input is all caps!

Accessing Environment Values

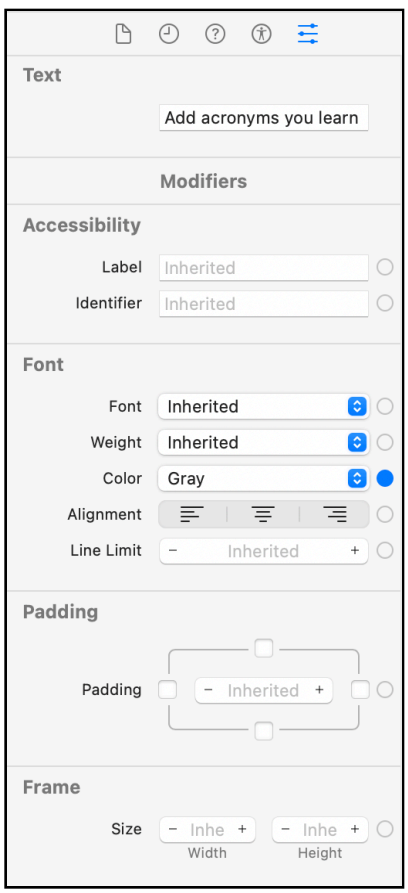
A view can access many environment values like `accessibilityEnabled`, `colorScheme`, `lineSpacing`, `font` and `dismiss`. Apple's SwiftUI documentation has a full list of environment values (https://apple.co/37cOxak).

A view's environment is a kind of inheritance mechanism. A view inherits environment values from its ancestor views, and its subviews inherit its environment values.

➤ To see this, open **ContentView.swift** and click anywhere in this line:

```
Text("Add acronyms you learn")
```

➤ With the canvas showing, open the Attributes inspector:



Text view attributes: Many are inherited.

Accessibility, Font, Weight, Line Limit, Padding and Frame Size are *Inherited*. Font Color would also be inherited if the `foregroundColor` modifier hadn't set it to `gray`.

A view can override an inherited environment value. It's common to set a default font for a stack then override it for the text in a subview of the stack. You did this in Chapter 3, "Prototyping the Main View", when you made the first page number larger than the others:

```
HStack {
  Image(systemName: "1.circle")
    .font(.largeTitle)
  Image(systemName: "2.circle")
  Image(systemName: "3.circle")
  Image(systemName: "4.circle")
}
.font(.title2)
```

Modifying Environment Values

`AddThingView` already uses the `dismiss` environment value, declared as a view property the same as in HIITFit's `SuccessView`. But, you can also set environment values by modifying a view.

Setting an environment variable is yet another way to auto-capitalize your acronyms.

➤ In **AddThingView.swift**, comment out `.autocapitalization(.allCharacters)`

➤ Then, in **TILApp.swift**, add this modifier to `ContentView()`:

```
.environment(\.textCase, .uppercase)
```

You set `uppercase` as the default value of `textCase` for `ContentView` and all its subviews.

> **Note**: The shortcut syntax `textCase(.uppercase)` also works, but the `.environment` syntax highlights the fact that `textCase` is an environment value.

➤ To see it in Live Preview, also add this modifier in **ContentView.swift** to `ContentView()` in `previews`.

➤ In **Live Preview**, tap +, then type an acronym.



Automagic uppercase

Your strings are automatically converted to uppercase.

The environment value applies to *all* text in your app, which looks a little strange. No problem — you can override it.

➤ In **AddThingView.swift**, add this modifier to the `VStack`:

```
.environment(\.textCase, nil)
```

You set the value to `nil`, so none of the text displayed by this `VStack` is converted to uppercase.

➤ In the `ContentView` **Live Preview**, tap +, type **icymi** then tap **Done**:



No upper case conversion in AddThing

Now, the button label and placeholder text are back to normal. The `uppercase` environment default still converts your input to all caps on the main screen.

Managing Model Data Objects

`@State`, `@Binding` and `@Environment` only work with value data types. Simple built-in data types like `Int`, `Bool` or `String` are useful for defining the state of your app's user interface.

You can use custom value data types like `struct` or `enum` to model your app's data. And, you can use `@State` and `@Binding` to manage updates to these values, as you did earlier in this chapter.

Most apps also use *classes* to model data. SwiftUI provides a different mechanism to manage changes to class *objects*: `ObservableObject`, `@StateObject`, `@ObservedObject` and `@EnvironmentObject`. To practice using `@ObservedObject`, you'll refactor TIL to use `@StateObject` and `@ObservedObject` to update `ThingStore`, which conforms to `ObservableObject`. You'll see a lot of similarities, and a few differences, to using `@State` and `@Binding`.

> **Note**: You *can* wrap a class object as a `@State` property, but its "value" is its address in memory, so dependent views will redraw themselves only when its address changes — for example, when the app reinitializes it.

Class & Structure

Actually, `ThingStore` *should* be a class, not a structure. `@State` and `@Binding` work well enough to update the `ThingStore` source of truth *value* in `ContentView` from `AddThingView`. But `ThingStore` isn't the most natural use of a structure. For the way your app uses `ThingStore`, a class is a better fit.

A class is more suitable when you need *shared* mutable state like a `HistoryStore` or `ThingStore`. A structure is more suitable when you need multiple *independent* states like `ExerciseDay` structures.

For a class object, change is normal. A class object expects its properties to change. For a structure instance, change is exceptional. A structure instance requires advance notice that a method might change a property.

A class object expects to be shared, and any reference can be used to change its properties. A structure instance lets itself be copied, but its copies change independently of it and of each other.

You'll find out more about classes and structures in Chapter 15, "Structures, Classes & Protocols".

Managing ThingStore With @StateObject & @ObservedObject

You've already used `@EnvironmentObject` in Chapter 6, "Observing Objects", to avoid passing `HistoryStore` through `WelcomeView` to reach `HistoryView`.

To use it as an `@EnvironmentObject`, you converted `HistoryStore` to a class that conforms to `ObservableObject`. This is also the first step before you can use `@StateObject` and `@ObservedObject` with `ThingStore`. Once that's done, you'll create it as a `@StateObject` and pass it to a subview that uses it as an `@ObservedObject`. Sounds a lot like "create a `@State` property and pass its `@Binding`", doesn't it?

> **Note**: You can pass a `@State` value or a `@StateObject` to a subview as a `@Binding` or `@ObservedObject` property, even if that subview needs only read access. This enables the subview to redraw itself whenever the `@State` value or `ObservableObject` changes. You did this with `selectedTab` in `HeaderView`, in Chapter 5, "Moving Data Between Views".

➤ In **ContentView.swift**, replace the `ThingStore` structure with the following:

```
final class ThingStore: ObservableObject {
  @Published var things: [String] = []
}
```

Just like you did with `HistoryStore`, you make `ThingStore` a class instead of a structure, then make it conform to `ObservableObject`. You mark this class `final` to tell the compiler it doesn't have to check for any subclasses overriding properties or methods.

Like `HistoryStore`, `ThingStore` publishes its array of data. A view *subscribes* to this publisher by declaring it as a `@StateObject`, `@ObservedObject` or `@EnvironmentObject`. Any change to `things` notifies subscriber views to redraw themselves.

You used `@EnvironmentObject` in Chapter 6, "Observing Objects". In HIITFit, `ExerciseView` and `HistoryView` declared a dependency on a `HistoryStore` object:

```
@EnvironmentObject var history: HistoryStore
```

If a view uses an `@EnvironmentObject`, you must create the model object by calling the `environmentObject(_:)` modifier on an *ancestor* view.

You first created the `HistoryStore` object in `ContentView`, applying the modifier to the `TabView`:

```
TabView(selection: $selectedTab) {
...
}
.environmentObject(HistoryStore())
```

Then, in Chapter 8, "Saving History Data", you elevated its initialization up one level to `HIITFitApp` and declared it as a `@StateObject`.

> **Note**: Initializing `HistoryStore` in the `environmentObject` modifier works while you're prototyping. To make sure the app never reinitializes an environment object, declare and initialize it as a `@StateObject`, then pass the property in the `environmentObject` modifier.

In TIL, `AddThingView` will use an `@ObservedObject`, so you must instantiate the model object as a `@StateObject` in an ancestor view, then pass it as a parameter to its subviews. The owning view creates the `@StateObject` exactly once.

➤ In `ContentView`, replace `@State private var myThings = ThingStore()` with this line:

```
@StateObject private var myThings = ThingStore()
```

`ThingStore` is now a class, not a structure, so you can't use the `@State` property wrapper. Instead, you use `@StateObject`. The `@StateObject` property wrapper ensures `myThings` is instantiated only once. It persists when `ContentView` redraws itself.

➤ In the call to `AddThingView(someThings:)`, remove the binding symbol `$`:

```
AddThingView(someThings: myThings)
```

As a class object, `myThings` is already a reference.

➤ In **AddThingView.swift**, replace `@Binding` in `AddThingView` with `@ObservedObject`:

```
@ObservedObject var someThings: ThingStore
```

➤ And fix its `previews`:

```
AddThingView(someThings: ThingStore())
```

The argument isn't a binding anymore.

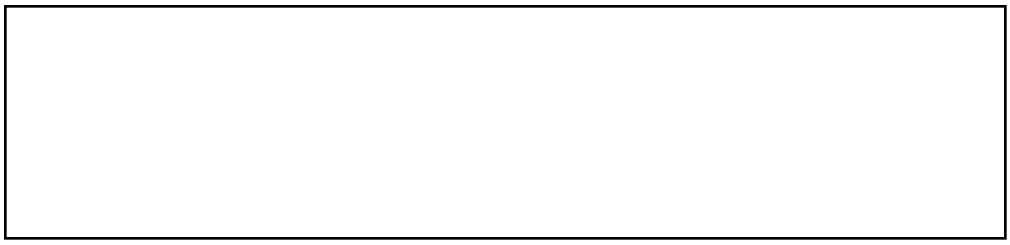
➤ In **Live Preview**, tap +, type **yolo** then tap **Done**:



TIL in action

No surprise: The app still works the same as before.

SwiftUI App Design Pattern

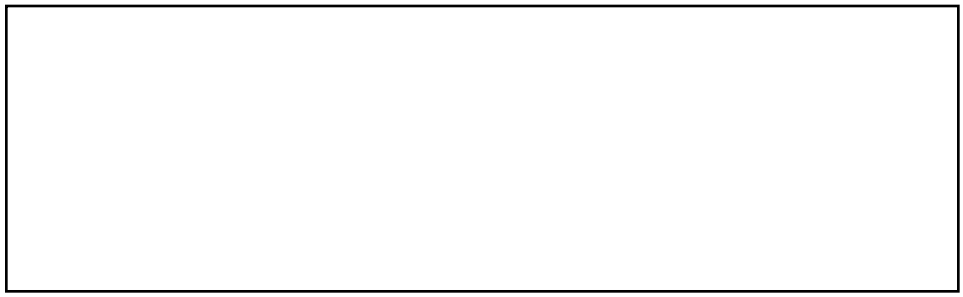


Model-View-Controller

You may be familiar with Model-View-Controller (**MVC**) architecture for apps in other settings, like web apps. Your data model knows nothing about how your app presents it to users. The view doesn't own the data, and the controller mediates between the model and the view.

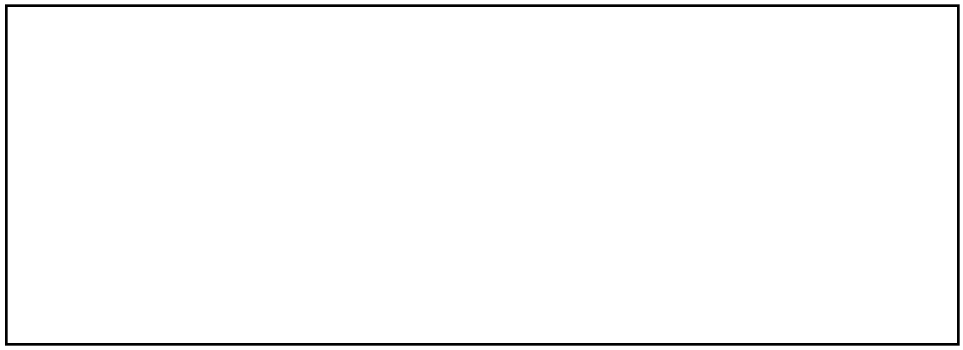
SwiftUI apps don't need a controller. Views subscribe to observable objects and update themselves when observed values change.

A common design pattern sets up a model as a `@StateObject` or `@EnvironmentObject`, as you've been doing in HIITFit and TIL.



Model-View

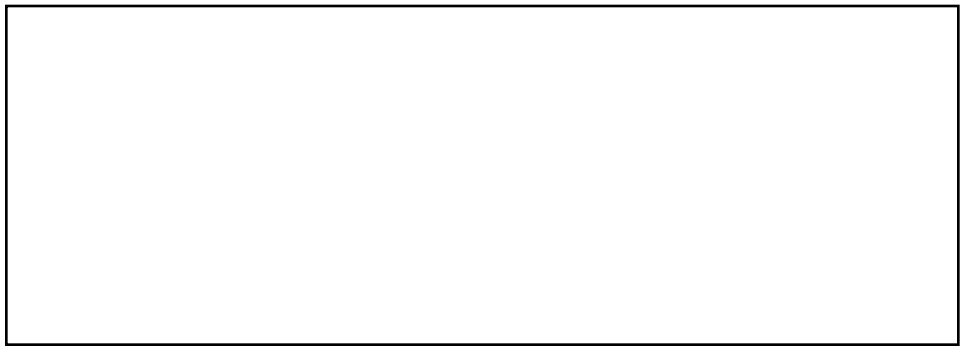
When a SwiftUI app's views display a collection of objects or values, its model manages the data collection. In simple apps like HIITFit and TIL, this is the model's only job. So the model's name often includes the word "Store".



MV in HIITFit

HIITFit's model, `HistoryStore`, saves and loads the user's exercise history. The model consists of the `Exercise` and `ExerciseDay` structures. `HistoryStore` publishes the `exerciseDays` array. `ExerciseView` and `HistoryView` subscribe to `HistoryStore`.

The `ExerciseView`'s tap-Done event updates the `exerciseDays` array, which changes the state of `HistoryView`.



MV in TIL

TIL's model, `ThingStore`, saves the user's array of acronyms. An acronym is simply a `String` and the model publishes the `things` array. `ContentView` and `AddThingView` subscribe to `ThingStore`. The `AddThingView`'s tap-Done event updates the `things` array, which changes the state of `ContentView`.

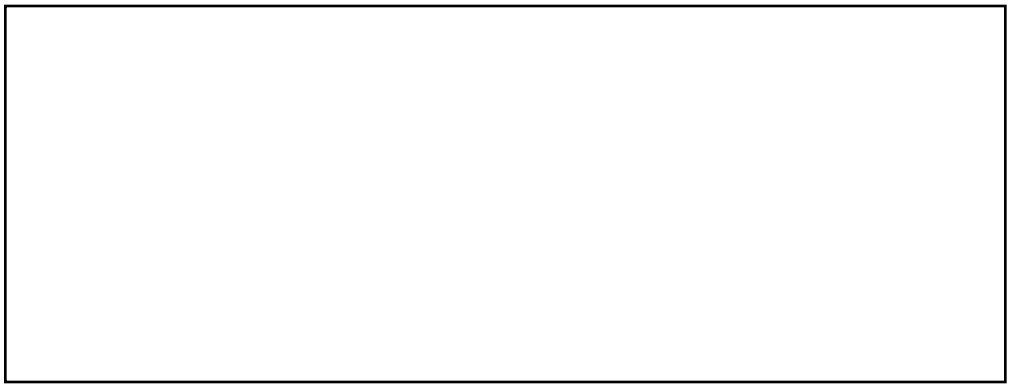
> **Note**: User actions can initiate network activity that updates the model. In the Section 3 app, TheMet, the user enters a query term, causing the model to download data from The Metropolitan Museum of Art, New York (https://www.metmuseum.org) and decode the data into its published array of `Object` values.

Wrapping Up Property Wrappers

Here's a summary to help you wrap your head around property wrappers.

First, decide whether you're managing the state of a value or the state of an object. Values are mainly used to describe the state of your app's user interface. If you can model your app's data with value data types, you're in luck because you have a lot more property wrapper options for working with values.

But at some level, most apps need reference types to model their data, often to add or remove items from a collection.



Property wrappers for values and objects

Wrapping Values

`@State` and `@Binding` are the workhorses of value property wrappers. A view owns the value if it doesn't receive it from any parent views. In this case, it's a `@State` property — the single source of truth. When a view is first created, it initializes its `@State` properties. When a `@State` value changes, the view redraws itself, resetting everything except its `@State` properties.

The owning view can pass a `@State` value to a subview as an ordinary read-only value or as a read-write `@Binding`.

When you're prototyping an app and trying out a subview, you might write it as a stand-alone view with only `@State` properties. Later, when you fit it into your app, you just change `@State` to `@Binding` for values that come from a parent view.

Your app can access the built-in `@Environment` values. An environment value persists within the subtree of the view you attach it to. Often, this is simply a container like `VStack`, where you use an environment value to set a default like font size.

> **Note**: You can also define your own custom environment value — for example, to expose a view's property to ancestor views. This is beyond the scope of this book, but check out SwiftUI by Tutorials, Chapter 9, "State & Data Flow – Part II" (https://bit.ly/3IffPOt).

You can store a few values in the `@AppStorage` or `@SceneStorage` dictionary. `@AppStorage` values are in `UserDefaults`, so they persist after the app closes. You use a `@SceneStorage` value to restore the state of a scene when the app reopens. You can think of scenes as multiple windows on an iPad.

Wrapping Objects

When your app needs to change and respond to changes in a reference type, you create a class that conforms to `ObservableObject` and publishes the appropriate properties. In this case, you use `@StateObject` and `@ObservedObject` in much the same way as `@State` and `@Binding` for values. You instantiate your publisher class in a view as a `@StateObject` then pass it to subviews as an `@ObservedObject`. When the owning view redraws itself, it doesn't reset its `@StateObject` properties.

If your app's views need more flexible access to the object, you can lift it into the environment of a view's subtree, still as a `@StateObject`. You **must** instantiate it here. Your app will crash if you forget to create it. Then you use the `.environmentObject(_:)` modifier to attach it to a view. Any view in the view's subtree can *subscribe* to the publisher object by declaring an `@EnvironmentObject` of that type.

To make an environment object available to every view in your app, attach it to the root view when the `App` creates its `WindowGroup`.

Key Points

- Every data value or object that can change needs a single source of truth and a mechanism to enable views to update it.
- Use `@State` and `@Binding` to manage changes to user interface values.
- Access environment values as `@Environment` view properties or by using the `environment` view modifier.
- Use `@StateObject` and `@ObservedObject` to manage changes to data model objects. The object type must conform to `ObservableObject` and should publish at least one value.
- If only a few subviews need access to an `ObservableObject`, instantiate it as a `@StateObject` then pass it in the `environmentObject` view modifier. Declare an `@EnvironmentObject` property in any subviews that need access to it.
- When prototyping your app, you can use `@State` and `@Binding` with structures that model your app's data. When you've worked out how data needs to flow through your app, refactor your app to accommodate data types that need to conform to `ObservableObject`.
- A commonly used architecture for SwiftUI apps is the Model-View pattern, where the model is an `ObservableObject`. Changes to the model's published properties cause updates to the view.

Chapter 12: Apple App Development Ecosystem

By Audrey Tam

Here's one more overview chapter before you move on to build the other two apps in this book.

While building HIITFit, you learned a lot about Xcode, Swift and SwiftUI at a detailed level. In the previous chapter, you got a kind of "balcony view" of how the various property wrappers help you manage the state of your app's data. This chapter provides a bird's eye view of the whole Apple app development ecosystem, demystifying many of the terms you might hear when iOS developers get together for a chat. You'll start to build your own mental model of how all the parts fit together, creating your own framework for all the new things that Apple adds every year.

A Brief History of SwiftUI

You've been using SwiftUI to build HIITFit, but this is only the most recent app development paradigm from Apple.

Apple announced **SwiftUI** at its World Wide Developers Conference in June 2019. SwiftUI builds on the Swift programming language, which Apple announced in June 2014. SwiftUI is a **Domain Specific Language** (DSL), written in Swift using these new Swift features:

- Property wrappers, like `@State` to monitor the state of properties.
- Result builders, like `@ViewBuilder` to create view hierarchies.
- Opaque result types, like `some View` to avoid explicitly writing out the view hierarchy.

Swift creates faster, safer apps than Objective-C and is more *protocol-oriented* than object-oriented. Chapter 15, "Structures, Classes & Protocols", explains the difference between class inheritance and protocols.

By January 2018, the Redmonk Programming Language Rankings (https://bit.ly/3rsZmwq) ranked Swift and Objective-C in a tie at number 10.

Objective-C entered Apple history when Apple bought NeXT in 1997, which also brought Steve Jobs back to Apple. Jobs had resigned from Apple in 1985 after losing a boardroom battle with CEO John Sculley over the future of the Macintosh computer. Jobs, with five other former Apple executives, then founded **NeXT Computers**.

A brief history of SwiftUI

The NeXTSTEP operating system, written in Objective-C, formed the basis of **Mac OS X**, released in 2001. Apple provided the **Cocoa API** for developers to create apps for OS X. Cocoa consists of three frameworks reflecting the Model-View-Controller principles: **Core Data**, **AppKit** and **Foundation**. The "NS" prefix in AppKit and Foundation acknowledges their NeXTSTEP heritage.

Apple announced the **iPhone** in 2007 and the **iPhone SDK** (Software Development Kit) in 2008. This included **Cocoa Touch**, with **UIKit** replacing AppKit. Now called the **iOS** SDK, it helps you create apps that appear and behave the way users expect.

Fun Facts

- The first World Wide Web server was a NeXT Computer, and id Software developed the video games Doom and Quake on machines running the NeXT operating system **NeXTSTEP**. In 1996, NeXT Software, Inc. released **WebObjects**, a framework for Web application development. Apple used WebObjects to build and run the Apple Store, MobileMe services and the iTunes Store.
- Cocoa != Java for kids: Before Jobs returned to Apple, the Apple Advanced Technology Group created KidSim, an app to teach kids to program. KidSim programs were embedded in web pages to run, so they renamed and trademarked the app as **Cocoa** — "Java for kids". The Cocoa program was one of the many axed in 1997, and Apple reused the name for the OS X API to avoid the delay of registering a new trademark.
- While developing the iPhone, Steve Jobs didn't want non-Apple developers to build native iPhone apps. They were supposed to be content making web applications for Safari. This stance changed in response to a backlash from developers, and the iPhone SDK was released on March 6, 2008.

SwiftUI vs. UIKit

Although you've used only SwiftUI to create HIITFit, UIKit has a lot of resources that can help you add features to your app or fine-tune how it looks and functions.

Most Popular Episode

"SwiftUI vs. UIKit" (https://bit.ly/3V0lwEY) is the most popular free episode at kodeco.com. Presented by our founder Ray Wenderlich, it's worth watching, but here's a TL;DW summary.

> **Note**: Like a lot of the content on this site, this episode is aimed at people who want to work for iOS app development companies. If this isn't you, skip to the next section.

Three reasons why there are still more developers using UIKit than SwiftUI:

1. SwiftUI only works on iOS 13 or later. Some companies still need to support iOS 12 or earlier, so they can't switch to SwiftUI quite yet.
2. SwiftUI is still not as mature as UIKit. Apple released UIKit in 2008, and it built on macOS AppKit, which came from NeXTSTEP, so there was a lot of time to get things right. SwiftUI still has missing features or rough edges, so some companies want to give SwiftUI a little more time to mature.
3. Many companies have already written their apps using UIKit, and it would simply be too much work at this point to rewrite the entire thing in SwiftUI, so a lot of that old UIKit legacy code will remain.



SwiftUI or UIKit?

Q: Which should you learn: SwiftUI or UIKit?

A: *If you're serious about being a professional iOS developer*, our recommendation is to learn both SwiftUI and UIKit. If you end up working at a company that already has shipped an iOS app, there's a high chance that it's been made with UIKit. So if you want to work at one of those companies, it's important for you to be able to work with those codebases, too. We created a special and optional learning path, called "iOS User Interfaces with UIKit" (https://bit.ly/3zRe5Y2). If you need to learn UIKit development for your job, you should definitely check that out. But if you only care about SwiftUI, you can safely skip it.

It's not all or nothing: It's possible to make a certain part of your app with SwiftUI and the rest with UIKit. As companies begin to transition from UIKit to SwiftUI, we expect to see many codebases with a mixture of both SwiftUI and UIKit code in the years ahead.

Thanks, Ray! That's the perfect segue into the next section…

Integrating New & Old

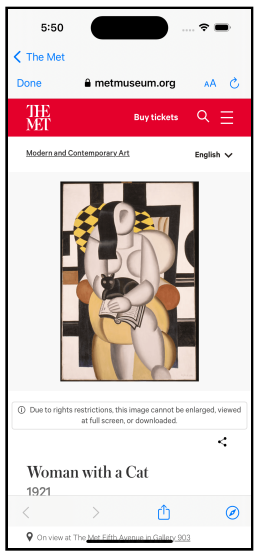
Apple always provides support for developers to transition to new things. The Carbon API enabled developers to port "classic" Mac OS apps to OS X. Bridging headers enable developers to use Objective-C code in Swift apps and vice versa.

Developers can still create Objective-C apps without Swift and Swift apps without SwiftUI. UIKit has many more features than SwiftUI and provides much more control over the appearance and operation of user interface elements.

But no FOMO (fear of missing out)! You can use UIKit views in your SwiftUI apps:

Integrating a UIKit view

You can also create and manage `UIViewController` objects: You'll use `SFSafariViewController` in Section 3, "Your Third App".



SFSafariViewController in TheMet app

It's also really easy to use `UIColor` to access system and UI element colors, and you can create an `Image` view from a `UIImage`.

Apple Developer

Despite Steve Jobs' initial intentions, Apple would like everyone to be an Apple developer. Your needs and interests might be shared by a few other people or by a lot of other people. But maybe not by professional iOS developers. If you create an app *you* need or want, it becomes available to those other people too. Even better if your app uses some technology that only works on the newest Apple gadgets, so they have to upgrade to use your app. ;]

Apple provides tons of resources at developer.apple.com to help you become a developer and stay up to date, including:

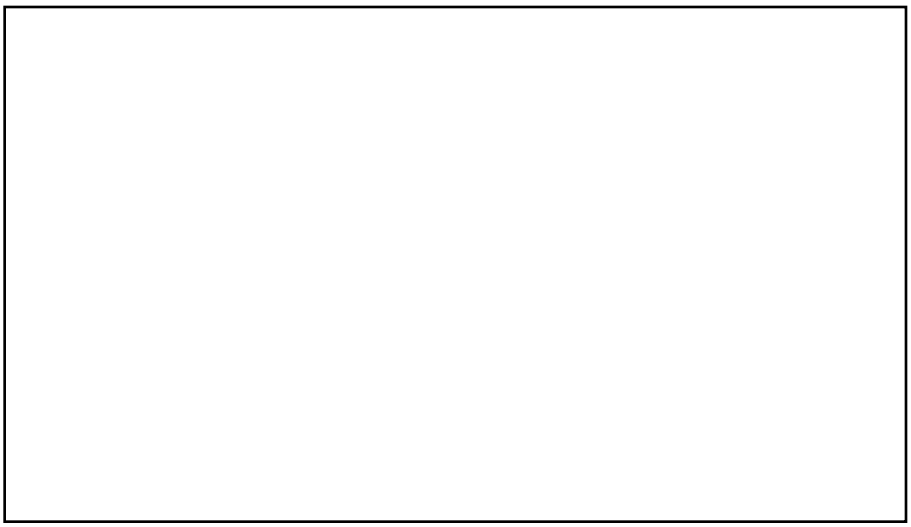
- Apple Developer Documentation (https://apple.co/3v9YVcL) and Xcode's **Help ▸ Developer Documentation** (**Shift-Command-0**)
- Apple's Human Interface Guidelines (https://apple.co/3cgQJPk) and Xcode's **Help ▸ Human Interface Guidelines** (**Shift-Command-H**)

WWDC

Every June, Apple holds the 5-day **World Wide Developers Conference** — WWDC, often pronounced dub-dub-dee-cee or shortened to dub-dub. The keynote on day 1 shows off all the features planned for the new versions of iOS, macOS and all the other OSes. These launch later in the year, around September or October.

For iOS developers, the more important presentation is the **Platforms State of the Union** later on day 1, where you get your first look at the APIs for adding these new features to your apps, as well as improvements to developer tools like Xcode. During the rest of the week, you can watch presentations that introduce and dive deeper into the new features.

Apple provides the multi-platform Apple Developer app (https://apple.co/3eoIfs1), where you can view WWDC videos and bookmark or download your favorites.



Apple Developer app

If you're a paid member of the **Apple Developer Program**, you can download the beta versions of Xcode and the operating systems and immediately start exploring all the new things. Your goal is to include the new features in new or existing apps, all set to go into the App Store when the new iOS launches.

A word of caution: The WWDC presenters use a special in-house version of Xcode. It's different from the Xcode beta you can download, so not everything you see in the presentations actually works in beta 1. Or beta 2. Or ever. Details of the API often change by the time Apple releases the final version, and some promised features get postponed or quietly disappear.

Platforms

Using SwiftUI to build new iOS apps makes it easier to create similar apps on Apple's other platforms: macOS, watchOS and tvOS. It's not that your iOS app will "just work" on another platform. It probably won't.

You can use many SwiftUI views on other platforms, but how they look or function might be a little different. And other platforms have views that don't exist for iOS. Also, some features of your iOS app won't make sense on a more stationary platform like tvOS or on a smaller screen like watchOS.

But, the way you assemble a SwiftUI app remains the same, no matter which platform you're targeting. Apple expresses it this way: Learn once, apply anywhere.

Mac Catalyst is Apple's program to make it easier to create a native Mac app from an iPad app. You turn on Mac Catalyst in the iPad app's project settings, then modify the user interface to be more Mac-like. Some iPad UI elements aren't quite right for the Mac user experience, and some iPad frameworks just aren't available in macOS. Your code controls what to include using this compiler directive:

```
#if targetEnvironment(macCatalyst)
...
#endif
```

Check out Apple's tutorial for Mac Catalyst (https://apple.co/3qPaen7). For an even more in-depth look, browse our book Catalyst by Tutorials (https://bit.ly/32ppGwM).

> **Note**: What about **Apple Silicon**? It's Apple's program to design and manufacture its own Mac processors. Since its launch in 1984, the Mac has used Motorola 68000, PowerPC and Intel CPU chips. The Apple M1 Chip integrates Apple's new CPU with its new GPU, neural engine and more. You can install Rosetta 2 on an Apple Silicon Mac to run apps written for Intel Macs.

Frameworks

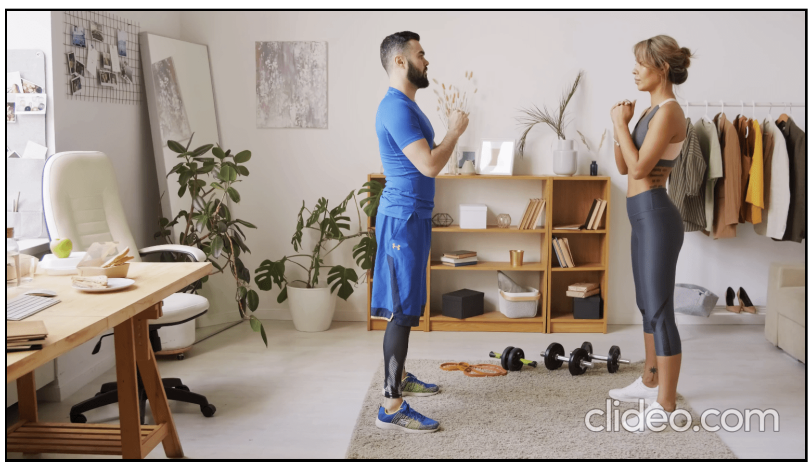
The SDK has *a lot* of frameworks, and Apple adds new ones every year. The ones every app needs are modernized versions of the original Cocoa:

- Core Data or some other database technology for data persistence.
- SwiftUI and/or UIKit for user interface.
- Foundation to manipulate and coordinate data and views.

> **Note**: Core Data is a massive topic and, if you'd like to learn more, we have a book, Core Data by Tutorials (https://bit.ly/39lo2k3) and video courses Beginning Core Data (https://bit.ly/2OGjuwG) and Intermediate Core Data (https://bit.ly/3bE2H6z) to help you on your way.

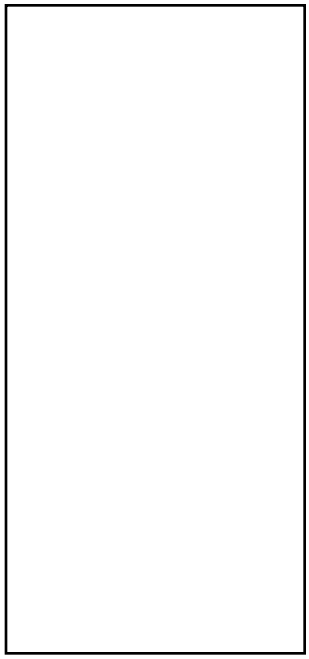
The apps in this book use these frameworks:

In HIITFit, `AVKit` for the `AVPlayer` in `VideoPlayer`:



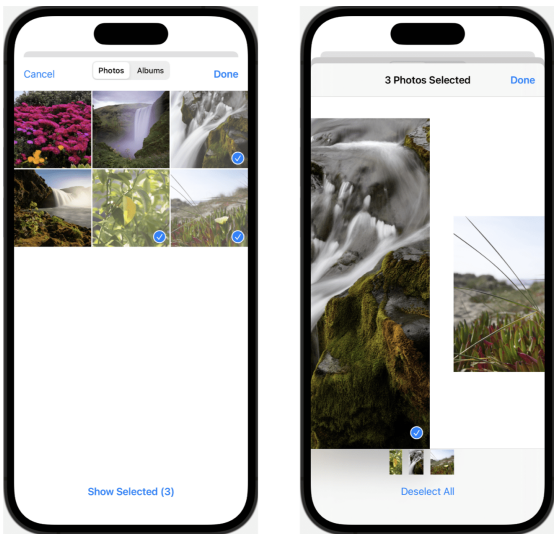
AVKit for AVPlayer in VideoPlayer

Also in HIITFit, `Charts` in `HistoryView`:



Charts in HistoryView

In Cards, `PhotosUI` with `PhotosPicker`:



PhotosUI with PhotosPicker

These next two frameworks are important, but beyond the scope of this book:

- **Accessibility**: Paying attention to accessibility is one of the easiest ways to grow the audience for your apps. See SwiftUI by Tutorials, Chapter 12, "Accessibility" (https://bit.ly/32oFTCs) and our three-part tutorial "iOS Accessibility in SwiftUI Tutorial" (starting at Part 1 (https://bit.ly/2WYD9sI)).
- **Combine**: This framework is a major change to the way iOS apps handle concurrency. See Combine: Asynchronous Programming in Swift (https://bit.ly/3KuGurx) and the video course Reactive Programming in iOS with Combine (https://bit.ly/3mnu1Oq).

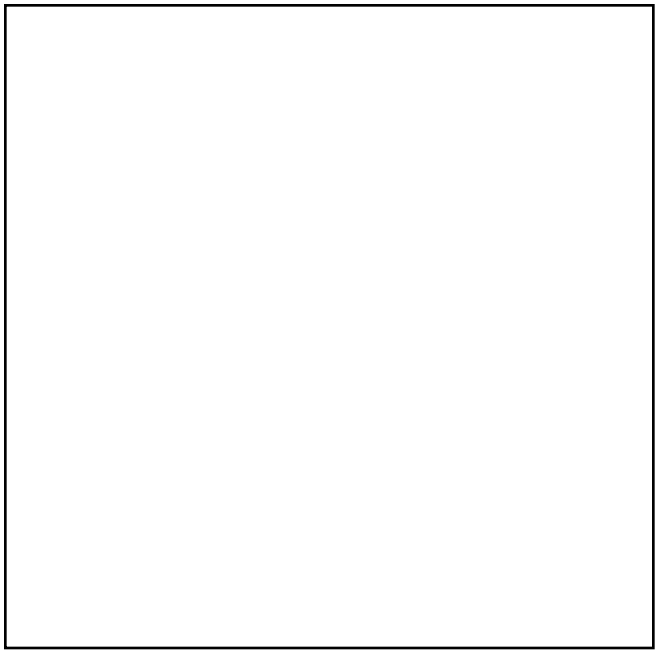
Some other frameworks you might want to explore:

`WidgetKit` to add widgets, like this one from our video course SwiftUI Charts for WidgetKit (https://bit.ly/3EIRrnV):



SwiftUI Charts widget

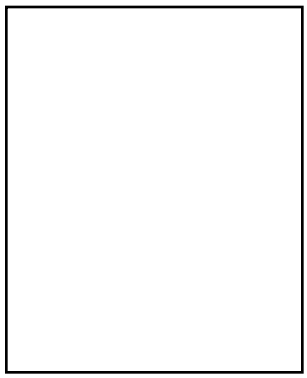
`MapKit` to add maps, user location, routing or overlay views to your apps:



Newest MapKit tutorials

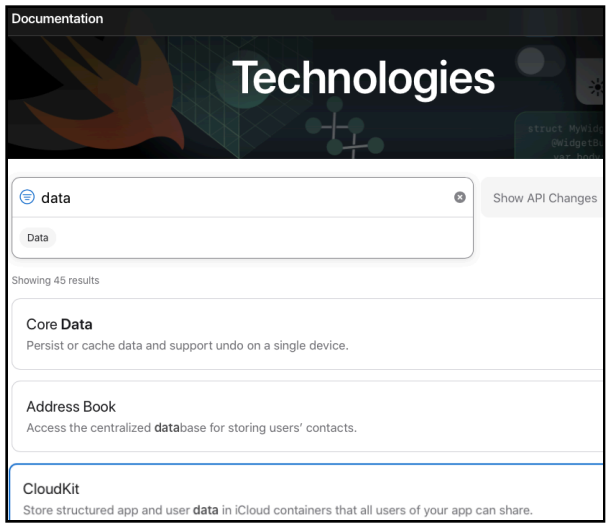
`WatchKit` to create apps for Apple Watch: See watchOS With SwiftUI by Tutorials (https://bit.ly/3Ok0iQ2).

`ARKit` for augmented reality: See Apple Augmented Reality by Tutorials (https://bit.ly/3OpnzQz). Be prepared for Apple's mixed reality headset (https://bit.ly/3XibhgR), predicted for Real Soon Now™. ;]



Apple Augmented Reality by Tutorials

Explore all the Technologies (https://apple.co/3rwyxrj):



Apple Developer Technologies filtered for 'data'

And you can browse topic-specific videos in the Apple Developer app:

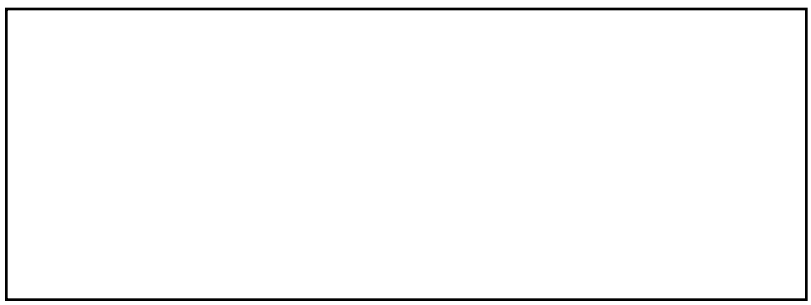


Apple Developer app: Videos classified by topic

Capabilities

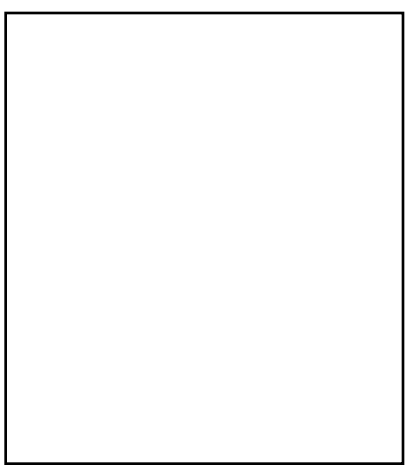
Many of the frameworks are for adding special features to your apps. Apple calls these *capabilities*.

➤ To see a list of capabilities, open one of your Xcode projects or create a new one. On the project page, select a target, click **Signing & Capabilities**, then click **+ Capability**:



Capabilities

If you're not in the **Apple Developer Program**, you can add only some of these capabilities to your apps. They're listed in the third column of Supported capabilities (iOS) (https://apple.co/3rOhlNW).



Capabilities Available to Developers

➤ Scroll down the page to see all the capabilities you can add if you join the Apple Developer Program.

Developer Programs

So what are the three types of Developer?

Apple Developer:

- No annual fee.
- Access to documentation and videos.
- Free provisioning to install Xcode projects on your devices: Xcode creates a *provisioning profile* that lets you install a few apps. The provisioning profile expires after seven days, so the apps will stop working on your device. Simply delete the profile and reinstall your app.

Apple Developer Program (https://apple.co/3emR2ur):

- Annual fee approximately equivalent to US$99 in your currency.
- Access to beta operating systems and applications.
- A web interface for managing your certificates, identifiers and profiles.
- Access to **App Store Connect**: Distribute your beta apps to testers with **TestFlight**; submit your apps to the App Store and access App Analytics.
- During virtual WWDC: You can request a lab appointment or post forum questions to Apple engineers about WWDC content. When in-person WWDCs resume, you can register for the ticket lottery.

What if you have apps in the App Store but you don't renew your membership? Here's Apple's answer:

> **Expired Memberships**: If your Apple Developer Program membership expires, your apps will no longer be available for download and you will not be able to submit new apps or updates. You will lose access to pre-release software, Certificates, Identifiers & Profiles, and Technical Support Incidents.

Apple Enterprise Program is for companies that want to distribute apps only to employees. The fee is US$299 or equivalent per year. Enterprise apps aren't submitted to the App Store so don't have to comply with Apple's requirements. But there are a lot of legal requirements (https://apple.co/3coUHVU), and it's probably easier to just use TestFlight.

App Distribution

The actual procedure for getting your app into the App Store changes a little bit every year. Apple's documentation can be confusing.

Check out our book iOS App Distribution & Best Practices (https://bit.ly/3al3Hez) or video course Publishing to the App Store (https://bit.ly/3tckW8Z).

Housekeeping & Trouble-Shooting

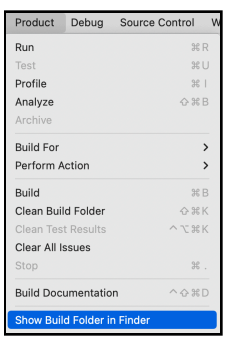
Hopefully, you haven't run into any issues while creating HIITFit. Xcode's error and warning messages are often helpful, but sometimes they're just wrong. At first, you won't be sure if it's something you need to fix but, as you gain more experience, you'll get a feeling for when Xcode is wrong or confused. It does happen. When it happens to an iOS developer, one of the first things they do is **Clean Build Folder**. Read on to find out what this involves.

DerivedData/Build

Xcode maintains a lot of files in **~/Library/Developer/Xcode**. Of particular interest is **DerivedData**, where Xcode creates a folder for every project you've ever created or opened. This is where Xcode stores intermediate build results, indexes and logs.

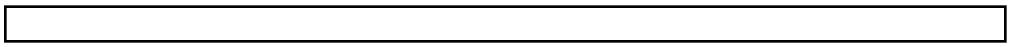
The easiest way to locate your project's derived data folder is with the Xcode menu item **Show Build Folder in Finder**.

➤ Open one of your Xcode projects or create a new one. If it's new, press **Command-B** or refresh a preview to build it. Then select **Product ▸ Show Build Folder in Finder**:



Show Build Folder in Finder.

➤ In the **Finder** column view, scroll *left* to see your project's folder in **DerivedData**:



Scroll left in Finder.

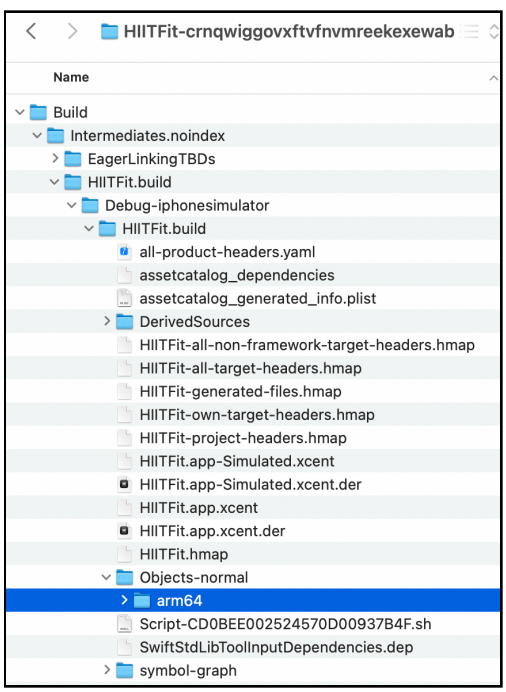
The folder name starts with the name of your project followed by a hash value.

➤ Select this folder, then view it as a list and open the **Build** folder:



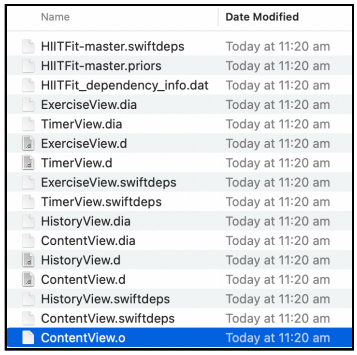
Open Build folder.

➤ Open **Intermediates.noindex** and drill down through its **.build**, **Debug** and **.build** folders to find **Objects-normal/x86_64** or, for us lucky M1 owners, **arm64**:



Locate x86_64 or arm64 folder.

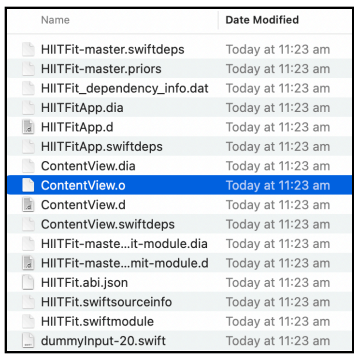
➤ Double-click **x86_64** or **arm64** to open it, then sort on **Date Modified**:



Sort x86_64 or arm64 by Date Modified.

➤ Note the location and timestamp of **ContentView.o**.

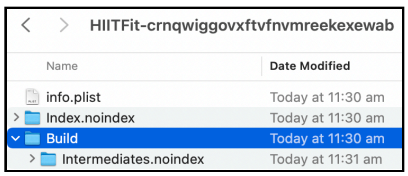
➤ In Xcode, make a change in **ContentView.swift**, press **Command-B** to rebuild, then look at **ContentView.o** in **x86_64** or **arm64**:



Build after code change

Xcode updated several files, including most of the **ContentView** files. Xcode recompiled **ContentView.swift** but not the other Swift files. It knows (most of the time) which files have changed and doesn't recompile files that haven't changed. But sometimes, something goes wrong with this system, and Xcode complains about code that is correct, or weird errors appear for no apparent reason. Then it's time to *clean the build folder*.

➤ In Xcode, press **Shift-Command-K** or select **Product ▸ Clean Build Folder** from the Xcode menu:



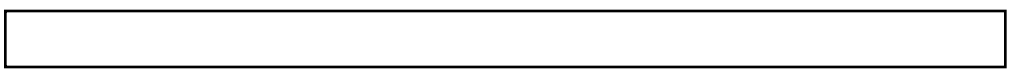
Empty Build folder

This command deletes almost everything in your project's **Build** folder, giving you a fresh start.

DerivedData

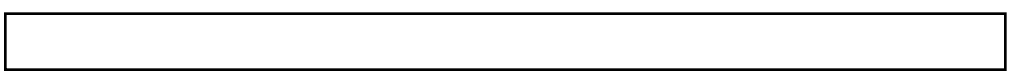
The rest of your app's folder in DerivedData stores data and indexes that Xcode uses for search, Open Quickly and refactoring. Again, sometimes these get mixed up, causing strange Xcode behavior. There's no menu command to delete these files. You just have to delete the whole derived data folder and let Xcode re-create it.

➤ Press **Command-up-arrow** to get back to your derived data folder or double-click its name in the path bar:



Go to derived data folder.

➤ In **Finder**, press **Command-delete** to delete the whole folder, then in Xcode, press **Command-B** to rebuild:



New derived data folder

Xcode creates a new derived data folder, with the same name.

When Xcode Behaves Strangely

Xcode is a hugely complex app, and sometimes it needs a nudge or something stronger to "clear its head".

Here are the escalating levels of intervention that most developers follow:

1. Clean Build Folder.
2. Delete the project's DerivedData folder.
3. Restart Xcode.
4. Restart Mac.

> **Strange but true**: Deleting earlier versions of Xcode can fix some weird issues, like no color-coding in the editor or **Command**-/ not working.

Reclaiming Disk Space

Weird Xcode behavior isn't the only reason to delete derived data folders. Long after you've finished working on a project — or even deleted it — its derived data folder is still there, taking up disk space. Many developers routinely delete the entire DerivedData folder every month or so, reclaiming gigabytes of space. If you're running low on disk space, it's certainly the first place you should look.

Command-delete just moves it to Trash, where it still takes up space. To really delete the folder, you can enter this command in **Terminal**:

```
rm -rf ~/Library/Developer/Xcode/DerivedData/*
```

Or, if you're running **Big Sur** or later, you can open **Trash** and selectively erase the folder. But, since Big Sur, macOS storage management provides a way to clear even more space.

➤ In the Apple menu, select **About this Mac** and click **More Info…**. In the **General** screen, scroll down to **Storage**, click **Storage Settings…**, then click the **Developer** info icon. In the Developer window, select **Xcode Caches**, hold down the **Command** key to select any **Device Support** items you don't need, then click **Delete…**:

macOS Developer storage management

Seeking Help From the Community

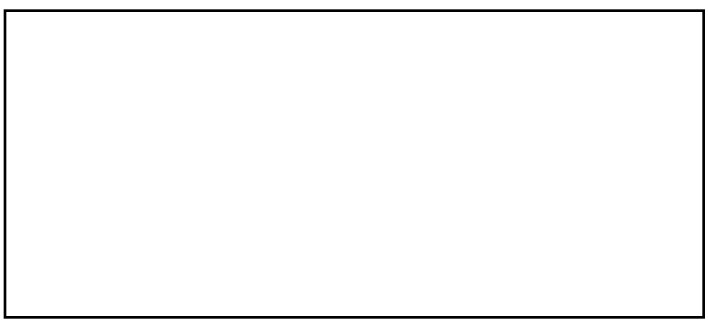
SwiftUI's error messages can be mysterious. If you get an error message and can't figure out what it wants you to do, select the whole message, then select **Search With Google** from the right-click menu.

Many search results will be Stack Overflow or Apple developer forum questions, hopefully with answers.

Also check out Kodeco forums (https://bit.ly/3rHLXRj) and Discord (https://bit.ly/3lajOjf).

The Kodeco team and members are a terrific resource, but there's also a large worldwide community of iOS developers. They're almost universally friendly, welcoming and generous with their time and expertise.

An easy path to join this community is to attend the monthly online events at iOS Dev Happy Hour (https://www.iosdevhappyhour.com):



iOS Dev Happy Hour

Key Points

- SwiftUI is a Domain Specific Language built on Swift, a faster and safer programming language than Objective-C.
- "SwiftUI vs. UIKit" is the most popular free episode at Kodeco and answers the big question: Which should you learn?
- You can use UIKit views and view controllers in your SwiftUI apps.
- Apple provides a lot of resources to help you become a developer and stay up to date: Documentation and human interface guidelines are available on the website and in Xcode. Use the Apple Developer app to watch WWDC videos.
- The iOS Software Development Kit has a lot of frameworks, many for adding special features (capabilities) to your apps.
- Members of the Apple Developer Program can add all the capabilities and also get early access to beta operating systems and developer tools. And, only members can participate fully in WWDC.
- Xcode stores intermediate build results, indexes and logs in your project's derived data folder. Sometimes you need to Clean Build Folder or delete the entire derived data folder. To reclaim disk space, periodically delete the whole DerivedData directory.

Section II: Your Second App: Cards

Now that you've completed your first app, it's time to apply your knowledge and build a new app from scratch. In this section, you'll build a photo collage app called Cards and you'll start from a blank template. Along the way, you'll:

- Dive more deeply into Swift's ways of representing data.
- Learn how to support user gestures.
- Discover how Xcode and iOS manage app assets such as images and colors.
- Explore more robust ways of saving and restoring data.
- Translate a designer's vision into reality in your app.

Chapter 13: Outlining a Photo Collage App

By Caroline Begbie

Congratulations, you've written your first app! HIITFit uses standard iOS user interaction with lists and swipeable page views. Now you'll get your teeth into something a bit more complex with custom gestures and custom views.

Photo collage apps are very popular, and in this section, you'll build your own collaging app in which you'll create cards to share. You'll be able to add images — from your photos or from the internet — and add text and stickers too. This app will be real-world with real-world problems to match.

In this chapter, you'll take a look at a sketch outline of the app idea and create a view hierarchy that will be the skeleton of your app.

At the end of Section 2, your finished app will look like this:

Final app

Initial App Idea

The first step to creating a new app is having the idea. Before writing any code, you should do research as to whether your app is going to be a hit or a miss. Work out who your target audience is and talk to some people who might use your app. Find out what your competition is in the App Store and explore how your app can offer something new and different.

Once you've decided that you have a hit on your hands, sketch your app out and work out feasibility and where technical difficulties may lie.

Your photo collaging app will have a primary view — where you list all the cards — and a detail view for the selected card — where you can add photos and text. This might be the back-of-the-napkin sketch:

Back of the napkin sketch

In future chapters, you'll set up the data model and data storage, but for now, examine the design and think about possible implementation difficulties that you'll need to overcome. Always take a modular approach and test each aspect of the app as separately from the main app as possible.

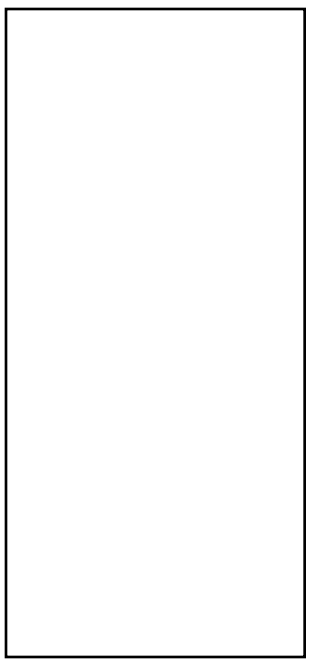
SwiftUI is great for this, because you can construct views and controls independently using SwiftUI's live preview. When you're happy with how a view works, add it to your app.

Creating the Project

In the previous section, you began with a starter app containing all the assets you needed to create HIITFit. In this section, you'll start with a new app, and you'll find out how to add assets as you move through the next few chapters.

➤ Open Xcode and choose **File ▸ New ▸ Project...** and create a new project called **Cards** using the **iOS App** template. If you need a refresher on how to create a new SwiftUI project, you'll find all the information in Chapter 1, "Checking Your Tools".

➤ Click the **run destination** button and select **iPhone 14 Pro**. Build and run your app using **Command-R** to make sure that everything works OK. Your iPhone 14 Pro simulator should start and show `ContentView`'s "Hello, world!" text.



Initial view

You should take these steps every time you create a new app just in case something in your environment has changed.

Creating the First View for Your Project

Skills you'll learn in this section: `ScrollView`

➤ Create a new SwiftUI View file called **CardsListView.swift**.

This view will show a scrolling thumbnail list of all the cards you create in your app.

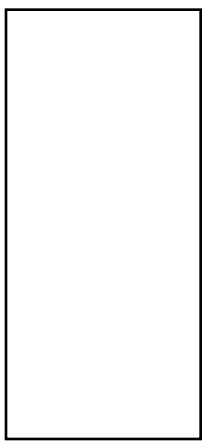
Creating a List of Cards

Instead of cards, for the moment, you'll show a placeholder list of rounded rectangles.

➤ In **CardsListView.swift**, replace `body` with:

```
var body: some View {
  ScrollView {
    VStack {
      ForEach(0..<10) { _ in
        RoundedRectangle(cornerRadius: 15)
          .foregroundColor(.gray)
          .frame(width: 150, height: 250)
      }
    }
  }
}
```

This places ten shapes in a scrollable `VStack`.

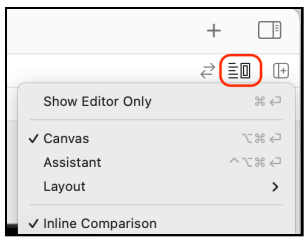


Placeholder thumbnails

A `ScrollView` can be vertical or horizontal. The default, which you use here, is vertical, but you can specify a horizontal axis with `ScrollView(.horizontal)`.

➤ In **live preview**, scroll the list. When you scroll, you can see an ugly scroll bar by the side of the cards.

In case you can't see the canvas, you can enable it using the icon at the top right of Xcode:



Show canvas

➤ Change `ScrollView {` to:

```
ScrollView(showsIndicators: false) {
```

This turns the scroll bar off.

With and without the scroll bar

Refactoring the View

Skills you'll learn in this section: refactoring views

As you add views, you'll recognize that later on, some views will become more complex. The `RoundedRectangle` is such a view. You've given it basic styling, but you'll probably want to style it a bit further down the line.

It's much easier to refactor views early on, so you'll create a new view for the placeholder card now. You extracted a view in Chapter 3, "Prototyping the Main View", so this should be a refresher for you.

➤ Create a new SwiftUI View file called **CardThumbnail.swift**.

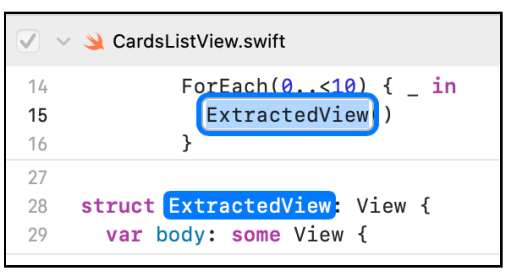
➤ Back in **CardsListView.swift**, **Command-click** `RoundedRectangle` and choose **Extract Subview**.

Xcode will copy `RoundedRectangle()` to a new `View` and rename the reference to `ExtractedView()`.

```
var body: some View {
  ScrollView(showsIndicators: false) {
    VStack {
      ForEach(0..<10) { _ in
        ExtractedView()
      }
    }
  }
}
```

Extracted View

➤ Right click `ExtractedView()`, choose **Refactor > Rename** and rename `ExtractedView` to `CardThumbnail`.



Refactor the view

The extracted view is now at the end of the current file and looks like this:

```
struct CardThumbnail: View {
  var body: some View {
    RoundedRectangle(cornerRadius: 15)
      .foregroundColor(.gray)
      .frame(width: 150, height: 250)
  }
}
```

➤ Cut this code and open **CardThumbnail.swift**.

➤ Select the entire `CardThumbnail` structure and paste in the cut code.

➤ Open **CardsListView.swift**. Your list still looks the same, but it will be easier for you to add colors and shadows to the thumbnails later.

Setting Up the Single Card View

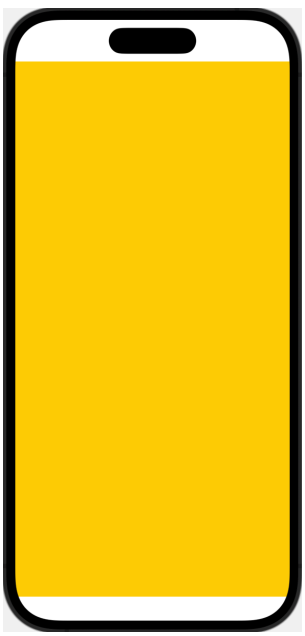
A card will have a colored background to which you'll add photos, stickers and text.

➤ Create a SwiftUI View file called **SingleCardView.swift**.

➤ Replace `body` with:

```
var body: some View {
  Color.yellow
}
```

This color will eventually come from the card data, but for the moment you'll just make the card yellow.



A yellow card

Transitioning From List to Card

Skills you'll learn in this section: full screen modal

When you tap a card in the scrolling list in `CardsListView`, you want to show `SingleCardView`. You can achieve this in several ways:

- Replace `CardsListView` in the view hierarchy. With this option, when you return to the thumbnails after editing the card, you'd lose your current scrolling position in the cards list.
- Use a `NavigationStack` with a `NavigationLink` destination that pushes `SingleCardView` to the front. You'll learn about `NavigationStack` in your third app in Section III.
- Present a full screen modal view. The view slides up from the bottom and covers the whole screen. This is the option you'll use here.
- By toggling a view state property, you can show `SingleCardView` in a new layer in front of `CardsListView`. This is the best option if you need a different transition to the modal's slide from the bottom. It is, however, more complex to implement, as you have to pass the view state property to other subviews.

Creating a Full Screen Modal View

➤ In `CardsListView`, create a new property to track the modal presentation:

```
@State private var isPresented = false
```

➤ Add a new modifier to `ScrollView`:

```
.fullScreenCover(isPresented: $isPresented) {
  SingleCardView()
}
```

When `isPresented` is `true`, you'll show `SingleCardView` as a full screen modal.

➤ Add a modifier to `CardThumbnail()`:

```
.onTapGesture {
  isPresented = true
}
```

➤ In **live preview**, tap one of the cards.

Transition from thumbnail to card

`isPresented` becomes `true` and `SingleCardView` slides up from the bottom of the device's screen. Currently there is no way to dismiss the view and return to the list of cards, so you'll need to create a **Done** button.

Before tackling the button, set up your app to run in Simulator, so that if **live preview** fails, you can still see your app.

➤ Open **CardsApp.swift** and change `ContentView()` to:

```
CardsListView()
```

You call the view that will show the list of cards instead of `ContentView`.

➤ Build and run and make sure that your app works in Simulator, just as it does in the preview.

You aren't using **ContentView.swift** any more, but you can leave it in the project to experiment with other SwiftUI layouts.

The Navigation Toolbar

Skills you'll learn in this section: toolbars; navigation bar; `NavigationStack`; tuples

A **Done** button in `SingleCardView` will dismiss the full screen modal view. You can set up buttons at the top and bottom of the screen using a navigation toolbar.

➤ Open **SingleCardView.swift** and add the environment object that dismisses a modal to `SingleCardView`:

```
@Environment(\.dismiss) var dismiss
```

➤ Add a new toolbar modifier to `Color.yellow`:

```
.toolbar {
  ToolbarItem(placement: .navigationBarTrailing) {
    Button("Done") {
      dismiss()
    }
  }
}
```

You place a **Done** button at the top right of the screen. When the user taps this button, SwiftUI dismisses the `SingleCardView` modal.

`toolbar(content:)` allows multiple `ToolbarItems`. `placement` can be:

- `navigationBarLeading`: The leading edge of the top navigation bar.
- `navigationBarTrailing`: The trailing edge of the top navigation bar.
- `principal`: On iOS, the principal placement is in the center of the navigation bar.
- `bottomBar`: The bottom toolbar.

You'll use the bottom toolbar placement shortly.

➤ Preview `SingleCardView`.



No Done button

NavigationStack

Notice that the button doesn't show up. This is because `ToolbarItem(placement:)` is using `navigationBarTrailing`, so any item will only show up if the view is inside a `NavigationStack`.

➤ In `SingleCardView`, **Command-click** `Color` and choose **Embed...**.

➤ Change the placeholder `Container` to `NavigationStack`.

Your button will now show up.



Navigation bar Done button

When you use `Lists`, you often use `NavigationStack` and `NavigationLink` together, which have built-in push and pop transitions and titles. You'll explore this more in Section 3. Currently, you're using a `NavigationStack` not for transitions but simply to make the **Done** button show up in the `SingleCardView`.

➤ Open **CardsListView.swift** and test your app so far. Tap a thumbnail to show the yellow card and dismiss the card by tapping the **Done** button.

Navigation with Done button

The Bottom Toolbar

The single card view will have four buttons at the bottom, allowing you to add elements to your card:

- **Photos**: Pick photos from your Photo Library.
- **Frames**: Change the shape of the photo element.
- **Stickers**: Add some fun to your card with app stickers.
- **Text**: Add words.

Each of these buttons will show a separate modal view. When you have discrete values, such as these four destinations, you can use an enumeration to create your set of values. Enumerations make your code easier to read and ensure that values are restricted to those defined in the enumeration.

➤ Create a new Swift file called **ToolbarSelection.swift** and add this code:

```
enum ToolbarSelection {
  case photoModal, frameModal, stickerModal, textModal
}
```

These cases correspond to each of the buttons.

➤ Create a new SwiftUI View file called **BottomToolbar.swift**.

In this view, you'll set up the four buttons at the bottom of the screen.

➤ Above `BottomToolbar`, add a new `View` for a single toolbar button:

```
struct ToolbarButton: View {
  var body: some View {
    VStack {
      Image(systemName: "heart.circle")
        .font(.largeTitle)
      Text("Stickers")
    }
    .padding(.top)
  }
}
```

Each modal button will use this view, and you'll style this to be more generic shortly.

➤ In `BottomToolbar`, add a binding for the current modal:

```
@Binding var modal: ToolbarSelection?
```

➤ Replace `body` with:

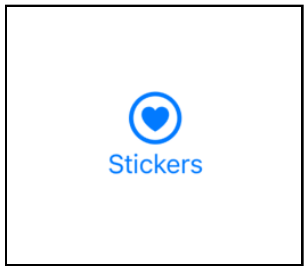
```
var body: some View {
  HStack {
    Button {
      modal = .stickerModal
    } label: {
      ToolbarButton()
    }
  }
}
```

Here you create an `HStack` containing a button that will change the modal state. Because the text label for the button is a custom view, rather than a string, you use the `Button(action:label:)` initializer. You'll add more toolbar items in a moment to this `HStack`.

➤ Fix the preview to send a modal binding:

```
struct BottomToolbar_Previews: PreviewProvider {
  static var previews: some View {
    BottomToolbar(modal: .constant(.stickerModal))
      .padding()
  }
}
```

In **live preview**, you'll see your **Stickers** button and icon:



Stickers button

Adding the Bottom Toolbar

➤ Open **SingleCardView.swift** and add a new property to `SingleCardView`:

```
@State private var currentModal: ToolbarSelection?
```

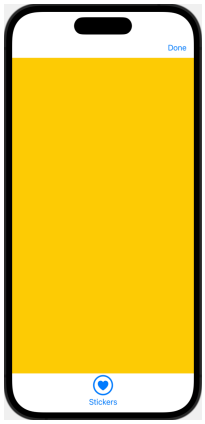
When you tap a button on the bottom bar, the button will update this property. Later on, you'll show the corresponding modal view.

➤ In `body`, locate `.toolbar {`. This is where you currently have the **Done** button.

➤ Add a new toolbar item inside `toolbar(content:)`, under the previous toolbar item:

```
ToolbarItem(placement: .bottomBar) {
  BottomToolbar(modal: $currentModal)
}
```

Here you add your new toolbar at the bottom of the screen.



Bottom toolbar

Adding the Other Buttons

➤ Open **BottomToolbar.swift** and add a new property to `ToolbarButton`:

```
let modal: ToolbarSelection
```

`ToolbarButton` is the view that displays the toolbar button. You'll send in the modal that the button is tied to and show the correct image for that button. You'll get a compile error until you fix `BottomToolbar`.

You already set up `body` to show an image and text for the **Stickers** button. You could do a `switch` in body and show the appropriate image for all the modal options. However, it's more succinct to set a dictionary of all the possible options with the text and image name. In case you need a refresher on dictionaries, you first used them in Chapter 7, "Saving Settings".

➤ Add this property to `ToolbarButton`:

```
private let modalButton: [
    ToolbarSelection: (text: String, imageName: String)
  ] = [
    .photoModal: ("Photos", "photo"),
    .frameModal: ("Frames", "square.on.circle"),
    .stickerModal: ("Stickers", "heart.circle"),
    .textModal: ("Text", "textformat")
  ]
```

Here you set up a dictionary of type `[ToolbarSelection: (String, String)]` containing values for all the possible button states. You could have set up a structure that contains `text` and `imageName`, but if you're only using a type once in an object with not much code, you can set up an "ad hoc" data type called a **tuple**.

Tuples

A tuple is a group of values. For example, you could initialize a tuple with three elements like this:

```
let button = ("Stickers", "heart.circle", 1)
```

And access the data:

```
let text = button.0
let number = button.2
```

It's obviously good practice to name your types rather than using numbers to access the data, which is why you defined your `modalButton` tuple with `(text:imageName:)`

➤ In `ToolbarButton`, replace `body` with:

```
var body: some View {
  if let text = modalButton[modal]?.text,
    let imageName = modalButton[modal]?.imageName {
  VStack {
    Image(systemName: imageName)
      .font(.largeTitle)
    Text(text)
  }
  .padding(.top)
  }
}
```

Using your dictionary, you access the text and image name and use those for the button instead of the hard coded Stickers values.

To show all the buttons in `BottomToolbar`, you can iterate through all the values of `ToolbarSelection`. Swift enumerations have a built-in array called `allCases`, but to use it, the enumeration must conform to the `CaseIterable` protocol.

➤ Open **ToolbarSelection.swift** and conform `ToolbarSelection`:

```
enum ToolbarSelection: CaseIterable {
```

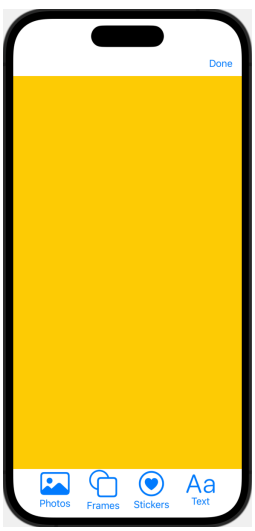
You can now iterate through the values using `ToolbarSelection.allCases`.

➤ Open **BottomToolbar.swift**. In `BottomToolbar` replace the contents of `body` with:

```
HStack {
  ForEach(ToolbarSelection.allCases, id: \.self) { selection in
    Button {
      modal = selection
    } label: {
      ToolbarButton(modal: selection)
    }
  }
}
```

You iterate through all the available modals. Each button shows the correct image and text for the modal, and the action sets the new modal state.

➤ Open **SingleCardView.swift** and, in **live preview**, admire your layout so far.



Button Preview

Adding Modal Views

Skills you'll learn in this section: multiple modal sheets; `Identifiable` enumerations; `Hashable`

As of now, the buttons don't do anything, so you'll attach text views to each button. As you progress through the book, you'll replace the text view with the appropriate content.

In Section 1, you used `View.sheet(isPresented:onDismiss:content:)`, where you passed in a Boolean state property. When you have multiple sheets to show conditionally, you can choose a different method of presentation, by passing the sheet an optional data source binding.

This data source can be of any type that conforms to `Identifiable`.

➤ In **SingleCardView.swift**, add this modifier to `Color.yellow`.

```
.sheet(item: $currentModal) { item in
  switch item {
  default:
    Text(String(describing: item))
  }
}
```

Here, for every modal selection, you print out the description of the modal. Later, you'll add cases to this switch statement when you configure each of the four modal views.

`currentModal` is of type `ToolbarSelection`, which doesn't conform to `Identifiable`. Because of this, you'll get a compile error: _Instance method `sheet(item:onDismiss:content:)` requires that `ToolbarSelection` conform to `Identifiable`_.

Making an Enumeration Identifiable

➤ Open **ToolbarSelection.swift** and conform `ToolbarSelection` to `Identifiable`:

```
enum ToolbarSelection: CaseIterable, Identifiable {
```

As you already know, to conform to `Identifiable`, you have to provide an `id`.

Add the id to `ToolbarSelection`:

```
var id = UUID()
```

You'll immediately get a compiler error, saying *"Enums must not contain stored properties"*. Remember that you can't make a copy of an enumeration by instantiating it, so you can't add stored `vars` to an enumeration. Yet, you need to include `var id` in order to conform to `Identifiable`.

Making an Object Hashable

You need a value that uniquely identifies an object. That describes a **hash value**. Hashing algorithms calculate values from any data to provide a digital fingerprint that identifies an object.

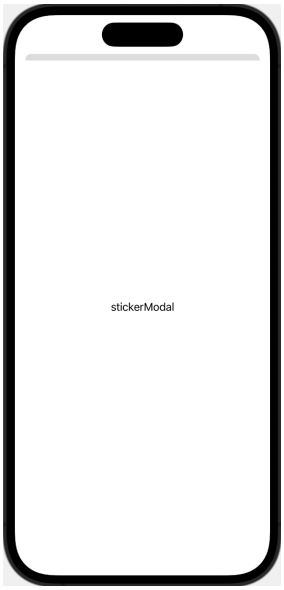
Fortunately, enumerations automatically conform to `Hashable`, which provides a hash value.

➤ Replace `var id = UUID()` with:

```
var id: Int {
  hashValue
}
```

Instead of a stored property, this `var` is a computed property. Now, when you create a `ToolbarSelection` object, each object will have a different ID calculated from the enumeration's hash value.

➤ Open **CardsListView.swift** and tap a card. In **live preview**, tap the bottom buttons to preview each of your modal views. The modal's description displays on each modal view.



A modal view

Cleaning Up

➤ Open **BottomToolbar.swift**, and in `BottomToolbar`, remove `, id: \.self` from the `ForEach` loop.

Because `ToolbarSelection` is now `Identifiable`, the extra `id:` parameter is superfluous.

The `ForEach` loop is now:

```
  ForEach(ToolbarSelection.allCases) { selection in
```

➤ Build and run your app in Simulator and check out your app's outline so far.

The app outline

You now have a prototype of the main views of your app and can visualize how they will fit together. A prototype is useful, even at this early stage, so that you can show it to other people to find out what they think of it and whether the interface is intuitive enough for them to navigate without help. It's better to find out that your app is not useful as early as possible in its development so that you can either incorporate feedback or pivot entirely.

Challenge

Make it a habit to regularly tidy up the code and files in your app.

Challenge 1: Tidy up Files

Look down the list of files and see which ones you can group together. **Command-click** each file that you want to group together, then **Control-click** the selected files and choose **New Group from Selection**. Name the group. If you miss any files, just drag them into the group later.

As an example, you can group all the files with **View** in their name a group called **Views**. You can then have a sub group for the views used for a single card. You'll find suggested groups in the **challenge** project for this chapter.

Challenge 2: Refactor Code

You don't always have to create new structures for views. Sometimes, if it's a simple view and you're only using it once, it's easier to keep track of views as properties or methods.

In **CardsListView.swift**, refactor `ScrollView` to be a property called `list`. Leave the full screen cover modifier in `body` as a modifier on `list`. Try and keep `body` as short as possible so that it's easier to read.

Key Points

- Prototypes are always worth doing. With a prototype it's easier to see what's missing and what the next steps should be. They don't have to be complicated. So far, you aren't creating or saving any data, but you can tell how the app will flow. Writing an app is more than just writing code. It's finding your target audience, creating a good design and overcoming technical problems.
- Refactor your views early and often. Whenever you have a stack enclosing a number of views with multiple modifiers, it's worth extracting those views to either a new `View` structure or to a new `View` property within the existing `View` structure.
- There are several ways you can navigate apps. You can use built-in `NavigationStacks`, completely customize with buttons or tapped views and modal views or layered views.
- SwiftUI toolbars are great for standard placement. You must enclose them in a `NavigationStack` to be able to see them.
- Dictionaries are useful for holding disparate values. For example, you can use them to hold options to format views. Using tuples, you can create ad hoc types.
- You can show modal views conditionally using a type that conforms to `Identifiable`.
- Enumerations have a unique identifier called a hash value.

Chapter 14: Gestures

By Caroline Begbie

Gestures are the main interface between you and your app. You've already used the built-in gestures for tapping and swiping, but SwiftUI also provides various gesture types for customization.

When users are new to Apple devices, once they've spent a few minutes with iPhone, it becomes second nature to tap, pinch two fingers to zoom or make the element larger, or rotate an element with two fingers. Your app should use these standard gestures. In this chapter, you'll explore how to drag, magnify and rotate elements with the built-in gesture recognizers.

Back of the napkin design

In the single card view, you'll drag around and resize photo and text elements. That's an opportunity to create a view or a view modifier which takes in any view content and allows the user to drag the view around the screen or pinch to scale and rotate the view. Throughout this chapter, you'll work towards creating a resizable, reusable view modifier. You'll be able to use this in any of your future apps.

Creating the Resizable View

To start with, the resizable view will simply show a colored rectangle but, later on, you'll change it to show any view content.

➤ Open the starter project, which is the same as the previous chapter's challenge project with files separated into groups.

➤ Create a new SwiftUI View file named **ResizableView.swift**. Replace `ResizableView` with this code:

```
struct ResizableView: View {
  // 1
  private let content = RoundedRectangle(cornerRadius: 30.0)
  private let color = Color.red

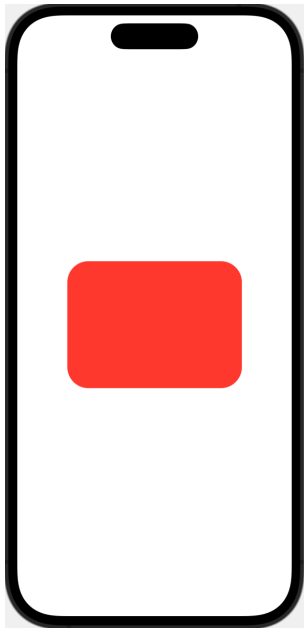
  var body: some View {
    // 2
    content
      .frame(width: 250, height: 180)
      .foregroundColor(color)
  }
}
```

Going through the code:

1. Create a `RoundedRectangle` view property. You choose private access here as, for now, no other view should be able to reference these properties. Later on, you'll change the access to pass in any view.

2. Use `content` as the required `View` in `body` and apply modifiers to it.

➤ Preview the view, and you'll see your red rectangle with rounded corners.



Preview rounded rectangle

Creating Transforms

Skills you'll learn in this section: transformation

Each card in your app will hold multiple images and pieces of text called, generically, **elements**. For each element, you'll store a size, a location on the screen and a rotation angle. In mathematics, you refer to these spatial properties collectively as a **transformation** or **transform**.

➤ Create a new group called **Model** that will hold data structure files.

➤ In the **Model** group, create a new Swift file called **Transform.swift** to hold the transformation data.

➤ Replace the code in the file and create a structure with initialized spatial properties:

```
import SwiftUI

struct Transform {
  var size = CGSize(width: 250, height: 180)
  var rotation: Angle = .zero
  var offset: CGSize = .zero
}
```

You set up defaults for size, rotation and offset. `Angle` is a SwiftUI type which conveniently works with both degrees and radians.

Notice the use of `.zero` here. `Angle.zero` and `CGSize.zero` are both **type properties** that return zero values. You'll discover more about type properties later in this chapter. When the type is obvious to the compiler, as it is here, the compiler will work out which type to use for `.zero`.

Often, transforms hold a scale value too, but in this case you'll update the size of the element instead of holding a scale value.

➤ Open **ResizableView.swift** and add a new property:

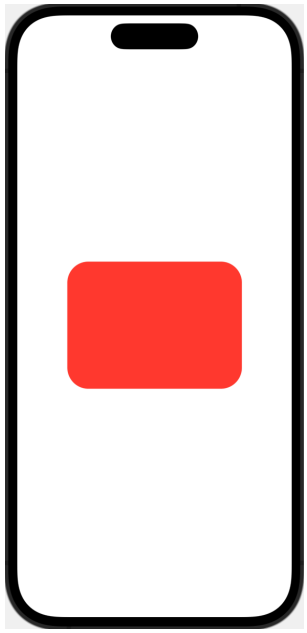
```
@State private var transform = Transform()
```

You hold the transform that you will apply to `ResizableView` as a state property. Later on, you'll pass the element's saved transform in, but for now, just hold the transform locally.

➤ Change `frame(width:height:alignment:)` to use `transform` instead of the hard-coded size:

```
.frame(
  width: transform.size.width,
  height: transform.size.height)
```

Because `transform` holds the same default size, the view does not change. Now you're ready to create gestures to move your view around.



Rounded rectangle with transform sizing

Creating a Drag Gesture

Skills you'll learn in this section: drag gesture; operator overloading

You'll start off with the **drag** gesture, where the user moves one finger across the screen. This is also called a **pan** gesture. When the user touches down on a `ResizableView` and drags a finger, the view will follow that finger. When they lift the finger, the view will remain at that location.

You'll give the view a modifier which will update the offset of `ResizableView` from the center of its parent view. To position the view, you have a choice of using either `position(_:)` or `offset(_:)` view modifier. You're saving an `offset` value into `transform`, so that's what you'll use here.

➤ Create a new `Gesture` property in `ResizableView`:

```
var dragGesture: some Gesture {
  DragGesture()
    .onChanged { value in
      transform.offset = value.translation
    }
}
```

The gesture updates `transform`'s `offset` property as the user drags the view.

`onChanged(_:)` has one parameter of type `Value`, which contains the gesture's current touch location and the translation since the start of the touch.

The center of the screen is at `offset.zero`.

Offset and translation

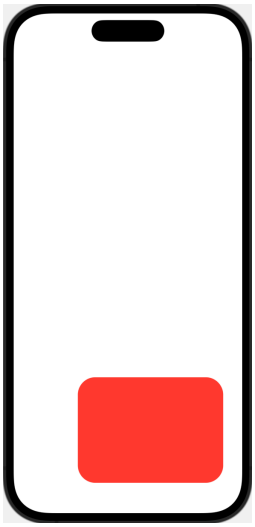
The amount of translation is the amount to offset the view. The translation is a `CGSize`, so when you travel across the screen, that's `translation.width`, and up and down the screen is `translation.height`.

➤ Add new modifiers to `content` at the end of `body`:

```
.offset(transform.offset)
.gesture(dragGesture)
```

Order of modifiers is important — `gesture(_:)` needs to go after any positioning modifiers.

➤ Live preview the view and drag it around the screen.



Dragging the view

The first drag works well, but on second and subsequent drags, the view does a jump at the start of the drag. This is because the drag gesture sets `value.translation` to zero at the start of the drag, so you'll need to take into account any previous translations.

➤ Add a new property to `ResizableView` to hold the transform's offset before you start dragging:

```
@State private var previousOffset: CGSize = .zero
```

➤ Change `dragGesture` to:

```
var dragGesture: some Gesture {
  DragGesture()
    .onChanged { value in
      transform.offset = CGSize(
        width: value.translation.width + previousOffset.width,
        height: value.translation.height +
previousOffset.height)
    }
    .onEnded { _ in
      previousOffset = transform.offset
    }
}
```

In `onChanged(_:)`, you update `transform` with the user's drag translation amount and include any previous dragging.

In `onEnded(_:)`, you replace the old `previousOffset` with the new offset, ready for the next drag. You don't need to use the `value` provided, so you use _ as the parameter for the action method.

➤ Try it out in the live preview again.

This works well. You can now drag your view around and position it wherever you want.

The `CGSize` code is a bit long-winded though, with having to do the math on both `width` and `height`. You can shorten this code by overloading the + operator.

Operator Overloading

Operator overloading is where you redefine what operators such as +, –, ∗ and / do.

To add `translation` to `offset`, you must add `width` to `width` and, at the same time, add `height` to `height`. To do this, you'll redefine + with a new method.

➤ Create a new Swift file called **Operators.swift**. Any time you want to overload an operator for a particular type, you can add the method in this file.

➤ Replace the code with the new method:

```
import SwiftUI

func + (left: CGSize, right: CGSize) -> CGSize {
  CGSize(
    width: left.width + right.width,
    height: left.height + right.height)
}
```

Here you specify what the + operator should do for a `CGSize` type. The parameters are `left` and `right`, which are the items to the left and right of the + sign. You return the new `CGSize`.

This is a simple example of how you want the + sign to work for `CGSize`. It makes sense here to add the `width` and `height` together. However, you can redefine this operator to do anything, and you should be very careful that the method makes sense. Don't do things like redefining a multiply sign to do division!

➤ Now, return to **ResizableView.swift** and change `dragGesture` to:

```
var dragGesture: some Gesture {
  DragGesture()
    .onChanged { value in
      transform.offset = value.translation + previousOffset
    }
    .onEnded { _ in
      previousOffset = transform.offset
    }
}
```

You can see how overloading the + operator reduces the code and increases clarity.

Creating a Rotation Gesture

Skills you'll learn in this section: rotation gesture

Now that you can move your view around the screen, it's time to rotate it. You'll use two fingers on the view and set up a `RotationGesture` to track the angle of rotation.

Just as you did with tracking the previous offset of the view, you'll track the previous rotation.

➤ In **ResizableView.swift**, set up a new property for this:

```
@State private var previousRotation: Angle = .zero
```

This will hold the angle of rotation of the view going into the start of the gesture.

➤ Add the new gesture to `ResizableView`:

```
var rotationGesture: some Gesture {
  RotationGesture()
    .onChanged { rotation in
      transform.rotation += rotation - previousRotation
      previousRotation = rotation
    }
    .onEnded { _ in
      previousRotation = .zero
    }
}
```

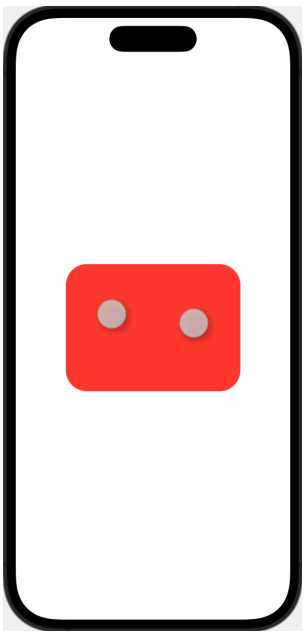
`onChanged(_:)` provides the gesture's angle of rotation as the parameter for the action you provide. You add the current rotation, less the previous rotation, to `transform`'s rotation.

`onEnded(_:)` takes place after the user removes his fingers from the screen. Here, you set any previous rotation to zero.

➤ In `body`, replace `.gesture(dragGesture)` with:

```
  .rotationEffect(transform.rotation)
  .gesture(dragGesture)
  .gesture(rotationGesture)
```

To test your rotation effect in the live preview, because you don't have a touch screen available to you, you can simulate two fingers by holding down the **Option** key. Two dots will appear, representing two fingers. (You may have to click the preview before they show up.)



Two fingers to rotate

Move your mouse or trackpad to change the distance between the two dots. Make sure that they are both on the rectangle `View`, and click and drag. Your view should rotate. If you have the distance between the dots correct, but you want the dots to be elsewhere on the screen, you can hold down the **Shift** key as well as **Option** to move the dots. Still holding **Option**, let go the **Shift** key when they are in the right place.

Order of modifiers is again important here. The pivot point of the rotation is around the center of the view without taking any offset into consideration.

➤ Drag the view and then rotate it, and you'll see that the view's pivot point is around the center of the screen. This is the view's center point without the offset applied.

Sometimes this may be what you want. But in your case here, you want to rotate the view before offsetting it.

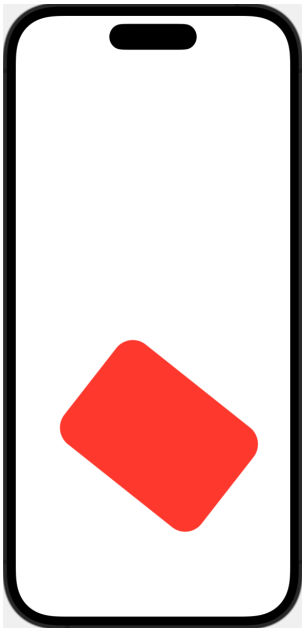
> **Swift Tip**: `rotationEffect(_:anchor:)` by default rotates around the center of the view, but you can change that to another point in the view by changing `anchor`.

➤ Move `.offset(transform.offset)` to *after* `rotationEffect(transform.rotation)`, but *before* `gesture(dragGesture)`.

> **Note**: The shortcut keys **Option-Command-[** and **Option-Command-]** move lines of code up and down.

The order of gestures is also important. If you place the drag gesture after the rotation gesture, then the rotation gesture will swallow up the touches.

➤ Try rotating the view in the live preview



Rotation

➤ Gestures always feel better on a real device, so to run this on a device, open **CardsApp.swift**

➤ Temporarily, change `CardsListView()` to:

```
ResizableView()
```

➤ Change the run destination to your device.

> **Note**: If you haven't yet run an app on your device, take a look at **Running your Apps on an iOS Device** in Chapter 2, "Planning a Paged App". You'll need an Apple developer account set up in Settings to run the app on a device.

➤ Set your team identifier on the Cards app's **Signing & Capabilities** tab.

➤ Build and run and try your gestures to see how fluid they feel. Two fingers on the device feels much more natural than trying to manipulate the simulator gesture dots.

Creating a Scale Gesture

Skills you'll learn in this section: magnification gesture; simultaneous gestures

Finally, you'll scale the view up and down. `MagnificationGesture` operates as a pinch gesture, so you'll be able to rotate and scale at the same time, using two fingers.

You'll do the scale slightly differently from rotate and offset. The view will always be at a scale of `1.0` unless the user is currently scaling. At the end of the scaling operation, you'll calculate the new size of the view and set the scale back to `1.0`.

➤ Open **ResizableView.swift**, and create a property to hold the current scale:

```
@State private var scale: CGFloat = 1.0
```

➤ Add the scale gesture property to `ResizableView`:

```
var scaleGesture: some Gesture {
  MagnificationGesture()
    .onChanged { scale in
      self.scale = scale
    }
    .onEnded { scale in
      transform.size.width *= scale
      transform.size.height *= scale
      self.scale = 1.0
    }
}
```

`onChanged(_:)` takes the current gesture's scale and stores it in the state property `scale`. To differentiate between the two properties called the same name, use `self` to describe `ResizableView`'s `@State` property.

When the user has finished the pinch and raises his fingers from the screen, `onEnded(_:)` takes the gesture's scale and changes `transform`'s `width` and `height`. You then reset `ResizableView.scale` to `1.0` to be ready for the next scale.

➤ In `body`, after `.rotationEffect(transform.rotation)`, add the scale modifier:

```
.scaleEffect(scale)
```

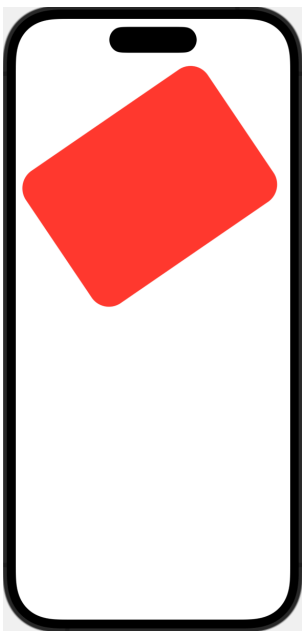
Creating a Simultaneous Gesture

Whereas the drag is a specific gesture with one finger, you can do rotation and scale at the same time with two fingers. To do this, change `.gesture(rotationGesture)` to:

```
.gesture(SimultaneousGesture(rotationGesture, scaleGesture))
```

You can now perform the two gestures at the same time.

➤ Try your three gestures in Live Preview. Then, build and run your app and try them in Simulator or, if possible, on a device.



Completed gestures

Creating Custom View Modifiers

Skills you'll learn in this section: creating a `ViewModifier`; `View` extension; using a view modifier; advantages of a view modifier

You've made a very useful view, one that can be used in many app contexts. Rather than hard-coding the view you want to resize, you can change this view and make it a modifier that acts on other views.

➤ In **ResizableView.swift**, change `struct ResizableView: View {` to:

```
struct ResizableView: ViewModifier {
```

Here, you declare the new view modifier. For the moment, ignore all the compile errors until you've completed the modifier.

➤ Change `var body: some View {` to:

```
func body(content: Content) -> some View {
```

Because `ViewModifier` takes in an existing view, instead of a `var`, it requires a method with the view content as a parameter. The content will be a view, such as a `Rectangle` or an `Image` or any custom view you create.

`ResizableView` should only operate on expected properties of a view. For resizing, you would expect a `Transform` property, but `color` has nothing to do with resizing. You'll set up color and content outside of the modifier.

➤ Remove:

```
private let content = RoundedRectangle(cornerRadius: 30.0)
private let color = Color.red
```

➤ Also remove `.foregroundColor(color)` from `body(content:)`.

➤ To preview the modifier, change the preview provider at the end of **ResizableView.swift**

```
struct ResizableView_Previews: PreviewProvider {
  static var previews: some View {
    RoundedRectangle(cornerRadius: 30.0)
      .foregroundColor(Color.blue)
      .modifier(ResizableView())
  }
}
```

Here, you set up the content that the view should use and add the `modifier(_:)` with your custom view modifier.

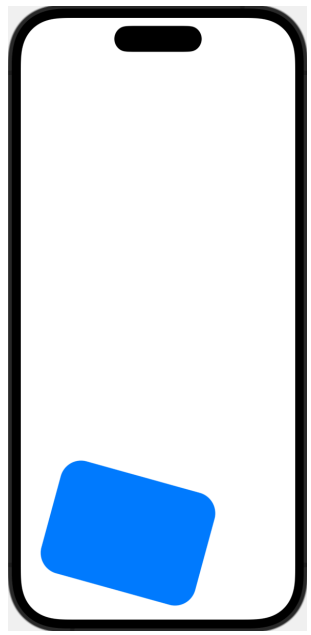
It's always a good idea to keep your previews working. With view modifier previews, you can provide an example to future users of your code how to use the modifier. Always remember that "future users" includes you in a few weeks' time!

➤ In **CardsApp.swift**, revert `ResizableView()` back to:

```
CardsListView()
```

Your project will now compile.

➤ In **ResizableView.swift**, resume your live preview and check out your new modifier.



View Modifier preview

It works exactly the same as `ResizableView`, but you can now apply the modifier to any view and make it resizable.

Using Your Custom View Modifier

In the preview, you used `.modifier(ResizableView())`. You can improve this by adding a "pass-through" method to `View`.

➤ Add this to the end of **ResizableView.swift**:

```
extension View {
  func resizableView() -> some View {
    modifier(ResizableView())
  }
}
```

You extend the `View` protocol with a default method. `resizableView()` is now available on any object that conforms to `View`. The method simply returns your modifier, but it does make your code easier to read.

➤ In `ResizableView_Previews`, replace `.modifier(ResizableView())` with:

```
.resizableView()
```

➤ Open **SingleCardView.swift** and add a new view property:

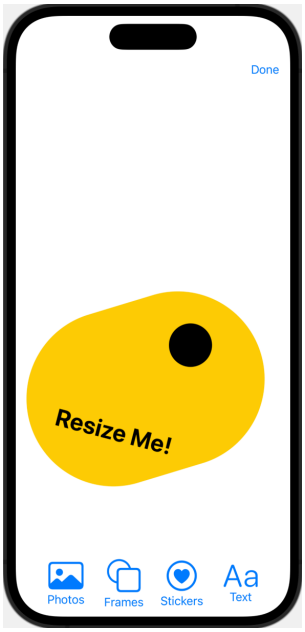
```
var content: some View {
  ZStack {
    Capsule()
      .foregroundColor(.yellow)
      .resizableView()
    Text("Resize Me!")
      .font(.largeTitle)
      .fontWeight(.bold)
      .resizableView()
    Circle()
      .resizableView()
      .offset(CGSize(width: 50, height: 200))
  }
}
```

➤ In `body`, replace `Color.yellow` with:

```
content
```

Eventually, `content` will show card elements, but for now you can test your new resizable view. Here you test your modifier with two different types of views — two `Shapes` and one `Text`. The `Circle`'s offset is applied on top of the offset in `resizableView()`. Everything is put together inside a `ZStack`, which is a container view that allows its children to use absolute positioning.

➤ Check out your new resizing abilities in Live Preview.



Resize multiple views

There is a problem with the `Text`. `Capsule` remembers its size, because of the `frame(width:height:alignment:)` modifier inside `ResizableView`. However, `Text` has a `font(_:)` modifier. Because the modifier is applied directly to the view, it takes priority over `frame(width:height:alignment:)`.

There is a trick to scaling text on demand. Give the font a huge size, say 500. Then apply a minimum scale factor to it, to reduce it in size.

➤ Remove `.font(.largeTitle)` from `content`.

➤ After `.fontWeight(.bold)`, add:

```
.font(.system(size: 500))
.minimumScaleFactor(0.01)
.lineLimit(1)
```

`.lineLimit(1)` ensures the text stays on one line and doesn't wrap around.

➤ Try resizing the text again in live preview. This time the text retains its size.



Resize the text

View Modifier Advantage

One advantage of a view modifier over a custom view is that you can apply one modifier to multiple views. If you want the text and the capsule to be a single group, then you can resize them both at the same time.

➤ Group `Capsule` and `Text` together inside the `ZStack`, and apply `resizableView()` to `Group` instead of the two views:

```
Group {
  Capsule()
    .foregroundColor(.yellow)
  Text("Resize Me!")
    .fontWeight(.bold)
    .font(.system(size: 500))
    .minimumScaleFactor(0.01)
    .lineLimit(1)
}
.resizableView()
```

Here, you grouped the two views together so they combine to a single view.

➤ Live Preview the view.



Grouped Views

When you resize the capsule now, you drag and resize both capsule and text at the same time. This could be useful where you have a caption or a watermark on an image and you want them both at the same scale.

Other Gestures

- **Tap gesture**

You used `onTapGesture(count:perform:)` in the previous chapter when tapping a card. There is also a `TapGesture` structure where you can use `onEnded(_:)` in the same way as with the other gestures in this chapter.

- **Long press gesture**

Similarly, you can use either the structure `LongPressGesture` to recognize a long-press on a view, or use `onLongPressGesture(minimumDuration:maximumDistance:pressing:perform:)` if you don't need to set up a separate gesture property.

Type Properties

Skills you'll learn in this section: type properties; type methods

So far, you've hard coded the size of the card thumbnail, and also the default size in `Transform`. In most apps, you'll want some global settings for sizes or color themes.

You do have the choice of holding constants in global space. You could, for example, create a new file and add this code at the top level:

```
var currentTheme = Color.red
```

`currentTheme` is then accessible to your whole app. However, as your app grows, sometimes it's hard to immediately identify whether a particular constant is global or whether it belongs to your current class or structure. An easy way of identifying globals, and making sure that they only exist in one place, is to set up a special type for them and add **type properties** to the type.

Swift Dive: Stored Property vs Type Property

To create a type property, rather than a stored property, you use the `static` keyword.

You already used the type property `CGSize.zero`. `CGPoint` also has a type property of `.zero` and defines a 2D point with values in `x` and `y`. Examine part of the `CGPoint` structure definition to see both stored and type properties:

```
public struct CGPoint {
  public var x: CGFloat
  public var y: CGFloat
}

extension CGPoint {
  public static var zero: CGPoint {
    CGPoint(x: 0, y: 0)
  }
}
```

This is an example of using a `CGPoint`:

```
var point = CGPoint(x: 10, y: 10)
point.x = 20
```

When you create an instance of the structure `CGPoint`, you set up `x` and `y` properties on the structure. These `x` and `y` properties are unique to every `CGPoint` you instantiate.

To use `CGPoint`'s type property, you use the name of the type:

```
let pointZero = CGPoint.zero  // pointZero contains (x: 0, y: 0)
```

This sets up an instance of a `CGPoint`, named `pointZero`, with `x` and `y` values of zero.

When you instantiate a new structure, that structure stores its properties in memory separately from every other structure. A `static` or `type` property, however, is constant over all instances of the type. No matter how many times you instantiate the structure, there will only be one copy of the `static` type property.

In the following diagram, there are two copies of `CGPoint`, `pointA` and `pointB`. Each of them has its own memory storage area. `CGPoint` has a type property `zero` which is stored once.

Type property storage

Swift Tip: `CGPoint.zero` is defined as a computed property. It has a return value of `CGPoint(x: 0, y: 0)`, and you can't set it to any other value. There is no effective difference between defining `.zero` as a computed property or as `static let zero = CGPoint(x: 0, y: 0)`. It is a stylistic choice.

Creating Global Defaults for Cards

Going back to your hard coded size values, you'll now create a file that will hold all your global constants.

➤ Create a new group called **Config**.

➤ In **Config**, create a new Swift file called **Settings.swift** and replace the code with:

```
import SwiftUI

struct Settings {
  static let cardSize =
    CGSize(width: 1300, height: 2000)
  static let thumbnailSize =
    CGSize(width: 150, height: 250)
  static let defaultElementSize =
    CGSize(width: 250, height: 180)
  static let borderColor: Color = .blue
  static let borderWidth: CGFloat = 5
}
```

Here you create default values for the final card size, the card thumbnail size, the card element size and for a border that you'll use later.

Notice that you created a structure. While this works, it could become problematic, because you could instantiate the structure and have copies of `Settings` throughout your app.

```
let settings1 = Settings()
let settings2 = Settings()
```

However, if you use an enumeration, you can't instantiate it, so it ensures that you will only ever have one copy of `Settings`.

➤ Change `struct Settings {` to:

```
enum Settings {
```

Using an enumeration and type properties in this way future-proofs your app. Later on, someone else might want to add another setting to your app. They won't need to change the enumeration itself, but they'll simply be able to create an extension.

For example, they could add a new type property like this:

```
extension Settings {
  static let aNewSetting: Int = 0
}
```

Extensions can hold type properties, but not stored properties.

➤ Open **CardThumbnail.swift**. Instead of defining the frame size here, you can rely on your settings defaults.

➤ Change `.frame(width: 150, height: 250)` to:

```
.frame(
  width: Settings.thumbnailSize.width,
  height: Settings.thumbnailSize.height)
```

➤ Similarly, open **Transform.swift** and change `var size = CGSize(width: 250, height: 180)` to:

```
var size = CGSize(
  width: Settings.defaultElementSize.width,
  height: Settings.defaultElementSize.height)
```

If you want to change these sizes later on, you can do it in `Settings`.

Creating Type Methods

As well as `static` properties, you can also create `static` methods. To illustrate this, you'll extend SwiftUI's built-in `Color` type. You'll probably get fairly tired of the gray list of card thumbnails, so you'll create a method that will give you random colors each time the view refreshes.

➤ Create a new group and name it **Extensions**. In **Extensions**, create a new Swift file called **ColorExtensions.swift** and replace the code with:

```
import SwiftUI

extension Color {
  static let colors: [Color] = [
    .green, .red, .blue, .gray, .yellow, .pink, .orange, .purple
  ]
}
```

You created an array of `Colors` that's available throughout the app by referencing `Color.colors`.

➤ Create a new method inside `Color`:

```
static func random() -> Color {
  colors.randomElement() ?? .black
}
```

This method returns a random element from the colors array and, if the colors array is empty, returns black.

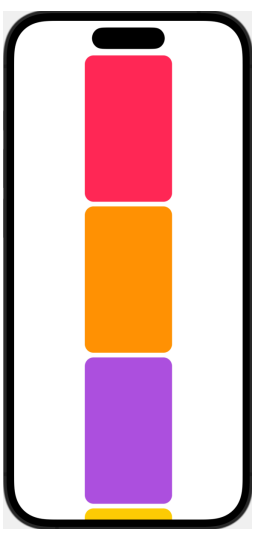
> **Swift Tip**: Astute readers will notice that this method could just as easily have been a `static var` computed property. However, conventionally, if you're returning a value that may change often, or there is complex code, use a method.

➤ Open **CardThumbnail.swift** and change `.foregroundColor(.gray)` to:

```
.foregroundColor(.random())
```

Here you use the `static` method that you created on `Color`. Each time you list the thumbnails, they will use different colors.

➤ Preview **CardsListView.swift** and see your random card colors. Each time you press the **Live** icon, the colors change.



Random color

Challenge

Challenge: Make new View Modifiers

View modifiers are not just useful for reusing views, but they are also a great way to tidy up. You can combine modifiers into one custom modifier. Or, as with the toolbar modifier in `SingleCardView`, if a modifier has a lot of code in it, save yourself some code reading fatigue, and separate it into its own file.

Your challenge is to create a new view modifier that takes the toolbar code and moves it into a modifier called `CardToolbar`.

To do this, you'll:

1. Create a new file to hold the view modifier.
2. Create a structure `CardToolbar: ViewModifier` and create a new method `body` that returns `content`, as you did when you made `ResizableView` a `ViewModifier`.
3. Remove the preview, as it doesn't make sense to have one for this modifier.
4. For `body`, cut the toolbar and sheet modifier code from `SingleCardView` and paste the modifiers on `CardToolbar`'s `content`.
5. In `CardToolbar`, you'll need the `dismiss` environment object and `currentModal` as a binding.
6. In `SingleCardView`, in `body`, add to `content` your new custom modifier: `.modifier(CardToolbar(currentModal: $currentModal))`.

When you've completed the challenge, your code should work the same, but, with this refactoring, `SingleCardView` is easier to read.

As always, you'll find the solution in the **challenge** folder for this chapter.

Key Points

- Custom gestures let you interact with your app in any way you choose. Make sure the gestures make sense. Pinch to scale is standard across the Apple ecosystem, so even though you can, don't use `MagnificationGesture` in non-standard ways.
- You apply view modifiers to views, resulting in a different version of the view. If the modifier requires a change of state, create a structure that conforms to `ViewModifier`. If the modifier doesn't require a change of state, you can make code more readable by adding a method to a `View` extension and use that method to modify a view.
- `static` or type properties and methods exist on the type. Stored properties exist per instance of the type. `Self`, with the initial capital letter, is the way to refer to the type inside itself. `self` refers to the instance of the type. Apple uses type properties and methods extensively. For example, `Color.yellow` is a type property.

Where to Go From Here?

By now you should be able to understand a lot of technical jargon. It's time to check out Apple's documentation and articles. Adding Interactivity with Gestures (https://apple.co/3isFhBO) is an article that describes updating state during a gesture. Read this article and check your understanding of the topic so far.

The Apple article Composing SwiftUI Gestures (https://apple.co/36meaVo), describes combining gestures in various ways.

Create your own modifiers. Any time you repeat your view's design, you should look at creating a method or a modifier that encapsulates that code.

Think about parts of your app in modules. In this chapter, you created a useful resizable view modifier which you can now use in any app that you create. When creating views, consider how you could abstract them and make them more generic.

Chapter 15: Structures, Classes & Protocols

By Caroline Begbie

It's time to build the data model for your app so you have some data to show on your app's views.

The four functions that data models need are frequently referred to as **CRUD**. That's **C**reate, **R**ead, **U**pdate, **D**elete. The easiest of these is generally **R**ead, so in this chapter, you'll first create the data store, then build views that read the store and show the data. You'll then learn how to **U**pdate the data and store it. That will leave **C**reate and **D**elete. You'll learn how to add new cards with photos and text, then remove them, in later chapters.

Starter Project Changes

There are a few differences between the challenge project from the last chapter and the starter project of this chapter:

- **Operators.swift**: contains new operators.
- **Preview Assets.xcassets**: contains three cute hedgehogs from https://pexels.com.
- **PreviewData.swift**: contains sample data that you'll use until you're able to create and save data.
- **TextExtensions.swift**: contains a new view modifier to scale text.

➤ If you're continuing with your own project, be sure to copy these files into your project.

Data Structure

Take another look at the back of the napkin sketch:

Back of the napkin sketch

Even with this rough sketch, you can get an idea of how to shape your data.

You'll need a top level data store that will hold an array of all the cards. Each card will have a list of elements, and these elements could be an image or text.

Data structure

You don't want to constrain yourself to image or text though, as you might add new features to your app in the future. Any data model you create now should be *extensible*, meaning as flexible as possible, to allow future capabilities.

Value and Reference Types

Skills you'll learn in this section: differences between value and reference types

Before creating the data model, you'll need to decide what types to use to store your data. Should you use structures or classes?

A Swift data type is either a value type or a reference type. Value types, like structures and enumerations, contain data, while reference types, like classes, contain a reference to data.

Value and reference types

At runtime, your app instantiates properties and assigns them to separate areas of memory, called the **stack** and the **heap**. Value types go on the stack, which the CPU manages and optimizes, so it's fast and efficient. You can instantiate structures, enumerations and tuples without counting the cost. The heap, however, is much more dynamic and allows an app to allocate and deallocate areas of memory, while maintaining reference counts. This makes allocating reference types less efficient. When you instantiate a class, that piece of data should stick around for a while.

Swift Dive: Structure vs Class

Skills you'll learn in this section: how to use structures and classes

When initializing classes and structures in code, they look very similar. For example:

```
let iAmAStruct = AStruct()
let iAmAClass = AClass()
```

The important difference here is that `iAmAStruct` contains immutable data, whereas `iAmAClass` contains an immutable reference to the data. The data itself is still mutable and you can change it.

```
iAmAStruct.number = 10  // compile error
iAmAClass.number  = 10  // no error - `number` will update to 10
```

When you assign value types, such as a `CGPoint`, you make a copy. For example:

```
let pointA = CGPoint(x: 10, y: 20)
var pointB = pointA    // make a copy
pointB.x = 20          // pointA.x is still 10
```

`pointA` and `pointB` are two different objects.

With a reference type, you access the same data. For example:

```
let iAmAClass = AClass()
let iAmAClassToo = iAmAClass
iAmAClassToo.number = 20     // this updates iAmAClass
print(iAmAClass.number)      // prints 20
```

Swift keeps a count of the number of references to the `AClass` object created in the heap. The reference count here would be two, and Swift won't deallocate the object until its reference count is zero.

Changing the data like this can be a source of errors for unwitting developers. One of Swift's principles is to prevent accidental errors, and if you favor value types over reference types, you'll end up with fewer of those accidents. In this app, you'll favor structures and enumerations over classes where possible.

Creating the Card Store

Skills you'll learn in this section: when to use classes and structures

Returning to the complex matter of deciding how to store your data, you need to choose between a structure and a class.

In general, when you hold a simple piece of data, such as a `Card` or a `CardElement`, those are lightweight objects that you won't need forever. Typically, you'd make those a structure. However, when you hold a data store that you're going to use throughout your app, that's a good candidate for a class. In addition, if your data has publisher properties, it must conform to `ObservableObject`, which requires you to use a class.

Now, you'll get started creating your data model, beginning at the bottom of the data hierarchy with the element.

➤ In the **Model** group, create a new Swift file called **CardElement.swift** and replace the code with:

```
import SwiftUI

struct CardElement {
}
```

This is the file where you'll describe the card elements. You'll come back to this shortly to define the data you'll hold.

➤ Create a new Swift file called **Card.swift** and replace the code with:

```
import SwiftUI

struct Card: Identifiable {
  let id = UUID()
  var backgroundColor: Color = .yellow
  var elements: [CardElement] = []
}
```

You set up Card to conform to `Identifiable` by defining the protocol's required property `id`. Later, you can use this unique id to locate a card and to iterate through the cards.

You also hold a background color for the card and an array of `elements` for all the images and text that you'll place on the card.

➤ Create a new Swift file named **CardStore.swift** and replace the code:

```
import SwiftUI

class CardStore: ObservableObject {
  @Published var cards: [Card] = []
}
```

`CardStore` is your main data store and your single *source of truth*. As such, you'll make sure that it stays around for the duration of the app. It isn't, therefore, a lightweight object, and you choose to make it a class.

There is a second reason for it to be a class. The protocol `ObservableObject` requires any type that conforms to it to be a class.

`ObservableObject` is part of the **Combine** framework. A class that conforms to `ObservableObject` can have published properties in it. When any changes happen to these properties, any view that uses them will automatically refresh. So when any card in the published array changes, views will react.

You've now set up a data model that SwiftUI can observe and write to. There is a difficulty with card elements, however. These can be either an image or text.

Class Inheritance

Skills you'll learn in this section: class inheritance; composition vs inheritance

You might have come across **object oriented programming** (OOP) in Swift or other languages. This is where you have a base object, and other classes derive — or **inherit** — from this base object. Swift classes allow inheritance. Swift structures do not.

You might set up your card element data in this way:

```
class CardElement {
  var transform: Transform
}

class ImageElement: CardElement {
  var image: Image?
}

class TextElement: CardElement {
  var text: String?
}
```

Here you have a base class `CardElement` with two sub-classes inheriting from `CardElement`. `ImageElement` and `TextElement` both inherit the `transform` property, but each type has its own separate relevant data.

As discussed earlier, however, lightweight objects such as card elements should be value types, not classes.

Composition vs Inheritance

With **inheritance**, you have tightly coupled objects. Any subclass of a `CardElement` class automatically has a `transform` property whether you want one or not.

You might possibly decide in a future release to require some elements to have a color. With inheritance, you could add `color` to the base class, but you'd then be holding redundant data for the elements that don't use a color.

An alternative scenario is to use **composition** with protocols, where you add only relevant properties to an object. This means that you can hold your data in structures.

This diagram shows a `CardElement` protocol with `ImageElement` and `TextElement` structures. It also shows a possible future expansion if you want to include a new `ColorElement`. This would be much harder with inheritance.

Composition

Traditionally, inheritance is considered to be an "is a" relationship, while composition is a "has a" relationship. But, you should avoid tightly-coupled objects as much as you can, and composition gives you much more freedom in design.

Protocols

> **Skills you'll learn in this section**: create protocol; conform structures to protocol; protocol method

You've used several protocols so far — such as `View` and `Identifiable` — and, possibly, been slightly mystified as to what they actually are.

Protocols are like a contract. You create a protocol that defines requirements for a structure, a class or an enumeration. These requirements may include properties and whether they are read-only or read-write. A protocol might also define a list of methods that any type adopting the protocol must include.

Protocols can't hold data; they are simply a blueprint or template. You create structures or classes to hold data and they, in turn, conform to protocols.

`View` is the protocol you've used most. It has a required property `body`. Every view that you've created has contained `body` and, if you don't provide one, you get a compile error.

You've also used `Identifiable`. `id` is a required property, so each time you conform a type to `Identifiable`, you create an `id` property that is guaranteed to be unique.

In your app, every card element will have a `transform`, so you'll change `CardElement` to be a protocol that requires any structure adopting it to have a `transform` property.

➤ Open **CardElement.swift** and replace the structure with:

```
protocol CardElement {
  var id: UUID { get }
  var transform: Transform { get set }
}
```

Here you create a blueprint of your `CardElement` structure. Every card element type will have an `id` and a `transform`. `id` is read-only, and `transform` is read-write.

➤ In the same file as `CardElement`, create the image element:

```
struct ImageElement: CardElement {
  let id = UUID()
  var transform = Transform()
  var image: Image
}
```

`ImageElement` conforms to `CardElement` with its required `id` and `transform`. It also holds an image.

➤ Create the text element after the image element:

```
struct TextElement: CardElement {
  let id = UUID()
  var transform = Transform()
  var text = ""
  var textColor = Color.black
  var textFont = "Gill Sans"
}
```

`TextElement` also conforms to `CardElement` and holds a string for text, the default text color and the default font.

With protocols, you future-proof the design. If you later want to add a new card element that is just a solid color, you can simply create a new structure `ColorElement` that conforms to `CardElement`.

`Card` holds an array of `CardElements`. `Card` doesn't care what type of `CardElement` it holds in its `elements` array, so it's easy to add new element types.

Creating a Default Protocol Method

A protocol blueprint might require the conforming type to implement a method. For example, this protocol requires all types that conform to it to implement `find()`:

```
protocol Findable {
  func find()
}
```

Sometimes you want a default method that is the same across all conforming types. For example, in your app, a card will hold an array of card elements. Later, you'll want to find the index for a particular card element.

The code for this would be:

```
let index = card.elements.firstIndex { $0.id == element.id }
```

This is quite hard to read and you have to remember the closure syntax. Instead, you can create a new method in `CardElement` to replace it.

➤ In **CardElement.swift**, under the protocol declaration, add a new method in an extension:

```
extension CardElement {
  func index(in array: [CardElement]) -> Int? {
    array.firstIndex { $0.id == id }
  }
}
```

This method takes in an array of `CardElement` and passes back the index of the element. If the element doesn't exist, it passes back `nil`. The way you'll use it is:

```
let index = element.index(in: card.elements)
```

This is a lot easier to read than the earlier code, and the complicated closure syntax is abstracted away in `index(in:)`. Any type that conforms to `CardElement` can use this method.

Now that you have your views and data model implemented, you have reached the exciting point of showing the data in the views. Your app doesn't allow you to add any data, so your starter project has some preview data to work with until you can add your own.

The Preview Data

Skills you'll learn in this section: using preview data

➤ In the **Preview Content** group, take a look at **PreviewData.swift** and remove the comment tags `/* */`. This code was commented to remove compile errors while you built your data model.

There are five cards. The first card uses the array of four elements, which are a mixture of images and text. You'll use this data to test new views. The card elements are positioned for portrait orientation on iPhone 14 Pro. As they are hard-coded, if you run the app in landscape mode or on a smaller device, some of the elements will be off the screen. Later, your card will take on a fixed size, and the elements will scale to fit in the available space.

➤ Open **CardStore.swift** and add an initializer to `CardStore`:

```
init(defaultData: Bool = false) {
  if defaultData {
    cards = initialCards
  }
}
```

When you first instantiate `CardStore`, the initializer will load the preview data when `defaultData` is `true`.

Later, when you can save and load cards from files, you'll update this to use saved cards. For the moment, you'll use the preview data.

You'll need to instantiate `CardStore`, and the best place to do that is at the start of the app.

➤ Open **CardsApp.swift** and add a new property to `CardsApp`:

```
@StateObject var store = CardStore(defaultData: true)
```

You use `@StateObject` to ensure that the data store persists throughout the app.

➤ Add a modifier to `CardsListView()` so you can address the data store through the environment:

```
.environmentObject(store)
```

➤ Open **CardsListView.swift** and add the new environment object to `CardsListView`:

```
@EnvironmentObject var store: CardStore
```

Whenever you create an environment object property, you should make sure that the SwiftUI preview instantiates it. If you don't do this, your preview will crash mysteriously with no error message.

➤ In `previews`, add a modifier to `CardsListView`:

```
.environmentObject(CardStore(defaultData: true))
```

Listing the Cards

Skills you'll learn in this section: observing full screen cover property

➤ Still in **CardsListView.swift**, in `list`, change `ForEach(0..<10) { _ in` to:

```
ForEach(store.cards) { card in
```

Here you iterate through `store.cards`. Remember that `ForEach` in this format requires `Card` to be `Identifiable`.

➤ Open **CardThumbnail.swift** and add a new property to `CardThumbnail`:

```
let card: Card
```

You don't need `card` to be mutable here, as you'll only read from it to get the card's background color for the thumbnail.

➤ Replace `.foregroundColor(.random())` with:

```
.foregroundColor(card.backgroundColor)
```

Instead of a random color, you use the background color of the card for the thumbnail.

➤ Update the preview to use the first card in the provided preview data:

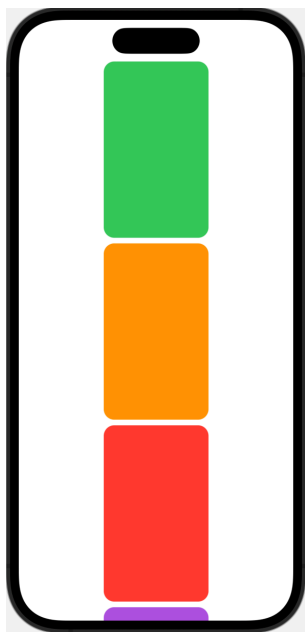
```
CardThumbnail(card: initialCards[0])
```

➤ Back in **CardsListView**, change `CardThumbnail()` to:

```
CardThumbnail(card: card)
```

You pass the current card to the thumbnail view.

➤ Preview the view and check that the scrolling card thumbnails use the background colors from the preview data:



The card thumbnails

Choosing a Card

When you tap a card, you set `isPresented` to `true`, which triggers the full screen modal for the single card. `SingleCardView` should now use the data for the selected card.

➤ Add a new property to `CardsListView`:

```
@State private var selectedCard: Card?
```

➤ Remove the property `isPresented` as you won't need it any more.

➤ In `.onTapGesture`, replace `isPresented = true` with:

```
selectedCard = card
```

➤ Replace `.fullScreenCover(isPresented: $isPresented) {` with:

```
.fullScreenCover(item: $selectedCard) { card in
```

When `selectedCard` is not `nil`, the system will show `SingleCardView` in the full screen modal. When you tap the **Done** button and dismiss the modal, the system will reset `selectedCard` to `nil`.

Displaying the Single Card

You can now pass the selected card to the single card view.

➤ Still in **CardsListView.swift**, change `SingleCardView()` to:

```
SingleCardView(card: card)
```

➤ Open **SingleCardView.swift** and add a new property to `SingleCardView`:

```
let card: Card
```

➤ Update `SingleCardView_Previews` to:

```
SingleCardView(card: initialCards[0])
```

➤ Change `content` to:

```
var content: some View {
  card.backgroundColor
}
```

With the background color, you'll be able to tell whether the app is displaying the correct selected card.

➤ Return to **CardsListView.swift** and Live Preview the app.

Selected card passed

As you select each card, the correct color for the card shows on the single card view.

Mutability

Skills you'll learn in this section: mutability

But wait! In `SingleCardView`, is `card` mutable? You'll want to add images and text to the card later on, so it does need to be mutable.

The answer, of course, is that you passed `card` with a `let` and therefore it is read-only. To get a mutable card, you need to access the selected card in the data store's `cards` array by index.

➤ Open **CardStore.swift** and create a new method:

```
func index(for card: Card) -> Int? {
  cards.firstIndex { $0.id == card.id }
}
```

This finds the first card in the array that matches the selected card's id and returns the array index, if there is one.

➤ Open **CardsListView.swift** and, in `body`, replace `SingleCardView(card:)` with:

```
if let index = store.index(for: card) {
  SingleCardView(card: $store.cards[index])
} else {
  fatalError("Unable to locate selected card")
}
```

You work out the array index of the selected card in the data store's `cards` array and pass it as a binding to `SingleCardView`. This should never fail but, just in case, you add a fatal error message.

➤ Open **SingleCardView.swift** and change `let card: Card` to:

```
@Binding var card: Card
```

The selected card is now mutable in this view.

➤ Change `SingleCardView_Previews` to:

```
SingleCardView(card: .constant(initialCards[0]))
```

You update the preview with a binding to the preview data.

➤ Preview **CardsListView.swift** and the result is the same as previously, but you're now all set up to update the card with new elements.

Adding Elements to the Card

➤ In the **Single Card Views** group, create a new SwiftUI View file named **CardDetailView.swift**.

This view will contain only the card and its elements.

➤ Replace the code in **CardDetailView.swift** with:

```
import SwiftUI

struct CardDetailView: View {
  // 1
  @EnvironmentObject var store: CardStore
  @Binding var card: Card

  var body: some View {
    // 2
    ZStack {
      card.backgroundColor
```

```
    }
  }
}

struct CardDetailView_Previews: PreviewProvider {
  static var previews: some View {
    // 3
    CardDetailView(card: .constant(initialCards[0]))
      .environmentObject(CardStore(defaultData: true))
  }
}
```

Here you:

1. Add a reference to the `CardStore` environment object and a `Card` binding.
2. Use the card's background color and put it inside a `ZStack`.
3. Pass a constant binding to `CardDetailView` and an instance of `CardStore` using `environmentObject(_:)`.

➤ Preview the view to see the background color from the first card in your preview data.



Card background from the preview data

Creating the Card Element View

➤ In the **Single Card Views** group, create a new SwiftUI View file named **CardElementView.swift**. This view will show a single card element.

➤ Under the existing `CardElementView`, create a new view for an image element:

```
struct ImageElementView: View {
  let element: ImageElement

  var body: some View {
    element.image
      .resizable()
      .aspectRatio(contentMode: .fit)
  }
}
```

This simply takes in an `ImageElement` and uses the stored image as the view.

➤ Create a new view for text under `ImageElementView`:

```
struct TextElementView: View {
  let element: TextElement

  var body: some View {
    if !element.text.isEmpty {
      Text(element.text)
        .font(.custom(element.textFont, size: 200))
        .foregroundColor(element.textColor)
        .scalableText()
    }
  }
}
```

In the same way, this view takes in a `TextElement` and uses the stored text, color and font.

> **Swift Tip**: To find out what fonts are on your device, first list the font families in `UIFont.familyNames`. A font family might be "Avenir" or "Gill Sans". For each family, you can find the font names using `UIFont.fontNames(forFamilyName:)`. These are the weights available in the family, such as "Avenir-Heavy" or "GillSans-SemiBold".

`scalableText(font:)` is in your starter project in **TextExtensions.swift** and is the same code as you used for scaling text in the previous chapter, refactored into a method for easy reuse.

Depending on whether the card element is text or image, you'll use one of these two views. Note the `!` in front of `!element.text.isEmpty`. `isEmpty` will be `true` if `text` contains `""`, and `!` reverses the conditional result. This way you don't create a view for any blank text.

With these two views as examples, when "future you" adds a new type of element, it will be easy to add a new view specifically for that element.

➤ Change `CardElementView` to this code:

```
struct CardElementView: View {
  let element: CardElement

  var body: some View {
    if let element = element as? ImageElement {
      ImageElementView(element: element)
    }
    if let element = element as? TextElement {
      TextElementView(element: element)
    }
  }
}
```

When presented with a `CardElement`, you can find out whether it's an image or text depending on its type.

➤ Change the preview to:

```
CardElementView(element: initialElements[0])
```

Here you show the first element which contains a hedgehog image. To test the text view, change the parameter to `initialElements[3]`.

➤ Preview the view.

The card element view

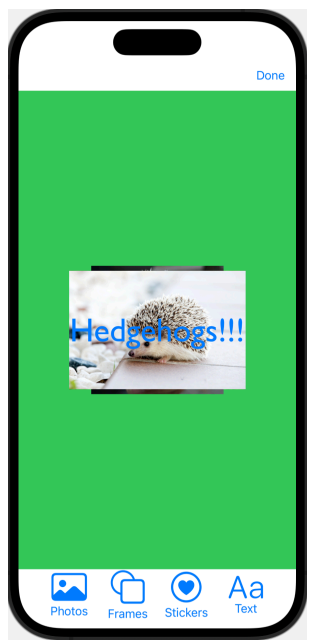
Showing the Card Elements

➤ Open **CardDetailView.swift** and, in `body`, add this after `card.backgroundColor`:

```
ForEach($card.elements, id: \.id) { $element in
  CardElementView(element: element)
    .resizableView()
    .frame(
      width: element.transform.size.width,
      height: element.transform.size.height)
}
```

Always be aware of whether your data is mutable. Later, you'll update the element's `Transform` within this `ForEach` loop. Generally, when you iterate through an array in a loop, the individual item is immutable. However, this variant of `ForEach` allows binding syntax by adding the `$` in front of the array and the individual item.

➤ Live Preview the view and see the elements all in the center of the view:



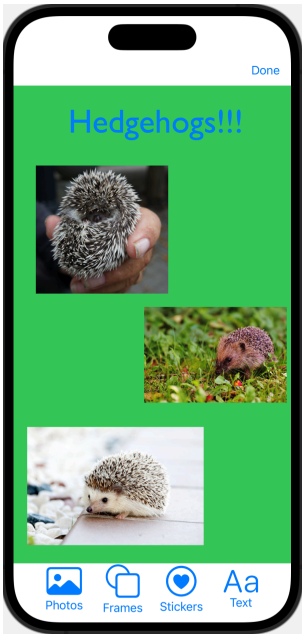
The card elements

➤ Open **SingleCardView.swift** and, in `body`, replace `content` with:

```
CardDetailView(card: $card)
```

➤ Remove the `content` property, as you no longer need it.

➤ Live Preview the app, or run in Simulator. Load the first card and move around the elements.



Move the card elements

Notice that when you return to the card list and reload the card, the element positions all reset to the center.

You've now completed the **R** in **CRUD**. Your views read and display all the data from the store. You'll now move on to **U** — updating the model when you resize, move and rotate card elements.

Understanding @State and @Binding Property Wrappers

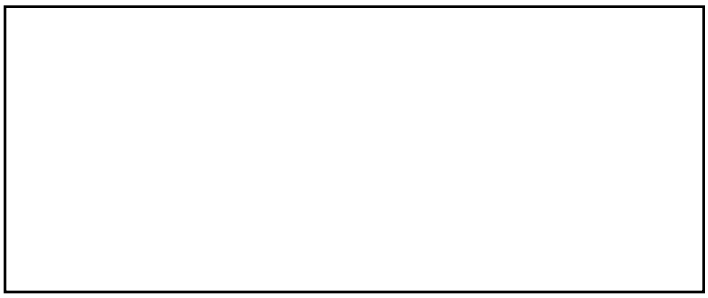
> **Skills you'll learn in this section**: `@State`; binding; generics

At the moment, you're using a state property `transform` inside `ResizableView`. You'll replace this with a binding to the current element's `Transform` property.

As you've learned already, inside a `View`, all properties are immutable unless they are created with a special property wrapper. A state property is the owner of a piece of data that is a source of truth. A binding connects a source of truth with a view that changes the data.

Your source of truth for all data is `CardStore`. When you select a particular card, you pass a binding to the card to `SingleCardView`.

➤ Open **SingleCardView.swift** and locate where you call `CardDetailView`. **Option-click** the `card` parameter to see the declaration.



CardDetailView declarations

`CardDetailView` expects an environment object and a binding. The type of these are in angle brackets. `store` is an environment object of type `CardStore`, and `card` is a binding of type `Card`.

Swift Dive: A Very Brief Introduction to Generics

Swift is a strongly typed language, which means that Swift has to understand the exact type of everything you declare. `Binding` has a **generic** type parameter `<Value>`. A generic type doesn't actually exist except as a placeholder. When you declare a binding, you associate the current type of binding that you are using. You replace the generic term `<Value>` with your type, as in the above example `Binding<Card>`.

Another common place where you might find this language construct is an `Array`. You defined an array in `CardStore` like this:

```
var cards: [Card] = []
```

That is actually syntactic sugar for:

```
var cards: Array<Card> = []
```

`Array` is a structure defined as `Array<Element>`. When you declare an array, you specify what the generic type `Element` actually is. In this example, `Element` is a `Card`. If you try and put anything other than a `Card` into that array, you'll get a compile error.

Binding Transform Data

Now that you've seen how generics work when composing a binding declaration, you'll be able to pass the element's transform to `resizableView()`, and `ResizableView` will connect to this binding instead of updating its own internal state `transform` property.

➤ Open **ResizableView.swift** and replace `@State private var transform = Transform()` with:

```
@Binding var transform: Transform
```

`transform` is now connected to the transform property in the parent view.

➤ In the `View` extension, replace `resizableView()` with:

```
func resizableView(transform: Binding<Transform>) -> some View {
  modifier(ResizableView(transform: transform))
}
```

The method receives a binding that is of `Transform` type and passes it on to the view modifier.

➤ In `ResizableView_Previews`, change `.resizableView()` to:

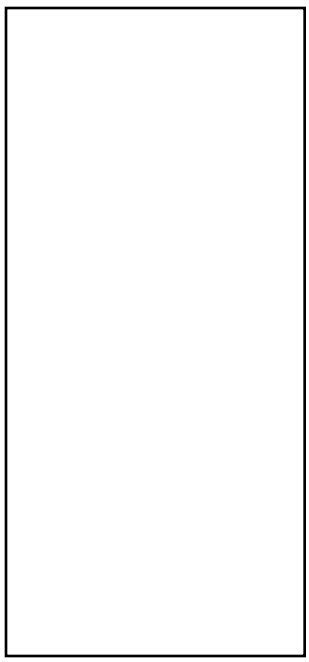
```
.resizableView(transform: .constant(Transform()))
```

This passes in a new transform instance as a binding.

➤ Open **CardDetailView.swift** and, in `body`, replace `.resizableView()` with:

```
.resizableView(transform: $element.transform)
```

`ResizableView` will now operate on the mutable element's transform property, and the preview places the card elements in the correct position.



Elements in their correct position

Updating the Previews

Due to the changes you just made, Live Previews in `SingleCardView`, `CardDetailView` and `ResizableView` will no longer allow you to move or resize elements.

➤ In **CardDetailView.swift**, locate the preview.

The parameter to `CardDetailView` is `.constant(initialCards[0])`. As its name implies, the binding value sent to `CardDetailView` is constant and, therefore, doesn't allow updates. You can get around this by creating a separate `View` structure.

➤ Replace `CardDetailView_Previews` with:

```
struct CardDetailView_Previews: PreviewProvider {
  struct CardDetailPreview: View {
    @EnvironmentObject var store: CardStore

    var body: some View {
      CardDetailView(card: $store.cards[0])
    }
  }

  static var previews: some View {
    CardDetailPreview()
      .environmentObject(CardStore(defaultData: true))
  }
}
```

In the preview, you're more closely reproducing the parent code that calls `CardDetailView`, and you're able to move and resize the elements. Any changes you make in position or size will save to the data store.

> **Note**: If you want your Live Previews to work in **SingleCardView.swift** and **ResizableView,swift**, you can reproduce this technique in those files.

There is still one problem. When you first reposition any element except for the central one, it jumps to a different position.

➤ Open **ResizableView.swift** and look at `dragGesture`.

`dragGesture` relies on `previousOffset` being set to an existing offset. On first loading the view, you should copy `transform.offset` to `previousOffset`.

➤ In `body`, add a new modifier to `content`:

```
.onAppear {
  previousOffset = transform.offset
}
```

When the view first appears, you initialize `previousOffset`. This will happen only once.

➤ Build and run and choose the first card. In the detail view, the initial position jump has gone away, and you can now move, rotate and resize the card elements.

Updating the card

You have now achieved both **R**ead and **U**pdate in the **CRUD** functions. In the next chapter, you'll learn how to **C**reate new image elements, and later, you'll tackle **D**eletion.

Key Points

- Use value types in your app almost exclusively. However, use a reference type for persistent stored data. Your stored data should be in one central place in your app.
- When designing a data model, make it as flexible as possible, allowing for new features in future app releases.
- Use protocols to describe data behavior. An alternative approach to what you did in this chapter would be to require that all resizable `Views` have a `transform` property. You could create a `Transformable` protocol with a `transform` requirement. Any resizable view must conform to this protocol.
- You had a brief introduction to generics in this chapter. Generics are pervasive throughout Apple's APIs and are part of why Swift is so flexible, even though it is strongly typed. Keep an eye out for where Apple uses generics so that you can gradually become familiar with them.
- When designing an app, consider how you'll implement CRUD. In this chapter, you implemented Read and Update. Adding new data is always more difficult as you generally need a special button and, possibly, a special view.

Where to Go From Here?

You covered a lot of Swift theory in this chapter. Our team's book Swift Apprentice (https://bit.ly/3RZhRWL) contains more information about how and when to use value and reference types. It also covers generics and protocol oriented programming.

If you're still confused about when to use class inheritance and OOP, watch this classic WWDC video (https://apple.co/3k9GUEM) where the protagonist, Crusty, firmly declares "I don't do object-oriented".

Chapter 16: Adding Assets to Your App

By Caroline Begbie

Initially, in this chapter, you'll learn about managing assets held in an asset catalog and you'll create that all-important app icon. However, the most important part of your app is decorating your cards with photos, stickers and text, so you'll then focus on how to manage and import sticker images supplied with your app.

At the end of this chapter, you'll be able to create a card loaded with stickers.

The Asset Catalog

Skills you'll learn in this section: app icons; vector vs bitmap; managing images in asset catalogs; screen resolution

Asset catalogs are by far the best place to manage image and color sets.

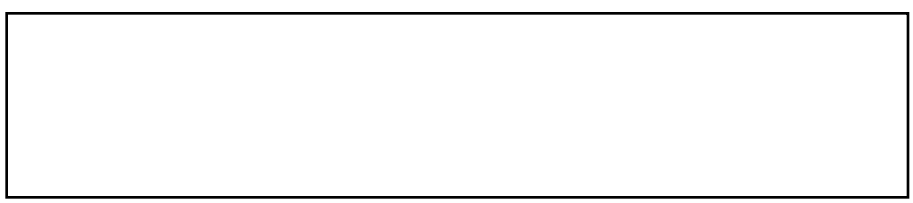
Within an asset catalog, under one image set, you can define multiple images for different themes, different devices, different scales and even different color gamuts. When you use the name of the image set in your code, the app will automatically load the correct image for the current environment. When you supply differently scaled images for different devices in an asset catalog, the app store will automatically do **app thinning** and only download the relevant images for that particular device. This is potentially a huge reduction in app download size.

The asset catalog also holds the app icon, the launch screen image and launch screen background color.

Adding the App Icon

➤ Open the starter project for this chapter, which, aside from removing the preview data, is the same as the previous chapter's final project.

➤ Click the project name **Cards** at the top of the Project navigator. Choose the target **Cards**. On the **General** tab, find **App Icons and Launch Screen**:

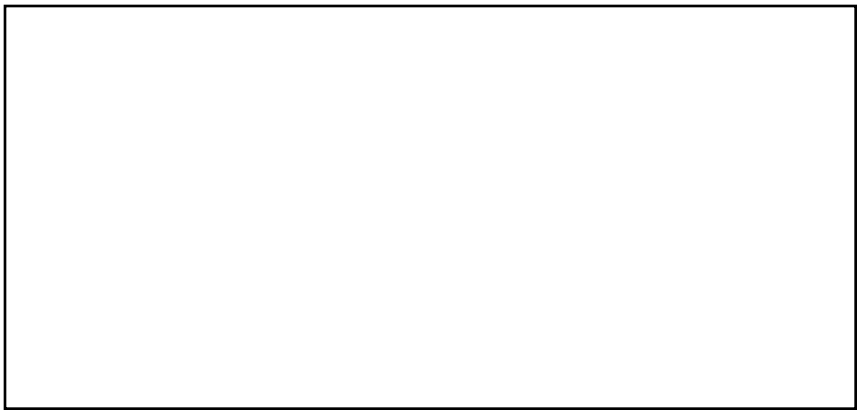


Source for app icons

This is where you specify which icon set to use for your app. You can choose to hold the icons in folders instead of asset catalogs, but it's much easier to keep them in the asset catalog as Apple intended. You can even change the icon for your app dynamically, by checking **include all app icon assets**, and defining multiple icon sets in **Assets.xcassets**.

➤ Open **Assets.xcassets** and select **AppIcon**.

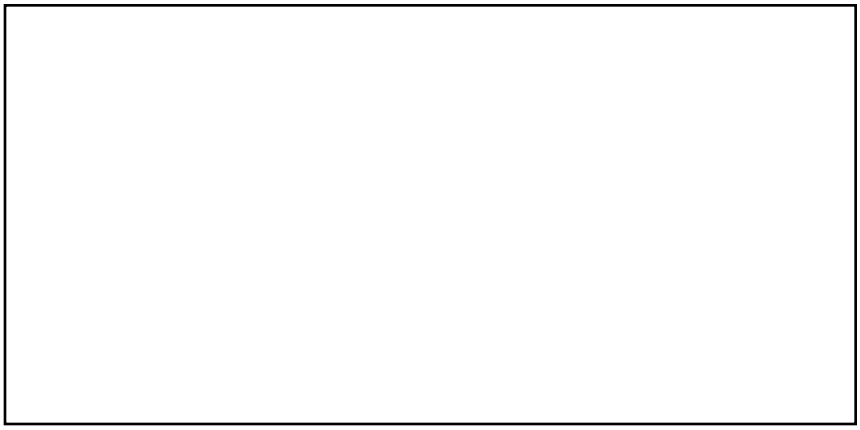
The iOS app template created an empty icon set called **AppIcon** when you first created your project.



Waiting for an icon

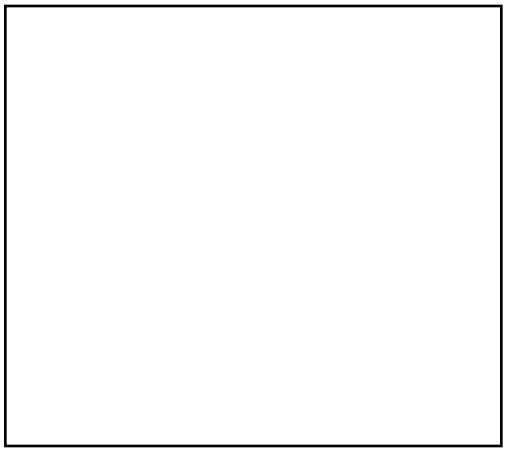
If you're lucky enough to have a designer for your app, as we are, they will distribute a design file, not code. This might be Sketch files or, as in our case, a Figma file.

The designer for this app, Lea Marolt (https://twitter.com/hellosunschein), created all the assets for the app in Figma, a "freemium" vector graphics prototyping tool. You can use Figma in the web interface at https://www.figma.com or download the companion app available from that link. In the **assets** folder for this chapter, you'll find a `.fig` file, which you can import into Figma. As you will see, some design suggestions don't always make it to the shipped product.



A Figma design file

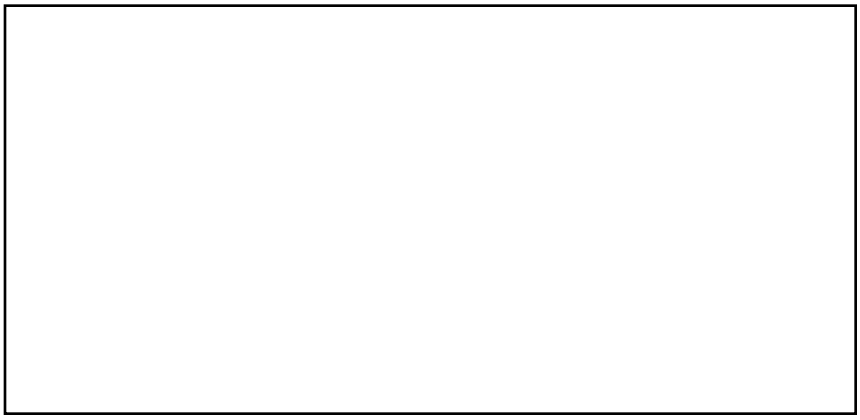
This figma file includes the app icon, itself designed in Figma with different vector shapes:



The app icon in Figma

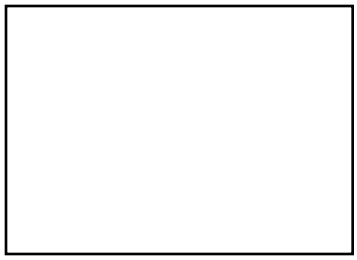
➤ In Finder, open the **assets** folder for this chapter and locate **app-icon.png**. This file has been exported from Figma and is 1024 by 1024 pixels.

➤ With the **AppIcon** set selected in Xcode, drag **app-icon.png** to the icon area.



The app icon

➤ Build and run, and swipe up from the bottom to exit your app. You'll see your new icon instead of the old placeholder icon.



App icon in use

Any asset you import into the asset catalog is highly configurable. For example, app icons are different sizes on different devices. If you wish to have different icons for different devices, you can.

➤ Select the new icon you just dragged in, and show the **Attributes inspector**. Change **iOS** from **Single Size** to **All Sizes**.



Configure multiple icons

You can now drag different **PNG** image files to the various icon spots.

➤ That's a lot of work, so change **All Sizes** back to **Single Size**.

Vector vs Bitmap

You imported a bitmap PNG image for the icon. For other assets, you can use vector formats, such as PDF or SVG. When possible, it's always better to use vector formats. These are made up of lines, curves and fills. For a vector line, you can set a start point and an end point. When you scale the line, the vector resizes without losing any of its resolution. With a bitmap line, you must stretch or compress pixels.

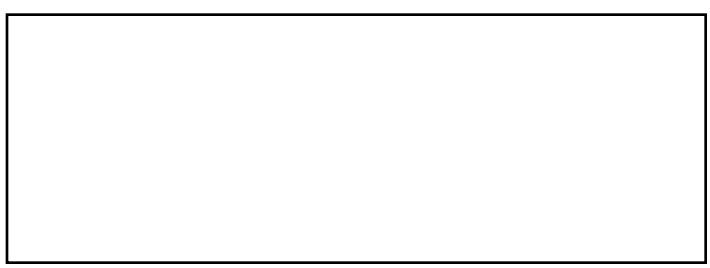
This image shows two 50 pixel wide images scaled up by twelve to 600 pixels. One is bitmap and the other is vector. You can see the vector image loses none of its sharpness.

Bitmap vs vector

Adding a Vector Image

Later, your app will need a placeholder image to show whether there are any errors in loading an image.

➤ In Finder, drag in **error-image.svg** from the **assets** folder for this chapter to the asset catalog panel under **AppIcon**.



Error image

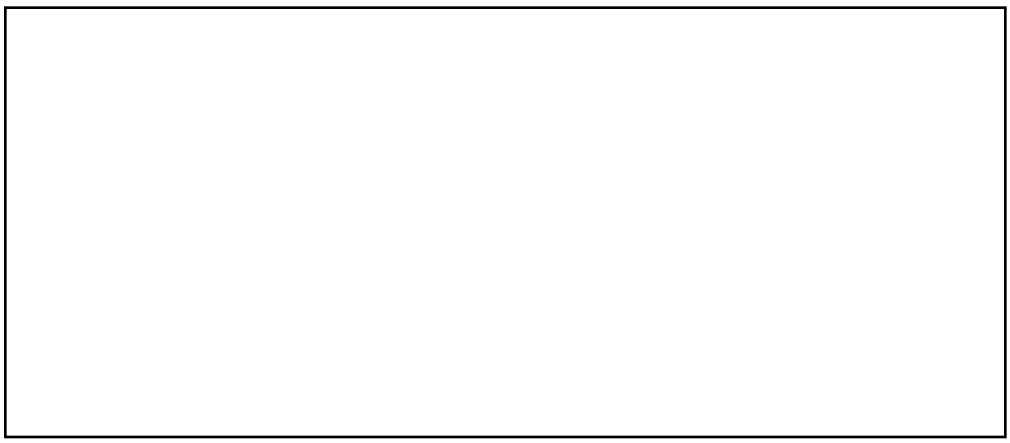
The image imports into Xcode in the **1x** space, and leaves the **2x** and **3x** spaces empty. These spaces are for devices with different resolutions.

Device Resolutions and Image Scale

Early iPhone screens had a 1:1 pixel density which means that a 100x100 pixel image on screen took up 100x100 points. iPhone 4 introduced the **retina** screen. Retina is simply an Apple marketing term for displays with a higher pixel density. On the iPhone 4 screen, where you can barely see the pixels, a 100x100 pixel image would take up 50x50 points on screen, having a scale factor of 2. iPhone 6s Plus came along, introducing a 3:1 pixel density. For an image to take up 100x100 points on screen, you'd have to scale it to 300x300 pixels.

When you provide bitmap assets, you must provide them for every device resolution. However, **error-image.svg** is a vector format image with a native size of 512x512. You don't need to scale it by 2x and 3x as Xcode can do this for you.

➤ With the error image selected, in the **Attributes inspector**, change **Scales** from **Individual Scales** to **Single Scale**.



Single scale

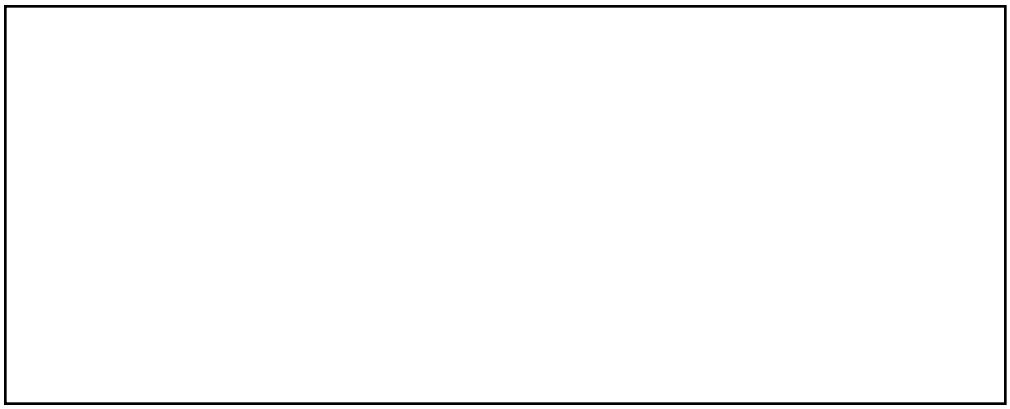
Xcode removes the 2x and 3x options in the center panel. When you build for a 2x resolution device, Xcode will automatically add to your app bundle a 512x512 optimized bitmap image scaled to the correct 2x resolution. Bundle images are held in a .car file, the format of which is not publicly available, so you can't inspect what Xcode has done.

Launch Screen

Skills you'll learn in this section: launch screen; size classes

Another use for the asset catalog is to hold a launch screen image and background color that displays while your app is launching. You've already come across **Info.plist** in Chapter 7, "Saving Settings". This **.plist** file is where you'll set the launch image and color.

➤ Click **Cards** at the top of the Project navigator and choose the **Cards** target. Choose the **Info** tab, and you'll see the contents of **Info.plist** in the **Custom iOS Target Properties** section.



Info.plist

You can add new items either by right-clicking an entry and choosing **Add Row** or by moving your cursor over an item and clicking the + sign that appears. You can delete items by clicking the - sign.

➤ Click the disclosure control next to **Launch Screen**, then add items for **Image Name** and for **Background Color**.

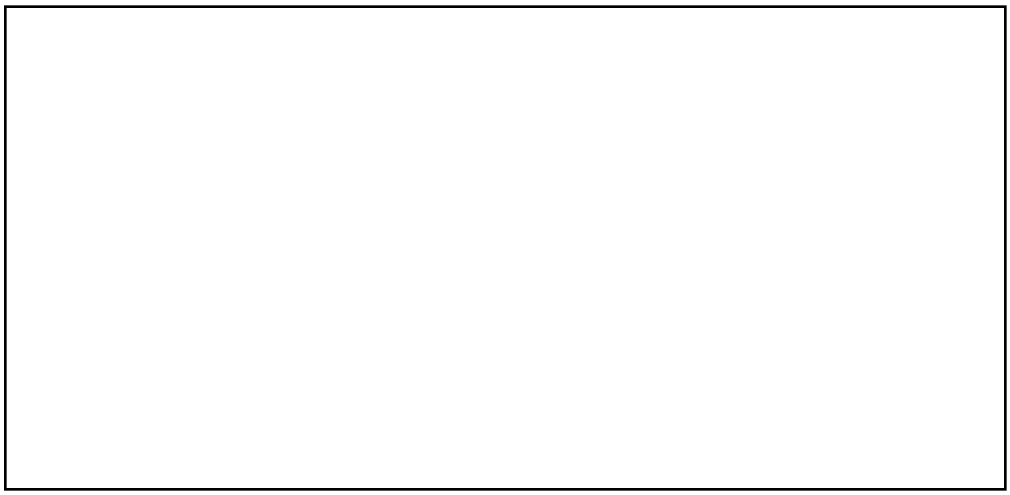
➤ Double-click in the **Value** field for **Image Name** and enter:

```
LaunchImage
```

➤ Double-click in the **Value** field for **Background color** and enter:

```
LaunchColor
```

You may need to resize the columns to show the **Value** column.



Launch configuration

➤ Open **Assets.xcassets**, click the + sign at the bottom of the assets panel and choose **Image Set**. Rename **Image** to **LaunchImage**.

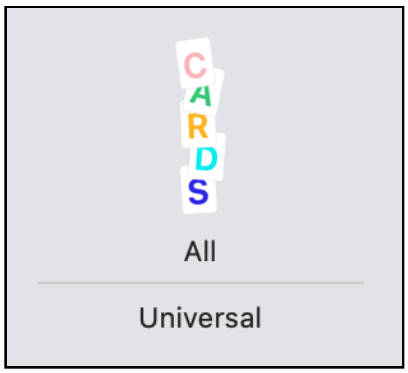
➤ Click the + sign at the bottom of the assets panel again and choose **Color Set**. Rename **Color** to **LaunchColor**.

When you run your app now, the app will use these for the launch screen. Unfortunately, the simulator doesn't clear launch screen caches, so if you change your launch image or color, in Simulator, you'll have to go to **Device > Erase All Contents and Settings...** and clear the simulator completely. On a device, deleting the app should be sufficient, but you might have to restart the device as well.

➤ Click **LaunchImage** in the catalog. You have the option of filling the three images. However, just as with the error image, you'll use a single scale SVG image.

➤ In Finder, open **assets/Launch Screen**. Drag in **launch-screen-light.svg** to the 1x spot.

➤ In the **Attributes inspector**, change **Scales** to **Single Scale**.



Launch Image

This SVG with a transparent background has a native size of 200x500px. At build time, Xcode will create the appropriately scaled bitmap image from this and display it in the center of the screen. When you launch the app in landscape on iPhone, you'll need an image with a smaller height, so you'll use size classes to decide which image to load.

Size Classes

Size classes represent the content area available using horizontal and vertical traits. These two traits can be either regular or compact. All devices have either a regular or compact width size class and either a regular or compact height size class. You can find a list of these size classes in Apple's Human Interface Guidelines (https://apple.co/348lVx0) under the section **Device size classes**.

This is an illustration of some iPhones and iPads laid on top of each other:

Size classes

For height in portrait mode, all devices fit into the regular height section. In landscape, all iPhones use compact height, but some larger iPhones use regular width rather than the smaller compact width.

iPads are always regular width and regular height. However, you still have to take into account size classes on iPad, because of split screen and resizing app windows. When in portrait mode, split screen apps are both compact width. In landscape mode, the user can size between compact width and regular width.

For your app, the current launch image will fit on all devices except for iPhones in landscape. So you'll specify a sideways image for compact height.

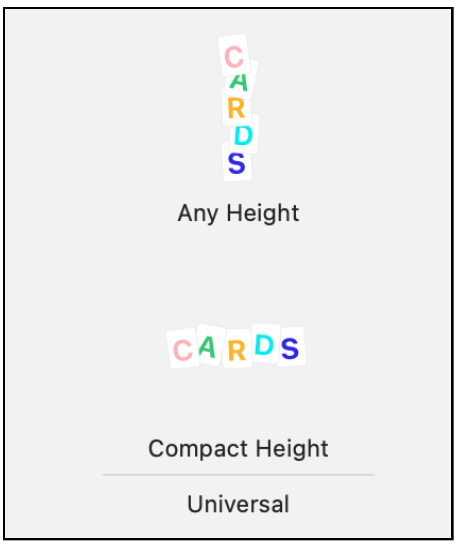
➤ In the **Attributes inspector**, change **Height Class** to **Any & Compact**.



Any & Compact height size class

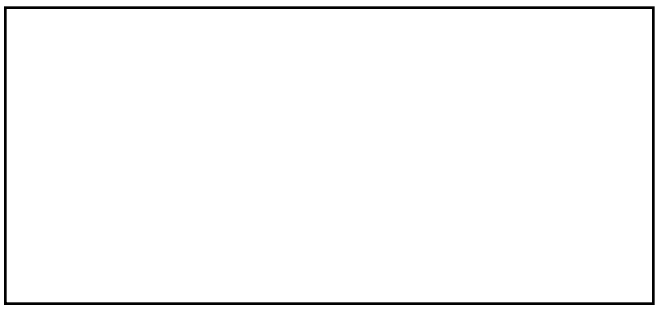
A new space opens up for the compact height image.

➤ In Finder, from **assets/Launch Screen**, drag in **launch-screen-landscape-light.svg** to the compact height image space.



Launch screen images

➤ Build and run, and your launch screen should show up briefly before your app does. Try rotating the simulator to get the different landscape launch screen. If your launch screen doesn't show up, remember to erase the simulator contents.



Launch screen in landscape

Now that you've got the hang of adding new images for checked conditions in the Attributes inspector, you'll be able to complete the challenge for Dark Mode at the end of the chapter.

> **Note**: At the time of writing, there appears to be a bug in scaling. The SVG image sometimes stretches to full screen. If this bug persists, you would have to resize the image yourself to fit an iPad screen, instead of relying on the asset catalog to manage scaling.

Adding Sticker Images to Your App

> **Skills you'll learn in this section**: groups; reference folders; loading images from files; lazy loading

There's one thing that an asset catalog does not allow you to do. You can't enumerate all the images it contains.

When you release your cards app, one way of making it stand out from the crowd is to have some excellent stickers.

You could still add the stickers to an asset catalog, but you'd have to keep track of how many there are and ensure that you have a strict naming convention. All items in asset catalogs need to have names that are unique in the app bundle.

As your app becomes more popular, you'll probably add more stickers and, maybe, categorize them into themes. It would be cumbersome to list each asset by name. You might have multiple artists working on stickers, and you wouldn't want them to have access to your project.

A solution to this is to use **reference folders**. Instead of using an asset catalog, you'll keep your sticker folder outside of your project and access it from the project as a reference folder.

➤ In Finder, take a look at the **assets/Stickers/Camping** folder. Shortly, you'll add all these PNG files to your app.

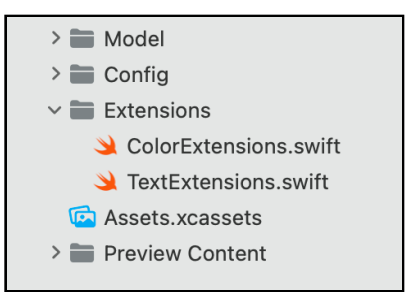


Camping stickers

> **Note**: These stickers are from Pixabay (https://bit.ly/3vojAJf). There are several sites, such as https://unsplash.com and https://www.pexels.com, where creators share their work and allow reuse of images. Before adding an image to your app, always check that the license allows commercial use and follow the license instructions. The stickers use Pixabay's license: free for commercial use with no attribution.

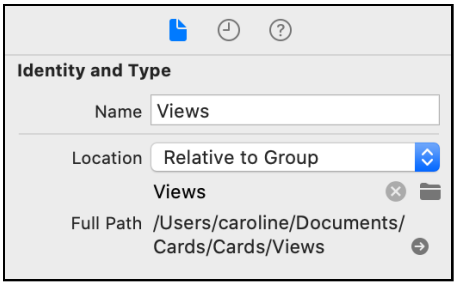
Xcode Groups

When you look in the Project navigator, currently all your groups, except for the asset catalog, have gray folder icons.



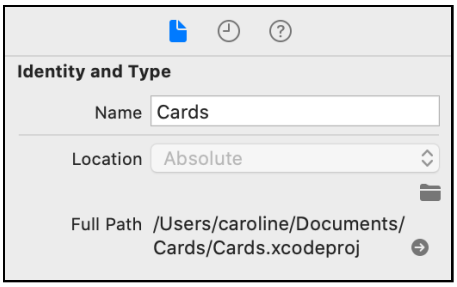
Gray folder icons

➤ Click the **Views** group and open the **File inspector**.



Identity and Type for group

Location describes how to store the group. **Relative to Group** means that Xcode traverses up the hierarchy of groups and folders to find an **Absolute** path. In your app's case, this is the very top **Cards** item in the Project navigator.

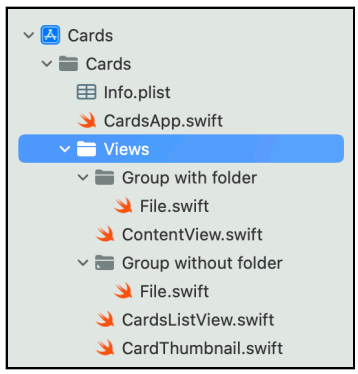


Absolute location

When you create a new group, you can choose to mirror that group with a folder on disk. If you have a file selected inside a group connected to a disk folder, and you create a new group with **File ▸ New ▸ Group**, then Xcode will create both a new group and a new folder.

If your current selection is inside a logical group without a mirrored folder on disk, then Xcode won't create a new folder for a new group. The option under **File ▸ New ▸ Group** will do the opposite. It changes between **Group with Folder** or **Group without Folder** depending upon whether your currently selected file is inside a mirrored group or not.

If your current file is in a logical group, the gray icon has a tiny triangle at the bottom left.



Groups

This is the file and folder organization in Finder:



Organization in Finder

Notice that **File.swift** isn't in a physical subfolder because it's contained in a logical group in Xcode.

Reference Folders

Unlike groups, Xcode doesn't organize reference folders at all. When you bring a reference folder into the project, it will have a blue icon and its hierarchy will reflect the disk hierarchy.

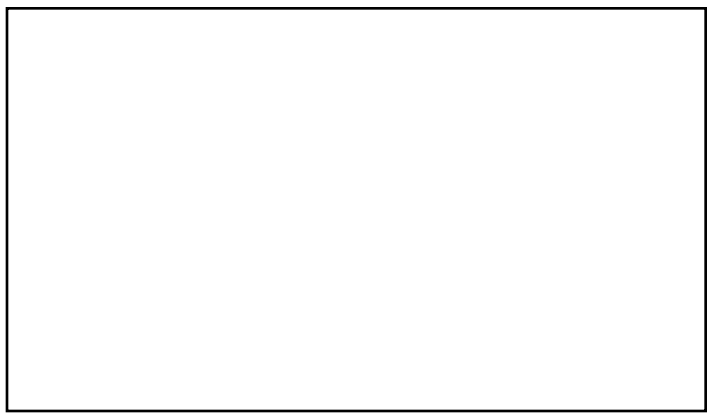
➤ In Finder, locate **assets/Stickers**.

You'll treat this folder as the master folder for your sticker assets. Any stickers that your artists create should go into this folder.

➤ Drag the Stickers folder into the Xcode Project navigator.

Uncheck **Copy items if needed** and choose **Create folder references**.

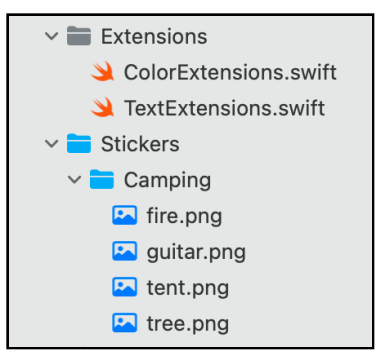
➤ Fill out the import screen with the following:



Add a reference folder

> **Warning**: Whenever you drag a file or folder into Xcode, make sure you examine these settings. You'll usually check **Copy items if needed**, and generally, you want to create groups, not folder references.

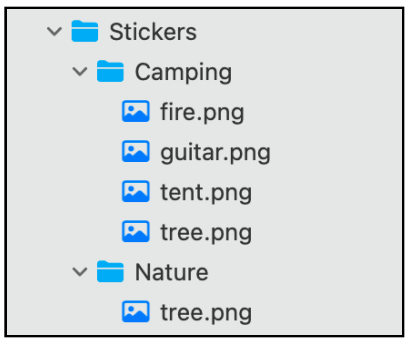
You now have a blue folder called **Stickers** in your project, with a blue sub-folder of **Camping**. The blue folder marks it as a reference folder. Xcode will only allow you to create folders inside this folder, not groups.



Reference folder

➤ In Finder, create a new folder inside Stickers called **Nature** and copy **Camping/tree.png** to this folder. Xcode will immediately update its hierarchy to reflect what's happening on disk.

If you tried this with gray groups (**don't!**), Xcode wouldn't be able to find any files you moved in Finder.



Reference folder with new sub-folder

In this folder hierarchy, you have two images with the same name **tree.png**. In an app bundle or in an asset catalog, you can't have images with the same name, but this works here because the app contains the reference folder hierarchy.

Another advantage with reference folders, is that with **Stickers** as the top level folder, your artists can create new themes in different folders without touching the Xcode project.

➤ Delete the **Nature** folder as you don't need it.

> **Note**: Sometimes your project may lose the reference to the Stickers folder of images. In this case, **Stickers** will appear in the Project navigator in red. Choose the red folder name and, in the Attributes inspector, tap the folder icon under Location. Navigate to your Stickers folder and click **Choose**. Alternatively, you can delete this red item and re-import the Stickers folder as a reference folder. If you ever want confirmation of where the folder is, right-click the folder and choose **Show in Finder**.

Loading Files From Reference Folders

Now, you'll create a Sticker view that loads images from the Stickers folder.

➤ In **Single Card Views**, create a new sub-group called **Card Modal Views**, and in this group, create a new SwiftUI View file called **StickerModal.swift**.

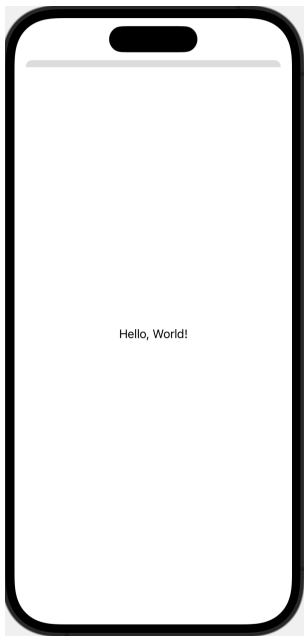
➤ Open **CardToolbar.swift** and locate `.sheet(item: $currentModal)`.

➤ Add a new case to the `switch` statement before the `default` case:

```
case .stickerModal:
  StickerModal()
```

➤ Open **SingleCardView.swift**, Live Preview it, and pin the preview, so you can access it from other views.

➤ Try out your new stickers modal by tapping **Stickers** in live preview.



New Sticker modal view

Loading a SwiftUI `Image` from a folder is not as easy as loading from an asset catalog. Asset catalogs do a lot of the heavy lifting. For example, if you add a vector file to an asset catalog, Xcode will convert to a native pixel format automatically, whereas it's not an easy task to load a vector file from a folder.

When you load an image from a folder, you load it into an instance of UIKit's `UIImage`. You also need to provide the full app bundle resource path.

➤ Open **StickerModal.swift** and replace `body` with:

```
var body: some View {
  // 1
  if let resourcePath = Bundle.main.resourcePath,
    // 2
```

```
      let image = UIImage(named: resourcePath +
        "/Stickers/Camping/fire.png") {
        Image(uiImage: image)
    } else {
      EmptyView()
    }
  }
```

Going through the code:

1. Get the full resource path of the app bundle.
2. Load the `UIImage` using the full name and path of the sticker and use the `uiImage` parameter for creating the `Image` view.

➤ In Live Preview, on the pinned Single Card View, tap the **Stickers** button and you'll see the sticker image.



Fire sticker

However, you don't only want one sticker, you want to see all of them. Depending on how many stickers you have, you shouldn't load up all the `UIImages` at once, as loading images is resource heavy and will block the user interface.

You can load the file names up front and, as the user scrolls, load the image when it's needed. This is called **lazy loading**.

➤ In `StickerModal`, create a new type method to load the sticker names:

```
static func loadStickers() -> [String] {
  var themes: [URL] = []
  var stickerNames: [String] = []
}
```

You'll first load the folder names in the top level in the **Stickers** folder. These will be themes. You'll be able to add new themes to your app in the future simply by adding a new folder inside the **Stickers** folder in **Finder**. You won't have to change any code to do this.

➤ Add this code at the end of `loadStickers()`:

```
// 1
let fileManager = FileManager.default
if let resourcePath = Bundle.main.resourcePath,
  // 2
  let enumerator = fileManager.enumerator(
    at: URL(fileURLWithPath: resourcePath + "/Stickers"),
    includingPropertiesForKeys: nil,
    options: [
      .skipsSubdirectoryDescendants,
      .skipsHiddenFiles
    ]) {
      // 3
      for case let url as URL in enumerator
      where url.hasDirectoryPath {
        themes.append(url)
      }
}
```

Going through the code:

1. Load the default file manager and bundle resource path.
2. Get a directory enumerator, if it exists, for the **Stickers** folder. For the `options` parameter, you skip subdirectory descendants and hidden files. Unless you skip the subdirectories, an enumerator will continue down the hierarchy. You currently just want to collect the top folder names as the themes.
3. If the URL is a directory, add it to `themes`.

Next you'll iterate through all the theme directories and retrieve the file names inside.

➤ Add this code after the previous code:

```
for theme in themes {
  if let files = try?
  fileManager.contentsOfDirectory(atPath: theme.path) {
    for file in files {
      stickerNames.append(theme.path + "/" + file)
    }
  }
}
return stickerNames
```

For each theme folder, you retrieve all the files in the directory and append the full path to `stickerNames`. You then return this array from the method.

➤ Create a new method in `StickerModal` to load a `UIImage` from a path:

```
func image(from path: String) -> UIImage {
  print("loading:", path)
  return UIImage(named: path)
    ?? UIImage(named: "error-image")
    ?? UIImage()
}
```

You temporarily print out the path name so that you can check whether you're lazily loading the image. You then return the `UIImage` loaded from the path name. If you can't load the image, return the error image from the asset catalog that you created earlier. As this is still optional and you need to return a non-optional, if everything fails, create a blank `UIImage`.

➤ Create a new property in `StickerModal` to hold the file names:

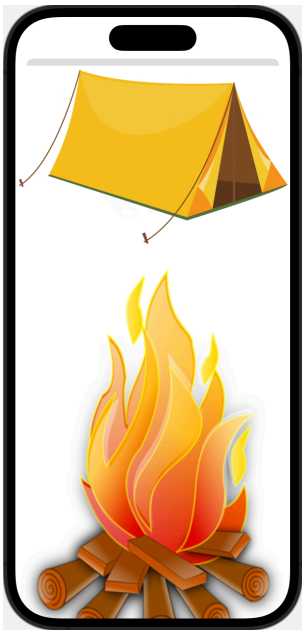
```
@State private var stickerNames: [String] = []
```

➤ Change `body` to:

```
var body: some View {
  ScrollView {
    ForEach(stickerNames, id: \.self) { sticker in
      Image(uiImage: image(from: sticker))
        .resizable()
        .aspectRatio(contentMode: .fit)
    }
  }
  .onAppear {
    stickerNames = Self.loadStickers()
  }
}
```

Instead of just showing one sticker, you iterate through all the sticker names and create an `Image` from the `UIImage`.

➤ To see the print output, build and run. Choose the first card and tap **Stickers**. Watch the debug console output, and you'll see all the images are loading up front, ending with the tree and the guitar. As mentioned before, with a lot of stickers, this will block the user interface.



Stickers loaded



Debug console output

➤ To get the stickers to load lazily, in `body`, **Command-click** `ForEach` and embed it in a `VStack`.

➤ Change `VStack {` to:

```
LazyVStack {
```

➤ Build and run, and display the stickers modal screen again. Now, only the images that show on screen, plus the one just after, load. Scroll down, and you'll see in the debug console that the guitar image loads as you approach it. Your images are now loading lazily.

> **Note**: At the time of writing, there appears to be a SwiftUI bug that duplicates the calling of `image(from:)`, causing the name of the image to print out twice in the debug console.

These images are much too big and would look much better in a grid. Fortunately, as well as lazy `VStack` and `HStacks`, SwiftUI provides a lazy loading grid view.

Using Lazy Grid Views

> **Skills you'll learn in this section**: grids

`LazyVGrid` and `LazyHGrid` provide vertical and horizontal grids. With the `LazyVGrid`, you define how to layout columns and, with the `LazyHGrid`, you layout rows.

➤ Add a new property to `StickerModal`:

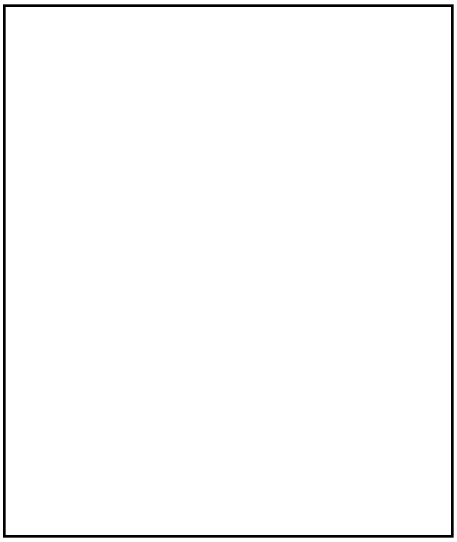
```
let columns = [
  GridItem(spacing: 0),
  GridItem(spacing: 0),
  GridItem(spacing: 0)
]
```

➤ Change `LazyVStack {` to:

```
LazyVGrid(columns: columns) {
```

You still use the same `ForEach` and `Image` views but they now fit into the available space in the grid instead of taking up the whole width of the screen. The grid uses all the horizontal available space and divides it equally among the specified `GridItems`.

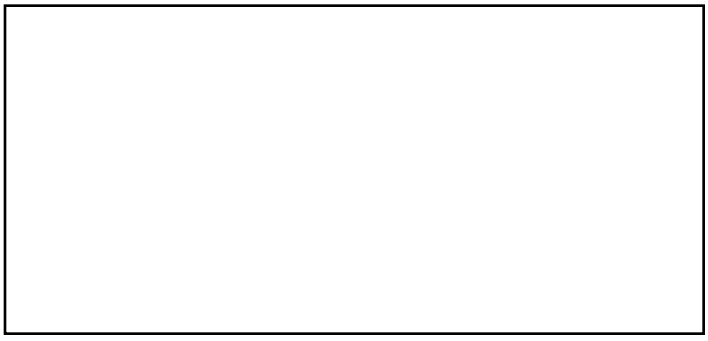
➤ To visualize this, in the design canvas panel, view **Sticker Modal** and click the **Selectable** icon. In the code panel, place the cursor on `Image` in `body`. The outlines of the `Images` will show in the preview.



Vertical grid

Swift Tip: If this were a `LazyHGrid`, you would define `rows` in the same way as you have `columns`, and the grid would divide up the available *vertical* space. To scroll horizontally, add a horizontal axis: `ScrollView(.horizontal)`.

➤ In the design canvas, click the **Variants** icon, and choose **Orientation Variants**.



Orientation variants

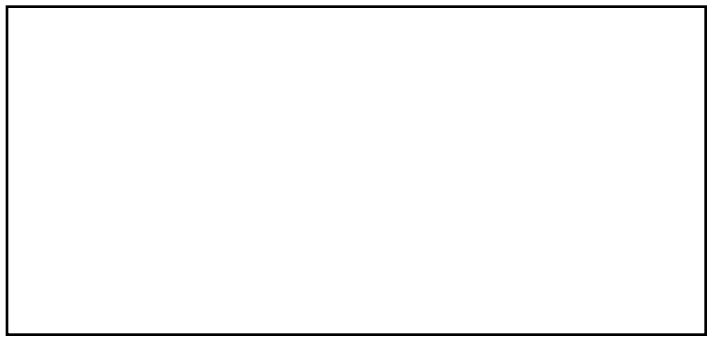
Although the grid looks good in portrait mode, it would look better with more images horizontally when in landscape.

➤ Change the `columns` property declaration to:

```
let columns = [
  GridItem(.adaptive(minimum: 120), spacing: 10)
]
```

The columns will size to `120` points, separated by ten points.

The design canvas updates the views:



Adaptive grid

> **Swift Tip**: As well as `adaptive`, `GridItem.size` can be `fixed` with a fixed size, or `flexible`, which sizes to the available space.

Selecting the Sticker

Now that you have the stickers showing, you'll tap one to select it, dismiss the modal and add the sticker to the card as a card element.

➤ In `StickerModal`, add a property to hold the selected image:

```
@Binding var stickerImage: UIImage?
```

The parent of the modal will pass in a state property to hold the selected image.

➤ Add the environment property that holds the dismiss action:

```
@Environment(\.dismiss) var dismiss
```

You'll use this property to dismiss the modal.

➤ In `body`, add a modifier to `Image`:

```
.onTapGesture {
  stickerImage = image(from: sticker)
  dismiss()
}
```

When the user taps an image, you'll update the bound sticker image and dismiss the modal.

➤ Update the preview:

```
StickerModal(stickerImage: .constant(UIImage()))
```

You can now select a sticker and at the same time dismiss the modal. `CardDetailView` will then take over and store and show the selected sticker.

➤ Open **CardToolbar.swift** and add new properties to `CardToolbar`:

```
@Binding var card: Card
@State private var stickerImage: UIImage?
```

You receive the current card from the parent view and hold the current sticker chosen from `StickerModal`.

➤ Locate the `sheet(item:)` with the `stickerModal` case. This will have a compile error as you're not yet passing the state property to `StickerModal`.

➤ Change `StickerModal()` to:

```
StickerModal(stickerImage: $stickerImage)
  .onDisappear {
    if let stickerImage = stickerImage {
      card.addElement(uiImage: stickerImage)
    }
    stickerImage = nil
  }
```

On dismissal of the modal, you should store the sticker as a card element and reset the sticker image to `nil`. You'll get a compile error until you've added the `card` binding and written `addElement(uiImage:)`.

➤ Open **SingleCardView.swift** and, in `body`, change the modal toolbar modifier to:

```
.modifier(CardToolbar(
  currentModal: $currentModal,
  card: $card))
```

Ensure that your `CardToolbar` parameters are in the same order that you listed the bindings in **CardToolbar.swift**, otherwise you will get a compile error.

Storing the Data

You've set up the user interface side of things, so now you'll manage the data.

➤ In the **Model** group, open **CardElement.swift**.

Now that you're adding image data, instead of storing an `Image`, which is a `View`, you'll store the data and construct a view from that data.

➤ Add a new property to `ImageElement`:

```
var uiImage: UIImage?
```

This will hold the sticker image data.

Replace the `image` definition with:

```
var image: Image {
  Image(
    uiImage: uiImage ??
      UIImage(named: "error-image") ??
      UIImage())
}
```

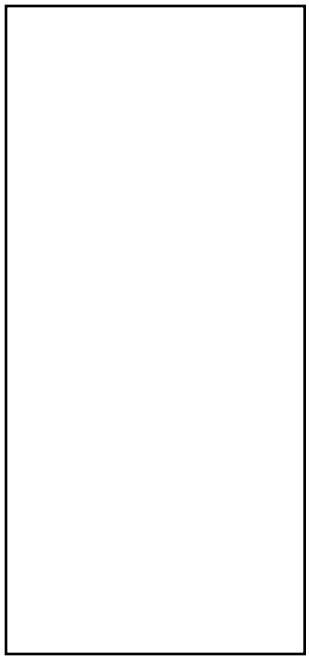
You create a computed property that builds an `Image` from `uiImage`. If this is `nil`, present the error image in the asset catalog. If, for some reason this doesn't exist, create a blank `UIImage`.

➤ Open **Card.swift** and add this new method to `Card`:

```
mutating func addElement(uiImage: UIImage) {
  let element = ImageElement(uiImage: uiImage)
  elements.append(element)
}
```

Here you take in a new `UIImage` and add a new `ImageElement` to the card. In the following chapter, you'll be able to use this method for adding photos too.

➤ Build and run, select a card and add some stickers to it. Resize and reposition the stickers as you want and create a masterpiece :].



Make a fun picture!

Challenges

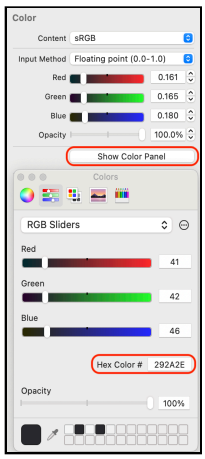
Challenge 1: Set Up a Dark Mode Launch Screen

Your app currently has different launch screens for portrait and landscape, when the height size class is compact. Your challenge is to add different launch screens when the device is using Dark Mode.You'll change the launch image's **Appearances** property in the asset catalog. You'll find the dark launch screen images in the **assets** folder. Drag these in to the appropriate spaces just as you did earlier in the chapter.

When you test on the simulator, to get the new launch screen to show, you'll need to erase all device contents and settings.

Challenge 2: Set Up Launch Colors

This chapter did not cover colors specifically, but you can change appearance and device in the same way as with images. You've already set up a launch color in **Info.plist** to use as the launch background color. Change the launch color in the asset catalog. Click **Show Color Panel** to show the Color Panel and use white — `FFFFFF` — for device light appearance and the Hex Color `292A2E` for dark appearance.



The Color Panel

If you get stuck, the asset catalog in the project in the challenge project will show you what to do.

Key Points

- Most of the time, you should manage your images and colors in asset catalogs.
- If the asset catalog is not suitable for purpose, then use reference folders.
- In asset catalogs, favor vector images over bitmaps. They are smaller in file size and retain sharpness when scaled. Xcode will automatically scale to the appropriate dimensions for the current device.
- Think about how you can make your app special. Good app design together with artwork can really make you stand out from the crowd.

Where to Go From Here?

In this chapter you used app icons and launch screens. The Apple Human Interface Guidelines (https://apple.co/3qUopKd), often referred to as the HIG, will point you at best use.

For example, rather than using branding on the launch screen, they suggest making your launch screen similar to the first screen in your app, so that it appears that the app loads quickly.

You should study the HIG so that you know what Apple is looking for in an app. People who use Apple devices enjoy clean, crisp interfaces, and the guidelines will help you follow this quest. If you follow the guidelines diligently, you might even be featured by Apple in the app store.

Chapter 17: Adding Photos to Your App

By Caroline Begbie

In the previous chapter, you learned how to add stickers to your card. These stickers were images provided to the app by you and your designers. Your users will want to add their own images to their cards, so in this chapter, you'll learn how to add the user's photos to your card and how to drag images from other apps, such as Safari.

The PhotosUI Framework

With the stickers, you load the sticker images lazily, and when the user selects one, you use that one image. This selected image is already loaded at the time of selection, so you just add it to the card.

Loading photos is not as simple as loading stickers, because the user's media library might number in the tens of thousand of assets. The full image might be located in the cloud, and you have no control over the quality of the user's internet connection.

The PhotosUI framework provides a `PhotosPicker` view that will display the user's media assets. The user then selects photos, and each selected item goes into an array. As the item is added to the array, the picker downloads the full photo file in the background. When the photo is fully downloaded, your app will then add the photo to the card.

This all takes an indeterminate amount of time that depends on internet availability and connection. Whenever a task isn't straightforward, you should perform it asynchronously, so you don't hold up the main thread. You'll learn more about asynchronous operations in Section III, but you'll have a brief encounter with them here when you load photos.

The PhotosPicker View

> **Skills you'll learn in this section**: `PhotosPicker`

Instead of using your own modal view, you'll use PhotosUI's `PhotosPicker`.

➤ Open the starter project for this chapter, which is the same as the previous chapter's challenge project.

➤ In the **Card Modal Views** group, create a new SwiftUI View file called **PhotosModal.swift** and import the framework:

```
import PhotosUI
```

➤ Replace `PhotosModal` with:

```
struct PhotosModal: View {
  @Binding var card: Card
  // 1
  @State private var selectedItems: [PhotosPickerItem] = []

  var body: some View {
    // 2
    PhotosPicker(
      // 3
      selection: $selectedItems,
      // 4
      matching: .images) {
      // 5
        ToolbarButton(modal: .photoModal)
    }
  }
}
```

Going through the code:

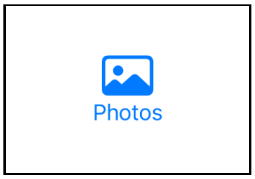
1. Create an array to hold the selected images. The type `PhotosPickerItem` doesn't contain the actual image data. Instead, it contains only an identifier and the type of content, such as `jpeg`, that the item supports.
2. Display the photos picker view.
3. As the user taps and selects media assets, the photos picker adds them to `selectedItems`.
4. You can filter the photo library in various ways, such as screenshots or videos. For **Cards**, you filter images. You can see the other available filters here (https://apple.co/3flsc0C).
5. `PhotosPicker` requires a label to start it, so you include the image and text you already set up in `ToolbarButton`.

> **Note**: Be careful with your file names. If you create a structure called `PhotosPicker`, that will override the one used by PhotosUI without any warning. You can still use `PhotosUI.PhotosPicker`, but you have to specifically reference `PhotosUI` when you do.

➤ In `PhotosModal_Previews`, change `PhotosModal()` to:

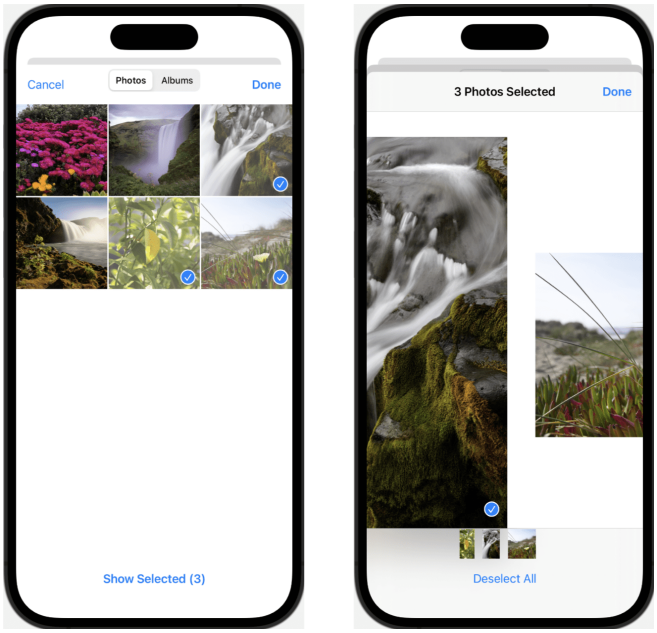
```
PhotosModal(card: .constant(Card()))
```

In Live Preview, you'll see the button with the label you provided:



PhotosPicker view button

➤ Tap the button to see how the system photos picker works. You can select multiple photos and also select from your photo albums. You can also show larger versions of all the images you've selected.



The system photos picker

Adding the Photos Picker to Your App

➤ Open **CardToolbar.swift**, and locate `.sheet(item: $currentModal)`.

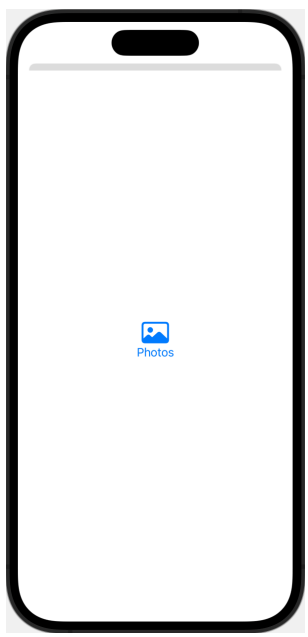
This is where you display the modal views when the user taps a button on the bottom toolbar.

➤ Add a new case to `switch item`:

```
case .photoModal:
  PhotosModal(card: $card)
```

Here you set up the photo button on the toolbar to display your photos modal view.

➤ Open **SingleCardView.swift** and pin the preview. In Live Preview, tap the Photos button on the bottom toolbar.



Two Photos buttons

You may have anticipated this. Your `PhotosModal` view pops up from your button. This view contains the system `PhotosPicker` view which you defined with its own label. You obviously don't want to compel your user to press two buttons.

➤ Open **BottomToolbar.swift** and, in `BottomToolbar`, locate `Button {...}` inside `ForEach...`. Command-Click `Button`, and select **Embed...**.

➤ Change `Container {` to:

```
switch selection {
default:
```

➤ Add a new case to `switch selection` before the `default` case:

```
case .photoModal:
  Button {
  } label: {
    PhotosModal(card: $card)
  }
```

The resulting view from `PhotosModal` is the button you supply to `PhotosPicker`, so this replaces the previous `ToolbarButton`.

➤ `BottomToolbar` needs to contain the binding, so add the new property to `BottomToolbar`:

```
@Binding var card: Card
```

➤ Replace `BottomToolbar` in the preview with:

```
BottomToolbar(
  card: .constant(Card()),
  modal: .constant(.stickerModal))
```

Remember to add the parameters in the order they appear in `BottomToolbar`.

➤ Open **CardToolbar.swift** and remove:

```
case .photoModal:
  PhotosModal(card: $card)
```

`BottomToolbar` loads the photos view now, so this is no longer needed.

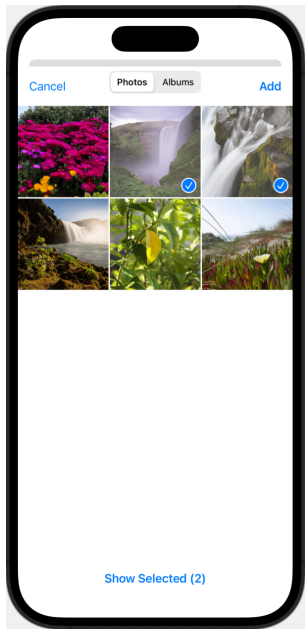
➤ Inside `ToolbarItem(placement: .bottomBar) {`, locate `BottomToolbar(modal: $currentModal)`. Replace it with:

```
BottomToolbar(
  card: $card,
  modal: $currentModal)
```

You now pass the binding and your app compiles.

➤ Resume Live Preview on **Single Card View**, and select the Photos button on the bottom toolbar.

This time you see the system photos picker. When you tap **Cancel**, the Photos modal disappears. So far, when you select photos and tap **Add**, nothing happens. The system retains the selection, however, as you'll see if you return to the photos picker.



The system photos picker

The Transferable Protocol

Skills you'll learn in this section: `Transferable`; Uniform Type Identifiers; add photos to Simulator

It's not only photos that you might want to add to your app. You might want to be able to copy and paste text, or even custom types, such as files created by another app. You'll also want to share your card with your friends, which means exporting your card from your app. `Transferable` is a flexible protocol that allows you to describe how to import and export any types.

Some existing data types, such as `Data`, which is a string of bytes, already conform to `Transferable`. When you add photos, these will be of type `UIImage`, which unfortunately does not conform.

It's easy to add conformance, and you'll do that later in the chapter. For the moment, though, to get you quickly adding photos, you'll transfer the photos as `Data`.

➤ Open **PhotosModal.swift** and add this modifier to `PhotosPicker`:

```
  .onChange(of: selectedItems) { items in
    for item in items {
      print(item)
    }
    selectedItems = []
  }
```

Whenever `selectedItems` changes, you'll print out each element in the array. After you've processed each item, clear the array.

➤ Build and run the app in Simulator, choose a card, tap the **Photos** button on the bottom toolbar and select the pink flowers photo and one other. Tap **Add**.

The details of each item selected print out in the debug console. Notice the `_supportedContentTypes`. These are the supported content types for the item. The pink flowers photo has two types: `public.jpeg` and `public.heic`. The other photo has just one type: `public.jpeg`.

```
PhotosPickerItem(_itemIdentifier:
  Optional("CC95F08C-88C3-4012-9D6D-64A413D254B3/L0/001"), _shouldExposeItemIdentifier:
  false, _supportedContentTypes: [<_UTCoreType 0x10a900af0> public.jpeg (not dynamic,
  declared), <_UTCoreType 0x10a900c50> public.heic (not dynamic, declared)],
  _itemProvider: <PUPhotosFileProviderItemProvider: 0x6000002afa80> {types = (
    "public.jpeg",
    "public.heic"
)})
PhotosPickerItem(_itemIdentifier:
  Optional("ED7AC36B-A150-4C38-BB8C-B6D696F4F2ED/L0/001"), _shouldExposeItemIdentifier:
  false, _supportedContentTypes: [<_UTCoreType 0x10a900af0> public.jpeg (not dynamic,
  declared)], _itemProvider: <PUPhotosFileProviderItemProvider: 0x6000002af180> {types =
  (
    "public.jpeg"
)})
```

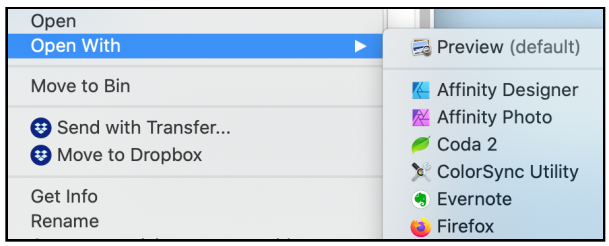
Console output

Uniform Type Identifiers

Uniform Type Identifiers, or UTIs, identify file types. For example, JPG is a standard UTI, with the identifier `public.jpeg`. It's a subtype of the base image data type `public.image`.

`public.text` encompasses all text data, including `public.plainText` and `public.rtf`.

Most apps have associated data types. For example, when you right-click a macOS file and choose **Open With**, the menu presents you with all the apps associated with that file's data format. When you right-click a .png file, you might see a list like this:



.png app list

These are the apps that are able to open .png files.

There are many standard system UTIs which you can find at https://apple.co/3xASdxD.

If you have a custom data format, you can create your own type in a `UTType` extension:

```
extension UTType {
  static var myType: UTType =
    { UTType(exportedAs: "com.kodeco.myType") }
}
```

`public.data` is a base type representing a stream of bytes. Using this type, you can load the photos as a data stream and then convert the data to a `UIImage`.

Adding Photos to Your App

➤ Still in **PhotosModal.swift**, in the `for` loop, replace `print(item)` with:

```
item.loadTransferable(type: Data.self) { result in
  Task {
  // create a UIImage
  }
}
```

You load the item as a `Data` type. `result` is of type `Result<Success, Failure>`. `Success` contains the image data, and `Failure` contains a failure value.

For each item, you load the image on a background thread using `Task {}`.

➤ Replace `// create a UIImage` with:

```
switch result {
case .success(let data):
  if let data,
    let uiImage = UIImage(data: data) {
    card.addElement(uiImage: uiImage)
  }
case .failure(let failure):
  fatalError("Image transfer failed: \(failure)")
}
```

If the result succeeds, use the data to create a `UIImage` and add that image to the card's element array. If the result fails, produce a fatal error.

> **Note**: At the time of writing, the simulator pink flowers photo in `HEIC` format causes an error. This appears to be an Apple bug, but it does give you the chance to make sure that your `PhotosPicker` error checking works. When you run your app in Simulator and choose that photo, in the console, you should see *Fatal error: Image transfer failed:* with information about the failure. `HEIC` format files will work on a device with your own photos.

➤ Live Preview **Single Card View** and add some photos (not the pink flowers) to the card.



Photos added to the card

Adding Photos to Simulator

If you want more photos than the ones Apple supplies, you can simply drag and drop your photos from Finder into Simulator. Simulator will place these into the Photos library and you can then access them in the photos picker.

Drag and Drop From Other Apps

Skills you'll learn in this section: Split view; drag and drop; data representation

The photos library is not the only place you can access photos. Modern apps should accept photos and images that you drag from any other app.

First set up Simulator so that you'll be able to do the drag and drop.

➤ Build and run your app on an iPad simulator and turn the iPad to landscape mode. You can use the icon on the top bar, or use **Command-Right Arrow**.

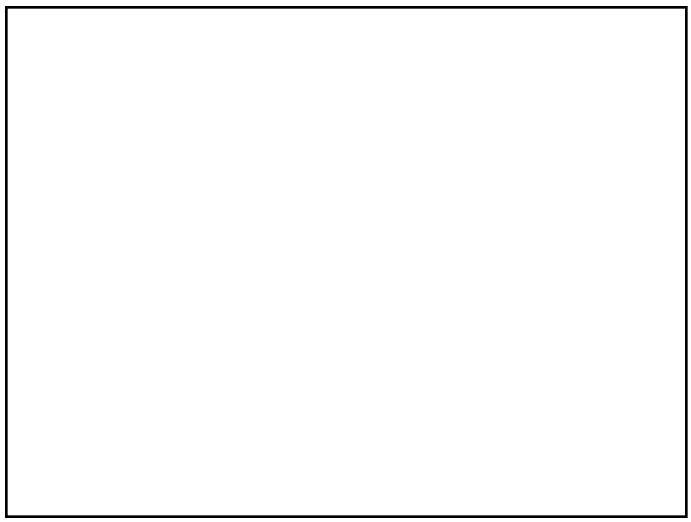
➤ Tap the three dots at the top of Simulator's screen and choose **Split View**.



Split View

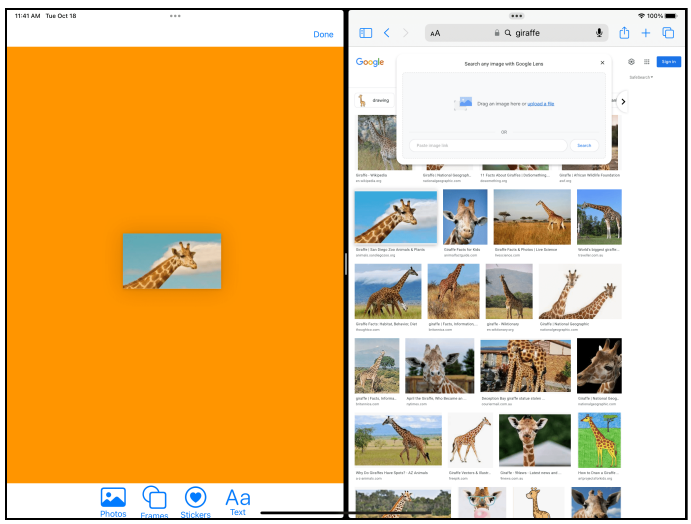
➤ Locate the Safari icon and tap it.

Safari will load using half of the iPad screen.



Cards and Safari in Split View

➤ In the Cards app, tap a card. In Safari, Google your favorite animal and tap Images. Long press an image until it gets slightly larger and drag it onto your card.



Drag a giraffe

Cards is not ready to receive a drop yet, so nothing happens. If the drop area were able to receive an item, you would get a plus sign next to the image.

Adding the Dropped Item to Your App

➤ Open **CardDetailView.swift** and, in `body`, add this modifier to `ZStack`:

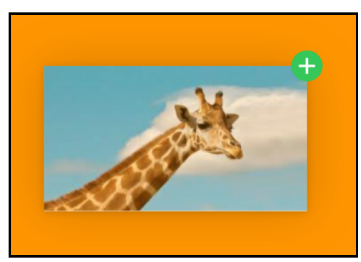
```
.dropDestination(for: Data.self) { receivedData, location in
  print(location)
  for data in receivedData {
    if let image = UIImage(data: data) {
      card.addElement(uiImage: image)
    }
  }
  return !receivedData.isEmpty
}
```

Just as you did with your photos, you receive the dragged image or images as an array of data streams. You create a `UIImage` from the data and add the image to the card's array of elements. You return whether the operation was successful.

Currently you don't use `location`, so any dropped items are added to the center of the card. To calculate the `offset` for the element's `transform`, you'll need to convert the location point on the card to an offset from the center of the card. This involves knowing the screen size of the card. You'll revisit this problem in Chapter 20, "Delightful UX — Layout".

➤ Build and run again, and drag in photos from Safari.

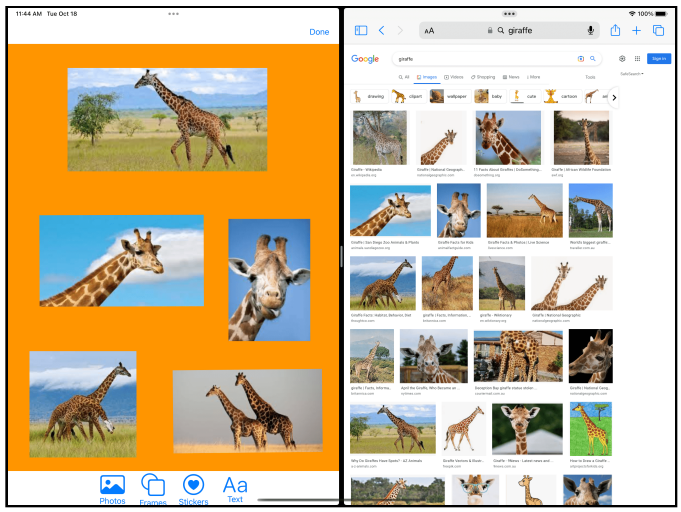
As you drag over the drop area, a plus sign will appear on the drop pile, indicating that the drop destination is allowable for this data type. When you drop the photo, it's added to the card at the center.



Drop is active

In Simulator, to select multiple images in Safari at the same time, pick up an image and start dragging it. That small drag is important — you won't be able to multiple select without it. Then hold down **Control**. Release the click and then Control. A gray dot appears on the image representing your finger on a device. Click other images to add them to the drag pile.

When you've collected all the images, drag them to Cards.



A tower of giraffes

Conforming Types to Transferable

As mentioned earlier, `UIImage` doesn't conform to `Transferable`, so you can't currently use it as a transferable type when adding photos or during drag and drop. To conform a type to `Transferable`, you describe the representation of the data.

➤ Create a new Swift file in the **Extensions** group, called **UIImageExtensions.swift** and replace the code with this:

```
import SwiftUI
// 1
extension UIImage: Transferable {
  // 2
  public static var transferRepresentation: some
TransferRepresentation {
    // 3
    DataRepresentation(importedContentType: .image) { image in
      // 4
      UIImage(data: image) ?? errorImage
    }
  }

  public static var errorImage: UIImage {
    UIImage(named: "error-image") ?? UIImage()
  }
}
```

This code needs some explanation:

1. Add a new extension to `UIImage` to conform to `Transferable`.
2. `transferRepresentation` is a required property. `TransferRepresentation` describes how to transfer an item. You can describe how to import and export the item.
3. When the imported `UTType` is an image, you'll import the image as data and construct a `UIImage` from that data. `DataRepresentation` expects the return type to be the same type as `Self`, in this case, a `UIImage`.
4. Create the `UIImage`. If the operation fails, create an error image using the image in the asset catalog.

If you want to transfer a custom object that conforms to `Codable`, instead of `DataRepresentation`, you can use `CodableRepresentation` in the same way.

Updating the Drag and Drop

You can now import dropped photos using `UIImage` instead of `Data`. This makes your code correspond more closely to your intent. When you see the word "data", it's not always obvious what type that data is.

➤ Open **CardDetailView.swift** and replace the `dropDestination` modifier with:

```
.dropDestination(for: UIImage.self) { images, location in
  print(location)
  for image in images {
    card.addElement(uiImage: image)
  }
  return !images.isEmpty
}
```

You can now use `UIImage.self` as a `Transferable` type. The data representation in the `UIImage` extension reads in the data and converts it to a `UIImage`, which you can add to the the card directly.

Drag and Drop of Custom Types

You can now drag in images from another app, but you also want to drag in text. You can try this naively and see how it will work.

➤ First, open **Card.swift** in the **Model** group and add a new method to `Card` that adds text to the card elements.

```
mutating func addElement(text: TextElement) {
  elements.append(text)
}
```

You receive in a `TextElement` and add it to the `elements` array.

➤ Open **CardDetailView.swift** and locate the `UIImage` `dropDestination` modifier.

Unfortunately, having two drop destination modifiers doesn't work.

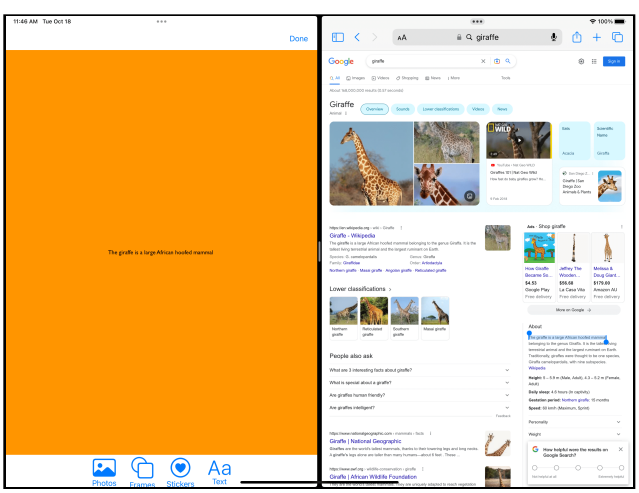
➤ Replace the `UIImage` drop destination modifier with this code:

```
.dropDestination(for: String.self) { strings, _ in
  for text in strings {
    card.addElement(text: TextElement(text: text))
  }
  return !strings.isEmpty
}
```

You add a new drop destination modifier that will take in a `String` and add a text element to the card.

➤ Build and run the app in an iPad simulator with Safari open in Split View. In Safari, highlight a small amount of text and drag it on to your card.

Cards will add the text element to the center of the card.



Dropped text

So far, so good. You can drag an image to your card, and you can drag some text to your card. The only problem is that you have to change the code each time.

You can overcome this problem with a custom transfer type that conforms to `Transferable`.

➤ In the **Model** group, create a new Swift file named **CustomTransfer.swift** and replace the code with:

```
import SwiftUI

struct CustomTransfer: Transferable {
  var image: UIImage?
  var text: String?

  public static var transferRepresentation: some
TransferRepresentation {
    DataRepresentation(importedContentType: .image) { data in
      let image = UIImage(data: data)
        ?? UIImage(named: "error-image")
      return CustomTransfer(image: image)
    }
    DataRepresentation(importedContentType: .text) { data in
      let text = String(decoding: data, as: UTF8.self)
      return CustomTransfer(text: text)
    }
  }
}
```

`CustomTransfer` contains two properties, one for text and one for image. The transfer representation takes into account the image type and the text type and fills in the relevant property. For images, the data representation is the same as you did for `UIImage`, and for text, you create a `String` from the data. `UTF8` is the most common Unicode encoding system.

Once `CustomTransfer` has created either an image or some text from the transferred data, you'll add an element to the card.

➤ Open **Card.swift** and add this new method:

```
mutating func addElements(from transfer: [CustomTransfer]) {
  for element in transfer {
    if let text = element.text {
      addElement(text: TextElement(text: text))
    } else if let image = element.image {
      addElement(uiImage: image)
    }
  }
}
```

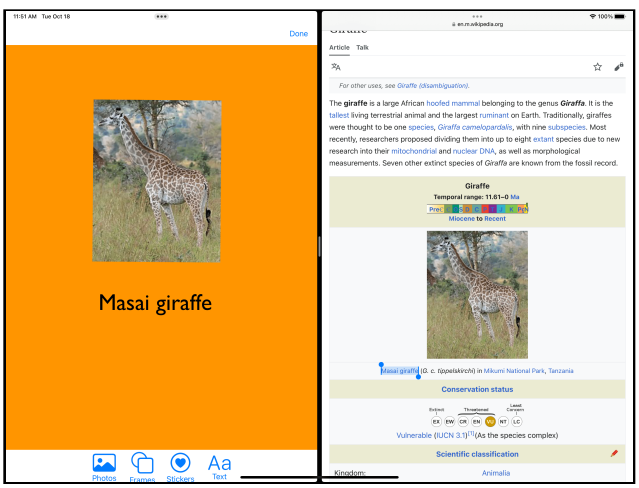
Using your custom `Transferable` structure, you can add both text and image elements to the card appropriately.

➤ Open **CardDetailView.swift**, and in `body`, replace the drop destination modifier with:

```
.dropDestination(for: CustomTransfer.self) { items, location in
  print(location)
  Task {
    card.addElements(from: items)
  }
  return !items.isEmpty
}
```

➤ Build and run the app in Simulator.

When you drop the transferred items, the view will print the location to the debug console for later use. A new task will start that adds the elements to the card.



Drag and drop text and image

Pasting From Another App

> **Skills you'll learn in this section**: Cut and paste

Once you've set up your `CustomTransfer`, as well as dragging photos from another app, you can instead copy them and paste them on your card.

SwiftUI provides `PasteButton` for this. It doesn't allow a lot of customization, and you can't add it to a `Menu`, but it is easy to implement.

➤ Open **CardToolbar.swift**.

This is where you place your toolbar items.

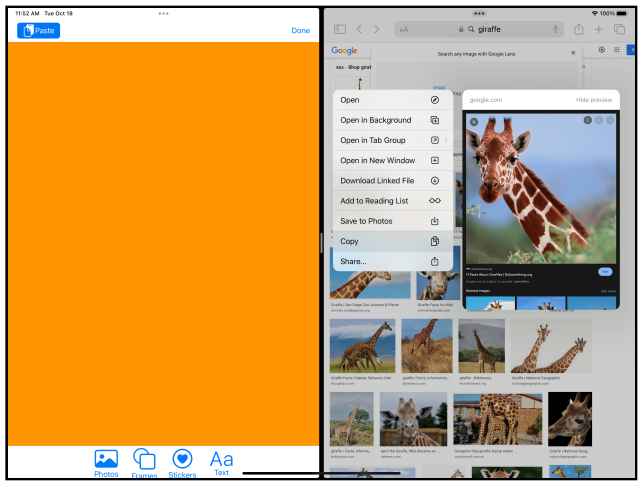
➤ Add a new item inside `toolbar(content:)`:

```
ToolbarItem(placement: .navigationBarLeading) {
  PasteButton(payloadType: CustomTransfer.self) { items in
    Task {
      card.addElements(from: items)
    }
  }
}
```

You've now implemented paste in your app. I told you it was easy! `PasteButton` will be disabled unless it detects a `CustomTransfer` item. Then when you tap `Paste`, the items will be added to your card in the same way as the drop.

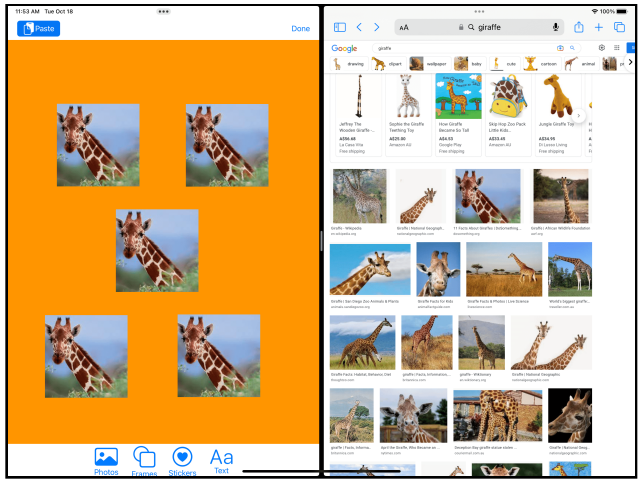
➤ Build and run the app on an iPad simulator with Safari in split screen and choose a card. Long press an image in Safari, and choose **Copy**.

The image is now in the pasteboard (also known as clipboard) ready to paste. Copy-and-paste will also work with text.



Copy an image

➤ Tap **Paste** a few times to add copies of the image to your card.

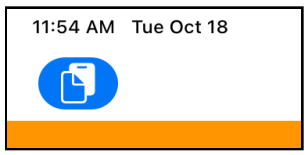


Several paste operations

➤ Add these modifiers to `PasteButton(payloadType:)`:

```
.labelStyle(.iconOnly)
.buttonBorderShape(.capsule)
```

This removes the word "Paste" leaving only the icon and gives the button a capsule shape. The paste button is now a little less obtrusive, but the design still doesn't fit well.



Styled paste button

Adding a Pop-up Menu

Skills you'll learn in this section: Pop-up menu; context menu; `UIPasteBoard`; remove from array

As you build up your app, you'll probably want to add a few extra buttons for operations that don't really need to be always on screen. You can add a pop-up menu for all these operations. Unfortunately, `PasteButton` won't work on this menu, so you'll use a `Button` which updates UIKit's `UIPasteboard`.

➤ Replace the `PasteButton` `ToolbarItem` with:

```
ToolbarItem(placement: .navigationBarTrailing) {
  menu
}
```

➤ This toolbar item will be more complicated than the previous one, so add a new property to `CardToolbar`:

```
var menu: some View {
  // 1
  Menu {
    Button {
      // add action here
    } label: {
      Label("Paste", systemImage: "doc.on.clipboard")
    }
    // 2
    .disabled(!UIPasteboard.general.hasImages
      && !UIPasteboard.general.hasStrings)
  } label: {
    Label("Add", systemImage: "ellipsis.circle")
  }
}
```

There are a couple of things to note here:

1. You add a `Menu` to the top toolbar just to the left of the **Done** button. A `Menu` is a list of buttons. For this app, you'll only have one button, but you can very easily add more under the **Paste** button.
2. You only want the paste button to be enabled when there is something to paste, so you check `hasImages` and `hasStrings`. If both are false, you disable the button.

➤ Build and run the app and tap the ellipsis.



Ellipsis pop-up menu

Your paste button shows up on the menu.

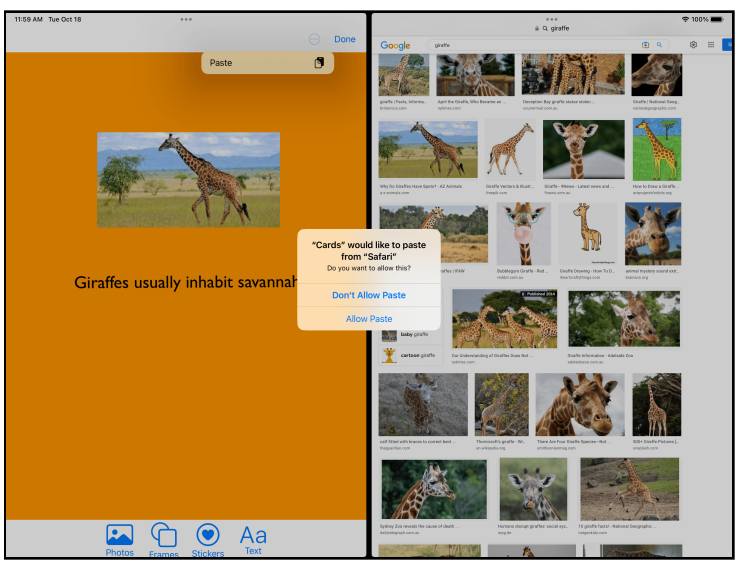
➤ Back in **CardToolbar.swift**, in `menu`, replace `// add action here` with:

```
if UIPasteboard.general.hasImages {
  if let images = UIPasteboard.general.images {
    for image in images {
      card.addElement(uiImage: image)
    }
  }
} else if UIPasteboard.general.hasStrings {
  if let strings = UIPasteboard.general.strings {
    for text in strings {
      card.addElement(text: TextElement(text: text))
    }
  }
}
```

You can check whether the pasteboard contains images or strings. Apple's documentation states not to test `images` or `strings` to see whether they contain data, but to check `hasImages` and `hasStrings`.

➤ Build and run your app on iPad with Safari in split screen. Then, try copying and pasting text and images.

When pasting from another app, iOS will ask permission whether to paste.



Allow paste

> **Note**: Apple's Universal Clipboard is very powerful. For example, if you run **Cards** on a device, you can select and copy photos in the macOS **Photos** app and paste them into **Cards** on the device.

Copying Elements

You can copy from other apps, so it makes sense to implement copying elements within your own app.

You do this with `contextMenu(menuItems:)` modifiers on card elements. You activate the context menu with a long press, just as you did when you copied from Safari. When you choose **Copy** from the context menu, the system will add the element — text or image — to the pasteboard. You can then paste the text or image in your app, or even in another app.

➤ Open **CardDetailView.swift**.

If you add a context menu with several buttons to `CardElementView(element:)`, the view will get over-complicated. Instead of adding the context menu here, you'll create it in a new view modifier file.

➤ In the **SingleCardViews** group, create a new Swift file called **ElementContextMenu.swift** and replace the code with:

```
import SwiftUI

struct ElementContextMenu: ViewModifier {
  @Binding var card: Card
  @Binding var element: CardElement

  func body(content: Content) -> some View {
    content
  }
}
```

The context menu will need access to the current card and current element. Creating a view modifier should be familiar to you from Chapter 14, "Gestures", when you created `resizableView()`.

➤ Add a new modifier to `content`:

```
.contextMenu {
  Button {
    if let element = element as? TextElement {
      UIPasteboard.general.string = element.text
    } else if let element = element as? ImageElement,
      let image = element.uiImage {
        UIPasteboard.general.image = image
    }
  } label: {
    Label("Copy", systemImage: "doc.on.doc")
  }
}
```

The context menu will pop up when you perform a long press on a card element. When you tap **Copy**, the pasteboard will record the text or image element details ready for pasting elsewhere.

Your modifier is ready to use, but, as you did with `ResizableView`, you should make it easier to use.

➤ Add this to the end of **ElementContextMenu.swift**.

```
extension View {
  func elementContextMenu(
    card: Binding<Card>,
    element: Binding<CardElement>
  ) -> some View {
    modifier(ElementContextMenu(
      card: card,
      element: element))
  }
}
```

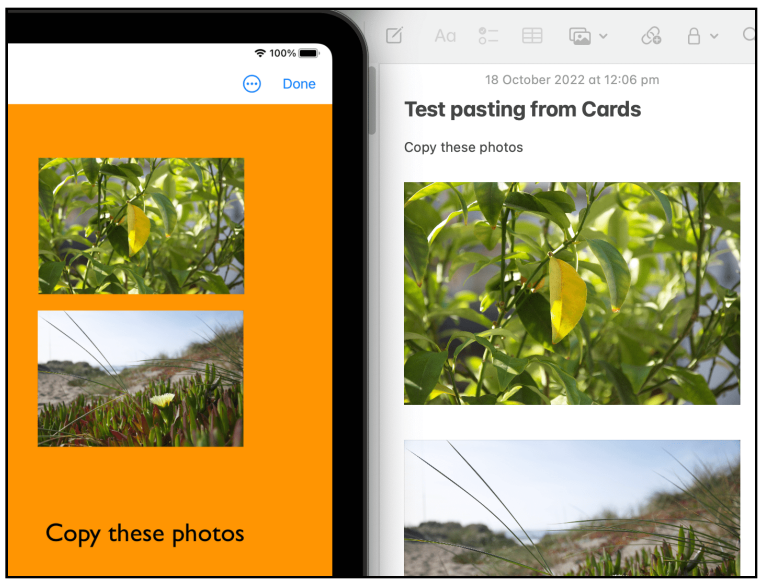
This extension to `View` simply calls your new modifier with a card and an element value.

➤ Open **CardDetailView.swift**, and, in `body`, add this code to `CardElementView(element: element)` as the first modifier:

```
.elementContextMenu(
  card: $card,
  element: $element)
```

You have now added a new context menu to each element that you can access with a long press on that element. You must place the modifier before the following ones so that the context menu appears in the correct place on the screen.

➤ Build and run the app and experiment with copying elements and pasting them in other cards, or even in other apps. Even when copying the element in Simulator, you can paste it into another macOS app.



Copy Cards elements to Notes

Deletion

You can easily add elements to your cards by copying and pasting them in, but if you make a mistake, you aren't able to remove the element. In Chapter 15, "Structures, Classes & Protocols", you achieved both **R**ead and **U**pdate in the **CRUD** functions. Next, you'll take on **D**eletion.

You'll add an entry to the context menu. When you tap the menu item, your app will remove the selected card element from the card's array.

➤ Open **Card.swift** and add this code to `Card`:

```
mutating func remove(_ element: CardElement) {
  if let index = element.index(in: elements) {
    elements.remove(at: index)
  }
}
```

Here you retrieve the index of the card element. You then remove the element from the array using the index.

➤ Open **ElementContextMenu.swift** and add a new button to the context menu:

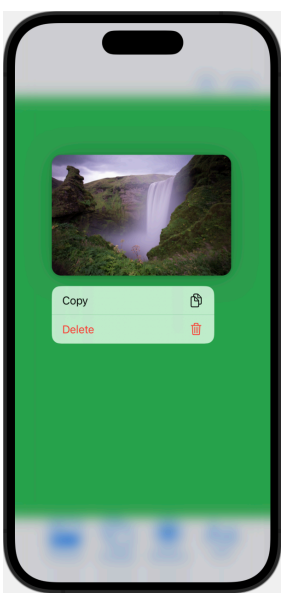
```
Button(role: .destructive) {
  card.remove(element)
} label: {
  Label("Delete", systemImage: "trash")
}
```

Your delete button should be highlighted as dangerous, and that's what the `destructive` role does for you. The menu item will be in red.

➤ Live Preview **Single Card View**, add a photo to the card, and then, long press the photo.

You'll see the context menu pop up.

➤ Tap **Delete** to delete the element, or tap away from the menu if you decide not to delete it.



Delete an element

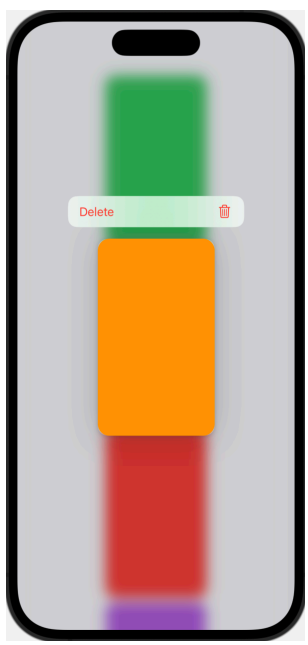
In summary, when you delete the element, you delete it from `card.elements`. `card` is bound to `cards` in the data store, and `cards` is a published property. When `cards` changes, all views containing `cards` will redisplay their content.

Challenge

Challenge: Delete a Card

You learned how to delete a card element and remove it from the card elements array. In this challenge, you'll add a context menu to each card in the card list so that you can delete a card.

1. In `CardStore`, create a similar remove method as the one in `Card` to remove a card from the cards array.
2. In `CardsListView`, add a new context menu to a card with a delete option that calls your new method to remove the card.



Delete a card

You'll find the solution to this challenge in the **challenge** folder for this chapter.

Key Points

- Instead of having to implement your own photos picker view, Apple provides the PhotosUI framework with a `PhotosPicker` view. It's an easy way to select photos and videos from the photo library.
- Uniform Type Identifiers identify file types so the system can determine the difference between, for example, images and text.
- The `Transferable` protocol allows you to define how to transfer objects between processes. You use `Transferable` for drag and drop, pasting and sharing. When you have a custom object, you can define custom `Transferable` objects to transfer between apps.
- A `Menu` is a list of `Buttons`. Each `Button` can have a `role`. By making the role `destructive`, the menu item will appear in red.
- `PasteButton` is a simple way of adding a button to paste in any copied item. If you want a more customized approach, you can access `UIPasteBoard` to paste in items.
- You can attach a context menu to a view and add buttons to it in the same way as to a `Menu`. You access the context menu by a long press. SwiftUI brings the view to the foreground and darkens the other views. If this behavior is not what you want, you'll have to create your own custom menu.

Chapter 18: Paths & Custom Shapes

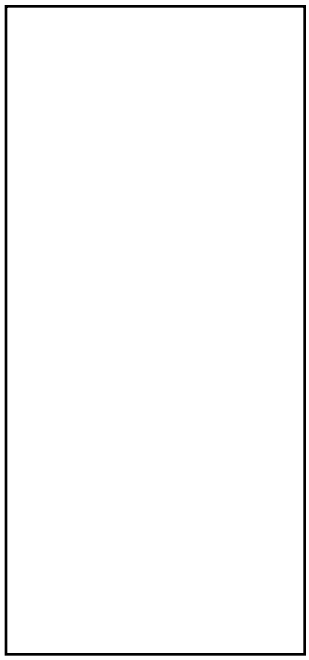
By Caroline Begbie

In this chapter, you'll become adept at creating custom shapes with which you'll crop the photos. You'll tap a photo on the card, which enables the **Frames** button. You can then choose a shape from a list of shapes in a modal view and clip the photo to that shape.

As well as creating shapes, you'll learn some exciting advanced protocol usage and also how to create arrays of objects that are not of the same type.

The Starter Project

The starter project is the same as the challenge project from the previous chapter, with the exception that, just for a change, your project now contains preview data of giraffes rather than hedgehogs.



The starter project

Shapes

Skills you'll learn in this section: predefined shapes

➤ In the **Model** group, create a new SwiftUI View file called **Shapes.swift**. This file will hold all your custom shapes.

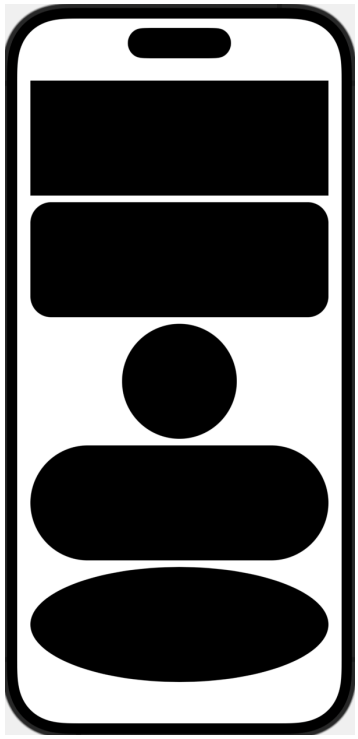
➤ Remove `Shapes`. You'll preview your shapes using `Shapes_Preview`.

➤ Replace `previews` with

```
static var previews: some View {
  VStack {
    Rectangle()
    RoundedRectangle(cornerRadius: 25.0)
    Circle()
    Capsule()
    Ellipse()
  }
  .padding()
}
```

These are the five built-in shapes, which fill as much space as they can.

➤ Live Preview the view.



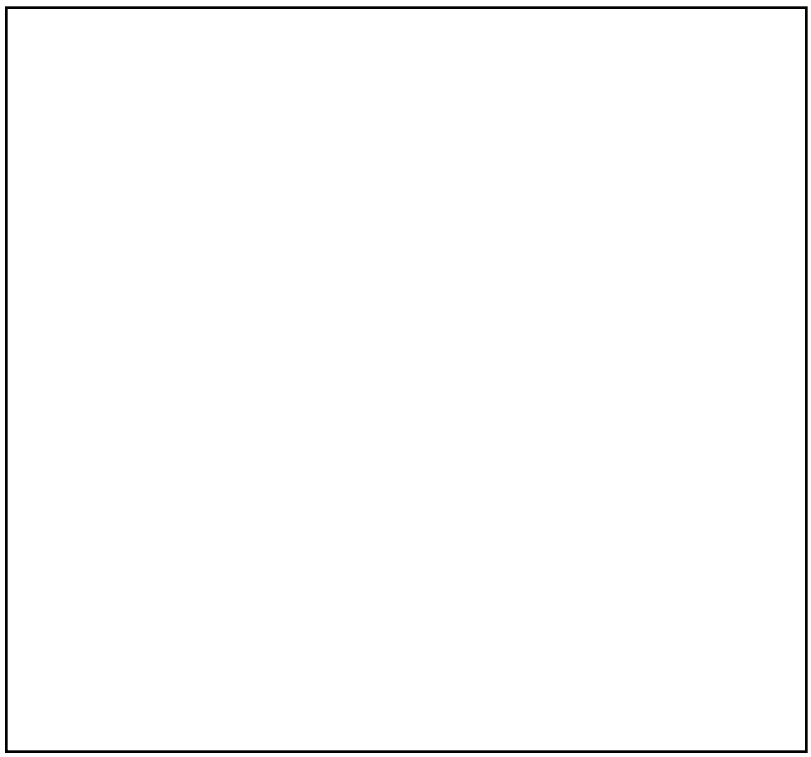
Five predefined shapes

These shapes conform to the `Shape` protocol, which inherits from `View`. Using `Shape`, you can define any shape you want using **paths**.

Paths

Skills you'll learn in this section: paths; lines; arcs; quadratic curves

This is the triangle shape you'll draw first. You'll create a **path** made up of lines that go from point to point.



Triangle

Paths are simply abstract until you give them an outline stroke or a fill. SwiftUI defaults to filling paths with the primary color, unless you specify otherwise.

➤ At the end of **Shapes.swift**, add a new shape with this code:

```
struct Triangle: Shape {
  func path(in rect: CGRect) -> Path {
    var path = Path()
    return path
  }
}
```

`Shape` has one required method which returns a `Path`. `path(in:)` receives a `CGRect` containing the drawing canvas size in which to draw the path.

Lines

➤ Create a triangle with the same coordinates as in the diagram above. Add this to `path(in:)` before `return path`:

```
// 1
path.move(to: CGPoint(x: 20, y: 30))
// 2
path.addLine(to: CGPoint(x: 130, y: 70))
path.addLine(to: CGPoint(x: 60, y: 140))
// 3
path.closeSubpath()
```

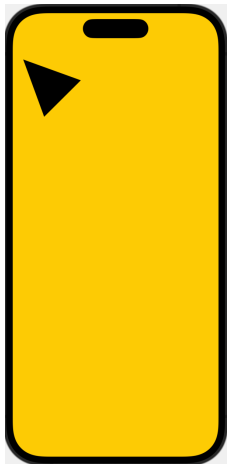
Going through the code:

1. You create a new subpath by moving to a point. Paths can contain multiple subpaths.
2. Add straight lines from the previous point. You can alternatively put the two points in an array and use `addLines(_:)`.
3. Close the subpath when you've finished to create the polygon.

➤ Change `Shapes_Previews` to:

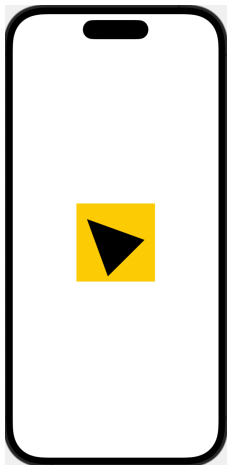
```
struct Shapes_Previews: PreviewProvider {
  static let currentShape = Triangle()

  static var previews: some View {
    currentShape
      .background(Color.yellow)
  }
}
```



Triangle Shape

Shapes fill as much space as they can. The filled path is using the fixed numbers from `path(in:)`. But `Triangle` itself is filling the whole yellow area. Your code only replicates the triangle in the previous diagram when you add `.frame(width: 150, height: 150)` to `currentShape`.



Fixed Triangle

If you want the triangle to retain its shape, but size itself to fill the available size, you must use relative coordinates, rather than absolute values.

➤ In `Triangle`, replace `path(in:)` with:

```
func path(in rect: CGRect) -> Path {
  let width = rect.width
```

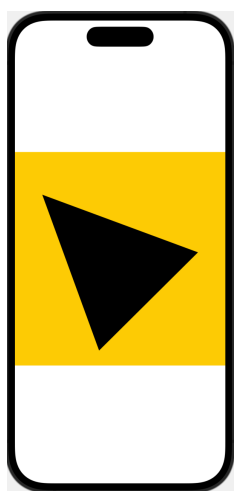
```
    let height = rect.height
    var path = Path()
    path.addLines([
      CGPoint(x: width * 0.13, y: height * 0.2),
      CGPoint(x: width * 0.87, y: height * 0.47),
      CGPoint(x: width * 0.4, y: height * 0.93)
    ])
    path.closeSubpath()
    return path
  }
```

Here you use `addLines(_:)` with an array of points to make up the triangle. You replace the hard coded coordinates with relative ones that depend upon the width and height. You can calculate these coordinates by dividing the hard coded coordinate by the original frame size. For example, `20.0 / 150.0` comes out at about `0.13`.

➤ In `Shapes_Previews`, change the contents of `previews` to:

```
  currentShape
    .aspectRatio(1, contentMode: .fit)
    .background(Color.yellow)
```

You maintain the square aspect ratio, and the triangle will now resize to the available space.



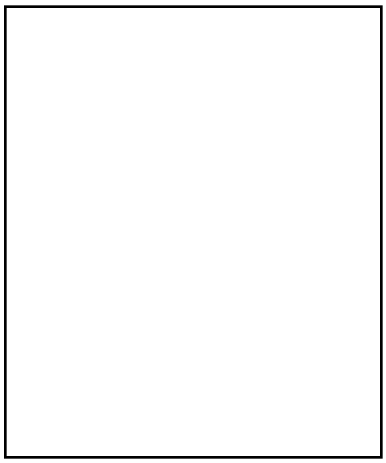
Resizable Triangle

➤ In `Shapes_Previews`, add this modifier to `currentShape`:

```
.previewLayout(.sizeThatFits)
```

➤ Switch Live Preview to **Selectable**.

Now the preview will only show the part of `Shapes` that holds the triangle.



Resized preview

Arcs

Another useful path component is an arc.

➤ At the bottom of **Shapes.swift**, add this code to create a new shape:

```
struct Cone: Shape {
  func path(in rect: CGRect) -> Path {
    var path = Path()
    // path code goes here
    return path
  }
}
```

Here you create a new shape in which you'll describe a cone. To draw the cone, you'll draw an arc and two straight lines.

➤ Add the arc to `path(in:)` before `return path`

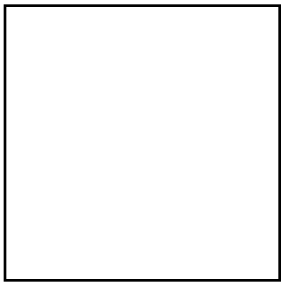
```
let radius = min(rect.midX, rect.midY)
path.addArc(
  center: CGPoint(x: rect.midX, y: rect.midY),
```

```
    radius: radius,
    startAngle: Angle(degrees: 0),
    endAngle: Angle(degrees: 180),
    clockwise: true)
```

Here you set the center point to be in the middle of the given rectangle with the radius set to the smaller of width or height.

➤ In `Shapes_Previews`, replace the `currentShape` property with:

```
  static let currentShape = Cone()
```



The arc

Forget everything you thought you knew about the clockwise direction. In iOS, angles always start at zero on the right hand side, and clockwise is reversed. So when you go from a start angle of 0° to an end angle of 180° with `clockwise` set `true`, you start at the right hand side and go anti-clockwise around the circle.

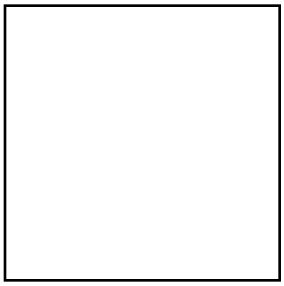
Describe an arc

This is for historical reasons. In macOS, the origin — that's coordinate (0, 0) — is at the bottom left, as in the standard Cartesian coordinate system. When iOS came out, Apple flipped the iOS drawing coordinate system on the Y axis so that (0, 0) is at the top left. However, much of the drawing code is based on the old macOS drawing coordinate system.

➤ In `Cone`'s `path(in:)`, add two straight lines to complete the cone before the `return`:

```
path.addLine(to: CGPoint(x: rect.midX, y: rect.height))
path.addLine(to: CGPoint(x: rect.midX + radius, y: rect.midY))
path.closeSubpath()
```

You start the first line where the arc left off and end it at the middle bottom of the available space. The second line ends at the middle of the right hand side.



The completed cone

Curves

As well as lines and arcs, you can add various other standard elements to a path, such as rectangles and ellipses. With curves, you can create any custom shape you want.

➤ At the end of **Shapes.swift**, add this code to create a new shape:

```
struct Lens: Shape {
  func path(in rect: CGRect) -> Path {
    var path = Path()
    // path code goes here
    return path
  }
}
```

The lens shape will consist of two quadratic curves, like an ellipse with a point at each end.

If you have used vector drawing applications, you'll have used control points to draw curves. To create a quadratic curve in code, you set a start point, an end point and a control point that defines where the curve goes.

Quadratic curve

The two mid points shown are calculated and define the curvature. It can take some practice to work out the control point for the curve.

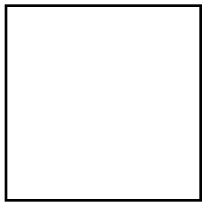
➤ In `path(in:)`, add this code before the `return`:

```
path.move(to: CGPoint(x: 0, y: rect.midY))
path.addQuadCurve(
  to: CGPoint(x: rect.width, y: rect.midY),
  control: CGPoint(x: rect.midX, y: 0))
path.addQuadCurve(
  to: CGPoint(x: 0, y: rect.midY),
  control: CGPoint(x: rect.midX, y: rect.height))
path.closeSubpath()
```

The first curve here is the same as in the diagram above, and the second curve mirrors it.

➤ In `Shapes_Previews`, replace the `currentShape` property to use this shape:

```
static let currentShape = Lens()
```



Lens shape

Strokes and Fills

Skills you'll learn in this section: stroke; stroke style; fill

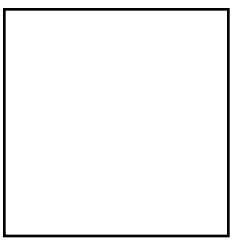
SwiftUI is currently filling the paths with a solid **fill**. You can specify the fill color or, alternatively, you can assign a **stroke**, which outlines the shape.

Stroke and fill

In `previews`, add this modifier to `currentShape`:

```
.stroke(lineWidth: 5)
```

You can only use `stroke(_:)` on objects conforming to `Shape`, so you must place the modifier directly after `currentShape`.



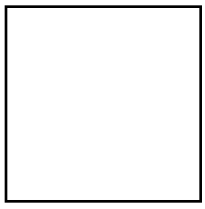
Stroke

Stroke Style

When you define a stroke, instead of giving it a `lineWidth`, you can give it a `StrokeStyle` instance.

For example:

```
currentShape
  .stroke(style: StrokeStyle(dash: [30, 10]))
```



StrokeStyle with dash

With a stroke style, you can define what the outline looks like — whether it is dashed, how the dash is formed and how the line ends look.

To form a dash, you create an array which defines the number of horizontal points of the filled section followed by the number of horizontal points of the empty section.

The example above describes a dashed line where you have a 5 point vertical line, followed by a 10 point space, followed by a one point vertical line, followed by a 5 point space.

This second example adds a dash phase, which moves the start of the dash to the right by 15 points, so that the dash starts with the one point line.

> **Swift tip**: You haven't done much animation so far as you'll cover this later in Chapter 21, "Delightful UX — Final Touches", but these dashed line parameters are animatable, so you can easily achieve the "marching ants" marquee look.

You can choose to change how the ends of lines look with the `lineCap` parameter:



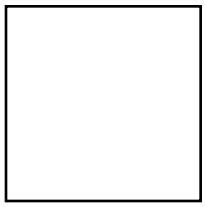
Line caps

`lineCap: .square` is similar to `.butt`, except that the ends protrude a bit further.

➤ In `previews`, replace `.stroke(lineWidth:)` with:

```
.stroke(
  Color.primary,
  style: StrokeStyle(lineWidth: 10, lineJoin: .round))
.padding()
```

Here you give the stroke an outline color and, using the `lineJoin` parameter, the two sections of lens shape are now nicely rounded at each side:



Line join

You've now created a few shapes and feel free to experiment with more. The challenge for this chapter suggests a few shapes for you to try.

Selecting an Element

Skills you'll learn in this section: borders; clip shapes

As well as displaying a shape view, you can use a shape to clip another view. You'll list all your shapes in a modal so that the user can select a photo element on the card and clip it to a chosen shape.

Before creating the modal, you'll set up a property to hold the selected element. You could pass this element around as a binding, but in this case you'll add it to `CardStore` as a published property. As you'll see shortly, when this property changes, all views affected by the property will redraw.

➤ Open **CardStore.swift** and add the new property:

```
@Published var selectedElement: CardElement?
```

You'll update this selected element when the user taps a card element.

When adding a selection property like this one, you should consider where best to place it. When listing and selecting cards, you added `selectedCard` to `CardsListView`. In that case, the selected card was only needed by one view, so it was an easy decision. If your app gets more complex, you may choose to move that `selectedCard` to `CardStore` as a published property, so that any view that uses `selectedCard` will redraw when the property changes. `selectedElement` will be used in several places, so it's easier to place the property in `CardStore`.

➤ Open **CardDetailView.swift** and locate `CardElementView(element: element)`. Add a new modifier to `CardElementView(element:)`:

```
.onTapGesture {
  store.selectedElement = element
}
```

When the user taps this element, you save it as the selected element.

➤ Add a new modifier to `card.backgroundColor`:

```
.onTapGesture {
  store.selectedElement = nil
}
```

When the user taps the card background, the selection clears.

The user leaves the card by tapping **Done**, but you've defined the **Done** button in a different file, and it's a good idea to keep similar code in close proximity.

➤ Add this new modifier to `ZStack`:

```
.onDisappear {
  store.selectedElement = nil
}
```

When the user taps **Done**, `CardDetailView` disappears and performs this closure.

The user will want to know which element he's selected, so you'll add a border to the element.

➤ First, add a new method to `CardDetailView` to determine whether the current element is selected:

```
func isSelected(_ element: CardElement) -> Bool {
  store.selectedElement?.id == element.id
}
```

This allows you to determine if a particular element is the currently selected element by comparing their ids.

➤ Add a new modifier to `CardElementView(element:)`. Because you want the border to resize and reposition together with the element, this should be the first modifier in the list.

```
.border(
  Settings.borderColor,
  width: isSelected(element) ? Settings.borderWidth : 0)
```

All views have a border, but if the element is not currently selected, the line width of the border is 0.

➤ Test your selection and border in Live Preview. Tap the background to clear the selection.



Border selection

You can only have one element selected at a time. When you update `store.selectedElement`, this affects the border of all element views. Because `store.selectedElement` is a published property, all views are redrawn with the correct border.

Clip Shapes Modal

Now that you can select an element, you'll apply a clip shape selected from frames displayed on a modal view.

➤ In the **Card Modal Views** group, create a new SwiftUI View file called **FrameModal.swift**. This will be very similar to **StickerModal.swift**, but will load your custom shapes instead of stickers into a grid.

First, set up an array of all your shapes for the modal to iterate through.

Open **Shapes.swift** and add a new enumeration to hold all the shapes:

```
enum Shapes {
}
```

Initially, you might think you can define the array in `Shapes` like this:

```
static let shapes: [Shape] = [Circle(), Rectangle()]
```

However, this will give you a compile error:

```
Use of protocol 'Shape' as a type must be written 'any Shape'
```

So, how can you solve this? Read on!

Associated Types

Skills you'll learn in this section: protocols with associated types; type erasure

Swift Dive: Protocols With Associated Types

Protocols with associated types (**PAT**s) are advanced black magic Swift and, if you haven't done much programming with generics, the subject will take some time to learn and absorb. Apple APIs use them everywhere, so it's useful to have an overview.

`Shape` inherits from `View`, and this is how `View` is defined:

```
public protocol View {
    associatedtype Body : View
    @ViewBuilder var body: Self.Body { get }
}
```

`associatedType` makes a protocol generic. When you create a structure that conforms to `View`, the requirement is that you have a `body` property, and you tell the `View` the real type to substitute. For example:

```
struct ContentView: View {
  var body: some View {
    EmptyView()
  }
}
```

In this example, `body` is of type `EmptyView`.

Earlier, you created the protocol `CardElement`. This doesn't use an associated type, and so you were able to set up an array of type `CardElement`. This is how you defined `CardElement`:

```
protocol CardElement {
  var id: UUID { get }
  var transform: Transform { get set }
}
```

All of the property types in `CardElement` are **existential types**. That means they are types in their own right and not generic. However, you might have a requirement for `id` to be either a `UUID` or an `Int` or a `String`. In that case you can define `CardElement` with a generic type of `ID`:

```
protocol CardElement {
  associatedtype ID
  var id: ID { get }
  var transform: Transform { get set }
}
```

When you create a structure conforming to `CardElement`, you tell it what type `ID` actually is. For example:

```
struct NewElement: CardElement {
  let id = Int.random(in: 0...1000)
  var transform = Transform()
}
```

In this case, whereas the other `CardElement` ids are of type `UUID`, this `id` is of type `Int`.

Once a protocol has an associated type, because it is now a generic, the protocol is no longer an existential type. The protocol is constrained to using another type, and the compiler doesn't have any information about what type it might actually be. For this reason, you can't set up an array containing protocols with associated types, such as `View` or `Shape`.

Going back to the code at the start of this section which doesn't compile:

```
static let shapes: [Shape] = [Circle(), Rectangle()]
```

Even though `Circle` and `Rectangle` both conform to `Shape`, they are `Shapes` with different associated types and, as such, you can't put them both in the same `Shape` array.

Type Erasure

You are able to place different `Views` in an array by converting the `View` type to `AnyView`:

```
// does not compile
let views: [View] = [Text("Hi"), Image("giraffe")]
// does compile
let views: [AnyView] = [
  AnyView(Text("Hi")),
  AnyView(Image("giraffe"))
]
```

`AnyView` is a **type-erased** view. It takes in any type of view and passes back an existential, non-generic type of `AnyView`.

Similarly, Apple provides `AnyShape`.

A Type Erased Array

➤ In **Shapes.swift**, add a new property to `Shapes`:

```
static let shapes: [AnyShape] = [
  AnyShape(Circle()), AnyShape(Rectangle()),
  AnyShape(Cone()), AnyShape(Lens())
]
```

This holds a type-erased list of all your defined shapes. When you create more shapes, add them to this array.

Shape Selection Modal

Now that you have all your shapes in an array, you can create a selection modal, just as you did for your stickers.

➤ Open **FrameModal.swift** and replace `FrameModal` with:

```
struct FrameModal: View {
  @Environment(\.dismiss) var dismiss
  // 1
  @Binding var frameIndex: Int?
  private let columns = [
    GridItem(.adaptive(minimum: 120), spacing: 10)
  ]
  private let style = StrokeStyle(
    lineWidth: 5,
    lineJoin: .round)

  var body: some View {
    ScrollView {
      LazyVGrid(columns: columns) {
      // 2
        ForEach(0..<Shapes.shapes.count, id: \.self) { index in
          Shapes.shapes[index]
          // 3
            .stroke(Color.primary, style: style)
            // 4
            .background(
              Shapes.shapes[index].fill(Color.secondary))
            .frame(width: 100, height: 120)
            .padding()
            // 5
            .onTapGesture {
              frameIndex = index
              dismiss()
            }
        }
```

```
        }
      }
      .padding(5)
    }
  }
```

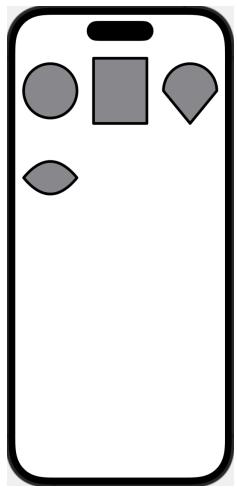
This is almost exactly the same code as you wrote for `StickerModal`. The exceptions are:

1. Pass in an integer that will hold the index of the selected shape in the `Shapes` array.
2. Iterate through the array of shapes by index.
3. Outline the shape with the primary color.
4. Fill the shape so that you have a touch area. If you don't fill the shape, the tap will only work on the stroke.
5. When the user taps the shape, update `frameIndex` and dismiss the modal.

➤ Change the preview to:

```
struct FrameModal_Previews: PreviewProvider {
  static var previews: some View {
    FrameModal(frameIndex: .constant(nil))
  }
}
```

➤ Live Preview `FrameModal` to see all your shapes in a grid:



Shapes Listing

Adding the Frame Picker Modal to the Card

Skills you'll learn in this section: conditional modifiers; disabling button

➤ Open **CardToolbar.swift** and add two new properties to `CardToolbar`:

```
@EnvironmentObject var store: CardStore
@State private var frameIndex: Int?
```

You load the card store into the environment so you can access the selected element. You also define the index of the frame shape that you'll pass to `FrameModal`.

➤ In `body(content:)`, locate `.sheet(item:)` and add a new case to the `switch` statement before the `default` case:

```
case .frameModal:
  FrameModal(frameIndex: $frameIndex)
    .onDisappear {
      if let frameIndex {
        card.update(
          store.selectedElement,
          frameIndex: frameIndex)
      }
      frameIndex = nil
    }
```

Here you call the modal and update the card element with the frame index. As you haven't written `update(_:frameIndex:)` yet, you'll get a compile error.

Adding the Frame to the Card Element

➤ Open **CardElement.swift** in the **Model** group and add a new property to `ImageElement`:

```
var frameIndex: Int?
```

This will hold the element's frame index. You add it solely to `ImageElement` because the frame will clip only images, not text.

> **Note**: If later, you create another type of element such as `ColorElement`, to which you also want to be able to add clip frames, you could create a protocol `Clippable`, with `frameIndex` as a required property. Instead of testing to see if an element is an `ImageElement`, you can test to see whether the element is `Clippable`.

➤ Open **Card.swift** and add the new update method to `Card`:

```
mutating func update(_ element: CardElement?, frameIndex: Int) {
  if let element = element as? ImageElement,
    let index = element.index(in: elements) {
      var newElement = element
      newElement.frameIndex = frameIndex
      elements[index] = newElement
  }
}
```

Here you pass in the element and the frame index. Because `element` is immutable and you need to update its `frameIndex`, you create a new mutable copy and update `elements` with this new instance.

All that's left to do now is to clip the image element.

➤ Open **CardDetailView.swift** and locate `CardElementView`. Add a new modifier to `CardElementView` before `border(_:width:)`:

```
.clipShape(Shapes.shapes[0])
```

The first element in `Shapes.shapes` is a circle.

➤ Live Preview the view:



Clipped elements

Conditional Modifiers using @ViewBuilder

You applied `.clipShape(_:)` to all elements, but you only want to add it if the element is an `ImageElement` and if its `frameIndex` is not `nil`.

Surprisingly, it's not easy to add a conditional modifier in SwiftUI. You can show `Views` conditionally using `if {} else {}`, and with simple conditions, you could show the same view with and without a modifier. However, when you have multiple modifiers on a view, this leads to heavily duplicated code.

In Chapter 9, "Refining Your App", when you wanted to conditionally show a button shape, you created a method with a `ViewBuilder` attribute, and that's what you'll do here too.

➤ First, remove the previous code `.clipShape(Shapes.shapes[0])` from `CardDetailView`.

➤ Open **CardElementView.swift** and at the end of the file, add a new extension:

```
// 1
private extension ImageElementView {
  // 2
  @ViewBuilder
  func clip() -> some View {
    // 3
    if let frameIndex = element.frameIndex {
      // 4
      let shape = Shapes.shapes[frameIndex]
      self
        .clipShape(shape)
    } else { self }
  }
}
```

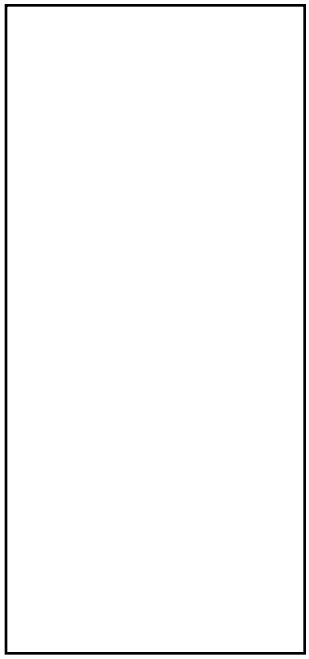
Going through the code:

1. The modifier is specific to this view, so you create it as a private extension. Creating the extension on `ImageElementView` means that clipping will only apply to this type. Clipping a text element makes little sense.
2. The `ViewBuilder` attribute allows you to build up views and combine them into one. Check out Chapter 9, "Refining Your App" if you need a refresher on how this works.
3. Use `if-let` to get the `frameIndex`.
4. If there's a value in `frameIndex`, clip the view with the element's frame shape. Otherwise, return the unmodified view.

➤ In `CardElementView`, add the new modifier to `ImageElementView`:

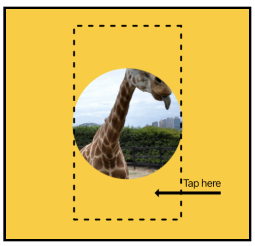
```
ImageElementView(element: element)
  .clip()
```

➤ Build and run the app and choose the first yellow card. Tap a giraffe and choose **Frames**. Select a frame and the giraffe photo gets clipped to that shape. The selection border is still rectangular, but you'll fix that in the challenge at the end of this chapter.



Clipped giraffe

When you tap the background near an unselected clipped image, but inside the area of the original unclipped image, SwiftUI still thinks you're tapping the image.



Tap area doesn't match shape

In `clip()`, add this modifier after `.clipShape(shape)`

```
  .contentShape(shape)
```

This will clip the tap area to the frame.

Disabling the Frames Button

It doesn't make sense to show the list of clip frames unless you have selected an element ready for clipping. So, until you select an element, the **Frames** button should be disabled.

➤ Open **BottomToolbar.swift** and add this new property to `BottomToolbar`:

```
@EnvironmentObject var store: CardStore
```

➤ In `BottomToolbar_Previews`, add this modifier to `BottomToolbar(card:modal:)`:

```
.environmentObject(CardStore())
```

By choosing to place `selectedElement` as a published property in `CardStore`, any view in the hierarchy where `CardStore` is defined as an environment object has access to the store. In Live Preview, the hierarchy starts at `BottomToolbar_Previews`. When you run your app, the hierarchy starts at `CardsListView`, loaded in **CardsApp.swift**.

In `BottomToolbar`, in `body`, you'll duplicate the default button and use it for `frameModal`.

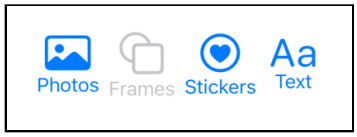
➤ To reduce code duplication, extract the default button into a new method:

```
func defaultButton(_ selection: ToolbarSelection) -> some View {
  Button {
    modal = selection
  } label: {
    ToolbarButton(modal: selection)
  }
}
```

➤ In `body`, in `switch selection`, replace the entire `default:` condition with:

```
case .frameModal:
  defaultButton(selection)
    .disabled(
      store.selectedElement == nil
      || !(store.selectedElement is ImageElement))
default:
  defaultButton(selection)
```

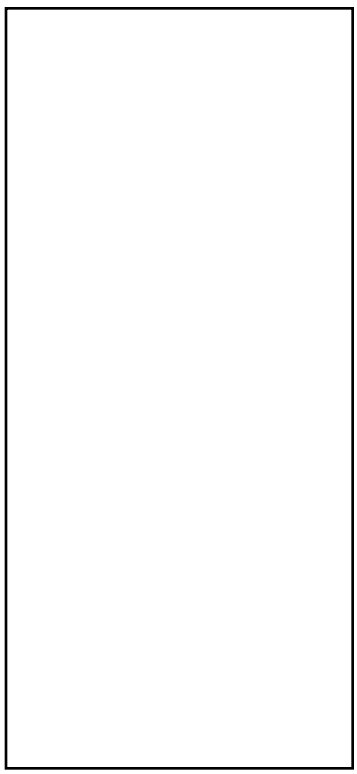
By separating out `frameModal`, you can disable the button when there is no selected element. It also makes no sense to be able select clip frames on a `TextElement`, so you check that the selected element is an `ImageElement`.



Disabled Frames button

In Live Preview, the **Frames** button is disabled, as `CardStore` is initialized by `BottomToolbar_Previews`.

➤ Build and run the app and check that you can still add frames to selected image elements:



Button is still disabled when text is selected

Challenges

Challenge 1: Create new Shapes

Practice creating new shapes and place them in the frame picker modal. Here are some suggestions:

Try these shapes

The last two are a `Polygon` shape with a number of sides property, so try that out, and take a look at the code in the challenge folder.

Challenge 2: Clip the Selection Border

Currently, when you tap an image, it gets a rectangular border around it. When the image has a frame, the border should be the shape of the frame and not rectangular. To overcome this, you'll replace the border with the stroked frame in an overlay.

1. In **CardDetailView.swift**, add a new `View` extension and create a new method similar to `ImageElementView.clip()`. Pass in the element and whether the element is selected. Call it `overlay(element:isSelected)`.
2. In the new method, if the element is selected, when it is an image and has a frame, replace the border modifier with an overlay of the stroked frame. If the element is selected and doesn't have a frame or is not an image, add a border modifier as before.
3. In `CardDetailView`, replace the border modifier with your new overlay.

Check your changes out by running the app or by Live Previewing `SingleCardView`.



Clipped photos

Key Points

- The `Shape` protocol provides an easy way to draw a 2D shape. There are some built-in shapes, such as `Rectangle` and `Circle`, but you can create custom shapes by providing a `Path`.
- `Path`s are the outline of the 2D shape, made up of lines and curves.
- A `Shape` fills by default with the primary color. You can override this with the `fill(_:style:)` modifier to fill with a color or gradient. Instead of filling the shape, you can stroke it with the `stroke(_:lineWidth:)` modifier to outline the shape with a color or gradient.
- With the `clipShape(_:style:)` modifier, you can clip any view to a given shape.
- Associated types in a protocol make a protocol generic, making the code reusable. Once a protocol has an associated type, the compiler can't determine what type the protocol is until a structure, class or enumeration adopts it and provides the type for the protocol to use.
- Using type erasure, you can hide the type of an object. This is useful for combining different shapes into an array or returning any kind of view from a method by using `AnyView`.
- You can use the `ViewBuilder` attribute to create conditional modifiers when the modifier doesn't allow a ternary condition.

Chapter 19: Saving Files

By Caroline Begbie

You've set up most of your user interface, and it would be nice at this stage to have the card data persist between app sessions. You can choose between a number of different ways to save data.

You've already looked at `UserDefaults` and property list (plist) files in Section 1. These are more suitable for simple data structures, whereas, when you save your card, you'll be saving images and sub-arrays of elements. While Core Data could handle this, another way is to save the data to files using the JSON format. One advantage of JSON is that you can easily examine the file in a text editor and check that you're saving everything correctly.

This chapter will cover saving JSON files to your app's **Documents** folder by encoding and decoding the JSON representation of your cards.

The Starter Project

To assist you with saving `UIImages` to disk, the starter project contains methods in a `UIImage` extension to resize an image and to save, load and remove image files. These are in **UIImageExtensions.swift**.

In the first challenge for this chapter, you'll store the card's background color. **ColorExtensions.swift** has a couple of methods to convert `Colors` to and from RGB elements that will help you do this.

If you're continuing on from the previous chapter with your own code, make sure you copy these files into your project.

The Saved Data Format

When you save the data, each card will have a JSON file with a **.rwcard** extension. This file will contain the list of elements that make up the card. You'll save the images separately. The data store on disk will look like:

Data store

When your app first starts, you'll read in all the **.rwcard** files in the **Documents** folder and show them in a scroll view. When the user taps a selected card, you'll process the card's elements and load the relevant image files.

When to Save the Data

Skills you'll learn in this section: when to save data; `ScenePhase`

There are two ways you can proceed, and each has its pros and cons.

You can choose to save the card file every time you change anything, such as adding, moving or deleting elements. This means that your data on disk is always up-to-date. The downside is that your saving is spread out all over your app.

Alternatively, you could choose to save when you really need to:

1. When `SingleCardView` disappears, which happens when the user taps **Done**.
2. When the app becomes inactive through the user switching apps or an external event such as a phone call.

The downside of this method is that if your app crashes before you've done the save, then the last few changes the user made might not be recorded. You'll also need to remember when testing that the app doesn't save in the simulator until you press **Done**.

In this app, you'll choose a hybrid approach. You'll perform the first method of saving whenever you create or delete card data. This is primarily because of saving the image element's `UIImage`. You'll save the `UIImage` when you choose it from the **Photos** or **Stickers** modal, and you'll store the file id in the `ImageElement`. To maintain data integrity, it's a good idea to store the `ImageElement` at the same time as the `UIImage`.

However, moving and resizing elements happens regularly, and saving every time can be quite inefficient. To save the transform data, you'll choose the second method: saving when the user taps **Done** or leaves the app.

Saving When the User Taps Done

➤ Open the starter project. In the **Model** group, open **Card.swift** and create a new method in `Card`:

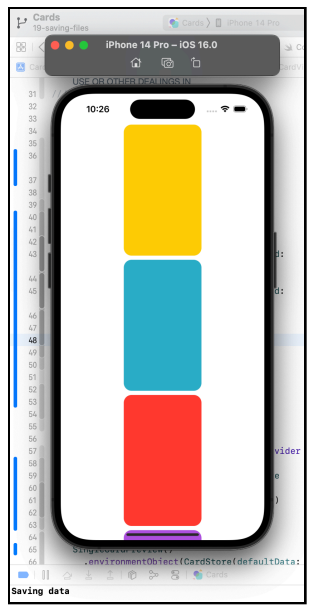
```
func save() {
  print("Saving data")
}
```

You'll come back to this method to perform the saving later in this chapter.

➤ Open **SingleCardView.swift** and add a new modifier to `CardDetailView(card:)` inside `body`:

```
.onDisappear {
  card.save()
}
```

➤ Build and run the app, tap a card, then tap **Done**. You'll see "Saving data" appear in the console.



Saving data

Using ScenePhase to Check Operational State

When you exit the app, surprisingly, the view does not perform `onDisappear(_:)`, so the card won't get saved. However, you can check what state your app is in through the environment.

Views presented through `fullScreenCover(item:)` don't inherit the environment, so you'll check the environment in `SingleCardView`'s parent view.

➤ Open **CardsListView.swift** and add a new environment property:

```
@Environment(\.scenePhase) private var scenePhase
```

`scenePhase` is a useful member of `EnvironmentValues`. It's an enumeration of three possible values:

- `active`: the scene is in the foreground.
- `inactive`: the scene should pause.
- `background`: the scene is not visible in the UI.

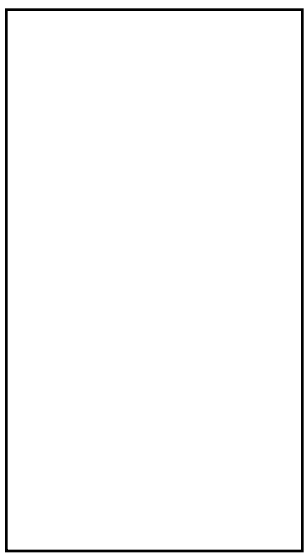
You'll save when `scenePhase` becomes `inactive`.

➤ In `body`, locate `fullScreenCover(item:)` and add a modifier to `SingleCardView(card:)`:

```
.onChange(of: scenePhase) { newScenePhase in
  if newScenePhase == .inactive {
    store.cards[index].save()
  }
}
```

`onChange(of:)` is called whenever `scenePhase` changes. If the new value is `inactive` then save the card. Be mindful that `card` is not the same instance as `store.cards[index]` here, so `card.save()` would not save correctly.

➤ Build and run the app, tap a card to open it and exit your app by swiping up from the bottom. You'll see the console message **"Saving data"**.



Saving data

➤ Return to the app in the simulator. It will resume inside the card where you left it. There is no way to simulate a phone call on the simulator, but you can activate Siri to test external events. Choose **Device ➤ Siri** and, once again, you'll see the console message **"Saving data"**.

You've now implemented the skeleton for the saving part of your app. The rest of the chapter will take you through encoding and decoding data so you can perform `save()`.

JSON Files

Skills you'll learn in this section: the JSON format

JSON is an acronym for JavaScript Object Notation. JSON data is formatted like this:

```
{
  "identifier1": [data1, data2, data3],
  "identifier2": data4
}
```

Each `data` item can be a nested chunk of JSON.

To explore how easy it is save simple data to JSON files, you'll create a temporary structure and save it.

Codable

Skills you'll learn in this section: `Encodable`; `Decodable`

The `Codable` protocol is a type alias for `Decodable & Encodable`. When you conform your structures to `Codable`, you conform to both these protocols. As its name suggests, you use `Codable` to encode and decode data to and from external files.

➤ Open **CardsApp.swift** and add this code to the end of the file:

```
struct Team: Codable {
  let names: [String]
  let count: Int
}

let teamData = Team(
  names: [
  "Richard", "Libranner", "Caroline", "Audrey", "Sandra"
  ], count: 5)
```

After you've seen how `Codable` works, you'll delete this code and apply your knowledge to the more complex data in your app.

This structure contains straightforward data of types that JSON supports — an array of `Strings` and an `Int`. `Team` conforms to `Codable` and makes `Team` a type that can encode and decode itself.

Encoding

➤ In `Team`, create a new method:

```
static func save() {
  do {
  // 1
    let encoder = JSONEncoder()
    encoder.outputFormatting = .prettyPrinted
    // 2
    let data = try encoder.encode(teamData)
    // 3
    let url = URL.documentsDirectory
      .appendingPathComponent("TeamData")
    try data.write(to: url)
} catch {
  print(error.localizedDescription)
  }
}
```

Going through this code:

1. Initialize the JSON encoder. `prettyPrinted` means that the encoded data will be easier for you to read.
2. Encode the data from `teamData` to a byte buffer of type `Data`.
3. Write the data to a file called `TeamData` in the **Documents** folder.

➤ In `CardsApp`, create a temporary initializer:

```
init() {
  Team.save()
}
```

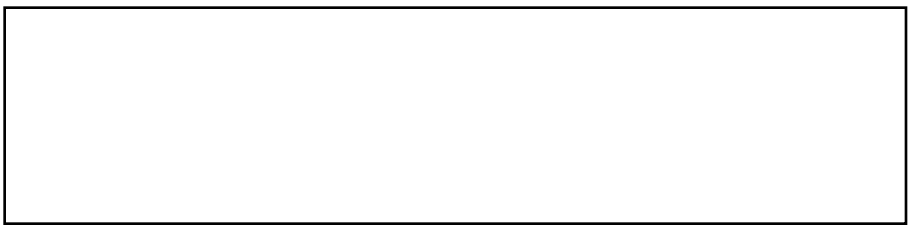
This will save the team data at the very start of the app so that you can examine it.

➤ In `body`, add a new modifier to `CardsListView()`:

```
.onAppear {
  print(URL.documentsDirectory)
}
```

You print out the URL of the **Documents** folder so you can find the file you've saved.

➤ Build and run the app. Highlight the **Documents** URL that shows up in the debug console, then right-click it and choose **Services ➤ Show in Finder**. Drag the parent folder to your **Favorites** sidebar as you'll be visiting this folder often while you're testing.



Team data

➤ In Finder, right-click **TeamData** and open the file in **TextEdit**:

```
{
  "names" : [
    "Richard",
    "Libranner",
    "Caroline",
    "Audrey",
    "Sandra"
  ],
  "count" : 5
}
```

This is your structure data stored in JSON format. The identifiers are the names you used in the structure. As you can see, using `Codable`, it's easy to store data.

Decoding

Reading the data back in is just as easy.

➤ In `Team` add a new method:

```
static func load() {
  // 1
  let url = URL.documentsDirectory
    .appendingPathComponent("TeamData")
  do {
  // 2
    let data = try Data(contentsOf: url)
    // 3
    let decoder = JSONDecoder()
    // 4
    let team = try decoder.decode(Team.self, from: data)
    print(team)
  } catch {
    print(error.localizedDescription)
  }
}
```

Going through this code:

1. Get the URL.
2. Read the data from the URL into a `Data` type.
3. This time you're decoding, so you initialize a JSON decoder.
4. Decode the data into an instance of `Team` and print it to the console so that you can see what you've decoded.

➤ In `CardsApp`, change `init()` to:

```
init() {
  Team.load()
}
```

➤ Build and run, and see the new instance of `Team` loaded from **TeamData** printed out in the console before the **Documents** URL.

Loaded team data

You can see that the theory of saving and loading data using `Codable` is simple. But naturally, with real life data, there are always complications.

Encoding and Decoding Custom Types

Skills you'll learn in this section: encoding; decoding; `compactMap(_:)`

Data types that you want to store must conform to `Codable`. If you check the developer documentation for the properties contained by `Team`, which are `String` and `Int`, you'll see they both conform to `Decodable` and `Encodable`.

Custom types which store only `Codable` types present no problem. But how about one of your custom types that contain types that do not conform to `Codable`?

Before continuing, remove the sample `Team` code you created.

➤ In **CardsApp.swift**, remove `init()`, all of `Team` and `teamData`.

➤ Open **Transform.swift** and add this new extension:

```
extension Transform: Codable {}
```

You get a compile error: "Type `Transform` does not conform to protocol `Decodable`".

`Transform` contains two data types: `CGSize` and `Angle`. When you check the documentation, you'll find that `CGSize` conforms to `Encodable` and `Decodable`, whereas `Angle` does not.

When you conform your custom type to `Codable`, there are two required methods: `init(from:)` and `encode(to:)`.

When all the types in your custom type conform to `Codable`, then all you have to do is add `Codable` conformance to your custom type, and `Codable` will automatically **synthesize** (create) the initializer and encoder methods.

Codable synthesized methods

When the structure contains types that don't conform to `Codable`, you must implement the two synthesized methods yourself.

➤ In the **Extensions** group, create a new Swift file called **AngleExtensions.swift**.

➤ Replace the code with:

```
import SwiftUI

extension Angle: Codable {
  public init(from decoder: Decoder) throws {
    self.init()
  }

  public func encode(to encoder: Encoder) throws {
  }
}
```

You conform `Angle` to `Codable` and provide the two required methods. Because all the types used by `Transform` are now `Codable`, your code will now compile. However, the encoder and decoder methods you just created aren't doing anything useful. You'll have to tell the coders how to encode and decode every property that you want saved and loaded.

To do this, you create an enumeration that conforms to `CodingKey`, listing all the properties you want saved.

➤ Add this to the `Angle` extension:

```
enum CodingKeys: CodingKey {
  case degrees
}
```

You list only the properties that you want to save and restore. `radians` is another `Angle` property, but `Angle` can construct that internally from `degrees`, so you don't need to store it.

➤ Add this to `encode(to:)`:

```
var container = encoder.container(keyedBy: CodingKeys.self)
try container.encode(degrees, forKey: .degrees)
```

You create an encoder container using `CodingKeys`. Then you encode `degrees`, which is of type `Double`. This is a `Codable` type, so the container can encode it.

Decoding is similar.

➤ Replace the contents of `init(from:)` with:

```
let container = try decoder.container(keyedBy: CodingKeys.self)
let degrees = try container
  .decode(Double.self, forKey: .degrees)
self.init(degrees: degrees)
```

You create a decoder container to decode the data. As `degrees` is a `Double`, you decode a `Double` type. Then, you can initialize the `Angle` from the decoded degrees.

With `Angle` taken care of, and `CGSize` already conforming to `Codable`, `Transform` will now be able to synthesize the encoding and decoding methods and encode and decode itself, so your app will now compile.

You're eventually going to save a `Card`, so all types in the data hierarchy will need to to be `Codable`. Going up from `Transform` in your data structure hierarchy, the next structure that you'll tackle is `ImageElement`.

Encoding ImageElement

➤ Open **CardElement.swift** and take a look at `ImageElement`.

When saving an image element, you'll save `transform`, `uiImage` and `frameIndex`. You don't need to save the UUID, as it will get reconstructed when you initialize the element. `transform` and `frameIndex` conform to `Codable`, however `UIImage` does not.

You could save out the `UIImage` data into the card data, but it's good practice to record binary files separately and save the binary file name with the card data.

➤ Add a new property to `ImageElement`:

```
var imageFilename: String?
```

This will hold the name of the saved image file, which will be a UUID string.

➤ Open **Card.swift** and replace `addElement(uiImage:)` with:

```
mutating func addElement(uiImage: UIImage) {
// 1
  let imageFilename = uiImage.save()
  // 2
  let element = ImageElement(
    uiImage: uiImage,
    imageFilename: imageFilename)
  elements.append(element)
}
```

The changes from the previous code are:

1. You now save the `UIImage` to a file using the code provided in **UIImageExtensions.swift**. `uiImage.save()` saves the **PNG** data to disk and returns a UUID string as the filename. Before saving, `save()` resizes large images, as you don't need to store the full resolution for the card.
2. You create the new element with the string filename and the original uiImage.

You'll also need to remove the image file from disk when the user deletes the element.

➤ In `remove(_:)`, add this to the top of the method:

```
if let element = element as? ImageElement {
  UIImage.remove(name: element.imageFilename)
}
```

You check that the element is an `ImageElement` and use the method provided in **UIImageExtensions.swift** to remove the file from disk.

➤ Back in **CardElement.swift**, add a new extension after `ImageElement`:

```
extension ImageElement: Codable {
}
```

When adding a second initializer to the main definition of a structure, you lose the default initializer and have to recreate it yourself. However, adding initializers to extensions doesn't have this effect. When you conform `ImageElement` to `Codable`, you provide the decoding initializer `init(from:)`. By adding the initializer to this extension, you keep both the default initializer and the new decoding one.

➤ Add the `CodingKey` enumeration containing the properties to save to the `ImageElement` extension:

```
enum CodingKeys: CodingKey {
  case transform, imageFilename, frameIndex
}
```

You'll save the transform, filename and frame index to disk.

➤ Add the decoder:

```
init(from decoder: Decoder) throws {
  let container = try decoder
    .container(keyedBy: CodingKeys.self)
  // 1
  transform = try container
    .decode(Transform.self, forKey: .transform)
  frameIndex = try container
    .decodeIfPresent(Int.self, forKey: .frameIndex)
  // 2
  imageFilename = try container.decodeIfPresent(
    String.self,
    forKey: .imageFilename)
  // 3
  if let imageFilename {
    uiImage = UIImage.load(uuidString: imageFilename)
  } else {
    // 4
```

```
      uiImage = UIImage.errorImage
    }
  }
```

Going through the decoding:

1. Decode the transform and frame index. They are `Codable`, so they take care of themselves.
2. When decoding optionals, such as `frameIndex` and `imageFilename`, if you decode something that doesn't exist, the decoder will throw an error. Check whether the data exists using `decodeIfPresent(_:forKey:)`.
3. If the filename is present, load the image using the filename.
4. If there's an error loading the image, use the error image in **Assets.xcassets**.

➤ Add the encoder to the `ImageElement` `Codable` extension:

```
  func encode(to encoder: Encoder) throws {
    var container = encoder.container(keyedBy: CodingKeys.self)
    try container.encode(transform, forKey: .transform)
    try container.encode(frameIndex, forKey: .frameIndex)
    try container.encode(imageFilename, forKey: .imageFilename)
  }
```

Here you're encoding the transform, the frame index and the image filename.

Decoding and Encoding the Card

➤ Open **Card.swift** and add a new extension with the list of properties to save:

```
  extension Card: Codable {
    enum CodingKeys: CodingKey {
      case id, backgroundColor, imageElements, textElements
    }
  }
```

For `Card`, you'll save the id. This will be the name of the JSON file that you'll store all the data in, so it's important to keep track of the id to ensure data integrity. You'll store the background color in the first challenge at the end of the chapter. You'll also store image elements and text elements in two separate arrays.

➤ First add the decoder in the extension:

```
init(from decoder: Decoder) throws {
  let container = try decoder
    .container(keyedBy: CodingKeys.self)
  // 1
  let id = try container.decode(String.self, forKey: .id)
  self.id = UUID(uuidString: id) ?? UUID()
  // 2
  elements += try container
    .decode([ImageElement].self, forKey: .imageElements)
}
```

Going through the decoder:

1. Decode the saved id string and restore `id` from the UUID string.
2. Load the array of image elements. You use the += operator to add to any elements that may already be there, just in case you load the text elements first. You'll load the text elements in the challenge at the end of the chapter.

As you're restoring `id`, you'll need to make it a `var`.

➤ In `Card`, change `let id = UUID()` to:

```
var id = UUID()
```

➤ Add the encoder to `Card`'s `Codable` extension:

```
func encode(to encoder: Encoder) throws {
  var container = encoder.container(keyedBy: CodingKeys.self)
  try container.encode(id.uuidString, forKey: .id)
  let imageElements: [ImageElement] =
    elements.compactMap { $0 as? ImageElement }
  try container.encode(imageElements, forKey: .imageElements)
}
```

Here you encode the id as a UUID string. You also extract all the image elements from `elements` using `compactMap(_:)`

Swift Dive: compactMap(_:)

`compactMap(_:)` returns an array with all the non-nil elements that match the closure. `$0` represents each element.

When code is more complex than the above, you can replace the closure with:

```
let imageElements: [ImageElement] =
  elements.compactMap { element in
  element as? ImageElement
}
```

This replaces the non-descriptive `$0` with `element`.

The code is equivalent to:

```
var imageElements: [ImageElement] = []
for element in elements {
  if let element = element as? ImageElement {
    imageElements.append(element)
  }
}
```

The biggest advantage of using `compactMap(_:)` is that `imageElements` is a constant. This is safer because you can't accidentally add data to it at a later time. It's also less code and more readable once you're accustomed to using array methods such as `map(_:)` and `filter(_:)`. When necessary, you can compose them them together to create arrays from complex operations.

Saving the Card

With all the encoding and decoding in place, you can finally implement `save()`.

➤ Still in **Card.swift**, replace `save()` with:

```
func save() {
  do {
  // 1
    let encoder = JSONEncoder()
    // 2
    let data = try encoder.encode(self)
    // 3
    let filename = "\(id).rwcard"
    let url = URL.documentsDirectory
      .appendingPathComponent(filename)
    // 4
    try data.write(to: url)
  } catch {
    print(error.localizedDescription)
  }
}
```

To save the data, you:

1. Set up the JSON encoder.
2. Set up a `Data` property. This is a buffer that will hold any kind of byte data and is what you will write to disk. Fill the data buffer with the encoded `Card`.
3. The filename will be the card id plus a **.rwcard** extension.
4. Write the data to the file.

Perform this method whenever there are changes to the card.

➤ Add:

```
save()
```

to the end of:

- `remove(_:)`
- `addElement(uiImage:)`

These are the methods that update the image file, so you do an additional save to protect data integrity. You already call `save()` when the user presses the **Done** button and, also, when the app scene phase changes.

➤ Build and run in Simulator.

➤ Open the **Documents** folder in Finder. The folder path prints out in the console, but you should have the folder in your **Favorites** sidebar.

➤ In Simulator, choose the yellow card and add a new photo. (Don't use the pink flowers as currently that file format does not work.) When the card adds the new element, it saves the photo to a **PNG** file and itself to a file with the **.rwcard** extension. In Finder, open the **.rwcard** file in TextEdit — you should just be able to double click it to open it.

```
{"id":"9E91BACF-8ABB-47AA-8137-80FD5FDB7F9B","imageElements":
[{"frameIndex":null,"imageFilename":null,"transform":{"offset":
[27,-140],"size":[250,180],"rotation":{"degrees":0}}},
{"frameIndex":null,"imageFilename":null,"transform":{"offset":
[-80,25],"size":[380,270],"rotation":{"degrees":0}}},
{"frameIndex":null,"imageFilename":null,"transform":{"offset":
[80,205],"size":[250,180],"rotation":{"degrees":0}}},
{"frameIndex":null,"imageFilename":"010050AF-A949-4DA4-9284-
CF998EF6885E","transform":{"offset":[0,0],"size":
[250,180],"rotation":{"degrees":0}}}]}
```

You'll see something like the above. This is the JSON format as described earlier. You can see that you're saving the card `id`, which matches the filename and, also, an array of four `imageElements`. The first three elements will have `null` in the filename as they were provided by the preview data and never saved to a file. The last element will contain the name of the saved image file in `imageFilename`.

If you want to make the output more human readable, in `save()`, after initializing `encoder`, you can add:

```
encoder.outputFormatting = .prettyPrinted
```

Loading Cards

> **Skills you'll learn in this section**: file enumeration

Now that you've saved a card, you'll start the app by loading them.

File Enumeration

To list the cards, you'll iterate through all the files with an extension of **.rwcard** and load them into the cards array.

➤ Open **CardStore.swift** and create a new extension with the method to load the files:

```
extension CardStore {
  // 1
  func load() -> [Card] {
    var cards: [Card] = []
    // 2
    let path = URL.documentsDirectory.path
    guard
      let enumerator = FileManager.default
        .enumerator(atPath: path),
      let files = enumerator.allObjects as? [String]
    else { return cards }
    // 3
    let cardFiles = files.filter { $0.contains(".rwcard") }
    for cardFile in cardFiles {
      do {
        // 4
        let path = path + "/" + cardFile
```

```
          let data =
            try Data(contentsOf: URL(fileURLWithPath: path))
          // 5
          let decoder = JSONDecoder()
          let card = try decoder.decode(Card.self, from: data)
          cards.append(card)
        } catch {
          print("Error: ", error.localizedDescription)
        }
      }
      return cards
    }
  }
```

Going through the code:

1. You'll return an array of `Cards` from `load()`. These will be all the cards in the **Documents** folder.
2. Set up the path for the **Documents** folder and enumerate all the files and folders inside this folder.
3. Filter the files so that you only hold files with the **.rwcard** extension. These are the `Card` files.
4. Read each file into a `Data` variable.
5. Decode each `Card` from the `Data` variable. You've done all the hard work of making all the properties used by `Card` and its subtypes `Codable`, so you can then simply add the decoded `Card` to the array you're building.

➤ Replace the implementation of `init(defaultData:)` with:

```
  cards = defaultData ? initialCards : load()
```

Instead of using the default data, you can choose to load the cards from disk.

➤ In **CardsApp.swift**, initialize `store` without the default data:

```
  @StateObject var store = CardStore()
```

➤ Build and run the app and check out your saved data.

If there are no file data errors, you'll see this result:

Loading your data

Remember that you created the card using default asset images, which hadn't been saved to disk. You'll see error images replacing those, but you'll see the image that you added to the card inside the app, which was duly saved.

Creating new Cards

Without the default data, you'll need some way of adding cards. You'll create an **Add** button that you'll enhance in the following chapter.

First, you'll need a method to add a new card.

➤ Open **CardStore.swift** and add this new method to `CardStore`:

```
func addCard() -> Card {
  let card = Card(backgroundColor: Color.random())
  cards.append(card)
  card.save()
  return card
}
```

You create a new card with a random background color, add it to the array of cards and save it to disk.

➤ Open **CardsListView.swift** and, in `body`, embed `list` in a `VStack`.

➤ At the end of the `VStack`, so that it shows up under `list`, add the new button:

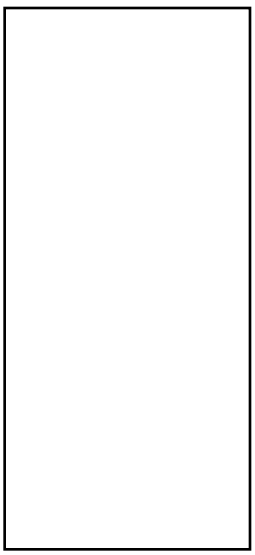
```
Button("Add") {
  selectedCard = store.addCard()
}
```

When you tap the **Add** button, you call your new `addCard()` method in `store`. This adds a new `Card` to the store's cards array and saves the card file to disk.

By changing `selectedCard`, you trigger `fullScreenCover(item:)`, which displays the new card.

➤ Open your app's **Documents** folder in Finder and remove all the files from the folder. This will reset your app's data.

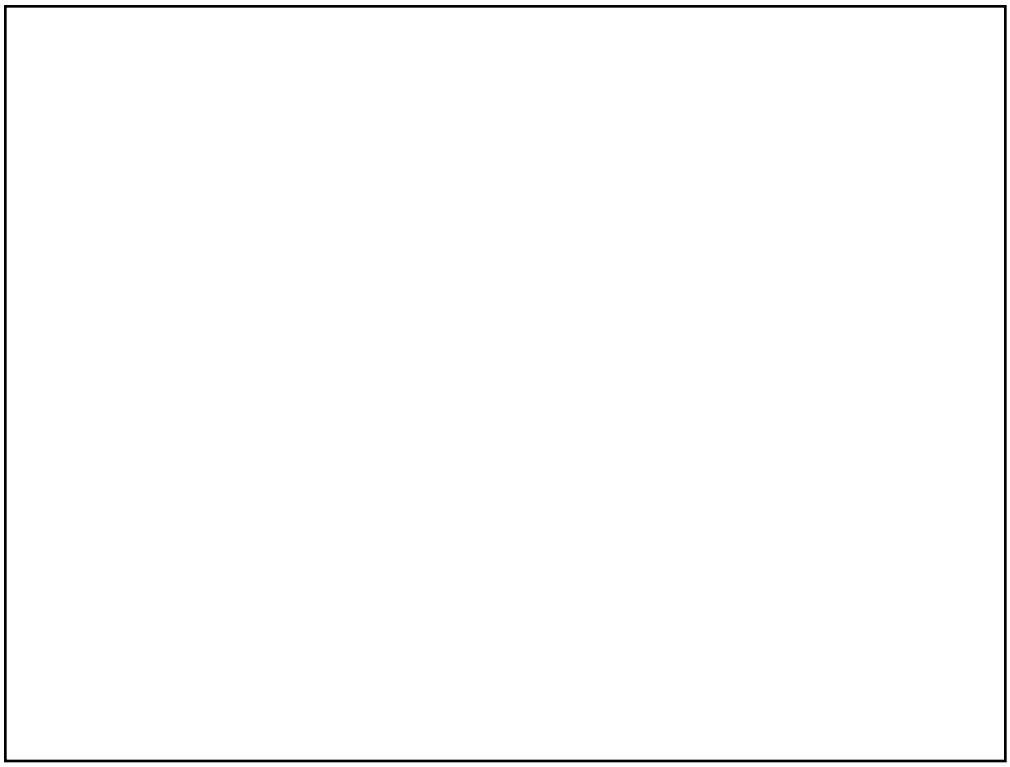
➤ Build and run your app.



No app data

➤ Tap **Add** to add a new card. The background color is random and won't be saved until you complete the challenge at the end of the chapter.

A new **.rwcard** file will appear in your app's **Documents** folder. Add a couple of photos and stickers to the card. These will get saved right away. Move them around and tap **Done** to save the transforms. Your new card will show in the list of cards. When you re-run your app, any cards will appear just as you created them (but with a yellow background).



Adding elements to the card

Your app is in great shape now. You've implement all the **CRUD** operations and you're saving the data between sessions.

Challenges

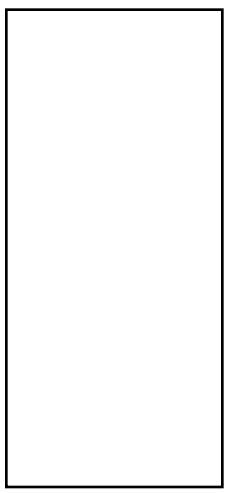
Challenge 1: Save the Background Color

One of the properties not being stored is the card's background color, and your first challenge is to fix this. Instead of making `Color` `Codable`, you'll store the color data in `CGFloats`. In **ColorExtensions.swift**, there are two methods to help you:

- `colorComponents()` separates a `Color` into red, green, blue and alpha components. These are returned in an array of four `CGFloats`. `CGFloat` conforms to `Codable`, so you'll be able to store the color.
- `color(components:)` is a static method which initializes a `Color` from four `CGFloats`. This is commonly called a **factory method**, as you're creating a new instance.

In **Card.swift**, encode and decode the background color using these two methods.

Before testing your solution, remove all files from the app's **Documents** folder. When you change the format of the file, it becomes unreadable. When adding properties to files in an app that you've already released, you would have to take this into account, as you wouldn't want to lose your users' data. Generally you'd store a version number in your files and have a startup method that does an upgrade of files if the data is an older version.



Card background colors saved

Challenge 2: Save Text Data

This is a super-challenging challenge that will test your knowledge of the previous chapters too. You're going to save text elements into your `Card` **.rwcard** file. Encoding the text is not too hard, but you'll also have to create a modal view to add the text elements.

1. Create a new SwiftUI View file for your text entry modal. You will need to hold a `TextElement` binding property sent from `CardToolbar` to hold the text data temporarily, just as you've done for your other picker modals with `frameIndex` and `stickerImage`. This time, though, in `CardToolbar`, instantiate the state property and don't make `textElement` an optional. You can check whether text is empty with `if textElement.text.isEmpty`.

2. In your new modal view file, add an environment `dismiss` property as you did for your other modals and replace `body` contents with:

```
let onCommit = {
  dismiss()
}
TextField(
  "Enter text", text: $textElement.text, onCommit: onCommit)
.padding(20)
```

The text field will show a placeholder and update the text `String` with the user's input. When the user presses **Return**, the modal will close.

3. In **CardToolbar.swift**, change `sheet(item:)` to add the text picker modal just as you did the other modals. In `onDisappear(_:)`, if the text is not empty, add the new text element to the card.

4. Make `TextElement` `Codable` so that you save and restore the text with the card.
5. In `Card`'s `Codable` extension, make sure that you encode and decode the text elements with the image elements.

Text entry and added text

This looks like a substantial challenge, but each step is one that you have done before, so you shouldn't have any trouble. Learning how to add features to an existing app is an important skill. If you do have any difficulties, then take a look at the project in this chapter's **challenge** folder.

When you finish this challenge, give yourself a big pat on the back, as you've now created an app that has a complex UI and persists data each time you run the app. This is the meat and vegetables of app development. The following chapters cover making your app look gorgeous and round off the meal with an exotic dessert.

Key Points

- Saving data is the most important feature of an app. Almost all apps save some kind of data, and you should ensure that you save it reliably and consistently. Make it as flexible as you can, so you can add more features to your app later.
- `ScenePhase` is useful to determine what state your app is in. Don't try doing extensive operations when your app is inactive or in the background as the operating system can kill your app at any time if it needs the memory.
- JSON format is a standard for transmitting data over the internet. It's easy to read and, when you provide encoders and decoders, you can store almost anything in a JSON file.
- `Codable` encompasses both decoding and encoding. You can extend this task and format your data any way you like.

Chapter 20: Delightful UX — Layout

By Caroline Begbie

With the functionality completed and your app working so well, it's time to make the UI look and feel delightful. Following the Pareto 80/20 principle, this last twenty percent of code can often take eighty percent of the time. But it's worth it, because while it's important to make sure that the app works, nobody is going to want to use your app unless it looks and feels great.

The Starter app

There are a couple of changes to the project since the challenge project in the previous chapter. These are the major changes:

- The asset catalog has more pleasing random colors to use for backgrounds, as well as other colors that you'll use in these last chapters. **ColorExtensions.swift** now uses these colors.
- `ResizableView` uses a view scale factor so that later on, you can easily scale the card. The default scale is 1, so you won't notice it to start with.
- `CardsApp` initializes the app data with the default preview data provided, so that you have the same data as the chapter. Remember to change to `@StateObject var store = CardStore()` in **CardsApp.swift** when you want to start saving your own cards again.
- Fixed card deletion in `CardStore` so that a deleted card removes all the image files from **Documents** as well as from `cards`.
- **Settings.swift** contains a method you'll use to complete the challenge.

This is the view hierarchy of the app you've created so far.

View Hierarchy

As you can see, it's very modular. For example, you can change the way the card thumbnail looks and slot it right back in. You can easily add buttons to the toolbar and add a corresponding modal.

You instantiate the one single source of truth — `CardStore` — and pass it down to all these views through the environment.

Designing the Cards List

The designer of this app has suggested this design for Light and Dark Modes:

App Design

When there are no cards, the user will see a large add button. There will also be a wide **Create New** button at the bottom. This is the design that you'll attempt to duplicate.

Adding the List Background Color

➤ Before adding anything to the project, build and run the app in Simulator and choose **Device ▸ Erase All Contents and Settings...**.

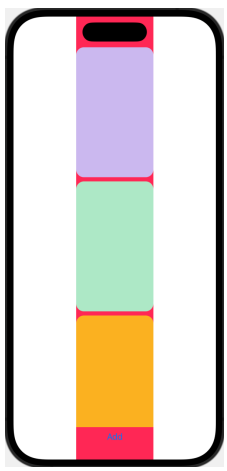
This will delete all the data you have so far created for the app. For the moment, you'll use the default data provided with the app.

➤ Open **CardsListView.swift** and add a modifier to the top `VStack`:

```
.background(
  Color("background")
    .ignoresSafeArea())
```

This will use a color from the asset catalog named **background** for the background color. This is defined as light gray for light appearance and dark gray for dark appearance. By using default parameters for `ignoresSafeArea(_:edges:)`, you ensure the background covers all the screen.

➤ Preview the view. In this image, the background color is pink for clarity; yours will be light gray.



Background Color not showing up

Instead of the background color showing across the whole view, even though you're ignoring all the safe areas, the background color is only showing up in the area of the scroll view. This is because `VStack` only takes up as much space as required by its child views.

Layout

Skills you'll learn in this section: control view layout

It's time to take a deeper look at how SwiftUI handles view layout.Most of the time, SwiftUI views lay themselves out and look great, and you don't have to think about the layout at all. But then comes the time where you want exact positioning, or a view isn't behaving the way that you thought it would, and you might start fighting the system. Once you understand layout and treat it logically, then it all becomes much easier.

Layout starts from the top of the view hierarchy. The parent view tells its children, "I propose this size". Each child then takes as much room as it needs within the parent's available space and tells the parent "I only need this size". This continues all the way down the view hierarchy. The parent then resizes itself to the size of its child views.

➤ Create a new SwiftUI View file named **LayoutView.swift** to experiment with various layouts.

➤ In `LayoutView_Previews`, add a new modifier to `LayoutView`:

```
.previewLayout(.fixed(width: 500, height: 300))
```

This gives a fixed size to the preview of 500 x 300.

➤ In the canvas, switch from **Live** to **Selectable**, to see the correct view preview size.

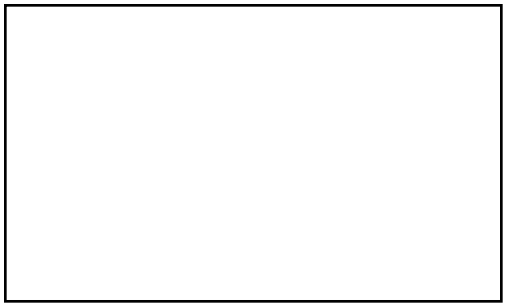


Selectable

➤ In `LayoutView`, add a new modifier to `Text`:

```
.background(Color.red)
```

In the canvas, the red color shows how much space the `Text` view takes up on screen.



Text with red background

There are three views in the view tree hierarchy here:

```
LayoutView ➤ Text (modified) ➤ Red
```

`LayoutView` has a fixed size of 500 by 300 points. `Text` takes up the amount of space needed for the letters in the assigned font size. `Color` is a bit different. It's a **late binding token**, which means that the size is assigned at the last moment.

A `Color` view fills the whole space of its parent.

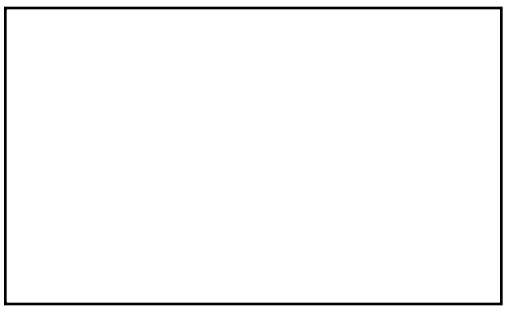


Laying out views

➤ Change `LayoutView` to:

```
struct LayoutView: View {
  var body: some View {
    HStack {
      Text("Hello, World!")
        .background(Color.red)
      Text("Hello, World!")
        .padding()
        .background(Color.red)
    }
    .background(Color.gray)
  }
}
```

Here you create a horizontal stack with two `Text` views. The second `Text` has padding.



Laying out views

The view tree is now:

```
LayoutView ➤ HStack ➤ Text (modified) ➤ Red
                    ➤ Text (modified) ➤ Padding (modified) ➤ Red
                    ➤ Gray
```

`LayoutView` still has the fixed size of 500 by 300 points. `HStack` presents 500 by 300 points to its children. The first `Text` returns the space it needs, but the second text has a padding modifier, so returns its space plus the padding. `HStack` then takes up only the space required by its two child views plus `HStack`'s default padding between the two child views. `HStack`'s gray background color fills out the space taken up by `HStack` underneath the two `Text` views.

Every time you add a modifier, you create a new layer in the view hierarchy. But don't worry about the efficiency of this — SwiftUI views are lightweight and adding new views is incredibly fast.

The Frame Modifier

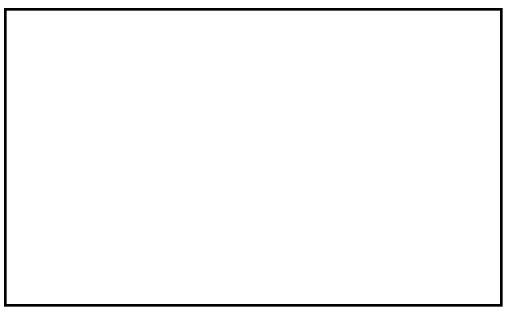
In previous code, you changed the default size of views using `frame(width:height:alignment:)`, giving absolute values to `width` and `height`.

When you want to lay out views relative to parent view sizes, you can specify minimum and maximum widths and heights using `frame(minWidth:idealWidth:maxWidth:minHeight:idealHeight:maxHeight:alignment:)`.

➤ Before `.background(Color.gray)`, add this:

```
.frame(maxWidth: .infinity)
```

The `HStack` now tells its parent that it wants the maximum available width, so `HStack`, with its gray color, expands to the whole width of the view.



Maximum width

Remember your earlier problem with the background color only taking up the width of the `ScrollView`? Specifying a frame with `maxWidth` and `maxHeight` of `infinity` would be one way of filling up the entire available background.

> **Note**: If you want to visualize how much space views take up, try adding `.background(Color.red.clipped())` as a modifier to the various views. You clip the color, as the background can sometimes render outside the view frame.

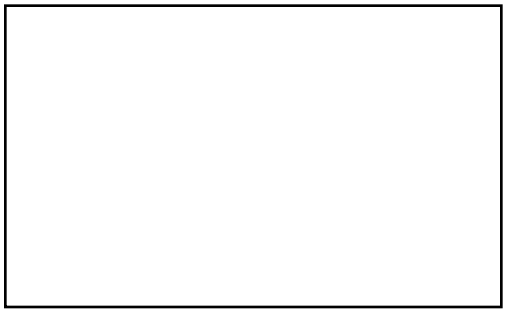
Views That use Their Parents' Size

Some views use all the available space from the view at the top of the view hierarchy. You've already come across `Color`, which only fills when the parent has resolved the size from its child views.

`Lazy` views fill the vertical or horizontal space, depending on the type of view, of the top level view. These views are finalized late, as their content is only loaded when it is necessary.

➤ In `LayoutView`, remove `.frame(maxWidth: .infinity)` and change `HStack` to:

```
LazyHStack
```



LazyHStack fills the container vertically

Because the stack can't determine the height of items that might be loaded later, it takes the vertical size of the parent container, and the color fills that area. Similarly, `LazyVStack` and `LazyVGrid` fill the horizontal size of the parent container view.

➤ Undo `LazyHStack` back to `HStack`.

Later in the chapter, you'll explore `GeometryReader`, which takes up the entire available space of its parent and returns the size in points. Use `GeometryReader` as a last resort as there are usually other ways to achieve a fluid, animatable layout.

Adding a Lazy Grid View

Skills you'll learn in this section: shadows; accent color

Instead of showing one column of scrolling cards, you'll add a `LazyVGrid` to show the cards in multiple columns. This should be adaptive depending on the device's current display width. The `LazyVGrid` expands horizontally to fit the parent's size, so you'll coincidentally solve the problem of the background color that you had earlier.

➤ Open **CardsListView.swift** and add a new property to `CardsListView`:

```
var columns: [GridItem] {
  [
    GridItem(.adaptive(
      minimum: Settings.thumbnailSize.width))
  ]
}
```

This returns an array of `GridItem` — in this case, with one element — you can use this to tell the `LazyVGrid` the size and position of each row. This `GridItem` is adaptive, which means the grid will fit as many items as possible with the minimum size provided.

➤ In `list`, replace the `VStack` inside `ScrollView` with:

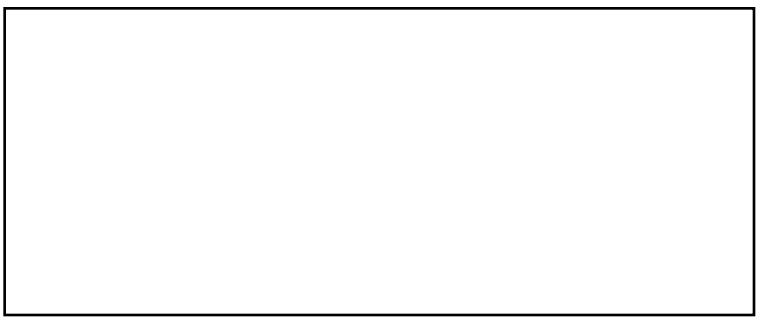
```
LazyVGrid(columns: columns, spacing: 30)
```

You now have a flexible grid with vertical spacing of 30 points.

➤ Add some padding to `ScrollView`:

```
.padding(.top, 20)
```

In Live Preview, the background color now fills the entire screen. Check out the orientation variants to see how the number of columns changes.



Orientation variants

Setting the Card Thumbnail Size

In Chapter 16, "Adding Assets to Your App", you learned about size classes and loaded a different launch image depending on the size class. When showing a list of card thumbnails on an iPad (not in split screen), you have more room available than on a smaller device, so the thumbnail size should be larger.

However, when you change the layout to split screen, the thumbnail should size smaller. You'll test for the size of the device by using the compact or regular layout.

Currently, you set the size of the thumbnail in `CardThumbnail`, but since the number of columns in `CardsListView` depends on the size of the thumbnail, you'll size the thumbnail in `CardsListView`.

➤ Still in **CardsListView.swift**, add these new properties to `CardsListView`:

```
@Environment(\.horizontalSizeClass) var horizontalSizeClass
@Environment(\.verticalSizeClass) var verticalSizeClass
```

These environment properties contain whether the size class is `compact` or `regular` so that you'll know how much space you have available.

➤ Add another new property to `CardsListView`:

```
var thumbnailSize: CGSize {
  var scale: CGFloat = 1
  if verticalSizeClass == .regular,
    horizontalSizeClass == .regular {
    scale = 1.5
  }
  return Settings.thumbnailSize * scale
}
```

If both size classes are regular, you can show a larger size thumbnail.

➤ Change the grid items in `columns` to:

```
GridItem(.adaptive(
  minimum: thumbnailSize.width))
```

➤ In `list`, add this modifier to `CardThumbnail(card:)`:

```
.frame(
  width: thumbnailSize.width,
  height: thumbnailSize.height)
```

You use your new conditional size for the thumbnail and for the column layout.

➤ Open **CardThumbnail.swift**, cut the frame modifier from `RoundedRectangle(cornerRadius:)` and paste it to modify `CardThumbnail(card:)` in `CardThumbnail_Previews`.

➤ Change the run destination to iPad and build and run the app. On iPad with split screen, you can check both compact and regular sizes.

➤ Add a split screen with Safari and change the size of the split screen. Check out the changing size of the thumbnails.

Thumbnails resize according to size class

Creating the Button for a new Card

You'll now place a button at the foot of the screen to create a new card.

➤ Open **CardsListView.swift** and, in `CardsListView`, create a new button property:

```
var createButton: some View {
// 1
  Button {
    selectedCard = store.addCard()
  } label: {
    Label("Create New", systemImage: "plus")
  }
  .font(.system(size: 16, weight: .bold))
// 2
  .frame(maxWidth: .infinity)
  .padding([.top, .bottom], 10)
// 3
  .background(Color("barColor"))
}
```

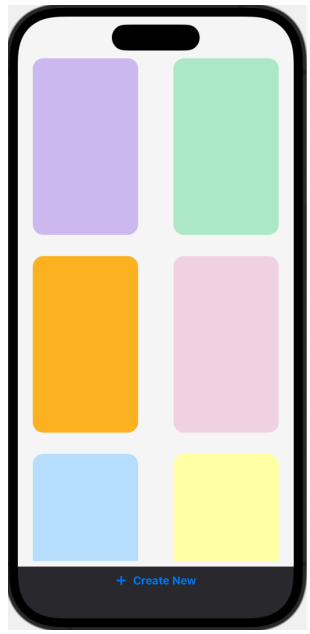
Going through this code:

1. Create a simple button using a `Label` format so that you can specify a system image. When tapped, you create a new card and assign it to `selectedCard`. When `selectedCard` changes, `SingleCardView` will show.
2. The button stretches all the way across the screen, less the padding.
3. The background color is in the asset catalog. You'll customize the button text color shortly.

In `body`, replace the current **Add** button with:

```
createButton
```

➤ Test your new button in Live Preview.



Create button

The button code has a "gotcha". Although the button frame extends all the way across the screen, only the text is tappable.

➤ In `createButton`, move `frame(maxWidth: .infinity)` from being a modifier on `Button` to a modifier on `Label`:

```
Button {
  selectedCard = store.addCard()
} label: {
  Label("Create New", systemImage: "plus")
    .frame(maxWidth: .infinity)
}
...
```

The button looks the same but is tappable all the way across.

Outlining the Cards

➤ Open **CardThumbnail.swift**.

An alternative to using a `RoundedRectangle` is to use the card background color as the view.

➤ Change `RoundedRectangle(cornerRadius:)` and `foregroundColor(_:)` to:

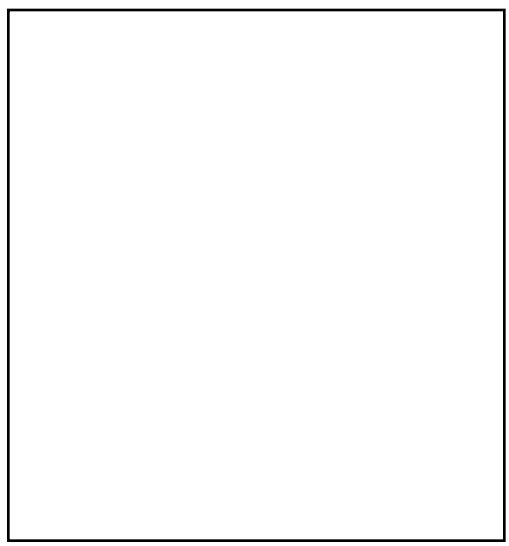
```
card.backgroundColor
  .cornerRadius(10)
```

This changes the corner radius to match the design, but otherwise produces the same result as before.

➤ Add a modifier to `card.background`, after `cornerRadius(10)`:

```
.shadow(
  color: Color("shadow-color"),
  radius: 3,
  x: 0.0,
  y: 0.0)
```

Here you add a shadow with your specified color and a radius of `3`. With the `x` and `y` positions both being zero, the shadow will be three points all around the view.



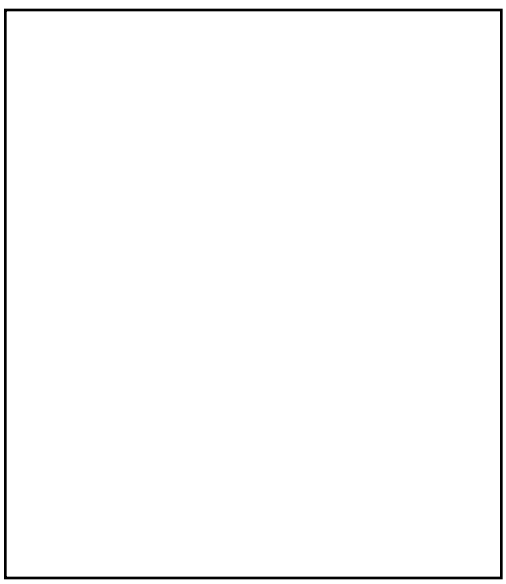
Outline Colors

This is a very subtle outline color, just to raise up the cards from the background slightly, but if your designer tells you to add it, trust the designer. :]

➤ Temporarily change `card.backgroundColor` to:

```
Color(UIColor.systemBackground)
```

In the Variants preview, the card color is now the same as the screen's background color and you'll be able to see the shadow.



Outline Colors with temporary card color

➤ Change `Color(UIColor.systemBackground)` back to:

```
card.backgroundColor
```

This restores your card's background color.

Adding a Button When There Are No Cards

When users first open your app, they need some prompting to add a new card. As well as the **Create New** button, you'll add a single card with a plus sign.

➤ Open **CardsListView.swift** and create the new initial view:

```
var initialView: some View {
  VStack {
    Spacer()
      let card = Card(
        backgroundColor: Color(uiColor: .systemBackground))
    ZStack {
      CardThumbnail(card: card)
      Image(systemName: "plus.circle.fill")
        .font(.largeTitle)
    }
    .frame(
      width: thumbnailSize.width * 1.2,
      height: thumbnailSize.height * 1.2)
    .onTapGesture {
      selectedCard = store.addCard()
    }
    Spacer()
  }
}
```

This creates a new temporary card with a plus symbol on it. With the device in Dark Mode, the card should be black, and in Light Mode, the card should be white. `Color.primary` gives black in Light Mode, and sometimes you can use `colorInvert()` for the inverse. However, this results in `some View` rather than the `Color` you need here. UIKit provides a `systemBackground` color, so you can use that instead.

Use `Spacers` to center the card in the view, keeping `createButton` at the foot of the screen.

➤ In `body`, replace `list` with:

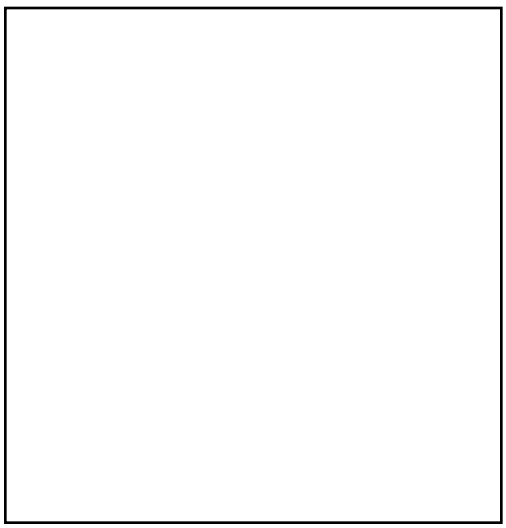
```
Group {
  if store.cards.isEmpty {
    initialView
  } else {
    list
  }
}
```

You place these two views in a `Group` so that `fullScreenCover(item:onDismiss:content:)` modifies both views.

➤ In `CardsListView_Previews`, replace `CardStore(defaultData: true)` with:

```
CardStore(defaultData: false)
```

Your new card shows up in place of the `ScrollView`, and you can either use the **Create New** button or this card to create a new card.



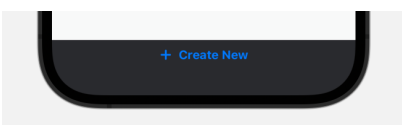
Add card prompt showing Color Scheme variants

➤ Test your card prompt in Live Preview and see that, when you add a card, the prompt no longer appears. Delete all the cards and the card prompt reappears. When you're happy that your card prompt works, in `CardsListView_Previews`, replace `CardStore(defaultData: false)` with:

```
CardStore(defaultData: true)
```

Customizing the Accent Color

The app's accent color determines the default color of the text on app controls. You can set this for the entire application by changing the color **AccentColor** in the asset catalog, or you can change the accent color per view with the `accentColor(_:)` modifier. The default is blue, which doesn't work at all well for the text button:

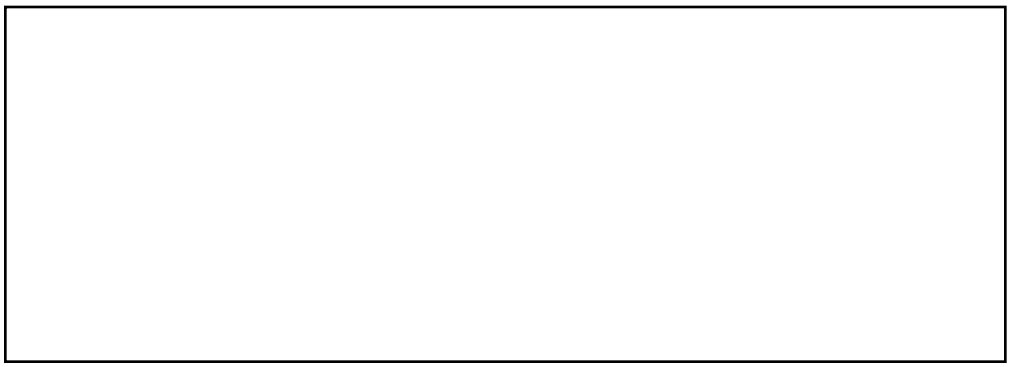


The default accent color

➤ Open **Assets.xcassets** and select **AccentColor**.

AccentColor is automatically created when you create a new project using the App template.

➤ Change the color to black for **Any Appearance** and white for **Dark Appearance**.

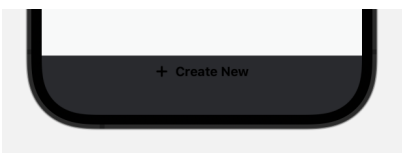


Change the accent color

This will change the default accent color of all the controls throughout the app.

➤ Open **CardsListView.swift** and preview it.

The **Create** button text is now black and doesn't show on the black bar. Black is a great color for buttons the card detail view, but not so great for this button.



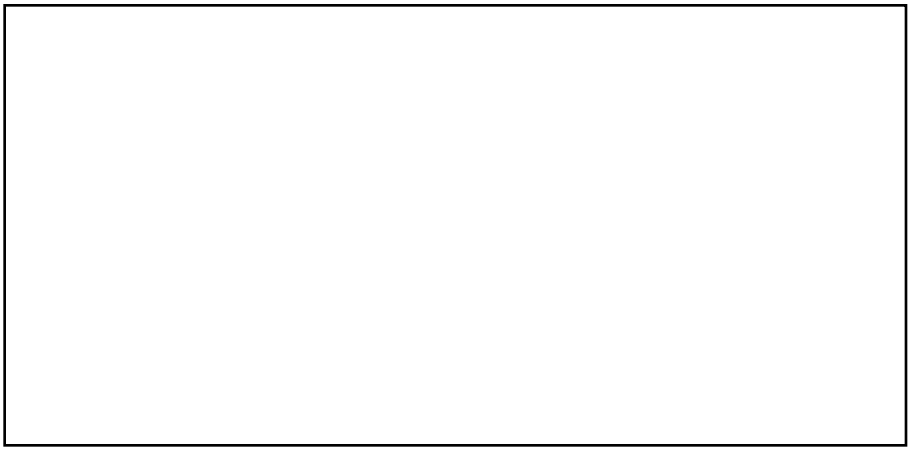
Black text

➤ In `createButton`, add a new modifier after `background(Color("barColor"))`:

```
.accentColor(.white)
```

As the button is dark in both light and dark appearances, you set the button's accent color to always be white, overriding the app's default color.

➤ Preview the color scheme variants of both `CardsListView` and `SingleCardView`.



Accent color

Throughout the app, text takes on **AccentColor** as defined in **Assets.xcassets**, except for where you specify `accentColor(_:)` on specific views.

Scaling the Card to fit the Device

> **Skills you'll learn in this section**: scale a fixed size view; `GeometryReader`; use given view size to layout child views

Currently a card takes up the full size of the screen, less the top and bottom safe areas, no matter what device or orientation you're using. This obviously doesn't work when you've created a portrait card and then turn the device to landscape.

You're going to create cards with a fixed size of 1300 by 2000. The entire card will be visible at one time, no matter the orientation, and you'll calculate the appropriate size of the card view using a geometry reader proxy size.

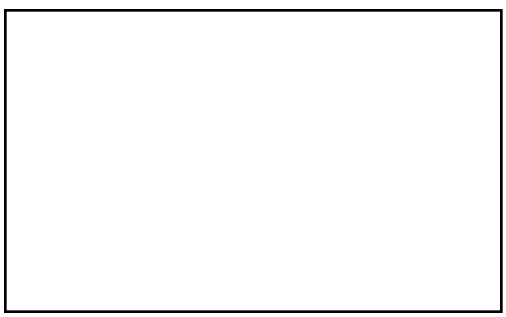
GeometryReader

`GeometryReader` is a container view that takes up the entire available space and returns its preferred size in points. Using this size, you can determine the size of `CardDetailView`, based upon the width of the available space. Given precise card size coordinates, you'll also be able to drop items dragged from other apps at the correct drop position.

➤ To see how this will work, open **LayoutView.swift**, embed `HStack` in a `GeometryReader` and give it a yellow background:

```
GeometryReader { proxy in
  HStack {
    ...
  }
  .background(Color.gray)
}
.background(Color.yellow)
```

`GeometryReader` takes up the size of the parent, in this case the whole 500 x 300 point view. It returns a value of type `GeometryProxy`, which includes a `size` property so that you can find out exactly the size of the view. You can then lay out child views using this size.



GeometryReader

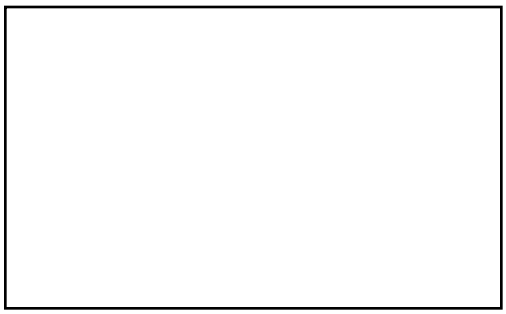
Notice that `GeometryReader` changes alignment behavior. Instead of `HStack` being centered in its parent view, it is now aligned to the top left of its parent view. You'll discover more about alignment later in this chapter.

➤ Change `HStack`'s modifiers to:

```
.frame(width: proxy.size.width * 0.8)
.background(Color.gray)
.padding(
  .leading, (proxy.size.width - proxy.size.width * 0.8) / 2)
```

`frame(width:height:alignment)` now uses a relative value of 80% of the width of the available area. If the parent view gets larger, for example on device rotation, `proxy.size` will update and refresh the view. The view will resize to 80% of the new parent size.

To center `HStack`, you calculate the leading padding, using the geometry proxy width.



GeometryProxy size

Notice the order of the modifiers. If you change the order of any one of these, you'll get a different result. Before filling with color, you must set the size of the view. If you calculate the padding before filling with gray, then you'll center the text views but not the background gray color.

Now, you'll put this knowledge into action.

➤ Open **SingleCardView.swift** and, in `body`, embed `CardDetailView(card:)` in a `GeometryReader`:

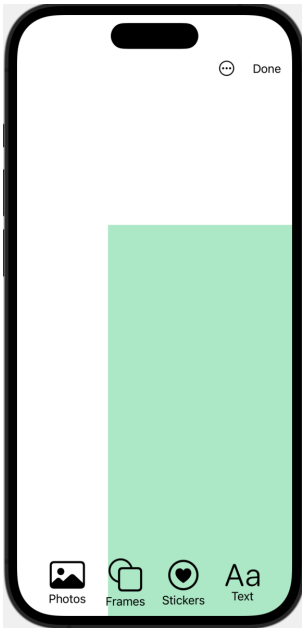
```
var body: some View {
  NavigationStack {
    GeometryReader { proxy in
      CardDetailView(card: $card)
        .modifier(...
```

You can now calculate the frame of `CardDetailView` using the geometry reader proxy size.

➤ Temporarily add these modifiers to `CardDetailView(card:)`:

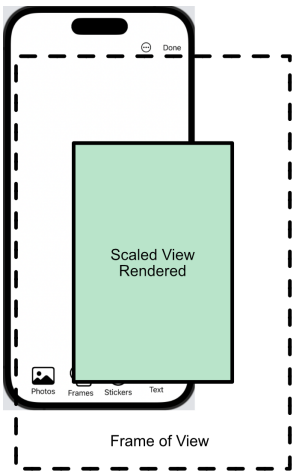
```
.frame(
  width: Settings.cardSize.width,
  height: Settings.cardSize.height)
.scaleEffect(0.8)
```

You set the card frame to the final card size and scale it by 80%.



Card scaled to 80%

Knowing that views lose their alignment under `GeometryReader`, you might be surprised to see that the card now appears offset. However, it's only the rendered view that is showing up. `CardDetailView`'s frame is still taking up 1300 x 2000.



View frame stays original size

You have to take into account the size of `CardDetailView` on any size device, so it's easier to convert the frame to the correct size than use `scaleEffect(_:anchor:)`. However, this does mean that in views further down the view hierarchy, you'll have to take into account how much `CardDetailView` is scaled.

➤ Open **Settings.swift** and add these new methods to `Settings`:

```
static func calculateSize(_ size: CGSize) -> CGSize {
  var newSize = size
  let ratio =
    Settings.cardSize.width / Settings.cardSize.height

  if size.width < size.height {
    newSize.height = min(size.height, newSize.width / ratio)
    newSize.width = min(size.width, newSize.height * ratio)
  } else {
    newSize.width = min(size.width, newSize.height * ratio)
    newSize.height = min(size.height, newSize.width / ratio)
  }
  return newSize
}

static func calculateScale(_ size: CGSize) -> CGFloat {
  let newSize = calculateSize(size)
  return newSize.width / Settings.cardSize.width
}
```

These methods calculate the size and scale of a view with the correct aspect ratio using a given size. This size comes from the view's' `GeometryReader`'s `GeometryProxy`.

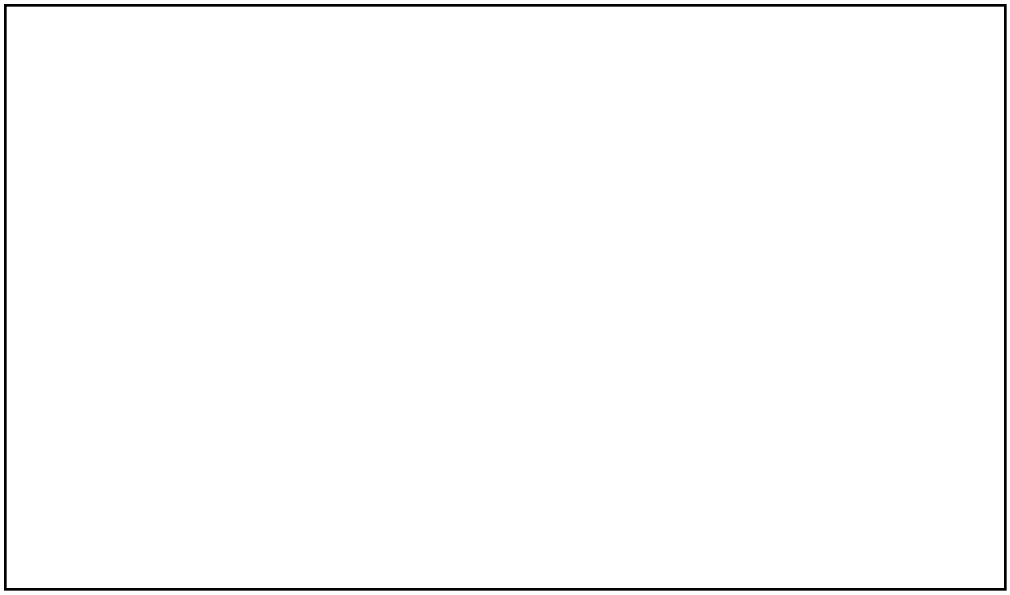
➤ Open **SingleCardView.swift** and replace the existing frame and scale modifiers on `CardDetailView(card:)` with these:

```
// 1
.frame(
  width: Settings.calculateSize(proxy.size).width,
  height: Settings.calculateSize(proxy.size).height)
// 2
.clipped()
// 3
.frame(maxWidth: .infinity, maxHeight: .infinity)
```

There's a lot of layout going on in these few modifiers:

1. Calculate the size of the card view given the available space.
2. The background color will spill out of the frame, so clip it.
3. Make sure that `CardDetailView` takes up all of the space available to it. This will center the card view in the geometry reader.

In portrait mode, the card probably looks fine in the canvas. The problem comes when you view the card in landscape, and find that the elements aren't scaling properly.



Incorrect scaling

➤ In the **Views** group, open **ResizableView.swift**.

Notice that the new changes in this file adjust all the offsets and sizes to be scaled to `viewScale`. This defaults to `1`, so you don't have to specify a view scale if you don't want to.

➤ Open **CardDetailView.swift** and add a new property:

```
var viewScale: CGFloat = 1
```

You'll pass in the calculated view scale from `SingleCardView`.

➤ Locate `.resizableView(transform: $element.transform)` and replace it with:

```
.resizableView(
  transform: $element.transform,
  viewScale: viewScale)
```

When `ResizableView` transforms the size of each element, it now uses the new view scale.

➤ Open **SingleCardView.swift** and replace `CardDetailView(card: $card)` with:

```
CardDetailView(
  card: $card,
  viewScale: Settings.calculateScale(proxy.size))
```

You calculate the scale of the view and pass it down the hierarchy.

With the view scaled, the element's default size will be too small.

➤ Open **Settings.swift** and change `defaultElementSize` to:

```
static let defaultElementSize =
  CGSize(width: 800, height: 800)
```

➤ In **CardsApp.swift**, change `@StateObject var store = CardStore(defaultData: true)` to:

```
@StateObject var store = CardStore(defaultData: false)
```

➤ Build and run on various devices and orientations and check out your newly scaled card view. The card stays in portrait and is fixed to a scaled 1300 by 2000 size. The elements are also scaled and you can manipulate them in the same way as you did before.

Scaled card in portrait and landscape

Alignment

Skills you'll learn in this section: stack alignment

The final subject in layout that you'll cover is alignment. Take another look at the previous image. Currently, the images in your toolbar buttons are different sizes which misaligns the button text. Your attention-to-detail gene should have been crying inwardly because of this.

`VStack(alignment:spacing:)` and `HStack(alignment:spacing:)` have optional alignment parameters.

Stack Alignment

With an `HStack`, you describe how child views should align vertically, and with a `VStack`, you describe the horizontal view alignment

➤ Open **BottomToolbar.swift** and preview the view.

> **Xcode Tip**: Don't forget your keyboard shortcut **Shift-Command-O** to quickly open a file by name. To see the current file in the Project navigator, press **Shift-Command-J**.



Misaligned preview of the toolbar buttons

Currently, in `BottomToolbar`, your toolbar buttons are in a center aligned `HStack`. This means that the `ToolbarButtons`, which consist of a `VStack` with an `Image` above and `Text` below, are all center aligned.

➤ In `BottomToolbar`, change `HStack {` to:

```
HStack(alignment: .top) {
```

This aligns the buttons at the top of the `HStack`.



Top aligned buttons

➤ Now try bottom alignment. Change the alignment to:

```
HStack(alignment: .bottom) {
```

This is the best result as all the text is now aligned.



Bottom aligned buttons

Challenges

Challenge 1: Resize the Bottom Toolbar Icons

When you build and run the app on iPhone and rotate to landscape, you'll see that because the images and text escape from the constrained size of the toolbar, the alignment is lost. In addition, the home bar covers the text.



Escaping buttons

For your challenge, you'll check the size class of the device use a different icon view for each size class. The compact size class will only show the image, whereas the regular size class will show both image and text.

To achieve this:

1. In **BottomToolbar.swift**, in `ToolbarButton`, add a `regularView` that shows the image and text and a `compactView` that only shows the image. You'll construct these views in methods that take in the image name and the text, if necessary.
2. Use the environment's vertical size class to determine whether to show either `regularView` or `compactView`.

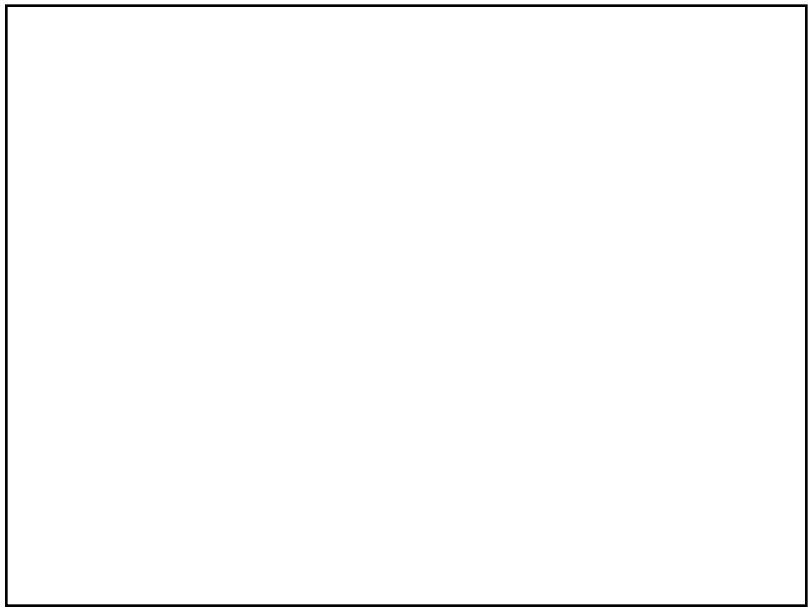
Toolbar view dependent on size class

Challenge 2: Drag and Drop to the Correct Offset

In Chapter 17, "Adding Photos to Your App", you implemented drag and drop. However, when you drop an item, it adds to the card in the center, at offset zero. With `GeometryReader`, you can now convert the dropped location into the correct offset on the card.

`Settings.calculateDropOffset(proxy:location:)` returns the offset calculated from the geometry proxy and the drop location. Use this method in **CardDetailView.swift** to drop items at the correct location. You'll need to pass the geometry proxy from `SingleCardView` to `CardDetailView`. You'll also need to amend the methods where you add the element to include the offset.

Try out drag and drop on iPad, and you have an infinite number of Google images to decorate your card.



Drag and Drop

Key Points

- Even though your app works, you're not finished until your app is fun to use. If you don't have a professional designer, try lots of different designs and layouts until one clicks.
- Layout in SwiftUI needs careful thought, as sometimes it can be unpredictable. The golden rule is that views take their size from their children.
- `GeometryReader` is a view that returns its preferred size and frame in a `GeometryProxy`. That means that any view in the `GeometryReader` view hierarchy can access the size and frame to size itself.
- Stacks have alignment capabilities. If these aren't enough, you can create your own custom alignments, too. The Apple video, Building Custom Views with SwiftUI (https://apple.co/39uamSx), examines SwiftUI's layout system in depth.

Chapter 21: Delightful UX — Final Touches

By Caroline Begbie

An iOS app is not complete without some snazzy animation. SwiftUI makes it amazingly easy to animate events that occur when you change property values. Transition animations are a breeze.

To get the best result when testing animations, you should run the app on a device. Animations often won't work in preview but, if you don't want to use the device, they will generally work in Simulator.

The Starter Project

➤ Open the starter project for this chapter.

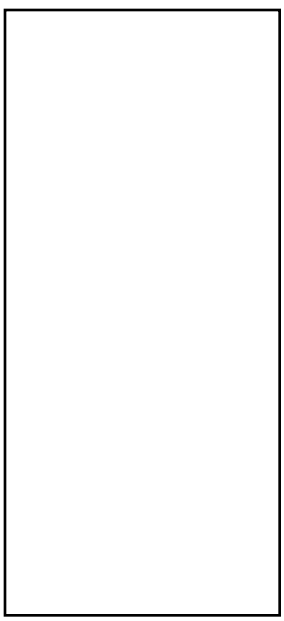
The project has an additional group called **Supporting Code**. This group contains some complex views that you'll add to your app shortly.

Animated Splash Screen

> **Skills you'll learn in this section:** set up properties for animation

Sometimes in a more complex app, after showing the launch screen, your app will take a few seconds to do all the loading housekeeping. To prevent the UI from appearing to stall, the app can perform an animation to distract the user. Apps such as Twitter and Uber use animation to reflect their branding.

You'll create an animated splash screen where the letters **C-A-R-D-S** will drop down from the top and, when that animation is complete, the animation view will slide to the main cards view.



Final animation

➤ In the **Cards** group, under **CardsApp.swift**, create two new SwiftUI View files named **AppLoadingView.swift** and **SplashScreen.swift**.

➤ Open **AppLoadingView.swift**. This view will determine whether you're showing the animation or not.

➤ Create a new property in `AppLoadingView`:

```
@State private var showSplash = true
```

➤ Change `body` to:

```
var body: some View {
  if showSplash {
    SplashScreen()
      .ignoresSafeArea()
  } else {
    CardsListView()
  }
}
```

When `showSplash` is true, you'll show the splash animation, otherwise you'll show the main `CardsListView`. At the moment, you never set `showSplash` to false, so `CardsListView` will never show. Sometimes the live preview doesn't show animations correctly — or at all — so in order to see the animation on the simulator, you'll keep it this way until you perfect your splash animation.

➤ In `AppLoadingView_Previews`, add this modifier to `AppLoadingView`:

```
.environmentObject(CardStore(defaultData: true))
```

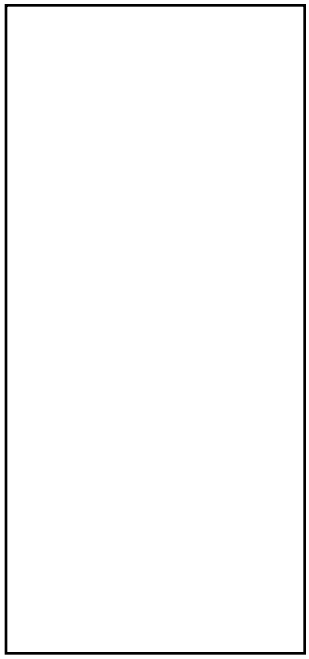
This sets up the card store so that the app will still work in Live Preview.

➤ Open **CardsApp.swift** and change `CardsListView()` to:

```
AppLoadingView()
```

You show the intermediate view which contains the splash screen.

➤ Build and run, and you'll see the default "Hello World" from `SplashScreen`.



Hello, World

➤ Open **SplashScreen.swift** and add this new method to `SplashScreen`:

```
func card(letter: String, color: String) -> some View {
  ZStack {
    RoundedRectangle(cornerRadius: 25)
      .shadow(radius: 3)
      .frame(width: 120, height: 160)
      .foregroundColor(.white)
    Text(letter)
      .fontWeight(.bold)
      .scalableText()
      .foregroundColor(Color(color))
      .frame(width: 80)
  }
}
```

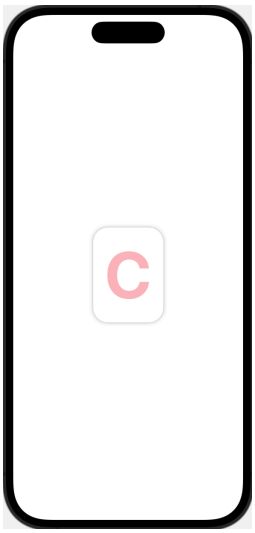
Here you create a view, with a shadow, that takes in a letter and a color.

➤ In `body`, change `Text("Hello, World!")` to:

```
card(letter: "C", color: "appColor7")
```

Here you create the view with the letter "C" and the name of a color set up in your asset catalog.

➤ Preview the view.



The card

You now have a stationary card. You'll separate out the animation movement into a new view modifier.

➤ In **SplashScreen.swift**, add a new structure:

```
private struct SplashAnimation: ViewModifier {
  @State private var animating = true
  let finalYPosition: CGFloat
  let delay: Double

  func body(content: Content) -> some View {
    content
      .offset(y: animating ? -700 : finalYPosition)
      .onAppear {
        animating = false
      }
  }
}
```

To drop the card from the top, you'll animate `content`'s offset. If `animating` is `true`, then the card's offset is off the top of the screen at -700 points. When `false`, the offset will be the final designated position. You change `animating` to `false` when the view appears.

You'll use the `delay` property shortly.

➤ At the end of **SplashScreen.swift**, add a new extension where you can improve the modifier's ease-of-use:

```
private extension View {
  func splashAnimation(
    finalYposition: CGFloat,
    delay: Double
  ) -> some View {
    modifier(SplashAnimation(
      finalYPosition: finalYposition,
      delay: delay))  }
}
```

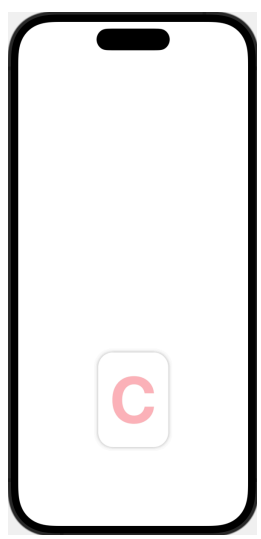
This is simply a pass through method to make your code prettier.

➤ In `SplashScreen`, replace `body` with:

```
var body: some View {
  card(letter: "C", color: "appColor7")
    .splashAnimation(finalYposition: 200, delay: 0)
}
```

Here, you call the view modifier with the final Y position of the card.

➤ Live Preview the view, and you'll see your card 200 points below the center, but not animated yet.



The card before animation

SwiftUI Animation

Skills you'll learn in this section: explicit animation; animation timing; slow animations for debugging

SwiftUI makes animating any view parameter that depends on a property incredibly easy. You simply surround the dependent property with a closure:

```
withAnimation {
  property.toggle()
}
```

And that's it! Any parameter in your entire app that depends on `property`, will animate automatically.

In `SplashAnimation`, the offset of your card depends on `animating`.

➤ In `onAppear(_:)`, change `animating = false` to:

```
withAnimation {
  animating = false
}
```

➤ Live preview the view. Your card now animates from the top and ends up at a Y offset of 200.

➤ Build and run the app in Simulator and choose **Debug ▸ Slow Animations**.

This menu option is a debug feature to slow down animations, so that you can see them properly. You'll now see a check mark next to the menu item.

➤ Build and run the app again to see the animation in slow motion.

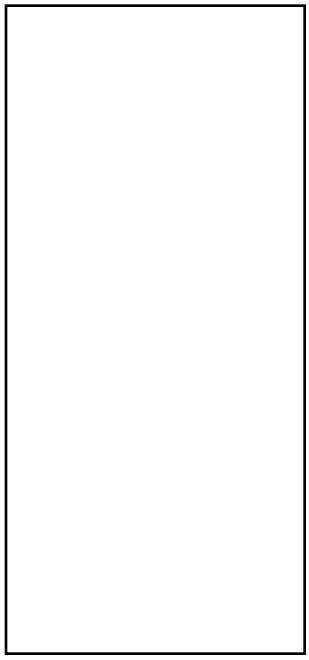
➤ In `SplashScreen`, change the contents of `body` to:

```
ZStack {
  Color("background")
    .ignoresSafeArea()
  card(letter: "S", color: "appColor1")
    .splashAnimation(finalYposition: 240, delay: 0)
  card(letter: "D", color: "appColor2")
    .splashAnimation(finalYposition: 120, delay: 0.2)
  card(letter: "R", color: "appColor3")
    .splashAnimation(finalYposition: 0, delay: 0.4)
  card(letter: "A", color: "appColor6")
```

```
      .splashAnimation(finalYposition: -120, delay: 0.6)
    card(letter: "C", color: "appColor7")
      .splashAnimation(finalYposition: -240, delay: 0.8)
  }
```

This sets up all the card letters with their final positions and colors. The `delay` parameter doesn't do anything yet, but you'll use it shortly. The background color is in your asset catalog.

➤ Live Preview or run in Simulator. In this animation, all the cards animate downwards with the same timing, which isn't aesthetically pleasing.



Animating with the same timing

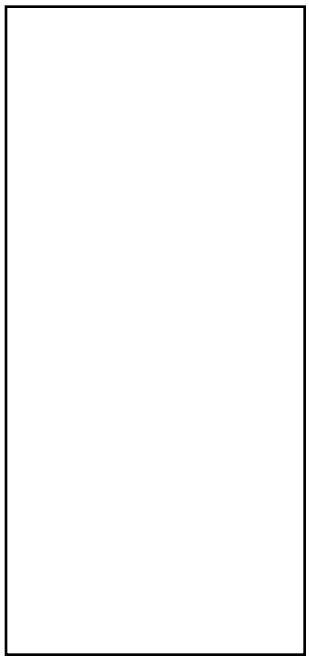
When you use `withAnimation(_:_:)`, you can specify what sort of `Animation` you want to use. You can specify the timing of the animation, the duration and whether it has a delay.

➤ In `SplashAnimation`, in `onAppear(_:)`, change `withAnimation {` to:

```
  withAnimation(Animation.default.delay(delay)) {
```

Here you're using the default animation with a delay modifier. You've already set up the cards with their delay. Each card has a 0.2 second delay greater than the previous card.

➤ Live Preview the result. With the delays, the card animation is staggered.



Animation delay

An `Animation` can have various qualities. The most common are:

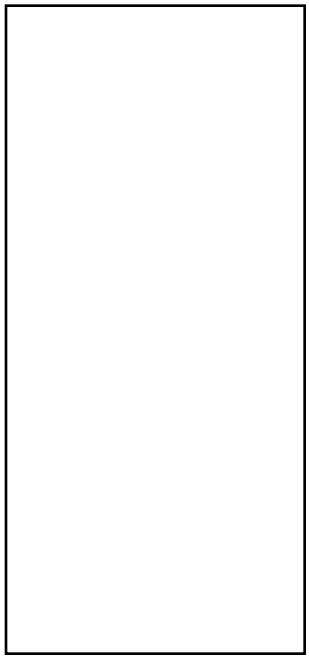
- **easeIn**: where the animation starts slowly but speeds up to the end.
- **easeOut**: where the animation starts at speed but slows down toward the end.
- **easeInOut**: a combination of the previous two.
- **linear**: where the animation speed is constant all the way through.

➤ Replace `withAnimation(Animation.default.delay(delay)) {` with:

```
withAnimation(Animation.easeOut(duration: 1.5).delay(delay)) {
```

This animation lasts for 1.5 seconds and slows gradually at the end of the animation.

➤ Live Preview first to see the animation in 1.5 seconds. Then build and run on the simulator with slow animations. You can see that the cards fall closer together toward the end of the animation.



Ease out animation timing

A more interesting `Animation` is a spring, where the view bounces like a spring. You can specify how stiff it is and how fast the bouncing stops.

➤ In `SplashAnimation`, create a new property:

```
let animation = Animation.interpolatingSpring(
  mass: 0.2,
  stiffness: 80,
  damping: 5,
  initialVelocity: 0.0)
```

This creates a spring animation.

➤ In `SplashAnimation`, replace the `withAnimation(_:_:)` closure with:

```
withAnimation(animation.delay(delay)) {
  animating = false
}
```

➤ Live Preview this, and you'll see that each card bounces as it hits its offset position. Experiment with the values of each of the animation spring properties to see how they affect the animation.

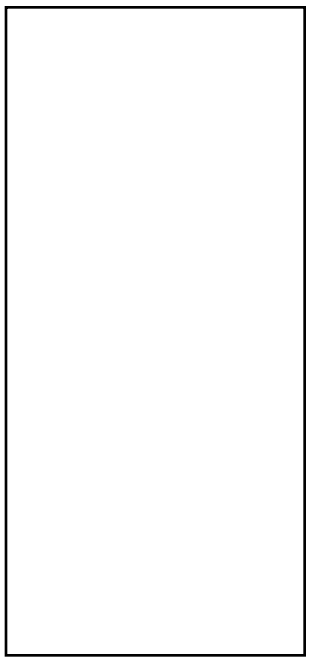
To finish off this animation, add a random rotation to each card.

➤ In `SplashAnimation`, after `offset(y:)`, add this:

```
.rotationEffect(
  animating ? .zero
    : Angle(degrees: Double.random(in: -10...10)))
```

The card animates to a random rotation between -10 and 10 degrees as it drops.

➤ Live Preview, and you'll see your final animation.



Random rotation

Explicit and Implicit Animation

Skills you'll learn in this section: implicit animation

`withAnimation(_:_:)` **explicitly** causes animations with parameters affected by the property within its closure. If you have multiple properties changing, you can explicitly change the animation for each of them.

For **implicit** animation, you animate any view with an animatable parameter automatically.

➤ In `SplashAnimation`, remove the `withAnimation(_:_:)` closure, so that `onAppear(_:)` is:

```
.onAppear {
  animating = false
}
```

This removes all animation.

➤ After the rotation effect modifier add this:

```
.animation(animation.delay(delay), value: animating)
```

This adds an implicit animation to the view. The view watches the property `animating`, and whenever `animating` changes, the view animates with the `Animation` provided.

➤ Live Preview the animation.

In this case, as you are only animating views with one animatable property, the implicit animation will appear exactly the same as the explicit animation. Explicit animations can be less code, but implicit animations give you more control by being able to animate each view depending on the animated property with different animations.

Animated Transitions

Skills you'll learn in this section: transitions

You'll now transition your splash screen to the main `CardsListView`. SwiftUI makes this easy with built-in transition effects, but you can also have complete control over how the view transitions.

➤ Open **AppLoadingView.swift**. After `ignoresSafeArea()`, add:

```
.onAppear {
  DispatchQueue.main.asyncAfter(deadline: .now() + 1.5) {
    withAnimation(.linear(duration: 5)) {
      showSplash = false
    }
  }
}
```

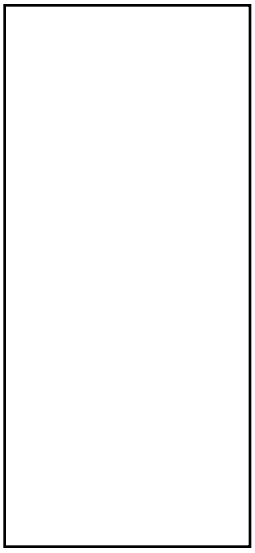
Here you set `showSplash` to `false` after a delay and use explicit animation. `showSplash` controls which view shows. You want the splash screen to show for a second or two and then transition to the main view.

Slowing the animation in Simulator doesn't work well when testing this transition, so you give the transition animation a slow duration of 5 seconds to see what's happening.

➤ In Simulator, choose **Debug ▸ Slow Animations** to turn off the slow animations.

➤ As Live Preview doesn't work well with transition animations, build and run the app.

The default transition does an opacity fade from one view to another.

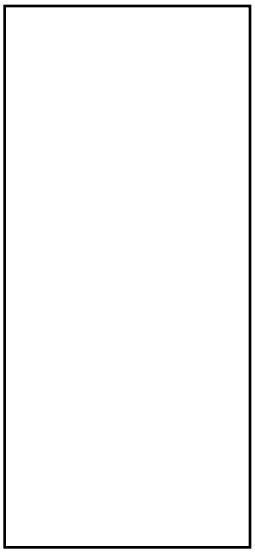


Fade transition

➤ In `AppLoadingView`, add a modifier to `CardsListView()`:

```
.transition(.slide)
```

➤ Build and run to see the slide transition over the specified five second duration.



Slide transition

As well as `opacity` and `slide`, there are a couple more automatic transitions:

- **move**: allows you to specify the edge that the new view moves in from.
- **scale**: the new view scales up.

You can also have a different transition for each direction by using:

```
.transition(.asymmetric(insertion: .slide, removal:.scale))
```

➤ Change the transition to:

```
.transition(.scale(scale: 0, anchor: .top))
```

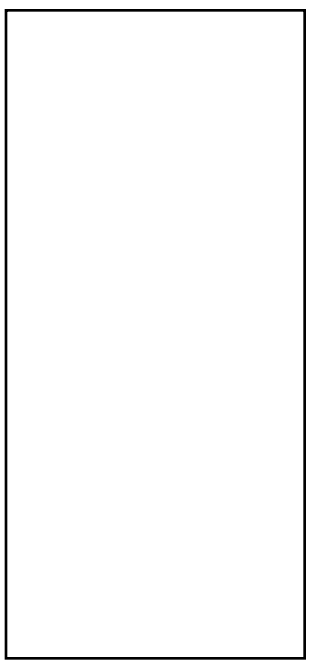
This will scale the new view in from the top.

➤ Replace `withAnimation(.linear(duration: 5)) {` with:

```
withAnimation {
```

This replaces the five second duration with the default transition duration.

➤ Build and run to see your completed splash screen animation and transition.



Scale transition

Supporting Multiple View Types

Skills you'll learn in this section: picker control

You'll add a picker view to the top of the list of cards to choose how you view the cards. You can either view them in the scrolling list or in a carousel. When you have a set of mutually exclusive values, you can use a picker control to decide between them.

There are various picker styles for mutually exclusive picking. For example, `WheelPickerStyle` shows the options in a scrollable wheel. Apple's Clock app uses a wheel picker for the Timer. You'll use a `SegmentedPickerStyle`, which is a horizontal control that holds one value at a time.

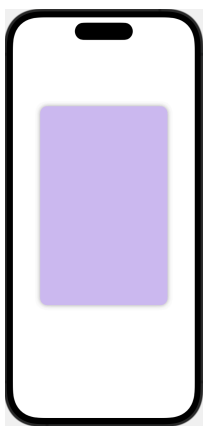


Picker with two segments

The Carousel

Carousel.swift, included in the starter project in the **Supporting Code** group, is an alternative view for listing the cards. It's a an example of a `TabView`, similar to the one you created in Section 1.

➤ Open **Carousel.swift** and Live Preview the view. Swipe to view each card.



Carousel

Each card should take up most of the device's screen, so the code uses `GeometryReader` to determine the size. There should be nothing new to you in this code. One of SwiftUI's great advantages is that you can be given a view like this, and it's an easy matter to slot it into your own code.

Adding a Picker

➤ In the **Views** group, under **CardsListView.swift**, create a new SwiftUI View file named **ListSelection.swift**.

➤ At the top of the file, after `import SwiftUI`, create a new enumeration that describes how you are viewing the list of cards:

```
enum ListState {
  case list, carousel
}
```

You'll either view the cards as a list or as a carousel.

➤ Add a new Binding to `ListSelection`:

```
@Binding var listState: ListState
```

`listState` holds the current picker selection and you'll pass this in from `CardsListView`.

➤ Update `ListSelection_Previews` to pass the initial selection of `list`:

```
static var previews: some View {
  ListSelection(listState: .constant(.list))
}
```

➤ In `ListSelection`, replace `body` with:

```
var body: some View {
  // 1
  Picker(selection: $listState, label: Text("")) {
  // 2
    Image(systemName: "square.grid.2x2.fill")
      .tag(ListState.list)
    Image(systemName: "rectangle.stack.fill")
      .tag(ListState.carousel)
  }
  // 3
  .pickerStyle(.segmented)
  .frame(width: 200)
}
```

Going through this code:

1. You use a `Picker`, passing in the selection property to update.
2. You assign SFSymbols for each option. When the user chooses an option, the `tag(_:)` modifier will update `listState` with the specified value.
3. You tell the `Picker` what picker style to use. Other picker styles include `menu` and `wheel`, which displays options in a scrollable wheel.

➤ Preview the picker.



Segmented picker

In the app, when you tap the right segment, the cards should display in the carousel; tapping the left segment will display them in the scrolling list.

➤ Open **CardsListView.swift** and add a new property to `CardsListView`:

```
@State private var listState = ListState.list
```

This property controls how you view the cards.

➤ In `body`, add the picker to `VStack`, before `Group`:

```
ListSelection(listState: $listState)
```



The picker in place

➤ Change `list` to:

```
Group {
  switch listState {
  case .list:
    list
  case .carousel:
    Carousel(selectedCard: $selectedCard)
  }
}
```

You show the scrolling list or the carousel depending on `listState`. Similar to the list, when you select a card from the carousel, changing the value of `selectedCard` will run the full screen modal.

➤ Live Preview to see the picker in action.

The two card list views

Sharing the Card

Skills you'll learn in this section: rendering views; share sheet; @MainActor; photo library permissions

At the moment, when you create a card, you're the only person who can admire it. As a final feature, you'll add sharing.

You'll create a share button on the navigation bar. On tapping this button, you'll screen capture the card. You'll then use this screenshot in the built-in Share sheet for sharing to other apps such as email or your Photos library.

➤ In the **Supporting Code** group, open **ShareCardView.swift**.

`ShareCardView` is a cut-down version of `CardDetailView`, without any of the modifiers that make the card interactive. You'll be able to render this view to an image and then share the image.

Rendering a View to an Image

➤ In the **Extensions** group, open **UIImageExtensions.swift**. Add a new extension at the end of the file:

```
extension UIImage {
  // 1
  @MainActor static func screenshot(
    card: Card,
    size: CGSize
  ) -> UIImage {
    // 2
    let cardView = ShareCardView(card: card)
    let content = cardView.content(size: size)
    // 3
    let renderer = ImageRenderer(content: content)
    // 4
    let uiImage = renderer.uiImage ?? UIImage.errorImage
    return uiImage
  }
}
```

There's a lot to unpack here:

1. `MainActor` ensures that a method is performed on the main dispatch queue. Any time you are dealing with views, you should be on the main thread. Note that any method that calls `UIImage.screenshot(card:size:)` must also be marked with `MainActor`, otherwise it will not compile.
2. Load the card into a view and extract the content. Specifying the size of the content, means that you can scale it to any size preview you want.
3. Render the image from the view. `ImageRender<Content>` initializes with a view and draws it to a `Canvas`. You can render shapes or text or any other `View` to an image.
4. Extract a `UIImage` from the rendered image, but if there's an error, use the error image in the asset catalog.

➤ In the **Single Card Views** group, open **CardToolbar.swift** and add this code after the **Done** button `ToolbarItem`:

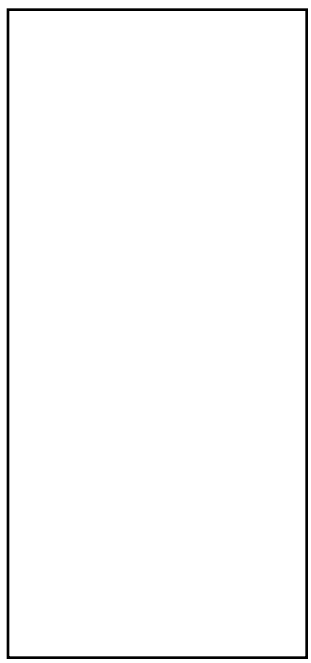
```
ToolbarItem(placement: .navigationBarLeading) {
  let uiImage = UIImage.screenshot(
    card: card,
    size: Settings.cardSize)
  let image = Image(uiImage: uiImage)
  // Add ShareLink here
}
```

You create a new toolbar item at the leading edge of the navigation bar and load an `Image` ready for sharing.

Sharing Images

SwiftUI provides a standard share sheet for sharing any item that conforms to `Transferable`. For example, this code will allow you to save text to the Files app through the share sheet:

```
ShareLink("Share Text", item: "Hello world")
```



Sharing text from your app

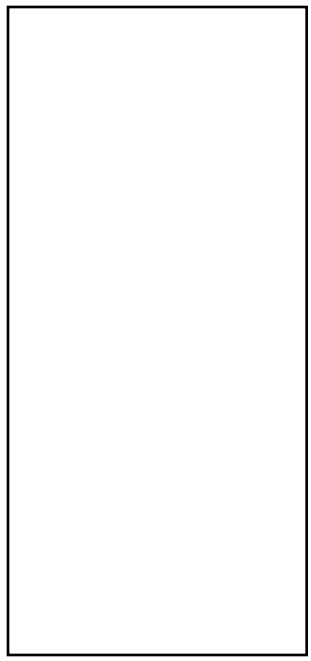
`ShareLink` will add an icon, seen here at the top left of the screen, where you can start the share. A sheet will pop up, and you'll see a preview of the text at the top left of the sheet. The share sheet determines what apps to show from the type of the item.

➤ Replace `// Add ShareLink here` with this code:

```
ShareLink(
  item: image,
  preview: SharePreview(
    "Card",
    image: image)) {
      Image(systemName: "square.and.arrow.up")
}
```

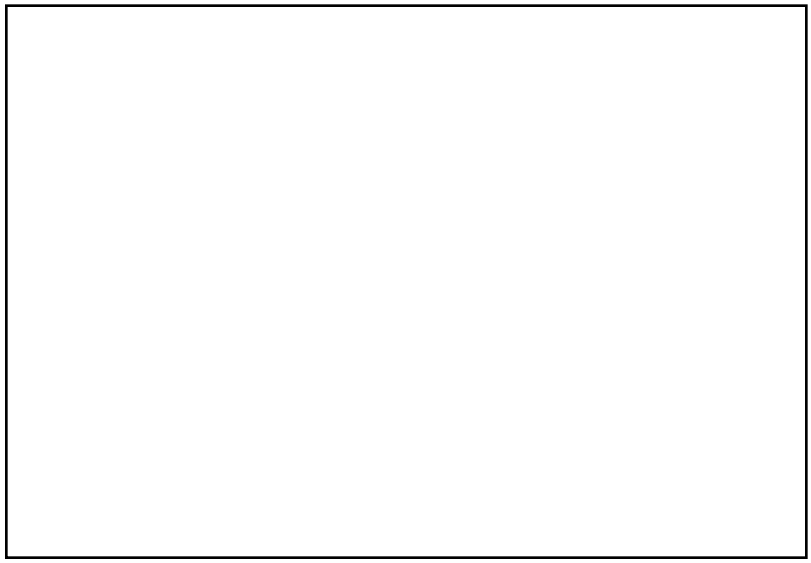
Here you use a longer `ShareLink` initializer. In place of text, you share your screen-capture image. You create your own preview image and provide a custom icon.

➤ Build and run your app in Simulator. Open the first card and tap the share icon at the top left. Pull up the sheet to see where you can share the card.



Sharing your card from your app

➤ As no option appears to save your card to Photos, tap **Save to Files**, then **Save**, and then open the Files app in Simulator. In the Files app, locate your card. Long press your card and choose **Get Info** to see the properties of the imported file.



Your card in the Files app

Notice that the dimensions of the PNG file are `1300 x 2000`, which is what you specified for your card size.

There's only one problem. You'd much rather have it in Photos than Files.

Configuring Your App to Save Photos

Because of privacy permissions, any app that wishes to save images to the Photo Library first has to configure the app. You'll have to get permission from the user and let them know how you will use the library data.

App properties are held in your app's **Info.plist**. You'll save a property here to allow Photo Library additions, and the option to save a photo will automatically appear in the share sheet's list of actions.

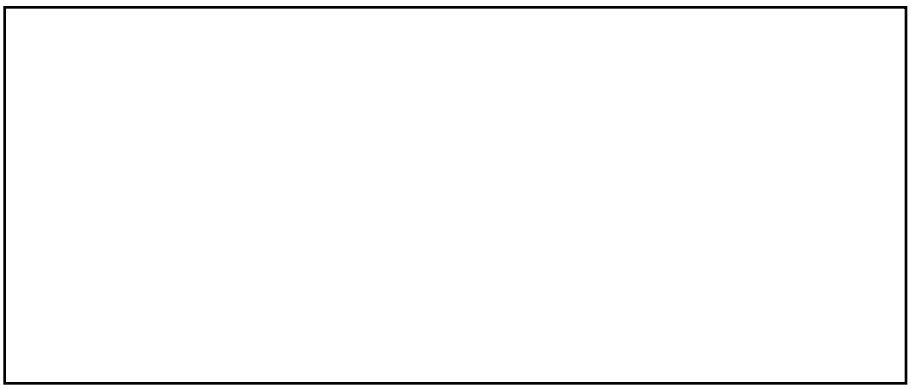
In the Project navigator, select the topmost **Cards** and choose the **Cards** target, then choose **Info** from the options across the top.

➤ Add a new key **NSPhotoLibraryAddUsageDescription**, or **Privacy - Photo Library Additions Usage Description**.

➤ In the Value field, add:

```
Cards will save your card to the Photo Library
```

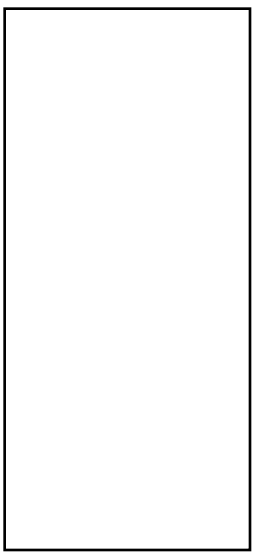
This is the message your users will see, so you might add something soothing about not using their personal data for nefarious purposes.



Key to ask user for permission to use photo library

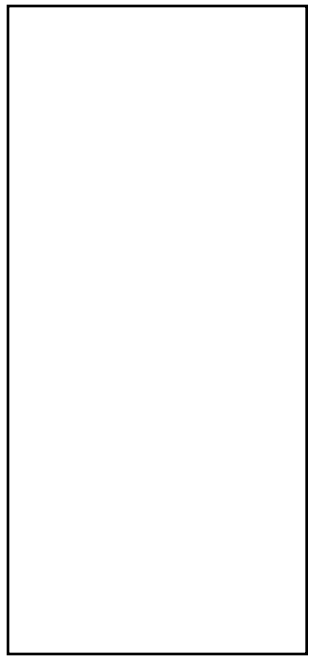
➤ Build and run the app again and choose a card. Share the card and this time, **Save Image** appears as an option. Save the image to the Photo Library.

The app asks for permission to save to photos, showing the message you entered in the **Info** key.



Asking user for permission to use photo library

➤ Tap **OK** and the card will save to the photo library. Check out the Photos app on the simulator to see your photo library.



Your shared card in the Photos Library

If you run the app on a device with Mail, Messages or any sharing app installed, you can share the image through those, too.

Challenges

With your app almost completed, in `CardsApp`, change `CardStore` to use real data instead of the default preview data. Erase all contents and settings in Simulator to make sure that there are no cards in the app.

Challenge 1: Save & Load the Card Thumbnail

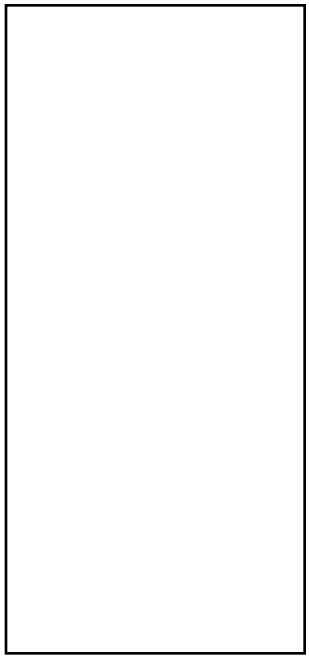
Currently, the list of cards doesn't show a preview of the card. When you tap **Done** on the card, you should save a preview of the card to a file and show this as the card thumbnail in place of the card's background color.

To achieve this:

1. Locate the code where you save the card in **SingleCardView.swift**. First use `UIImage.screenshot(card:size:)` to generate a `UIImage` and then save the `UIImage` to a file. **UIImageExtensions.swift** contains a method `UIImage.save(to:)` to save the file. Use `card.id.uuidString` as the filename.
2. In **CardThumbnail.swift**, load this image file. There's a `UIImage.load(uuidString:)` method in **UIImageExtensions.swift**. If the load is successful, show the image. If not, show the card's background color. Enclose the two alternative views in a `Group` and place the modifiers on the group, rather than on the background color.

If you have done this part correctly, when testing this in Simulator, the card thumbnail image will load when you first run the app, but not when you change a card by moving one of the elements. In **CardsListView.swift**, `CardThumbnail` will only refresh if there are published changes. `CardThumbnail` uses `card` from `store.cards`, and this is the property that you need to update.

3. Add a `uiImage: UIImage?` property to `Card` and update this property when you load the card image in **SingleCardView.swift**. Updating this property means that you update the published property `cards` in `CardStore`, and the card thumbnail will redraw.



The thumbnail image

Challenge 2: Change the Text Entry Modal View

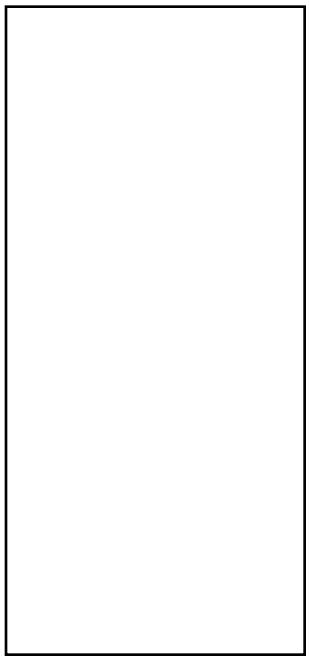
In the **Supporting Code** group, you'll find an enhanced Text Entry view, called **TextView.swift**, that lets users pick fonts and colors when they enter text. There's a list of some of the fonts available on iOS in **AppFonts.swift**.

First, preview and examine `TextView` and make sure you understand it. SwiftUI views look complicated, but you have encountered almost everything in this file before.

Your challenge is to add this view to the modal view `TextModal` under the current `TextField`.

With the new font and color, style the text currently being entered in the `TextField`. Use `.font(.custom(textElement.textFont, size: 30))` to style the font.

To test the view, run the app in Simulator or Live Preview `SingleCardView`.



Text entry with fonts and colors

When you've completed these challenges, you should be well pleased with yourself. You've worked hard to construct an app with some very tricky features. Don't rest on your laurels, though. You still have Section 3 to work through!

Key Points

- Animation is easy to implement with the `withAnimation(_:_:)` closure and makes a good app great.
- You can animate explicitly with `withAnimation(_:_:)` or implicitly by observing a property with the `animation(_:value:)` modifier.
- Transitions are also easy with the `transition(_:)` modifier. Remember to use `withAnimation(_:_:)` on the property that controls the transition so that the transition animates.
- Picker views allow the user to pick one of a set of values. You can have a wheel style picker or a segmented style picker.
- Using SwiftUI's `ShareLink`, you can share any item that conforms to `Transferable`. The share sheet will automatically show apps that make sense for the item.

Where to Go From Here?

You probably want to animate everything possible now. The book iOS Animation by Tutorials (https://bit.ly/3VsvLmf) is available with the Pro subscription and has two chapters dedicated to animations and transitions with SwiftUI.

A great example of an app with complex layout and animation is Apple's Fruta sample app (https://apple.co/2XE8tNF). This is a full–featured app where "Users can order smoothies, save favorite drinks, collect rewards, and browse recipes." Fruta also has various features, such as widgets. Download the app and see if you can work out how it all fits together.

Section III: Your Third App: TheMet

You've now built two apps with beautiful user interfaces. But, you're probably wondering how to build an app that accesses resources on the internet. Fear not! In this section, you'll build TheMet, an app that allows you to view items in the collection at the Metropolitan Museum of Art — better known as The Met. Along the way, you'll:

- Learn how to build lists of information and navigate between views using SwiftUI.
- Discover the intricacies of REST APIs and how to use them.
- Explore iOS's networking support using a Swift Playground.
- Learn to how add support for iOS Widgets to your app.

Chapter 22: Lists & Navigation

By Audrey Tam

Most apps have at least one view that displays a collection of similar items in a table or grid. When there are too many items to fit on one screen, the user can view more items by scrolling — vertically, horizontally or both. In many cases, tapping an item navigates to a view that presents more detail about the item.

In this section, you'll start implementing **TheMet**, an app that searches The Metropolitan Museum of Art, New York (https://www.metmuseum.org) for objects matching the user's query term.

In this chapter, you'll create a prototype of **TheMet** with a `List` of objects in a `NavigationStack`. Tapping a list item pushes a detail view onto the navigation stack. The starter project already contains **ObjectView.swift**, which displays some of the object properties.

Getting Started

➤ Open the **TheMet** app in the **starter** folder. For this chapter, the starter project initializes the `Object` data in **Preview Content**. In Chapter 24, "Downloading Data", you'll fetch this data from collectionapi.metmuseum.org.

List

You encountered the SwiftUI `List` view in Chapter 10, "Working With Datasets", where you learned how to let users edit the history of exercises in HIITFit.

`List` is the easiest way to present a collection of items in a view that scrolls vertically. You can display individual views *and* loop over arrays within the same `List`. In this chapter, you'll start by just listing objects, then you'll embed the list in a navigation stack so users can navigate to a detail view for each list item.

To present a list of objects, the syntax looks a lot like `ForEach`.

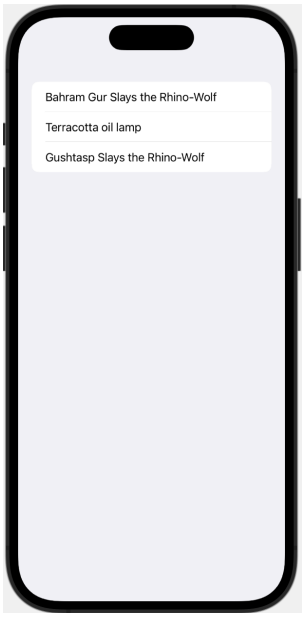
➤ In **ContentView.swift**, replace the contents of `ContentView` with the following code:

```
@StateObject private var store = TheMetStore()

var body: some View {
  List(store.objects, id: \.objectID) { object in
    Text(object.title)
  }
}
```

You initialize `TheMetStore`, which calls `createDevData()` to create a sample `objects` array. Then, you tell `List` to loop over `objects`, and you provide an `id`. Like `ForEach`, `List` expects each item to have an identifier, so it knows which item is in which row. The argument `\.objectID` tells `List` that each item is identified by that property value. For each `object` in the list, you display its title.

In Live Preview, it doesn't look like much but, later in this chapter, you'll spruce it up with a title, custom colors and a search button.



List with development data

NavigationStack

In Chapter 13, "Outlining a Photo Collage App", you used `NavigationStack` so you could add toolbar buttons to `SingleCardView`. Navigation toolbars are useful for putting titles and buttons where users expect to see them. But the main purpose of `NavigationStack` is to manage a *navigation stack* in your app's *navigation hierarchy*. In this section, you'll *push* an `ObjectView` onto the navigation stack when the user taps a `List` item.

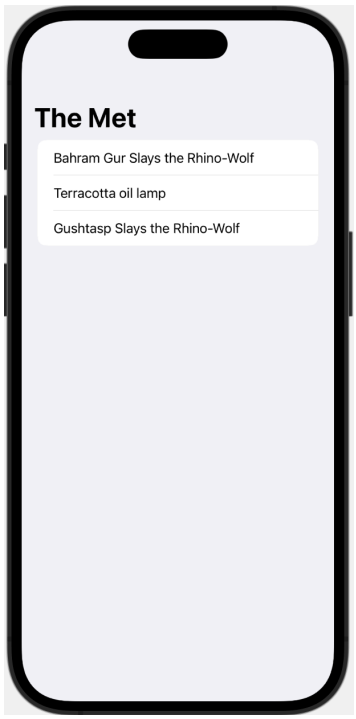
Start by adding a navigation bar with a title.

➤ In **ContentView.swift**, replace `List` with the following to embed it in a `NavigationStack` and set the screen's title:

```
NavigationStack {
  List(store.objects, id: \.objectID) { object in
    Text(object.title)
  }
  .navigationTitle("The Met")
}
```

Notice `navigationTitle` modifies `List`, not `NavigationStack`. A `NavigationStack` can contain alternative root views, each with its own `.navigationTitle` and toolbars.

In **Live Preview**, you get a large title by default:



Navigation title defaults to large title.

Navigating to a Detail View

Now, you'll navigate to the object's `ObjectView` when the user taps the list item.

➤ In the `List` closure, replace `Text(object.title)` with this:

```
NavigationLink(object.title) {
  ObjectView(object: object)
}
```

A `NavigationLink` takes two arguments — a label and a destination. Here, you label the link with the object's title and, in a trailing closure, set its destination to `ObjectView`. Each `List` row acquires a *disclosure indicator*, telling the user there's more to see.

> **Note**: When the label is only text, **NavigationLink** has a convenience initializer that creates a **Text** view from a **String**.

➤ In Live Preview, tap an item:

Navigation link to ObjectView

`ContentView` is currently the only view in the navigation stack. When you tap a list item, `NavigationStack` *pushes* `ObjectView` onto the navigation stack: It's now the top view on the stack, so it's the view that's visible.

`NavigationStack` gives you a "back" button, labeled the same as the root view's `navigationTitle`.

➤ Tap the back button to *pop* this view off the navigation stack, revealing `ContentView` again.

Using the Internet

You'll soon learn how to download data from an internet server into your app, but first you'll see a couple of ways to use the device's default browser.

One of the `Object` properties is `objectURL` — the URL of the object's page at metmuseum.org. There's an easy way to open this page in the device's default browser.

Link Button

➤ In **ContentView.swift**, comment out the `NavigationLink(...) { ... }` code and type the following code:

```
Link(object.title, destination: URL(string: object.objectURL)!)
```

You create a special button whose label is the object's title. Tapping this button opens its destination `URL` in the associated app. You create the `URL` from the `Object` property `objectURL`. The associated app is Safari (in a simulator) or your device's default browser.

> **Note**: To be safe, you should check `URL(string: object.objectURL)` isn't `nil` but, for now, you can assume metmuseum.org always supplies a valid URL string in `objectURL`.

➤ Build and run in the simulator or on your device: There's no disclosure indicator anymore because the whole list row is a button. Tap an item to open the object's web page in Safari or your device's default browser:



Open object's metmuseum.org page in browser app.

`Link` takes users from your app to their browser app, giving them access to their browser settings and saved passwords. They can easily explore the site without sharing any secure data or history with your app. It's the normal browser app, so your users can even enter another URL in the location field and go anywhere on the internet.

➤ To return to your app, tap the **TheMet** back button.

SFSafariViewController

You might prefer your users don't leave your app. You can open a Safari browser **in** your app. Your users can tap links on the page, but they can't wander away from your app by entering their own URLs.

➤ Look at **SafariView.swift**:

```
import SwiftUI
import SafariServices

struct SafariView: UIViewControllerRepresentable {
  let url: URL

  func makeUIViewController(
    context: UIViewControllerRepresentableContext<SafariView>
  ) -> SFSafariViewController {
    return SFSafariViewController(url: url)
  }

  func updateUIViewController(
    _ uiViewController: SFSafariViewController,
    context: UIViewControllerRepresentableContext<SafariView>)
{}
}
```

➤ Option-click **SFSafariViewController** and read its documentation.

`SFSafariViewController` is a `UIViewController` that provides Safari features to your users, but your app cannot access their activity or private information.

You use **Representable** protocols to insert UIKit views or view controllers into your SwiftUI apps. Instead of creating a structure that conforms to `View` for a SwiftUI view, you create a structure that conforms to either `UIViewRepresentable` — for a single view — or `UIViewControllerRepresentable` — to use a view controller for complex management of views.

`UIViewControllerRepresentable` requires methods to make and update the view controller. `SFSafariViewController` needs only a URL and doesn't really need any updating.

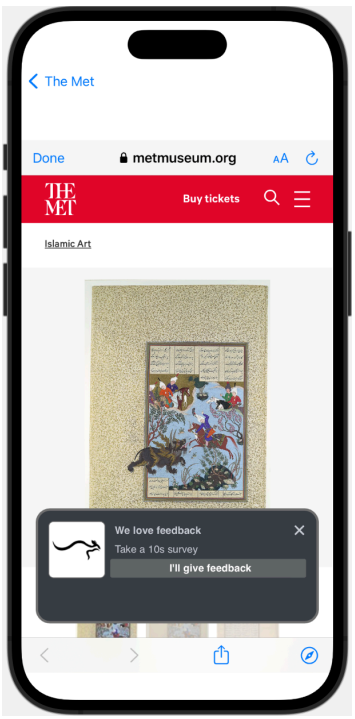
> **Note**: For a more complex example of `UIViewControllerRepresentable`, see SwiftUI by Tutorials, Chapter 21, "Complex Interfaces" (https://bit.ly/3VQb5Uu).

➤ Go back to **ContentView.swift** and delete the `Link(...) { ... }` code and type the following code in its place:

```
NavigationLink(
  destination: SafariView(url: URL(string: object.objectURL)!))
{
  HStack {
    Text(object.title)
    Spacer()
    Image(systemName: "rectangle.portrait.and.arrow.right.fill")
      .font(.footnote)
  }
}
```

The destination is a `SafariView` that loads the object's web page. This time, you define the label in the trailing closure because it's more complex than a `String` — you add an icon to indicate that tapping the item takes the user to a web page.

➤ In Live Preview, tap an item:



Open object's metmuseum.org page in Safari, in your app.

Now, `NavigationStack` pushes `SafariView` onto the navigation stack and gives you the standard **The Met** back button. There's no location field, although there are buttons for the share sheet, and the user can open this page in their default browser.

➤ Tap the back button to pop this view off the navigation stack and return to the list view.

AsyncImage

In the next chapters, you'll learn how to use `URLSession` methods to download `Object` data from metmuseum.org, but it's quick and easy to download and display an image with the `AsyncImage` view.

If an object is in the public domain, then its images are available for use without restriction under the Met's Open Access program, and its `primaryImageSmall` property is a non-empty string — a web address.

➤ In **ObjectView.swift**, locate the `if object.isPublicDomain` closure and replace its contents with this code:

```
AsyncImage(url: URL(string: object.primaryImageSmall)) { image
in
  image
    .resizable()
    .aspectRatio(contentMode: .fit)
} placeholder: {
  PlaceholderView(note: "Display image here")
}
```

Leave the `else` closure as it is.

The `url` argument is `URL?` so, if `primaryImageSmall` yields a valid `URL`, the view returns an image. You modify this image with the usual image modifiers and reuse the `PlaceholderView` "picture frame" as a placeholder while the image is downloading.

> **Note**: Bear in mind that you can't apply modifiers directly to `AsyncImage`, instead you need to apply them to `image`. There's more you can do with `AsyncImage`, like animate the way the image appears or handle possible errors. If you want to learn about it, take a look at AsyncImage's official documentation (https://apple.co/3XBOqfx).

In Live Preview, the image for “Bahram Gur Slays the Rhino-Wolf” appears:



Download and display the object's primary image.

That was super easy!

navigationDestination

This app can download two kinds of objects from metmuseum.org. Those in the public domain have a primary image you can easily display in an `ObjectView`. What should your app do for objects that aren’t in the public domain? Well, you can just as easily load their web page into a `SafariView`. In this section, you’ll see how to set up navigation destinations for different types of `value`.

Extracting Web Indicator View

➤ First, in **ContentView.swift**, to keep your code neat, extract the `HStack` with the web indicator into its own view:

```
struct WebIndicatorView: View {
  let title: String

  var body: some View {
    HStack {
      Text(title)
      Spacer()
      Image(systemName:
"rectangle.portrait.and.arrow.right.fill")
        .font(.footnote)
    }
  }
}
```

➤ And, your `NavigationLink` becomes:

```
NavigationLink(destination: SafariView(url: URL(string:
object.objectURL)!)) {
  WebIndicatorView(title: object.title)
}
```

Handling Both Kinds of Objects

➤ Now, comment out the `SafariView` navigation link and uncomment the `ObjectView` navigation link:

```
NavigationLink(object.title) {
  ObjectView(object: object)
}
```

You'll soon set up the `List` with both navigation links, but first, see what happens:

➤ In Live Preview, tap the second item — **Terracotta oil lamp**:

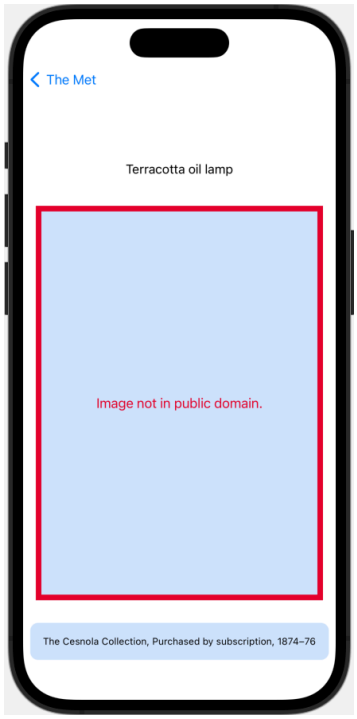


Image not in public domain.

This object isn't in the public domain, so its `primaryImageSmall` is an empty string, and `ObjectView` has no image to display. Displaying this message is … OK, but it's a better user experience to open the object's page in `SafariView`.

> **Note**: Actually, this lamp *is* in the public domain, but **MetStoreDevData.swift** creates it as if it isn't.

Now, here's one way to display public-domain objects with the `object.title` label and non-public-domain objects with the `WebIndicatorView` label.

➤ Uncomment the `WebIndicatorView` navigation link closure and carefully enclose it and the `ObjectView` navigation link closure in an if-else:

```
if !object.isPublicDomain,
   let url = URL(string: object.objectURL) {
  NavigationLink(destination: SafariView(url: url)) {
    WebIndicatorView(title: object.title)
  }
} else {
  NavigationLink(object.title) {
    ObjectView(object: object)
  }
}
```

You send non-public-domain objects to `SafariView(url:)` and public-domain objects to `ObjectView(object:)`. And, this is a good opportunity to safely unwrap `URL(string: object.objectURL)` and pass `url` to `SafariView(url:)`

➤ In Live Preview, tap a public-domain item, then the oil lamp:

Navigate to ObjectView or SafariView.

This works fine: The poor rhino-wolf gets slain in `ObjectView`, and `SafariView` loads the oil lamp's web page. But, there's another `NavigationLink` initializer you can use with the `navigationDestination` modifier.

Using navigationDestination

➤ Replace your if-else code with the following:

```
if !object.isPublicDomain,
   let url = URL(string: object.objectURL) {
  NavigationLink(value: url) {
    WebIndicatorView(title: object.title)
  }
} else {
  NavigationLink(value: object) {
    Text(object.title)
  }
}
```

You use the `value` initializer for `NavigationLink`, so both label views are in the trailing closures. This version expects you to modify the enclosing `List` with a matching `navigationDestination` for each type of `value`.

➤ Below `.naviationTitle("The Met")`, add these `List` modifiers:

```
.navigationDestination(for: URL.self) { url in
  SafariView(url: url)
    .navigationBarTitleDisplayMode(.inline)
    .ignoresSafeArea()
}
.navigationDestination(for: Object.self) { object in
  ObjectView(object: object)
}
```

For non-public-domain objects with a valid `objectURL`, `NavigationLink` passes `url`, which is a `URL`. It matches `.navigationDestination(for: URL.self)`, which receives the value as `url`, so the destination is still `SafariView(url: url)`. The two modifiers of `SafariView` reduce the gap below the back button and make the `SafariView` toolbar color match the app's navigation toolbar. It simply looks a little nicer.

For public-domain objects or non-public-domain objects without a valid `objectURL`, `NavigationLink` passes an `object`, which matches `.navigationDestination(for: Object.self)`, so the destination is still `ObjectView(object: object)`.

➤ In Live Preview, try both kinds of navigation link to confirm they still work the same.

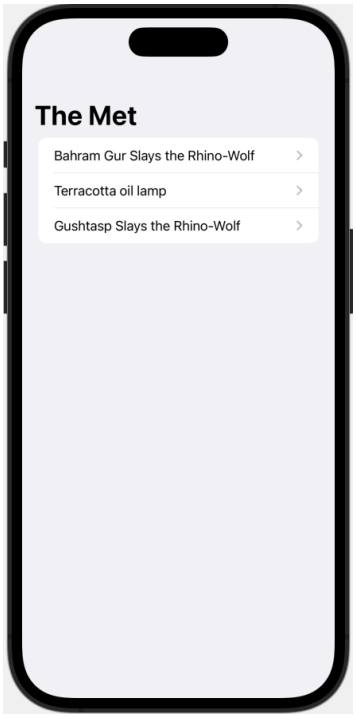
Testing for Invalid objectURL

Now that you're checking for a valid URL before calling `SafariView(url:)`, how do you test for an invalid URL? Well, you've got your own sample data in **MetStoreDevData.swift**, and you can set any values you like.

➤ In **MetStoreDevData.swift**, comment out the `objectURL` line for the terracotta oil lamp and type this line below it:

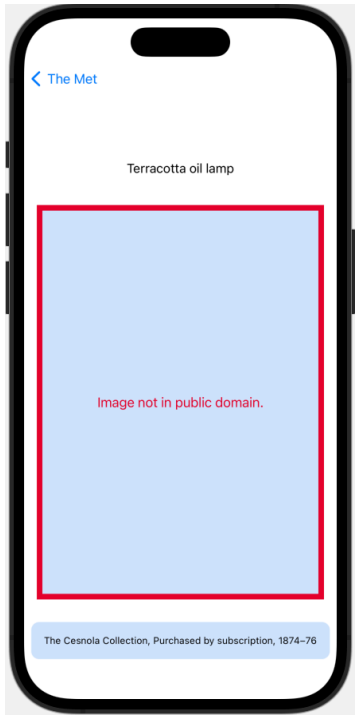
```
  objectURL: "",    // don't forget the comma!
```

➤ Back in **ContentView.swift**, refresh Live Preview to see the oil lamp item no longer uses `WebIndicatorView(title:)`:



List: Non-public-domain object with invalid URL

➤ Tap the oil lamp item:



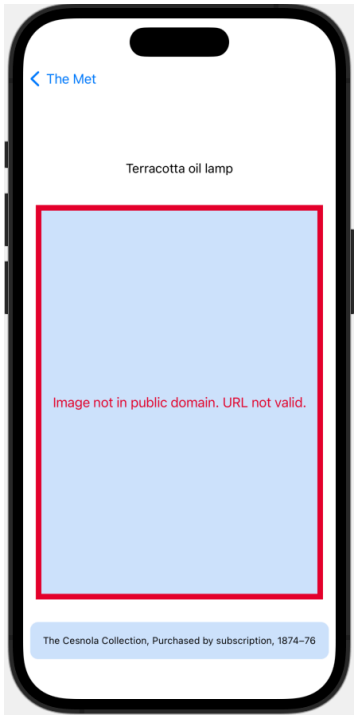
ObjectView: Non-public-domain object with invalid URL

The message "Image not in public domain." is useful, but you could add a little more information to explain why the user sees this view instead of a metmuseum.org web page.

➤ In **ObjectView.swift**, add "URL not valid." to the `note`:

```
PlaceholderView(note: "Image not in public domain. URL not
valid.")
```

➤ Back in **ContentView.swift**, refresh Live Preview, then tap the oil lamp item:

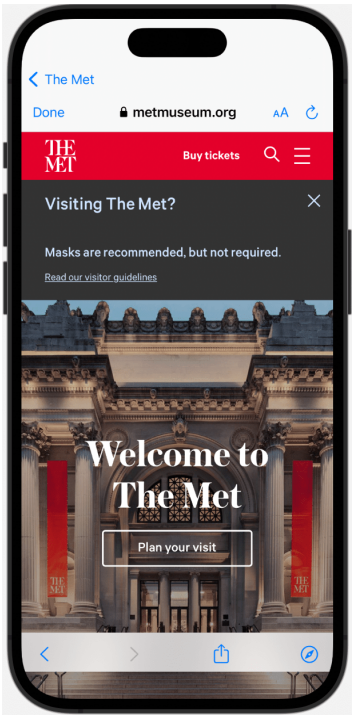


ObjectView: Non-public-domain object with invalid URL

That will do — you don't really expect this situation will happen, but you've got it covered anyway.

Using Custom Colors

➤ In **MetStoreDevData.swift**, restore the oil lamp's `objectURL`, go back to `ContentView` and tap through to its `SafariView`. Then tap **THE MET** (on the web page) to go to the home page:



Metropolitan Museum home page

Notice the main colors? The header has a red background and the "Visiting The Met?" text is light blue, like the sky in the photo below.

In your app, **Assets.xcassets** defines these two colors as **met-background** and **met-foreground**. **ColorExtension.swift** extends Color to add `metBackground` and `metForeground` as static properties. You'll use these colors to differentiate the public-domain and non-public-domain rows.

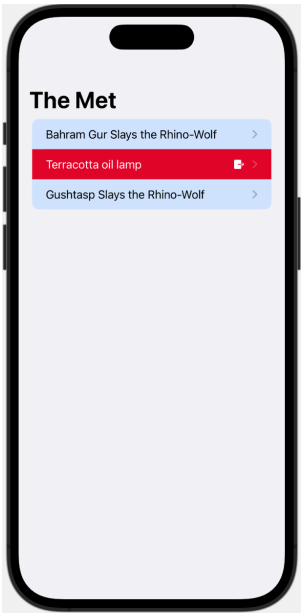
Color-Coding the List Rows

➤ Add these modifiers to the `NavigationLink` of the non-public-domain objects:

```
.listRowBackground(Color.metBackground)
.foregroundColor(.white)
```

➤ And add this modifier to the `NavigationLink` of the public-domain objects:

```
.listRowBackground(Color.metForeground)
```



Color-coded list rows

You've made the non-public-domain rows red, changing the text color to white, so it shows up on the red background. And you made the public-domain rows sky blue

Linking to Met From ObjectView

Tapping a public-domain object navigates to its `ObjectView`, which downloads and displays its primary image. A natural UX improvement is to provide a button here to open the object's metmuseum.org page.

➤ In **ObjectView.swift**, replace `Text(object.title)` and its three modifiers with this if-else code:

```
if let url = URL(string: object.objectURL) {
  Link(destination: url) {
    WebIndicatorView(title: object.title)
      .multilineTextAlignment(.leading)
      .font(.callout)
      .frame(minHeight: 44)
      // add these four modifiers
      .padding()
      .background(Color.metBackground)
      .foregroundColor(.white)
      .cornerRadius(10)
  }
} else {
  Text(object.title)
    .multilineTextAlignment(.leading)
    .font(.callout)
    .frame(minHeight: 44)
}
```

If the object's `objectURL` is valid, you wrap its `title` in a `WebIndicatorView` and style it to look like the non-public-domain rows in your `List`. Then, you create a `Link` button with this view as its label.

It looks great in Live Preview:



Link to metmuseum.org matches non-public-domain list rows.

Build and run the app in a simulator to try it out.

One Last Thing

Soon, you'll implement the code to download objects from metmuseum.org. These objects will match the user's query term, like "rhino" or "persimmon". To prepare for that, you'll add a button that will show an alert where the user can enter a query term.

➤ In **ContentView.swift**, add these two `@State` properties:

```
@State private var query = "rhino"
@State private var showQueryField = false
```

You provide a starting query term and initialize the value that shows or hides the alert.

➤ Next, below `.navigationTitle("The Met")`, add the toolbar button:

```
.toolbar {
  Button("Search the Met") {
    query = ""
    showQueryField = true
  }
  .foregroundColor(Color.metBackground)
  .padding(.horizontal)
  .background(
    RoundedRectangle(cornerRadius: 8)
      .stroke(Color.metBackground, lineWidth: 2))
}
```

When the user taps this button, it resets `query` to the empty string and sets `showQueryField` to show the alert. You set the color of the button's text and border to `metBackground` to make it stand out.



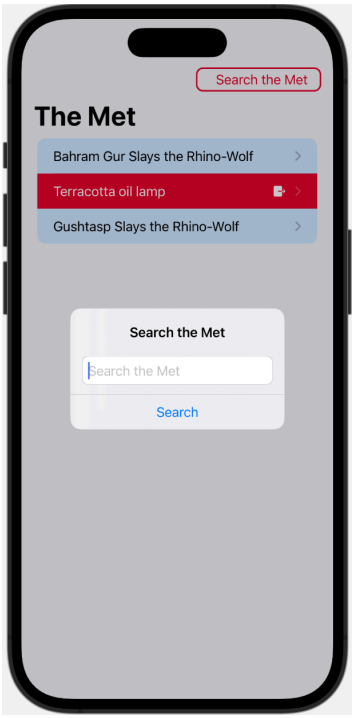
Search button in toolbar

➤ Now, below the `toolbar` modifier, add the `alert` modifier:

```
.alert("Search the Met", isPresented: $showQueryField) {
  TextField("Search the Met", text: $query)
  Button("Search") { }
}
```

You pass a binding to `query` to the text field. You'll fill in the Search button's action after you write the download code in Chapter 24, "Downloading Data".

➤ In Live Preview, tap the toolbar button:



Alert with text field to get query term

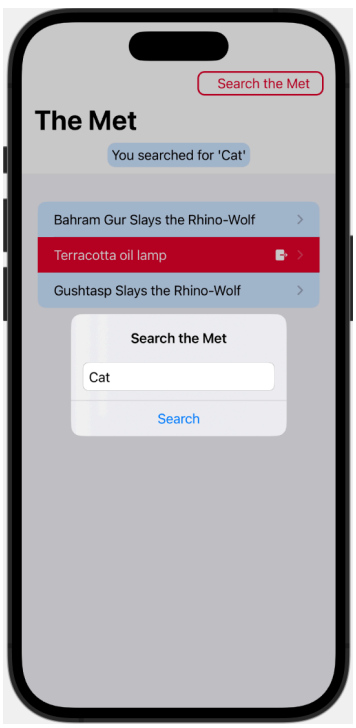
The Very Last Thing

Users expect to see a reminder of what they searched for, so you'll add a message above the list.

➤ First, embed `List` in a `VStack`, then add this code at the top of the `VStack`:

```
Text("You searched for '\(query)'")
  .padding(5)
  .background(Color.metForeground)
  .cornerRadius(10)
```

➤ In Live Preview, tap the search button, then type some text:



You-searched-for message

The message updates to **You searched for ‘’**, then it shows whatever you typed into the text field. Looking good! Now, you're all set to learn how to download data from a server, after the next chapter, which covers some HTTP and REST API basics.

Key Points

- The SwiftUI `List` view is the easiest way to present a collection of items in a view that scrolls vertically. Call `.listRowBackground` on the view in the row, not on the `List` itself.
- `NavigationStack` manages a navigation stack in your app's navigation hierarchy. Tapping a `NavigationLink` pushes its destination view onto the navigation stack. Tapping the back button pops this view off the navigation stack.
- A `NavigationStack` can contain alternative root views. You modify each with its own `navigationTitle` and toolbars.
- A `NavigationLink` has an initializer that takes two arguments — a label view and a destination view. You can supply a `String` for the label, and `NavigationLink` will create a `Text` view.
- You can navigate using values thanks to the initializer `NavigationLink.init(value:label:)`.
- Your app can open a web link in the device's default browser using `Link` or as a Safari view within your app.
- It's easy to download an image and display it with the `AsyncImage` view.

Chapter 23: Just Enough Web Stuff

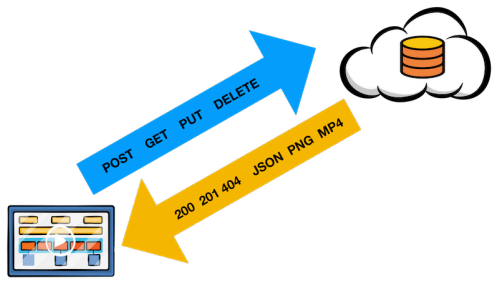
By Audrey Tam

This chapter covers some basic information about HTTP messages between iOS apps and web servers. It's just enough to prepare you for the following chapter, where you'll implement downloads from the metmuseum.org server.

There's no SwiftUI in this chapter.

If you already know all about HTTP messages, skip down to the section "Exploring metmuseum.org" to familiarize yourself with the API you'll use in the following chapters.

Servers & Resources



HTTP requests and responses between client and server

Many apps communicate with computers on the internet to access databases and other resources. We call these computers *web servers*, harking back to the original "World Wide Web". Or *cloud servers* because nowadays everything is "in the Cloud". "Host" is another term for "server".

Apps like Safari and TheMet are *clients* of these servers. A client sends a *request* to a server, which sends back a *response*. This communication consists of plain-text messages that conform to the Hypertext Transfer Protocol (**HTTP**). Hypertext is structured text that uses hyperlinks between nodes containing text. Web pages are written in HyperText Markup Language (HTML).

HTTP has several methods, including POST, GET, PUT and DELETE. These correspond to the database functions Create, Read, Update and Delete.

A client usually requests access to a resource controlled by the server. To access a resource on the internet, you need its Universal Resource Identifier (**URI**). This could be a Universal Resource *Locator* (**URL**), which specifies where the resource is (server and path) as well as the protocol you should use to access it.

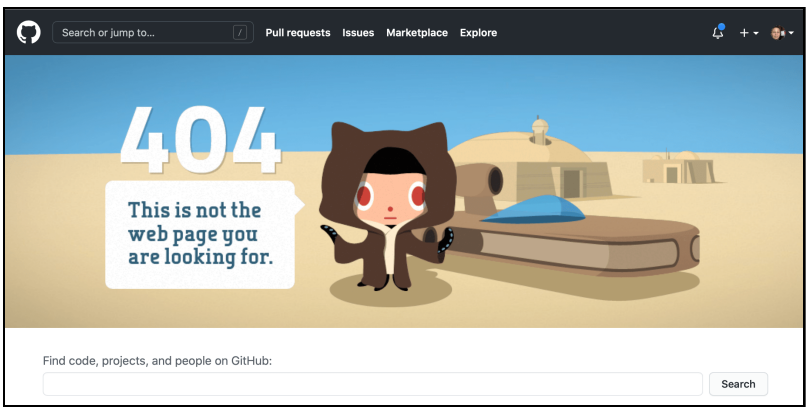
For example, **https://www.metmuseum.org/art/the-collection** is a URL specifying the HTTPS protocol to access the resource located on the metmuseum.org server with the path `art/the-collection`.

> **Note**: **HTTPS** is the secure, encrypted version of HTTP. It protects your users from eavesdropping. The underlying protocol is the same but, instead of transferring plain-text messages, everything is encrypted before it leaves the client or server.

HTTP Messages

A client's HTTP **request** message contains *headers*. A POST or PUT request has a *body* to contain the new or updated data. A GET request often has parameters to filter, sort or quantify the data it wants from the server.

A server's HTTP **response** message also has headers and a body. A key part of the response is the **status code** — ideally, **200 OK** in response to a GET request or **201 Created** in response to a POST request. You don't want to see any error status codes like **404 Not Found**:



GitHub's 404 page

There are many many HTTP response status codes. You'll find a fun representation of them at http.cat. For example:



418 I'm a teapot

Mozilla (mzl.la/3o6qWNM) provides more conventional descriptions of status codes:

*The HTTP **418 I'm a teapot** client error response code indicates that the server refuses to brew coffee because it is, permanently, a teapot. A combined coffee/tea pot that is temporarily out of coffee should instead return 503. This error is a reference to Hyper Text Coffee Pot Control Protocol defined in April Fools' jokes in 1998 and 2014. Some websites use this response for requests they do not wish to handle, such as automated queries.*

> **Note**: The 1998 HTCPCP (https://bit.ly/3olM42q) April Fools' joke was inspired by the Trojan Room coffee pot (https://bit.ly/3pkCDSb), the subject of the world's first web cam. It was set up in 1991, long before the Internet of Things (IoT).

If an HTTP message has a body, it also has a **Content-Type** header. Content-Type specifies the internet media type of the data in the HTTP message body.

Usually, you'll work with three content types for *text data*, depending on the structure:

- **JSON** (JavaScript Object Notation) is the most common data format used for HTTP communication by app clients. It's a structured data format consisting of numbers, strings, and arrays and dictionaries that can contain strings, numbers and nested arrays and dictionaries.
- Web forms use **form-encoded**, which looks like a query string. A query string is a collection of key-value pairs, separated by `&` and preceded by `?`.
- Web pages are **HTML**.

When working with *binary data* some of the most used types are PDF, image formats and multi-part form data, when the client sends any kind of binary file along with text elements.

REST API

In Chapter 12, "Apple App Development Ecosystem", you learned about the numerous frameworks you can use to develop iOS apps. An Apple framework is one kind of Application Programming Interface (**API**). It tells you how to use the standard components created by Apple engineers.

Another kind of API is the set of rules for clients to request resources from a server. Most of the APIs you'll use for your apps are REST APIs, which use HTTP. For each resource available on the server, the REST API documentation tells you how to construct a request:

- The resource's URL, called its *endpoint*.
- Which HTTP method to use.
- Which HTTP headers to include.
- What to put in the request body.

> **Note**: REST is the acronym of "REpresentational State Transfer", the name created by Roy Fielding for the architectural style underlying the World Wide Web. The term describes how a well-designed Web application works: A user selects a resource identifier from a network of Web resources (a virtual state-machine) and uses methods like GET or POST to create a state transition that transfers the resource's representation to the user.

In the next chapter, you'll set up TheMet to communicate with the museum's REST API. In this chapter, you'll explore this API's documentation (https://metmuseum.github.io).

Sending & Receiving HTTP Messages

Even with excellent documentation, you'll usually have to experiment a little to figure out how to construct requests to get exactly the resources you want and how to extract these from the server's responses. So how do you send requests and examine responses?

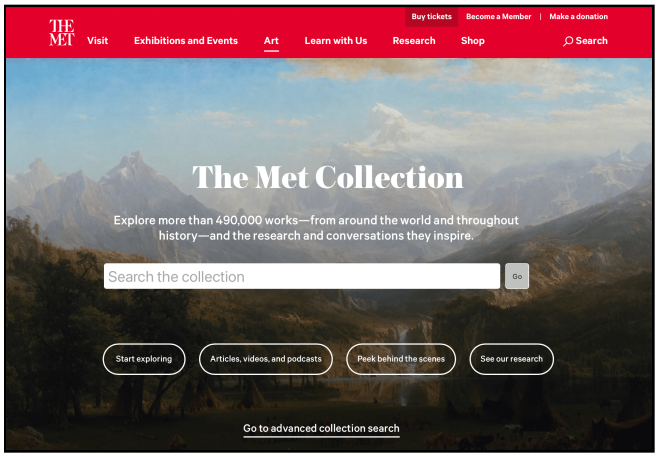
Browser

The easiest way to make a simple HTTP GET request is to enter the URL in a browser app like Safari.

➤ Enter this URL in your favorite browser:

```
https://www.metmuseum.org/art/the-collection
```

This is the endpoint of the metmuseum.org art collection. You get a page similar to this:



HTTP response to metmuseum.org/art/the-collection request

This is the body of the server's response, but you don't get to see the headers. And, you can't do much more than a simple GET request.

cURL

A browser is a fully-automated HTTP tool. At the other end of the spectrum is the command-line tool cURL (https://curl.se) — "the internet transfer backbone for thousands of software applications".

The documentation for a REST API often provides sample requests to show you how to use it. Very often, these use cURL.

➤ Open **Terminal** and enter this command:

```
curl https://api.github.com/zen
```

You send an HTTP request to GitHub's API server. The response is a random item from their design philosophies, like "Favor focus over features" or "Avoid administrative distraction". There are lots more request examples at GitHub's Getting started with the REST API (https://bit.ly/3iD717R).

But, you exclaim, `curl` doesn't show any response headers either! Well, like all Unix commands, `curl` has a wealth of options, including `--include` and its shortcut `-i`, to include the HTTP response headers in its output.

➤ Enter this command:

```
curl -i https://api.github.com/zen
```

And you see quite a lot more output:

```
HTTP/2 200
server: GitHub.com
date: Fri, 09 Dec 2022 01:49:10 GMT
content-type: text/plain;charset=utf-8

...

x-ratelimit-reset: 1670551677
x-ratelimit-resource: core
x-ratelimit-used: 2
accept-ranges: bytes
x-github-request-id: F968:61FE:7DF301:85B2E3:63929416

Design for failure.
```

Headers beginning with **x-** are custom headers set up by the organization. For example, `x-ratelimit-limit` and `x-ratelimit-used` indicate how many requests a client can make in a rolling time period (typically an hour) and how many of those requests the client has already made.

The `curl --verbose` or `-v` option displays *request* headers and a *lot* more.

➤ Enter this command:

```
curl -v https://api.github.com/zen
```

Replacing `-i` with `-v` produces quite a lot more output — every handshake interaction between the terminal and the server, the encryption algorithms used, the server certificate details, as well as the response headers. The *request* headers are just the five lines that start with `>`:

```
> GET /zen HTTP/2
> Host: api.github.com
> user-agent: curl/7.79.1
> accept: */*
>
```

Lines that start with `<` are *response* headers, and lines that start with `*` are additional information provided by cURL.

You might not enjoy typing long structured command lines, especially something like this sample cURL command to create a new GitHub repository:

```
curl -i -H \
  "Authorization: token
5199831f4dd3b79e7c5b7e0ebe75d67aa66e79d4" \
  -d '{ \
      "name": "blog", \
      "auto_init": true, \
      "private": true, \
      "gitignore_template": "nanoc" \
    }' \
  https://api.github.com/user/repos
```

This POST command sends authorization data in a request header and the request body as data in JSON format. The endpoint doesn't name a specific user because GitHub knows that from the token value.

Another problem with using cURL: If the response is complex, it's hard to examine it in the terminal.

➤ Enter this command:

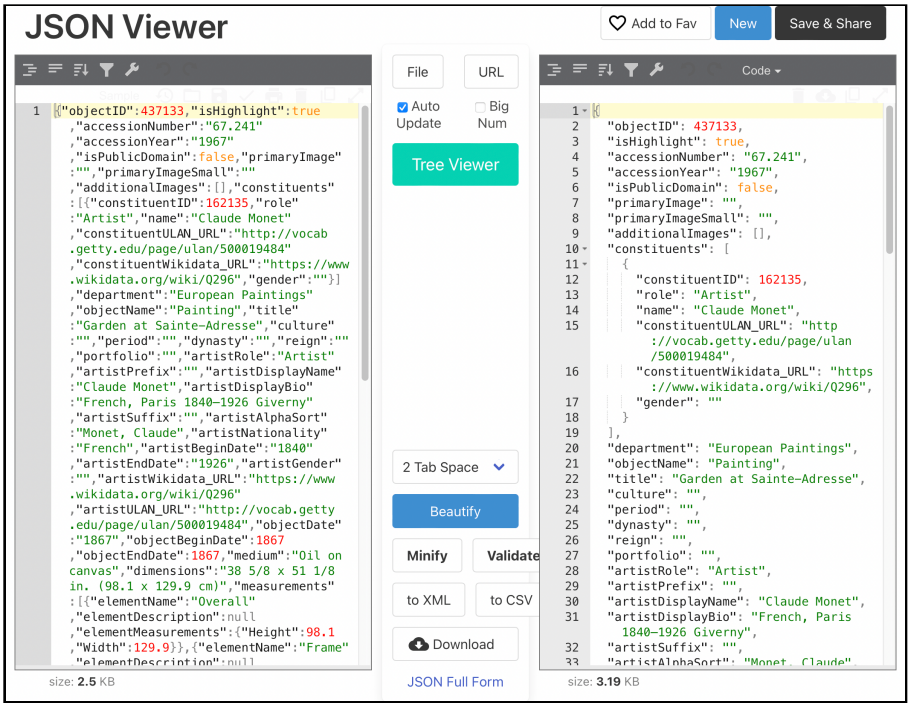
```
curl https://collectionapi.metmuseum.org/public/collection/v1/
objects/437133
```

This is a request to the API you'll use for TheMet. The response is pretty mind-numbing:



Response body using cURL

If you concentrate, you *might* be able to see from this output that the response body is a dictionary where a couple of items are arrays of dictionaries. You can use a tool like codebeautify.org/jsonviewer to format and beautify this so it's easier to read.



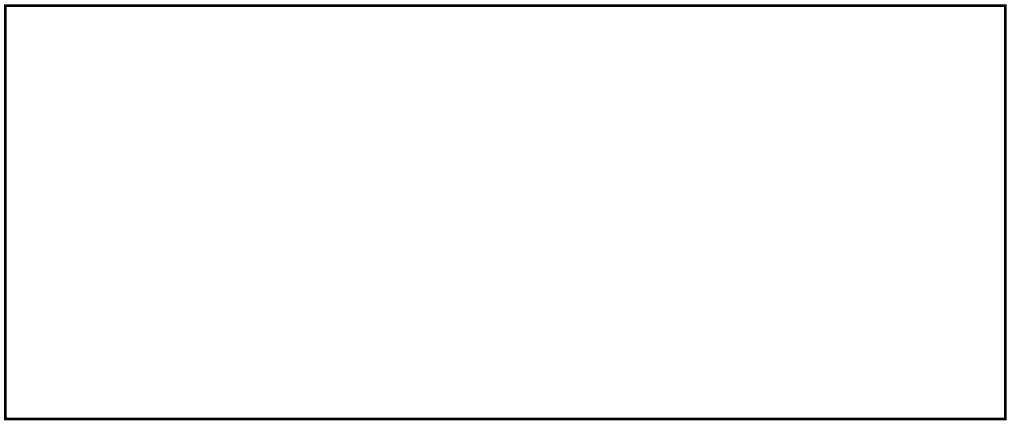
Response body at codebeautify.org/jsonviewer

But there's a better solution: apps that make your HTTP messaging easier.

Exploring metmuseum.org

Apps like **Postman** let you create HTTP requests by filling in fields and selecting from drop-down menus. You can pretty-print responses, and this also gives you syntax highlighting.

➤ Download Postman (https://bit.ly/3VTp6B4) for your Mac's chip and open the app. If you don't want to create an account, click **Skip and go to the app**. Then, in **Scratch Pad**, under **Get started**, click **Create a request** and hide the side bar:



Postman ▸ Scratch Pad ▸ Get started ▸ Create a request

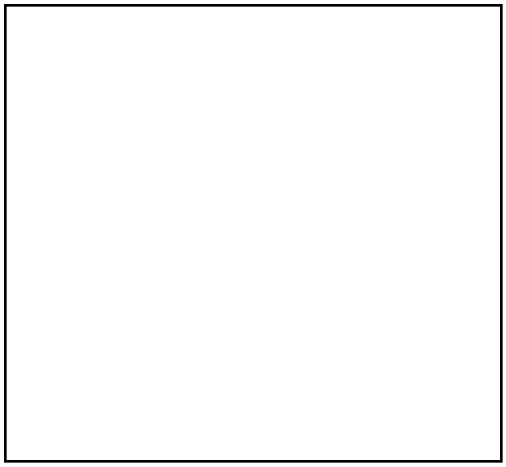
Requesting Objects

➤ Open **Postman** and enter this URL in the GET field:

```
https://collectionapi.metmuseum.org/public/collection/v1/
objects/437133
```

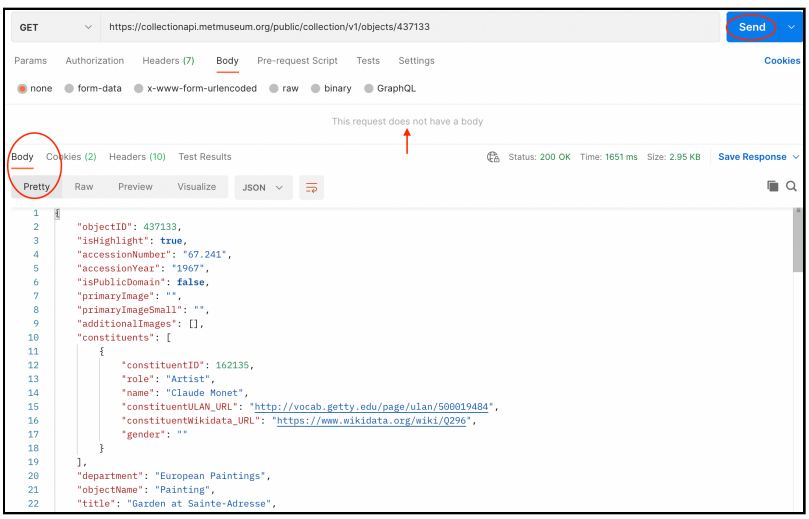
You set the *resource endpoint*. How do you know what to ask for? Scroll down in metmuseum.github.io: In the **Endpoints** section, you'll see four API endpoints: Objects, Object, Departments and Search. Scroll further to see, for each endpoint, its description, request parameter list, request endpoint, response field list and examples of requests and responses.

The URL you entered in the GET field is an **Object** request for the example **objectID**:



Object request for objectID 437133

➤ Back in **Postman**, click **Send**, then drag the response window up and select **Body** and **Pretty**:



Pretty view of response body

Scrolling through the response body, it's much easier to see all the keys in the top level dictionary: `objectID`, `isHighlight` and so on. Your app needs only a few of these keys.

➤ Click **Headers** to see the status code 200 OK, response time, size and other response headers:



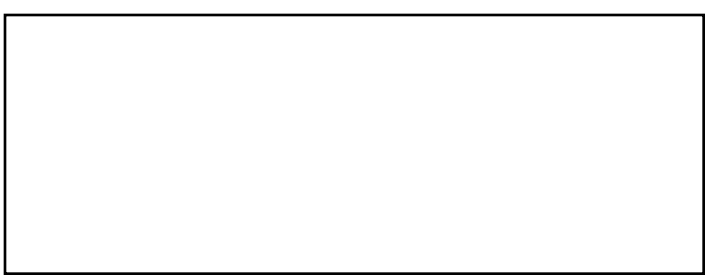
Response headers

Content-Type is `application/json; charset=UTF-8`. The key information here is **json**. This tells you how to decode the response body. In the next chapter, you'll use `JSONDecoder` to extract the attributes you want and store them in the `Object` structure so you can display them in your app.

> **Note**: **UTF-8** string encoding is a version of Unicode that is very efficient for storing regular text, but less so for special symbols or non-Western alphabets. Still, it's the most popular way to deal with Unicode text today.

Media URLs

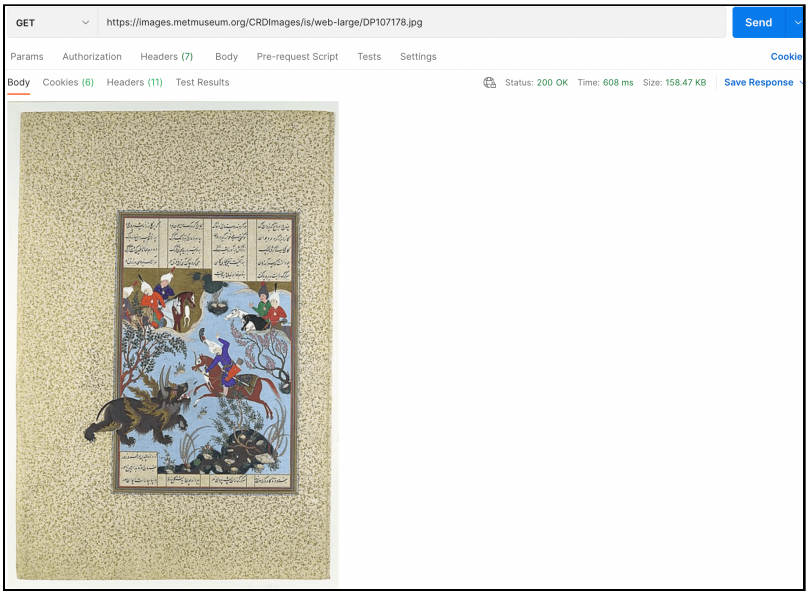
➤ Go back to the **Body** tab: This object isn't in the public domain, so it has empty strings for its primary image values. In the **GET** field, replace the object ID 437133 with our old friend **452174**, then click **Send**:



Rhino-wolf object

This is the “Bahram Gur Slays the Rhino-Wolf” object. In the previous chapter, you used `AsyncImage` to download its `primaryImageSmall`.

➤ The value of the `primaryImageSmall` key is a link. Click it to insert it into the GET field, then send the request.



Primary image

Postman is able to display the image. You can also **Command-click** the link to open it in your browser.

➤ Select the response’s **Headers** tab.

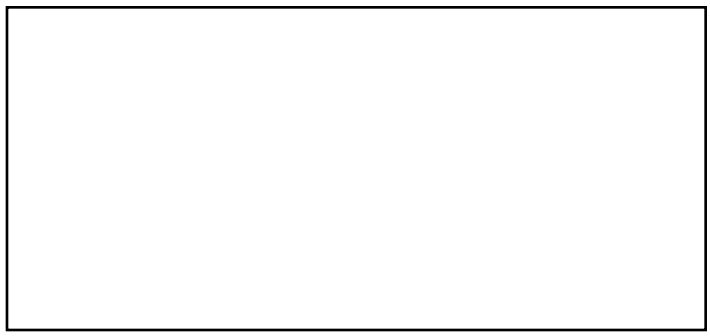


Content-Type: image/jpeg

The **Content-Type** is now `image/jpeg`.

➤ At the top-level, click the + to add a new request and send a request for this URL:

```
https://collectionapi.metmuseum.org/public/collection/v1/search?
q=rhino
```



Search for 'rhino'.

This request has a query parameter with key `q` and value `rhino`. The response contains an `objectIDs` array and the `total` number of object IDs in the array.

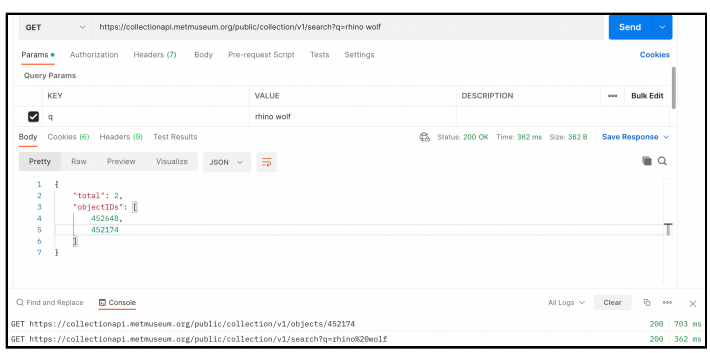
This is the request you'll send from your app when the user enters a query term. Then, to display the list of matching objects, you'll request the object for each object ID in the `objectIDs` array.

URL-encoding

➤ Now, send a search request for "rhino wolf":

```
https://collectionapi.metmuseum.org/public/collection/v1/search?
q=rhino wolf
```

➤ Below the response window, open the **Console**:



Search for 'rhino%20wolf'.

The console shows the "official" request URL, where the query string is actually `rhino%20wolf`.

Postman *URL-encoded* the space between `rhino` and `wolf` to `%20`. URLs sent over the internet can contain only letters, digits and these punctuation marks: `-`, `_`, `.` and `~`.

Other punctuation marks, including `/`, `?` and `%`, are encoded as a pair of hexadecimal digits preceded by the escape character `%`. The hexadecimal value is the character's byte value in ASCII, for example, 20 (32 in decimal) for the space character and 25 (37 in decimal) for the `%` character. The space character can also be encoded as `+`. For a non-ASCII character, URL-encoding uses its UTF-8 byte value.

When `/` and `?` are delimiters in the URL, they don't get encoded.

POST Request & Authentication

TheMet doesn't need anything from this section, but your future apps might.

TheMet only needs to GET resources from the server, and your users don't need to authenticate.

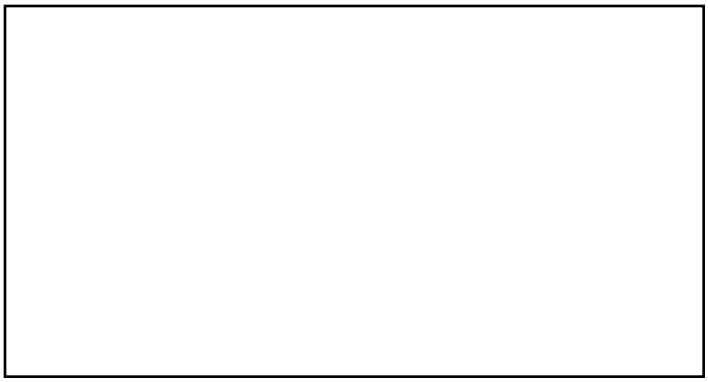
You usually need to implement authentication for apps that let users access restricted materials or create, update or delete server records. If you're building an app that requires this capability, consider using **Sign In with Apple**. You can learn more by following our video course Sign in with Apple (https://bit.ly/3VOGbMG).

To try out a POST request, you'll use Postman to send something like this GitHub `curl` example:

```
curl -i -H \
  "Authorization: token
5199831f4dd3b79e7c5b7e0ebe75d67aa66e79d4" \
  -d '{ \
      "name": "blog", \
      "auto_init": true, \
      "private": true, \
      "gitignore_template": "nanoc" \
    }' \
  https://api.github.com/user/repos
```

This example shows how to create a new GitHub repository, so it requires GitHub-user authentication. Remember when you set up your GitHub account in Xcode, you had to generate a personal access token? You'll need one here, too.

➤ If you haven't saved a plain-text copy of your GitHub personal access token, generate a new one (https://bit.ly/2Y71Ofh) with **repo** scope:



GitHub access token with repo scope

➤ In Postman, select the request's **Authorization** tab, then select **Bearer Token** from the **Type** menu. Paste **your** personal access token in the Token field:



Authorization tab with (dummy) bearer token

➤ Set the method to **POST** and the endpoint to **https://api.github.com/user/repos**. In the **Body** tab, select **form-data** and set the following keys and values:

- KEY: **name**, VALUE: **api-test-repo**
- KEY: **auto_init**, VALUE: **true**



POST request data: Form-encoded

➤ Click **Send**:



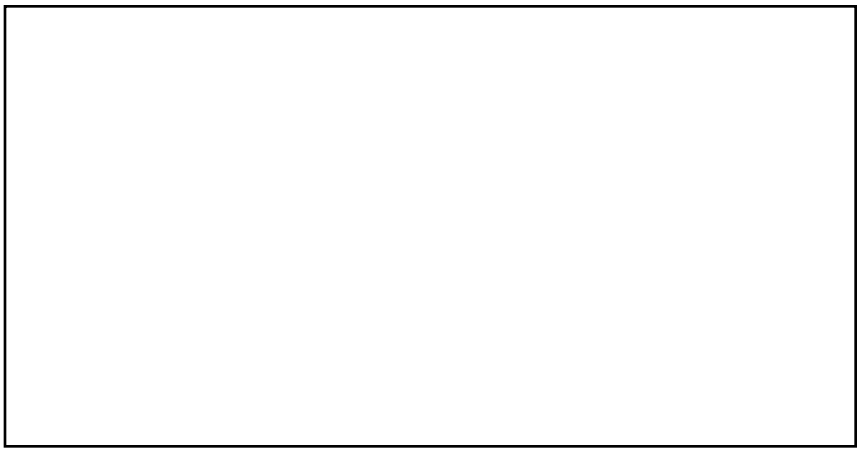
Response: 400 Bad Request. Problems parsing JSON

This was a deliberate "oops" to show you what happens if the server expects the POST request body to be in JSON but you send form-encoded data instead. You get a helpful link to a `documentation_url`.

➤ In the request Body tab, select **raw**, check the format is **JSON**, then type this in the text view:

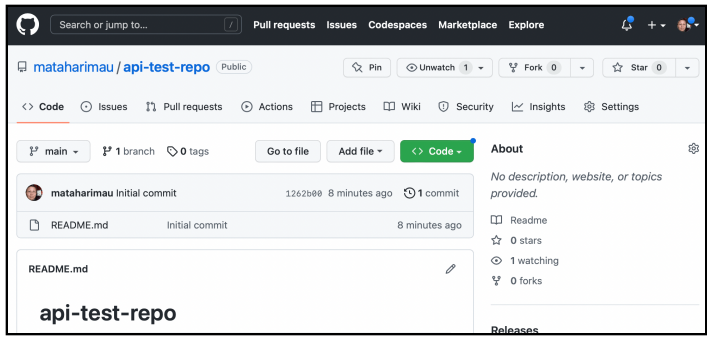
```
{
  "name": "api-test-repo",
  "auto_init": true
}
```

➤ Click **Send**:



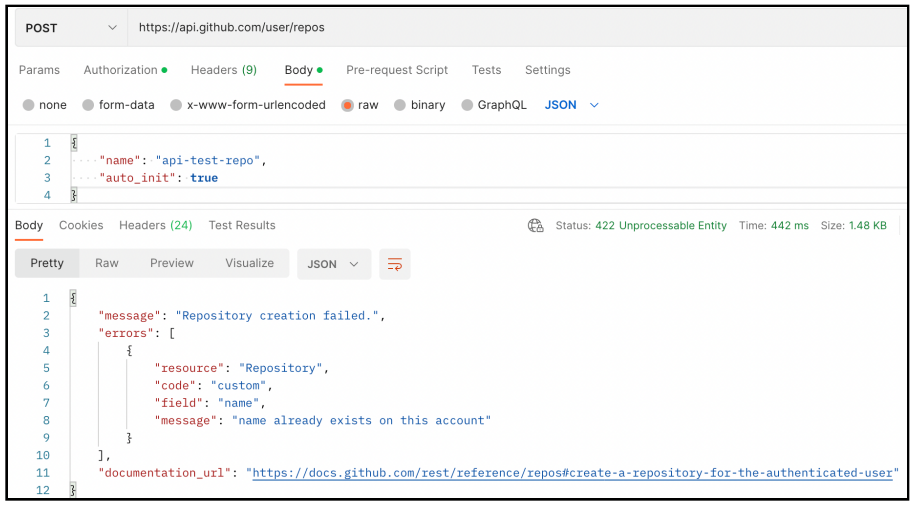
Response to POST request with JSON data: 201 Created

That worked! Check your GitHub account to see there really is a new repository named **api-test-repo**:



GitHub: New repository created

➤ Click **Send** again.



GitHub: 422 Unprocessable Entity

If you try to create the same repo again, the server returns the error message “name already exists on this account”.

Key Points

- Client apps send HTTP requests to servers, which send back responses.
- An HTTP response from an API endpoint contains a status code and some content. Text content is usually in JSON format and may contain URIs the client app can use to access media resources.
- HTTP requests follow the rules of the server's REST API, whose documentation specifies resource endpoints, HTTP methods and headers, and how to construct POST and PUT request bodies.
- You can send simple GET requests in a browser app. Use cURL or an app like Postman to create and send requests and inspect responses.

Chapter 24: Downloading Data

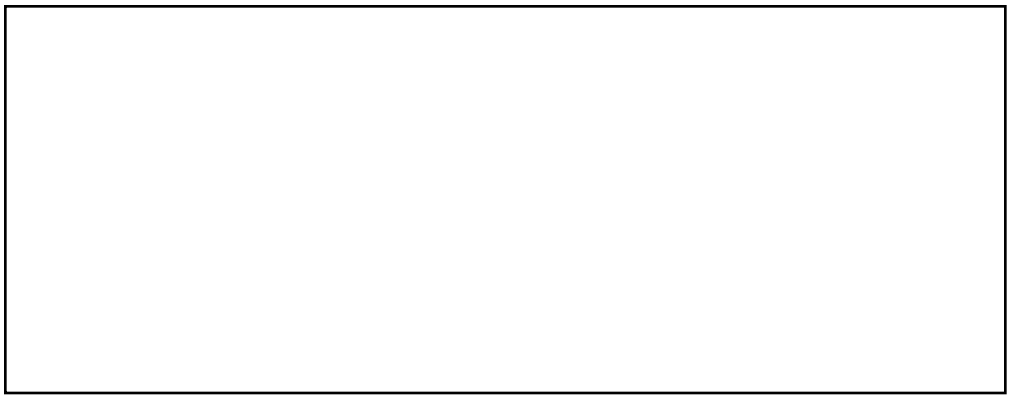
By Audrey Tam

Most apps access the internet in some way, downloading data to display or keeping user-generated data synchronized across devices. TheMet app needs to create and send HTTP requests and process HTTP responses. Downloaded data is usually in JSON format, which your app needs to decode into its data model.

If your app downloads data from your own server, you might be able to ensure the JSON structure matches your app's data model. But TheMet needs to work with the metmuseum.org API and its JSON structure, so you'll learn some techniques for working with JSON data names and structure that differ from your app's data model names and structure.

Getting Started

Open the **DownloadingData** playground in the starter folder. If the editor window is blank, show the Project navigator (**Command-1**) and select **DownloadingData** there.



Open DownloadingData playground.

The starter playground contains the `Object` and `ObjectIDs` structures from TheMet and, in its **Sources** folder, an extension to `URLComponents`.

Playgrounds are useful for exploring and working out code before moving it into your app. You can quickly inspect values produced by code and methods without needing to build a user interface or search through a lot of debug console messages.

URLSession

`URLSession` is Apple's framework for HTTP messages. Apple's documentation page includes this note:

> **Note**: The `URLSession` API involves many different classes that work together in a fairly complex way which may not be obvious if you read the reference documentation by itself.

To send a request to a server, you need to perform several steps:

1. Specify a `URLSession` to coordinate data-transfer tasks. For the simple download tasks in TheMet, you'll use the built-in `shared` session, which provides a reasonable default behavior.
2. Create a `URL` from a `String` like `https://metmuseum.org`. This could fail — for example, if the string is empty.
3. Create a `URLRequest` from this `URL`: This object contains the HTTP method and headers and other request properties.
4. Send the `URLRequest` in the `shared` session and `await` a `Data` instance and a `URLResponse` object.
5. Decode the data into your app's data model: Most REST APIs send JSON data. You'll use a `JSONDecoder` to decode this into your data model. The code is much simpler than what you did in Chapter 19, "Saving Files" because TheMet needs only a few properties from the metmuseum.org API, and its JSON names and structures are a good match for TheMet's data model.

> **Note**: `URLSession` and the broader topic of networking have their own video courses: Beginning Networking with URLSession (https://bit.ly/3GtY2DE) and Advanced Networking with URLSession (https://bit.ly/3GwHE5m).

Asynchronous Methods

Most `URLSession` methods involve network communication, so you can't predict how long they'll take to complete. In the meantime, the system must continue to interact with the user. To make this possible, `URLSession` methods are *asynchronous*: They dispatch their work onto another *queue* and immediately return control to the *main queue*, so it can respond to user interface events. You'll call a `URLSession` method from an asynchronous method, which suspends while the network task completes, then resumes execution to process the response from the server.

> **Note**: Learn more about asynchronous methods and concurrency in our book Modern Concurrency in Swift (https://bit.ly/3i15FrM) and its companion video courses Modern Concurrency: Getting Started (https://bit.ly/3If528z) and Modern Concurrency: Beyond the Basics (https://bit.ly/3vtbIsd).

Creating a REST Request URL

A REST request URL often includes query parameters. Here's one from the metmuseum.org's API (https://metmuseum.github.io/):

```
https://collectionapi.metmuseum.org/public/collection/v1/search?
medium=Quilts|Silk|Bedcovers&q=quilt
```

This URL lists query parameter names and values after the `?` separator. The query parameter `medium=Quilts|Silk|Bedcovers` matches any object whose `medium` is Quilts or Silk or Bedcovers. You can send the request just like this in Postman or Safari but, to send it from your app, the `|` character must be URL-encoded as `%7C`:

```
https://collectionapi.metmuseum.org/public/collection/v1/search?
medium=Quilts%7CSilk%7CBedcovers&q=quilt
```

You certainly don't want to do the URL-encoding yourself! Fortunately, you can hand this work over to `URLComponents` and `URLQueryItem`. You'll use these in this playground to create a flexible approach to composing REST requests, so you can easily change query parameter values.

URLComponents

`URLComponents` enables you to construct a URL from its parts and, also, to access the parts of a URL. Components include `scheme`, `host`, `port`, `path`, `query` and `queryItems`. `URL` itself gives you access to URL components like `lastPathComponent`.

➤ Add this code to the playground:

```
let baseURLString = "https://collectionapi.metmuseum.org/public/
collection/v1/"
var urlComponents = URLComponents(
  string: baseURLString + "search")!
urlComponents.queryItems = [
  URLQueryItem(name: "medium", value: "Quilts|Silk|Bedcovers"),
  URLQueryItem(name: "q", value: "quilt")
]
urlComponents.url
urlComponents.url?.absoluteString
```

You set the URL string for the API's base endpoint and add the `search` endpoint to create a `URLComponents` instance. Then, you create an array of `URLQueryItem` values. The `URLQueryItem` parameters are the parameter names and values in the sample request.

The last line displays the final URL *string* in the sidebar. The line above it displays the final URL. What's the difference? Time to find out!

> **Note**: In a playground, you can write an expression on its own line to display its value.

➤ Click the **Execute Playground** arrow on the last line number or at the bottom of the playground:

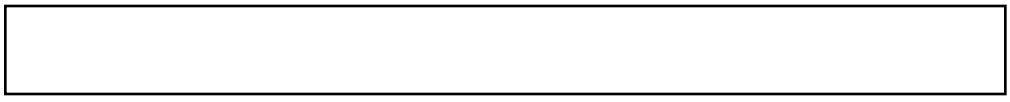


Execute-Playground arrows on the code line and in the bottom bar

> **Note**: Clicking the arrow next to a line of code runs the playground only up to that line.

The sidebar displays values for some lines with buttons for **Quick Look** and **Show Result**.

➤ Click the **Show Result** button of the last code line to show the result below the code line. You can click the display window to resize it, if necessary.

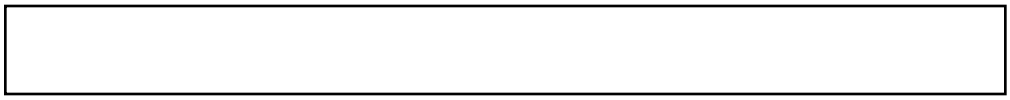


Show Result of the code line.

```
"https://collectionapi.metmuseum.org/public/collection/v1/
search?medium=Quilts%7CSilk%7CBedcovers&q=quilt"
```

Thanks to `urlComponents`, your query parameters are safely URL-encoded and appended to the base URL.

➤ Now look at the `url` on the line above. Notice it's not in quotation marks, because it's not a `String`. In fact, it's an `Optional`. Click its **Show Result** button:



Playground trying to display a URL

The playground tries its best to open the URL.

You can create a `URL` from a `String`, if the `String` has all the right parts. Then, you can access these parts as properties of the `URL` instance: `host`, `baseURL`, `path`, `lastPathComponent`, `query` and so on.

If you try to create a `URL` from a `String` that wouldn't work in a browser, the initializer returns `nil`. That's why `urlComponents.url` is an `Optional` and there's a `url?` in the last code line: If `url` is `nil`, it doesn't have an `absoluteString` property.

➤ Click their **Hide Result** buttons to close the result windows.

> **Note**: You can also use `print(urlComponents.url?.absoluteString)` to see the printed value in the Debug area below. If you're not able to see the Debug area, click the button in the lower right corner or press **Shift-Command-C**.

URLComponents Helper Method

`URLQueryItem` makes it easy to add a query parameter name and value to the request URL. The `name` and `value` arguments of `URLQueryItem` look like dictionary `key` and `value` items, so it's easy to create a dictionary of parameter names and values, then transform this dictionary into a `queryItems` array. It's especially easy when Alfian Losari has already done it (https://bit.ly/3pRtT6t). :] It's in **Sources/URLComponentsExtension.swift** in this playground.

➤ Replace the `urlComponents.queryItems` definition with:

```
var baseParams = [
  "hasImages": "true",
  "q": ""
]
urlComponents.setQueryItems(with: baseParams)
```

You create a dictionary whose keys are query parameter names. The first parameter `hasImages` matches any object whose web page displays at least one image. You include `q` (search term) with the default value `""`, and this dictionary lets you easily change its value.

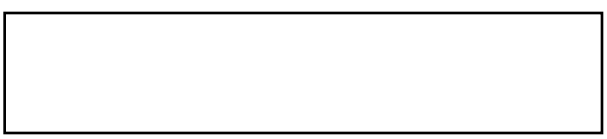
The `setQueryItems(with:)` helper method defined in the `URLComponents` extension creates a `URLQueryItem` for each dictionary item and sets the `queryItems` array of the `URLComponents` instance.

➤ To set a query term, add this code below `urlComponents.url?.absoluteString`:

```
baseParams["q"] = "rhino"
urlComponents.setQueryItems(with: baseParams)
urlComponents.url?.absoluteString
```

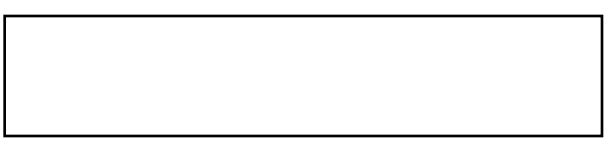
You're adding the query parameter `q=rhino` to the request URL by setting `baseParams["q"] = "rhino"` then calling `setQueryItems(with:)`.

➤ Execute the playground and show the `absoluteString` results:



hasImages first, q second

Or the parameters might be ordered the other way around:



q first, hasImages second

Because `baseParams` is a dictionary, you can't control the order of its elements.

➤ Copy and paste your `absoluteString` into your browser to see what you get.

If `q=rhino` is at the end of your `absoluteString`, you'll get:

```
{"total":3,"objectIDs":[452648,241715,452174]}
```

The response body contains JSON data that maps directly into your `ObjectIDs` structure. Only six objects match `q=rhino`, and only three of these have images.

If `q=rhino` appears before `hasImages=true`, you'll get:

```
{"total":103,"objectIDs":
[551786,852562,472562,317877,544740,329077,437422,459027,459028,
544320,200668,824771,438821,460281,310453,53660,452102,207157,23
7451,450605,549236,451725,436607,435997,436098,712539,192770,399
01,435848,776714,439327,204587,197460,197461,448280,485416,38388
3,334030,811772,811771,687615,377933,436102,452032,437059,850659
,430812,736196,626692,759529,822751,435702,435621,452740,40080,4
36658,488221,764636,436105,39742,437585,228995,437878,60470,4523
64,228990,200840,53238,838076,53162,436838,436803,452651,437868,
453385,201718,437174,437508,435991,464118,451287,436884,436885,4
35864,437368,438779,73651,44759,436529,435844,437873,341703,4371
59,453895,437173,844492,39895,436099,733847,437936,450761,435678
,437061]}
```

This behavior is … unexpected. And not correct: None of these objects matches `q=rhino`. It seems the metmuseum.org API has an undocumented requirement that the `q` parameter must be the final parameter. So, for this API, you'll need to add the `q` parameter manually.

➤ In the `baseParams` initialization, delete `q=""` and the comma at the end of `"hasImages": "true"`:

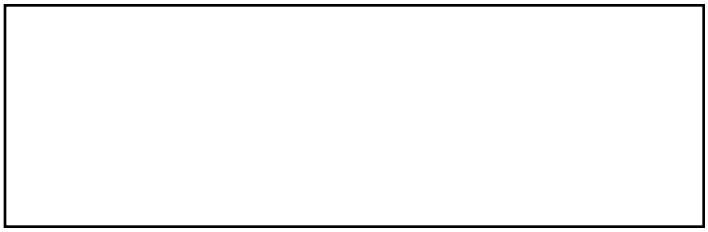
```
var baseParams = [
  "hasImages": "true"
]
```

➤ Replace the last three lines with this code:

```
let queryTerm = "rhino wolf"
urlComponents.queryItems! += [URLQueryItem(name: "q", value:
queryTerm)]
urlComponents.queryItems
urlComponents.url?.absoluteString
```

The `queryItems` property of `urlComponents` is an array of `URLQueryItem` name-value pairs, so you add the `q` parameter as a single-item array. Because it's an array, the `q` parameter is always the final parameter, every time you run the playground. It's actually an optional, but you know it isn't `nil` because you already called `urlComponents.setQueryItems(with:)`, so it's safe to force-unwrap it.

➤ Execute the playground to check `urlComponents.url?.absoluteString`:



URL to search for `rhino wolf`

And `URLComponents` takes care of URL-encoding the space in `"rhino wolf"`.

Now, you're all set to send a request with this URL in the playground, then decode the data.

Sending the Request With URLSession

You've got your URL. Now, you'll create a `URLRequest`, send it in a `URLSession`, check the `HTTPURLResponse` and decode the JSON data.

➤ Add this code:

```
let queryURL = urlComponents.url  // 1
let request = URLRequest(url: queryURL!)
let session = URLSession.shared  // 2
let decoder = JSONDecoder()  // 3

Task {  // 4
  let (data, response) = try await session.data(for: request)
  guard
    let response = response as? HTTPURLResponse,
    (200..<300).contains(response.statusCode)
  else {
    print(">>> response outside bounds")
    return
  }
  let objectIDs = try decoder.decode(ObjectIDs.self, from: data)
// 5
  objectIDs.objectIDs
}
```

1. You create a `URLRequest` with `queryURL`. In this playground, you know `queryURL` is a valid URL, so it's safe to force-unwrap it. When this code is in a method, you would do this assignment in a `guard` statement and exit if the value is `nil`.

2. `URLSession` methods run in a session. The framework provides a `shared` session with default configuration that you can use for simple requests. If you need a custom configuration, you can create your own session. For example, here's how you'd create a session that waits 300 seconds for a network connection:

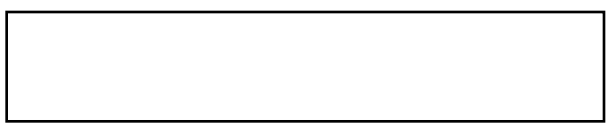
```
let config = URLSessionConfiguration.default
config.waitsForConnectivity = true
config.timeoutIntervalForResource = 300
let session = URLSession(configuration: config)
```

3. You create a `JSONDecoder` to decode the JSON data you're about to download.

4. `data(for:)` is asynchronous. To run it in a playground, you create an asynchronous `Task` to send the request, await the `data` and `response`, then check that the status code is in the success range.

5. Finally, you decode the data into an `ObjectIDs` instance, then display the array of object IDs.

➤ Execute the playground to see the IDs of the two matching objects:



IDs of matching objects

Next, you'll send requests for each of these objects but first, a brief discussion about decoding JSON.

Decoding JSON

If there's a good match between your data model and a JSON value, the default `init()` of `JSONDecoder` is poetry in motion, letting you decode complex structures of arrays and dictionaries in a single line of code. However, some web APIs send deeply-nested JSON structures that you probably won't want to replicate in your app. Then, you'll need to use `CodingKey` enumerations and custom `init(from:)` methods.

Decoding What You Need

In Chapter 19, "Saving Files", you saw how easy it is to encode and decode the `Team` structure as JSON because all its properties are `Codable`. You were saving and loading your app's own data model, so item names and structure in JSON format exactly matched your `Team` structure.

JSON values sent by real-world APIs rarely match the way you want to name or structure your app's data. Very often, your app doesn't need all the JSON values. You saw in the previous chapter the enormous number of fields in an object's record:



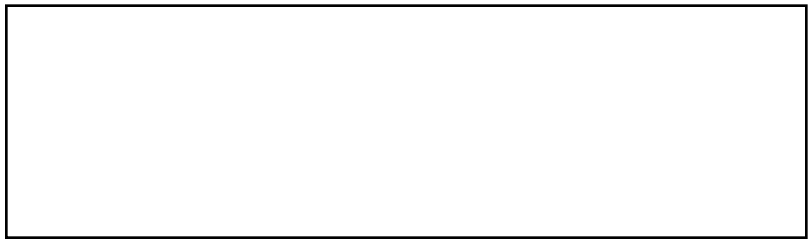
Response body at codebeautify.org/jsonviewer

The `Object` structure in your app contains only six of these:

```
struct Object: Codable, Hashable {
  let objectID: Int
  let title: String
  let creditLine: String
  let objectURL: String
  let isPublicDomain: Bool
  let primaryImageSmall: String
}
```

This will work perfectly fine: `JSONDecoder` will decode only these values from the response data and ignore the rest.

Note: If you need Swift structures and coding keys for the complete JSON schema, paste a sample response body into quicktype (https://app.quicktype.io) and select the **Swift** and **Struct** options.



Complete Object structure app.quicktype.io

Decoding When JSON Name != Property Name

JSON values sent by real-world APIs might not match the way you want to name or structure your app's data.

Many APIs use `snake_case` for JSON names, but Swift property names use `camelCase`. There's an easy fix; you simply tell the decoder to do the translation:

```
let decoder = JSONDecoder()
decoder.keyDecodingStrategy = .convertFromSnakeCase
```

This takes care of translating a JSON name like `artistWikidata_URL` to a Swift property name like `artistWikidataURL`.

If the JSON structure matches your app's data model, but some of the names are different, you simply have to define a `CodingKey` enumeration to assign JSON item names to your data model properties. For example, `Object` has a `primaryImageSmall` property that matches the JSON item's name. The value of this key is a URL string, so you might want to name your app's property `imageURL`. Here's how you'd do it:

```
enum CodingKeys: String, CodingKey {
  case imageURL = "primaryImageSmall"
  case objectID, title, creditLine, objectURL, isPublicDomain
}
```

Unfortunately, as soon as you create a `CodingKey` enumeration for *one* property name, you must include **all** the property names, even those that already match JSON item names.

Decoding Nested JSON

Many APIs send JSON data whose structure is very different from the way you want to organize your app's data. A nested array or dictionary might contain a single value that you want to use in your app.

Fortunately, this isn't the case with the metmuseum.org data you need for TheMet, although the `Object` record does contain a few arrays. Suppose you want to use the `Wikidata_URL` of one of the `term` items in the `tags` array:

```
"tags": [
    {
        "term": "Animals",
        "AAT_URL": "http://vocab.getty.edu/page/aat/300249525",
        "Wikidata_URL": "https://www.wikidata.org/wiki/Q729"
    },
    {
        "term": "Poetry",
        "AAT_URL": "http://vocab.getty.edu/page/aat/300055931",
        "Wikidata_URL": "https://www.wikidata.org/wiki/Q482"
    },
    {
        "term": "Men",
        "AAT_URL": "http://vocab.getty.edu/page/aat/300025928",
        "Wikidata_URL": "https://www.wikidata.org/wiki/Q8441"
    },
    {
        "term": "Horses",
        "AAT_URL": "http://vocab.getty.edu/page/aat/300250148",
        "Wikidata_URL": "https://www.wikidata.org/wiki/Q726"
    }
]
```

There are two approaches to decoding nested JSON:

1. **Define your data model to mirror the JSON value** and see how nifty automatic JSON decoding can be.
2. **Flatten the JSON value into your data model**: In exchange for more decoding work, you'll get sensible data structures that are easier and more natural to work with.

To see how to do this and much more, check out our tutorial Encoding and Decoding in Swift (https://bit.ly/3bqsrBY).

Downloading Objects

Now, back to your playground code, where you sent a query request then decoded the response data into an `ObjectIDs` instance. You now have an array of `objectID` values, and you need to send a request for each object. You'll store the downloaded objects in an array.

➤ Above the `Task` closure, add this line:

```
var objects: [Object] = []
```

You create an empty array of `Object` items.

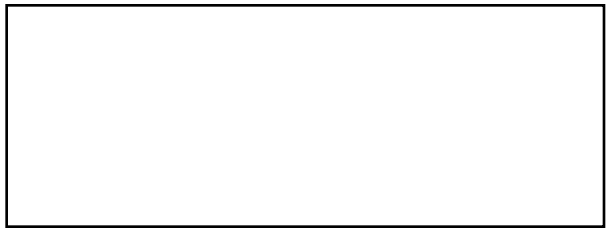
➤ In the `Task` closure, add this code:

```
for objectID in objectIDs.objectIDs {
  let objectURLString = baseURLString + "objects/\
(objectID)"  // 1
  let objectURL = URL(string: objectURLString)
  let objectRequest = URLRequest(url: objectURL!)
  let (data, response) = try await session.data(for:
objectRequest)  // 2
  guard
    let response = response as? HTTPURLResponse,
    (200..<300).contains(response.statusCode)
  else {
    print(">>> response outside bounds")
    return
  }
  let object = try decoder.decode(Object.self, from: data)  // 3
  objects.append(object)
}
objects
```

You loop over the values in `objectIDs.objectIDs`. For each `objectID`, you run code that's very similar to what you did for the query request:

1. You construct the endpoint URL for this `objectID` and use it to create a `URLRequest`.
2. You send the request, await the data and response, then check the status code.
3. You decode `data` into an `Object` instance, then append it to your `objects` array. To check the array, you display it.

➤ Execute the playground, show the result for `objects` and open an object:



Array of matching objects

Your JSON decoding is all working, and this is as much as you can test in a playground. You're ready to copy and adapt all this code into your app to make everything work!

Downloading Data in Your App

Your playground code is working fine, sending requests for objects that match a query term and decoding the JSON responses. Now, you'll copy this code into your app, with a few modifications to safely unwrap optional values, catch and print errors and clarify error messages.

➤ Open the project in this chapter's starter folder: It's the same as the final project from Chapter 22, plus **URLComponentsExtension.swift**. If you prefer to continue with your project, open the playground's navigation panel and drag this file into your project from the playground's **Sources** folder.

➤ In **Project navigator**, in the **Preview Content** group, delete **MetStoreDevData.swift**. You'll initialize the `objects` array from `URLSession` response data.

➤ In **TheMetStore.swift**, replace `init()` with this code:

```
func fetchObjects(for queryTerm: String) async throws {
}
```

Next, you'll use your playground code to implement some helper methods that `fetchObjects(for:)` will call. The helper methods will be asynchronous and might throw errors, so `fetchObjects(for:)` also has these keywords.

TheMetService

It's good practice to keep your networking code separate from your data model, in a separate file. And, it's common to use either "networking" or "service" in the filename.

➤ Create a new **Swift file** and name it **TheMetService.swift**. Below `import Foundation`, add this structure:

```
struct TheMetService {
  let baseURLString = "https://collectionapi.metmuseum.org/
public/collection/v1/"
  let session = URLSession.shared
  let decoder = JSONDecoder()

  func getObjectIDs(from queryTerm: String) async throws ->
ObjectIDs? {
    // insert code here
    return nil
  }

  func getObject(from objectID: Int) async throws -> Object? {
    return nil
  }
}
```

The first three lines are from your playground, and you'll adapt playground code to implement the two methods. `fetchObjects(queryTerm:)` in `TheMetStore` will first call `getObjectIDs(from:)`, then loop over the `objectIDs` array, calling `getObject(from:)` for each `objectID`.

Both methods are `async` because they'll call `session.data(for:)`, and both methods can rethrow errors thrown by `session.data(for:)`. The `JSONDecoder` can also throw errors, but these errors are extremely useful for finding any JSON-decoding problems, so you'll catch and print them right away. For now, both methods return `nil`, so the compiler doesn't complain.

getObjectIDs(from:)

➤ In `getObjectIDs(from:)`, insert this code *above* `return nil`:

```
let objectIDs: ObjectIDs?  // 1

guard
  var urlComponents = URLComponents(string: baseURLString +
"search")
else {  // 2
```

```
    return nil
  }
  let baseParams = ["hasImages": "true"]
  urlComponents.setQueryItems(with: baseParams)
  urlComponents.queryItems! += [URLQueryItem(name: "q", value:
  queryTerm)]
  guard let queryURL = urlComponents.url else { return nil }
  let request = URLRequest(url: queryURL)
```

1. You'll decode `data` into `objectIDs`, then return this structure.
2. You create the `URLRequest`, taking greater care to unwrap most of the optional values.

➤ Now, replace the final `return nil` with this code:

```
let (data, response) = try await session.data(for: request)  //
1
guard
  let response = response as? HTTPURLResponse,
  (200..<300).contains(response.statusCode)
else {
  print(">>> getObjectIDs response outside bounds")
  return nil
}

do {  // 2
  objectIDs = try decoder.decode(ObjectIDs.self, from: data)
} catch {
  print(error)
  return nil
}
return objectIDs  // 3
```

1. This is the playground code that calls `data(for:)`, awaits `data` and `response`, then checks the status code. You add `getObjectIDs` to the print message, so you know which method had the problem. Because `getObjectIDs` is an asynchronous method, it already runs in an asynchronous context, so you don't need to embed this code in a `Task`.
2. The decoder can throw errors, so you call it in a do-catch statement. You print the raw `error` value, as this gives you more information about what went wrong.
3. If execution reaches this line, everything has worked without errors, and you return `ObjectIDs`.

getObject(from:)

`fetchObjects(queryTerm:)` in `TheMetStore` will loop over the `objectIDs` array, calling `getObject(from:)` for each `objectID`.

➤ In `getObject(from:)`, replace `return nil` with this code:

```
let object: Object?  // 1

let objectURLString = baseURLString + "objects/\(objectID)"  //
2
guard let objectURL = URL(string: objectURLString) else { return
nil }
let objectRequest = URLRequest(url: objectURL)

let (data, response) = try await session.data(for:
objectRequest)  // 3
if let response = response as? HTTPURLResponse {
  let statusCode = response.statusCode
  if !(200..<300).contains(statusCode) {
    print(">>> getObject response \(statusCode) outside bounds")
    print(">>> \(objectURLString)")
    return nil
  }
}

do {  // 4
  object = try decoder.decode(Object.self, from: data)
} catch {
  print(error)
  return nil
}
return object  // 5
```

1. You'll decode `data` into `object`, then return this structure.
2. You create the `URLRequest`, taking greater care to unwrap the optional value. You don't need to use `URLComponent` to construct `objectURLString` because `objectID` is an `Int`, so there won't be any characters that need to be URL-encoded.

3. This is modified from the playground code that calls `data(for:)`, awaits `data` and `response`, then checks the status code. Some `objectID` values return 404-not-found, so you print the actual `statusCode` and the problem URL string.
4. The decoder can throw errors, so you call it in a do-catch statement.
5. If execution reaches this line, everything has worked without errors, and you return the resulting `Object`.

Now, head back to `TheMetStore` to use these two methods.

Fetching Objects in ContentView

`TheMetStore` is closely connected to your SwiftUI views — its main responsibility is to publish values for the views to present to users. It uses `TheMetService` to do this.

fetchObjects(for:)

➤ In **TheMetStore.swift**, add these properties to `TheMetStore`:

```
let service = TheMetService()
let maxIndex: Int
```

You create an instance of `TheMetService` so you can call its methods. `getObjectIDs(from:)` could return a very large array, so you'll use `maxIndex` to cap the number of calls to `getObject(from:)`. This is a courtesy to The Metropolitan Museum, which asks "Please limit request rate to 80 requests per second." You'll also use this value in the next chapter to keep your widget testing manageable.

➤ Next, add this initializer:

```
init(_ maxIndex: Int = 30) {
  self.maxIndex = maxIndex
}
```

You set 30 as the default `maxIndex` value. You don't need to change the `@StateObject` line in `ContentView` unless you want a different `maxIndex` value.

➤ Now, add this code to `fetchObjects(for:)`:

```
if let objectIDs = try await service.getObjectIDs(from:
queryTerm) {  // 1
  for (index, objectID) in objectIDs.objectIDs.enumerated()  //
2
  where index < maxIndex {
    if let object = try await service.getObject(from: objectID)
{
      objects.append(object)
    }
  }
}
```

1. First, you call `getObjectIDs(from:)` and wait for it to return `objectIDs`.
2. Then, you loop over `objectIDs.objectIDs` — at most `maxIndex` of them — calling `getObject(from:)` for each `objectID`. If it returns an `Object`, you append it to your `objects` array.

You're nearly there! Head back to **ContentView.swift** to call `fetchObjects(for:)`.

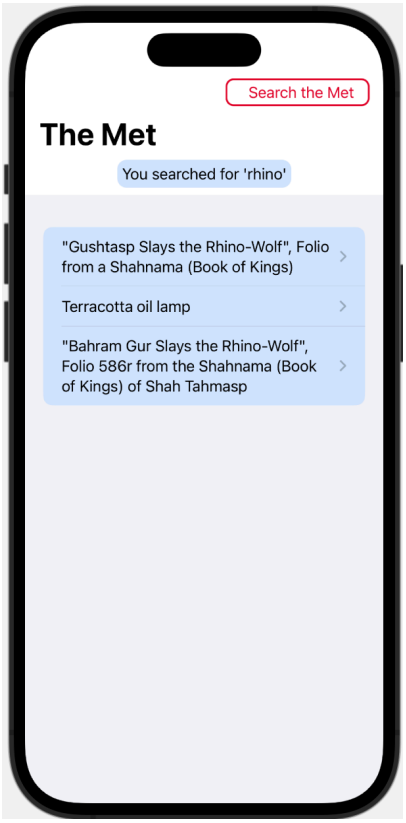
Sending the First Request

➤ In the `body` of `ContentView`, fold the `VStack` so you can see the closing brace of `NavigationStack`, then add this modifier to `NavigationStack`:

```
.task {
  do {
    try await store.fetchObjects(for: query)
  } catch {}
}
```

When the app starts, `ContentView` appears and this task runs. Because it modifies `NavigationStack`, it runs only once, no matter how often you navigate to `ObjectView` and back to `ContentView`.

➤ Refresh **Live Preview**:



Search for rhino when the app starts.

The app lists the three public-domain objects that match "rhino". Tap **Terracotta oil lamp** to see a rhino tossing a lion with its horn — that explains why this object matches "rhino".

Next, you need to call `fetchObjects(for:)` when the user enters a new query term.

Sending a New Request

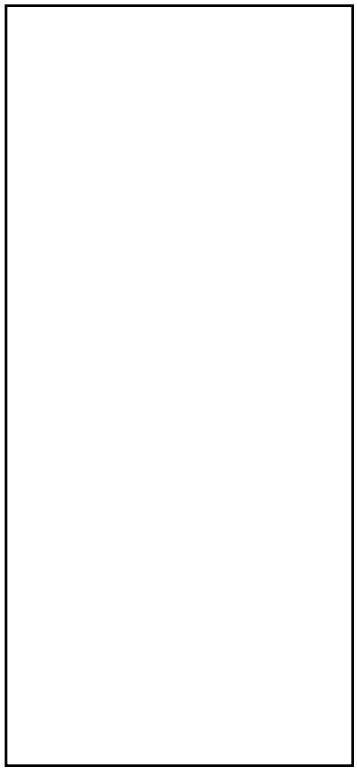
Tapping the **Search the Met** button shows an alert where you can enter a new query term. Tapping **Search** or the return key should call `fetchObjects(for:)`.

➤ Unfold the `VStack` and locate the `alert` modifier. Add this code inside the button's action closure:

```
Task {
  do {
    store.objects = []
    try await store.fetchObjects(for: query)
  } catch {}
}
```

SwiftUI views run synchronously, so you embed the call to `fetchObjects(for:)` in a `Task`.

➤ Build and run the project in a simulator, then tap the search button and enter a term like "persimmon":

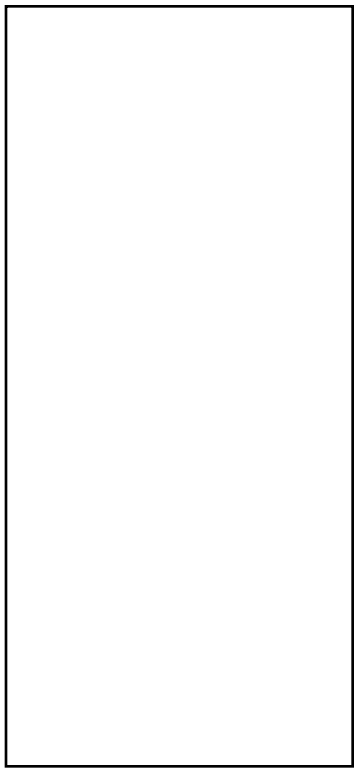


Search for persimmon.

You get a much longer list, including two non-public-domain objects that have images.

What if you decide to search for a different term while the app is still downloading objects for the current query term?

➤ Search for something that fetches **a lot** of objects, like "cat". While these are still downloading, search for "rhino" — you know this query returns only three objects.



Search for cat then search for rhino.

Although the alert button's action resets `objects` to the empty array, the "cat" task continues to run, so cat objects swamp the three rhino objects.

➤ Stop the project in Xcode to stop the cat downloads.

Canceling the Running Task

You need to cancel the running task before starting the next task. To cancel a task, you must give it a name.

➤ Add this property to `ContentView`:

```
@State private var fetchObjectsTask: Task<Void, Error>?
```

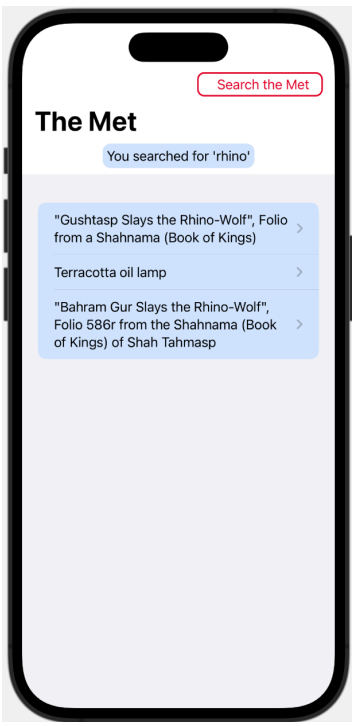
You'll store the alert button task in `fetchObjectsTask` and cancel it before you start the next task.

➤ In the alert button's action code, replace `Task {` with:

```
fetchObjectsTask?.cancel()
fetchObjectsTask = Task {
```

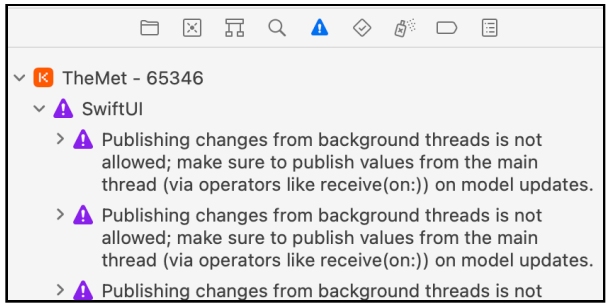
If there's a `fetchObjectsTask`, you cancel it. In any case, save the new `Task` to `fetchObjectsTask`.

➤ Build and run the project in a simulator, search for "cat", then search for "rhino":



Cancel search for cat before searching for rhino.

That worked! But, what are all these purple errors?

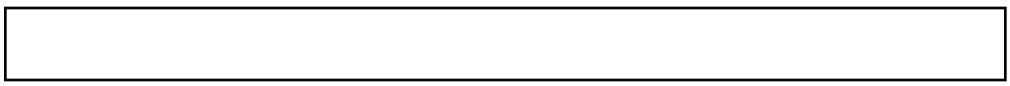


Publishing changes from background threads is not allowed...

Publishing Changes on the Main Thread

"Publishing changes ...": Whenever you call `fetchObjects(for:)` to run a new search, `store` publishes updates to `objects`, which updates the list in `ContentView`.

➤ Open **TheMetStore.swift**:



Purple published property

The purple points to the published property `objects`. `ContentView` subscribes to this property by creating an instance of `TheMetStore`. The asynchronous method `fetchObjects(for:)` changes `objects`, which changes `ContentView`.

"... from background threads": What's this about threads? Your app runs its code on threads — the main thread and one or more background threads. The user interface of every iOS app runs on the main thread. A SwiftUI app's user interface consists of SwiftUI views, so these always run on the main thread. The main thread must always be responsive to the user, so asynchronous methods run on a background thread to avoid slowing down or blocking the main thread.

Here's the problem: Any code that updates the user interface **must** run on the main thread. If an asynchronous method contains code that updates the user interface, it must somehow run that code on the main thread.

The "old concurrency" way to do this was to *dispatch* this code to the main queue. The new Swift concurrency way to do this is to run the code on `MainActor`.

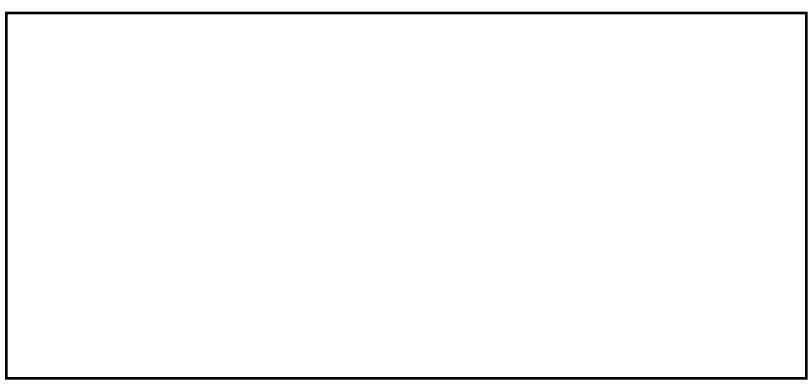
➤ In `fetchObjects(for:)`, replace `objects.append(object)` with this code:

```
await MainActor.run {
  objects.append(object)
}
```

This is the only line of code that **must** run on the main thread, so you embed it in a `MainActor.run` closure. You use `await` to call it asynchronously, so the system can suspend and resume execution on the correct actor.

> **Note**: You can annotate a method with `@MainActor` to ensure it runs on the main thread or annotate a property to ensure it can only be updated from the main thread. Or you can annotate an entire class with `@MainActor`, if almost all its properties and methods need to be on the main thread, then mark any exceptions with the `nonisolated` keyword. Learn more about Swift concurrency from our book Modern Concurrency in Swift (https://bit.ly/3i15FrM) or video courses Modern Concurrency: Getting Started (https://bit.ly/3If528z) and Modern Concurrency: Beyond the Basics (https://bit.ly/3vtbIsd).

➤ Build and run the project in a simulator, then enter a search term:



Purple problem solved.

Purple problem solved! Your app is working perfectly, but there's one last thing...

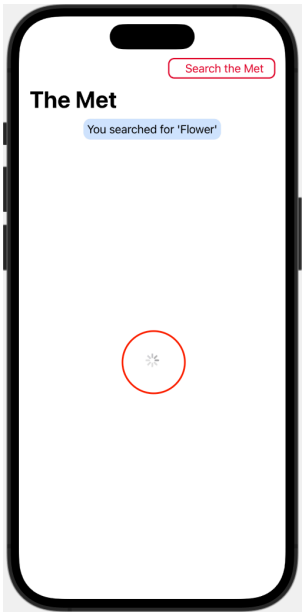
Showing a Progress View

After entering a new query term, there's a brief moment when the list is blank. Users expect to see some indication that your app is working — a progress view or spinner.

➤ In **ContentView.swift**, add this modifier to `VStack`:

```
.overlay {
  if store.objects.isEmpty { ProgressView() }
}
```

➤ Refresh Live Preview, then enter a new search term:



Progress view while objects is empty

A progress view spins until the first object appears.

Congratulations, you've built a working app for exploring the collections of The Metropolitan Museum of Art, New York. And, you're now ready to apply everything you've learned about `URLSession`, `URLComponents` and `JSONDecoder` to your own apps.

Key Points

- The `URLSession` API involves many different classes that work together in a fairly complex way. For simple download tasks, use the built-in `shared` session. Use `URLComponents` to create a URL-encoded `URL` from the endpoint string and query parameters, then create a `URLRequest` from this `URL`. Send this request with `data(for:)` and `await` the `Data` instance and `URLResponse` object. If the status code indicates success, use a `JSONDecoder` to decode the data into your data model.
- Create a separate structure to define your networking methods, then instantiate this in your data model.
- Asynchronous methods run on background threads. Any code that updates the user interface must run on the main thread. One way to do this is to call `MainActor.run`.
- Playgrounds are useful for working out code. You can quickly inspect values produced by methods and operations.

Chapter 25: Widgets

By Audrey Tam

Ever since Apple showed off its new home screen widgets in the 2020 WWDC Platforms State of the Union, everyone has been creating them. It's definitely a useful addition to TheMet, providing convenient and quick access to objects listed in your app.

> **Note**: The WidgetKit API continues to evolve at the moment, which may result in changes that break your code. Apple's template code has changed a few times since the 2020 WWDC demos. You might still experience some instability. That said, Widgets are cool and a ton of fun!

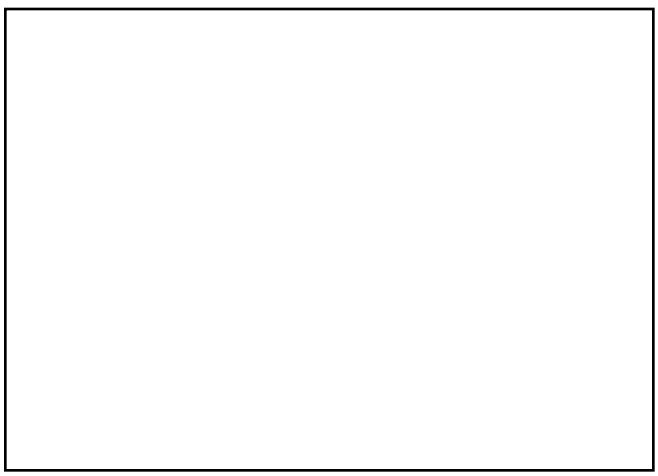
Getting Started

▸ Open the starter project or continue with your app from the previous chapter.

WidgetKit

WidgetKit is Apple's API for adding widgets to your app. The *widget extension* template helps you create a *timeline* of *entries*. You decide what app data you want to display and the time interval between entries.

You also define a view for each size of widget — small, medium, large, extra large — you want to support. The extra large size is available only in iPadOS.



Widget timeline

Here's a typical workflow for creating a widget:

1. Add a widget extension to your app. Configure the widget's display name and description.
2. Select or adapt a data model type from your app to display in the widget. Create a timeline entry structure — a `Date` plus your data model type. Create sample data for snapshot and placeholder entries.

3. Decide which widget sizes to support: Create small, medium or large views to display one or more data model values. In iOS 16, you can also create accessory views to display on the device's lock screen.
4. Create a timeline to deliver timeline entries. Decide on the *refresh policy*.

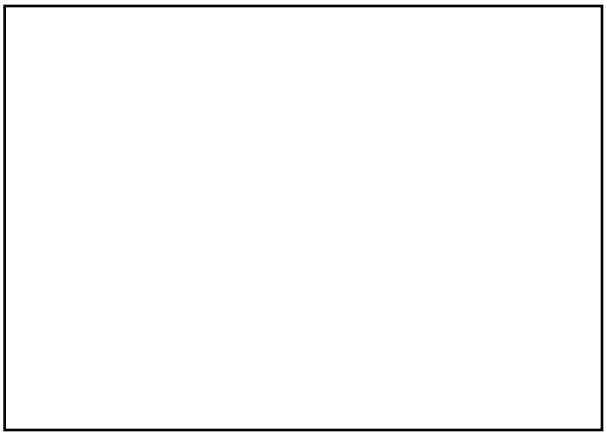
Adding a Widget Extension

➤ Start by adding a widget extension with **File ▸ New ▸ Target...**.



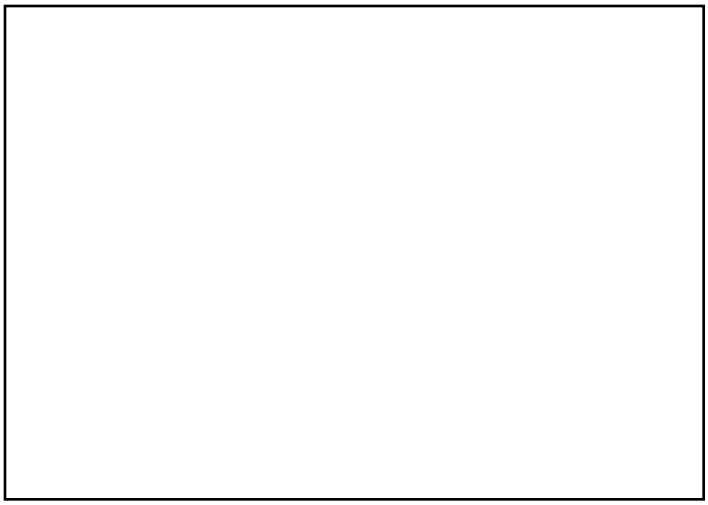
Create a new target.

➤ Search for **widget**, select **Widget Extension** and click **Next**:



Search for 'widget'.

➤ Name it **TheMetWidget**, select your team and make sure **Include Live Activity** and **Include Configuration Intent** are **not** checked:



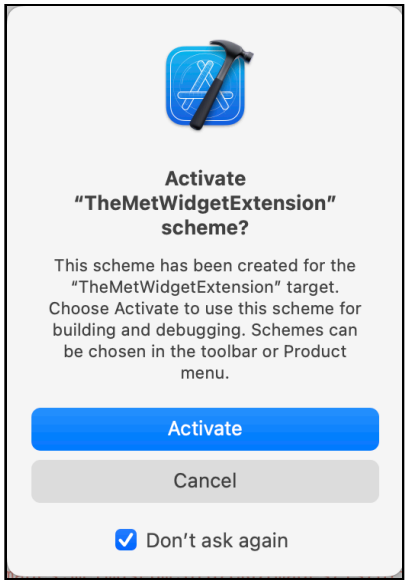
Don't select Include Live Activity or Include Configuration Intent.

A **Live Activity** display shows an app's most current data on the iPhone Lock Screen and in the Dynamic Island. This chapter doesn't implement Live Activity.

There are two widget configurations: **Static** and **Intent**. A widget with `IntentConfiguration` uses Siri intents to let the user customize widget parameters. Your widget for TheMet will be static.

> **Note**: `IntentConfiguration` is covered in our tutorial Getting Started With Widgets (https://bit.ly/2MS7K9U)

➤ Click **Finish** and agree to the activate-scheme dialog:



Activate scheme for new widget extension.

Configuring Your Widget

A new target group named **TheMetWidget** appears in the **Project** navigator. It contains two Swift files.

TheMetWidgetBundle.swift is like **TheMetApp.swift**, but it instantiates your widget instead of the first view of your app. The `@main` attribute means this is the widget's entry point.

```
@main
struct TheMetWidgetBundle: WidgetBundle {
  var body: some Widget {
    TheMetWidget()
  }
}
```

➤ Open **TheMetWidget.swift**, then find `TheMetWidget` and edit the last two modifiers: `configurationDisplayName(_:)` and `description(_:)`.

```
struct TheMetWidget: Widget {  // 1
  let kind: String = "TheMetWidget"

  var body: some WidgetConfiguration {
    StaticConfiguration(
      kind: kind,
      provider: Provider()  // 2
    ) { entry in
      TheMetWidgetEntryView(entry: entry)  // 3
    }
    // 4
    .configurationDisplayName("The Met")
    .description("View objects from the Metropolitan Museum.")
  }
}
```

Here's what the template code does:

1. The structure's name and its `kind` property are the name you gave it when you created it.
2. You define your widget's timeline, snapshot and placeholder entries in `Provider`.
3. You create your widget view(s) in `TheMetWidgetEntryView`.
4. In this structure, you only need to customize the name to **The Met** and the description to **View objects from the Metropolitan Museum.** Your users will see these in the widget gallery.

Doing a Trial Run

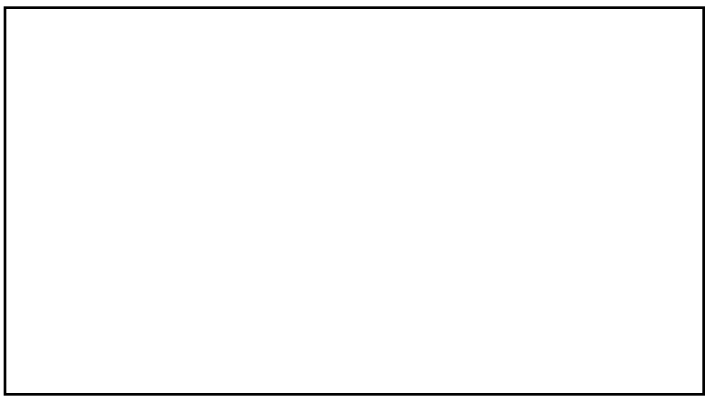
The widget template provides a lot of boilerplate code you simply have to customize. It works right out of the box, so you can try it out now to make sure everything runs smoothly when you're ready to test your code.

➤ You can try out your widget in a simulator. If you want to install your app on your iOS device, you need to sign both targets. In the **Project** navigator, select the top level **TheMet** folder. Use your organization instead of "com.yourcompany" in the bundle identifiers and set the team for each target.

> **Note**: Your widget's bundle ID prefix must be the same as your app's. This isn't a problem with TheMet but, if your project has different bundle IDs for Debug, Release and Beta, you'll need to edit your widget's bundle ID prefix to match.

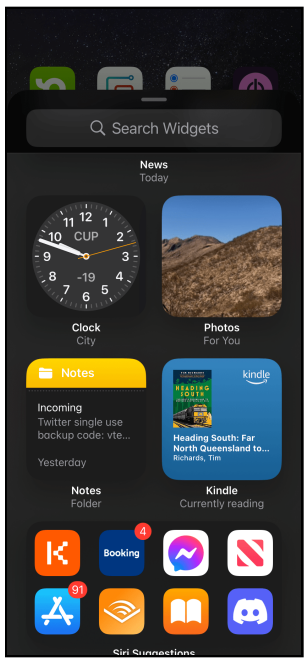
➤ TheMetWidget is a second target, and it's probably the currently selected scheme. Make sure you select the **TheMet** scheme, then build and run. Tap the **Home** button in the simulator tool bar to close the app, then press on some empty area of your home window until you see delete buttons on the app icons.

> **Note**: The app's scheme **TheMet** might disappear from the scheme menu. If this happens, select **Manage Schemes...** from the menu, then click **Autocreate Schemes Now**:



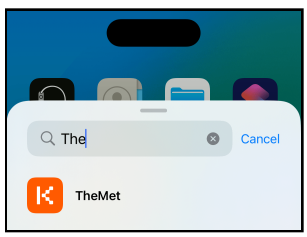
If necessary, autocreate TheMet scheme.

➤ Tap + in the upper left corner. If you've installed the app on a device, your gallery looks something like this:



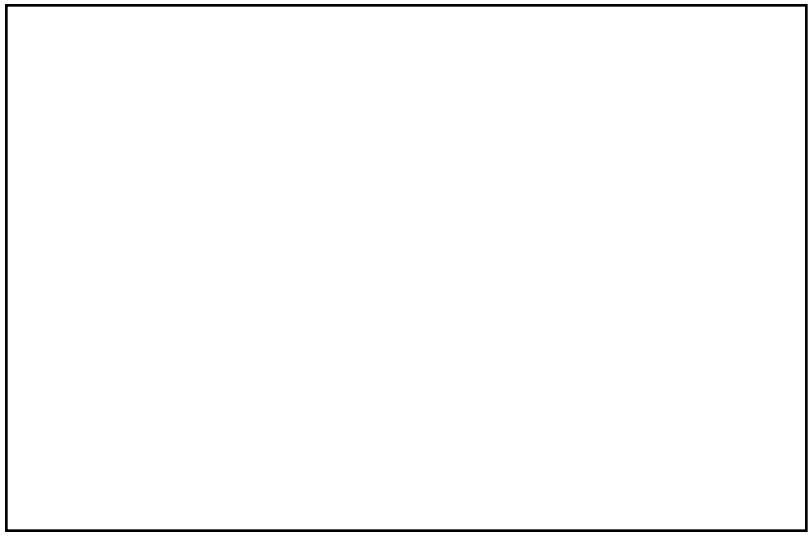
Widget gallery on iPhone

➤ The quickest way to find your widget is to start typing **TheMet** in the **Search Widgets** field:



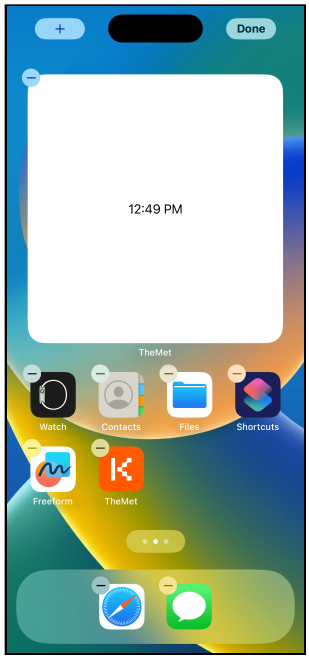
Search for your widget.

➤ Select **TheMet** to see snapshots of the three sizes:



Snapshots of the three widget sizes.

➤ Tap **Add Widget** to see your widget on the screen:



Your widget on the home screen.

➤ Tap **Done** in the upper right corner.

➤ Tap the widget to reopen TheMet.

Your widget works! Now, you simply have to make it display information from TheMet.

➤ Close the app then long-press the widget to open its menu and select **Remove Widget**. Get into the habit of removing the widget after you've confirmed it's working. This is especially important if you've installed the app on your device. While you're developing your widget, it will display a new view every three seconds, and that's a real drain on your battery.

Creating Entries From Your App's Data

It makes sense for your widget to display some of the information your app shows for each object, using the properties in **Object.swift**.

Adding App Files to the Widget Target

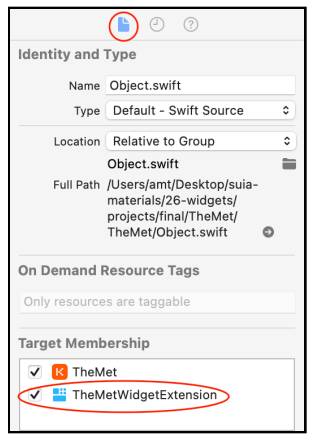
➤ In **TheMetWidget.swift**, find `SimpleEntry`. Add this line below `date`:

```
let object: Object
```

Your widget will display an object from the app's list.

An Xcode error appears, because the widget doesn't know about `Object`. You need to add **Object.swift** to the widget target.

➤ In **Project** navigator, select **Object.swift**. Show the **File** inspector and check the **Target Membership** box for **TheMetWidgetExtension**:



Add Object.swift to widget target.

Provider Methods

Adding the `object` property to `SimpleEntry` causes errors in `Provider` because it creates `SimpleEntry` instances in its methods `placeholder(in:)`, `getSnapshot(in:completion:)`, `getTimeline(in:completion:)` and also in the preview. `Provider` methods are called by WidgetKit, not by any code you write.

- To display your widget for the first time, WidgetKit calls `placeholder(in:)` and applies a `redacted(reason: .placeholder)` modifier to mask the view's contents. This method is *synchronous*: Nothing else can run on its queue until it finishes, so don't do any network downloads or complex calculations in this method.
- WidgetKit calls `getSnapshot(in:completion:)` whenever the widget is in a transient state, waiting for data or appearing in the widget gallery.
- WidgetKit calls `getTimeline(in:completion:)` to get an array of time-stamped entries to display.

Creating Sample Objects

First, you need a sample `Object` for the parameter value.

➤ In **Object.swift**, add this extension to `Object`:

```
extension Object {
  static func sample(isPublicDomain: Bool) -> Object {
    if isPublicDomain {
      return Object(
        objectID: 452174,
        title: "Bahram Gur Slays the Rhino-Wolf",
        creditLine: "Gift of Arthur A. Houghton Jr., 1970",
        objectURL: "https://www.metmuseum.org/art/collection/
search/452174",
        isPublicDomain: true,
        primaryImageSmall: "https://images.metmuseum.org/
CRDImages/is/original/DP107178.jpg")
    } else {
      return Object(
        objectID: 828444,
        title: "Hexagonal flower vase",
        creditLine: "Gift of Samuel and Gabrielle Lurie, 2019",
        objectURL: "https://www.metmuseum.org/art/collection/
search/828444",
        isPublicDomain: false,
        primaryImageSmall: "")
    }
  }
}
```

This method returns a sample object, either in the public domain or not. The method is static, so you can call it with `Object.sample(isPublicDomain: true)`.

➤ Now, in **TheMetWidget.swift**, fix the errors one by one, or use the handy shortcut **Control-Option-Command-F** for **Editor ▸ Fix All Issues** to insert the missing argument. Replace all the `Object` placeholders in **TheMetWidget.swift** with:

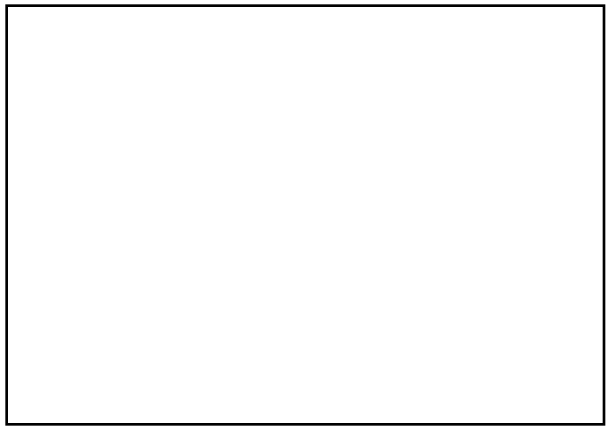
```
Object.sample(isPublicDomain: true)
```

Then, in `getSnapshot(in:completion:)`, change `true` to **false** so you'll be able to see which one appears in the widget gallery.

Creating Widget Views

When you've decided what data to display, you need to define a widget view to display it. It would be nice to display the primary image of an object in your widget view, but `AsyncImage(url:)` doesn't work in a widget, so you'll simply display the object's title.

➤ Create a new **SwiftUI View** file named **WidgetView.swift**, making sure you set **TheMetWidgetExtension** as its target:



Create new SwiftUI view file for widget view.

➤ Replace the content of `WidgetView` with this:

```
let entry: Provider.Entry

var body: some View {
  Text(entry.object.title)
}
```

The widget needs an `Entry` to display and, to start, you'll display the object's title.

➤ Now, fix the preview: Copy the content of `TheMetWidget_Previews` from **TheMetWidget.swift**, then change `TheMetWidgetEntryView` to `WidgetView`:

```
WidgetView(
  entry: SimpleEntry(
    date: Date(),
    object: Object.sample(isPublicDomain: true)))
  .previewContext(WidgetPreviewContext(family: .systemSmall))
```

➤ To use `WidgetPreviewContext`, you need to import `WidgetKit`:

```
import WidgetKit
```

➤ Refresh the preview to see what you've got:



Simplest small widget

> **Note**: Xcode might fail to build, complaining that "Embedded binary is not signed with the same certificate as the parent app" or "Reference to invalid associated type 'Entry' of type 'Provider'". Changing `Timeline<Entry>` to `Timeline<SimpleEntry>` in the `getTimeline(in:completion:)` signature in **TheMetWidget.swift** can get rid of this problem.

It works! Now, to make it look more like the app's list view, you'll need the `WebIndicatorView` from **ContentView.swift** and the `metBackground` color defined in **Assets** and **ColorExtension.swift**.

➤ From **TheMet** group, add **Assets.xcassets** and **ColorExtension.swift** to the widget target.

Instead of adding the whole **ContentView.swift** to the widget target, you'll move `WebIndicatorView` to a separate SwiftUI View file, along with `PlaceholderView`.

➤ In **TheMet** group, add a new **SwiftUI View** file named **SupportingViews.swift**, delete its structures, then move `WebIndicatorView` from **ContentView.swift** and `PlaceholderView` from **ObjectView.swift** into your new file. Add **SupportingViews.swift** to the widget target.

➤ Back in **WidgetView.swift**, add this structure:

```
struct DetailIndicatorView: View {
  let title: String

  var body: some View {
    HStack(alignment: .firstTextBaseline) {
      Text(title)
      Spacer()
      Image(systemName: "doc.text.image.fill")
    }
  }
}
```

The app displays a detail view for public-domain objects, with some text and an image. By the end of this chapter, you'll implement a deep-link to the object's detail view, so here you include a little system image to suggest what the user will see.

➤ Now, in `WidgetView`, replace `Text(entry.object.title)` with the following:

```
VStack {
  Text("The Met")  // 1
    .font(.headline)
    .padding(.top)
  Divider()  // 2

  if !entry.object.isPublicDomain {  // 3
    WebIndicatorView(title: entry.object.title)
      .padding()
      .background(Color.metBackground)
      .foregroundColor(.white)
  } else {
    DetailIndicatorView(title: entry.object.title)
      .padding()
      .background(Color.metForeground)
  }
}
.truncationMode(.middle)  // 4
.fontWeight(.semibold)
```

Here's what you're doing:

1. You can't use `NavigationStack` in a widget view, so you create your own title with `headline` font size and `top` padding to push it away from the top edge.
2. You add a divider line, to make it look more like a title.
3. You display the object's title so it looks similar to how it appears in the app's list.
4. You apply `truncationMode` and `fontWeight` to the `VStack` so it works for both `WebIndicatorView` and `DetailIndicatorView`.

➤ Refresh the preview to see your improved widget:



Small widget: public-domain object

A Group of Previews

You can preview both sample objects by creating a `Group`:

➤ In `WidgetView_Previews`, replace the contents of `previews` with:

```
Group {
  WidgetView(
    entry: SimpleEntry(
      date: Date(),
      object: Object.sample(isPublicDomain: true)))
```

```
    .previewContext(WidgetPreviewContext(family: .systemSmall))
    // non-public-domain sample object
    WidgetView(
      entry: SimpleEntry(
        date: Date(),
        object: Object.sample(isPublicDomain: false)))
    .previewContext(WidgetPreviewContext(family: .systemSmall))
  }
```

This embeds `WidgetView` in a `Group`, duplicates it and its `.previewContext modifier`, and changes `true` to `false`.

➤ In the preview canvas, select the second **Widget View** to see the `WebIndicator` view:



Small widget: non-public-domain object

➤ Now, in the second `WidgetView`, change `.systemSmall` to `.systemMedium`:



Medium widget: non-public-domain object

This is how your `WidgetView` layout looks in the medium size widget.

➤ To see how the medium size might be useful, go to **Object.swift** and replace the `title` of the public-domain object:

```
title: "\"Bahram Gur Slays the Rhino-Wolf\", Folio 586r from the
Shahnama (Book of Kings) of Shah Tahmasp",
```

This is the full title that the app downloads.

➤ And back to **WidgetView.swift** to change `.systemSmall` to `.systemMedium` in the *first* `WidgetView`:



Medium widget: public-domain object with very long title

➤ Finally, in the first `WidgetView`, change `.systemMedium` to `.systemLarge`:



Large widget: public-domain object with very long title

Supporting Widget Sizes

If you think one of the sizes looks best, or if you definitely don't want to support one of the sizes, you can restrict your widget to specific size(s). For TheMet, long titles look better in the medium or large size.

➤ In `TheMetWidget`, add this modifier to `StaticConfiguration`, below `description(_:)`:

```
.supportedFamilies([.systemMedium, .systemLarge])
```

Finally, you need to set up `TheMetWidgetEntryView` to use `WidgetView`. Replace the `body` contents with this code:

```
WidgetView(entry: entry)
```

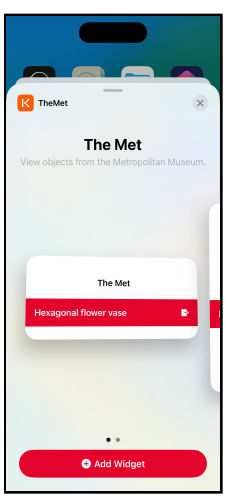
Here, you set `WidgetView` as the view to use when you want to display content.

➤ And in `previews`, change `.systemSmall` to `.systemMedium` or `.systemLarge`:

```
.previewContext(WidgetPreviewContext(family: .systemMedium))
```

➤ Make sure the scheme is **TheMet**, then build and run the app. After it launches, close the app.

If you had a small widget installed before this, it's now gone. And, when you add a widget, the small size isn't an option:

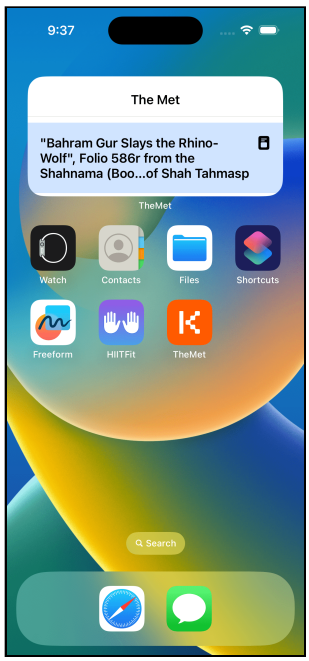


Widget gallery: medium or large

Notice the gallery uses the non-public-domain sample object, which means it's calling `getSnapshot(in:completion:)`.

> **Note**: If your widget doesn't appear in the gallery, or doesn't work correctly, delete the app then build and run again. If the problem persists, restart the simulator or device.

➤ Add a widget, then tap the screen or the **Done** button to exit screen-editing mode.



Widget displays timeline entry.

The widget view displays the `SimpleEntry` you set up in `getTimeline(in:completion:)`.

Providing a Timeline Of Entries

The heart of your widget is the `Provider` method `getTimeline(in:completion:)`. It delivers an array of time-stamped entries for WidgetKit to display. The template code creates an array of five entries one hour apart:

```
let currentDate = Date()
for hourOffset in 0 ..< 5 {
  let entryDate = Calendar.current.date(
    byAdding: .hour,
    value: hourOffset,
    to: currentDate)!
  let entry = SimpleEntry(
    date: entryDate,
    object: Object.sample(isPublicDomain: true))
  entries.append(entry)
}
```

This code creates each entry with the same `Object.sample`. You'll modify the method so it displays items in the app's `objects` array. Waiting an hour between entries is no good for testing purposes, so you'll shorten the interval to a few seconds.

First, you must populate your `objects` array.

Creating a Local TheMetStore

The quickest way — fewest lines of code — to get objects is to create an instance of `TheMetStore` in the widget.

➤ In **TheMetWidget.swift**, add these properties to `Provider`:

```
let store = TheMetStore(6)
let query = "persimmon"
```

While debugging, you limit the number of downloaded objects to a small number. You set the query term to something that returns objects with distinct titles.

➤ To get rid of the error flags, add **TheMetStore.swift**, **TheMetService.swift** and **URLComponentsExtension.swift** from **TheMet** group to the widget target.

➤ Now, in `getTimeline(in:completion:)`, replace the `for` loop with the following code:

```
let interval = 3

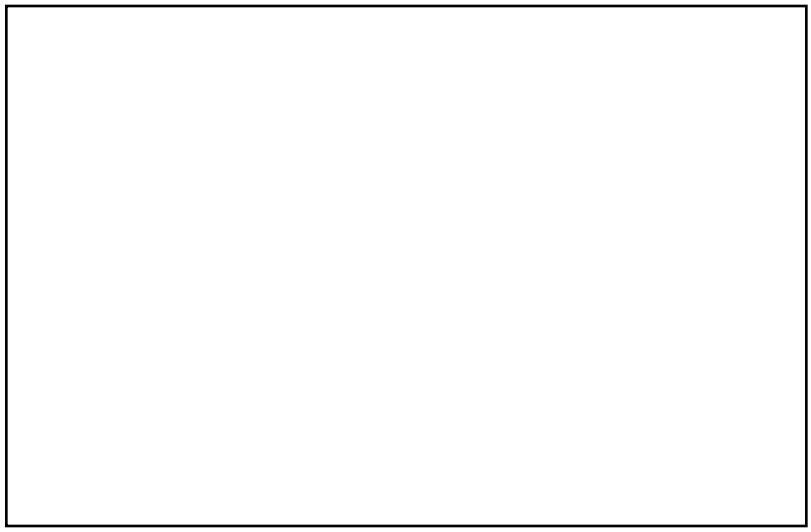
Task {  // 1
  do {
    try await store.fetchObjects(for: query)
  } catch {
    store.objects = [
      Object.sample(isPublicDomain: true),
      Object.sample(isPublicDomain: false)
    ]
  }
}

for index in 0 ..< store.objects.count {
  let entryDate = Calendar.current.date(
    byAdding: .second,  // 2
    value: index * interval,
    to: currentDate)!
  let entry = SimpleEntry(
    date: entryDate,
    object: store.objects[index])  // 3
  entries.append(entry)
}
```

1. You call `fetchObjects(for:)` to fill the `objects` array and use this to create an array of `SimpleEntry` values, three seconds apart. If `fetchObjects(for:)` fails, you fill the array with the two sample objects.
2. You change the interval between entries to 3 seconds.
3. You display an object from `store.objects`.

➤ At the top of `ContentView`, change `query` to **"persimmon"**: You download the same objects as the widget, so you can compare the app's list with what your widget displays.

➤ Build and run, then close the app. Look for your widget and add it. Then watch it display the first six persimmon objects:



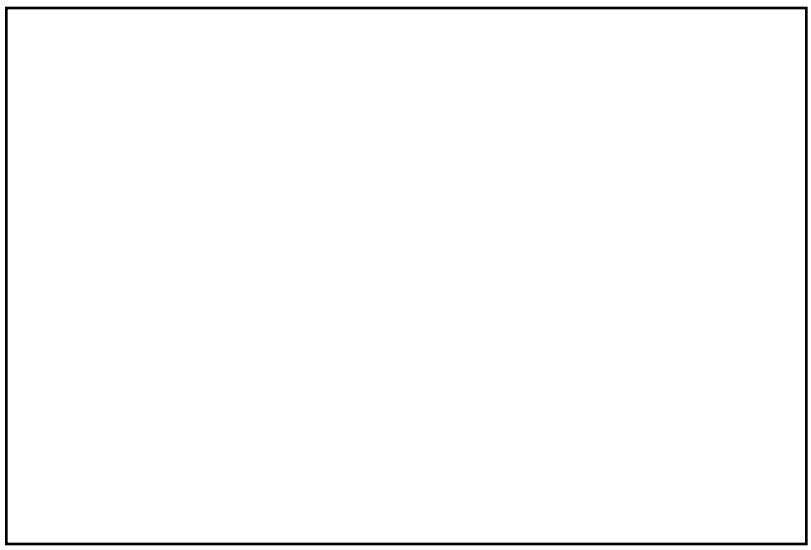
Widget showing persimmon objects

> **Note**: If you already had a Widget added in the home screen and it isn't showing the objects, remove it and add it again.

The widget might take a while to start displaying. In the meantime, it displays the placeholder view. If nothing happens after a couple of minutes, build and run the app again. After the sixth object, there's a longer interval while the widget re-fetches the same six objects.

> **Note**: At the time of writing, the widget doesn't work correctly on my iPhone. It displays the first object, but doesn't update.

➤ Tap the widget to reopen your app. Set a new query term, wait for the list to reload, then close the app. Your widget is still displaying persimmon objects:



Widget still showing persimmon objects

Your widget's `TheMetStore` is separate from your app's `TheMetStore`, so it's still using *persimmon* as the query term. You need to decide between these two design options:

1. Keep the widget's array in sync with the app's.
2. Allow the user to set a different query term for the widget.

Later in this chapter, you'll implement a deep link from the widget into your app to open the detail view of the widget's entry. This won't make sense if the widget's array could be different from the app's array. So this chapter chooses the first design option.

> **Note**: The second option requires you to create a widget with an `IntentConfiguration`, covered in our tutorial Getting Started With Widgets (https://bit.ly/2MS7K9U).

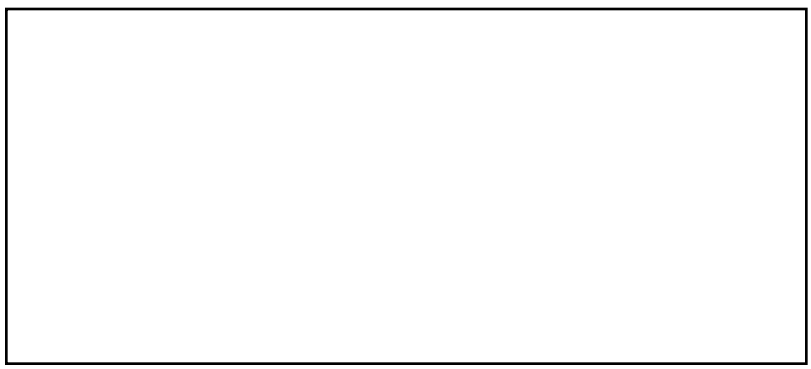
Creating an App Group

> **Xcode Tip**: App group containers allow apps and targets to share resources.

Whenever the user changes the query term in your app, `fetchObjects(for:)` downloads and decodes a new `objects` array. To share this array with your widget, you'll create an app group. Then, in **TheMetStore.swift**, you'll write a file to this app group, which you'll read from in **TheMetWidget.swift**.

➤ If you haven't signed the targets yet, do it now: In the **Project** navigator, select the top level **TheMet** folder. For each target, change the bundle identifier prefix to your organization instead of **com.yourcompany** and set the team.

➤ Now select the **TheMet** target. In the **Signing & Capabilities** tab, click **+ Capability**, then drag **App Groups** into the window. Click **+** to add a new container.



Add new app group.

➤ Name your container **group.your.prefix.TheMet.objects**. Be sure to replace **your.prefix** with **your** bundle identifier prefix. Click the reload button if the color of your group doesn't change from red to black.

➤ Now select the **TheMetWidgetExtension** target and add the **App Groups** capability. If necessary, scroll through the **App Groups** to find and select **group.your.prefix.TheMet.objects**.

Reloading the Widget's Timeline

Next, you'll set up `TheMetStore` so it tells the widget to reload its timeline whenever `fetchObjects(for:)` finishes downloading and decoding an array of objects.

➤ Back in **TheMetStore.swift**, add this `import` statement:

```
import WidgetKit
```

`fetchObjects(for:)` needs to call a `WidgetCenter` method to reload your widget's timeline.

➤ In `fetchObjects(for:)`, add this line *after* the for-loop:

```
WidgetCenter.shared.reloadTimelines(ofKind: "TheMetWidget")
```

When the navigation stack in `ContentView` first loads, it calls `fetchObjects(for:)` to create the `objects` array, but this is an asynchronous task, so a user might install the widget while its `objects` array is empty. You tell the widget to reload its timeline when the array is ready.

Writing the App Group File

➤ At the top of **TheMetStore.swift**, just below the `import WidgetKit` statement, add this code:

```
extension FileManager {
  static func sharedContainerURL() -> URL {
    return FileManager.default.containerURL(
      forSecurityApplicationGroupIdentifier:
        "group.your.prefix.TheMet.objects")!
  }
}
```

This is simply some standard code for getting the app group container's URL. Be sure to substitute *your* bundle identifier prefix.

It makes sense to write this app group file just after you've decoded the `data` into the `objects` array. To write an array to a file, you JSON-encode it. Then the widget JSON-decodes the file contents.

➤ To write the file, add this helper method to `TheMetStore`:

```
func writeObjects() {
  let archiveURL = FileManager.sharedContainerURL()
    .appendingPathComponent("objects.json")
  print(">>> \(archiveURL)")

  if let dataToSave = try? JSONEncoder().encode(objects) {
    do {
      try dataToSave.write(to: archiveURL)
    } catch {
      print("Error: Can't write objects")
    }
  }
}
```

Here, you convert your array of `Object` values to JSON and save it to the app group's container.

➤ In `fetchObjects(for:)`, add the following to call your new helper method before the call to `WidgetCenter`:

```
writeObjects()
```

The existing call to `WidgetCenter` now tells the widget to reload its timeline whenever your app has written a new array of objects into a file in the app group.

Next, go and set up the widget to read this file.

Reading the Objects File

➤ Open **TheMetWidget.swift**.

Now, you can read your `objects` array from the app group file.

➤ Add this helper method to `Provider`:

```
func readObjects() -> [Object] {
  var objects: [Object] = []
  let archiveURL =
    FileManager.sharedContainerURL()
    .appendingPathComponent("objects.json")
  print(">>> \(archiveURL)")

  if let codeData = try? Data(contentsOf: archiveURL) {
    do {
      objects = try JSONDecoder()
        .decode([Object].self, from: codeData)
```

```
      } catch {
        print("Error: Can't decode contents")
      }
    }
    return objects
  }
```

This reads the `Object` values from the file that `fetchObjects(for:)` saved into the app group's container.

➤ Delete these lines from `Provider`:

```
  let store = TheMetStore(6)
  let query = "persimmon"
```

You won't be using a local `TheMetStore` anymore.

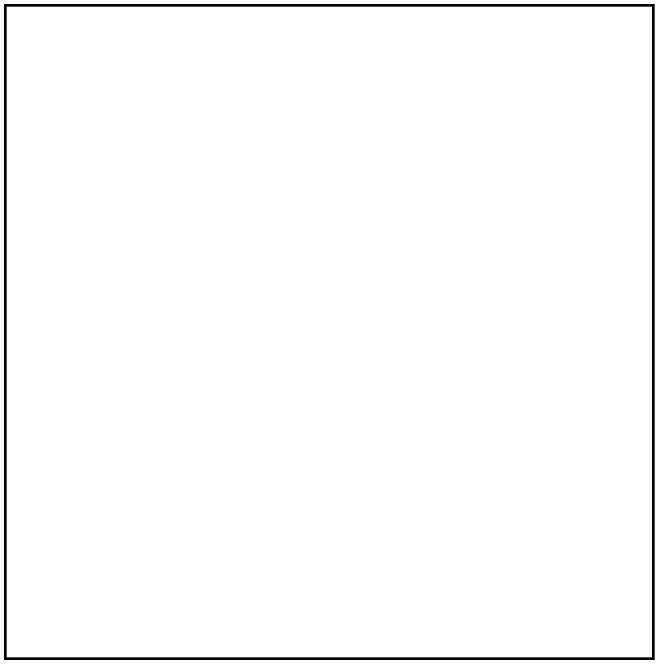
➤ Also delete the `Task` in `getTimeline(in:completion:)` that calls `fetchObjects(for:)`. Then, in `getTimeline(in:completion:)`, replace the `for` loop with the following code:

```
  let objects = readObjects()
  for index in 0 ..< objects.count {
    let entryDate = Calendar.current.date(
      byAdding: .second,
      value: index * interval,
      to: currentDate)!
    let entry = SimpleEntry(
      date: entryDate,
      object: objects[index])
    entries.append(entry)
  }
```

You read the `objects` array from the app group file and use it instead of `store.objects` to create entries.

> **Note**: If you changed the bundle identifier, you'll end up having two apps. Delete the old one before running the project.

➤ Build and run, then close the app. Look for your widget and add it. Watch it display a few persimmon objects, then tap the widget to reopen your app. Set `query` to **giraffe**, wait for the list to reload, then close the app. After a while, your widget will start displaying giraffe objects:



Widget reloaded with giraffe objects

> **Note**: If the widget keeps displaying persimmon objects, tap the widget or the app's icon to reopen the app, then close the app again.

Your widget is working well, and you could happily install it on your device now. If you want to do so, skip down to the end of this chapter to change the timeline back to one-hour intervals.

The next section adds a feature many users expect: When you tap the widget, the app should open in the `ObjectView` for the current widget entry, if it's public domain. Tapping a non-public-domain object should open the metmuseum.org page for the object.

Deep-Linking Into Your App

You can set up your widget with a deep link to activate a `NavigationLink` that opens the `ObjectView` or `SafariView` of the widget entry object. Here's your workflow:

1. Create a URL scheme.
2. Modify a suitable view in `WidgetView` with `widgetURL(_:)`.
3. In your app, implement `onOpenURL(perform:)` to activate the correct `.navigationDestination` modifier.

> **Note**: When you install the app on a device, deep-linking works when the app is running in the background. If the app isn't running at all, tapping the widget opens the app and shows the list.

Creating a URL Scheme

"URL scheme" sounds very grand and a little scary but, because it's just between your widget and your app, it can be quite simple. You're basically creating a tiny API between widget and app. The widget needs to send enough information to the app so the app knows which view to display. Formatting this information as a URL lets you use `URL` or `URLComponents` properties to extract the necessary values.

For this app, the `objectID` property of `Object` uniquely identifies it. So the URL to open "Hexagonal flower vase" is simply:

```
URL(string: "TheMet://828444")
```

And you can access this `objectID` value as the `host` property of the URL. So simple!

In Your Widget

➤ In **WidgetView.swift**, add this modifier to the top-level `VStack`, where you set `truncationMode` and `fontWeight`:

```
.widgetURL(URL(string: "themet://\(entry.object.objectID)"))
```

> **Note**: In the medium and large widget sizes, you can use `Link(_:destination:)` to attach links to different parts of the view.

In Your App

In your app, you implement `.onOpenURL(perform:)` to process the widget URL. You attach this modifier to either the root view, in `TheMetApp`, or to the top level view of the root view. For TheMet, you'll attach this to the `NavigationStack` in `ContentView`, because the `perform` closure must assign a value to a `@State` property of `ContentView`.

You need to trigger navigation *programmatically*: You'll add the widget's object to a *navigation path* to make the app open the correct navigation destination.

➤ In **ContentView.swift**, add this `@State` property to `ContentView`:

```
@State private var path = NavigationPath()
```

When the widget sends a `widgetURL` to the app, you'll check whether the object is in the public domain or not. Then, you'll append either the object or its URL to `path`, and `NavigationStack` will use this to select the correct `navigationDestination`.

> **Note**: If your `NavigationStack` presents only one type of view, `path` can be an array of the data type you pass to that view: `[Object]` for `ObjectView` or `[URL]` for `SafariView`. You'll still need to use `NavigationLink(value:)` with the `.navigationDestination(for:)` modifier.

➤ To use this path, replace `NavigationStack {` with this:

```
NavigationStack(path: $path) {
```

You pass a binding to `path` to the navigation stack. Now, you can observe the current state of the stack or modify `path` to specify where to navigate.

➤ Now, add this modifier to `NavigationStack`, at the same level as the `task` that calls `fetchObjects(for:)`:

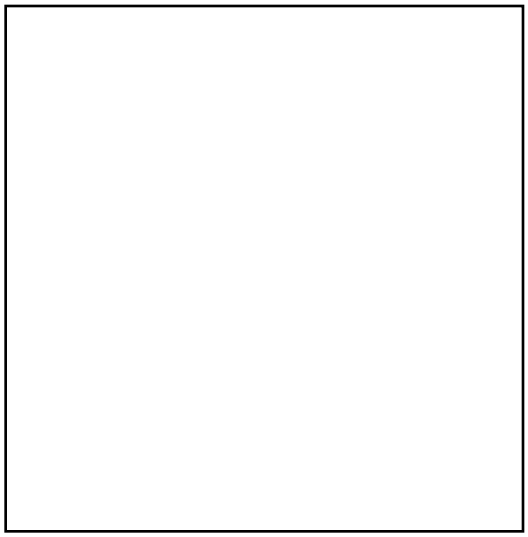
```
.onOpenURL { url in
  if let id = url.host,
    let object = store.objects.first(
      where: { String($0.objectID) == id }) {  // 1
    if object.isPublicDomain {  // 2
      path.append(object)
    } else {
      if let url = URL(string: object.objectURL) {
        path.append(url)
      }
    }
  }
}
```

Here's what this does:

1. Extract an `id` value from the widget URL, then find the first `object` whose `objectID` matches this `id` value. Because `url.host` is a `String`, convert the `objectID` value to `String` before comparing.
2. If the object is in the public domain, append it to `path`. Otherwise, append the `URL` created from its `objectURL`.

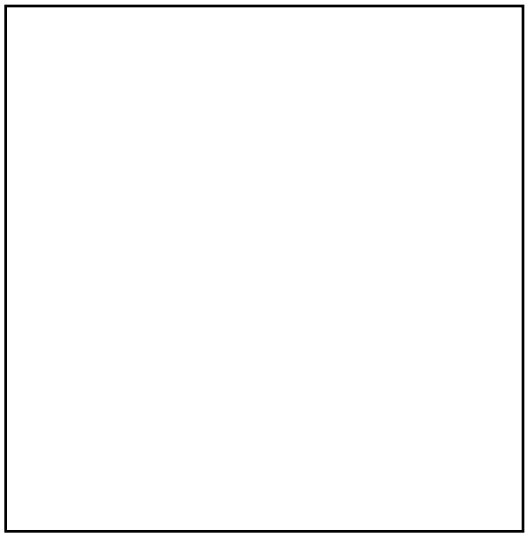
➤ At the top of `ContentView`, change `query` to **peony**: This query returns more non-public-domain objects, so you'll be able to test that tapping these objects opens the app in `SafariView`.

➤ Build and run, wait for the list to load, then close the app and add your widget. Tap a public-domain entry to see it open the `ObjectView` for that object:



Deep link opens widget entry's ObjectView.

➤ Tap the app's back button to return to the list, then close the app and tap a non-public-domain entry to see it open the `SafariView` for that object:



Deep link opens widget entry's SafariView.

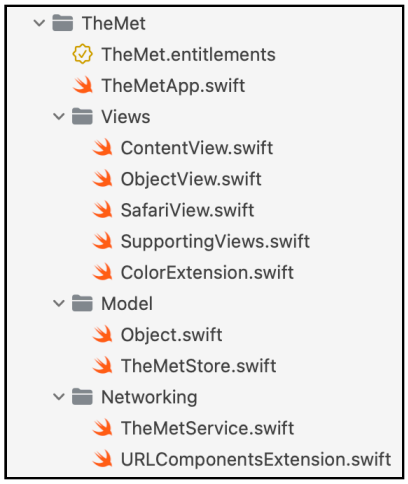
Well done!

A Few Last Things

A couple of housekeeping items before you go.

Organizing TheMet Group

➤ Organize your app files by grouping them into Views, Model and Networking:



Views, Model and Networking groups

Using Normal Timing

You've been using a three-second interval in your timeline to make testing simpler. You definitely don't want to release your widget with such a short interval. If you want to use TheMet on your device as a real app, set up the timeline to change every hour instead of every three seconds.

> **Note**: The project in the final folder still displays every three seconds.

➤ In **TheMetWidget.swift**, in the for-loop of `getTimeline(in:completion:)`, change the `entryDate` code to this:

```
let entryDate = Calendar.current.date(
  byAdding: .hour,
  value: index,
  to: currentDate)!
```

You're restoring the template code's original timing. Now, your widget will add entries one hour apart from each other. You can add it to your device's home screen with no worries about excessive battery use.

➤ Also remove the declaration of `interval`, as Xcode so helpfully suggests, since you're no longer using it.

Refresh Policy

In `getTimeline(in:completion:)`, after the `for` loop, you create a `Timeline(entries:policy:)` instance. The template sets `policy` to `.atEnd`, so WidgetKit creates a new timeline after the last date in the current timeline. As you saw when the widget was downloading a small number of its own objects, the new timeline doesn't start immediately.

Of course, your current timeline fires at 3-second intervals, which is far from normal. With a more normal interval, like one hour, you probably won't notice any delay.

There are two other `TimelineReloadPolicy` options:

- `after(_:)` : Specify a `Date` when you want WidgetKit to refresh the timeline. Like `atEnd`, this is more a suggestion to WidgetKit than a hard deadline.
- `never`: Use this policy if your app uses `WidgetCenter` to tell WidgetKit when to reload the timeline. This is a good option for TheMet. You've already seen the timeline reload almost immediately when you change a query option in your app. You could add code to your app to call `fetchObjects(for:)` at the same time every day, and this would also refresh your widget's timeline.

Key Points

- WidgetKit is still a relatively new API. You might experience some instability. You can fix many problems by deleting the app or by restarting the simulator or device.
- To add a widget to your app, decide what app data you want to display and the time interval between entries. Then, define a view for each size of widget — small, medium, large — you want to support.
- Add app files to the widget target and adapt your app's data structures and views to fit your widgets.
- Create an app group to share data between your app and your widget.
- Deep-linking from your widget into your app is easy to do.

26 Conclusion

We hope you're excited by the new world of iOS and SwiftUI development that lies before you!

By completing this book, you've gained the knowledge and tools you need to build beautiful iOS apps. Set your imagination free and couple your creativity with your newfound knowledge to create some impressive apps of your own.

There is so much more to the iOS ecosystem and Kodeco has the resources to help your continued growth as an iOS developer:

- *SwiftUI by Tutorials* will expand your knowledge of SwiftUI and explores more advanced developer topics.
- *iOS App Distribution & Best Practices* will guide you through the process of publishing your app on the App Store.
- The many video courses and free tutorials on Kodeco explore diverse topics from MapKit to Core Data to animation and much more.

If you have any questions or comments as you work through this book, please stop by our forums at https://forums.kodeco.com and look for the particular forum category for this book.

Thank you again for purchasing this book. Your continued support is what makes the tutorials, books, videos, conferences and other things we do at Kodeco possible, and we truly appreciate it!

— Audrey, Caroline, Libranner and Richard

The *SwiftUI Apprentice* team